

计算机组成原理与操作系统

第一册

从计算、状态、地址、隔离和设备需求到内核抽象

CPU Books

本册是组成原理与操作系统合并理论卷。前半部从“程序要能计算、保存现场、寻址、隔离、通信和恢复”这些需求推导机器结构，再进入内核对象、调度、虚拟内存、I/O、文件系统、网络入口和恢复；完整模拟器内核和代码工程放入实践卷。

前言

操作系统不是一组系统调用名，也不是“进程、内存、文件、网络”这些名词的并列清单。它是一套把硬件资源变成可控抽象的工程协议：CPU 时间被包装成可调度实体，物理内存被包装成带权限的地址空间，设备被包装成文件和事件，崩溃风险被包装成日志、提交点和恢复规则。

因此，本版把计算机组成原理和操作系统放在同一条主线上讲。前半部不会假设读者已经理解 CPU、DMA、MMU、cache、TLB、总线、特权级和中断控制器，而是先搭一个可手算的 x86-64 教学子集：从 bit、byte、word、内存数组、RIP、寄存器、指令编码、取指译码执行、栈、中断、定时器、特权级、系统调用和分页开始，一步一步解释这些硬件为什么出现。只有看清硬件怎样执行指令、保存状态、翻译地址、投递中断和隔离权限，后面的系统调用、进程、虚拟内存、驱动和文件系统才不是空概念。

本册从最简单的硬件和单片机裸机程序开始。先看一位数据怎样被电路保存，一条指令怎样经过取指、译码、执行和访存，再看一个 `while(true)` 主循环怎样直接拥有整台机器；任务变多后为什么需要任务表和状态机；事件变多后为什么需要中断；任务可能死循环后为什么需要定时器和抢占；多个任务共享数据后为什么需要锁、条件变量和 IPC；任务互相破坏内存后为什么需要页表和地址空间；用户不能直接碰硬件后为什么需要系统调用；设备不应由 CPU 搬每个字节后为什么需要 DMA、IOMMU 和 completion；数据写一半会崩溃后为什么需要日志、`fsync`、`rename` 和 `manifest`。

本册采用 CPU 项目中的写法：每个机制都从失败场景进入，再给出原理、不变量、实践观察和测试口径。读者不应该只记住“进程是资源分配单位，线程是调度单位”这样的短句，而要能回答：状态在哪里，谁能改变状态，错误路径怎样退出，哪些证据证明系统确实按这个规则运行。

本册暂时只写理论卷，不展开实践卷工程。正文中的“实践”指纸上状态机、命令观察、实验设计和验收报告口径；完整 C++ 实现、模拟器上的内核和更大规模实验，放到后续实践卷再看。

核心思想

操作系统的每个特性都应该能回答三个问题：它依赖哪个硬件事实，它修复了前一阶段的哪个失败，又引入了哪些新的状态、成本和测试责任。

缩写与术语表

本表只收录本册反复出现、且读者读源码时会立刻遇到的缩写。缩写不是背诵目标；它们的价值是帮助你把教材概念映射到真实内核对象和代码路径。

缩写	全称或常见含义	本册语境
ABI	Application Binary Interface	系统调用参数、调用约定、用户栈、ELF 装载契约
ACPI	Advanced Configuration and Power Interface	固件向内核描述 CPU、NUMA、设备和电源信息
ALU	Arithmetic Logic Unit	CPU 数据通路中执行整数算术和逻辑运算的部件
APIC	Advanced Programmable Interrupt Controller	x86 中断投递、本地定时器、IPI 和多核启动
PCID	Process Context ID	x86-64 TLB 中区分地址空间的标签，减少切换时全局刷新
BIO	Block I/O	块层从文件系统接收的页片段读写意图
BPF	Berkeley Packet Filter	seccomp、eBPF tracing、网络和安全钩子的执行载体
CFS	Completely Fair Scheduler	Linux 经典公平调度模型，以 <code>vruntime</code> 解释按权重分配 CPU
CFI	Control Flow Integrity	控制流保护，限制间接跳转和返回路径被劫持
CPL	Current Privilege Level	x86-64 当前特权级，决定是否可执行特权操作
COW	Copy-on-Write	<code>fork</code> 、 <code>MAP_PRIVATE</code> 和 COW 文件系统的共享后复制语义
CR/MSR	Control Register / Model-Specific Register	x86-64 控制寄存器和模型专用寄存器，保存页表根、系统调用入口和特权控制状态
DMA	Direct Memory Access	设备直接读写内存，要求映射、缓存一致性和生命周期管理
EEVDF	Earliest Eligible Virtual Deadline First	Linux 自 6.6 起逐步采用的公平调度策略，结合 lag 与虚拟截止期选择任务
EOI	End Of Interrupt	中断处理后通知控制器可以投递后续中断
FUA	Force Unit Access	块设备写入要求到达稳定介质，而非仅进入设备缓存
GDT	Global Descriptor Table	x86-64 早期保护和段描述结构，长模式下仍参与入口设置
GFP	Get Free Page flags	内核分配上下文，决定是否可睡眠、可 I/O、可回收
IDT	Interrupt Descriptor Table	x86-64 中断、异常和陷入入口表
IOMMU	I/O Memory Management Unit	设备 DMA 地址翻译和隔离
IPI	Inter-Processor Interrupt	CPU 间重调度、TLB shutdown、多核控制消息
IRQ	Interrupt Request	外设中断请求及其内核处理路径

ISA	Instruction Set Architecture	软件可见的指令、寄存器、异常、内存和特权契约
LSM	Linux Security Module	SELinux、AppArmor、BPF LSM 等安全钩子框架
MMU	Memory Management Unit	执行虚拟地址翻译和页表权限检查的硬件单元
MMIO	Memory-Mapped I/O	CPU 通过地址空间读写设备寄存器
NAPI	New API	Linux 网络收包中断转轮询机制，控制中断风暴
NUMA	Non-Uniform Memory Access	多插槽内存局部性、调度和页分配策略
OOM	Out Of Memory	内存耗尽、回收失败和 OOM killer 决策
PCIe	Peripheral Component Interconnect Express	现代外设互联，承载配置空间、BAR、MSI/MSI-X 和 DMA
PTE/PMD/PUD/PGD	Page Table Entry 等页表层级	多级页表、权限位、缺页和 TLB 失效
RCU	Read-Copy-Update	读路径低成本、写路径延迟释放的同步机制
RSS	Resident Set Size	进程实际驻留物理页，不等于虚拟地址空间大小
skb	socket buffer	Linux 网络栈中承载包数据和协议元数据的核心对象
SMP	Symmetric Multiprocessing	多核并发、per-CPU 数据、runqueue 和 IPI
TLB	Translation Lookaside Buffer	地址翻译缓存，页表修改后需失效旧项
VFS	Virtual File System	统一 superblock、inode、dentry、file 的文件系统抽象
VMA	Virtual Memory Area	进程虚拟地址区间策略，用于缺页判断和权限控制
WAL	Write-Ahead Log	先写日志再提交数据的崩溃恢复协议

怎样阅读本册

本册必须按顺序阅读，但顺序不是“名词清单”的顺序，而是一条失败推动机制出现的路线。先看一次普通保存请求为什么不能靠单个函数解释，再看硬件怎样提供执行现场、地址翻译、特权入口、中断和 DMA，随后才进入进程、调度、系统调用、虚拟内存、文件、设备、网络和安全。全书真实架构统一采用 x86-64，汇编统一采用 Intel 语法：目的操作数在前，源操作数在后，例如 `mov rax, qword ptr [rbx]`、`add rax, 1`、`syscall`、`iretq`。

第一部分回答“操作系统为什么必须存在”。先看 CPU、MMU、中断、DMA 和设备只暴露低层状态，再看裸机主循环为什么会被忙等、异步事件、死循环和启动约束击穿。读完这一部分，读者应该能把 task、address space、fd、request 这些 OS 对象追溯到具体机器事实。

第二部分回答“多个执行流怎样共享 CPU 且不丢事件”。它从进程、线程和 runqueue 进入，再补调度器、多核、等待队列、同步、锁和死锁。这里不要只背算法名，要看每个状态转换：谁从 runnable 变成 sleeping，谁负责唤醒，唤醒后怎样重新进入 runqueue。

第三部分回答“用户程序怎样被限制在授权边界内”。系统调用、`exec`、ELF、虚拟内存、缺页、`mmap`、page cache 和内核分配器被放在一起，是因为它们都在改变“谁能访问什么”。这一部分的主线是验证、准备、提交、回滚，避免半提交状态。

第四部分回答“异步设备怎样变成文件、网络和持久化语义”。驱动、DMA、块层、VFS、文件系统、socket、日志和恢复不是并列名词，而是一条从 request 身份、buffer 所有权、completion、page cache 到 `fsync` 的路径。

第五部分回答“系统出错、被攻击或性能异常时怎样给出证据”。安全隔离、panic、错误恢复、可观测性和验收方法统一收束到工程报告能力：结论必须能指向状态、边界、失败路径和可观察证据。

阅读阶段	先问的问题	不合格读法
机器事实	哪个硬件限制逼出内核对象	只背 CPU、MMU、DMA 名词
任务并发	状态怎样进入和离开 runqueue/wait queue	只背调度算法复杂度
边界内存	何时验证，何时提交，失败 怎样回滚	把系统调用当普通函数
I/O 持久化	请求身份、buffer 所有权 和 completion 在哪	把 <code>write</code> 成功当落盘
安全观测	哪条证据证明契约成立或失败	把工具输出直接当结论

学习每章时都要落到五个问题：

1. 状态在哪里：进程表、runqueue、锁等待队列、页表、VMA、fd 表、socket、inode、page cache、日志还是设备队列。
2. 硬件在哪里：RIP、RSP、RFLAGS、通用寄存器、cache、TLB、页表、MMIO 寄存器、DMA 描述符、APIC/MSI-X 还是定时器。
3. 权限在哪里：CPU 特权级、页表权限、文件权限、capability、namespace 还是对象引用。
4. 等待在哪里：runnable 等 CPU、blocked 等事件、mutex 等持有者、page fault 等页面、socket 等缓冲、I/O 等 completion。
5. 提交在哪里：`exec` 切换地址空间、状态更新完成、写入 page cache、块设备 flush、目录持久化还是 manifest 生效。

每章可以按固定节奏读：先找失败场景，再看直接补丁为什么不够，再看 OS 机制如何建立状态对象，最后用不变量和证据收尾。若一节讲 fd，就要画出 fd table、open file description、inode 和 page cache 的关系；若一节讲调度，就要写出 runnable、running、blocked、zombie 和 reaped 的状态转换；若一节讲锁，就要画出锁顺序图和等待队列；若一节讲 `exec`，就要标出失败回滚边界和提交点；若一节讲持久化，就要写出每个崩溃点后恢复程序应该相信什么。

阅读实现小节时，不能只看伪代码能不能运行。要追问四件事：数据结构是什么，复杂度是多少，维护了哪些不变量，失败路径是否会留下半提交状态。一个教学模型可以比真实内核小，但不能把真实问题讲错。

不要把工具输出当成结论。工具只是证据来源；结论必须说明哪条 OS 契约被验证，哪条仍然只是合理假设。

目录

前言	ii
缩写与术语表	iii
怎样阅读本册	v
第一部分 从上电到内核运行	1
第 1 章 裸机、引导与内核初始化	3
1.1 从单片机裸机程序开始	3
1.2 中断、定时器与内核入口	8
1.3 引导、链接脚本与启动状态机	16
1.4 内核初始化：从 kernel_main 到第一个任务	21
1.5 源码级补充：MCU 启动、UART 与中断路径	24
1.6 实践	26
1.7 测试	27
第 2 章 硬件事实与内核对象	29
2.1 一、操作系统必须知道的硬件事实	29
2.2 二、硬件事实怎样逼出内核对象	31
2.3 三、CPU 入口逼出 task 和 trap frame	32
2.4 四、地址翻译逼出 address space、VMA 和 PTE	34
2.5 五、设备请求逼出 request、wait queue 和 completion	37
2.6 六、映射总结与检查表	39
第 3 章 操作系统地图与契约	44
3.1 从保存按钮建立 OS 地图	44
3.2 保存请求的完整账本	59
3.3 四条完整路径：从硬件事件到内核决策	66
3.4 从硬件事实到 OS 抽象	70
3.5 保存路径的五个关口	71
3.6 动手验证：把本章落到证据	74
第二部分 进程、调度与并发	76
第 4 章 进程、线程与调度	78
4.1 原理	78
4.2 实践	79
4.3 算法与实现分析	79
4.4 测试	85
第 5 章 系统调用、权限与内核边界	92
5.1 原理	92
5.2 实践	96
5.3 算法与实现分析	96
5.4 测试	99
5.5 源码级补充：entry_SYSCALL_64、capability、LSM 与 vDSO	100

第 6 章 exec、ELF 装载与用户栈	112
6.1 原理	112
6.2 实践	112
6.3 算法与实现分析	114
6.4 测试	116
6.5 源码级补充: load_elf_binary、auxv 与 binfmt	117
第 7 章 同步、IPC 与锁	120
7.1 原理	120
7.2 实践	121
7.3 算法与实现分析	122
7.4 测试	125
7.5 源码级补充: 锁实现、IPC 与 lockdep	126
第三部分 内存管理	130
第 8 章 虚拟内存与地址空间	132
8.1 原理	132
8.2 实践	133
8.3 算法与实现分析	133
8.4 测试	137
8.5 源码级补充: PTE 标志、THP、swap、mlock 与 PSI	139
第 9 章 mmap、page cache 与文件映射	160
9.1 原理	160
9.2 实践	161
9.3 算法与实现分析	162
9.4 测试	164
9.5 源码级补充: do_mmap、XArray、writeback、O_DIRECT 与 DAX	165
第 10 章 内核内存分配器	168
10.1 原理	168
10.2 实践	168
10.3 算法与实现分析	168
10.4 测试	170
10.5 源码级补充: buddy、SLUB、GFP、vmalloc 与 kmemleak	170
第四部分 设备、I/O 与文件系统	173
第 11 章 设备驱动、中断下半部与 DMA	175
11.1 原理	175
11.2 实践	176
11.3 算法与实现分析	176
11.4 测试	178
11.5 源码级补充: Linux 设备模型、PCI、NVMe、virtio 与 IOMMU	179
第 12 章 块设备、I/O 调度与文件路径	183
12.1 块层原理与对象	183
12.2 I/O 调度算法	184
12.3 持久化语义与多队列	185
12.4 块层的统计、限流与错误处理	186

12.5 VFS 文件层	187
12.6 I/O 路径	190
12.7 实践	194
12.8 测试	195
12.9 文件系统恢复与崩溃一致性	195
第 13 章 文件系统：从磁盘字节到现代设计	197
13.1 为什么需要文件系统	197
13.2 块设备接口：从 CPU 到磁盘字节	199
13.3 最小文件系统：四类对象	204
13.4 inode 的深层结构	210
13.5 块映射：从直接指针到 extent 树	217
13.6 空间分配：bitmap、块组与局部性	221
13.7 目录结构：线性表、hash、B-tree	224
13.8 VFS：统一接口与分发	226
13.9 page cache 与写回	230
13.10 崩溃一致性与日志	241
13.11 Copy-on-Write 文件系统	249
13.12 闪存文件系统	255
13.13 现代优化技术	263
13.14 设计选择与对比	272
13.15 测试	273
13.16 源码级补充	275
第 14 章 socket、网络入口与内核边界	282
14.1 原理	282
14.2 实践	282
14.3 算法与实现分析	283
14.4 测试	286
14.5 源码级补充：sk_buff、协议栈入口、NAPI 与 Netfilter	287
第五部分 安全、恢复与观测	291
第 15 章 安全、隔离与最小权限	293
15.1 原理	293
15.2 实践	294
15.3 算法与实现分析	295
15.4 测试	295
15.5 源码级补充：DAC、MAC、capability、namespace、Landlock 与硬化	296
第 16 章 持久化、崩溃一致性与恢复	301
16.1 原理	301
16.2 实践	301
16.3 算法与实现分析	302
16.4 测试	303
16.5 源码级补充：WAL、COW 文件系统、RAID、LVM 与 media failure	303
第 17 章 panic、错误处理与可观测性	305
17.1 原理	305
17.2 实践	306
17.3 算法与实现分析	307
17.4 测试	310

17.5 源码级补充：oops、kdump、ftrace、perf 与 eBPF	311
能力清单	316
补充材料阅读地图	318
Linux 源码阅读索引	320
参考资料与版本基线	322

第一部分

从上电到内核运行

本部分导读

这一部分建立全书地基：从按下电源开关开始，追踪电流怎样变成指令、指令怎样变成内核入口、内核入口怎样初始化出第一个可以调度的任务。

第一章从裸机开始：单片机的 `while(1)` 循环能做什么、为什么不够用、中断怎样打破顺序执行、引导程序怎样把内核加载到内存。第二章把硬件事实收束成内核对象：CPU 特权级怎样逼出 `task`、页表怎样逼出地址空间、设备队列怎样逼出 `request`。第三章给出操作系统地图：一次保存请求牵出执行现场、用户地址、`fd`、`page cache`、设备 `completion` 和持久化边界。

读完这一部分，读者应该能回答三个问题：上电后到内核运行经过哪些阶段；一个 OS 对象背后对应哪条硬件事实；为什么后面的调度、内存、驱动和文件系统都必须先有状态、权限、等待和提交边界。

第 1 章 裸机、引导与内核初始化

本章把单片机裸机程序、中断与定时器、引导与内核初始化三条线索合并成一条路径，解释一台机器从上电到 `kernel_main` 运行的全过程。我们先从一块最小的温控板出发，看清裸机主循环为什么会失败；再用中断和定时器补上事件驱动与抢占这两块缺口；最后回到真实机器的复位向量、链接脚本和分阶段初始化，追踪内核如何一步步建立可运行环境。理解这条路径后，后续进程、内存、文件等章节讨论的内核服务才有立足之地。

1.1 从单片机裸机程序开始

理解操作系统，最稳的起点不是 Linux，也不是进程和虚拟内存，而是一块最小单片机。先看一个具体任务：做一块温控板。

这块板子要做四件事：

1. 每 1ms 读取温度传感器。
2. 每 1ms 根据温度更新电机或风扇输出。
3. 随时接收串口命令，例如修改目标温度。
4. 每 100ms 通过串口发一行日志。

第一版最容易写成这样：

```
init_clock();
init_sensor();
init_motor();
init_uart();

while (true) {
    read_sensor();
    update_motor();
    handle_uart_command_if_any();
    send_log_if_due();
}
```

这就是裸机程序：没有操作系统，程序直接拥有整台机器。所有内存都能读写，所有外设寄存器都能访问，任何函数都可以关中断、改时钟、写串口、改 GPIO。它的优点是简单、启动快、路径短；缺点也同样直接：只要某个函数等硬件，后面的函数就全被拖住。

假设 `send_log_if_due()` 里要发 80 个字节，而串口暂时发送缓冲区满。最朴素写法会忙等：

```
void uart_putc(char c) {
    while (!uart_tx_ready()) {
        // wait until hardware can accept one byte
    }
    uart_write(c);
}
```

如果这一等花了 50ms，那么 1ms 周期的 `read_sensor()` 和 `update_motor()` 就会迟到 50 次。程序不是“逻辑错”，而是结构错：它把需要等待的 I/O 当成了立刻完成的普通函数。

所以本章的过渡线不是从术语开始，而是从这个失败开始：

1. 暴力写法：所有工作按顺序塞进一个 `while (true)`。
2. 发现问题：相邻工作共享同一个 CPU，慢 I/O 会拖死周期任务。
3. 第一步优化：把任务的周期、下次运行时间和状态显式记录下来。
4. 第二步优化：把阻塞发送改成状态机，硬件没准备好就先返回。
5. 再往后：让硬件用中断通知事件，把“主动一直问”改成“事件到了再处理”。

核心思想

裸机程序的失败不是因为代码写得不仔细，而是因为它把“需要等待的 I/O”和“能立刻完成的计算”放在同一条顺序执行流里。操作系统不是凭空出现的奢侈品：只要程序里出现多个活动、共享资源、等待硬件、错误恢复和时间预算，就已经需要 OS 式的状态管理。理解 OS 的最低门槛，就是看清这一步为何不可避免。

一个 MCU 芯片里面通常有什么。现在再看硬件部件。单片机，也叫 MCU，是一颗芯片上集成了一套很小的计算机。温控板不需要插显卡和硬盘，但它需要 CPU 核执行指令，需要 Flash 保存程序，需要 SRAM 放栈和全局变量，需要 GPIO 控制电机引脚，需要 UART 收发命令，需要定时器提供 1ms 节拍。很多外设不是运行时枚举出来，而是在芯片手册里固定了基地址和寄存器布局。

部件	作用	与 OS 的关系
CPU core	执行指令和异常入口	上下文、特权模式、中断响应
Flash	保存程序镜像和只读数据	类似启动介质和只读代码段
SRAM	保存栈、堆、全局变量	物理内存管理的最小形态
中断控制器	管理外设中断	IRQ 分发和优先级的原型
SysTick/Timer	周期时间基准	tick、超时和抢占的原型
GPIO、UART、SPI、I2C	连接外部设备	字符设备和总线驱动的原型
DMA controller	在外设和内存间搬运数据	DMA 生命周期和 completion 的原型

这些部件通常共享同一个地址空间。CPU 访问 `0x20000000` 可能是 SRAM，访问 `0x40000000` 可能是 UART 寄存器。区别不在指令，而在地址译码后连到哪个硬件。地址译码可以理解为“硬件看地址范围，把这次访问交给内存或某个外设”。这个事实会一路延伸到大型 OS：驱动通过 MMIO 地址访问 PCIe 设备寄存器，本质仍是 load/store 触发硬件状态机。

芯片设计者怎样选择寄存器接口。外设寄存器是硬件和软件的协议。设计一个 UART 外设时，芯片设计者通常会暴露控制寄存器、状态寄存器、数据寄存器、波特率寄存器和中断使能寄存器。软件写控制寄存器启用发送接收；读状态寄存器判断 TX ready 或 RX available；读写数据寄存器收发字节。

UART register model:

```
CTRL    bit0 enable_rx, bit1 enable_tx
STATUS  bit0 rx_ready, bit1 tx_empty, bit2 overrun
DATA    read pops received byte, write submits transmit byte
BAUD    divisor for baud rate generator
IRQ_EN  enable rx/tx/error interrupt
```

注意 **DATA** 寄存器不是普通内存。读它可能弹出接收 FIFO 的一个字节，写它可能启动发送状态机。状态位也可能有特殊清除规则，例如“写 1 清除”。驱动必须严格按手册顺序访问这些寄存器，否则问题不是编译错误，而是硬件进入不可预期状态。

从外设状态机看驱动。一个 UART 发送器可以看成硬件状态机：空闲时等待软件写 DATA；收到字节后按波特率移位输出；完成后设置 tx_empty；若开启中断，则向中断控制器发请求。软件驱动的任务不是“调用发送函数”，而是和这个状态机同步。

```
UART_TX_IDLE:
    if software writes DATA:
        shift_reg = DATA
        state = UART_TX_BUSY

UART_TX_BUSY:
    shift one bit per baud tick
    if all bits sent:
        STATUS.tx_empty = 1
        if IRQ_EN.tx_empty:
            raise_irq()
        state = UART_TX_IDLE
```

这就是为什么驱动要处理 ready 位、超时、FIFO 满、错误位和中断确认。ready 位表示“硬件现在能不能接收下一步动作”；FIFO 是硬件里的小队列，用来暂存还没处理完的字节；错误位记录溢出、校验失败或线路异常。硬件不会执行 C 函数返回值协议，它只会按寄存器位和状态机推进。操作系统把这种硬件状态机包装成 read/write/poll/ioctl，但底层仍然必须尊重硬件协议。

暴力写法：一个主循环包办所有工作。

```
init_clock();
init_gpio();
init_uart();

while (true) {
    if (uart_has_byte()) {
        byte b = uart_read();
        handle_command(b);
    }
    if (timer_expired()) {
        update_control_loop();
    }
    flush_logs_if_needed();
}
```

这段代码看起来不像 OS，但它已经暴露了 OS 的胚胎问题。第一，多个活动共享一个 CPU，谁先执行？第二，事件到达时间不可控，怎样避免漏事件？第三，某个处理函数太慢，怎样不拖死其他工作？第四，共享状态由谁保护？第五，故障后怎样恢复到可解释状态？

注意这里的“任务”还不是 Linux 里的进程或线程。它只是一个需要周期性或事件性执行的函数。先把这个简单含义抓住，后面再逐步升级成调度器里的 task。

从超级循环到任务表。裸机主循环通常被称为超级循环。它的结构是一个大循环里按固定顺序调用多个任务函数。温控板的第一版可以直接写：

```
while (true) {
    task_read_sensor();
    task_control_motor();
    task_send_telemetry();
    task_blink_led();
}
```

这种写法的问题是隐含了调度策略。调度策略就是“谁先用 CPU，谁后用 CPU，最多能用多久”的规则。超级循环没有把规则写成数据；它把规则藏在代码顺序里：每

个任务的优先级就是它在代码里的顺序，每个任务的时间预算完全靠自觉，任何任务卡住，后面的任务都不会运行。

要让它可分析，第一步是把任务放进表里，并给每个任务明确的周期、deadline 和上次运行时间。deadline 是“最晚必须完成的时间点”。例如控制电机每 1ms 必须更新一次，超过这个时间点，系统还能跑，但控制效果已经变差。

```
struct Task {
    const char* name;
    uint32_t period_ms;
    uint32_t next_release_ms;
    void (*run)();
};

while (true) {
    uint32_t now = ticks_ms();
    for each task in tasks:
        if (now >= task.next_release_ms) {
            task.run();
            task.next_release_ms += task.period_ms;
        }
    }
```

这个算法是最小合作式调度器。合作式的意思是：任务必须主动返回，系统才有机会运行下一个任务。它没有抢占；抢占是“任务还没主动返回，定时器或内核入口就把 CPU 收回来”。这里先不要急着上抢占，先看合作式的边界。

它的复杂度是每轮扫描 $O(n)$ ， n 是任务数量；它的关键不变量是 `next_release_ms` 单调前进，每个任务函数必须短小、不能无限循环、不能执行不可控阻塞。

合作式调度器已经比手写超级循环好，因为任务的释放条件变成数据。但它仍然无法处理一个任务超时的问题。如果 `task_send_telemetry` 等串口缓冲区空等了 50ms，那么 1ms 周期的控制任务就会迟到。于是我们需要区分“能立刻做的计算”和“需要等待的 I/O”，这会把系统推向事件队列和中断。

最小调度算法的复杂度和失败边界。周期任务表的扫描算法很容易写，但它把每轮成本变成 $O(n)$ 。如果任务数量很少，成本可忽略；如果任务很多或周期很短，调度器本身会吃掉可观 CPU 时间。更重要的是，它没有强制时间预算，只能在任务返回后继续扫描。

策略	算法成本	失败边界
固定顺序超级循环	每轮 $O(n)$	任务顺序即隐式优先级，慢任务拖累后续任务
周期任务表	每轮扫描 $O(n)$	任务不返回则全局停顿，deadline 只能事后发现
优先级就绪队列	取最高优先级可为 $O(1)$ 或 $O(\log n)$	低优先级可能饥饿，需要抢占和时间统计
时间片轮转	每次切换接近 $O(1)$	交互延迟和吞吐受时间片影响

这里已经能看见调度器设计的核心：策略不是口号，必须落到数据结构和复杂度。任务少时，简单扫描最可靠；任务多、优先级复杂、需要抢占时，就要引入 `runqueue`、时间统计和上下文切换。

状态机替代阻塞函数。裸机程序最常见的灾难是阻塞等待：

```
void send_packet(Packet p) {
    for each byte in p:
        while (!uart_tx_ready()) {
```



```
// busy wait
}
uart_write(byte);
}
```

这段代码在功能上正确，但会占住整个 CPU。它的问题在于：明明只有“串口暂时不能收下一个字节”这一点新情况，程序却把整个 CPU 都扣在原地。更好的写法是把发送过程改成状态机：如果硬件暂时不能发送，就保存进度并返回，让其他任务继续运行。

```
enum TxState { Idle, Sending };

void uart_tx_step() {
    if (tx_state == Idle || !uart_tx_ready()) {
        return;
    }
    uart_write(tx_buffer[tx_pos++]);
    if (tx_pos == tx_len) {
        tx_state = Idle;
    }
}
```

状态机的核心代价是复杂性。阻塞函数把“下一步在哪里”藏在调用栈里；状态机把下一步显式放在 `tx_state` 和 `tx_pos` 里。操作系统中的线程，就是另一种保存“下一步在哪里”的方法：线程把寄存器、栈和程序计数器保存为上下文，使阻塞代码看起来仍像顺序代码。但线程要付出调度和栈内存成本。

资源所有权和最小内核对象。没有 OS 时，所有模块都能直接操作硬件寄存器。小程序可以这样写；程序变大后，它会造成所有权混乱。串口驱动、日志模块、命令行模块如果都直接写 UART 寄存器，就无法保证输出顺序，也无法处理并发写入。

于是我们引入最小内核对象：一个对象拥有硬件资源，其他模块只能通过它的接口请求操作。比如串口对象维护发送队列、接收队列、错误计数和当前硬件状态。它不一定需要特权级，但已经有了内核思想：资源集中管理，状态对外封装，错误通过接口返回。

```
struct UartDevice {
    RingBuffer rx;
    RingBuffer tx;
    uint32_t overrun_count;
    bool tx_busy;
};

bool uart_submit(UartDevice* dev, Span bytes);
bool uart_read(UartDevice* dev, byte* out);
```

从这里可以看到后续 fd、socket、设备文件的影子。用户不应该直接改设备寄存器，而应该持有一个句柄，请求拥有者执行操作。差别只是大型 OS 里这个拥有者运行在内核态，并且有硬件权限保护。

从裸机模块边界到用户/内核边界。裸机程序里也可以规定“只有 UART 模块能写 UART 寄存器”，但这只是团队约定。任何 C 文件只要拿到寄存器地址，仍然能直接写硬件。通用 OS 不能依赖所有程序守规矩，它必须让硬件禁止用户程序访问设备寄存器。

```
bare-metal convention:
    app calls uart_submit()
    but app could still write UART_BASE directly
```

```
protected OS:
  user VA does not map UART MMIO
  user write to UART physical address triggers fault
  app must call write(fd, ...)
  kernel driver performs MMIO after permission checks
```

这就是模块化和隔离的区别。模块化让代码更清楚，但不能阻止恶意或错误代码；隔离由 CPU 特权级、页表权限和异常入口强制执行。OS 的很多复杂性，就是把裸机里的“请大家遵守约定”升级成“硬件保证不能绕过”。

轮询、中断和 CPU 占用率。裸机驱动要么轮询要么中断，二者不是非此即彼，而是同一根折中曲线上的两端。假设 UART 每秒收到 100 个字节，CPU 频率 100MHz。若主循环每 10 微秒轮询一次状态寄存器，一秒轮询 100000 次，其中只有 100 次有用；若每次轮询花 30 个周期，纯轮询检查就耗掉 300 万周期，约 3% CPU。若轮询间隔变大，CPU 占用下降，但输入延迟和溢出风险上升。

模式	优点	失败边界
忙轮询 周期轮询	实现最简单，延迟可控 降低 CPU 消耗	浪费 CPU，慢操作会漏事件 延迟取决于周期，burst 易溢出
中断驱动	空闲时不耗 CPU，事件到达即处理	ISR 竞态、队列满、中断风暴
DMA + 中断	大吞吐低 CPU	buffer 生命周期、cache、一致性复杂

这张表也适用于大 OS：epoll 减少无效轮询，NAPI 在高负载下把中断转为轮询，io_uring 用共享队列减少系统调用往返。机制不同，目标都是在 CPU 占用、延迟和复杂度之间折中。

MCU 上的 DMA：从串口接收到环形缓冲区。即使在单片机上，DMA 也不是高级服务器才有的东西。典型场景是 UART 持续接收数据，CPU 不想每个字节都进中断。驱动可以准备一个环形缓冲区，让 DMA 控制器把 UART 接收数据直接写入 SRAM，半满或满时再中断 CPU。

```
uart_rx_dma_setup():
  dma.src = UART_DATA_REGISTER
  dma.dst = rx_ring_buffer
  dma.len = RING_SIZE
  dma.mode = circular
  dma.enable_half_full_irq = true
  dma.enable_full_irq = true
  start_dma()

dma_irq():
  produced = compute_dma_write_position()
  publish_new_bytes_to_parser(produced)
```

这个例子把 DMA 的三条规则讲清楚。第一，buffer 在 DMA 运行期间不能随意改用途。第二，CPU 读取 DMA 写入的数据前，必须理解缓存一致性和写入位置。第三，completion 不一定表示全部 I/O 结束，也可能只是半缓冲区可消费。大型 OS 的网卡 RX ring 和块设备 completion，本质上是这个模型的扩展。

1.2 中断、定时器与内核入口

上一节的温控板已经有了主循环和任务表，但还有两个问题没有解决。

第一个问题是串口接收。主循环如果每 1ms 才检查一次串口，而用户连续发来

20 个字节，硬件 FIFO 可能很快满掉。FIFO 满后再来的字节会丢失，命令就被截断。

第二个问题是坏任务。假设日志任务里出现死循环：

```
void task_send_log() {
    while (true) {
        // bug: waits forever
    }
}
```

合作式调度器要求任务主动返回。这个任务不返回，1ms 控制任务就再也没有机会运行。也就是说，仅靠“大家自觉让出 CPU”不够。

先看轮询的暴力解。CPU 在主循环里反复问硬件：

```
while (true) {
    if (uart_has_byte()) {
        byte b = uart_read();
        ring_push(&rx_queue, b);
    }
    if (timer_expired()) {
        run_periodic_tasks();
    }
}
```

这个版本的问题是 CPU 每轮都重新问所有事件有没有发生，大多数检查没有新信息。检查太频繁，浪费 CPU；检查太慢，输入延迟变大，还可能丢字节。于是硬件提供另一种过渡：事件发生时主动打断 CPU。

中断 (interrupt) 可以先理解成硬件事件入口。外设或定时器发现事件后，向 CPU 发出请求；CPU 在合适的指令边界停下当前代码，保存必要现场，跳到内核登记好的入口函数。处理完后，CPU 再恢复被打断的代码。中断不是魔法，它只是把“CPU 一直问设备”改成“设备有事时通知 CPU”。

核心思想

轮询和中断不是对立的两条技术路线，而是同一条折中曲线的两端。轮询把判断时机的成本放在 CPU 上，延迟可控但浪费算力；中断把判断时机的成本放在硬件上，空闲时不耗 CPU 但要付出上下文保存、竞态和入口管理的代价。从“超级循环”走到“事件驱动”，本质是把“什么时候做”这件事从代码顺序交给硬件事件，再把硬件事件整理成可调度的内核对象。

中断处理函数必须短。原因很具体：中断期间可能屏蔽同级或低级中断；处理太久会增加其他事件延迟；处理函数如果访问复杂共享状态，会和主循环或其他中断产生竞态。因此常见设计是 top half 只读取硬件状态、清中断、把事件放入队列；bottom half 或普通任务稍后完成复杂处理。top half 指“中断入口里马上做的短路径”；bottom half 指“离开硬中断后继续做的延后工作”。

```
interrupt uart_rx_isr() {
    byte b = UART_DATA;
    ring_push(&rx_queue, b);
    UART_CLEAR_INTERRUPT();
}

void uart_task_step() {
    while (ring_pop(&rx_queue, &b)) {
        parse_byte(b);
    }
}
```

中断引入了共享状态竞争。主循环和中断处理函数可能同时访问 `rx_queue`。在单核单片机上，最简单的临界区是短暂关中断，修改队列指针，再开中断。这个方法有效，但必须短；如果关中断期间做慢操作，就会丢事件或增加控制延迟。

```
disable_interrupts();
bool ok = ring_push(&queue, value);
enable_interrupts();
```

这个临界区就是锁的祖先。临界区指“这段代码运行到一半时，共享数据结构暂时不完整，不能被另一个执行流插进来”。以后在多核机器上，关本核中断不能阻止另一个核心同时访问同一数据结构，于是需要自旋锁、互斥锁和原子操作。但基本问题没有变：某段代码需要在状态不变量被临时打破时不被并发打断。

中断控制器为什么不是可选部件。若只有一个外设和一个 CPU，外设可以直接把中断线连到 CPU。真实系统有串口、网卡、磁盘、定时器、多个 CPU，事件会同时出现。CPU 不能只收到一句“有事了”，它还要知道“哪个事件、是否允许现在处理、送到哪个入口”。中断控制器负责把混乱的硬件事件整理成 CPU 能处理的 `vector`。

`vector` 是 CPU 入口编号，可以理解成“这类中断应该跳到哪一个入口”。`pending` 位表示“这个中断源已经有事件等待处理”。`mask` 表示“软件暂时屏蔽这个中断源”。`EOI` 是 end of interrupt，表示处理方告诉控制器“这次中断处理完了，可以继续投递后续事件”。

```
interrupt_controller:
    pending[source] = device_irq_line
    enabled[source] = software_mask
    priority[source] = configured_priority
    target[source] = cpu_id

deliver():
    candidates = pending & enabled
    source = highest_priority(candidates)
    cpu = target[source]
    cpu.raise_interrupt(vector[source])
```

没有中断控制器，OS 就不能可靠地屏蔽某类中断，不能区分中断来源，不能把高吞吐设备分散到多个 CPU，也不能在多核系统中发送 IPI。中断控制器是“外设事件”和“CPU 异常入口”之间的路由层。

x86-64 中断控制器：local APIC、IOAPIC、MSI/MSI-X。x86-64 主线里，中断控制器不是一个单独芯片名，而是一组协作机制。先抓住最小含义：每个 CPU 旁边有本地中断入口管理，外部设备要通过路由机制把事件送到某个 CPU 的某个 `vector`。

local APIC 属于每个 CPU，用于本地 timer、IPI 和接收 `vector`。IPI 是 CPU 给另一个 CPU 发的中断，常用于让别的核心重新调度或刷新地址翻译缓存。IOAPIC 把传统外部 IRQ 重定向到目标 CPU 和 `vector`。MSI/MSI-X 让 PCIe 设备通过一次特殊内存写投递中断；它的好处是多队列设备可以给不同队列分配不同 `vector`。

机制	主要用途	OS 关心点
local APIC	每 CPU 中断、本地 timer、IPI	多核调度、TLB shutdown、timer tick
IOAPIC	把外部 IRQ 重定向到 CPU/vector	IRQ 路由、触发方式、亲和性
MSI/MSI-X	PCIe 设备通过内存写投递中断	多队列设备、vector 分配、隔离
x2APIC	扩展 APIC 编址和访问方式	大规模 CPU、虚拟化、IPI 成本

MSI/MSI-X 值得单独强调。传统中断像一根线；MSI 是设备向一个特殊地址写入消息来触发中断。这样 PCIe 设备可以为多个队列分配多个 vector，让网卡 RX queue 0 中断送到 CPU0，RX queue 1 中断送到 CPU1。多队列网络和 NVMe 的伸缩性很大程度依赖这种硬件能力。

定时器和时间片。现在回到坏任务。串口中断解决的是“外设事件来了怎么办”；定时器中断解决的是“没有外设事件，但当前任务一直不回来怎么办”。没有定时器，只有任务主动让出 CPU 时，系统才能切换任务；一旦任务死循环，整个系统就失控。

定时器最小形态是一个硬件计数器和一个比较值。计数器按时钟递增；计数器达到比较值时，硬件产生中断。OS 在中断中增加 tick、检查超时、唤醒等待任务，并在必要时设置“需要调度”的标志。tick 是内核维护的软件时间刻度，不等于现实世界里唯一的时间表示，但足够说明周期调度和超时。

```
interrupt timer_isr() {
    ticks++;
    for each timer in timer_wheel[current_slot]:
        if (timer.expired(ticks)) {
            make_task_runnable(timer.task);
        }
    if (current_task.time_slice_used++ >= current_task.time_slice) {
        need_reschedule = true;
    }
}
```

定时器从硬件计数器到 OS 超时。硬件定时器最小形态是一个递增或递减计数器，加一个比较寄存器。计数器来自固定频率时钟；当计数器达到比较值时，硬件产生中断。OS 在中断里更新软件时间，并设置下一次比较值。

```
program_next_timer(deadline):
    timer.compare = deadline
    timer.enable_interrupt = true

timer_interrupt():
    now = timer.counter
    run_expired_timers(now)
    next = earliest_deadline()
    program_next_timer(next)
```

周期 tick 的做法是每隔固定时间中断一次，例如 1ms 或 10ms。高精度和 tickless 的做法是按最近事件编程下一次中断。前者实现简单，后者减少无意义中断，但要求内核精确维护最近 deadline。无论哪种方式，抢占和超时都依赖硬件在未来把控制权交回内核。

异常和系统调用。中断来自外设，异常来自当前指令执行，例如除零、非法指令、缺页、权限错误。系统调用也是一种受控入口：用户程序主动执行特定指令，请求进入内核。三者共同构成内核入口。

入口	来源	典型处理
中断	外设或定时器	读取设备状态，唤醒任务，记录 tick
异常	当前指令失败	发送信号、修复缺页、终止进程
系统调用	用户主动请求	检查参数和权限，操作内核对象

从这里开始，OS 的边界清楚了：用户代码不能直接操作所有资源，而是通过受控入口进入内核；内核根据当前上下文、权限和对象状态决定是否允许请求。

异常入口保存的是诊断证据。异常不是“出错就跳到内核”这么简单。硬件必须保存足够证据，让内核判断能否修复。以 page fault 为例，内核需要知道 faulting address、访问类型、当前特权级、出错指令地址和错误码。以非法指令为例，内核需要知道是哪条指令，以便发送信号或模拟指令。

证据	用途	缺失后果
fault RIP	返回或报告出错指令	无法重试或定位错误
fault address	判断哪个虚拟地址出错	无法处理缺页或 SIGSEGV
access type	读、写、执行、用户/内核	无法区分 COW、NX、权限错误
RFLAGS/CPL	中断状态、特权级、条件位	返回路径不安全
error code/cause	架构提供的分类原因	只能粗暴终止，无法精细恢复

这也是内核崩溃日志要打印寄存器、RIP、栈和 fault address 的原因。它们不是噪声，而是硬件给出的最低层证据。没有这些证据，调试只能猜。

上下文保存。中断、异常和调度都需要保存上下文。x86-64 的最小上下文包括 RIP、RFLAGS、通用寄存器和 RSP。若要切换任务，还要保存当前任务的内核栈、地址空间标识和调度账本。

```
struct Context {
    uint64_t rip;
    uint64_t rsp;
    uint64_t rflags;
    uint64_t regs[16];
};

void switch_to(Task* prev, Task* next) {
    save_context(&prev->ctx);
    current = next;
    restore_context(&next->ctx);
}
```

这段伪代码隐藏了真实汇编细节，但说明了不变量：一个任务被切走时，它的下一条指令位置和栈必须能恢复；一个任务被恢复时，寄存器必须像它从未离开 CPU 一样。上下文保存不正确，任何高级调度策略都没有意义。

上下文分两类：异常上下文和调度上下文。异常上下文由硬件入口和低级入口共同保存，描述“被打断的执行现场”。调度上下文描述“线程被切走后下次如何继续”。二者有关但不完全相同。系统调用和中断进入内核时，需要 trap frame；线程主动睡眠或被调度出去时，需要保存 callee-saved 寄存器、内核栈位置和调度状态。

上下文	保存内容	典型时机
trap frame	用户 RIP、RSP、RFLAGS、系统调用参数、错误码	中断、异常、系统调用入口
switch context	内核栈指针、callee-saved 寄存器、返回地址	scheduler 切换内核线程
地址空间上下文	页表根、PCID、TLB 状态	切换进程地址空间
浮点/SIMD 上下文	FPU、向量寄存器、控制字	用户使用浮点/SIMD 或信号处理

把这几类上下文混在一起会导致很多误解。例如中断打断用户程序时，内核需要 trap frame 才能返回用户态；调度器在两个都已进入内核的线程之间切换时，通常只需要保存内核执行所需的最小寄存器集合。地址空间切换还要改页表根和处理

TLB。

trap frame、switch context 和 stack frame 的区别。“保存现场”这个说法太粗，会掩盖三种完全不同的结构。trap frame 是硬件入口和汇编入口为异常/中断/系统调用保存的用户或被打断现场；switch context 是调度器在内核线程之间切换时保存的最小恢复信息；stack frame 是普通函数调用在栈上的返回地址、旧帧指针和局部变量布局。

结构	谁创建	用来回答什么问题
trap frame	CPU 硬件入口 + 低级汇编入口	从哪个 RIP/RSP/RFLAGS 返回用户态或被打断点
switch context	调度器和架构切换代码	下次调度到该线程时，从哪个内核位置继续
stack frame	编译器按 ABI 生成的函数调用代码	当前函数返回到哪里，局部变量在哪里
signal frame	内核在用户栈上构造	用户态 signal handler 结束后如何恢复原现场

一个线程在系统调用中睡眠时，通常同时拥有这些结构：用户态曾经有 stack frame；进入内核后有 trap frame；内核 C 函数有自己的 stack frame；调度出去时还要保存 switch context。调度器切换的是“当前内核执行点”，不是直接切换到用户程序入口；返回用户态时才消费 trap frame。

```
user calls read()
-> user stack frame exists
-> syscall creates trap frame
-> kernel functions create kernel stack frames
-> task sleeps, scheduler saves switch context
-> later scheduler restores switch context
-> syscall completes and return path restores trap frame
```

这也解释了为什么每个线程需要独立内核栈。线程睡在内核里时，它的内核 stack frames 和 trap frame 都还活着；另一个线程进入内核不能覆盖这段栈。

中断入口的最小状态机。中断入口要做的第一件事不是调用 C 函数，而是保存足够现场，让被打断代码将来能继续。可以把入口写成四段：硬件自动保存、汇编入口补充保存、C 层分发、返回路径恢复。

```
interrupt_entry(vector):
    frame = hardware_saved_frame()
    save_scratch_registers(frame)
    switch_to_kernel_stack_if_needed(frame)
    dispatch_interrupt(vector, frame)
    maybe_preempt_on_return(frame)
    restore_registers(frame)
    iret(frame)
```

这个状态机维护两个不变量。第一，任何会被中断处理函数破坏的寄存器，要么已经保存，要么按 ABI 规定不需要保存。第二，返回用户态之前，内核必须处理挂起信号、抢占标志和可能的调度请求。若在中断中直接切换任务，必须保证旧任务的内核栈仍保存完整 trap frame。

精确中断和延迟中断。外部中断通常在指令边界被接收，CPU 完成当前指令的架构效果后进入中断入口。这样内核返回时可以从下一条指令继续。异常则和当前指令绑定：缺页异常通常要返回到同一条指令重试，除零或非法指令可能发送信号。精确性让 OS 能用同一套保存/恢复机制处理多种入口。

```
external interrupt:
    instruction N committed
    interrupt taken before N+1
    saved RIP = address of N+1

page fault:
    instruction N cannot complete
    saved RIP = address of N
    kernel may fix page and retry N
```

中断延迟来自多个来源：CPU 临时关中断、硬件优先级屏蔽、长时间持有自旋锁、中断控制器路由、cache miss、下半部积压。实时系统关心最大延迟；通用系统也要避免长时间关中断，因为它会让定时器、网络、块 I/O 和 RCU 都延后。

中断控制器与优先级。真实硬件通常通过中断控制器管理多路中断。控制器至少要完成屏蔽、优先级、确认和结束通知。教学模型可以抽象成 pending bitmap：

```
interrupt_controller_tick():
    pending = pending_bits & enabled_bits
    if pending == 0:
        return
    vector = highest_priority(pending)
    pending_bits.clear(vector)
    cpu_raise_interrupt(vector)

end_of_interrupt(vector):
    controller.eoi(vector)
```

若忘记 EOI，控制器可能不再投递后续同类中断；若过早 EOI，同一个设备可能在 handler 尚未处理完共享状态时再次打断。驱动通常先读取设备状态并清设备侧中断源，再向控制器确认完成。顺序错误会导致中断风暴或丢事件。很多“中断风暴”并非来自设备本身，而是来自清除顺序错误或设备侧 pending 条件未被真正消费。

IRQ、vector、handler 和 completion 的区别。设备 I/O 里还有一组容易混的词。IRQ 是“有中断事件需要处理”的硬件/内核资源；vector 是 CPU 入口编号；handler 是内核注册的处理函数；completion 是某个具体 I/O 请求完成的事实。一个 IRQ 可以对应一个队列，一次 handler 可以处理多个 completion，一个 completion 也可能在轮询模式下被发现而不是靠中断发现。

概念	表示什么	常见误解
IRQ	CPU 被通知有事件	误以为它等于某个请求完成
line/vector		
handler	处理该类事件的函数	误以为 handler 只能处理一个事件
completion	设备写回的请求结果	误以为一定伴随一次中断
entry		
wait queue	等待某个条件的任务集合	误以为中断直接唤醒某个用户函数
poll budget	一次下半部最多处理多少完成	误以为只要有中断就会处理完所有包/请求

高吞吐设备通常把中断当成“提醒你看队列”，而不是“这个请求完成了”的一

对一消息。网卡可能一次中断后处理几十个 RX completion; NVMe 可能一个 MSI-X vector 对应一个 completion queue; NAPI 甚至会暂时关闭中断, 改用轮询批量处理。这样吞吐更好, 但延迟、budget、队列深度和背压都必须进入设计。

电平触发、边沿触发和 MSI。中断触发方式影响 handler 的确认顺序。边沿触发在信号变化瞬间产生事件; 若软件没及时处理, 可能错过后续边沿。电平触发只要设备中断线保持有效, 控制器就认为中断仍 pending; 若 handler 没清设备状态, EOI 后会立刻再次进入。MSI/MSI-X 则是设备写一个中断消息, 通常不依赖共享物理线。

触发方式	特点	handler 重点
边沿触发	变化瞬间产生一次事件	不能丢边沿, 事件队列要足够
电平触发	条件保持则持续 pending	必须清设备侧原因再 EOI
MSI/MSI-X	设备写消息触发 vector	vector 分配、队列亲和性、屏蔽同步

OS 不能只从 CPU 侧看中断, 还要看设备侧 pending 条件是否被真正清掉。

top half 与 bottom half 的分工。中断上半部只做必须立即完成的工作: 读取硬件状态、清中断源、把事件入队、唤醒下半部。下半部可以在软中断、工作队列或普通内核线程中运行。

```
irq_top_half(dev):
    status = mmio_read(dev.status)
    mmio_write(dev.ack, status)
    push_event(dev.event_queue, status)
    schedule_bottom_half(dev)

bottom_half(dev):
    while pop_event(dev.event_queue, &event):
        process_event(event)
        wake_waiters(dev.waitq)
```

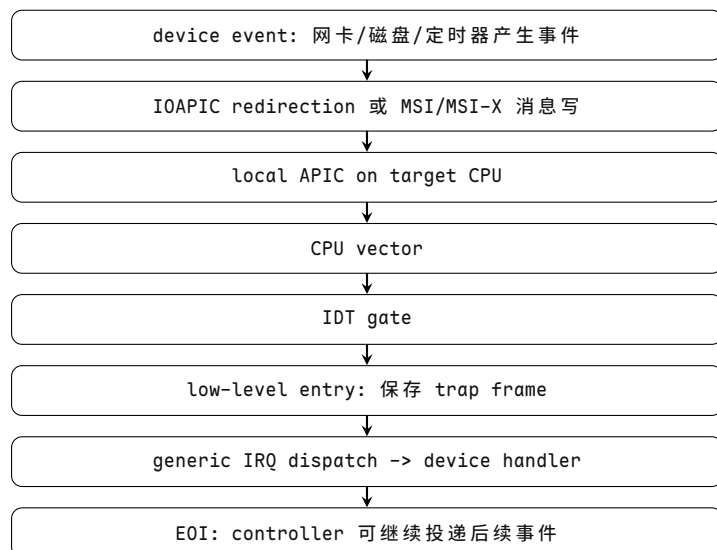
分工的理由是延迟控制。上半部越长, 其他中断越晚被处理; 下半部越慢, 队列越容易积压。因此测试不只看功能, 还要看最大关中断时间、队列峰值和事件丢失计数。

定时器数据结构选择。若每个 tick 扫描所有定时器, 复杂度是 $O(n)$ 。最小堆适合定时器数量中等、过期时间分散的系统; 时间轮适合大量短超时; 分层时间轮适合同时覆盖短超时和长超时。

结构	插入	取最近超时	适用场景
线性表	$O(1)$	$O(n)$	少量定时器, 教学 baseline
最小堆	$O(\log n)$	$O(1)$	通用超时, 精度清楚
单级时间轮	接近 $O(1)$	接近 $O(1)$	大量短超时, 槽宽可接受
分层时间轮	接近 $O(1)$	接近 $O(1)$	长短超时混合

选择定时器结构时要写清精度、最大超时、插入成本和 tick 成本。一个只在 10 个定时器上正确的实现, 不能直接推广到 100000 个 socket 超时。

下面的图把一次设备事件从产生到 EOI 的完整投递路径画出来。读这段路径时要记住: 每一跳都对应一种具体的硬件结构或内核数据结构, 任何一跳出错都会让事件丢失或重复投递。



图示：上半段是硬件路由：设备事件经 IOAPIC 或 MSI 消息送到目标 CPU 的 local APIC，再变成 vector。下半段是软件分发：IDT gate 找到入口，低级汇编保存 trap frame，通用 IRQ 层分发到设备 handler，最后 EOI 通知控制器可以继续。任何一跳的路由、亲和性或清除顺序出错，都会让事件丢失或重复投递。

1.3 引导、链接脚本与启动状态机

从裸机程序走向内核，第一道门是启动。先看一个最容易误判的写法：

```
power_on:
    call kernel\_main
```

这段伪代码看起来干净，但在真实机器上通常不能成立。C 函数不是孤立存在的，它默认栈已经可用，全局变量已经初始化，代码和数据在链接器预期的位置，必要地址能访问，最小输出设备已经准备好。上电瞬间，这些条件都不是天然存在的。

机器上电后，CPU 只知道从复位向量或固件指定入口取第一条指令。复位向量就是架构规定或固件约定的起始地址。CPU 不会自动理解“这是内核主函数”，也不会自动准备堆、栈、全局变量和设备。操作系统必须把“二进制文件如何进入内存、第一条指令在哪里、栈在哪里、全局变量如何初始化、设备何时可用”这些问题全部讲清。

在单片机上，启动路径通常是 reset handler：设置栈指针，把 `.data` 从 Flash 复制到 SRAM，把 `.bss` 清零，初始化时钟和最小串口，然后调用 `main` 或 `kernel_main`。`.data` 是带初值的全局数据段，例如 `int x = 7;`。`.bss` 是未显式初始化的全局数据段，C 语言要求它运行时从 0 开始。在 PC 或服务器上，路径更长：固件初始化硬件，bootloader 加载内核镜像和启动参数，切换 CPU 模式，建立最小页表，最后跳到内核入口。

启动阶段的特点是“没有服务可用”：还没有调度器不能睡眠，还没有内存分配器不能随便 `malloc`，还没有完整中断不能依赖异步事件，还没有文件系统不能读普通文件，调试输出只能靠串口或早期 console。任何 early init 函数偷偷调用需要调度器或内存分配器的路径，都会在小配置上“刚好能跑”、在大配置上随机损坏——这正是内核初始化必须按严格阶段组织的原因。

上电后 CPU 为什么不能直接运行内核 C 函数。C 函数依赖一组运行环境：栈指针有效，全局变量已初始化，BSS 为零，代码和数据位于链接器预期地址，调用约定可用，必要的地址映射存在。上电瞬间这些条件都不自动成立。CPU 只处在架构定义的复位状态，从固定入口取指；剩下的环境必须由启动代码一步步建立。

可以按下面这条线理解从“不能调用 C”到“可以调用 `kernel_main`”的过渡：

```
Reset:
    CPU only has architectural reset state

MinimalStack:
    set stack pointer, now simple calls can return

DataCopied:
    copy initialized globals to runtime memory

BssCleared:
    zero uninitialized globals

EarlyConsole:
    prepare minimal output for failure evidence

KernelMain:
    now ordinary early C code has a reliable base
```

每一步都只解决一个问题。先有栈，函数调用才不会把返回地址写到未知位置；再有 `.data/.bss`，全局锁、队列和指针才有可预测初值；再有 `early console`，后面的失败才有证据。

C 代码假设	谁建立	若缺失会怎样
栈可用	reset handler 或 bootloader	函数调用和局部变量破坏未知内存
全局变量已初始化	启动代码复制 <code>.data</code> 、清 <code>.bss</code>	锁、队列、指针从随机值开始
代码地址正确	链接脚本和装载器	跳转、全局地址、异常表错误
内存可访问	页表或物理映射	取指或访存 <code>fault</code>
最小输出可用	<code>early console</code> 初始化	出错时没有证据

所以内核入口往往先是汇编或极小 C 代码。它不是为了炫技，而是因为在运行时环境建立前，普通 C 抽象还没有可靠落脚点。

x86-64 早期入口、`_start` 与最小启动代码。早期启动的第一份“内核源码”通常不是 C 函数，而是入口汇编和链接脚本。x86-64 上，UEFI/bootloader 或 EFI stub 把内核镜像放入内存，传入启动参数和内存地图；早期入口建立最小栈、清 BSS、准备早期页表和 IDT，然后才调用 `kernel_main` 一类 C 入口。

```
early_x86_64_entry:
    lea rsp, [rip + boot_stack_top]
    copy .data from flash LMA to sram VMA
    zero .bss
    build early page tables
    load provisional IDT
    call early_console_init
    call kernel\_main
halt_loop:
    hlt
    jmp halt_loop
```

这里已经出现了 OS 启动的三个根概念：入口地址、内存布局和异常表。后面 IDT、内核入口和多核启动更复杂，但本质仍是“硬件和 `bootloader` 按固定规则跳到一段受信任代码，代码建立可运行环境”。

链接脚本决定内存布局。链接脚本把代码段、只读数据、已初始化数据、未初始化数据和栈安排到具体地址。代码段通常叫 `.text`，只读数据段通常叫 `.rodata`，已初始化全局数据是 `.data`，未初始化全局数据是 `.bss`。没有链接脚本，编译器和链接器不知道内核应该放在物理地址哪里，也不知道哪些符号表示段边界。

```
SECTIONS {
    . = 0x80200000;
    __kernel_start = .;
    .text : { *(.text*) }
    .rodata : { *(.rodata*) }
    .data : { *(.data*) }
    __bss_start = .;
    .bss : { *(.bss*) *(COMMON) }
    __bss_end = .;
    __kernel_end = .;
}
```

这些段边界符号不是装饰。启动代码需要 `__bss_start` 和 `__bss_end` 来清零 BSS；早期物理页分配器需要 `__kernel_end` 知道哪些内存已经被内核占用；页表初始化要知道哪些区域可执行、只读或可写。若链接脚本导出的段边界不单调递增或不按页对齐，后续清 BSS、页分配和页表权限设置都会出错。

链接地址、加载地址和重定位。链接地址是代码认为自己运行的位置，加载地址是 bootloader 把镜像放到内存的位置。二者可以相同，也可以不同。若不同，早期代码要么以位置无关方式运行，要么完成重定位，要么建立页表把链接虚拟地址映射到实际物理地址。重定位就是把“代码里原本按某个地址计算的引用”修正到实际运行地址。

```
kernel linked at virtual address: 0xffffffff81000000
kernel loaded at physical address: 0x00100000

early page table maps:
0xffffffff81000000 -> 0x00100000
```

这解释了为什么 64 位内核常有“高半区内核”概念。用户进程使用低地址范围，内核映射在高虚拟地址范围；每个进程切换页表时仍保留内核映射，系统调用和中断进入内核后能访问同一套内核代码和数据。这个设计依赖页表，而不是物理内存真的在高地址。

VMA 与 LMA：为什么 data 要复制。链接脚本里常见 VMA 和 LMA。VMA 是代码运行时使用的地址，LMA 是镜像中加载的位置。已初始化全局变量的初值保存在 Flash 中，但运行时变量要位于 SRAM，所以启动代码要从 LMA 复制到 VMA。

```
MEMORY {
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
    SRAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}

SECTIONS {
    .text : { *(.text*) *(.rodata*) } > FLASH
    .data : AT(ADDR(.text) + SIZEOF(.text)) {
        __data_start = .;
        *(.data*)
        __data_end = .;
    } > SRAM
```

```

__data_load = LOADADDR(.data);
.bss : {
    __bss_start = .;
    *(.bss*) *(COMMON)
    __bss_end = .;
} > SRAM
}

```

若 `.data` 未复制，带初值的全局对象会从错误内容开始；若 `.bss` 未清零，锁、队列和指针的初始值不可预测。这个失败和大型 OS 的早期页表、per-CPU 区域未初始化一样，通常不会在第一条错误处崩溃，而是在后续随机位置表现出来——这正是启动 bug 最难定位的原因。

启动阶段的状态机。前面的步骤可以收束成启动状态机：

```

Reset
-> MinimalStack
-> DataCopied
-> BssCleared
-> EarlyConsole
-> BootMemoryDiscovered
-> PageTablesReady
-> InterruptsReady
-> SchedulerReady
-> InitProcessStarted

```

每个阶段只能使用之前已经建立的能力。`EarlyConsole` 之前不能输出复杂日志；`BootMemoryDiscovered` 之前不能建立通用页分配；`SchedulerReady` 之前不能阻塞等待；`InterruptsReady` 之前不能依赖设备中断。



图示：启动是一条单向状态链。每个阶段只能使用之前已经建立的能力：栈可用之前不能调用普通 C 函数；全局变量初始化之前锁和队列不可预测；console 之前失败无证据；内存图之前不能分配页；页表之前地址翻译不可用；中断之前不能依赖设备事件；调度器之前不能阻塞。任何 early init 函数跨阶段使用未就绪能力，都会在小配置上巧合通过、在大配置上随机损坏。

BSS 清零和 data 复制。

```

boot_zero_bss():
    p = __bss_start
    while p < __bss_end:
        *p = 0
        p += word_size

boot_copy_data():
    src = __data_load_start
    dst = __data_start
    while dst < __data_end:
        *dst++ = *src++

```

这两个循环是 $O(n)$ ， n 是段大小。它们必须在使用对应全局变量之前完成。若 BSS 没清零，全局队列、锁和指针会从随机值开始，后续 bug 很难定位。

早期 bump allocator。完整页分配器建立前，内核常用 bump allocator 分配早期对象。它只维护一个 next 指针，每次按对齐向上推进，不支持释放。

```

early_alloc(size, align):
    next = align_up(early_next, align)
    end = next + size
    if end > early_limit: panic()
    early_next = end
    return next

```

分配是 $O(1)$ ，实现极简单。代价是不能释放，且必须在切换到正式分配器后停止使用。它适合页表、启动参数副本、早期设备表等生命周期贯穿启动阶段的对象。

x86-64 模式演进：实模式、保护模式、长模式。x86-64 的历史兼容层让启动路径明显分阶段。复位后 CPU 从兼容模式开始，早期代码或固件/bootloader 需要逐步建立 GDT、打开保护机制、准备页表、启用 PAE 和长模式，最后跳到 64 位代码。现代 UEFI 固件和 bootloader 会替内核完成大量工作，但内核仍要理解进入时的 CPU 模式、页表、内存地图和调用约定。

```

legacy shape:
    reset vector
    -> real mode firmware/bootloader
    -> load kernel setup code / image + initrd
    -> build provisional GDT and boot parameters
    -> enter protected mode
    -> build early page tables, enable PAE
    -> enable long mode (EFER.LME)
    -> jump to 64-bit kernel entry / start_kernel()

```

实模式只能直接访问有限地址空间，保护模式引入段和保护，长模式引入 64 位寄存器和分页模型。操作系统教材不需要把每条切换指令都背下来，但必须知道：64 位内核能运行，是因为早期代码已经设置好 GDT、控制寄存器、EFER.LME、页表和跳转序列。GDT 在长模式下不再像 32 位分段那样用于普通地址计算，但仍参与代码段、数据段、TSS、特权切换和系统调用入口设置；TSS 也不只是历史名词，x86-64 可用 IST 为 NMI、double fault 等关键异常提供专用栈，避免在当前栈损坏时继续崩溃。

BIOS、UEFI 和 bootloader 的职责边界。BIOS 风格启动和 UEFI 风格启动不同，但目标相同：把内核镜像和硬件描述交给内核。bootloader 不应替内核做所有工作，它只建立足够条件让内核接管。内核接管后，必须停止依赖大部分固件 Boot Services，改用自己的内存管理、驱动和中断。

阶段	可依赖对象	退出后的边界
----	-------	--------

固件早期	DRAM 初始化、平台基础设置	内核通常看不到细节，只接收结果
bootloader	读取磁盘/网络、加载镜像、构造参数	不再参与普通运行时服务
UEFI Boot Services	分配内存、访问协议、获取内存地图	<code>ExitBootServices</code> 后不可再调用
内核 early boot	启动参数、内存地图、早期 console	必须校验并建立自己的对象

`ExitBootServices` 是一个硬边界。退出前，固件仍可能拥有内存和设备；退出后，内核承担管理责任。若内核拿到内存地图后，固件又分配了内存，而内核没有重新获取地图，就可能把固件仍使用的页当成空闲页。

两种风格的具体差异值得一提：BIOS 常从磁盘引导扇区开始，靠中断调用访问受限固件服务，内存地图来自 `e820`，图形走 `VGA/vesa`；UEFI 则以 `EFI` 应用或 `EFI stub` 为入口，提供标准化的 `Boot/Runtime Services` 和 `GetMemoryMap`，图形走 `GOP framebuffer`，并以 `ExitBootServices` 作为硬边界。无论哪种风格，内核接管的标志都是同一条：固件退出后，内核必须靠自己的驱动和内存管理独立运行。

1.4 内核初始化：从 `kernel_main` 到第一个任务

`kernel_main`（或 Linux 的 `start_kernel`）被调用时，运行时环境刚刚勉强够用：栈、`.data/.bss`、`early console` 已经就绪，但调度器、完整中断、内存分配器和文件系统都还没有。本节追踪内核如何在这一起点之上，按依赖图把各子系统依次建立起来，直到第一个用户任务可以运行。

初始化依赖图。内核初始化不能只靠函数排列顺序。一个模块可能依赖 `console`，一个模块依赖内存分配器，一个模块依赖中断，一个模块必须在调度器启动前完成。把这些关系写成依赖图，才能解释为什么初始化常被分成 `early`、`core`、`subsys`、`device`、`late` 等阶段。

```
init_node {
  name: "page_allocator"
  requires: ["boot_memory_map", "early_console"]
  provides: ["alloc_pages"]
}

run_init_graph(nodes):
  ready = nodes with all requirements satisfied
  while ready not empty:
    n = pop(ready)
    run(n.init)
    mark_provided(n.provides)
    add newly unblocked nodes to ready
  if unfinished nodes remain:
    panic("init dependency cycle or missing provider")
```

核心理想

初始化依赖图的实质是拓扑排序，复杂度是 $O(V + E)$ 。它的工程价值不是算法本身，而是把“某函数必须先跑”从隐含的代码顺序变成可检查约束。若出现环，例如文件系统初始化依赖块设备、而块设备初始化又要从文件系统读取配置，就必须拆出更早的配置来源。可检查的依赖关系比“作者记得调用顺序”可靠得多。

启动参数与可信边界。bootloader 会把 UEFI 内存地图、命令行、initrd 和 ACPI 表交给内核。内核不能盲信这些数据，因为它们可能来自固件 bug、配置错误或攻击面。早期解析必须检查地址范围、长度、对齐和重叠。

```
validate_memory_region(region):
    require region.start aligned to PAGE_SIZE
    require region.length > 0
    require checked_add(region.start, region.length, &end)
    require end <= physical_address_limit
    require not overlaps(kernel_image)
    require not overlaps(previous_accepted_regions)
```

若把内核镜像所在物理页误标为空闲，后续页分配器会把正在运行的内核代码分配给别人。启动阶段的错误通常不是立刻崩溃，而是在很久以后变成随机损坏，因此必须在早期就把内存地图打印和校验做好。

ACPI 和 PCI 配置空间：硬件不是凭空出现的。内核要知道“有哪些 CPU、哪些内存、哪些设备、设备寄存器在哪、用哪个 IRQ、挂在哪个 NUMA node、有哪些电源关系”。在本书的 x86-64 主线里，这部分信息主要来自 UEFI/ACPI、PCI/PCIe 配置空间和固件内存地图。它们都不是设备驱动本身，而是硬件描述入口。

描述来源	x86-64 常见用途	描述内容
ACPI	PC、服务器	CPU 拓扑、NUMA、PCI routing、电源、热管理、平台设备资源
PCI config	可枚举 PCIe 设备	vendor/device、BAR、MSI、能力链
UEFI memory map	启动阶段内存交接	RAM、保留区、runtime services、ACPI reclaim 区域

驱动 probe 的输入很多来自这些描述。若 ACPI resource 或 PCI BAR 错，MMIO 映射到错误地址；若 IRQ routing 或 MSI-X 配置错，设备完成不会唤醒驱动；若 DMA mask 或 IOMMU domain 配置错，设备可能访问不到高地址内存。硬件描述错误会表现成软件 bug，所以启动阶段要尽早暴露和校验资源表。

从单核启动到多核启动。多核系统通常只有 bootstrap processor 先运行内核入口，其他 CPU 处于等待状态。主 CPU 建立全局页表、per-CPU 区域和调度数据后，再唤醒 application processors。每个 CPU 都需要自己的内核栈、当前任务指针、runqueue 和中断状态。

```
boot_cpu_main():
    setup_global_memory()
    setup_percpu_area(boot_cpu)
    for cpu in secondary_cpus:
        allocate_cpu_stack(cpu)
        send_startup_ipi(cpu, secondary_entry)
    wait_until_all_online()
    start_scheduler()

secondary_entry(cpu):
    load_kernel_page_table()
    set_percpu_base(cpu)
    init_local_interrupts()
    mark_cpu_online(cpu)
    idle_loop()
```

这里的不变量是：CPU online 之前不能被调度器选作迁移目标；per-CPU 数据初始化之前不能接收普通设备中断；所有 CPU 使用的内核文本映射必须一致。多核启动把“初始化顺序”从单线流程变成了并发协议。

per-CPU 数据为什么必须很早准备。多核 OS 不能让所有 CPU 频繁写同一批全局变量。每个 CPU 需要自己的当前任务指针、内核栈、runqueue、计数器、本地 timer 状态、抢占计数和中断统计。这些对象通常放在 per-CPU 区域，通过特殊基址寄存器或固定偏移快速访问。

```
per_cpu_area[cpu]:
    current_task
    kernel_stack_top
    runqueue
    irq_count
    preempt_count
    local_timer_state
```

secondary CPU 进入内核后，必须先设置自己的 per-CPU base，才能安全访问 `current` 这类隐式变量。否则 CPU1 可能误用 CPU0 的当前任务或 runqueue，造成调度和中断统计混乱。per-CPU 既是多核性能优化，也是正确性边界：在 base 设置完成前接收设备中断或被调度器选中，都会让一个 CPU 误操作另一个 CPU 的私有状态。

Linux start_kernel 调用序列。Linux 的 `start_kernel` 是阅读大型内核初始化的主干。它不是所有细节的开始，但它把架构初始化、内存、调度、RCU、中断、定时器、VFS、proc、driver model、init 进程串起来。

```
start_kernel()
    setup_arch()
    setup_command_line()
    mm_init()
    sched_init()
    radix_tree_init()
    trap_init()
    init_IRQ()
    tick_init()
    rcu_init()
    vfs_caches_init()
    rest_init()
```

不同版本顺序会变化，读源码时应抓依赖：`setup_arch` 解析固件和内存；`mm_init` 建立内存分配基础；`sched_init` 建立 runqueue；`trap_init/init_IRQ` 建立入口和中断；`rest_init` 创建 kernel thread 和第一个用户态 init 的路径。

initcall 机制。驱动和子系统不能全都手写进一个巨大初始化函数。Linux 用 initcall 等级把初始化函数放入特殊段，启动时按等级依次调用。常见等级包括 early、core、postcore、arch、subsys、fs、device、late。

```
early_initcall(foo_early);
core_initcall(mm_subsystem);
subsys_initcall(bus_subsystem);
fs_initcall(vfs_feature);
device_initcall(driver_init);
late_initcall(debug_late);
```

initcall 的实质是链接脚本和函数指针数组。它让模块能声明“我属于哪个初始化阶段”，但不能自动解决所有依赖。若一个 device initcall 假设某总线已注册，而总线初始化等级写错，启动会出现只在特定配置下复现的失败。

多核启动和 CPU hotplug。bootstrap processor 先运行全局初始化，然后通过 IPI 或固件接口唤醒 application processors。每个 CPU 上线前要设置 per-CPU base、内核栈、本地 APIC、idle task 和 runqueue。CPU hotplug 则要求运行中 CPU 可下线 and 上线，所以初始化/清理路径必须可重复、可回滚。

```
bringup_cpu(cpu):
    allocate_idle_task(cpu)
    allocate_percpu_area(cpu)
    send_startup_ipi(cpu)
    wait cpu reports online
    add cpu to scheduler domains
    enable interrupts on cpu
```

失败案例：CPU 已经标记 online 但本地 timer 未初始化，调度器可能把任务迁过去后无法抢占；CPU 下线时未迁走 timer 或 workqueue，后续事件投递到离线 CPU。多核启动不是“让其他核心开始跑”，而是把每个 CPU 纳入调度、中断和时间子系统的一致状态。

1.5 源码级补充：MCU 启动、UART 与中断路径

读真实代码时，先把一条最短路径追通，比试图一次理解整个框架更有效。本节给出从 MCU 启动文件到 Linux 中断路径的若干阅读入口。

GPIO 和 UART 寄存器。裸机驱动的核心是 MMIO。GPIO 通常有模式寄存器、输出数据寄存器、输入数据寄存器；UART 通常有状态寄存器、数据寄存器、波特率 divisor、控制寄存器。真实芯片位名不同，但结构类似。

```
#define UART_SR (UART_BASE + 0x00)
#define UART_DR (UART_BASE + 0x04)
#define UART_BRR (UART_BASE + 0x08)
#define UART_CR (UART_BASE + 0x0c)
#define UART_TXE (1u << 7)
#define UART_RXNE (1u << 5)

void uart_putc(char c) {
    uint32_t deadline = ticks + 10;
    while (!(read32(UART_SR) & UART_TXE)) {
        if (time_after(ticks, deadline)) {
            uart_errors.timeout++;
            return;
        }
    }
    write32(UART_DR, c);
}
```

这里的超时是工程边界。没有超时的驱动把硬件故障扩大成系统停顿。大型内核中的块设备超时、网络设备 reset、watchdog 都是同一思想：设备是异步状态机，不是可靠函数。

x86 异常入口保存了什么。x86-64 的中断和异常入口由 IDT 描述。CPU 根据 vector 找到 gate，切换特权级时切到内核栈，硬件压入返回所需状态。某些异常还会压入 error code。汇编入口随后保存通用寄存器，形成内核能理解的寄存器帧。

```
struct TrapFrame {
    // software-saved general registers
```

```
u64 r15, r14, r13, r12, r11, r10, r9, r8;
u64 rdi, rsi, rbp, rbx, rdx, rcx, rax;

// hardware-saved frame
u64 vector;
u64 error_code;
u64 rip;
u64 cs;
u64 rflags;
u64 rsp;
u64 ss;
};
```

真实布局会因入口类型和内核版本变化，但读源码时要抓住语义：`rip/cs/rflags/rsp/ss` 决定如何返回；`error code` 帮助 `page fault` 和保护异常分类；通用寄存器保存系统调用参数和被打断计算状态。

IDT、APIC、IOAPIC 和 MSI 的投递路径。IDT 是 CPU 本地表；APIC/IOAPIC/MSI 解决“设备事件如何送到哪个 CPU 的哪个 vector”。传统外设中断可经 IOAPIC 重定向到本地 APIC；PCIe 设备常用 MSI/MSI-X，通过一次内存写消息触发中断。多核重调度和 TLB shutdown 使用 IPI，由一个 CPU 通过本地 APIC 向另一个 CPU 投递。完整的投递路径见本节前面的 pdfdiagram，这里只补充性能要点。

中断亲和性影响性能。网卡 RX 队列中断若全部打到 CPU0，CPU0 会成为瓶颈；把队列中断分散到处理对应网络栈的 CPU，可以减少跨核 cache line 迁移。但过度分散也会让连接状态在 CPU 间跳动。真实系统因此有 `irq affinity`、`RSS/RPS/XPS` 等策略。

Linux 中断路径阅读点。读 Linux 中断路径可以从这些位置开始：

- `arch/x86/kernel/idt.c`: IDT gate 初始化。
- `arch/x86/entry/entry_64.S`: 低级入口、寄存器保存、返回路径。
- `kernel/irq/handle.c`: 通用 IRQ 分发。
- `kernel/irq/manage.c`: `request_irq`、`request_threaded_irq`。
- `kernel/time/timer.c`、`kernel/time/hrtimer.c`: 低精度 timer wheel 与 `hrtimer`。

阅读时不要只看 handler 函数。要同时看谁注册 handler、handler 运行在何种上下文、是否允许睡眠、谁发送 EOI、错误返回如何统计、设备移除时如何注销并等待 in-flight handler 完成。

四种下半部机制。

机制	特点	适用场景
softirq	固定类型，常用于网络和块层，高并发	极热路径，要求不能随意睡眠
tasklet	基于 softirq 的延后执行，同一 tasklet 串行	旧驱动常见，新代码逐步少用
workqueue	在内核线程中执行，可睡眠	需要分配内存、拿 mutex、访问慢设备
threaded IRQ	把 IRQ handler 线程化	降低硬中断时间，适合较复杂设备处理

选择机制的核心问题是上下文：是否需要睡眠，是否要高吞吐，是否允许同一事件并行，是否要限制 CPU 占用。把会睡眠的代码放进 `hard IRQ` 或 `softirq`，是典

型内核 bug。

hrtimer、timer wheel 与 tickless。低精度超时适合 timer wheel，例如 TCP 重传、普通超时、周期任务；高精度计时适合 hrtimer，例如纳秒级睡眠、音视频定时、精确 profiler。tickless 则在 CPU 空闲或单任务运行时减少周期 tick，降低功耗和噪声。

```
timer wheel:
    cheap insertion, coarse granularity, many ordinary timeouts

hrtimer:
    ordered by expiry in high-resolution time, higher overhead, precise wakeup

tickless:
    program next hardware timer to nearest event instead of fixed periodic tick
```

tickless 不表示没有时间。它表示内核不再无条件固定频率打断 CPU，而是计算下一个必要事件并设置硬件定时器。调度统计、RCU、负载均衡和超时都要适配这种模式。

真实代码阅读入口。读真实 MCU 代码时，建议按下面顺序找：启动汇编或 startup 文件（向量表、reset handler、栈顶符号）、链接脚本（Flash/SRAM 区域、`.text/.data/.bss`、堆栈布局）、HAL 或裸驱动（UART/GPIO/TIMER 寄存器定义和初始化序列）、中断文件（IRQ handler 如何清中断、入队、唤醒主循环）、示例 main（是否有阻塞等待、超时、错误计数）。x86-64 教学内核则可从 `boot/entry_64.S`、`kernel/head64.c`、`linker.ld`、`drivers/early_uart.c` 入手——不要先读完整驱动框架，先把 UEFI/bootloader 交接到第一个字符输出的路径追通。

1.6 实践

本章实践需要三组动手实验，覆盖裸机、中断和启动三个层面。

第一组是裸机实验，不需要复杂环境。拿任意一个单片机或模拟器，写三版程序：

1. 超级循环：固定顺序调用四个任务。
2. 周期任务表：每个任务有 period 和 next release。
3. 非阻塞状态机：把串口发送或传感器读取改成 step 函数。

报告要记录每个任务最长运行时间、周期、deadline、是否可能阻塞、是否访问共享状态。对于每个任务，写出下面的表：

字段	内容
输入	读取哪些寄存器、队列或全局状态
输出	写哪些寄存器、队列或全局状态
时间预算	最坏执行时间和周期
等待点	是否忙等硬件、锁或缓冲区
失败	超时、溢出、校验失败、设备错误

第二组是中断与定时器实验。建议写一个事件日志：每次进入中断、退出中断、唤醒任务、设置 need_reschedule，都记录 tick、事件类型和当前任务。然后用日志回答：哪个事件打断了谁，哪个任务被唤醒，什么时候真正运行。

```
tick=100 enter timer_isr current=control
tick=100 wake task=telemetry reason=period
tick=100 set need_reschedule
tick=100 exit timer_isr current=control
tick=100 switch control -> telemetry
```


同时实现两个定时器队列版本：全表扫描和最小堆。比较 n 从 10 到 100000 时，每个 tick 的处理成本。这个实验能说明为什么 OS 算法不能只看功能正确，还要看事件规模。

第三组是启动实验。先写一个启动依赖表：

阶段	可用能力	禁止事项
Reset	寄存器、固定入口	使用全局变量假设已初始化
MinimalStack	栈、少量汇编	调用复杂 C 运行库
BssCleared	零初始化全局变量	动态分配内存
EarlyConsole	最小输出	格式化复杂日志、大缓冲
BootMemory	物理内存范围	使用完整虚拟内存假设
PageTables	基础地址翻译	访问未映射设备
Scheduler	任务切换	在早期 init 中遗留永久锁

然后画出内核镜像布局：text、rodata、data、bss、boot stack、page tables、free memory。只要能解释“内核自己占了哪些页，哪些页可以交给分配器”，后续物理页管理才有根。

1.7 测试

本章覆盖三个层面的最低测试清单。
裸机层面的测试不是“灯会闪”，而是证明调度和状态机边界：

- 任一任务执行一次后必须返回主循环。
- 周期任务的 `next_release_ms` 单调前进，不因时间溢出立刻失控。
- 串口发送缓冲区满时，提交失败或施加背压，而不是覆盖旧数据。
- 非阻塞状态机在硬件未 ready 时不改变发送进度。
- 任一任务超时应被计数或记录，不能静默拖慢全系统。

中断与定时器层面的测试：

- 中断处理函数只做短路径工作，不调用可能阻塞的函数。
- 环形队列在主循环和 ISR 同时访问时不破坏 head/tail/size。
- 定时器 tick 单调增加，超时任务只被唤醒一次。
- 时间片用尽后设置 `reschedule` 标志，不能在任意位置破坏当前上下文。
- 上下文切换后，任务能从被切走的位置继续执行。
- 异常路径能区分可修复缺页、权限错误和不可恢复错误。

启动与内核初始化层面的测试：

- 链接脚本导出的段边界单调递增且按页对齐。
- BSS 清零后，全局计数器、指针和队列初值可预测。
- 早期分配器返回地址满足对齐，且不会覆盖内核镜像。
- 未初始化 console 前，日志路径不会访问空设备。
- 调度器 ready 前，任何初始化函数都不能睡眠。
- 页表 ready 后，text 只读可执行，data/bss 可写不可执行。

核心思想

本章的验收结论是一条贯穿三个层面的不变量：操作系统不是凭空出现的。裸机程序里出现多个活动、共享资源、等待硬件、错误恢复和时间预算时，就已经需要 OS 式的状态管理；中断和定时器让系统获得外部事件和抢占能力，但也迫使我们认真处理共享状态、上下文保存和入口权限；启动和内核初始化则把“硬件按固定规则跳到一段受信任代码”落实成可解释的阶段链。能从第一条指令解释到第一个用户任务启动，中间每一阶段可用能力和禁止事项都清楚，本章才算过关。

第 2 章 硬件事实与内核对象

用户已经有一本专门的硬件书，把晶体管、门电路、加法器、流水线、cache 层次和总线协议逐一推导过。本章不重讲这些内容。本章只做两件事：第一，把操作系统真正必须知道的硬件事实压成一份精简清单——CPU 特权级与受控入口、地址翻译与页表、MMIO 与 DMA、多核 cache 一致性、设备完成与持久化边界；第二，说明这些硬件事实怎样逼出 task、地址空间、VMA、fd、request、wait queue、completion 等内核对象。

task、地址空间、VMA、fd、request、wait queue 这些名字不是先验规定出来的。它们解决的都是同一个问题：硬件只给瞬时状态和低层事件，OS 必须把它们组织成可恢复、可检查、可等待、可取消、可释放的长期状态。只要这条线不清楚，后面讲调度、虚拟内存、文件系统和驱动时，就会像在背结构体字段。

核心思想

内核对象的第一原则是：把硬件给出的低层状态，变成可恢复、可检查、可等待、可释放的状态。进程不是“正在运行的代码”四个字，而是 CPU 现场、地址空间、文件表、信号、等待原因和调度状态的组合；I/O 请求也不是“调用设备函数”，而是 buffer、设备地址、request id、completion 和等待者的组合。

2.1 一、操作系统必须知道的硬件事实

硬件书里从需求一步步推出整台机器。OS 不需要重复这条推导，只需要抓住那些直接决定内核结构的事实。下面把这些事实分成五组，每组只说 OS 视角下“硬件给了什么、限制了什么”。如果某条事实的推导细节不清楚，请回到硬件书查阅；本章只保留它怎样影响内核。

CPU 特权级与受控入口。CPU 至少区分两种特权级：内核态（常称 ring0）和用户态（常称 ring3）。用户态不能直接修改页表、关中断或访问设备寄存器；执行特权指令会触发异常。用户程序进入内核只有几条受控路径，硬件在入口处保存返回 RIP、状态位，必要时切到内核栈。这是 trap frame 的硬件根。

入口	触发原因	特点
system call	用户主动执行受控入口指令，请求内核服务	主动而同步，参数按 ABI 放寄存器
page fault	MMU 无法完成某次取指、读或写	当前指令还没完成，fault address 和错误码由硬件给出
timer interrupt	定时器到期，CPU 被异步打断	用户或内核代码没有主动请求，抢占的硬件根
device interrupt	设备报告队列、completion 或错误状态	完成事实在设备侧，当前 CPU 上的 task 未必是提交者

没有受控入口，用户可以跳进内核任意位置；没有特权级，“用户程序”和“内核”就没有边界。入口路径必须极小心，因为它在系统最脆弱的位置运行：页表可能刚出错，用户指针不可信，栈也可能要切换。

地址翻译与页表。用户程序写下的是虚拟地址，不是物理地址。MMU 按页表把虚拟页翻译成物理页，并检查 present、R/W/X、U/S 等权限位。页表项叫 PTE。TLB 缓存最近翻译结果；TLB miss 时硬件自己 page walk 读页表；PTE 不存在或权限不允许时，CPU 进入 page fault。

路径	硬件怎么走	OS 是否接管
TLB 命中	虚拟页直接得到物理页号和权限，继续访问 cache	不接管，程序感觉只是一次普通读取
TLB miss 但 PTE 有效	page walker 读页表，检查权限，填 TLB	通常不接管，只是变慢
PTE 不存在	page walker 发现无有效映射	接管，内核查 VMA 后决定是否补页
权限失败	PTE 存在但写/执行/用户权限不允许	接管，可能 COW、发信号或杀进程

这里最容易混的是 TLB miss 和 page fault。TLB miss 只是翻译缓存没命中，硬件自己走页表，通常不进内核；page fault 是页表或权限告诉硬件这次访问不能直接完成，CPU 必须进内核。OS 修改 PTE 后要处理 TLB 中旧翻译——多核上还要做 TLB shutdown，否则别的 CPU 可能继续按旧权限访问旧物理页。

MMIO 与 DMA。设备不是函数，而是另一个按自己时钟前进的状态机。CPU 通过 memory-mapped I/O (MMIO) 访问设备寄存器：状态寄存器保存 READY/ERR，数据寄存器接收命令。MMIO 访问不能按普通 RAM 缓存或重排，因为读状态可能有副作用，写数据可能启动设备动作。

单字节设备靠 CPU 轮询状态位；慢设备或大块数据用 DMA，让设备直接读写内存。高速队列设备用 descriptor 加 submission/completion queue 加 doorbell：CPU 在内存里填描述符，写 MMIO doorbell 通知设备，设备 DMA 读写 buffer，完成后写 completion 并触发中断。IOMMU 给设备做地址翻译和权限检查，限制设备能访问的物理内存范围。

模型	机制	OS 关注
单字节 PIO	CPU 读状态寄存器，ready 后写数据寄存器	不能无限等，要有超时和等待队列
队列设备	descriptor ring + doorbell + completion + MSI-X	完成必须带 tag 才能找回具体请求
DMA	设备直接访存，buffer 所有权要明确	提交后不能释放，completion 前不能复用
IOMMU	IOVA 翻译成物理地址，按 domain 限制设备访问	没有 IOMMU，设备 DMA 错地址可覆盖任意内存

多核与 cache 一致性。多核 CPU 各有私有 L1/L2 cache。同一 cache line 在多个核心间由一致性协议转移所有权。counter++ 不是一步硬件动作，而是 load、add、store 三步；两个 CPU 同时执行就可能丢更新。硬件提供原子读改写指令（独占 cache line）和内存屏障（acquire/release）来约束可见顺序。store buffer 会延迟写入可见，源码顺序不等于其他 CPU 观察顺序。

多核硬件事实	直接问题	OS 应对方式
每核私有 cache	同一数据有多个副本	一致性协议、cacheline 对齐、per-CPU 数据
原子读改写要独占 line	热锁让 line 在 CPU 间迁移	queued spinlock、分片计数、减少全局锁
store buffer 延迟可见	源码顺序不等于观察顺序	acquire/release、内存屏障、锁语义

跨核通知靠 IPI

远端 CPU 要被打断配合

TLB shutdown、
reschedule IPI

内核经常把“同一件事”拆成每 CPU 状态，就是为了让热写路径留在本地 cache，不抢同一条 cache line。

设备完成与持久化边界。设备 completion 异步到达，可能乱序、合并或丢失。中断只是 CPU 入口，意思是“有设备事件需要处理”；completion 才是设备写下的结果，告诉驱动哪个 tag 完成、状态码是什么。若只根据中断返回用户态，内核不知道完成的是哪次 request。

completion 也不等于持久化。设备内部可能有易失缓存；用户看到 write 返回，并不等于数据已经越过设备缓存落到介质上。fsync 要把脏页、元数据和日志事务推到设备承诺的持久化边界（flush/FUA）。不同缓存层解决的问题不同：

缓存层	它让什么变快	OS 必须额外记录什么
CPU cache	CPU 读写内存更快	DMA 前后的可见性、内存屏障、cache 同步
page cache	文件读写复用内存页，合并小 I/O	dirty、uptodate、writeback、偏移到页索引
块层队列	请求排序、合并、限流	request 状态、队列深度、超时
设备内部缓存	设备接收写入后更快返回	flush/FUA、持久化错误回报、断电边界

核心思想

以上五组硬件事实就是本章其余部分的全部前提。CPU 只能从当前现场继续执行；虚拟地址要翻译和检查权限；设备靠 MMIO、队列和 DMA 协作；多核共享内存要靠一致性和屏障；完成异步到达且不等于持久化。后面每一个内核对象，都是在补这些事实缺失的语义。

2.2 二、硬件事实怎样逼出内核对象

先从一个非常普通的动作开始。一个进程往日志文件追加一行：

```
int fd = open("app.log", O_APPEND | O_WRONLY);
write(fd, user_buf, len);
```

直觉上，这像是“把 user_buf 里的字节交给磁盘”。但硬件并不认识“进程”“文件”“字符串”“追加写”这些概念。硬件只认识当前指令、寄存器里的数、虚拟地址、页表权限、cache line、设备寄存器、DMA 地址、中断向量和 completion 队列。于是内核必须把这次写日志拆成一串可验证的状态转换。

先写一个错误但很自然的版本：

```
write(fd, user_buf, len):
    disk.write(user_buf, len)
    return len
```

这个版本把五个硬件边界都抹掉了。

第一，CPU 正在执行用户程序，不会凭空同时运行内核代码。用户态进入内核时，硬件只提供一个受控入口。入口必须保存返回地址、栈指针、状态寄存器和参数寄存器，否则内核处理完以后不知道回到哪条用户指令，也不知道用户传了什么参数。

第二，user_buf 不是物理内存地址，也不是可信内核指针。它只是当前进程地址空间里的一个虚拟地址。它可能跨页，可能后半段不可读，可能指向尚未调入的

`mmap` 文件页，也可能在另一个线程里被并发 `munmap`。内核不能因为起始地址看起来正常，就把 `len` 字节当成一整段可靠内存。

第三，`fd` 不是文件本身。它只是当前进程文件表里的小整数索引。内核要用它找到 open file description，也就是“这个打开实例”的状态：访问权限、当前文件偏移、append 标志、引用计数、底层 inode 或设备对象。没有这个对象，两个线程同时写同一个 `fd` 时，谁更新偏移、谁持有引用、close 以后对象何时释放，都说不清。

第四，磁盘不是函数。高速存储设备靠队列工作：CPU 在内存里放描述符，写 MMIO doorbell 通知设备；设备经 DMA 读写内存；完成后写 completion 并触发中断。设备返回不是沿 C 调用栈返回，而是稍后通过硬件事件回来。

第五，失败不是一种。用户地址错误、文件不可写、磁盘超时、设备 reset、只写入一部分、数据只到 page cache、数据到设备缓存但未持久化，这些都不同。内核对象必须记录“推进到哪一步、哪些字节已经进入哪个层次、哪个请求还在飞、谁在等它、失败证据在哪里”。

把这条路径先压成一张总表，也是本章的阅读索引：

硬件事实	如果只写成普通函数会怎样	被逼出的内核对象
CPU 只能从当前现场继续执行	进入内核后覆盖寄存器，返回用户态时找不到原位置	trap frame、task、内核栈
用户地址只是虚拟地址	用户传坏指针会让内核读错页、越权读，或跨页时只检查了一半	address space、VMA、PTE、用户拷贝边界
<code>fd</code> 只是一个小整数	另一个线程 close 后对象可能被释放，权限和文件偏移也无处保存	fd table、open file description、引用计数
设备只暴露寄存器和队列	把设备当函数调用会忙等、丢失完成、无法取消和超时	driver、request、descriptor ring
完成事件异步回来	中断只说“有事件”，不能说明哪个请求完成、结果是什么	completion、request id、wait queue
DMA 会让设备直接访问内存	CPU 和设备同时改 buffer，或设备写到已经释放的页	buffer ownership、DMA mapping、IOMMU 约束

读到任何 OS 名词时，都应问一句：它是在补哪条硬件事实缺失的语义？下面几节把每条事实展开，看它怎样逼出对应的内核对象。

2.3 三、CPU 入口逼出 task 和 trap frame

用户程序执行 `write(fd, user_buf, len)` 后，硬件进入内核配置好的入口，并保存返回用户态所需的一部分状态。内核入口代码随后把更多通用寄存器保存成一份结构化现场，叫 trap frame。

trap frame 至少回答四个问题：从哪里来（用户 RIP、特权级）、怎么回去（用户 RSP、返回状态位）、带了什么参数（系统调用号和参数寄存器）、失败在哪里（faulting RIP、fault address、error code）。

问题	trap frame 里的证据	后续用途
从哪里来	用户 RIP、用户代码段或特权级状态	返回用户态、信号处理、调试、错误报告
怎么回去	用户 RSP、返回状态位、中断相关状态	恢复用户栈和受控状态
带了什么参数	系统调用号和参数寄存器	派发到具体系统调用并验证参数

失败在哪里	faulting RIP、fault address、error code	缺页、非法指令、权限错误和重启逻辑
-------	---------------------------------------	-------------------

如果不保存现场，内核自己的 C 函数也需要寄存器传参、使用栈、调用其他函数，原来的用户状态就会被正常的内核执行覆盖。trap frame 把“用户现场”从 CPU 瞬时状态变成内存里的对象。

但 trap frame 还不够。系统调用期间可能发生定时器中断，当前线程可能睡眠，调度器可能选择另一个线程运行。此时需要一个更大的归属对象：task。task 是一个可被调度和恢复的执行流，它把一组长期状态绑在一起：

```
Task:
  saved CPU context
  kernel stack
  state: running, runnable, blocked, stopped
  address space
  file table
  signal state
  wait reason and wait queue link
  accounting and scheduling metadata
```

用一次系统调用睡眠来走完整路径。用户线程调用 `write`，进入内核，内核发现设备请求需要等待，把当前 task 标成 blocked，挂到 request 的 wait queue 上。调度器选择另一个 runnable task，保存旧 task 的内核执行上下文，恢复新 task 的上下文。等设备完成后，中断处理函数把旧 task 从 wait queue 移回 runqueue。未来某个时间片里，调度器恢复这个 task，它继续从系统调用内部的睡眠点之后执行，最终返回用户态。

时刻	状态变化	为什么需要 task
用户执行	CPU 当前寄存器属于用户线程	这些状态要归属到某个可调度实体
系统调用入口	trap frame 记录用户现场，内核栈开始工作	进入内核后仍要知道将来回哪个用户线程
提交 I/O	request 记录 buffer、长度、tag 和等待者	等待关系要能从 request 回到 task
睡眠	task 状态从 running 变成 blocked，离开 CPU	CPU 可以运行其他 task，原 task 不丢失
中断完成	completion 找到 request，再找到等待 task	硬件事件变成对具体 task 的唤醒
再次调度	task 被放回 runnable 并恢复内核上下文	系统调用从睡眠点继续，而不是从头再来
返回用户态	trap frame 恢复用户寄存器和返回位置	用户程序看到一个普通 <code>write</code> 返回值

这张表解释了为什么“进程/线程”不能只说成“一个程序”。程序是代码和数据；task 是当前执行流的可恢复状态。两个线程可以运行同一份代码、共享同一个地址空间和文件表，但有不同的寄存器现场、栈、等待原因和调度状态。OS 需要的是后者，因为 CPU 只能在具体现场之间切换。

再看内核栈。用户态有用户栈，为什么进入内核后不能继续用用户栈？因为用户栈位于用户地址空间，权限和映射都由用户进程影响。用户可以把栈指针设到非法地址，也可以让栈页在进入内核后触发缺页，还可以把栈内容设计成让内核覆盖关键数据。所以每个 task 需要受内核保护的内核栈。

如果继续用用户栈	可能发生什么	内核栈解决什么
用户 RSP 指向非法页	入口保存现场时再次 fault, 甚至无法可靠报告错误	入口有可信保存位置
用户 RSP 指向共享映射	另一个线程或进程可修改内核临时数据	内核调用链不受用户写入影响
用户栈空间不足	深层文件系统或驱动调用覆盖用户数据或触发嵌套错误	每个 task 有受控大小和保护页
系统调用中睡眠	用户可能修改栈上“内核状态”	睡眠中的内核帧保存在内核地址空间

入口类型不同, trap frame 之外还要找的对象也不同。系统调用要找当前 task、fd table、address space; page fault 要找 fault address、VMA、PTE; timer interrupt 要找当前 task 的运行时间和 runqueue; device interrupt 要找 driver、device queue、request tag、wait queue。

device interrupt 的特点是“完成事实在设备侧”。设备中断到来时, 当前 CPU 上运行的 task 未必是当初提交 I/O 的 task, 甚至提交 I/O 的 task 可能在另一个 CPU 上睡眠。中断处理函数不能靠当前 task 来猜结果, 必须读取设备 completion, 按 tag 找 request, 再通过 request 找等待者。这一层关系非常重要: 中断入口保存的是“被打断的当前现场”, completion 保存的是“设备完成了哪次请求”。二者不是同一个对象。

核心思想

task 是 CPU 硬件事实的内核化: CPU 只能执行一个当前现场, OS 就把每个可恢复现场做成 task; CPU 入口只能给一次瞬时 trap, OS 就把入口证据保存成 trap frame; CPU 会被中断打断, OS 就把等待原因和调度状态也放进 task。

2.4 四、地址翻译逼出 address space、VMA 和 PTE

现在看 `user_buf`。用户传给内核的是一个虚拟地址。它不直接说明真实内存条上的位置, 也不说明当前是否允许读写。MMU 查页表把虚拟页翻译成物理页, 并检查权限。

PTE 只回答“当前硬件能不能直接完成这次访问”。它不回答“如果不能完成, 内核该不该补救”。这个补救策略来自 VMA。VMA 是 address space 里的一个合法区间说明: 这段虚拟地址从哪里来, 允许读写执行吗, 背后是匿名内存、文件映射、栈, 还是设备映射。

层次	它保存什么	谁使用它
address space	一个进程看到的虚拟地址世界, 包含页表根和 VMA 集合	task 切换、缺页处理、用户地址检查
VMA	一段虚拟区间的策略: 范围、权限、来源、增长规则	内核缺页处理和 <code>mmap/munmap</code>
PTE	硬件当前可执行的映射: present、物理页、权限位	MMU page walk 和 TLB 填充
TLB	最近 PTE 翻译结果的硬件缓存	CPU 快速完成 load/store

用前面跨页的 `user_buf` 走一遍:

```
page A: PTE present, user readable
page B: PTE not present, VMA says file mapping and readable
page C: no VMA covers this address
```

拷贝页 A 时，硬件翻译通过，CPU 从用户页读出字节。拷贝页 B 时，硬件发现 PTE 不 present，进入 page fault。注意，PTE 不 present 不等于一定非法。内核要查 VMA：若 VMA 说明这是一段合法文件映射，内核可以分配物理页、从文件读入数据、安装 PTE，然后重试原来的用户读取。拷贝页 C 时，没有任何 VMA 覆盖这个地址，内核就不能补页，因为这不是“暂时没准备好”，而是“这段地址不属于进程”。

访问结果	硬件看到什么	内核怎样解释
页 A 成功	PTE present 且用户可读，TLB 或 page walk 给出物理页	直接复制这一页内剩余字节
页 B 缺页	PTE not present，CPU 记录 fault address 和错误码	查 VMA，合法则补页后重试
页 C 非法	没有 VMA，或者 VMA 不允许读	停止拷贝，返回错误或短写语义
只读页写入	PTE present 但 writable 位不允许	可能是 COW，也可能是非法写
NX 执行失败	PTE 不允许执行	安全隔离，不应改成可执行后放行

这就是 address space、VMA、PTE 三层同时存在的原因。只有 PTE，不知道 not-present 是懒分配、文件页、换出页，还是非法地址。只有 VMA，硬件无法每条 load/store 都进内核询问策略。只有 TLB，修改页表后旧翻译还可能继续被 CPU 使用。OS 必须维护这三层的一致关系。

把几种 fault 分开看，会更清楚。page fault 不是简单的“程序访问了非法内存”。更准确的说法是：CPU 按当前页表和权限位无法完成这次内存访问，于是把证据交给内核。内核再根据 VMA 和对象状态判断这是正常按需工作，还是应该终止进程。

情况	硬件报告	内核解释
匿名内存第一次写	PTE not present，地址落在可写匿名 VMA 内	分配一页清零物理页，安装 PTE，然后重试
文件映射第一次读	PTE not present，地址落在可读 file-backed VMA 内	从文件对应位置读页，填入 page cache，再重试
换出页被访问	PTE not present，但软件记录说明页在 swap 中	从 swap 读回，恢复 PTE，再重试
COW 写入	PTE present 但不可写，VMA 允许写且页被共享	复制物理页，给当前进程安装私有可写 PTE
真正越权写	PTE 或 VMA 不允许写	返回 <code>EFAULT</code> 或向用户进程发送信号
NX 取指	PTE 不允许执行，CPU 正在取指	阻止把数据页当代码执行，通常发送信号

COW 能说明 PTE 权限位不只是安全检查，也是一种延迟复制机制。父进程 `fork` 后，父子进程先共享同一批物理页，PTE 都标成只读。某一方写入时，CPU 因为 writable 位不允许而 fault。内核查 VMA 后发现这不是非法写，而是 COW 写，于是分配新页、复制旧内容、给当前进程安装可写 PTE。若只看硬件错误码，会以为这是“写只读页”；只有结合 VMA 和页引用计数，内核才知道这是“该复制了”。

TLB 会让问题再多一层。CPU 为了不每次访存都走完整页表，会把最近的翻译缓存到 TLB。内核修改 PTE 后，如果旧翻译还留在某个 CPU 的 TLB 中，那个 CPU 可能继续按旧权限访问旧物理页。

PTE 变化	如果 TLB 没同步	后果
--------	------------	----

present 变	CPU 继续用旧翻译访问物理页	访问已释放页，破坏新对象
not present		
writable 变只	CPU 继续写旧可写映射	COW 或写保护失效
读		
user 变	用户态继续访问旧用户映射	权限隔离失效
kernel-only		
物理页号改变	CPU 继续访问旧物理页	读写错对象，数据难以追踪

用户拷贝和普通用户访存还有一个差别：fault 发生在内核代码执行期间。内核执行 `copy_from_user` 时，当前特权级在内核态，但被访问地址属于用户地址空间。若这个访问 fault，不能简单按“内核 bug”崩掉，也不能像用户态 fault 那样随便重启任意指令。内核必须知道这段代码是一个允许 fault 的用户拷贝窗口。

假设内核为了省事，在系统调用入口只检查 `user_buf` 起点：

```
bad_copy_from_user(user_buf, len):
    if address_is_valid(user_buf):
        memcpy(kernel_buf, user_buf, len)
```

这段代码在跨页时会错。起点所在页合法，不代表后续每一页合法。更严重的是，页表可能在拷贝过程中发生变化：另一个线程可以 `munmap` 某段地址，文件映射页可能第一次访问才调入，栈页可能按需增长。可靠的用户拷贝必须以“可能 fault、可能部分成功、必须能恢复”为前提设计。

```
copy_from_user_like(dst, user_addr, len):
    copied = 0
    while copied < len:
        check current user page
        if page fault can be resolved:
            resolve and retry
        if access is illegal:
            stop and report copied/error
        copy bytes until page boundary
        copied += bytes_this_page
```

这里的循环不是实现细节，而是语义边界。它告诉后面的系统调用章节：用户指针永远不能先验可信；告诉内存章节：缺页是硬件事件和 VMA 策略的交汇；告诉文件章节：短写和错误返回来自“请求推进到一半时遇到不可继续边界”。

地址空间还会和 DMA 发生关系。设备 DMA 使用的是设备可见地址，不一定等于 CPU 物理地址，更不等于用户虚拟地址。直接 I/O 中，用户传入的 `user_buf` 必须先被分解成一批用户虚拟页；内核检查 VMA 和 PTE，确保这些页允许这次方向的 I/O；再把对应物理页 pin 住；最后通过 IOMMU 建立设备可见映射。

地址名字	谁使用	不能混淆的原因
用户虚拟地址	用户程序和内核用户拷贝代码	只在某个 address space 中有意义
内核虚拟地址	内核代码	可能映射同一物理页，也可能没有直接映射用户页
物理页号	CPU 页表和内核页管理器	设备未必直接用这个编号访问
DMA 地址	设备和 IOMMU	生命周期受 DMA mapping 控制

如果一个进程提交直接 I/O 后马上 `munmap(user_buf)`，用户虚拟地址可以从 address space 中消失，但被 pin 的物理页不能立刻释放，因为设备 request 仍然引用它们。等 completion 到达、驱动同步 DMA、撤销 IOMMU 映射并放掉页引用

后，这些页才真正回到普通内存管理。地址空间解决 CPU 访问的合法性，request 和 DMA mapping 解决设备访问期间的所有权。

2.5 五、设备请求逼出 request、wait queue 和 completion

假设内核已经把用户数据安全地接收到内核对象里，下一步要让设备写出去。错误版本会写成：

```
device.write(kernel_buf, len);
```

真实设备通常暴露三类东西：MMIO 寄存器、内存中的队列、完成事件。先看最小串口模型：

```
STATUS bit0: tx_ready
DATA:       write one byte to start sending
CTRL bit0:  enable transmitter
```

朴素轮询版是 CPU 不断读状态寄存器直到 ready，再写数据寄存器。这个模型能说明 MMIO，但不适合通用 OS 的慢设备路径。若设备 10 ms 后才 ready，CPU 这 10 ms 都在原地读状态寄存器，不能运行别的 task。于是内核需要把“现在不能继续，但将来条件满足后要回来”表示成等待关系。等待关系不能只存在栈上，因为当前 task 会离开 CPU；它必须挂到内核对象上。

wait queue 可以理解成“等同一个条件的 task 链表”。等待串口 ready 的 task 挂在串口对象的 wait queue 上；等待块请求完成的 task 挂在 request 的 wait queue 上；等待锁的 task 挂在锁或 futex 对应的队列上。睡眠不是消失，而是 task 状态从 running 变成 blocked，并记录它等的条件。

高吞吐设备会进一步使用 request。request 是一次异步设备工作的内核身份牌。它至少保存：

```
Request:
  id or tag
  target device queue
  operation: read or write
  buffer list and length
  DMA mapping
  current state: new, submitted, completed, failed, canceled
  error code
  waiter or completion callback
```

用 NVMe 写请求走一遍：

阶段	发生什么	request 保存什么
创建	文件系统或块层把脏页转成块 I/O 请求	目标 LBA、长度、方向、关联页
映射	驱动通过 DMA API 建立设备可见地址	DMA 地址、方向、长度和待撤销映射
填描述符	CPU 在 submission queue 写 opcode、DMA 地址、tag	tag 把设备完成对回 request
敲门铃	CPU 写 MMIO doorbell，通知设备队列 tail 前进	request 状态变成 submitted
睡眠或异步返回	同步调用把 task 挂到 wait queue；异步路径记录回调	谁在等、完成后通知谁
设备 DMA 写	设备读取描述符并搬运数据	buffer 不能释放或复用
completion	设备把 tag、status、phase 写到 completion queue	request 结果等待驱动收割

中断处理	驱动读取 completion, 按 tag 找 状态变成 completed 或 到 request failed
唤醒与回收	唤醒等待 task, 撤销 DMA mapping, 生命周期结束条件清楚 释放引用

这里最容易混的是中断和 completion。中断只是 CPU 入口, 意思是“有设备事件需要处理”。completion 才是设备写下的结果, 告诉驱动哪个 tag 完成、状态码是什么。若只根据中断返回用户态, 内核不知道完成的是哪次 request; 若只看 completion 不处理等待者, 任务会永远睡着。

再看 DMA。DMA 不是“更快的 memcpy”, 而是设备获得了直接访问内存的能力。于是 buffer 所有权必须明确:

阶段	谁拥有 buffer	规则
提交前	CPU/内核拥有	可以填数据、建描述符、 建立 IOMMU 映射
提交后完成前	设备拥有或共享拥有	CPU 不能释放、复用或 无序改写; 必须遵守 cache/DMA 同步规则
completion 后	结果回到驱动, 但还没完全释放	先读取状态、同步 cache、 撤销 DMA mapping
回收后	CPU/内核重新拥有	页或 buffer 才能回到 分配器或上层对象

如果驱动在 completion 前释放 buffer, 设备迟到 DMA 可能写坏新对象。如果驱动没有建立正确 IOMMU 映射, 设备可能无法访问 buffer, 或者越界访问不该碰的页。如果驱动没有在 descriptor 写完后敲 doorbell, 设备可能读到旧描述符。request 对象把这些边界放到一处: 提交前准备, 提交后禁止释放, 完成后按顺序回收。

失败路径更能说明 request 的必要性。设备可能超时。超时不代表设备一定没做; 它只代表“在规定时间内没有观察到 completion”。此时内核不能马上释放 buffer, 因为设备可能稍后还会 DMA。可靠恢复通常要停止队列、屏蔽中断或 quiesce 设备、尝试 abort request、必要时 reset 控制器、确认没有 in-flight DMA 后再撤销映射。没有 request 列表, 内核不知道哪些请求还在飞; 没有 request state, 内核不知道哪些能重试、哪些已经失败、哪些不能再释放。

为了避免把设备统一想成“磁盘”, 再看三个设备路径。它们的规模不同, 但都在逼出同一组 OS 对象。

设备路径	典型动作	逼出的对象
PIO 串口发送一个字节	CPU 读状态寄存器, ready 后写 DATA 寄存器	wait queue、设备锁、寄存 器访问顺序
网卡接收一个包	设备 DMA 到预先给出的 buffer, 写 completion 或 更新 ring	buffer pool、descriptor ring、packet object、 NAPI/poll 状态
NVMe 写多个块	CPU 填 submission queue, 设备 DMA 用户或 page cache 页, 写 completion queue	request tag、DMA mapping、queue depth、 timeout/reset 状态

内存屏障也来自这个边界。CPU 和编译器可能为了性能重排普通内存写; 设备则按它观察到的内存和 MMIO 顺序行动。驱动填 descriptor 时, 必须保证 descriptor 内容对设备可见之后, 才写 doorbell。否则设备看到 doorbell 后去读 descriptor, 可能读到半旧半新的内容。抽象成代码就是:


```
fill descriptor fields
make descriptor writes visible to device
write MMIO doorbell
```

中间那句在不同架构上可能是内存屏障、DMA 同步或有序 MMIO 规则。它不是“玄学优化”，而是在告诉硬件：先看到队列内容，再看到通知。没有这层顺序，request 对象里保存的状态和设备实际执行的命令可能对不上。

再看丢中断。设备 completion 已经写到内存，但中断因为屏蔽、合并或硬件问题没有及时送到 CPU。若驱动只依赖“中断来了才检查 completion”，request 可能一直挂起。许多高性能驱动会把中断和轮询结合：中断负责提示“可能有活”，poll 循环负责批量收割 completion；超时定时器负责在长时间没有完成时检查 in-flight request。这里 wait queue 等的是 request 状态，而不是“某一次中断一定发生”。

设备 reset 是最复杂的失败路径之一。reset 不是把结构体清零这么简单。reset 前可能已有 request submitted，设备可能已经 DMA 了一半，completion 可能在 reset 过程中丢失，队列内存可能还留着旧 phase 位。可靠恢复至少要回答：

问题	必须有的状态
哪些请求已经提交给设备	in-flight request list 和 queue generation
哪些 buffer 仍可能被设备访问	DMA mapping、pinned pages、设备 quiesce 结果
哪些请求可以重试	request 操作类型、幂等性、文件系统上层语义
哪些请求必须失败上报	error code、等待 task、回调对象
旧 completion 怎样区分	tag、phase、queue generation 或 reset epoch

这张表说明“复杂硬件”不是因为名词多，而是因为异步边界多。CPU 可以继续运行，设备可以稍后 DMA，中断可能合并，completion 可能乱序，reset 可能跨过半完成状态。OS 要把这些边界收束成对象生命周期，否则用户看到的 read/write/fsync 就没有可靠语义。

2.6 六、映射总结与检查表

现在把前几节收束成一个最小内核模型。这个模型不是 Linux 完整结构体图，而是后文所有章节反复使用的检查框架：每个内核对象都必须回答“保存什么事实、谁能引用它、谁在等待它、它何时提交、它何时释放”。

```
Machine
  CPUs[]:
    current task
    trap/interrupt entry state
    current address-space root

  PhysicalPages[]:
    refcount
    dirty / locked / uptodate
    dma_pinned

  Devices[]:
    MMIO registers
    DMA queues
    interrupt vectors
    reset and error state
```



```

Kernel
Task:
    saved context
    kernel stack
    state and wait reason
    address space
    file table

AddressSpace:
    VMA list
    page table root
    active CPUs that may hold TLB entries

OpenFile:
    permissions
    file offset or append mode
    reference count
    backing inode, pipe, socket, or device

Request:
    id/tag
    buffer ownership
    DMA mapping
    status and error code
    wait queue or callback

```

用这套模型重新走一次 `write(fd, user_buf, len)`，每一步都能落到对象上：

动作	参与对象	不变量
进入内核	CPU、trap frame、task、内核栈	用户现场已保存，内核可安全使用自己的栈和寄存器
解析 fd	task、fd table、open file	file 引用有效，权限和偏移语义明确
验证地址	task、address space、VMA、PTE	用户范围逐页检查，fault 可修复才继续
接收数据	page cache 或 内核 buffer、physical page	已接收字节和未接收字节边界明确
提交 I/O	request、device queue、DMA mapping	descriptor、buffer、doorbell 顺序正确
睡眠等待	task、wait queue、request	task 等的是具体条件，不占 CPU 忙等
完成事件	device completion、interrupt entry、request	completion 能匹配 request，并记录成功或错误
返回用户态	task、trap frame、open file、request result	返回值只反映已经提交到相应语义边界的事实

再把这条路径按时间展开，把前面分散讲过的 CPU、MMU、cache、设备和 completion 放到同一条线里：

时间	硬件动作	内核对象变化	读者要抓住什么
T0	用户态 CPU 执行 <code>syscall</code> 指令	trap frame 开始形成，task 仍是当前 task	入口先保存现场，不先信参数

T1	CPU 切到内核入口和内核栈	task 的内核栈承载系统调用路径	内核不用用户栈保存状态
T2	内核按 fd table 查 open file	file 引用增加, 权限和 append 状态被检查	小整数变成有生命周期的对象
T3	CPU 读取用户虚拟地址	MMU 查 TLB/PTE, 可能触发 page fault	用户指针逐页变成可信字节
T4	缺页处理读取 VMA, 安装或拒绝 PTE	address space、VMA、page refcount 更新	not-present 要靠 VMA 解释
T5	CPU 把字节写入 page cache 或内核 buffer	page 变 dirty、locked 或 uptodate	write 可能只到内存层
T6	回写或 direct I/O 构造设备请求	request 分配 tag, buffer 建 DMA mapping	设备工作必须有身份牌
T7	CPU 写 descriptor, 再写 MMIO doorbell	request 进入 submitted 状态	顺序必须保证设备先看到描述符
T8	task 若需等待, 离开 CPU	task 挂到 wait queue, 调度器运行别人	等待不是忙等, 也不是丢失调用栈
T9	设备按 DMA 地址访问内存并执行命令	buffer 仍被 request 和 DMA mapping 持有	completion 前不能释放 buffer
T10	设备写 completion, 触发中断或等待轮询	driver 按 tag 找 request, 记录 status	中断不是结果, completion 才是结果证据
T11	内核唤醒 task, 撤销 DMA mapping, 释放引用	task 回到 runnable, request 走向 release	完成要伴随回收和错误传播
T12	CPU 恢复 trap frame 返回用户态	返回值放入用户可见寄存器	用户看到的是接口语义, 不是全部硬件细节

这条时间线也能解释为什么 OS 教材里同一个词会在不同章节反复出现。调度看到的是 T8 到 T12: task 为什么能睡下去又醒来。虚拟内存看到的是 T3 到 T4: 地址为什么要按页检查。文件系统看到的是 T5 和持久化边界: 写入何时对文件可见、何时对崩溃恢复可见。驱动看到的是 T6 到 T11: request 怎样提交, completion 怎样匹配, DMA 怎样收尾。

再把“提交边界”单独拎出来。OS 里很多 bug 都来自把不同层的完成混成一层:

完成说法	真实含义	不能误解成
系统调用入口完成	CPU 已进入内核并保存现场	参数已经可信
用户拷贝完成	字节已经进入内核 buffer 或 page cache	数据已经写到设备
request submitted	描述符和 doorbell 已提交给设备路径	设备已经完成 I/O
completion 到达	设备报告某个 tag 有结果	文件系统元数据已经持久
write 返回	对应接口语义下已接收或已提交一定字节	fsync 语义已经满足

<code>fsync</code> 返回	文件系统和设备承诺的持久化边界已完成或返回错误	所有未来硬件故障都不可能丢数据
-----------------------	-------------------------	-----------------

所有这些对象还有一个共同生命周期。它们沿着 `create`、`reference`、`use`、`wait`、`complete`、`release` 往前走：

阶段	含义	例子
<code>create</code>	把一次硬件事实或 API 请求变成对象	建 task、建 VMA、分配 request
<code>reference</code>	防止对象在使用期间被释放	fd lookup 增加 file 引用, I/O pin 用户页
<code>use</code>	按对象保存的权限和状态推进工作	拷贝用户页、填 descriptor、修改 page cache
<code>wait</code>	条件暂时不满足时让 task 离开 CPU	等页读入、等锁、等 completion
<code>complete</code>	外部事实回来后记录结果并唤醒等待者	page fault resolved、DMA done、journal committed
<code>release</code>	最后一个引用消失后撤销映射和释放资源	put file、unpin page、free request

这个生命周期能防住很多直觉错误。fd 查找之后要给 file 增加引用，是因为另一个线程可能马上 `close(fd)`；直接 I/O 要 pin page，是因为用户可能马上 `mmap` 或退出；request 完成后不能只唤醒 task，还要撤销 DMA mapping，是因为设备地址空间也曾经持有访问权；VMA 删除后要处理 TLB，是因为 CPU 可能还握着旧翻译。OS 对象的引用计数、锁、等待队列和状态位，都是在给这些生命周期边界补证据。

最后给一份贯穿后文的映射表。这张表也是后面每一章的阅读索引：

硬件事实	内核对象	后文会展开的问题
CPU 被同步或异步入口打断	trap frame、task、kernel stack	系统调用、异常、中断、调度
虚拟地址要翻译和检查权限	address space、VMA、PTE、TLB state	缺页、COW、mmap、用户拷贝
物理页会被共享、锁住、写脏或 DMA 引用	page、page cache、pin/refcount	内存分配、回收、回写、直接 I/O
fd 只是进程表里的索引	fd table、open file、inode/socket/pipe	VFS、权限、偏移、并发 close
设备通过队列和寄存器协作	driver、queue、descriptor、doorbell	块层、网络、字符设备、驱动模型
完成异步到达且可能失败	request、completion、wait queue、timeout	睡眠唤醒、超时、取消、reset
持久化不是普通内存写	dirty page、journal transaction、flush request	文件系统一致性、fsync、崩溃恢复

再给一份阅读 OS 机制的检查表。以后遇到任何机制，都按这六问拆：

问题	要找的证据	例子
----	-------	----

状态在哪里	哪个对象保存长期事实，不在 CPU 瞬时寄存器里	task、VMA、request、inode
入口在哪里	硬件或 API 从哪个受控点进入内核	syscall、page fault、IRQ、timer
权限在哪里检查	谁把不可信参数变成可用对象	fd lookup、VMA check、LSM、IOMMU
等待在哪里表达	task 睡在哪个条件上，谁负责唤醒	wait queue、completion、futex queue
提交点在哪里	哪一步之后对上层语义可见	PTE install、file size update、doorbell、journal commit
失败怎样收敛	哪些对象回滚、释放、保留错误码或重试	errno、request error、page refcount、reset recovery

核心思想

硬件不给应用直接共享整台机器的安全办法。CPU、MMU、cache、设备和持久化介质只暴露低层状态；内核必须把这些状态组织成 task、address space、open file、request、wait queue、completion 和恢复证据。后面每一章都在细化这套转换。

第 3 章 操作系统地图与契约

3.1 从保存按钮建立 OS 地图

编辑器里点“保存”，表面上只是把一段字节写进名为 `data` 的文件，再告诉用户“保存成功”。这一句话看起来很小，但它实际包含三个承诺：用户给出的内容被系统接收了；旧文件不会因为半途失败被随便破坏；如果程序或机器随后出错，系统还能说明哪个版本可信。

如果机器上只有一个函数、一个设备、一个永远不会失败的写入动作，保存可以被想象成普通函数调用。但真实机器不是这样。设备可能暂时不能接收数据；用户传入的地址可能无效；文件名和 `fd` 只是引用，不是文件本体；写入可能先停在内存缓存里；断电后还要判断旧版本、新版本、临时文件和目录项哪个可信。操作系统之所以复杂，不是因为它喜欢发明名词，而是因为这几个承诺不能靠一行函数返回值兑现。

保存按钮可以作为一张 OS 地图。简单写法一旦说不清“谁在等、等什么、失败在哪里、成功凭什么”，就说明状态还藏在函数调用里，没有进入内核账本。能解释这些状态的对象，才是进程、内存、文件系统和驱动这些子系统真正要解决的问题。

核心思想

操作系统的核心工作，是把“不可信的用户请求”和“会延迟、会失败的硬件事件”翻译成可管理的对象、可恢复的状态转移和可检查的完成证据。

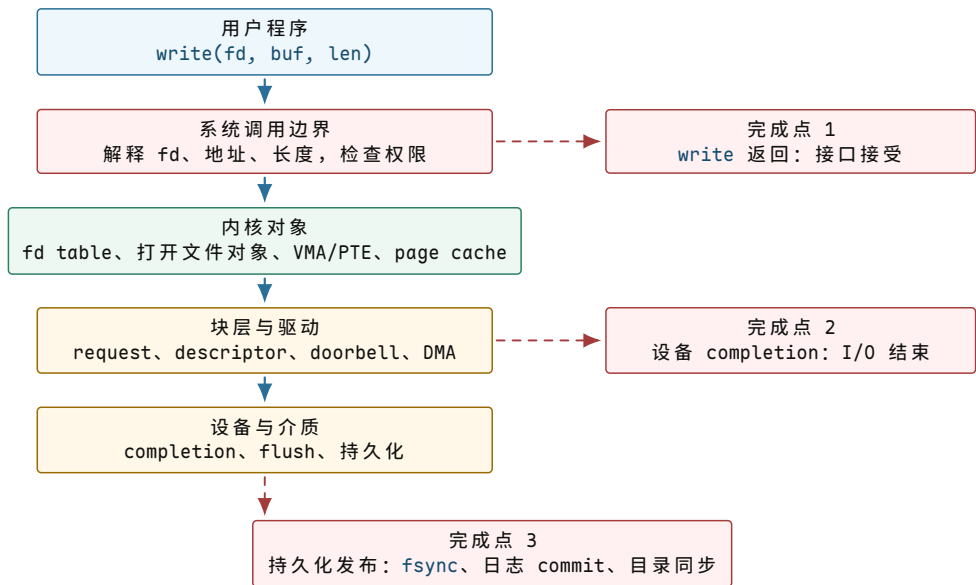
“契约”在这里不是法律比喻，而是可检查的技术承诺。用户调用 `write` 时，OS 要承诺它怎样解释参数、成功返回表示什么、失败返回表示什么；OS 提交设备请求时，硬件要用状态位、中断或完成队列告诉内核请求是否结束；文件系统宣布保存完成时，还要有崩溃恢复能识别的提交证据。只要其中一层说不清楚，“保存成功”就只是一句乐观打印。

一句“保存成功”	必须进一步证明什么	如果不证明会怎样
用户传入了内容	这段地址属于当前进程，且内核有权读取	可能读到非法地址，甚至泄漏内核数据
系统接收了请求	当前执行流、文件引用和目录对象都被记录下来	设备慢时只能忙等，或完成后找不到发起者
写入推进了	数据进入了哪个对象：内存缓存、设备队列还是持久介质	<code>write</code> 返回后掉电，用户以为保存了，其实磁盘仍是旧内容
失败能报告	错误发生在参数、权限、地址、设备、持久化还是目录发布	只能返回模糊失败，无法修复或恢复
完成能证明	哪个提交点之后，崩溃恢复可以相信这个结果	重启后无法区分成功版本和半成品

这些名字不是孤立术语。`task` 记录当前执行流，VMA/PTE 解释用户地址，`fd` 和打开文件对象解释整数句柄，page cache 解释写入为何能先停在内存，DMA/completion 解释设备异步完成，`fsync` 解释怎样把内存状态推进到持久化边界。每个名字都对应保存路径上的一个缺口。

判断一个 OS 机制是否讲清楚，要看四类可观察证据：用户程序写了哪行代码；进入内核后新增了哪份状态账本；硬件用什么事件、寄存器或权限检查支撑这份账本；失败时外部能看到什么结果。比如 `write(fd, buf, 4)` 这一行，在用户层只是一个函数调用，在内核层要解释 `fd`、buffer 和长度，在硬件层可能触发 page fault、DMA 和中断，在证据层则只返回字节数或错误码。

先把这张地图画出来。本章后面的每一步修正，都会回到这张图上的某一层或某一个完成点。



图示：保存请求自上而下穿过五层。三个“完成”在不同层生效：接口接受不等于设备完成，设备完成不等于崩溃后可恢复。掉电时能相信什么，取决于数据推进到了哪一层。

把环境压到最小：系统中只有一个用户程序，只有一个用来写数据的设备，暂不考虑掉电，也没有别的任务抢 CPU。在这个受限场景里，保存过程可能写成这样：

```

void save_request(const char* user_buf, size_t len) {
    while (!device_ready()) {
        // busy wait
    }

    for (size_t i = 0; i < len; ++i) {
        device_write_byte(user_buf[i]);
    }

    print("saved\n");
}
  
```

这段代码的问题不是“还不够工程化”，而是它没有交代任何 OS 坐标：CPU 等待设备时谁拥有执行权，设备完成时谁负责唤醒，`user_buf` 是否可信，`fd` 到底指向什么，`print("saved")` 凭什么代表保存成功。这些问题沿保存路径依次出现：CPU 先被无效占用，随后需要可靠睡眠和唤醒，接着要解释用户 buffer、fd 和文件对象，最后还要说明“保存成功”究竟越过了哪个边界。

先看等待设备的这一小段：

```

while (!device_ready()) {
    // CPU keeps spinning here.
}
  
```

它反复读取同一个事实：设备现在能不能接收下一个字节。这个会影响下一步行为、必须被记住的事实，就是状态。

核心思想

状态就是“会影响下一步行为的、必须被记住的事实”。在这里，`dev.ready = false` 表示设备还不能接收数据；`dev.ready == true` 表示保存路径可以继续往下走。

忙等的问题不是代码不好看，而是 CPU 一直在问同一个问题。假设设备 5 毫秒后才准备好，这 5 毫秒里 CPU 可以做很多别的事：运行另一个程序、处理网络包、响应键盘输入、刷新屏幕。但上面的 `while` 把 CPU 锁在原地，使设备延迟直接变成处理器浪费。

`device_ready()` 读取的是设备暴露给 CPU 的状态值。硬件设备常常会给 CPU 留出一些可读写的小格子，CPU 通过这些小格子知道设备是否空闲、是否出错、能不能接收下一个请求。这些小格子就是设备寄存器；在保存案例中，状态寄存器是 CPU 和设备之间交换状态的入口。

时刻	设备状态	CPU 正在做什么	问题
0 ms	not ready	读一次状态寄存器	发现不能写
1 ms	not ready	再读一次状态寄存器	没有任何新进展
2 ms	not ready	继续读状态寄存器	其他程序得不到 CPU
5 ms	ready	终于跳出循环	前面几毫秒被浪费

因此，等待设备不能写成原地自旋。设备没准备好时，当前执行流应当先停下，把 CPU 让出来；设备准备好之后，内核再把它恢复到可运行状态。

要让执行流可靠地停下并恢复，需要引入 `task`。`task` 是内核能暂停、恢复、选择运行的一条执行流。它不是用户写的一个普通函数；它必须有一份账本，记录自己运行到哪里、现在能不能继续、如果不能继续是在等什么。

这里第一次出现“对象”。对象不是面向对象编程里的类名，也不是把名词包装得更正式。对象是内核为了保存某类状态而建立的账本。普通函数的局部变量只在当前调用栈里有效；一旦函数要睡眠、被中断、被调度走，或者等待另一个硬件事件回来，状态就不能只放在局部变量里。它必须放进一个能被调度器、中断处理路径、超时路径和关闭路径共同找到的地方。

普通函数视角	OS 对象视角	为什么必须升级
局部变量 <code>i</code>	当前复制进度、请求偏移、已完成字节数	函数可能睡眠，醒来后要知道从哪里继续
一次函数调用	<code>task</code> 和保存的执行现场	当前执行流可能被切走，稍后从同一位置恢复
一个布尔判断	谓词和受保护状态	条件可能被中断路径或另一个 CPU 修改
等待一个结果	<code>wait queue</code> 和 <code>request/completion</code>	设备完成时要知道该唤醒谁、哪个请求完成

```
if (!device_ready()) {
    current->state = BLOCKED;
    wait_queue_push(&dev.waiters, current);
    schedule();
}
```

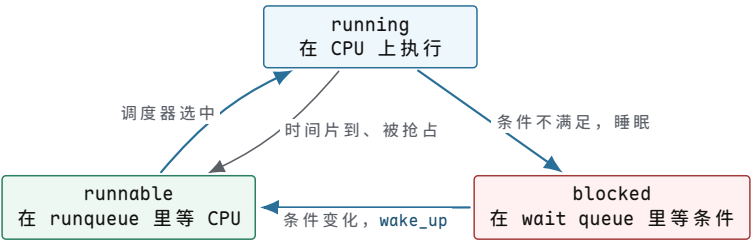
这段伪代码引入了三个内核账本，分别解决“能不能运行”“等在哪里”“谁来选择下一个执行流”三个问题。

词	在本例中的含义	反例
<code>BLOCKED</code>	当前 <code>task</code> 不能继续运行，因为它在等设备 <code>ready</code>	如果还把它放在可运行队列里，调度器可能又选中它，结果还是忙等
<code>wait queue</code>	等同一个条件的一组 <code>task</code> ，例如都在等 <code>dev.ready</code>	如果没有队列，设备完成时不知道该叫醒谁

<code>schedule()</code>	调度器入口：当前 task 让出 CPU，内核选择另一个能运行的 task	如果没有调度器，睡眠只会变成整台机器停住
-------------------------	---------------------------------------	----------------------

task 至少有三种基本状态：		
状态	含义	在保存例子中的位置
<code>running</code>	正在某个 CPU 上执行	保存函数正在跑
<code>runnable</code>	条件已经满足，只是暂时没轮到 CPU	<code>device_ready()</code> 或复制数据设备已经 ready，task 被唤醒，等调度器选中
<code>blocked</code>	现在不能推进，必须等某个条件变化	设备未 ready，task 睡在 <code>dev.waiters</code> 上

注意 `runnable` 不等于 `running`。一个 task 可以已经具备运行条件，但 CPU 正在跑别的 task；它只是排队等调度器。



图示：调度器只在 `running` 和 `runnable` 之间做选择；`blocked` 的 task 不在候选集合里，必须先由唤醒路径把它挪回 `runqueue`。

这三个教学状态在 Linux 里有真实名字。`task_struct` 的 `__state` 字段（`include/linux/sched.h`）记录的就是这张图：

真实状态名	对应教学状态	含义
<code>TASK_RUNNING</code>	<code>running</code> 或 <code>runnable</code>	在 CPU 上，或已在 <code>runqueue</code> 排队；两者共用一个状态名，靠 <code>on_rq</code> 字段区分
<code>TASK_INTERRUPTIBLE</code>	<code>blocked</code>	睡眠等待条件，可被信号打断
<code>TASK_UNINTERRUPTIBLE</code>	<code>blocked</code>	睡眠等待条件（多为 I/O），信号打不断
<code>__TASK_STOPPED</code>	<code>blocked</code> 的一种	被信号或调试器停住
<code>EXIT_ZOMBIE</code>	已退出待回收	不再是执行候选，但保留退出状态等父进程读取

这个状态不需要相信教科书，可以直接在真实机器上看到。`/proc/<pid>/status` 的 `State` 字段就是 `__state` 的用户态投影，下面两行是本书实验机上的真实输出：

```
$ grep '^State' /proc/self/status
State: R (running)
$ sleep 300 &
$ grep '^State' /proc/$!/status
State: S (sleeping)
```

`R` 表示 `TASK_RUNNING`——注意它同时覆盖“正在跑”和“在排队”两种情况，正好对应上面的状态机图；`S` 表示可中断睡眠，即 `TASK_INTERRUPTIBLE`。后面章节的等待队列、唤醒和调度讨论，都会落到这几个真实状态名上。

仅把 task 放入等待队列还不够。前面的写法比忙等前进了一步，但它仍然不正确，问题出在“检查条件”和“登记等待者”之间：

```
if (!device_ready()) {
    current->state = BLOCKED;
    wait_queue_push(&dev.waiters, current);
    schedule();
}
```

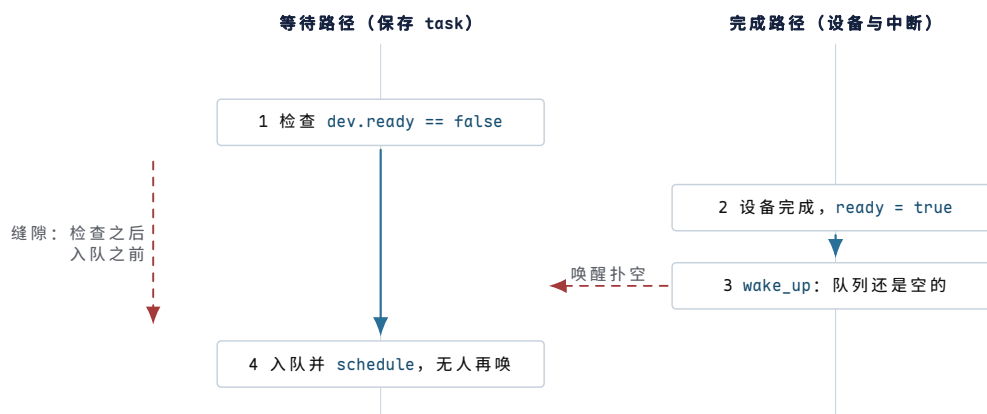
它把“检查条件”和“入等待队列”分成了几个互相可打断的动作。设备可能刚好在缝隙里完成：

顺序	保存 task 做了什么	设备完成路径做了什么
1	检查 <code>dev.ready == false</code>	还没发生
2	准备把自己睡下去	设备变成 ready
3	还没入队	中断处理调用 <code>wake_up(&dev.waiters)</code> ，但队列为空
4	task 入队并 <code>schedule()</code>	之后没有新的完成事件触发唤醒

这个错误叫 lost wakeup：唤醒确实发生过，但等待者还没登记，所以它错过了唤醒，之后可能永远睡下去。

这里的“唤醒”不是用户代码主动调用普通函数，而是设备完成后给 CPU 一个事件。这种设备主动打断 CPU 的事件就是中断。关键关系是：设备完成和用户函数继续执行不是同一条路径，中间必须由内核把“设备完成”翻译成“某个 task 可以继续运行”。

lost wakeup 最容易让初学者误解为“少调用了一次唤醒函数”。真正的问题不是函数少了，而是状态和等待者没有在同一个规则下更新。等待者判断 `dev.ready == false` 时，完成路径可能马上把它改成 true；完成路径调用 `wake_up` 时，等待者可能还没有进入队列。于是两个动作都发生了，却没有形成可靠关系。这就是为什么 OS 机制总要讲不变量：不是每一行代码看起来对就够了，还要保证任何时序交错下关系都成立。



图示：两条路径各自都“做了正确的事”，错误出在缝隙里：完成事件恰好落在等待者检查条件之后、登记入队之前。把检查和入队放进同一个受保护区，缝隙才被封死。

错误理解	正确理解	后果
wakeup 是一条消息	wakeup 只是提示条件可能变化	醒来后必须重新检查谓词

入队和检查可以分开写	检查谓词和登记等待者必须受同一规则保护	否则完成事件会从缝隙里丢失
睡眠只是暂停函数	睡眠会让出 CPU，并改变 task 的调度状态	blocked task 不能继续留在 runqueue 上抢 CPU
锁只是防止变量同时写	锁保护的是“条件与队列的一致关系”	没有锁会出现状态正确但等待关系错误

解决 lost wakeup，需要同时约束两个对象：一个是判断能否继续的条件，一个是保护这个条件和等待队列的互斥规则。

词	在本例中是什么意思	为什么必须要它
谓词	一个能回答“现在能不能继续”的条件，这里是 <code>dev.ready</code>	wakeup 不是消息本身，task 醒来后还要重新检查条件
锁	保护共享状态的一段互斥区间，这里保护 <code>dev.ready</code> 和 <code>dev.waiters</code>	不能让完成路径插在“检查条件”和“入队”之间

可靠的等待路径要把“检查条件”和“入等待队列”放在同一个受保护区间里：

```
lock(dev.lock);
while (!dev.ready) {
    prepare_to_wait(&dev.waiters, current);
    unlock(dev.lock);

    schedule();

    lock(dev.lock);
}
finish_wait(&dev.waiters, current);
unlock(dev.lock);
```

设备完成时，中断处理路径做相反的事：

```
irq_handler() {
    lock(dev.lock);
    dev.ready = true;
    complete_current_request();
    wake_up(&dev.waiters);
    unlock(dev.lock);
}
```

这段代码有三个关键点。第一，用 `while`，不用 `if`，因为醒来只表示“条件可能变了”，不保证条件已经满足。第二，`prepare_to_wait` 发生在锁保护下，避免 lost wakeup。第三，`schedule()` 前要释放锁，否则设备完成路径拿不到锁，无法把 `dev.ready` 改成 `true`。

由此可以看到 OS 的核心形状：正常路径不是一条直线，而是多条入口路径配合维护同一个谓词。

路径	它做什么	不能破坏的关系
等待路径	检查 <code>dev.ready</code> ，不满足就把当前 task 放入 wait queue	检查条件和入队不能被完成路径插到中间
完成路径	设备中断到来，更新请求状态，再唤醒等待者	必须先写完成状态，再唤醒；否则等待者醒来仍看不到结果
调度路径	<code>schedule</code> 选择另一个 runnable task 运行	blocked task 不能还留在 runqueue 里抢 CPU

返回路径

被唤醒后从 `schedule` 返回，重新检查条件`wakeup` 不是消息投递，只表示“条件可能变了”

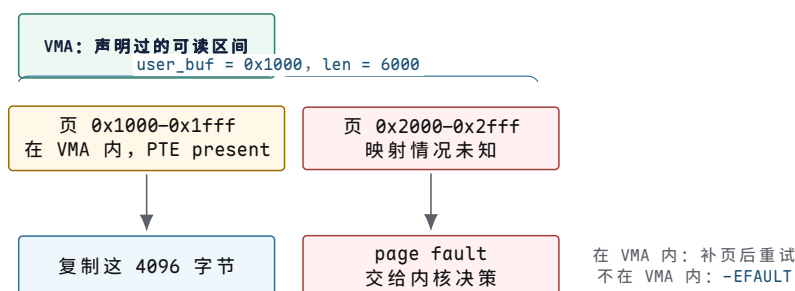
调度、锁、条件变量、`futex` 和 I/O completion 的共同骨架正是这组关系：先定义一个状态谓词，再定义谁能改它，最后用 `wait queue` 把“现在不能推进”的 `task` 挪开。

设备等待可靠之后，再处理 `user_buf`。在普通 C 函数里，`const char* user_buf` 看起来就是一个地址；但在 OS 里，它来自用户程序，内核不能直接相信。

具体地址如下：

```
char* buf = (char*)0x1000;
write(fd, buf, 6000);
```

假设一页是 4096 字节，那么这次写入跨过两页：`0x1000..0x1fff` 和 `0x2000..0x276f`。第一页可能合法，第二页可能根本没有映射。如果内核把它当普通指针直接读，轻则内核自己 `fault`，重则把不该读的内存泄漏出去。



图示：起点 `0x1000` 合法，只说明第一页可验证；`buffer` 是一个范围，检查必须按页推进到 `0x276f` 为止。第二页落入两种结局之一：合法但未装入（补页重试），或根本不属于任何 VMA（拒绝）。

这里先引入系统调用边界。`write` 不是普通函数调用到底层设备；它会通过受控入口进入内核。用户态只交出三个值：`fd`、`buffer` 地址、长度。内核必须把这些不可信值变成自己能负责的状态。

最容易写错的检查是只看起点：

```
if (is_user_pointer(user_buf)) {
    memcpy(kbuf, user_buf, len);
}
```

这段代码的问题和前面的忙等类似：它把一个连续过程压成了一个布尔判断。起点合法，只能说明第一个地址看起来属于用户空间；它不能证明后续 `len` 个字节都合法，也不能证明这些页现在都在内存里，更不能证明复制期间另一个线程不会改变映射。OS 需要的是按范围、按页、按权限推进，而不是对一个指针值投信任票。

天真判断	漏掉的问题	正确追问
指针不为空	非空地址仍可能没有映射	它是否落在当前进程允许的地址区间内
起点在用户空间	后续字节可能跨到非法页	整个范围是否逐页可读
页表现在有翻译	复制中可能发生缺页、换入或权限变化	<code>fault</code> 能否修复，修复后是否需要重试
地址可读	目标对象未必允许写入	<code>fd</code> 、打开文件对象和权限还要单独检查

核心思想

系统调用边界的第一件事不是“干活”，而是把用户传入的整数、指针和长度逐个解释清楚：这个 fd 是否存在，这段地址是否可读，这个长度是否会越界，失败时返回什么错误。

user_buf 在 OS 里要拆成三层判断：

- 1. 这个地址是否落在当前进程声明过的某个地址区间里。
- 2. 这个区间是否允许当前操作，比如写文件时内核要从用户 buffer 读。
- 3. 当前页表里是否已经有可用翻译；没有时，缺页处理能不能补上。

这三层判断落到四个概念上：

词	含义	保存例子里的判断
虚拟地址	用户程序看到的地址编号，不直接等于物理内存位置	0x1000 是用户给出的虚拟地址
VMA	virtual memory area，一段地址区间的“使用意图”	这段地址是不是用户栈、堆或 mmap 文件，是否允许读
PTE	page table entry，一页地址的“当前翻译事实”	这一页现在是否在内存里，是否允许用户态读
page fault	CPU 发现地址翻译或权限不满足，把问题交给内核解释	第二页没有 PTE 时，内核决定是补页还是报错

所以“读取用户 buffer”不是普通内存访问，而是一个可能失败、可能睡眠、可能只完成一部分的复制过程。

这里的“可能睡眠”很关键。若第二页是合法文件映射，但内容还不在内存，内核可能需要从磁盘把页读进来；读磁盘又会提交块 I/O，当前 task 可能睡在缺页等待或 I/O completion 上。也就是说，一个看似单纯的 copy_from_user，可能临时走到调度器和块设备路径。系统调用不是一条不被打断的直线，它会在地址、权限、等待和设备完成之间来回穿行。

```
size_t copied = 0;
while (copied < len) {
    int rc = copy_one_page_from_user(kbuf + copied,
                                     user_buf + copied);

    if (rc == PAGE_FAULT_FIXED) {
        copied += bytes_copied_this_round;
        continue;
    }
    if (rc == BAD_USER_ADDRESS) {
        break;
    }
}
```

两页复制过程如下：

步骤	内核检查到什么	下一步
第一页	地址落在 VMA 内，VMA 允许读，PTE present	复制第一页的数据
第二页	地址落在 VMA 内，但 PTE not-present	触发 page fault，能补页就重试
第二页另一种情况	找不到覆盖它的 VMA	停止复制，返回部分写入或 -EFAULT

内核地址 PTE 标着“只允许内核访问”，用户无权访问 拒绝，不能泄漏内核内存

用户传入的情况	内核看到什么	合理结果
buffer 全部合法且页已在内存	VMA 允许读，PTE present	复制成功，继续写入路径
buffer 合法但页未装入	VMA 允许读，PTE not-present	触发 page fault，分配或读入页面后重试
buffer 前半段合法，后半段非法	前几页复制成功，后一页找不到 VMA	返回已写字节数或 <code>-EFAULT</code> ，不能越界读
buffer 指向内核地址	用户态无权访问内核专用页	拒绝，不能把内核内存泄漏给用户
另一个线程同时修改 buffer	指针仍合法，但内容在变化	内核复制到的是某一时刻的快照，不能假设之后不变

这个边界可以直接观察。下面的程序故意把 buffer 指向 `0x1`。这个地址通常不属于进程的有效 VMA；用户态把它作为普通整数传给内核并不违法，但内核不能相信它真的可读。

```
int fd = open("data", O_WRONLY | O_CREAT | O_TRUNC, 0644);
const char* bad = (const char*)0x1;
ssize_t n = write(fd, bad, 16);
printf("n=%zd errno=%d\n", n, errno);
```

用系统调用跟踪工具看，重点不是记住具体 fd 号，而是看 `write` 怎样把坏地址变成用户可见错误：

```
$ strace -e trace=write ./badbuf
write(3, 0x1, 16) = -1 EFAULT (Bad address)
```

程序自己看到的返回值同样能证明这条受控路径。本书实验机上的真实输出：

```
$ ./badbuf
n=-1 errno=14 (Bad address)
```

这里没有出现“内核随便崩溃”，也没有出现“从地址 `0x1` 读出垃圾”。发生的是一条受控错误路径：CPU 在内核复制用户数据时发现地址访问不成立，进入 page fault；内核发现这次 fault 发生在 `copy_from_user` 这类允许恢复的位置，于是把它翻译成 `-EFAULT` 返回给系统调用。内核里通常会为这类可恢复访问准备异常表。异常表可以理解成一份小账本：如果某条特殊的内核访存指令 fault，不要按普通内核 bug 处理，而是跳到修复路径，设置错误结果并返回。

观察到的现象	背后的边界	说明
<code>0x1</code> 能作为参数传入	系统调用入口只接收整数值	入口接收参数不代表参数语义有效
<code>write</code> 返回 <code>EFAULT</code>	地址复制边界失败	错误发生在读取用户 buffer，而不是 fd 或磁盘
进程继续运行	fault 被内核修复路径接住	这类 fault 是可报告错误，不是必须 panic 的内核 bug
文件不应被当作完整写入	复制没有得到 16 个可信字节	没有可信输入，就不能推进保存语义

这里有两个边界需要同时成立。第一，系统调用边界会把不可信输入变成内核拥有的稳定状态；小数据通常复制，大 I/O 可能 pin page，也就是临时固定页面，保

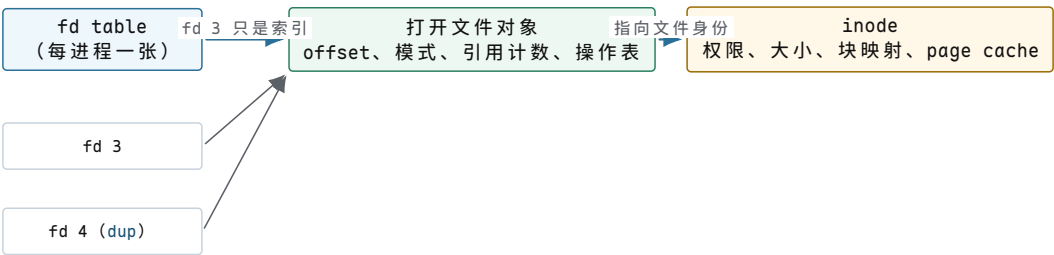
证设备 DMA 完成前页面不会被回收或移动。第二，page fault 不是单纯的“内存不够”，它是硬件把一个地址访问问题交给内核解释：能修就修，不能修就返回错误或发信号。

user_buf 解释完，还剩 fd。用户程序通常这样写：

```
int fd = open("data.tmp", O_WRONLY | O_CREAT | O_TRUNC, 0644);
write(fd, buf, len);
```

fd 看起来只是一个整数，例如 3。但它不是文件本体。它是当前进程 fd table 里的一个索引。fd table 可以先理解成“这个进程手里的资源号码表”。fd table 指向的 file object 可以理解为“打开文件对象”：它表示一次 open 或 dup 关系背后的内核对象，可对应 POSIX 语义里的 open file description。

层次	在本例中的含义	它回答的问题
fd 整数	用户态拿到的号码，例如 3	用户后续用哪个号码引用这次打开得到的对象
fd table	当前进程的号码表	3 号槽是否存在，指向哪个内核对象
打开文件对象 (file object)	一次打开文件形成的内核对象	当前 offset、读写模式、引用计数、具体操作函数
inode	文件系统里的文件身份和元数据	这个文件的权限、大小、块映射、page cache 归属



dup：两个 fd 槽共享同一个打开文件对象和 offset

图示：整数 fd 只是进程 fd table 的索引；offset 和引用计数属于打开文件对象；inode 才是文件身份。dup 复制的是槽位，不是对象——这是后面所有共享 offset 现象的根源。

为什么要分这么多层？一个 dup 例子能直接暴露 fd 和文件本体之间的差别：

```
int a = open("data", O_WRONLY);
int b = dup(a);
write(a, "A", 1);
write(b, "B", 1);
```

dup(a) 会创建另一个 fd，但它们通常共享同一个打开文件对象，也就是共享这个对象里的 offset。因此第一次写入把 offset 从 0 推到 1，第二次写入接着从 1 写，文件内容会是 AB。如果把 fd 误认为“文件本体”，就解释不了为什么两个整数会共享 offset，也解释不了 close(a) 后 b 还能继续写。

这个共享 offset 也可以用一个小程序逼出来：

```
int a = open("data", O_WRONLY | O_CREAT | O_TRUNC, 0644);
int b = dup(a);

write(a, "A", 1);           // offset: 0 -> 1
write(b, "B", 1);           // offset: 1 -> 2
lseek(a, 0, SEEK_SET);      // shared offset: 2 -> 0
write(b, "C", 1);           // offset: 0 -> 1
```

如果 `a` 和 `b` 是两个互不相关的文件对象，`lseek(a, 0, SEEK_SET)` 不应该影响 `b`。但 `dup` 的结果会共享打开文件对象，所以 `b` 的下一次写入从 0 开始，覆盖第一个字节。最终文件内容不是 `ABC`，而是 `CB`。

```
$ ./dup-offset
$ od -An -c data
C B
```

动作	共享 offset	文件内容
<code>write(a,"A",1)</code>	0 变成 1	A
<code>write(b,"B",1)</code>	1 变成 2	AB
<code>lseek(a,0)</code>	2 变成 0	AB，只是位置变了
<code>write(b,"C",1)</code>	0 变成 1	CB，第一个字节被覆盖

这比口头说“fd 是引用”更有力。用户手里确实有两个整数，但内核账本里只有一个共享的打开文件对象；offset 属于这个对象，不属于整数变量本身。若代码需要两个互不影响的文件位置，就不能用 `dup` 得到第二个位置，而要重新 `open`，让内核创建另一个打开文件对象。

再看一个更容易在工程里踩到的例子：

```
int fd = open("data", O_WRONLY);
int copy = dup(fd);
close(fd);
write(copy, "X", 1);
```

`close(fd)` 关闭的是当前进程 fd table 里的一个槽位；只要 `copy` 还引用同一个打开文件对象，这个打开对象就不能释放。于是“文件是否还能写”不取决于整数 `fd` 这个变量是否还存在，而取决于打开文件对象的引用计数和权限状态。在 `fork`、`exec`、`socket` 和设备文件中也会反复遇到同一条规则：用户态整数只是引用入口，真正的生命周期在内核对象上。

现象	如果把 fd 当文件本体会怎样误解	用对象账本怎样解释
<code>dup</code> 后共享 offset	以为两个整数各写各的	两个 fd 槽指向同一个打开文件对象
<code>close(a)</code> 后 <code>b</code> 还能写	以为文件已经关闭	只减少一个引用，打开对象仍被 <code>b</code> 持有
<code>fork</code> 后子进程继承 fd	以为复制了整个文件	父子 fd table 槽可引用同一打开对象
<code>exec</code> 后部分 fd 消失	以为程序替换会关闭全部资源	<code>FD_CLOEXEC</code> 决定哪些引用在提交点被裁剪

这层关系在真实系统里可以直接看到。下面的程序打开 `data.tmp`，用 `dup` 复制一个 fd，再打开所在目录，然后睡眠一分钟；趁它睡眠时去读它的 fd 目录（输出删去了完整路径）：

```
$ ./fd_demo &
pid=26635 fd=3 copy=4 dirfd=5
$ ls -l /proc/26635/fd
3 -> ../data.tmp
4 -> ../data.tmp
```

```
5 -> .../
$ grep -E '^(State|Threads)' /proc/26635/status
State: S (sleeping)
Threads: 1
```

fd 3 和 fd 4 指向同一个文件，这正是 dup 共享打开文件对象的直接证据；fd 5 是一个目录——目录也能被打开并占有 fd 槽位，后面的 fsync(dirfd) 依赖的正是这个事实。睡眠中的进程 State 是 S，对应前面讲的 TASK_INTERRUPTIBLE。

这三层在 Linux 里的真实名字是：fd table 对应 files_struct 和 fdtable (include/linux/fdtable.h)；打开文件对象是 struct file(include/linux/fs.h)，里面装着 f_pos (当前 offset)、f_count (引用计数) 和 f_op (具体操作函数表)；inode 是 struct inode，page cache 挂在它的 i_mapping 上。后面的章节会逐个打开这些结构；本章先记住一件事：整数 fd、打开文件对象、inode 是三层不同的对象，offset 属于中间那层。

为了避免把 dup 的结论误记成“同一个路径就共享 offset”，再看重新 open 的反例：

```
int a = open("data", O_WRONLY | O_CREAT | O_TRUNC, 0644);
int b = open("data", O_WRONLY);

write(a, "A", 1);      // a offset: 0 -> 1
write(b, "B", 1);      // b offset: 0 -> 1
```

这次最终内容通常是 B，不是 AB。原因不是路径名变了，而是两次 open 创建了两个打开文件对象；它们都指向同一个 inode，但各自有独立 offset。第一次写入用 a 的 offset 0，把文件变成 A；第二次写入用 b 的 offset 0，又从文件开头覆盖成 B。

```
$ ./open-twice-offset
$ od -An -c data
B
```

写法	内核对象关系	offset 结果
dup(a)	两个 fd 槽指向同一个打开文件对象	a 写完后，b 接着往后写
重新 open	两个打开文件对象指向同一个 inode	a 的 offset 不影响 b 的 offset
fork 继承	父子进程的 fd 槽可指向同一个打开文件对象	子进程推进 offset，父进程随后也从新位置写

fork 的情况也可以直接观察。下面的程序让子进程先写，父进程等子进程退出后再写：

```
int fd = open("data", O_WRONLY | O_CREAT | O_TRUNC, 0644);
pid_t pid = fork();

if (pid == 0) {
    write(fd, "C", 1);
    _exit(0);
}

waitpid(pid, NULL, 0);
write(fd, "P", 1);
```

输出会显示父进程不是从 offset 0 覆盖子进程写入的字节，而是接在后面写：

```
$ ./fork-offset
$ od -An -c data
C P
```

`waitpid` 只是在这个例子里固定执行顺序，真正决定 CP 的是父子进程共享同一个打开文件对象。子进程把共享 `offset` 从 0 推到 1；父进程被唤醒后继续使用同一个对象，所以从 1 开始写。由此可以把 `fd` 生命周期说清楚：进程可以复制，`fd` 槽可以复制，路径名可以相同，但打开文件对象才是 `offset`、状态标志和引用计数的承载者。

有了 `fd` 和 `buffer` 两条账，`write(fd, buf, len)` 的路径可以写成：

```
write(fd, buf, len)
-> find fd in current process fd table
-> check the open file object allows write
-> copy data from user buffer
-> update file/page-cache state
-> return byte count or -errno
```

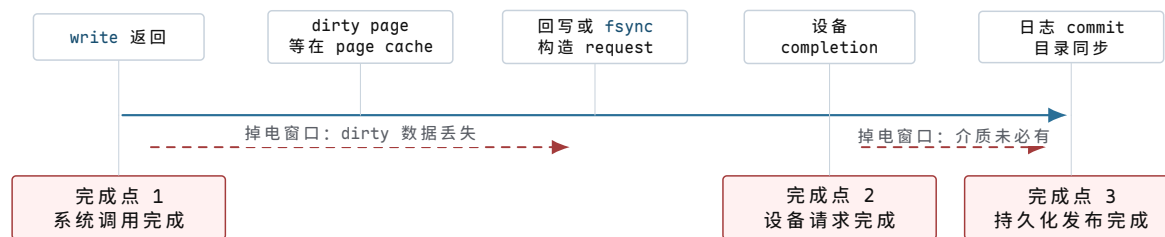
这一阶段会用到 `page cache`。它是内核用内存缓存文件内容的一组页面。写普通文件时，内核常常先把用户数据复制到 `page cache`，把相关页面标记为 `dirty`，然后 `write` 就可以返回。`dirty` 的意思是“内存里的文件页已经比磁盘上的版本更新，还需要写回”。

`page cache` 第一次出现时要特别小心：它不是“磁盘的另一个名字”，而是内核内存里的文件内容副本。把数据写进 `page cache`，通常能让后续读同一文件更快，也能把多个小写合并成更合适的块 I/O；代价是完成边界变多了。用户看到 `write` 返回时，数据可能只到了内核内存；后台写回或 `fsync` 之后，数据才会被组织成块请求交给设备；设备 `completion` 到来后，内核才知道那次设备请求结束；存储设备自己的缓存和文件系统日志还会继续影响崩溃后能不能相信这个结果。

边界	此时数据在哪里	还不能承诺什么
<code>copy_from_user</code> 结束	内核 <code>buffer</code> 或 <code>page cache</code> 中	目标文件名未必已经安全发布
<code>write</code> 返回 块请求提交	接口语义接受了若干字节 <code>request</code> 和 <code>DMA</code> 映射交给 驱动/设备	设备未必见过这批数据 <code>completion</code> 未到， <code>buffer</code> 不能释放
设备 <code>completion</code>	某次设备请求完成或失败	介质持久性仍可能依赖 <code>flush/FUA/日志</code>
<code>fsync</code> 成功	文件相关数据和必要元数据 推进到持久化边界	目录项更新仍可能需要目录 <code>fsync</code>

最后回到最初的 `print("saved")`。这行太早，因为保存请求至少有三个不同的完成点：

完成点	说明	反例
系统调用完成	<code>write</code> 返回，内核已经按接口语义接受了一部分数据	机器立刻掉电， <code>dirty page</code> 还没写到设备
设备请求完成	块层或驱动收到 <code>completion</code> ，某次 I/O 已结束	设备缓存还没 <code>flush</code> ，断电后介质上未必有数据
持久化发布完成	<code>fsync</code> 、日志 <code>commit</code> 、 <code>rename</code> 和目录同步形成可恢复状态	只同步文件内容，不同步目录项，崩溃后新名字可能丢失



图示：三个完成点在同一条真实时间线上，但彼此之间隔着可观的距离。两个红色区间是掉电会造成不同后果的窗口：第一段里数据只在内存，第二段里设备自己可能还攥着没 flush 的缓存。

这段距离不是修辞，可以直接实测。给 64KiB 的 `write` 和紧随其后的 `fsync` 分别计时，本书实验机（aarch64 容器、闪存介质，数字只用来说明量级）上的真实输出是：

```
write=83us fsync=3180us (wrote 65536 bytes)
write=43us fsync=195us  (wrote 65536 bytes)
write=26us fsync=140us  (wrote 65536 bytes)
```

`write` 几十微秒就返回，因为它只是把数据复制进 page cache 并标上 dirty；`fsync` 必须等设备真正确认，第一次甚至花了 3 毫秒多——完成点 1 和完成点 3 之间隔着一到两个数量级。中间那段距离不是空的，里面填满了 page cache、块层队列、驱动和设备固件各自的状态账本；本章后半部分的每一条路径，都是在解释这段距离里发生了什么。

按时间顺序展开后，三个完成点的差异更明显：

```
write(fd, buf, len)
-> copy from user
-> mark page cache dirty
-> return len

background writeback or fsync(fd)
-> build block request
-> map DMA buffer
-> device completion
-> record writeback error if any

rename("data.tmp", "data")
fsync(dirfd)
-> publish the name
-> make the directory update recoverable
```

三段边界各自承担不同含义。

`write` 返回，只说明内核按照 `write` 的接口语义接受了数据，或者接受了一部分数据。对普通文件来说，它经常停在 page cache 层：内存里有新内容，磁盘上未必有。

`fsync(fd)` 把语义往前推一层：内核要把这个文件相关的 dirty 数据和必要元数据推进到设备，并处理过程中发现的 I/O error。它比 `write` 强，但它通常只针对这个文件对象。

时间线里出现的 DMA 和 completion 是设备 I/O 的两个关键边界。DMA 是 direct memory access，意思是设备可以在内核授权后直接读写某段内存，不必让 CPU 一个字节一个字节搬。completion 是完成记录，表示某个提交给设备的请求结束了，内核可以根据它更新请求状态、释放 buffer、唤醒等待者。

`rename("data.tmp", "data")` 是发布新名字：让目录树里正式名字指向新文件。目录项本身也是元数据，所以很多崩溃安全写法还要打开目录并 `fsync(dirfd)`，让“名字更新”也跨过持久化边界。

如果崩溃发生在不同位置，恢复结果不同。

崩溃位置	可能看到的状态	为什么不能简单说“保存成功”
<code>write</code> 返回后、 <code>fsync</code> 前 <code>fsync(fd)</code> 中途	内存里有 <code>dirty page</code> ，磁盘 旧内容还在 部分块已写，错误可能稍后报 告	<code>write</code> 的契约不是持久化契约 I/O error 需要被记录并返回 给调用者
<code>rename</code> 后、目 录未 <code>fsync</code> 日志 <code>commit</code> 前	文件内容可能在，但名字更新 未必可恢复 日志里有准备信息，但不能当 成功事务	目录项本身也是元数据，也需 要提交边界 恢复时必须丢弃未 <code>commit</code> 的 更新
日志 <code>commit</code> 后	恢复程序能重放或完成更新	成功来自可验证记录，不来自 函数执行到最后

因此，“保存成功”不能只看一行返回值。它要说明数据进入了哪个对象、越过了哪个边界、失败时谁负责报告、崩溃后用什么证据恢复。`page cache`、块层、驱动、日志和观测工具，本质上都在追问这条时间线的不同边界。

前面的修正可以收束成“操作系统契约”。契约不是抽象口号，而是一组可检查的承诺：谁拥有状态，谁能改状态，不能继续时睡在哪里，失败怎么报告，外部用什么证据判断它真的完成。

继续沿保存请求走下去。最初的函数只有局部变量和设备寄存器；一旦要支持多个程序、多个请求和失败恢复，内核必须把隐含状态变成显式对象。

原来藏在哪里	内核对象	这个对象必须回答的问题
当前正在执行的 函数	<code>task</code>	它能否运行，睡在哪里，怎样从 同一位置恢复
用户传入的指针	<code>VMA</code> 、 <code>PTE</code> 、 <code>page</code>	这个地址是否合法，读写执行权 限是什么， <code>fault</code> 能否修复
一个整数句柄	<code>fd table</code> 、打开文件对象	这个 <code>fd</code> 指向谁，是否可写，关 闭或 <code>exec</code> 时怎样处理
设备内部队列	<code>request</code> 、 <code>descriptor</code> 、 <code>completion</code>	哪个请求完成了， <code>buffer</code> 归谁， 错误怎样返回
缓存中的文件页	<code>page cache</code> 、 <code>inode</code>	哪些字节已修改，何时写回，谁 能看到
落盘顺序	<code>journal</code> 、 <code>commit record</code>	崩溃后哪些更新可信，哪些必须 丢弃或重放

因此，OS 对象不是概念装饰，而是把“函数调用里说不清的状态”搬到内核账本上。账本一旦建立，就要有权限、生命周期、等待队列和证据。没有这些，系统就只能靠约定运行，无法在不可信程序、慢设备和故障路径里保持一致。

本章使用的 `task`、`wait queue`、`VMA` 这些名字是教学叫法，但每一本账在 Linux 里都有一个真实结构体。先给出对照表，后面章节会逐个打开它们：

本章叫法	Linux 真实对象	源码位置
<code>task</code>	<code>task_struct</code> (<code>__state</code> 、 <code>mm</code> 、 <code>files</code> 等)	<code>include/linux/sched.h</code>
<code>wait queue</code>	<code>wait_queue_head</code> 、 <code>wait_queue_entry</code>	<code>include/linux/wait.h</code>
<code>VMA</code> <code>PTE</code>	<code>vm_area_struct</code> <code>pte_t</code> 与页表操作宏	<code>include/linux/mm_types.h</code> <code>arch/x86/include/asm/pgt able*.h</code>
<code>fd table</code>	<code>files_struct</code> 、 <code>fdtable</code>	<code>include/linux/fdtable.h</code>

打开文件对象	struct file (f_pos、f_count、include/linux/fs.h f_op)	
page cache	address_space、folio/page	include/linux/pagemap.h
request	struct request (blk-mq)	include/linux/blk-mq.h
completion	struct completion 与设备完 成队列项	include/linux/completion .h
日志提交记录	jbd2 transaction、commit block	include/linux/jbd2.h

现在不需要读懂这些结构体的字段。给真名的目的是让每本账都有坐标：后面任何一章说“打开文件对象”时，读者可以打开 `include/linux/fs.h` 自己核对它到底记了什么，而不是停留在比喻层。

3.2 保存请求的完整账本

天真的保存函数只看见一段顺序代码。真实保存请求从进入内核到对外宣布完成，中间每一步都要有对象承载状态、有边界说明语义、有证据报告失败。把这些对象排成一张账本，保存路径就不再是一串松散名词。

先把同一个保存请求放到真实 OS 语义里。用户程序看到的是几行 API：

```
int fd = open("data.tmp", O_WRONLY | O_CREAT | O_TRUNC, 0644);
write(fd, buf, len);
fsync(fd);
close(fd);
rename("data.tmp", "data");
int dirfd = open(".", O_RDONLY | O_DIRECTORY);
fsync(dirfd);
close(dirfd);
```

这几行代码不是只能靠想象理解。可以用系统调用跟踪工具观察它大致穿过了哪些内核入口。下面的输出是删去无关参数后的示意，真实机器上路径名、fd 号码和标志细节可能不同，但边界顺序是要看的重点：

```
$ strace -e trace=openat,write,fsync,close,renameat ./save-demo
openat(AT_FDCWD, "data.tmp", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "hello\n", 6) = 6
fsync(3) = 0
close(3) = 0
renameat(AT_FDCWD, "data.tmp", AT_FDCWD, "data") = 0
openat(AT_FDCWD, ".", O_RDONLY|O_DIRECTORY) = 3
fsync(3) = 0
close(3) = 0
```

`strace` 只能看到用户态和系统调用边界，不能直接告诉你 page cache、DMA 或文件系统日志里发生了什么。但它给了第一层证据：保存不是一个黑盒函数，而是一串明确的内核入口。每一行返回 `0` 或正数，只说明对应系统调用自己的契约成立；它不能自动替后面的边界背书。

观察到的入口	这行真正越过的边界	还没有自动保证什么
openat	路径解析、权限检查、fd 分配成功	后续写入一定成功
write	内核接受了 6 个字节，并推进到目标对象的写路径	数据已经在断电后可恢复
fsync(fd)	文件数据和必要文件元数据被要求推进到持久化边界	目录项名字更新已经可恢复

<code>close</code>	fd 槽位释放，并可能暴露延迟错误	另一个 fd 不再引用同一个打开文件对象
<code>renameat</code>	目录项从临时名字发布到正式名字	崩溃后一定能看到新名字
<code>fsync(dirfd)</code>	目录对象的更新被要求推进到持久化边界	更上层业务事务已经提交

这类观察方式很重要：先看用户能写出的代码和能抓到的输出，再解释输出背后有哪些看不见的状态对象。没有这一步，`page cache`、`fd table`、`journal` 很容易变成互不相干的名词；有了这一步，它们都在回答同一个问题：这行返回值背后到底有什么证据。

这段代码只保留语义主线；真实工程还要检查返回值，循环处理 `partial write`、`EINTR` 和清理路径。即使只看这条主线，一个看似简单的“保存”也已经穿过了主要 OS 对象。

1. `open` 先走路径解析、权限检查和 fd 分配。成功后，用户拿到的只是当前进程 fd table 里的一个整数索引。
2. `write` 通过系统调用进入内核，复制或固定用户 buffer，把数据推进打开文件对象、page cache、socket buffer 或设备对象。返回值不等于持久化成功。
3. 如果用户页或文件页不在内存，路径可能触发 page fault、page cache miss、块 I/O 和调度睡眠。
4. 若块设备异步写入，内核要建立 request，映射 DMA buffer，提交描述符，等待 completion，再解除映射。
5. `fsync` 把“内核已经接受写入”推进到更强的持久化边界，但仍要处理设备错误、flush、日志提交和元数据顺序。
6. `rename` 把临时文件发布成正式名字；目录 `fsync` 让名字更新本身也有崩溃恢复语义。

把这些状态写成一张账本：

阶段	状态对象	关键边界	失败证据
入口	task、入口现场	用户态切到内核态，保存返回位置	错误码、审计记录
授权	fd、打开文件对象、inode	fd 是否存在，目标是否可写	<code>-EBADF</code> 、 <code>-EACCES</code>
复制缓存	VMA、PTE、page page cache、inode	用户 buffer 是否可读 dirty 页只是内存状态	<code>-EFAULT</code> 、部分写入 dirty/writeback 计数、回写错误
提交 I/O	request、DMA map	buffer 在 completion 前不能释放	timeout、I/O error、completion status
持久化	journal、flush、directory entry	数据和名字是否崩溃后仍可解释	<code>fsync</code> 错误、replay、fsck 日志

这张表把保存请求从一个 API 调用拆成了六个阶段：入口、授权、复制、缓存、异步提交和持久化发布。任何一个阶段说不清，程序就可能在错误路径里撒谎：返回了成功，但对象状态并没有达到用户以为的边界。

保存路径里最容易混淆的五个边界：

- `write` 返回成功，不等于数据已经落盘。
- 线程处于 `runnable`，不等于正在运行。
- `fd` 是整数句柄，不等于文件本体。
- 虚拟地址落在 `VMA` 内，不等于 `PTE` 已经存在。
- 唤醒函数被调用，不等于等待者已经继续执行。

再把这段保存代码当成一个完整案例看。很多程序会先写成这样：

```
bool save_bad(const char* path, const char* buf, size_t len) {
    int fd = open(path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) return false;
    if (write(fd, buf, len) != len) return false;
    close(fd);
    return true;
}
```

它的表面意思很直接：打开目标文件，截断旧内容，写入新内容，关闭，返回成功。但把前面的账本套上去，会发现它在三个位置同时冒险。

位置	发生了什么	失败后用户可能看到什么
<code>O_TRUNC</code>	旧文件长度先变成 0	后续写失败时，旧内容已经丢失
一次 <code>write</code>	可能只复制部分字节，或停在 <code>page cache</code>	返回短写时，新文件是半成品
<code>close</code> 后返回	未检查 <code>close/fsync</code> 的延迟错误	后台写回失败，用户却看到保存成功

把它放到一个具体文件上看，会更容易发现危险。假设正式文件 `data` 原来保存的是 `"old\n"`，这次要保存的新内容是 `"new\n"`。用户想要的结果其实很严格：失败时最好还能读到旧版本；成功时读到完整新版本；最糟糕的状态是旧版本被破坏，而新版本又不完整。

执行到哪里	<code>data</code> 可能处于什么状态	为什么危险
<code>open(... O_TRUNC)</code> 之后	长度已经变成 0	还没写任何新数据，旧版本已经被破坏
<code>write</code> 短写之后	可能只有 <code>"ne"</code> 这样的前缀	系统调用允许部分推进，不能假设一次写完
<code>write</code> 返回、未 <code>fsync</code>	<code>page cache</code> 里可能是新内容，磁盘上可能仍是旧内容或部分更新	函数返回和持久化不是同一个边界
<code>close</code> 未检查错误	<code>fd</code> 被释放，但延迟 I/O 错误可能被忽略	调用者看到 <code>true</code> ，实际保存可能失败

改进方向不是机械记住“要用临时文件”，而是从失败路径推出临时文件的必要性：旧文件不能在新内容可靠之前被破坏。更稳的写法先把新内容写进同目录下的临时文件，确认它的数据和必要元数据已经推进，再用 `rename` 一次性替换名字。

先把这个推导拆开。错误版本从正式文件本身下手，所以第一步 `O_TRUNC` 就已经破坏旧版本；后面任何失败都会让用户处在尴尬状态：旧内容没了，新内容也没完整提交。临时文件版本把“准备新内容”和“发布新名字”分成两个阶段。准备阶段

只影响临时名字，失败就删除临时文件；发布阶段用 `rename` 切换目录项，使正式名字尽量只在旧版本和新版本之间切换。

设计选择	它来自哪个失败	边界含义
不用 <code>O_TRUNC</code> 直接改正式文件	写到一半失败会破坏旧版本	旧文件在新版本准备好前保持可用
使用同目录临时文件	跨文件系统 <code>rename</code> 不能保证同样语义	临时文件和正式文件处在同一个目录事务范围内
循环 <code>write_all</code>	一次 <code>write</code> 可能短写或被信号打断	未写完整就不能进入发布阶段
先 <code>fsync(tmpfd)</code> 再 <code>rename</code>	page cache 中的新内容不等于持久内容 正式名字只能指向旧版本或新版本	新文件内容先越过文件持久化边界 发布动作集中到目录项切换
最后 <code>fsync(dirfd)</code>	目录项本身也是元数据	崩溃后正式名字更新有恢复证据

先把 `write_all` 摊开。它不是为了把代码写得“保险一点”，而是因为 `write` 的基本契约就是“返回本次实际推进了多少字节”。只要返回值小于请求长度，调用者就不能假装完整数据已经进入保存路径。

```
bool write_all(int fd, const char* buf, size_t len) {
    size_t off = 0;

    while (off < len) {
        ssize_t n = write(fd, buf + off, len - off);

        if (n > 0) {
            off += (size_t)n;
            continue;
        }

        if (n == 0) {
            return false;           // no progress
        }

        if (errno == EINTR) {
            continue;               // interrupted before a final result
        }

        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            wait_until_writable(fd);
            continue;
        }

        return false;               // ENOSPC, EIO, EFAULT, ...
    }

    return true;
}
```

假设要写入 6 个字节 `hello\n`。第一次 `write` 只返回 2，表示 `he` 已经推进；第二次被信号打断，返回 `-1/EINTR`，表示没有得到一个最终完成结果；第三次返回 4，才把剩下的 `llo\n` 推进。此时 `off` 的变化是保存程序必须记住的状态。

调用	返回	<code>off</code> 变化	保存程序该做什么
----	----	---------------------	----------

第 1 次 <code>write</code> 2	0 变成 2	继续写剩余 4 字节
第 2 次 <code>write -1/EINTR</code>	仍是 2	不能报成功，重试 同一段剩余数据
第 3 次 <code>write</code> 4	2 变成 6	才能进入 <code>fsync(tmpfd)</code>

`EINTR`、`EAGAIN` 和短写看起来像细枝末节，但它们对应三种不同边界。`EINTR` 表示调用被信号路径打断；`EAGAIN` 表示对象现在不能推进，调用者要等待可写条件；短写表示系统已经接受了一部分数据，下一次必须从新 `offset` 继续。若保存程序不记录 `off`，就会重复写前缀、漏写后缀，或者把半成品当完整版本发布。

```
bool save_better(const char* dir, const char* name,
                 const char* buf, size_t len) {
    int dirfd = open(dir, O_RDONLY | O_DIRECTORY);
    if (dirfd < 0) return false;

    char tmp_name[64];
    int tmpfd = open_temp_at(dirfd, tmp_name, sizeof(tmp_name));
    if (tmpfd < 0) goto fail_dir;

    if (!write_all(tmpfd, buf, len)) goto fail_tmp;
    if (fsync(tmpfd) < 0) goto fail_tmp;
    if (close(tmpfd) < 0) {
        tmpfd = -1;
        goto fail_unlink;
    }
    tmpfd = -1;

    if (renameat(dirfd, tmp_name, dirfd, name) < 0) goto fail_unlink;
    if (fsync(dirfd) < 0) goto fail_dir;

    close(dirfd);
    return true;

fail_tmp:
    close_if_open(tmpfd);
fail_unlink:
    unlinkat(dirfd, tmp_name, 0);
fail_dir:
    close(dirfd);
    return false;
}
```

这段代码不是完整库函数；真实工程还要处理 `EINTR`、权限、临时文件命名、保留文件模式、跨文件系统 `rename` 等细节。但它已经把关键边界摆清楚了。

逐行看它的状态推进会更清楚。打开目录不是多余动作，因为最后要同步目录项；创建临时文件不是为了名字好看，而是为了让新内容有独立承载对象；`write_all` 不是普通循环，而是在处理“接口允许短推进”这个契约；`fsync(tmpfd)` 把临时文件内容向持久化推进；`close(tmpfd)` 还可能暴露延迟错误；`renameat` 才把正式名字切到新文件；`fsync(dirfd)` 则让这个名字变化本身在崩溃后可恢复。

代码位置	此刻系统必须记住什么	如果失败该怎么解释
<code>open(dir)</code>	目录对象和权限	目录打不开，保存根本没有进入准备阶段
<code>open_temp_at</code>	临时 <code>inode</code> 、临时目录项、 <code>fd</code> 引用	只清理临时对象，不影响正式文件

<code>write_all</code>	已写入字节数、dirty page、可能的短写	未完整写入，不允许发布
<code>fsync(tmpfd)</code>	数据和必要元数据的提交结果	返回错误说明新版本不可信
<code>renameat</code>	目录项从旧 inode 切到新 inode	失败时正式名字仍应指向旧版本
<code>fsync(dirfd)</code>	目录项更新的持久化证据	失败时必须向上报告，不能宣称完全保存

动作	它保护什么	崩溃后恢复含义
同目录临时文件	保证 <code>rename</code> 不跨文件系统，旧名字暂时不变	旧文件仍可作为最后成功版本
<code>write_all</code>	处理短写、 <code>EINTR</code> 、 <code>EAGAIN</code> 等推进问题	临时文件要么完整写入，要么不发布
<code>fsync(tmpfd)</code>	推进临时文件数据和必要元数据	新内容有机会在崩溃后被读回
<code>rename</code>	原子切换目录项指向	名字层面看到旧版本或新版本，不应看到半个名字
<code>fsync(dirfd)</code>	推进目录项更新本身	崩溃后正式名字更新有恢复证据

再用同一个 `data = "old\n"`、新内容 `"new\n"` 的例子看崩溃点。下面的表不假设某一种具体文件系统实现细节，只抓通用语义边界：正式名字什么时候还能指向旧版本，临时文件什么时候可以被丢弃，什么时候才能对用户说“新版本已经发布”。

崩溃发生在	重启后合理看到的状态	保存程序应该如何解释
临时文件创建前	<code>data</code> 仍是旧版本	保存没有开始影响正式文件
<code>write_all</code> 中途	<code>data</code> 仍是旧版本，可能残留半个临时文件	临时文件不能发布，清理后重试
<code>fsync(tmpfd)</code> 失败	<code>data</code> 仍是旧版本，新内容不可信	必须返回失败，不能继续
<code>renameat</code> 前	<code>data</code> 仍指向旧版本，临时文件可能完整	<code>rename</code> 新版本还没有发布给正式名字
<code>renameat</code> 后、目录未同步	运行时能看到新名字，但崩溃恢复后名字更新可能没有证据	不能把目录项持久化当成已经完成
<code>fsync(dirfd)</code> 成功后	<code>data</code> 应当指向可恢复的新版本	可以向上层报告保存跨过发布边界

这张表比“用临时文件更安全”更重要。它说明安全不是来自某个函数名，而是来自一组顺序关系：先保护旧版本，再准备新版本，再持久化新版本，再发布名字，最后持久化名字。任何一步失败，都要停在还能解释的状态上。

这段保存代码把前面的对象串成了一条完整链路。`task` 解释了为什么等待 I/O 时不能忙等；`VMA/PTE` 解释了 `buf` 为什么要复制检查；`fd`、打开文件对象和 `inode` 解释了为什么整数句柄不是文件本体；`page cache` 解释了 `write` 为什么可以早于设备完成返回；`request/DMA/completion` 解释了内核如何跟踪异步设备结果；`fsync/rename/目录 fsync` 解释了“保存成功”如何从内存状态变成崩溃后可解释的证据。

再看“对象”这个词，它就不再是抽象名词。对象就是内核为了记住某类状态而建立的账本。前面保存案例已经见过几本账：

对象	它记住什么	没有它会怎样
task	当前执行流是否 running、runnable、blocked，睡在哪里	设备慢时只能忙等，或者睡了以后无法恢复
wait queue	哪些 task 在等同一个谓词	设备完成后不知道该叫醒谁，容易 lost wakeup
VMA/PTE	地址区间的意图和单页翻译事实	用户随便传一个指针，内核不知道该复制、补页还是拒绝
fd table/打开文件对象	整数 fd 指向哪个打开对象，offset 和读写权限是什么	dup、close、共享 offset 都解释不清
page cache	文件内容在内存中的页面，哪些是 dirty	write 返回后无法说明数据停在哪里
请求/完成记录	哪次设备 I/O 已提交，哪次已经完成	中断来了也不知道对应哪个保存请求
日志/提交记录	哪些更新崩溃后可信	写一半掉电后无法区分成功更新和半成品

这张表背后的判断规则是：只要某个状态会被多个路径共享，或者失败后需要恢复，就不能只藏在一个函数局部变量里。内核必须给它名字、所有者、修改入口、等待位置和失败证据。

同样的账本形状也会出现在别的对象上。socket 会把网络连接变成对象；地址空间会把整套页表和 VMA 组织起来；块层会把一次大写入拆成多个 request；安全子系统会把权限判断变成可审计的拒绝路径。对象一旦进入内核，就必须说明它记录什么、谁能改它、错了以后怎么证明。

保存案例还暴露出一个分层事实：同一个词在用户 API、内核对象和硬件事件三层里含义不同。用户说地址，可能只是一个指针值；内核说地址，要看 VMA、PTE 和权限；硬件说地址，要看页表翻译、TLB 和物理地址。用户说完成，可能只是函数返回；内核说完成，可能只是 page cache 或 request 状态更新；设备说完成，可能只是某次 I/O 结束。

因此，同一个动作要同时放进三层：用户 API 层、内核对象层、硬件事件层。

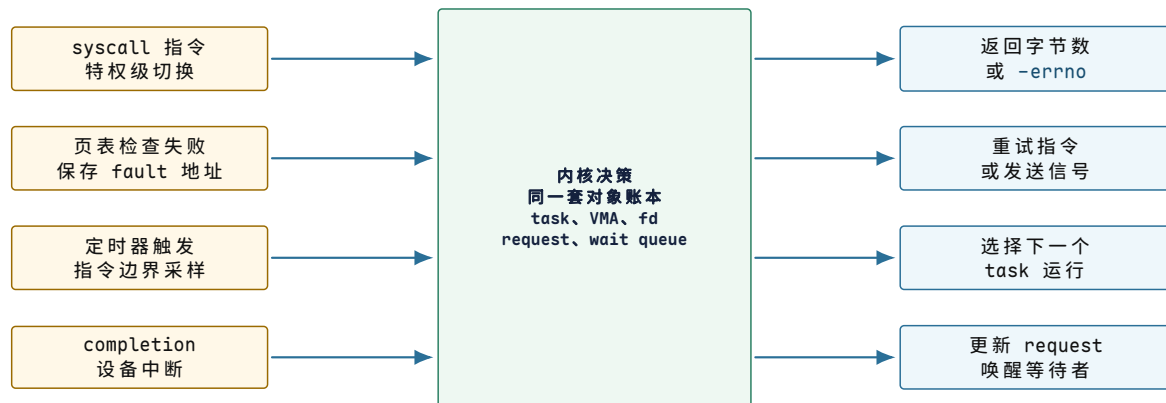
说法	用户 API 层	内核对象层	硬件层
地址	指针值，例如 0x1000	VMA/PTE 和权限	页表翻译、物理地址、设备寄存器地址
可读	read 不会立刻阻塞	buffer/队列中有数据或条件满足	中断、completion、DMA 写入
写完成	系统调用返回字节数	page cache 或 request 状态更新	设备接受请求，介质 flush 可能还没发生
上下文	用户调用链和线程身份	task、保存的返回位置、内核栈	程序计数器、栈指针、寄存器
错误	errno 或信号	对象状态拒绝、权限失败、生命周期结束	fault、设备错误、中断错误

以“写完成”为例。写普通文件时，用户看到 write 返回；内核看到 page cache 被改成 dirty；设备可能还什么都没看到。等后台回写或 fsync 提交 request 后，设备才开始处理这次 I/O。等 completion 到来，内核知道这次设备请求结束了。等 flush 或日志 commit 过了，崩溃恢复才有证据。这些都叫“完成”，但边界完全不同。

再看“可读”。普通文件几乎总是可读，因为内核可以从 page cache 或磁盘推进；socket 可读表示接收队列有数据、连接关闭或错误；设备 fd 可读可能表示驱动队列里有 completion。用户只看到 poll 返回，但内核必须把对象状态和硬件事件翻译成统一接口。

3.3 四条完整路径：从硬件事件到内核决策

一次保存请求不是一条顺着函数调用栈走到底的直线。它会被四类入口打断和续上：系统调用入口、缺页入口、定时器入口和设备完成入口。每条路径都从硬件事实进入内核决策，再回到用户能观察到的结果。



图示：四条路径入口不同、结果不同，但中间经过的是同一套内核对象账本。理解其中任何一条，都要能说清它读写了哪些共享状态——这正是后面每一章的展开方式。

第一条路径是系统调用入口。应用看见的是一个 C 库函数，内核看见的是一次受控入口和多个对象状态更新。这里的寄存器名字来自 x86-64 调用约定，用来表达四件事：系统调用号、fd、buffer 地址和长度被交给内核。

```

user:
    rax = SYS_write
    rdi = fd
    rsi = user_buf
    rdx = len
    syscall

kernel entry:
    save user return position and registers
    switch to kernel-owned stack
    locate current task and fd table
    validate fd refers to writable file, socket, or pipe
    validate user buffer range and copy or pin pages
    update target object state
    return byte count or -errno in rax
  
```

阶段	关键检查	失败表达
系统调用入口	syscall number 是否存在，当前上下文是否允许	-ENOSYS、拒绝或信号
fd 查找	fd 是否打开，是否有写权限	-EBADF、-EPERM
用户 buffer	地址格式是否有效、VMA 是否允许读、复制中是否 fault	-EFAULT 或部分写入
目标对象	文件、pipe 或 socket 是否可写、是否阻塞	-EAGAIN、睡眠、信号中断

提交语义	数据进入哪里：page cache、返回值不一定表示落盘或 socket buffer、设备队列	发出网络包
------	--	-------

这条路径说明了 OS 契约的层次。系统调用返回成功，只说明内核接受并完成了该接口定义下的一部分工作。写普通文件可能只是写入 page cache；写 socket 可能只是进入发送缓冲；写终端可能经过驱动队列和中断。若业务需要“崩溃后仍存在”，还要继续追 `fsync` 和存储 `flush`；若业务需要“对端已收到”，还要追协议确认。

第二条路径是 `page fault`。内核按用户地址复制 `user_buf` 时，CPU 可能发现页表里没有可用翻译，于是触发 `page fault`。硬件做的是检查和转交，内核做的是解释和修复。

```
CPU:
  compute the address being loaded
  check whether the address form is valid
  lookup TLB or walk page table
  if missing or permission denied:
    save faulting instruction position and error code
    record fault address
    enter page fault handler

kernel:
  find VMA covering fault address
  compare access type with VMA permissions
  inspect PTE state
  allocate a page, read a file page, copy a shared page, or reject
  install PTE and clear stale address-cache entries when needed
  return to the faulting instruction or deliver signal
```

硬件看到	内核进一步区分	结果
<code>present=0</code>	地址属于匿名 VMA	分配零页, <code>minor fault</code>
<code>present=0</code>	地址属于文件映射	<code>page cache</code> 命中或发起磁盘 I/O
<code>write denied</code>	PTE 标记 <code>shared-copy</code>	复制页并恢复写权限
<code>execute denied</code>	<code>NX</code> 或 VMA 不可执行	发送信号, 安全拒绝
<code>user bit denied</code>	用户访问内核专用页	发送信号或记录攻击/bug

缺页机制的关键不是“慢”，而是“可恢复边界”。如果 CPU 不能保存出错指令的位置，修好页表后就无法从原指令重试；如果内核不能判断 VMA 意图，`present=0` 就无法区分合法懒分配和非法访问；如果旧的地址翻译缓存没有清掉，PTE 更新后旧权限仍可能生效。

第三条路径是定时器中断。用户程序可能永远不主动让出 CPU。定时器中断给内核一个周期性入口，让它能统计时间、处理超时并决定是否切换任务。

```
hardware timer fires:
  CPU takes interrupt at instruction boundary
  entry saves interrupted return position and registers
  timer handler increments scheduler clock
  expired timers wake sleeping tasks
  current task runtime is charged
  if time slice or policy says preempt:
    set need_resched
  return path calls scheduler before going back to user
  scheduler saves current kernel context and restores next
```

这里有两个容易混淆的点。第一，中断处理函数通常不在任意深处直接把用户栈改成另一个任务；它设置调度标志，在安全返回点进入调度。第二，切换任务不是

只改一个 `current` 指针，还可能涉及内核栈、需要保留的寄存器、地址空间和地址翻译缓存。

动作	需要保存什么	保存错误的现象
中断进入	用户返回位置、栈指针和寄存器	返回后用户程序寄存器随机坏
调度统计	当前任务运行时间和状态	CPU 时间不公平或饥饿
切换内核上下文	内核栈指针、必须保留的寄存器	任务从错误调用链恢复
切换地址空间	当前页表身份、必要的翻译缓存处理	任务访问到别人的内存

第四条路径是设备完成。设备 I/O 是异步的。提交请求后，任务可能睡眠；设备通过 DMA 和中断报告完成；内核再把 `completion` 变成任务可见结果。

```
submit I/O:
  allocate request object
  map buffer for DMA
  fill device-visible descriptor
  publish descriptor in the required order
  notify the device
  if synchronous API:
    put task on wait queue and schedule away

device:
  reads descriptor by DMA
  transfers data
  writes completion entry
  raises interrupt

interrupt/bottom half:
  read completion queue
  unmap DMA and mark request done
  store status and byte count
  wake tasks waiting on request
```

这条路径解释了为什么 I/O 子系统一定有 `request identity`，也就是“请求身份”。中断到来时，CPU 不知道“哪个用户函数在等它”，只看到某个设备队列有 `completion`。内核必须通过队列位置、`tag`、`request` 指针或 `completion entry` 找回原请求，撤销 DMA 授权，更新上层对象，再唤醒等待者。没有这层身份映射，异步 I/O 无法可靠对应到发起者。

把这个过程压到一个设备队列里看。驱动把请求写进一块设备也能读取的 `descriptor ring`；`descriptor` 是一条描述符，里面记录 `buffer` 地址、长度、方向和 `tag`。`doorbell register` 是设备暴露给 CPU 的一个通知寄存器；驱动把 `descriptor` 写好后，再写 `doorbell`，告诉设备“队列里有新请求”。`completion queue` 则是设备写回结果的队列。

```
submit A:
  descriptor[5] = { tag: 17, buffer: page0, len: 4096, op: write }
  doorbell = 5

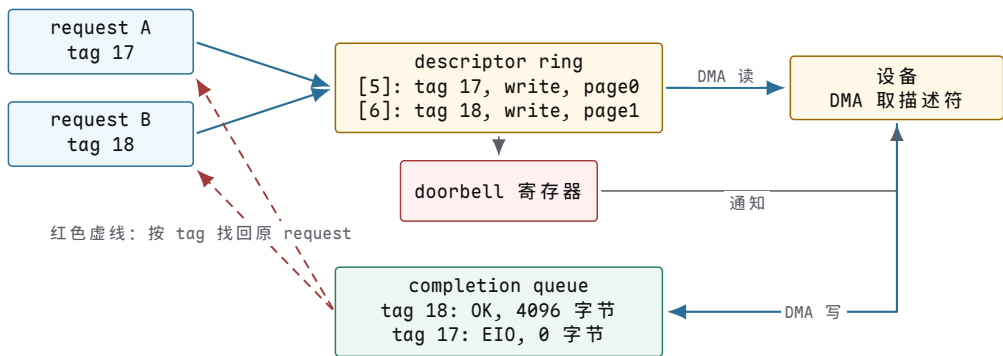
submit B:
  descriptor[6] = { tag: 18, buffer: page1, len: 4096, op: write }
  doorbell = 6

completion queue:
  { tag: 18, status: OK, bytes: 4096 }
```



```
{ tag: 17, status: EIO, bytes: 0 }
```

完成顺序不一定等于提交顺序。设备可以先完成 B，再报告 A 失败。若内核只按“第一个 completion 对应第一个等待者”处理，就会把 B 的成功错还给 A，把 A 的错误错还给 B。tag 的作用就是把设备事实重新接回内核对象账本。



图示：完成顺序不等于提交顺序：B 先成功、A 后失败是合法结果。completion entry 里的 tag 把设备事实重新接回正确的 request 账本，两条红色虚线就是这次“找回”。

阶段	硬件/驱动看到什么	内核必须维护什么
填 descriptor	buffer 地址、长度、操作类型、tag	request 对象仍然活着，buffer 不能释放
写 doorbell	设备被通知去取 descriptor	descriptor 字段必须已经完整可见
设备 DMA	设备直接读写内存 buffer	页面要被固定或映射，防止被回收复用
写 completion	completion entry 带回 tag 和 status	用 tag 找回 request，记录成功或错误
唤醒等待者	等待队列里的 task 重新变 runnable	醒来后读取 request status，不猜测结果

这里最危险的错误不是“设备慢”，而是生命周期错位。若 completion 之前释放 buffer，设备可能把数据写进已经分配给别人的页面；若 completion 没有 status，调用者只能知道“有中断”，不知道哪次写失败；若唤醒早于 request status 更新，等待者醒来仍可能读到旧状态。异步设备因此迫使 OS 明确三件事：请求身份、buffer 生命周期和完成证据。

四条路径讲完，同样给它们配上真实坐标。下表以 x86-64 主线为准（其他架构的入口名字不同，但职责相同）：

路径	Linux 入口函数（x86-64 主线）	深入章节
系统调用	<code>entry_SYSCALL_64</code> (<code>arch/x86/entry/entry_64.S</code>)、 <code>do_syscall_64</code> (<code>arch/x86/entry/common.c</code>)	第 5 章
缺页异常	<code>exc_page_fault</code> (<code>arch/x86/mm/fault.c</code>)、 <code>handle_mm_fault</code> (<code>mm/memory.c</code>)	第 8 章
定时器中断	<code>scheduler_tick</code> 、 <code>__schedule</code> (<code>kernel/sched/core.c</code>)	第 4 章

设备完成 驱动 irq handler、complete 第 11 章
 (kernel/sched/completion.c)、
 wake_up

现在不要求读懂这些函数。记住对应关系即可：本章每一条“硬件事实 → 内核决策 → 用户可见结果”的路径，在源码里都有一个能打开的入口；后续章节会逐个走进去。

3.4 从硬件事实到 OS 抽象

OS 账本必须落在硬件事实上。组成原理解释机器能做什么，操作系统解释这些能力如何被组织成可共享、可隔离、可恢复的服务。CPU 能保存执行位置，所以 task 可以睡眠再恢复；页表能检查地址，所以内核能拒绝坏 buffer；设备能异步完成，所以内核要维护 request 和 completion；存储可能掉电，所以文件系统要有提交边界。

组成原理事实	OS 抽象	关键问题
程序计数器、寄存器和栈可保存	task 上下文	切走后如何从同一位置恢复
特权级限制敏感指令	用户态/内核态	用户如何安全请求内核服务
异常入口能保存出错位置	page fault、signal、debug	错误能否修复并返回原指令
定时器可强制中断	抢占式调度	不合作任务如何被收回 CPU
MMU 检查页表权限	地址空间	进程如何互相隔离又能共享
cache/TLB 需要维护	性能和一致性策略	为什么迁移、锁和取消映射有成本
设备寄存器控制设备	驱动模型	寄存器副作用和顺序如何管理
DMA 异步访问内存	高性能 I/O	buffer 何时归 CPU，何时归设备

看到“进程”，需要同时看到一组保存的 CPU 上下文、地址空间和内核对象，而不只是 pid。看到“文件”，需要同时看到 fd 表、打开文件对象、inode、page cache、块层和设备队列，而不只是路径。看到“权限”，需要同时看到 CPU 特权级、页表权限、文件权限和对象引用，而不只是口令或用户 ID。

用户接口词、内核对象词和硬件词经常指向同一条路径的不同层。用户接口会说 process、file descriptor、signal、pipe、socket；内核对象会说 task、file、inode、wait queue；硬件路径会说寄存器、页表、中断、DMA、设备寄存器。三套词不是互相替代，而是共同描述一次动作怎样穿过系统。

用户语义	内核对象	硬件支点	关键边界
process	task、地址空间、fd table	寄存器、页表、定时器	身份、地址空间、权限分离
thread	task、内核栈、共享地址空间	寄存器、栈指针	可调度上下文，不复制全部资源
fd	fd table、打开文件对象	设备队列、page cache、DMA	整数句柄不是文件本体
signal	待处理信号、用户返回现场	保存的异常/中断现场	异步交付但必须恢复用户现场

socket	socket 对象、wait queue	网卡队列、中断、DMA	协议状态和设备完成分离
timer	定时器对象、等待队列	硬件定时器中断	未来事件需要硬件唤醒入口

例如 fd 是一个用户态整数，但内核要通过当前进程的 fd table 找到打开文件对象，再通过 file operations 调到 pipe、socket、inode 或 device 的具体实现。若底层是网卡，最终还会走网络缓冲区、驱动队列、DMA 和 completion。把 fd 当成“文件本体”，就解释不了 dup 后共享 offset、socket readiness、设备拔出后旧 fd 返回错误这些现象。

单片机和裸机程序能暴露 OS 的原始需求。一个裸机程序拥有全部内存和设备，所有代码同权，主循环靠自觉返回，中断直接修改全局状态。高级机制出现之前，失败通常已经在这种小系统里出现过。

裸机阶段的问题	局部处理	通用 OS 机制
函数忙等设备	改成状态机或中断	阻塞、等待队列、completion
任务靠自觉返回	任务表和显式 yield	线程和协作式调度
任务可能不合作	加硬件定时器	抢占和时间片
全局变量被中断处理函数打断	关中断临界区	锁、原子操作、内存屏障
所有代码同权	约定模块边界	CPU 特权级和系统调用
所有内存同权	手工内存布局	MMU、地址空间、页表权限
CPU 搬每个字节	DMA 控制器	异步 I/O 和驱动队列

如果不知道硬件能否强制中断、能否区分用户/内核、能否检查页表权限，就无法判断一个 OS 设计到底是“规范约定”还是“硬件保证”。

3.5 保存路径的五个关口

保存请求走到这里，已经不是一段顺序代码，而是一串状态和边界。每个边界都回答一个具体问题：对象是谁，谁拥有它，哪个入口能修改它，不能继续时停在哪里，完成时用什么证据说明结果可信。

问题	保存路径中的内容
状态对象	进程、线程、页、fd、inode、socket、设备请求、日志记录
所有者	用户态对象、内核对象、硬件寄存器、设备控制器
入口	系统调用、异常、中断、内核线程、后台回收
权限	特权级、页表权限、文件权限、资源能力、命名空间、引用计数
等待	runqueue、futex、page fault、I/O completion、锁、条件变量
提交	状态更新、引用释放、写入 page cache、落盘、目录项发布生效
证据	返回值、errno、trace、计数器、日志、测试断言

例如一次 write(fd, buf, len) 可以这样拆：用户态只有一个整数 fd 和一段用户 buffer；内核要先从进程 fd 表找到打开文件对象，再检查写权限，再复制用户 buffer，再把页标记为 dirty，最后返回写入字节数或错误。这个返回值通常不等于数据已经安全落盘。要证明持久化，还需要 fsync、目录同步或更高层提交记录。

再看一次 page fault。用户态执行 load/store，硬件发现页表项不存在或权限不符，CPU 进入异常入口，内核根据 VMA 判断这是合法缺页、权限错误还是非法地

址。合法缺页可能分配物理页或读取文件页；权限错误可能发送信号；非法访问可能终止进程。这里的核心不是“缺页很慢”，而是虚拟地址访问被拆成硬件检查和内核决策。

只看正常路径仍然不够。OS 机制真正麻烦的地方，是反例会从边界处钻出来。

机制	正常路径	必须补的反例
系统调用	参数合法，返回字节数或结果	用户指针跨页且后一页不可读，返回 <code>-EFAULT</code> 或部分成功
调度	runnable 任务被选中运行	任务睡在 wait queue 中，不能因为优先级高就运行
锁	持锁者修改共享状态后释放	两把锁顺序反过来，形成 wait-for 环
缺页	VMA 合法，分配或读入页面	地址不属于任何 VMA，必须发信号而不是分配页
DMA	completion 到达后回收 buffer	completion 前释放 buffer，设备可能写坏别人的内存
持久化	日志提交后恢复可重放	写了数据但没写 commit，崩溃后不能当成功

一个机制是否成立，先看没有它时哪里坏，再看新增了什么状态对象，最后看正常路径和反例能不能守住边界。

层次	必须回答的问题	例子
原始失败	没有这个机制时，哪个具体动作会坏	死循环霸占 CPU；用户指针越界；写一半掉电
状态对象	内核新增哪本账，记录哪些字段	task state、runqueue、VMA、PTE、request、journal record
状态转移	谁能改账本，入口是什么	timer interrupt 改调度状态；page fault 安装 PTE；completion 完成 request
不变量	哪些关系不能被破坏	sleeping 任务不能在 runqueue；DMA 完成前 buffer 不能释放
反例测试	怎样故意触发边界	lost wakeup、非法地址、共享页写入、I/O error、崩溃恢复

例如调度不能只列 FCFS、RR、CFS，而要回答：死循环为什么能霸占 CPU；定时器如何给内核抢占入口；task 在 runnable、running、blocked 之间怎样转移；runqueue 用队列、树或多队列时复杂度怎么变；lost wakeup 和饥饿怎样测出来。虚拟内存也一样：直接物理地址会互相破坏，VMA 表达地址区间意图，PTE 记录单页翻译事实，page fault 把硬件证据交给内核决策，非法地址、共享页写入、权限错误和 TLB 失效分别压在不同边界上。

保存路径最终会落到五个维度。

维度	追问	例子
身份	对象如何被引用	pid、tid、fd、inode number、socket 四元组、DMA tag
权限	谁能做什么	页表读写执行位、fd 标志、资源能力、命名空间

生命周期	何时创建和释放	fork/exit/wait、open/dup/close、page refcount、request completion
等待	无法立即推进时停在哪里	runqueue、wait queue、page fault、I/O queue、timer
提交	何时对外可见或持久	系统调用返回、PTE 安装、DMA 完成、fsync、提交记录

只说其中一个维度，很容易把机制讲偏。比如“socket 可读”只覆盖了等待维度，还要继续追：哪个 fd 引用它，谁有权限读，缓冲区里数据生命周期如何，读取后队列怎样变化，关闭时如何唤醒等待者。

这些维度还要放回层次关系中。操作系统经常呈现层次结构：硬件在底层，内核抽象在中间，用户 API 在上层。但真实问题会反向穿透层次。一个 write 系统调用看似属于文件 API，却可能等待内存回收、块层队列、驱动中断、设备 flush 和调度器唤醒。一个 page fault 看似属于内存管理，却可能读取文件页，触发块设备 I/O。一个安全拒绝看似属于权限系统，却依赖路径解析、命名空间和当前进程的身份信息。

```
user write()
-> fd table
-> open file object
-> page cache
-> writeback
-> block layer
-> driver DMA
-> interrupt completion
-> wakeup
```

真实系统不是互不相干的模块，而是交叉依赖的状态机网络。
保存请求不能只是一段普通函数代码，因为它至少要穿过五个关口。

关口	要解决的问题	保存案例中的落点
机器能保存状态	CPU、寄存器、栈、时钟和设备寄存器如何形成可恢复现场	task 能从睡眠处回来，设备 ready 位能被观察
机器能被打断	中断、异常和定时器怎样把外部事件交给内核	设备完成能唤醒等待者，死循环能被抢占
地址能被检查	MMU、页表、VMA 和 PTE 怎样区分合法缺页和非法访问	用户 buffer 跨页时，内核知道复制、补页还是拒绝
资源能被引用	fd、打开文件对象、inode、socket、request 怎样把一次操作变成对象账本	整数 fd 找到文件对象，I/O completion 找回请求
结果能被证明	page cache、writeback、journal、fsync 和 rename 怎样推进提交边界	“保存成功”从函数返回推进到崩溃后可恢复

这些关口也可以压成一张“失败、机制、代价”表：

失败	需要的机制	新增代价
固定电路不通用	存储程序 CPU	指令编码、取指译码、异常语义
单核状态无法被安全共享	多核一致性和原子操作	cache line 迁移、内存序、锁竞争

程序直接用物理地址	MMU 和虚拟地址	页表、TLB、缺页成本
CPU 搬运所有 I/O 数据	DMA 和描述符队列	buffer 生命周期、IOMMU、completion
轮询浪费 CPU	中断	异步共享状态
任务死循环霸占 CPU	定时器抢占	上下文保存和调度开销
任务互相覆盖内存	地址空间和权限	页表、缺页、TLB 成本
用户能随便改设备 I/O	系统调用和特权级	入口检查和复制成本
等待拖住计算	阻塞队列、异步和 completion	请求身份和 buffer 生命周期
写一半崩溃	日志、fsync、提交记录	写放大和恢复复杂度

OS 测试不是只看程序有没有跑完，而是验证契约有没有守住。正常路径、错误路径和边界路径都要能指出是哪条不变量断了。

- 系统调用能拆成用户态入口、内核对象、权限检查、状态更新和返回值。
- “函数返回成功”“内核接受请求”“设备完成”“业务提交生效”不能混成同一个完成点。
- 等待现象必须落到具体队列：CPU、锁、页面、I/O、子进程都不是同一种等待。
- 隔离边界必须落到硬件或内核对象：地址空间、文件权限、用户/内核态、命名空间各管不同问题。
- 任意机制都应有状态、入口、失败和证据四列。

反例同样重要。若仍然把 `fd` 当成文件本体，把 `write` 成功当成落盘，把线程 `ready` 当成 `running`，把 `malloc` 成功当成物理页已分配，把 `notify_one` 当成消息投递，就说明 OS 契约还没有建立。

3.6 动手验证：把本章落到证据

本章每个关键断言都可以在一台真实 Linux 机器上复查。下面这组实验不需要特殊环境；本章展示的输出就是在本书实验机（aarch64 容器）上跑出来的，你的数字可能不同，但边界关系应当一致。

1. 坏地址实验：把 `write` 的 `buffer` 指向 `0x1`，观察程序打印 `n=-1 errno=14 (Bad address)`；再用 `strace -e trace=write` 对照系统调用边界上的真实一行。
2. `offset` 归属实验：分别跑 `dup`、重新 `open`、`fork` 三个版本，用 `od -An -c data` 验证最终内容是 `CB`、`B`、`CP`，并解释差异来自打开文件对象的共享关系。
3. `fd` 观察：让 `fd_demo` 睡眠，读 `/proc/<pid>/fd` 和 `/proc/<pid>/status`，确认两个 `fd` 指向同一个文件、进程状态是 `S (sleeping)`。
4. 完成点距离实测：给 64KiB 的 `write` 和紧随其后的 `fsync` 分别计时，比较两者差几个数量级；换存储介质（`tmpfs`、SSD、机械盘）再测一次，观察差距怎样变化。
5. 崩溃点推演：在完成点时间线上任选一点假设“断电”，写出重启后用户、内核、设备各自能看到什么，并说明哪一层的账本足以支撑恢复结论。

做到这五条，本章的契约就不是背下来的，而是被自己亲手推翻过又重建的。

操作系统不是一堆彼此孤立的子系统，而是在维护一组可验证契约。遇到任何机制，都要先找没有它时哪个具体动作会坏，再问内核新增了哪本账，谁能通过哪个入口改这本账，不能继续时睡在哪里，成功和失败分别留下什么证据。保存按钮只是一个入口；同样的方法也适用于调度、虚拟内存、文件系统、网络和设备驱动。

第二部分

进程、调度与并发

本部分导读

第一部分说明了内核怎样获得入口和保存现场；这一部分回答入口之后最先遇到的问题：多个执行流怎样共享 CPU，怎样睡眠，怎样被唤醒，怎样避免共享状态互相破坏。

进程和线程先提供可切换的执行流；系统调用把不可信参数带入内核；`exec` 把旧程序映像替换成新映像；`futex`、IPC、信号和锁把”等待哪个对象、谁负责唤醒、醒来后是否还要复查条件”讲完整。

这一部分的出口是等待模型：线程不在 CPU 上时，必须能说清它是在等 CPU、锁、I/O、页面、信号、子进程还是定时器。说不清等待对象，后面的内存和 I/O 章节就会断线。

第 4 章 进程、线程与调度

4.1 原理

操作系统的第一个核心职责是分配 CPU 时间。硬件只有有限核心，用户却希望多个程序同时推进。进程提供资源边界，线程提供执行边界，调度器在可运行实体之间选择下一个执行者。这里的关键不是“轮流执行”四个字，而是每个执行实体必须处在可解释的状态里。

先从一个很小的服务器看问题：

```
while true:
    c = accept(listen_fd)
    data = read(c)
    write(log_fd, data)
    write(c, "ok")
```

如果这段代码只有一个执行流，问题马上出现：没有客户端连接时，`accept` 会等待；客户端发数据慢时，`read` 会等待；磁盘慢时，写日志会等待。CPU 在这些等待期间其实可以运行别的请求，但硬件不会自动知道“这个程序在等网络，那个程序可以继续算”。内核必须把“能继续执行”和“正在等待某个条件”区分成状态，再让调度器只选择能继续执行的实体。

最朴素的错误调度器会这样写：

```
for task in all_tasks:
    run(task)
```

它的问题不是代码短，而是没有状态。一个正在等磁盘的任务被运行也没有意义；一个已经退出的任务不该再运行；一个刚被网卡中断唤醒的任务应该重新进入候选集合；一个持有锁的内核路径不能在任意位置被切走。于是调度章真正要回答的不是“怎么排序”，而是四件事：任务状态怎么表示，等待条件怎么保存，唤醒怎样回到 `runqueue`，上下文怎样被保存和恢复。

进程不是一个正在运行的函数。它至少包含地址空间、文件描述符表、信号处理状态、凭据、资源限制、子进程关系和一个或多个线程。线程是内核可调度实体，拥有寄存器上下文、栈、调度状态和等待原因；同一进程内的线程共享地址空间和许多进程资源，所以它们能高效共享数据，也更容易互相破坏。

这里第一次出现两个容易混的词。进程是资源容器：它回答“这个程序能看见哪些地址、打开了哪些 `fd`、以什么身份运行”。线程是执行实体：它回答“当前 `RIP/RSP`/寄存器在哪里，能不能被调度到 CPU 上”。一个单线程进程看起来像两者合一；多线程进程把区别暴露出来：多个线程共享同一个地址空间和 `fd` 表，但每个线程有自己的栈、寄存器现场和等待状态。

概念	回答的问题	常见误解
进程	资源和权限边界是什么	误以为进程就是正在跑的那条函数调用链
线程	哪个执行现场可以放到 CPU 上	误以为线程只是一段代码，而不是保存现场的对象
<code>runqueue</code>	哪些线程现在可以运行	误把所有未退出线程都放进候选集合
<code>wait queue</code>	哪些线程在等同一个条件	误以为睡眠线程会自动被 CPU 记住
上下文切换	保存当前现场，恢复另一个现场	误以为只改一个 <code>current</code> 指针就够

进程生命周期可以简化为 `created`、`runnable`、`running`、`blocked`、`zombie`、

reaped。线程状态可以简化为 `runnable`、`running`、`sleeping`、`stopped`、`terminated`。真实内核状态更多，但第一版必须先守住这些边界：`blocked` 线程不能被调度执行，`terminated` 线程不能重新运行，`zombie` 进程已经退出但仍等待父进程回收退出状态。

调度策略必须回答：谁处于 `runnable`，谁在 `blocked`，时间片用完后怎么办，优先级是否影响选择，任务退出后资源如何回收，唤醒是否会造成饥饿。一个可解释的调度器必须把这些状态外化，而不是只返回一个线程编号。

上下文切换是调度的成本边界。切换时内核要保存当前线程的寄存器、栈指针和调度账本，选择下一个线程，恢复它的上下文，并可能改变地址空间和 TLB 状态。线程切换不一定改变地址空间，进程切换通常会带来更重的内存管理成本。高并发程序如果制造过多 `runnable` 线程，就会把时间花在 `runqueue` 等待、抢占和上下文切换上。

等待是调度的另一半。一个线程不运行，可能是被抢占，也可能是主动睡眠，可能等锁、等 I/O、等 `page fault`、等子进程、等信号、等定时器。分析性能和死锁时，必须区分 `on-CPU` 时间和 `off-CPU` 时间。只看 CPU 使用率会误导：CPU 低可能是线程都阻塞了，CPU 高可能是大量线程在抢锁或忙等。

后续章节的很多机制都会回到这套状态机。缺页时，线程可能睡在“等待磁盘把文件页读入”的队列上；`socket` 没数据时，线程睡在 `socket receive wait queue` 上；`waitpid` 睡在子进程退出事件上；`fsync` 睡在写回 `completion` 上。所谓操作系统“同时处理很多事”，本质是把不能前进的执行流从 CPU 上拿下来，并保证条件满足时能找回它。

4.2 实践

纸上实践先画三张表。第一张是进程表，列出 PID、父 PID、状态、退出码、地址空间和 `fd` 表引用。第二张是线程表，列出 TID、所属 PID、状态、等待原因、优先级、最近运行 CPU 和已消耗 `tick`。第三张是 `runqueue`，列出当前可运行线程的顺序和调度账本。

tick	事件	runqueue	运行线程	状态变化
0	创建 A/B	A,B	A	A running
1	时间片结束	B,A	B	A runnable, B running
2	B 发起 I/O	A	A	B blocked(io)
3	I/O 完成	A,B	A	B runnable
4	A exit	B	B	A zombie

这张表比一句“轮转调度”更有价值，因为它同时呈现了状态、队列和事件。后续任何复杂策略都可以在这张表上增加字段，例如虚拟运行时间、`deadline`、优先级、CPU affinity、NUMA node 或 `cgroup quota`。

实践报告必须区分三个时间：`wall time`、`runnable wait` 和 `on-CPU time`。`wall time` 是请求从开始到结束的总时间；`runnable wait` 是线程已经可运行但还没拿到 CPU 的时间；`on-CPU time` 是真正执行指令的时间。把三者混在一起，会把调度问题误判成算法问题。

4.3 算法与实现分析

FCFS：最简单也最容易产生长等待。先来最简单的调度算法：`first-come, first-served`。任务按进入 `runnable` 队列的顺序执行，一个任务运行到阻塞、退出或主动让出 CPU 后，才轮到下一个任务。

```
while true:
    task = runqueue.pop_front()
    run_until_block_or_exit(task)
```

```
if task.state == RUNNABLE:
    runqueue.push_back(task)
```

FCFS 的实现成本很低：入队 $O(1)$ ，出队 $O(1)$ 。它的问题是 convoy effect。一个长 CPU 任务排在前面，后面的短交互任务会长时间等待。若系统只跑批处理，FCFS 可以作为 reference；若系统需要交互响应，它通常不合格。

Round-robin：用时间片限制独占。Round-robin 在 FCFS 上增加时间片。每个 runnable 任务最多运行一个 quantum，时间片到期后被放回队尾。它解决了“一个任务长期霸占 CPU”的问题，但引入了时间片选择：时间片太大，交互延迟高；时间片太小，上下文切换成本高。

```
on timer_tick:
    current.used_ticks += 1
    if current.used_ticks == current.quantum:
        current.used_ticks = 0
        current.state = RUNNABLE
        runqueue.push_back(current)
        current = runqueue.pop_front()
```

假设上下文切换成本是 S ，时间片是 Q ，CPU 时间被切换消耗的比例大约是 $S / (S + Q)$ 。 Q 越小，响应越好，但浪费越多。这个公式不是精确模型，却能解释为什么时间片不能无限小。

优先级调度和饥饿。优先级调度把 runqueue 拆成多个队列，总是选择最高优先级的 runnable 任务。它适合表达“控制任务比日志任务重要”，但如果高优先级任务持续可运行，低优先级任务可能永远得不到 CPU。

```
for priority from HIGH to LOW:
    if !queue[priority].empty():
        return queue[priority].pop_front()
return idle_task
```

解决饥饿的常见方法是 aging：等待越久，优先级逐步提高。aging 的实现要小心，不能每个 tick 扫描所有任务，否则任务多时成本过高。可以在任务入队时记录等待起点，在选择或周期性维护时计算有效优先级。

MLFQ：用反馈区分交互和批处理。Multilevel feedback queue 使用多个优先级队列，新任务先进入高优先级；如果任务用完整个时间片，说明它偏 CPU 密集，下次降级；如果任务很快阻塞或让出 CPU，说明它可能是交互任务，保留较高优先级。MLFQ 不需要提前知道任务类型，而是从行为反馈中调整。

```
on_task_created(task):
    task.level = 0
    queues[0].push_back(task)

on_quantum_expired(task):
    task.level = min(task.level + 1, MAX_LEVEL)
    queues[task.level].push_back(task)

on_task_blocked_before_quantum(task):
    queues[task.level].push_back_when_woken(task)
```

MLFQ 的关键参数包括层数、每层时间片、降级规则、周期性 boost。boost 用来防止低层任务永久饥饿：每隔一段时间，把所有 runnable 任务提升回高层。实现上要记录任务当前层级、已用时间、最近 boost epoch。

公平调度：从 CFS 模型到 EEVDF。公平调度不只问“谁先来”，而是问每个 runnable 任务已经得到多少 CPU 服务。经典 CFS 模型给每个任务维护虚拟运行时间 `vruntime`：任务实际运行越久，`vruntime` 越大；权重越高，虚拟时间增长越慢。下面的伪代码是用于理解 CFS 核心思想的简化模型，不是当前 Linux 调度器的完整实现。

```
on_task_run(task, delta_exec):
    task.vruntime += delta_exec * NICE_0_WEIGHT / task.weight

pick_fair(rbtrees):
    return leftmost_node_by_vruntime(rbtrees)
```

若用红黑树保存 runnable 任务，插入、删除大约是 $O(\log n)$ ，取缓存的最左节点可接近 $O(1)$ 。这个模型解释了为什么 nice 值不是简单固定优先级：它影响获得 CPU 的比例，而不是让低优先级任务永远不运行。

Linux 从 6.6 起逐步把公平类的选择策略迁移到 EEVDF。EEVDF 仍使用虚拟时间核算服务量，但先用 lag 判断任务是否有资格运行，再在有资格的任务中选择虚拟截止期最早者。阅读 `kernel/sched/fair.c` 时，应把“CFS 的最小 `vruntime` 教学模型”和“当前 EEVDF 的 eligibility + virtual deadline 决策”分开。

deadline 调度与准入控制。实时或准实时任务还会声明运行时间和截止时间。例如任务说“每 20ms 需要 3ms CPU”。调度器若盲目接受所有请求，总利用率超过 100 后必然 miss deadline。因此 deadline 调度需要准入控制。

```
admit(task):
    new_util = total_runtime_over_period + task.runtime / task.period
    if new_util > CPU_CAPACITY:
        return -EBUSY
    add_to_deadline_class(task)
    return OK

pick_deadline():
    return task with earliest absolute_deadline
```

EDF 在单 CPU 理想模型中有很强性质，但真实系统还有抢占成本、临界区、缓存失效、中断关闭和共享资源阻塞。理论可调度不等于工程一定按时，测试必须记录 deadline miss、最大关抢占时间和锁等待。

调度指标。调度器优化必须有指标。常用指标包括平均响应时间、p95/p99 延迟、吞吐、上下文切换次数、公平误差、迁移次数、cache miss、deadline miss 和能耗。只报告“更快”没有意义。

指标	说明	常见误判
on-CPU time	真正执行指令的时间	把等待 I/O 误认为算法慢
runnable wait	可运行但没拿到 CPU 的时间	把调度拥塞误认为锁死
context switches	切换次数和成本	只看次数，不看每次成本
migration count	跨 CPU 迁移	均衡改善但缓存局部性变差
fairness error	实得 CPU 与权重目标差距	平均公平掩盖尾延迟

真实系统为什么更复杂。真实通用 OS 还要处理多核、NUMA、CPU affinity、实时任务、cgroup 配额、负载均衡和能耗。多核调度不是简单地把一个队列给所有 CPU，因为全局队列会产生锁竞争；每 CPU runqueue 能降低竞争，但需要负载均衡。迁移任务可以平衡负载，却会丢失 cache/TLB 热状态。调度器必须在公平、吞吐、延

迟、局部性和能耗之间折中。

task 不是一个正在运行的函数。硬件正在执行的是寄存器、RIP、RSP、RFLAGS 和页表上下文。内核把这组执行状态包装成 task，再让 task 指向地址空间、文件表、信号状态、凭据、namespace 和 cgroup。task 是可调度实体，不是进程所有资源的简单大盒子。

字段类别	保存什么	为什么重要
执行现场	内核栈、保存寄存器、返回路	被切走后还能从同一点继续
调度状态	state、policy、prio、vruntime、on_rq	判断能否运行、怎样排序、是否已入队
地址空间	mm、active_mm	进程切换可能要切页表和 TLB
资源指针	files、fs、signal、sighand	fork、exec、exit 要复制、共享或释放
安全身份	cred、namespace、cgroup	权限、限额和可见性都依赖它
亲和性	allowed CPUs、last CPU、NUMA 统计	负载均衡不能只看队列长度

state 和 on_rq 要分开理解。state 说明任务是 runnable、sleeping、stopped 还是 exiting；on_rq 说明它是否已经挂到某个 runqueue。一个 runnable 任务没有入队，会永远等不到 CPU；一个 sleeping 任务留在 runqueue，会被错误调度。调度器的大量锁和状态检查，就是为了让这两本账一致。

阻塞系统调用怎样离开 CPU。阻塞不是“函数停住”。内核要把当前任务从可运行集合移走，把它挂到等待队列，并保证事件到达时能找回它。以等待 pipe 或 socket 可读为例：

```
blocking_read(obj):
    lock(obj.lock)
    while !data_available(obj):
        prepare_to_wait(obj.read_waitq, current)
        current.state = TASK_INTERRUPTIBLE
        unlock(obj.lock)
        schedule()
        lock(obj.lock)
        finish_wait(obj.read_waitq, current)
        if signal_pending(current):
            unlock(obj.lock)
            return -EINTR
    copy_data_to_user()
    unlock(obj.lock)
    return bytes
```

检查谓词和入等待队列必须受同一把锁保护。若先检查发现没有数据，还没入队时生产者写入并唤醒，唤醒会丢失；随后当前任务睡下，就形成 lost wakeup。谓词循环也不是模板写法：任务可能被信号唤醒，可能被虚假唤醒，也可能多个等待者竞争同一份数据。

把这个错误具体走一遍，问题会更清楚。假设线程 A 正在 read(pipefd)，线程 B 稍后 write(pipefd)。错误实现若按“先看 pipe 为空，再准备睡眠”两步分开做，中间没有锁保护，就会出现下面的时序：

时刻	线程 A	线程 B 或内核状态
----	------	------------

1	A 检查 pipe, 发现没有数据	wait queue 仍为空
2	A 还没把自己挂入 wait queue	B 写入数据, 调用 wakeup, 但没有等待者可唤醒
3	A 把自己设为 sleeping 并调用 <code>schedule</code>	pipe 已经有数据, 但没有新的 wakeup 会来
4	CPU 去运行别的任务	A 可能永久睡眠, 除非信号、超时或其他写入再次触发事件

正确实现把“检查条件、登记等待者、改变 task 状态”放在同一个临界区里。这样生产者要么在 A 入队前看见锁被持有, 等 A 完成睡眠准备后再写入并唤醒; 要么在 A 检查条件前已经写入数据, A 重新检查时发现已有数据而不会睡下。这里的锁不是为了保护几行代码好看, 而是在保护一个跨 CPU 的事实: 若条件已经满足, 等待者不能睡丢; 若等待者已经睡下, 满足条件的一方必须能找到它。

还要注意, `schedule()` 返回不表示数据一定已经到手。A 可能因为信号返回, 可能因为超时返回, 可能被广播唤醒后发现另一个线程先取走了数据。因此睡眠后必须重新拿锁、重新检查谓词。许多初学者会把 `wake_up` 当成“把函数从睡眠点继续执行”, 这不准确。真实过程是: `wakeup` 修改任务状态并把它放回 runqueue; 真正执行要等调度器选择它; 继续执行后还要重新证明条件仍然成立。

唤醒怎样进入 runqueue。唤醒不等于立即运行。唤醒方通常只把等待任务改回 runnable, 放到目标 CPU 的 runqueue; 是否抢占当前任务, 要看调度类、优先级、抢占状态和 CPU 位置。

```

wake_up(waitq):
    lock(waitq.lock)
    for task in waitq:
        if task.wait_condition_is_satisfied:
            remove_from_waitq(task)
            task.state = TASK_RUNNING
            enqueue_task(select_runqueue(task), task)
            if task_should_preempt_current(task):
                set_need_resched(target_cpu)
    unlock(waitq.lock)

```

若目标 CPU 正在用户态运行, `need_resched` 常在返回用户态、定时器中断返回或显式调度点处理。若目标 CPU 是另一个核心, 内核可能发 IPI 让它尽快重新调度。这样看, `wakeup` 的成本包括等待队列锁、runqueue 锁、跨 CPU 通知和 cache line 迁移。

再用一个具体的 socket 接收案例串起来。线程 A 在 CPU0 上调用 `recv`, 接收队列为空, 于是它把等待原因记成“等 socket receive queue 非空”, 挂到 socket 的等待队列, 然后从 CPU0 的 runqueue 中消失。此时 A 没有“暂停在 CPU 里”, 它只是一个 task 对象, 保存了内核栈、寄存器现场、等待队列链接和状态。CPU0 可以去运行线程 C。

随后网卡在 CPU1 上产生中断。驱动或 NAPI poll 把包交给协议栈, 协议栈找到目标 socket, 把数据放进 receive queue, 然后调用 `wakeup`。wakeup 要做的不是直接跳回线程 A 的用户栈, 而是把 A 从 socket wait queue 摘掉, 改成 runnable, 选择一个目标 runqueue。若 A 上次在 CPU0 运行且 cache/TLB 仍有局部性, 调度器可能倾向放回 CPU0; 若 CPU0 很忙而 CPU1 空闲, 也可能迁移。若迁移, A 将来恢复时会付出 cache 冷启动成本。

阶段	状态变化	读者要抓住的边界
<code>recv</code> 阻塞	task 从 runqueue 移到 socket wait queue	到它不再竞争 CPU, 但内核必须记住等待条件

包到达	数据进入 socket receive queue	条件变为真，但等待线程还没有运行
wakeup	task 变成 runnable，进入某个 runqueue	runnable 只表示有资格运行，不表示已经运行
调度选择	CPU 在安全点切换到该 task	可能立即抢占，也可能等当前任务返回或时间片结束
重新检查	recv 继续后再次检查 receive queue	多等待者、信号和竞态要求谓词循环

这个案例解释了为什么排查“请求卡住”不能只看线程是不是 sleeping。若线程还在 wait queue，问题可能是事件没发生、唤醒路径漏了、等待条件写错；若线程已经 runnable 但很久没运行，问题在调度拥塞；若线程运行后又睡回去，可能是虚假唤醒、条件被别的线程抢走，或协议状态不满足。状态分清楚，才有办法定位。

抢占点和不可抢占区。定时器中断可以打断用户态程序，但内核不能在任意位置无条件切走当前任务。持有自旋锁、关闭中断、正在修改 runqueue、正在操作每 CPU 数据时，随意抢占会破坏不变量。内核用 preempt_count、中断状态和锁规则标记这些区域。

```
timer_interrupt:
    account_runtime(current)
    if current->timeslice_expired:
        set_need_resched(current)
    if returning_to_user and need_resched:
        schedule()
    else if kernel_preemptible and preempt_count == 0:
        schedule()
```

实时系统关心的不是平均时间片，而是最长不可抢占区、最长关中断区和最高优先级任务被低优先级持锁路径阻塞的时间。一个系统平均响应很好，也可能因为某条持锁路径过长而出现不可接受的尾延迟。

fork、exec、wait 和调度器的连接。fork 创建的新任务不能在半初始化状态进入 runqueue。内核必须先准备内核栈、保存的用户寄存器、调度实体、地址空间引用、文件表引用、信号状态和 PID，再让调度器看到它。子进程第一次运行时不是从程序开头执行，而是从系统调用返回路径回到用户态，只是返回值寄存器被设置为 0。

```
fork scheduling boundary:
    allocate child task
    prepare child saved registers
    copy or share mm/files/signal/cred
    allocate pid and link parent-child relation
    enqueue child as runnable
    parent returns child pid
    child later returns 0 from copied frame
```

exec 成功后仍是同一个 task 被调度，但地址空间、用户栈、入口 RIP、部分信号处理和 close-on-exec fd 改变。wait 要求已退出子进程保留 zombie 壳，直到父进程读取退出码。调度器只负责不再运行 zombie；进程管理还要保证退出状态只被回收一次，最后一个引用释放 task。

源码阅读入口。进程创建看 kernel/fork.c 的 copy_process；退出和回收看 kernel/exit.c 的 do_exit、release_task；调度主干看 kernel/sched/core.c 的 __schedule、try_to_wake_up；公平类看 kernel/sched/fair.c 的 enqueue/dequeue、vruntime 和负载均衡。阅读时把每一步标成四类动作：改 task 状态、改 runqueue、改资源引用、通知等待者。

本章压缩进来的并发与锁。调度章不能只到“谁获得 CPU”为止。只要两个线程可能同时碰同一份状态，就必须说明同步对象、等待对象和失败恢复。锁、futex、信号、IPC 和死锁原本可以展开成很多章，但主线里先抓住一个统一问题：某个线程为什么不能继续，它在等谁，谁有资格改变条件，条件改变后谁负责唤醒。

机制	它保存的等待事实	常见错误
mutex	持有者是谁，等待者排在哪个队列	忘记在循环中复查条件，或异常路径漏解锁
futex	用户态字值和内核等待队列的绑定	只看内核队列，不验证用户态谓词是否仍成立
condition variable	等待哪个共享状态变真	把 wakeup 当成状态已经满足
signal	异步事件和可中断睡眠的交汇点	忽略 EINTR，把部分完成误当失败
pipe/socket	缓冲区从空到非空、从满到非满	读写端关闭后仍然等待不可能发生的事件

锁的规则也必须落到调度状态。自旋锁适合短临界区，持锁时不能睡眠；mutex 可睡眠，适合较长临界区；读写锁提高读多写少场景的并发，但写者饥饿和升级死锁要单独处理；RCU 让读侧几乎不阻塞，但释放对象必须等 grace period，不能在最后一个指针消失时立即 free。这些机制的差别，不是 API 名字，而是“持有期间能不能调度、等待者在哪里、对象何时真正可释放”。

```
safe_wait(lock, waitq, condition):
    lock(lock)
    while !condition():
        prepare_to_wait(waitq, current)
        unlock(lock)
        schedule()
        lock(lock)
    finish_wait(waitq, current)
    unlock(lock)
```

这段伪代码压缩了等待路径的核心不变量：检查条件和入队必须受同一把锁保护；睡眠前要释放保护业务状态的锁；醒来后要重新加锁并复查条件；退出等待时要清理 wait queue 链接。若先检查条件再入队，中间事件可能已经发生并唤醒了空队列，线程随后睡下就永远没人叫醒。若醒来后不复查，多等待者会把同一个资源消费两次。

死锁分析也要用对象图，不要只说“锁顺序错了”。把每个线程正在持有的锁、正在等待的锁、是否可被信号打断、是否持有不可睡眠锁都列出来。如果图中出现环，而且环上没有超时、取消或外部释放路径，系统就无法自行前进。lockdep 一类工具本质上是在运行时收集锁类顺序，提前发现可能形成环的获取顺序。

调度与并发章的最低验收：给任一阻塞点，都能写出等待谓词、等待队列、唤醒者、是否可中断、被唤醒后的复查动作、持锁规则和失败清理路径。缺一项，代码即使在正常路径能跑，也没有可证明的并发语义。

4.4 测试

调度测试至少覆盖三类情况：多个 runnable 进程应轮转推进；blocked 进程不应获得 tick；terminated 进程不应重新出现。还要检查没有 runnable 进程时，调度器返回 idle，而不是访问空队列。

- fork 后父子进程关系可解释，父进程能回收子进程退出状态。
- exec 后地址空间替换，但 PID、部分 fd 和进程身份按规则保留。
- 线程阻塞后从 runqueue 消失，唤醒后回到 runnable 集合。
- 时间片结束只改变调度状态，不应破坏进程资源。
- 没有 runnable 线程时进入 idle，而不是忙等消耗 CPU。
- 优先级或 aging 不得让低优先级任务永久饥饿，除非策略明确允许。

后续可以加入优先级、deadline、affinity、负载均衡和 cgroup 配额，但必须保留基础回归。任何复杂策略都不能让 blocked 或 terminated 任务被错误调度。

进程与调度实现深讲：从 task 到 switch

前面的章节已经讲了调度算法。本节把进程和调度器按内核实现拆开：进程对象有哪些字段，fork 如何复制资源，exec 如何替换地址空间，wait 如何回收子进程，调度器如何维护 runqueue，睡眠唤醒如何避免丢事件，上下文切换到底切了什么。

进程不是一个结构体，而是一组引用。

Linux 中常用 `task_struct` 表示可调度实体。它不把所有资源都内嵌进去，而是引用地址空间、文件表、信号处理、凭据、命名空间、cgroup、调度实体等对象。这样线程可以共享一部分资源，fork 可以复制或共享不同对象，exec 可以替换地址空间但保留 PID 和部分文件描述符。

对象	回答的问题	生命周期事件
<code>task_struct</code>	这个可调度实体是谁、处于什么状态	fork 创建、exit 释放
<code>mm_struct</code>	用户地址空间是什么	fork COW、exec 替换、exit 释放
<code>files_struct</code>	fd 表指向哪些打开文件	fork 共享或复制、exec close-on-exec
<code>fs_struct</code>	cwd、root、umask 等进程文件系统上下文	chdir/chroot/fork
<code>sighand</code> <code>cred</code>	信号处理函数和 mask uid、gid、capability、安全标签	fork/exec/signal setuid、exec、LSM 检查
<code>sched_entity</code>	调度账本	enqueue、dequeue、account runtime

源码入口可以从 `include/linux/sched.h`、`kernel/fork.c`、`kernel/exec.c`、`fs/exec.c` 看。阅读时要抓引用计数和共享边界：线程共享 `mm` 和 `files` 时，一个线程关闭 fd 会影响同进程其他线程。

fork 的实现顺序。

fork 不是复制一整个进程镜像。内核先分配新的 task，复制或共享资源，建立父子关系，把子任务放入 runnable 集合，最后返回父子两个不同返回值。若中间失败，必须按已经成功的步骤逆序回滚。

```
fork sketch:
  allocate task_struct and kernel stack
  copy scheduler attributes
  copy or share mm_struct with COW PTEs
  copy or share files_struct
  copy signal and credential state
  assign pid
  link into parent children list
```



```
make child runnable
return child pid to parent, 0 to child
```

最容易错的是“什么时候可见”。子任务在所有必要状态初始化完成前不能进入 runqueue，否则另一个 CPU 可能立刻调度它，看到半初始化对象。父子关系也要在锁保护下建立，否则 wait 路径可能漏掉子进程。

fork 的 COW 页表细节。

fork 复制页表时，匿名可写页通常不复制物理页，而是父子 PTE 都改成只读并标记 COW。物理页引用计数增加。任一方写入时，触发写保护缺页，内核复制新页，再把写入方 PTE 改为可写。

```
fork COW:
  for each writable anonymous PTE:
    clear write bit in parent PTE
    install read-only PTE in child
    mark both as COW
    increment page refcount
  flush parent TLB entries if permissions were tightened

write fault:
  if page refcount == 1:
    make PTE writable
  else:
    allocate new page
    copy old page contents
    decrement old page refcount
    map new page writable
```

注意父进程 PTE 也要收紧权限，否则父进程写入不会 fault，就会直接修改共享页，破坏 fork 语义。权限收紧后还要处理 TLB，否则 CPU 可能继续使用旧的 writable 翻译。

exec 的关键承诺。

exec 成功后，当前进程身份大体保留，但用户地址空间被新程序替换。PID 不变，部分 fd 保留，设置了 close-on-exec 的 fd 关闭，信号处理按规则重置，凭据可能因 setuid 或文件 capability 改变。

```
execve sketch:
  copy argv/envp from user memory
  open executable and check permissions
  identify binary format
  create new mm_struct
  map program segments
  map interpreter if dynamically linked
  create user stack with argc/argv/envp/auxv
  apply credential transitions
  commit new mm atomically
  start at entry point
```

exec 失败要小心。理想情况下，在 commit 新地址空间前失败，旧程序仍能收到错误返回并继续运行；一旦已经不可逆地销毁旧地址空间，就只能终止进程。成熟内核会尽量把可能失败的检查和分配放在 commit 之前。

用户栈的构造。

新程序启动时不是 C 的 main 被内核直接调用。内核把初始用户栈构造成 ABI 规定的布局，包括 argc、argv 指针数组、envp 指针数组、auxiliary vector 和字符串数据。入口点通常是动态链接器或 _start。

```
initial user stack high to low:
```



```

strings for argv and envp
alignment padding
auxiliary vector entries
NULL
envp pointers
NULL
argv pointers
argc

```

`auxv` 给用户态传递页大小、随机数地址、程序头信息、`vDSO` 入口等。动态链接器用这些信息装载共享库、处理重定位和准备 `libc`。OS 的 `exec` 不是“把文件读到内存跳过去”这么简单，它要建立完整 ABI 环境。

`wait` 和 `zombie` 的意义。

子进程退出后不能立刻完全消失，因为父进程还可能调用 `wait` 获取退出状态和资源使用信息。于是退出进程进入 `zombie` 状态，保留 PID、退出码和少量统计；父进程 `wait` 后，内核释放最后对象。

```

exit:
    close files
    release mm
    record exit_code
    notify parent
    become ZOMBIE
    schedule away forever

wait:
    find child in ZOMBIE state
    copy exit status to parent
    unlink child from parent list
    free final task reference

```

若父进程不 `wait`，`zombie` 会留下。若父进程先退出，子进程会被 `reparent` 给 `init` 或 `subreaper`。这里的设计保证退出状态不丢，同时避免已退出任务再次运行。

`runqueue` 的最小不变量。

每 CPU `runqueue` 至少要维护当前任务、`runnable` 任务集合、锁、调度时钟和统计。一个任务在同一时刻只能处于一个位置：某个 CPU 的 `current`，某个 `runqueue`，某个等待队列，或者退出路径。

- `RUNNING` 任务必须等于某个 CPU 的 `current`。
- `RUNNABLE` 任务必须在且只在一个 `runqueue` 中。
- `SLEEPING` 任务不能留在 `runqueue` 中。
- `ZOMBIE` 任务不能被调度器选中。
- 迁移任务时，源 `runqueue` 和目标 `runqueue` 的锁顺序必须固定。
- 改变任务状态和队列位置必须在同一临界区内完成。

调度 bug 常常不是 pick 算法错，而是不变量破坏：任务重复入队，睡眠任务未出队，唤醒时状态已经变化，迁移时两个 CPU 同时看到同一任务。

CFS 的核心数据结构。

公平调度类用 `sched_entity` 记录虚拟运行时间、权重、统计和树节点。`runqueue` 中的 CFS 队列按 `vruntime` 排序，最左边是当前最“亏”的任务，优先运行。

```
CFS account:
```

```

delta_exec = now - curr.exec_start
weighted = delta_exec * NICE_0_LOAD / curr.weight
curr.vruntime += weighted
update rq.min_vruntime

CFS pick:
next = leftmost entity in rb_tree
return task_of(next)

```

`min_vruntime` 是队列的基准。新唤醒任务不能简单设置为 0，否则会长期占便宜；也不能设置得太大，否则交互响应差。实际实现会把新实体放在相对 `min_vruntime` 合理的位置，并配合唤醒抢占规则。

红黑树为什么合适。

公平调度需要频繁插入、删除和取最小 `vruntime`。普通链表插入简单但查找最小慢；堆取最小快，但删除任意 `sleeping` 任务不方便；红黑树能在 $O(\log n)$ 完成插入、删除和按 `key` 排序。

结构	优点	缺点
FIFO 队列	$O(1)$ ，简单	不能表达公平权重和 <code>vruntime</code> 排序
优先级数组	固定优先级快	动态公平排序不自然
堆	取最小快	删除任意实体复杂
红黑树	排序、插入、删除平衡	常数成本高于队列

算法选择来自操作需求。调度器不是为了展示数据结构，而是因为任务会频繁睡眠、唤醒、迁移、改变权重，必须支持这些操作。

睡眠唤醒的无丢失顺序。

等待队列正确性的核心是同一把锁保护“条件检查”和“入队睡眠”。否则唤醒可能发生在检查之后、入队之前，等待者永远睡下去。

```

correct wait:
lock(obj.lock)
while !condition:
    prepare_to_wait(waitq, current, SLEEPING)
    unlock(obj.lock)
    schedule()
    lock(obj.lock)
finish_wait(waitq, current)
unlock(obj.lock)

waker:
lock(obj.lock)
change condition
wake_up(waitq)
unlock(obj.lock)

```

很多真实内核代码会用更复杂的 `helper`，但它们都在维护这个原则。条件变量、`pipe`、`socket`、`futex`、`completion`、`wait event` 都逃不开同一条规则。

上下文切换切的是什么。

上下文切换分成调度器层和架构层。调度器选择 `next`，更新 `current`、统计、状态；架构层保存 `callee-saved` 寄存器、栈指针和必要控制状态，切到 `next` 的内核栈，再返回到 `next` 之前停下的位置。

```

schedule:
prev = current
next = pick_next_task(rq)

```

```

if prev != next:
    rq.current = next
    context_switch(prev, next)

context_switch:
    switch address space if needed
    switch kernel stack
    save prev callee-saved registers
    restore next callee-saved registers
    return as next task

```

这里有一个反直觉点：`context_switch` 返回时，执行流已经变成 `next` 任务。`prev` 将来被调度回来时，会从它当初离开的地方继续返回。这就是线程切换能伪装成普通函数返回的原因。

地址空间切换和 lazy TLB。

若两个线程共享同一个 `mm_struct`，切换时不需要换页表根。若切到不同进程，通常要切换页表根或地址空间标识。切换页表会影响 TLB；x86-64 PCID 可以减少全量刷新。

```

switch_mm(prev_mm, next_mm):
    if prev_mm == next_mm:
        keep current page table root
    else:
        load next page table root with PCID when enabled
        ensure stale translations cannot be used

```

内核线程可能没有自己的用户地址空间，常借用上一个进程的 `mm` 或使用特殊内核地址空间。lazy TLB 这类优化的目标，是减少无意义的页表切换和 TLB 刷新。

负载均衡和调度域。

多核系统不是一个平面。SMT sibling、同一核心、同一 LLC、同一 NUMA node、跨 socket 的迁移成本不同。调度器通常建立调度域，从近到远做负载均衡。

```

load balance levels:
    SMT siblings
    cores sharing LLC
    CPUs in same NUMA node
    CPUs across NUMA nodes

```

迁移一个任务可能减少 runnable wait，却损失 cache 热状态并增加远端内存访问。x86-64 服务器和工作站上，调度器要估计 SMT sibling、共享 LLC、NUMA node、跨 socket 迁移、turbo 预算和功耗封顶。任务放错位置会让锁竞争、cache miss、远端内存访问和尾延迟一起变差。

cgroup CPU 配额。

容器和服务隔离需要限制一组任务的 CPU 使用。cgroup CPU controller 可以按周期给 group 分配 runtime budget。预算耗尽后，该组任务被 throttled，直到下个周期补充预算。

```

cgroup quota:
    period = 100ms
    quota = 20ms
    tasks in group may run total 20ms per period
    after budget exhausted:
        throttle group
    at next period:
        refill runtime and unthrottle

```

这会产生典型现象：容器内 CPU 使用看似没满，但请求出现周期性延迟尖峰，因

为 group 被配额限流。性能分析必须看 cgroup throttling 统计，而不是只看宿主机总体 CPU。

调度实现测试。

- fork 后子任务不能在初始化完成前进入 runqueue。
- COW fork 后父子任一方写入都不能修改另一方可见内容。
- exec 失败路径不得破坏旧程序继续运行的必要状态。
- wait 必须能回收 zombie，并且不能回收非子进程。
- 睡眠和唤醒并发时不能 lost wakeup。
- 迁移任务时不能同时出现在两个 CPU 的 runqueue。
- cgroup quota 耗尽时 group 被 throttle，周期补充后能恢复。
- 上下文切换后 callee-saved 寄存器、栈和地址空间必须正确。

第 5 章 系统调用、权限与内核边界

5.1 原理

系统调用是用户态请求内核代为操作受保护资源的入口。用户程序不能直接修改页表、调度队列、磁盘控制器或其他进程的状态；它只能把请求和参数交给内核。CPU 通过特权级、异常入口和约定寄存器完成控制权转移，内核再检查参数、权限和对象状态。

系统调用不是普通函数调用。普通函数调用仍在同一特权级、同一地址空间和同一信任边界里执行；系统调用会从用户态进入内核态，切换到受控入口，使用内核栈或内核上下文，检查用户指针，并在返回前恢复用户态。这个边界使得内核可以拒绝非法请求，也使系统调用有固定开销。

特权到底保护了什么。特权级保护的不是一个抽象名分，而是一组具体硬件状态。用户态不能改页表根，不能关中断，不能改中断向量，不能写任意控制寄存器，不能访问只映射给内核的 MMIO，不能把自己的代码标成内核入口。CPU 在执行敏感指令、地址翻译和异常返回时都会检查当前模式。

受保护对象	为什么危险	合法路径
页表根/地址空间控制	可映射任意物理页，读取内核或其他进程	内核内存管理代码
中断使能和向量表	可阻止调度或劫持异常入口	内核入口和中断子系统
设备 MMIO 寄存器	可重置设备、窃取数据、破坏存储	驱动通过 fd/ioctl/sysfs 等受控接口
特权返回状态	可伪造内核态返回或跳转地址	低级 entry/return 代码检查
I/O 权限和端口	可直接访问平台设备	内核 I/O 权限管理或驱动

x86-64 用 Ring 0 和 Ring 3 表达常见用户/内核边界。Ring 3 运行普通用户程序，Ring 0 运行内核；CPU 在执行敏感指令、访问 supervisor-only 页、装载控制寄存器、返回低特权级时检查 CPL 和相关权限位。

层级	角色	系统调用入口
Ring 3	用户程序	执行 <code>syscall</code> 请求进入内核
Ring 0	内核	由 IA32_LSTAR MSR 指向的入口接管
VMX root	hypervisor	可选择拦截 guest 的敏感事件

特权边界的验收问题是：用户程序是否能绕过内核直接改变受保护状态？若答案是能，系统调用实现得再漂亮也没有隔离。

为什么进入内核要换栈。用户栈由用户程序控制。它可能没有映射，可能只读，可能指向恶意构造的数据，也可能在系统调用前被故意放到临界边界。内核不能在用户栈上保存自己的返回地址、锁状态和局部变量，否则用户就能破坏内核控制流。因此跨特权级入口必须切到可信内核栈或受控异常栈。

```
on syscall from user:
hardware saves minimal return state
entry code switches from user rsp to current.kernel_stack
entry code saves registers into pt_regs
```

C syscall code runs on kernel stack

每个线程通常有自己的内核栈。线程在用户态运行时使用用户栈；进入内核后使用内核栈；若阻塞在系统调用中，内核栈保存它在内核里的等待位置。调度器切换线程时，既要能恢复用户态 trap frame，也要能恢复睡眠中的内核调用链。

函数调用、库调用和系统调用的三层区别。很多初学者把 `printf`、`write`、系统调用混在一起。实际上它们处在三层边界上。普通函数调用只改变当前进程内部的 RIP 和栈；库调用仍在用户态，只是封装了更复杂逻辑；系统调用才跨越 CPU 特权边界进入内核。

调用类型	发生在哪里	边界含义
普通函数调用	同一进程、同一特权级	调用者和被调者互相信任同一地址空间
libc/API 调用	用户态库代码	可做缓冲、格式化、兼容处理，但不能直接改内核对象
系统调用	用户态陷入内核态	CPU 切换特权，内核检查权限和对象状态

`printf("x")` 通常先把格式化结果写到用户态 `stdio buffer`；`buffer` 满、遇到换行或显式 `flush` 时，`libc` 可能调用 `write`；`write wrapper` 把系统调用号和参数放进寄存器，执行 `syscall`。内核并不知道 `printf` 的格式化细节，它只看见 `fd`、用户 `buffer` 和长度。

```
printf("hello\n")
-> libc formats into FILE buffer
-> libc write wrapper
-> CPU syscall instruction
-> kernel sys_write
-> fd table, file object, device/file path
```

这层区分能解释许多现象：程序崩溃时 `stdio buffer` 可能没写出；`write` 返回成功不等于数据落盘；系统调用开销来自特权切换、参数检查和内核路径，不是 C 函数调用本身。

系统调用入口的硬件动作逐步展开。以 x86-64 的 `syscall` 为例，用户态执行指令后，CPU 不会查 C 函数表。它读取预先由内核配置的 MSR，切换到内核入口地址，保存返回地址和标志的一部分，改变特权相关状态。随后汇编入口完成 `swaps`、切换内核栈、构造 `pt_regs`。

```
user mode before syscall:
  rax = syscall number
  rdi, rsi, rdx, r10, r8, r9 = args
  rip = address after syscall
  rsp = user stack

hardware/syscall entry:
  save return rip into rcx
  save flags into r11
  load kernel entry rip from MSR
  enter privileged execution path

low-level kernel entry:
  swaps to kernel per-cpu base
  load current thread kernel stack
  save user registers into pt_regs
  call C syscall dispatcher
```


不同架构寄存器名和指令不同，但结构类似：约定参数位置，触发陷入，保存返回状态，切换到可信上下文，分发表项，返回前处理信号/抢占/审计。把这条路径记清，后面看 entry 汇编就不会把它当成神秘代码。

返回用户态比进入内核更危险。进入内核时，内核获得控制权；返回用户态时，内核要把控制权交还给不可信程序。返回前必须保证：目标 RIP 是用户地址，返回 CPL 是 Ring 3，用户栈指针不会让内核继续使用，挂起信号和抢占已处理，调试/审计状态一致，寄存器中不能泄露内核敏感数据。

```
return_to_user(regs):
    if need_resched: schedule()
    if signal_pending: deliver_signal(regs)
    if tracing_or_audit: syscall_exit_work(regs)
    sanitize_user_return_state(regs)
    restore_user_registers(regs)
    sysretq or iretq
```

许多安全漏洞发生在返回路径：错误使用快速返回指令、没有正确验证用户返回地址、泄露内核寄存器内容、返回前忘记处理单步/信号状态。系统调用不是“进入内核执行函数再回来”这么简单，入口和出口都是安全边界。

参数复制是系统调用安全性的核心。用户态传入的指针不能被内核直接相信，因为它可能无效、不可读、不可写、跨页、在检查后被另一个线程改掉，或指向内核不应访问的区域。内核必须使用 `copy_from_user/copy_to_user` 一类机制，把用户内存当作不可信输入处理。

用户指针为什么不能直接解引用。用户指针看起来只是一个地址，但它属于用户地址空间的契约，不属于内核可信内存。直接解引用会产生四类问题。第一，地址可能无效，内核访问会 `fault`。第二，地址可能指向用户不可读/不可写区域。第三，地址所在页可能跨页，前半可读后半不可读。第四，另一个线程可能在检查之后修改数据，造成 TOCTOU。

```
bad:
    // directly trusts user pointer
    header = *(struct Header*)user_ptr
    use(header.length)

better:
    header = copy_struct_from_user(user_ptr)
    validate(header.length)
    data = copy_bytes_from_user(user_ptr + sizeof(header), header.length)
```

`copy` 不是低效保守，而是把不可信输入转成内核拥有的稳定快照。对于大 I/O，内核可以 `pin` 用户页或使用零拷贝，但那会引入页生命周期、DMA 映射和 `completion` 责任，不能简单理解成“直接用用户指针”。

EFAULT 是如何产生的。当内核复制用户数据时，用户页可能不存在或权限不允许。若普通内核代码访问坏地址直接 `fault`，通常是内核 bug；但 `copy_from_user` 的 `fault` 是预期错误路径。内核通过异常表或等价机制告诉 `page fault handler`：如果 `fault` 发生在这段用户复制指令上，不要 `panic`，而是跳到 `fixup` 路径返回失败。

```
copy loop instruction at address A:
    load byte from user pointer
    store byte to kernel buffer

exception table:
    faulting instruction A -> fixup address F
```

```

page fault handler:
  if fault_pc found in exception table:
    regs.rip = fixup_address
    return
  else:
    handle normal kernel/user fault

```

这解释了为什么用户指针复制需要专用 API。它不仅做范围检查，还和低级异常处理配合，把硬件 fault 转换成系统调用错误码。教学 OS 也应该明确区分“内核自己访问坏地址”和“内核按用户请求复制时遇到坏用户页”。

权限检查不是一个 if。一次系统调用经常要穿过多层权限。以 `openat` 为例，路径解析要检查每级目录搜索权限；mount namespace 决定看到哪棵树；符号链接可能改变目标；LSM 可在路径或 inode 层拒绝；文件 mode、ACL、capability 决定读写执行；打开 flags 决定是否创建、截断、追加；资源限制决定还能否分配 fd。

```

openat(dirfd, path, flags):
  resolve dirfd in current fd table
  walk path under mount namespace
  check execute/search permission on directories
  handle symlink policy and flags
  get stable inode/dentry reference
  check DAC/ACL/capability
  call LSM hooks
  create file object with open flags
  allocate fd in current process fd table

```

这也是为什么错误码能提供信息。`ENOENT`、`ENOTDIR`、`EACCES`、`EPERM`、`ELoop`、`EMFILE` 分别表示不同层次失败。把所有失败都写成“权限不够”会掩盖状态机。

root、capability 和 namespace 的演进。早期 Unix 常把 uid 0 作为全能 root。现代系统把 root 权限拆成 capability，例如 `CAP_NET_ADMIN`、`CAP_SYS_ADMIN`、`CAP_DAC_OVERRIDE`。namespace 又让“这个 capability 对哪个对象集合有效”成为问题。容器内 root 不应自动拥有宿主全部权限。

机制	解决的问题	新复杂性
uid/gid/mode capability	简单主体和文件权限 把 root 权限拆小	粒度粗，root 过大 哪个路径检查哪个 capability 要准确
namespace	隔离进程看到的资源视图	capability 必须相对 user namespace 解释
LSM seccomp	策略化强制访问控制 收缩系统调用面	hook 覆盖面和策略复杂度 只看 syscall/参数，难懂对象语义

权限系统的本质是最小授权。内核应让调用者只获得完成任务所需能力；任何跳过对象所属 namespace 的 capability 检查，都可能把容器内权限放大成宿主权限。

权限不只有文件 mode 位。一个请求是否允许，可能取决于用户 ID、组 ID、capability、namespace、cgroup、seccomp、LSM、安全标签、mount 选项、fd 打开模式、对象引用和当前进程状态。OS 学习中的常见错误，是把“我是这个进程”误以为“我拥有所有资源”。

错误码也是契约。`EACCES` 表示权限不足，`ENOENT` 表示对象不存在，`EBADF` 表示 fd 无效，`EFAULT` 表示用户指针不可访问，`EINTR` 表示系统调用被信号打断，`EAGAIN` 表示非阻塞对象暂时不能推进。高质量 OS 程序必须按错误码分支，而不是把所有

失败都写成 fatal。

5.2 实践

分析一个系统调用时，按下面顺序写报告：

1. 用户态传入什么：整数、fd、地址、长度、flags、路径字符串。
2. 内核查找什么对象：进程表、fd 表、路径、inode、socket、VMA。
3. 检查哪些权限：对象存在、打开模式、读写执行、用户指针、资源限制。
4. 改变哪些状态：offset、buffer、引用计数、队列、页表、信号状态。
5. 返回什么：字节数、对象句柄、0/-1、errno、部分完成数量。
6. 哪些错误可重试，哪些错误必须失败，哪些错误表示调用者 bug。

以 `read(fd, buf, len)` 为例。内核先用 fd 查进程 fd 表，确认对象可读；再检查用户 buffer 是否可写；然后根据对象类型走普通文件、pipe、socket 或设备路径。普通文件可能从 page cache 复制，pipe 可能等待缓冲区，socket 可能返回当前已有字节。返回值为正时表示读取字节数，返回 0 可能是 EOF，返回 -1 时 errno 才解释失败原因。

以 `execve(path, argv, envp)` 为例。它不是创建新进程，而是在当前进程中替换用户态程序映像。PID 通常保持不变，地址空间被替换，用户栈重新构造，信号处理和 fd 状态按规则继承或关闭。若把 exec 理解成“启动新进程”，就无法解释 shell 中 fork+exec 的组合。

5.3 算法与实现分析

系统调用分发。系统调用入口通常把系统调用号和参数放在约定寄存器中。内核入口保存用户上下文，检查系统调用号范围，从系统调用表中找到处理函数，再把参数交给对应实现。返回时把结果或错误码放回约定位置。

```
syscall_entry(trap_frame):
    nr = trap_frame.regs[SYSCALL_NR]
    if nr >= syscall_table_size:
        return -ENOSYS
    current.user_context = trap_frame
    result = syscall_table[nr](trap_frame.args)
    trap_frame.return_value = result
    return_to_user(trap_frame)
```

这段算法的关键不是函数指针表，而是入口边界。内核必须确认当前线程确实来自用户态，不能让用户随便伪造内核上下文；必须处理信号、抢占、审计、seccomp 和 tracing；必须保证错误路径也能恢复到一致状态。

用户指针复制。用户指针复制要处理跨页、部分可访问和竞态。简单版本可以逐字节或逐页检查；高性能版本会利用硬件异常处理快速路径。下面定义的是系统调用层的教学包装函数 `copy_user_bytes`，不是 Linux `copy_from_user` 的真实函数签名。包装层把底层的部分复制结果转换成成功字节数或 `-EFAULT`，以便先讲清提交语义。

```
copy_user_bytes(dst, user_ptr, len):
    copied = 0
    while copied < len:
        page = translate_user_page(user_ptr + copied, READ)
        if page == fault:
            return -EFAULT
        n = min(bytes_until_page_end(user_ptr + copied), len - copied)
```

```
memcpy(dst + copied, kernel_map(page), n)
copied += n
return copied
```

真实内核还要处理 TOCTOU：检查之后用户线程可能修改内存。因此安全策略是把用户数据复制到内核缓冲区后再解析，不在解析过程中反复信任用户地址。路径字符串、数组参数和结构体参数都应遵守这个原则。

用 `write(fd, user_buf, 8192)` 看一次跨页复制。假设 `user_buf` 从页 A 的最后 100 字节开始，后面 8092 字节落在页 B 和页 C。页 A 可读，页 B 暂时没有 PTE 但 VMA 允许按需调入，页 C 不在任何 VMA 中。内核不能只检查起始地址，也不能在持有文件锁并已经修改文件状态后才发现页 C 不合法。稳妥路径通常是先把用户数据复制或 pin 到内核可控制的状态，再提交给文件或设备层。

片段	复制时发生什么	结果
页 A 尾部 100 字节	页表权限允许读，直接复制	<code>copied = 100</code>
页 B	PTE 不 present，缺页处理按 VMA 调入或分配页	成功后继续复制
页 C	找不到 VMA 或权限不允许读	教学包装层返回 <code>-EFAULT</code> ；真实 helper 报告未复制字节数

错误处理取决于提交点。如果系统调用还没有把任何字节交给文件对象，返回 `-EFAULT` 比较干净；如果某些接口允许部分完成，必须返回已经完成的字节数，并让调用者继续处理。关键是不能让内核对象处在“文件 offset 已经推进、page cache 半更新、但调用者只看到 `EFAULT`”的混乱状态。用户指针复制看似是内存问题，本质上是在保护系统调用的提交边界。

权限检查的顺序。权限检查要先定位对象，再检查主体是否有权操作对象，最后检查操作是否符合对象当前状态。以 `write(fd)` 为例：`fd` 必须存在；open file description 必须允许写；文件系统不能只读；用户 buffer 必须可读；长度不能超过限制；对象当前不能处于不允许写入的状态。

```
sys_write(fd, user_buf, len):
    file = fd_table_lookup(current, fd)
    if file == null: return -EBADF
    if !file.flags.can_write: return -EBADF
    if len > MAX_RW_COUNT: len = MAX_RW_COUNT
    kbuf = copy_user_bytes(user_buf, len)
    if kbuf.error: return -EFAULT
    return file.ops.write(file, kbuf, len)
```

不同错误码表达不同层次。`fd` 不存在和 `fd` 不可写都可能返回 `EBADF`；路径权限不足可能是 `EACCES`；文件系统只读可能是 `EROFS`；用户指针错误是 `EFAULT`。错误码越准确，调用者越能做正确恢复。

fork、exec、wait 的组合。Unix 风格进程创建不是一个巨大 `spawn`，而是 `fork`、`exec`、`wait` 的组合。`fork` 复制当前进程抽象；`exec` 替换当前进程映像；`wait` 回收子进程退出状态。这个设计让 shell 能在 `fork` 后、`exec` 前修改子进程 `fd`，例如重定向 `stdin/stdout`。

```
pid = fork()
if pid == 0:
    dup2(output_fd, STDOUT_FILENO)
    execve(program, argv, envp)
    _exit(127)
```

```
else:
    status = waitpid(pid)
```

算法不变量是：父进程和子进程在 `fork` 后分离推进；`exec` 成功后不会返回原用户程序；子进程退出后先变成 `zombie`，直到父进程 `wait` 取走退出状态。若没有 `zombie` 状态，父进程可能永远无法知道子进程怎样退出；若没有 `reaped`，进程表项会泄漏。

系统调用重启与信号。阻塞系统调用可能被信号打断。内核要决定返回 `EINTR`，还是在信号处理结束后自动重启系统调用。这个规则影响用户程序正确性：某些调用可安全重启，某些调用已经部分完成，不能假装从未发生。

```
blocking_read(fd, buf, len):
    while no_data_available(fd):
        ret = sleep_interruptible(fd.waitq)
        if ret == INTERRUPTED:
            if bytes_copied > 0:
                return bytes_copied
            if syscall_restartable(current):
                return -ERESTARTSYS
            return -EINTR
    return copy_available_data()
```

不变量是：已经对用户可见的部分完成不能被自动撤销。若 `read` 已经复制了 10 个字节，再收到信号，返回 10 比返回 `EINTR` 更能表达真实状态。调用者必须循环处理短读短写。

TOCTOU 与路径权限。`time-of-check to time-of-use` 是系统调用常见漏洞。比如先检查路径 `/tmp/a` 权限，再打开这个路径；中间攻击者把路径替换成符号链接，最终操作对象已经不是被检查对象。正确做法是把权限检查绑定到已经解析并持有引用的对象。

```
unsafe_open(path):
    if may_access_path(path):
        return open_path(path)

safe_open(path):
    inode = resolve_and_get_inode(path)
    if inode == null:
        return -ENOENT
    if !may_access_inode(current, inode):
        put_inode(inode)
        return -EACCES
    return make_file_from_inode(inode)
```

路径名是查找输入，不是稳定对象。`fd`、`inode` 引用和 `file` 引用才是内核能持有的对象身份。安全系统调用设计要尽量让检查和使用落在同一个对象引用上。

再看一个真实感更强的攻击时序。受害程序想安全打开用户指定目录下的 `data.txt`，先调用 `stat(path)` 检查它不是符号链接、权限也允许，然后再调用 `open(path)`。攻击者在两次调用之间把 `data.txt` 替换成指向 `/etc/shadow` 的符号链接。若内核或程序把“刚才检查过的路径字符串”当作对象身份，第二次打开的就可能是另一个对象。

时刻	动作	对象身份
1	受害者 <code>stat("/tmp/x/data.txt")</code>	检查的是旧 <code>dentry/inode</code>

2	攻击者 <code>rename</code> 替换路径分量	路径字符串不变，指向对象已变
3	受害者 <code>open("/tmp/x/data.txt")</code>	解析得到新 dentry/inode
4	程序继续写入	权限检查和使用对象不一致

更可靠的接口会缩短或消除这个窗口。例如先打开目录 `fd`，再用 `openat` 相对目录解析；使用 `O_NOFOLLOW` 拒绝末尾符号链接；在打开后对拿到的 `fd` 做 `fstat`，因为 `fd` 绑定的是已经打开的 `file` 对象；需要创建时用 `O_CREAT|O_EXCL` 把“检查不存在”和“创建”合成一个原子操作。原则不变：路径是名字，`fd/inode/file` 引用才是稳定对象。

5.4 测试

系统调用测试要验证边界，而不只是验证 happy path。

- 非法 `fd` 返回 `EBADF`，不应修改其他状态。
- 只读 `fd` 写入失败，错误应表达权限问题。
- 用户 `buffer` 不可访问时返回指针错误，不应让内核崩溃。
- 短读短写被调用者正确循环处理。
- `EINTR` 和 `EAGAIN` 被区分处理。
- `fork` 后父子进程共享 open file description 的 `offset`，独立 `open` 则 `offset` 分离。
- `exec` 后用户地址空间替换，但按规则保留或关闭 `fd`。
- `seccomp`、资源限制或 `capability` 缺失时，请求被拒绝且有可解释错误。

测试结论要写清“内核没有做什么”。例如权限失败时没有写入数据，非法指针没有部分污染内核对象，系统调用被信号打断后调用者能判断是否需要重试。负面保证是 OS 边界的核心。

本章压缩进来的边界账本。系统调用深讲材料可以展开很多低级细节：`entry` 汇编、异常修复表、`capability`、`LSM`、`seccomp`、`vDSO`、审计、`ptrace`、`restart block` 和资源限制。主线阅读时先把它们压成一张账本。任一系统调用都要能说清：入口寄存器保存在哪里，用户指针何时复制成内核快照，权限在哪个稳定对象上检查，失败是否已经改变状态，返回前是否处理信号、审计和抢占。

边界	必须回答的问题	错误后果
<code>entry/return</code>	用户 <code>RIP/RSP/flags</code> 和参数寄存器怎样保存与恢复	返回到错误特权级、泄露寄存器或跳过信号处理
用户内存	外层结构、内层指针和长度何时复制、何时 <code>pin</code> 页	<code>EFAULT</code> 变 <code>panic</code> ， <code>TOCTOU</code> ，整数溢出
对象解析	<code>fd</code> 、 <code>path</code> 、 <code>inode</code> 、 <code>socket</code> 、 <code>namespace</code> 是否已变成稳定引用	检查和使用不是同一对象
权限策略	<code>DAC</code> 、 <code>capability</code> 、 <code>LSM</code> 、 <code>seccomp</code> 、 <code>rlimit</code> 谁先失败	错误码混乱或绕过最小权限
提交点	哪一行之后状态对用户可见，失败如何回滚	<code>fd</code> 泄漏、引用计数泄漏、半初始化对象可见

这张账本能把看似分散的实现放回同一条路径。例如 `openat` 的提交点不是解析路径，也不是创建 `file` 对象，而是 `fd` 安装到进程 `fd` 表；在这之前失败必须释放 `file`、`dentry`、`inode` 和路径引用。在 `write` 中，返回正数可能表示部分完成；调用者不能把它和完整业务提交等同。在 `ioctl` 中，命令结构体版本、长度和 `padding` 都属于 ABI，不能因为“这是私有命令”就直接信任用户传入的内存布局。

5.5 源码级补充：entry_SYSCALL_64、capability、LSM 与 vDSO

x86-64 syscall 入口。x86-64 快速系统调用通常使用 `syscall/sysret`，入口地址和相关段选择子由 MSR 配置。进入内核后，低级入口要处理 `swapgs`、保存用户栈、切到内核栈、保存寄存器、检查 `tracing/seccomp`，再进入 C 层分发。

```
entry_SYSCALL_64:
    swapgs
    save user rsp into per-cpu area
    load kernel rsp
    push pt_regs
    call do_syscall_64
    check signals, reschedule, tracing
    restore regs
    sysretq or iretq
```

读 Linux 时看 `arch/x86/entry/entry_64.S`、`arch/x86/entry/common.c` 和 `arch/x86/entry/syscalls/syscall_64.tbl`。系统调用号表把 ABI 固定下来；`SYSCALL_DEFINE3(write,...)` 这类宏把 C 实现接入表项。不要把系统调用入口理解成普通函数指针调用，它先是特权级和寄存器协议。

access_ok 与 copy 用户。`access_ok` 只做范围和用户地址边界的快速判断，不保证后续复制一定成功。真正复制仍可能 `fault`，所以 `copy_from_user` 必须有异常修复表或等价机制。真实内核会为用户访问指令建立 `fixup`：访问失败时跳到错误处理地址，而不是让内核普通 `page fault` 变成 `panic`。

```
copy_from_user(dst, src, len):
    if !access_ok(src, len):
        return bytes_not_copied
    instrument_usercopy_before()
    n = raw_copy_from_user(dst, src, len)
    instrument_usercopy_after()
    return n
```

这里返回值语义也很重要：很多内核 `helper` 返回“未复制字节数”，上层再转换成 `EFAULT` 或短完成。写教学代码时可以返回错误码，但必须明确是否允许部分复制。

capable、ns_capable 和 user namespace。`capability` 判断不再只是“`uid 0`”。`capable(CAP_NET_ADMIN)` 检查当前主体是否有网络管理能力；`ns_capable(user_ns, cap)` 把能力放进某个 `user namespace` 里解释。容器依赖 `user namespace` 把容器内 `root` 映射成宿主上非特权身份。

```
if (!ns_capable(net->user_ns, CAP_NET_ADMIN))
    return -EPERM;
```

安全边界的常见错误是检查了全局 `capability`，却忘了对象所属 `namespace`；或者在 `user namespace` 中给了过宽能力，导致容器内操作影响宿主对象。读源码时要跟着对象走：这个 `network namespace`、`mount namespace`、`inode` 或 `cgroup` 属于哪个 `user namespace`。

LSM hook 和 seccomp。LSM 在 VFS、进程、socket、BPF 等路径插入 `security_*` hook。SELinux、AppArmor、BPF LSM 都可在这些点拒绝操作。seccomp 则在系统调用入口附近按 BPF filter 检查系统调用号和参数，返回 `allow`、`errno`、`trap`、`trace`、`kill`、`user notify` 等动作。

```
sys_openat:
    file = path_openat(...)
    ret = security_file_open(file)
    if ret:
        fput(file)
        return ret
    install_fd(file)
```

安全检查要放在对象已解析且引用稳定之后，否则路径 TOCTOU 仍可能存在。seccomp 适合收缩系统调用面，但它看不到所有内核对象语义；LSM 能看对象语义，但策略复杂。二者互补，不是替代关系。

vDSO 为什么能加速时间调用。有些“系统调用”只是读取内核维护的只读数据，例如当前时间。频繁进入内核会有上下文切换开销。vDSO 把一小段内核提供的用户态代码和只读数据映射到进程地址空间，使 `clock_gettime` 等调用能在用户态完成快路径。

```
clock_gettime():
    if vDSO available and clock mode stable:
        read seqlock-protected time data in user mapping
        return computed time
    else:
        syscall clock_gettime
```

vDSO 依赖 seqlock 一类协议：内核更新时改变序列号，用户态读取若发现并发更新就重试。它展示了一个重要设计：不是所有内核服务都必须每次陷入内核，稳定只读数据可通过受控共享映射给用户态。

系统调用表、ABI 和兼容层。系统调用 ABI 一旦发布，就很难改变。系统调用号、参数寄存器、返回值约定、错误码范围和结构体布局都属于用户态契约。Linux x86-64 的系统调用表在 [arch/x86/entry/syscalls/syscall_64.tbl](#) 一类文件中维护；32 位兼容进程还可能走 `compat syscall`，把 32 位结构体转换为内核内部格式。

```
syscall ABI:
    rax = syscall number
    rdi, rsi, rdx, r10, r8, r9 = arguments
    rax = return value or negative errno
```

这解释了为什么内核常新增系统调用而不是随意改变旧系统调用语义。用户态程序、libc、容器 seccomp profile、审计工具和 trace 工具都依赖 ABI 稳定。教学 OS 也应把 syscall 编号集中管理，并写兼容性测试。

审计、ptrace 和 restart block。系统调用入口不只分发表项。它还可能经过 audit、ptrace、seccomp、ftrace 和 signal 检查。调试器通过 ptrace 可在 syscall enter/exit 停住进程，修改寄存器或观察参数；安全审计记录主体、对象和结果；信号可能让阻塞系统调用返回 `EINTR` 或进入 restart。

```
do_syscall_64(regs):
    nr = regs.orig_ax
    if ptrace_or_seccomp_active:
        nr = syscall_trace_enter(regs, nr)
    ret = dispatch_syscall(nr, regs)
    regs.ax = ret
```

```
syscall_exit_work(regs)
```

restart block 用于保存重启系统调用所需状态。比如带 timeout 的阻塞调用被信号打断后，剩余 timeout 需要重新计算，而不能简单从头开始等待。系统调用重启是用户可见语义和内核内部状态的交叉点。

资源限制：rlimit 与 errno。权限检查之外，系统调用还要检查资源限制。进程可打开 fd 数、文件大小、地址空间、栈、锁定内存、进程数都可能受 rlimit 或 cgroup 限制。错误码要表达限制来源。

限制	触发路径	常见错误
open files	open/dup/socket/accept	EMFILE/ENFILE
file size	write/truncate	EFBIG 或信号
address space	mmap/brk	ENOMEM
locked memory	mlock	ENOMEM/EPERM
process count	fork/clone	EAGAIN

这类错误通常不是权限不足，而是资源边界。高质量程序应把 EMFILE、ENOMEM、EAGAIN 分开处理：释放资源、限流、等待或明确失败。

用户内存数组和二次复制。execve、sendmsg、ioctl 这类系统调用会传入指针数组或嵌套结构。内核不能只复制外层结构后继续信任内层用户指针。正确做法是逐层验证长度上限，把需要长期使用的数据复制到内核内存或 pin 住用户页。

```
copy_iovec(user_iov, count):
    if count > UIO_MAXIOV:
        return -EINVAL
    k_iov = copy_array_from_user(user_iov, count)
    for each entry in k_iov:
        if entry.len overflows total:
            return -EINVAL
        if !access_ok(entry.base, entry.len):
            return -EFAULT
    return k_iov
```

二次复制和整数溢出是系统调用漏洞高发区。长度相加要检查溢出，数组个数要有上限，结构体 padding 不能泄露内核栈数据，输出结构体应先清零再填字段。

系统调用、缺页和文件 I/O 实现深讲

本节把系统调用、虚拟内存、fd、VFS 和 page cache 连成一条实现路径。用户看到的是 open/read/write/mmap/fork/exec，内核看到的是 fd 表、file、inode、VMA、PTE、folio、bio、request 和设备 completion。深入理解 OS，必须能把这些对象按生命周期串起来。

系统调用入口的最小状态。

CPU 进入系统调用入口后，内核必须拿到系统调用号、参数寄存器、用户返回地址、用户栈、状态寄存器和当前线程。入口代码先建立可信执行环境，再调用 C 层分发。

```
syscall entry state:
    user RIP and RFLAGS
    user stack pointer
    syscall number
    argument registers
    current task pointer
    kernel stack pointer
    page table context
```

入口路径不能直接相信用户栈。用户栈可能不可访问、被另一个线程修改，甚至故意指向内核地址。参数寄存器只是整数，指针参数要在具体系统调用里通过 `copy_from_user` 或 `iov` 机制处理。

fd 表、file 和 inode。

文件描述符不是文件本身。fd 是进程 fd 表中的整数索引；索引指向 open file description，也就是内核的 `struct file`；`file` 再指向 `inode` 或 `socket` 等具体对象。多个 fd 可以指向同一个 `file`，共享文件 offset；多个 `file` 可以指向同一个 `inode`，各自有不同 offset 和 flags。

```
process files_struct:
  fd 0 -> file A -> inode terminal
  fd 1 -> file B -> inode output.txt
  fd 2 -> file B -> inode output.txt

dup(1):
  new fd points to same file B
  file offset is shared

open same path again:
  new file C points to same inode
  file offset is independent
```

这解释了 shell 重定向、`dup2`、`fork` 后 fd 共享、`close-on-exec` 的行为。若父子进程 `fork` 后共享同一个 `file`，它们写同一 fd 时共享 offset；若各自重新 open，同一 `inode` 也有不同 offset。

open 的路径解析。

`open("/a/b/c")` 不是一次查表。内核从当前 root 或 cwd 出发，逐级解析 dentry，处理挂载点、符号链接、权限、缓存命中和可能的文件系统 I/O。路径解析必须防止 race 和越权。

```
open path:
  choose starting point: root or cwd
  split path components
  for each component:
    lookup dentry in dcache
    if miss, ask filesystem lookup
    follow mount point if needed
    handle symlink subject to flags and limits
    check execute/search permission on directory
  check final file permissions and flags
  allocate struct file
  install into fd table
```

路径解析有大量安全边界。攻击者可能在检查后替换路径组件；符号链接可能指向意外位置；目录权限和文件权限不同；`O_NOFOLLOW`、`O_CLOEXEC`、`openat`、`openat2` 都是在修复具体 race 或权限边界。

read 的三条路径：缓存命中、缓存未命中、直接 I/O。

普通文件 `read` 首先查 page cache。若命中，内核把缓存页复制到用户 buffer；若未命中，文件系统和块层发起 I/O，设备完成后再复制；若使用 `O_DIRECT`，则尽量绕过 page cache，把用户页或内核 bounce buffer 直接交给块层。

路径	数据流	代价
page cache hit	cache page 到用户 buffer	copy 成本，低延迟
page cache miss	storage 到 page cache, copy	再 I/O 等待，后续可复用

direct I/O	用户页和块设备之间 DMA 或接 近 DMA	对齐、pin 页、生命周期 复杂
------------	---------------------------	---------------------

直接 I/O 不是“一定更快”。它减少 page cache 污染和复制，但增加对齐、页 pin、设备队列和应用缓存管理成本。数据库常用它，是因为数据库自己管理缓存和写入顺序；普通应用通常受益于 page cache。

page cache 的 xarray 思想。

文件的 page cache 需要按文件 offset 快速找到页。Linux 使用 address space 和 xarray 一类结构管理文件页。概念上可以理解为：

```
file address_space:
    key = page index within file
    value = folio containing file data

read page:
    index = file_offset / PAGE_SIZE
    folio = xarray_lookup(mapping, index)
    if folio exists and uptodate:
        copy from folio
    else:
        allocate folio and start read I/O
```

folio/page 的状态位很重要：uptodate 表示内容有效，dirty 表示需要写回，writeback 表示正在写，locked 表示某个路径正在操作。缺少这些状态，多个线程会重复读同一页或同时写回同一页。

write 的脏页路径。

普通 buffered write 通常先把用户数据复制到 page cache，把页标记 dirty，然后返回。后台 writeback 或 fsync 再把 dirty 页写到存储设备。

```
buffered write:
    lookup or create page cache folio
    copy_from_user into folio
    mark folio dirty
    update file size if extending
    return bytes copied

later writeback:
    pick dirty folios
    filesystem maps file blocks
    submit bios to block layer
    on completion clear writeback
```

这解释了为什么 write 可能很快而 fsync 很慢。前者只把数据推进到内核内存和文件系统状态，后者要等设备日志顺序满足持久化要求。

mmap 读文件：不经过 read 也用 page cache。

mmap 文件后，用户第一次访问对应地址会触发缺页。内核通过 VMA 找到文件和 offset，再从 page cache 找或读对应页，安装 PTE。之后用户态 load/store 直接访问映射页。

```
file mmap fault:
    CPU raises page fault at virtual address
    kernel finds VMA
    offset = vma.file_offset + (addr - vma.start)
    folio = page_cache_get_or_read(file.mapping, offset)
    install PTE mapping folio
    return to faulting instruction
```


所以 `read` 和 `mmap` 不是两套完全分离的缓存。它们常共享 `page cache`。一个进程 `read` 过的文件页，另一个进程 `mmap` 访问可能命中；反过来也成立。区别在于数据进入用户态的方式：`read` 复制，`mmap` 建 PTE。

COW fault 的实现分支。

`MAP_PRIVATE` 文件映射或 `fork` 后匿名页共享，都可能使用 COW。写入时，硬件发现 PTE 不可写，触发保护异常。内核检查 VMA 允许写且 PTE 是 COW，然后复制页。

```
COW write fault:
  locate VMA and PTE
  verify write is allowed by VMA
  if PTE is COW:
    old = pte.page
    new = allocate_page()
    copy_page(new, old)
    install writable PTE to new page
    decrement old refcount
    flush TLB for address
  else:
    signal permission fault
```

COW 的难点在并发。两个线程可能同时 `fault` 同一页；`fork`、`unmap`、`reclaim` 也可能并发修改页表。内核必须用页表锁、页引用计数和重试逻辑保证不会复制错误页或泄漏引用。

缺页错误如何变成信号或 EFAULT。

同一个坏地址，在不同上下文下结果不同。用户态直接访问坏地址，内核通常发送 `SIGSEGV`。系统调用 `copy_from_user` 过程中访问坏地址，系统调用应返回 `EFAULT`。内核自己访问坏内核地址，则是内核 bug，可能 `oops` 或 `panic`。

上下文	fault 结果	原因
用户指令	signal	用户程序违反地址空间规则
内核 <code>copy user</code>	<code>-EFAULT</code>	系统调用参数错误，可返回给调用者
内核普通指针	<code>oops/panic</code>	内核自身错误或硬件故障
缺页可修复	返回重试指令	懒分配、COW、文件页读入

这就是 `exception table` 的价值：内核某些 `copy` 用户路径可以声明“若这里 `fault`，跳到修复位置返回错误”，而不是让内核崩溃。

`epoll` 的 `readiness` 不是 `completion`。

文件和 `socket` 的 `readiness` 只表示“现在尝试某类操作可能不阻塞”，不是保证一定读到固定字节，也不是 I/O `completion`。多线程或边缘触发下，`readiness` 状态可能在应用处理前就变化。

```
epoll wait:
  if socket receive queue not empty:
    report readable
  user thread wakes
  another thread may consume data first
  read may still return EAGAIN on nonblocking fd
```

所以非阻塞 I/O 必须在循环中处理 `EAGAIN`。`epoll` 降低“等待很多 `fd`”的成本，但不取消对象状态变化和并发竞争。`readiness` API 和 `io_uring completion` API 的语义不同，不能混用理解。

io_uring 的提交和完成环。

io_uring 用共享 ring 减少系统调用次数。用户态写 submission queue entry，内核消费 SQE 并执行请求，完成后写 completion queue entry。关键仍是所有权、顺序和生命周期。

```
io_uring lifecycle:
  userspace fills SQE
  userspace advances SQ tail with store-release
  kernel observes SQ tail with load-acquire
  kernel validates fd, buffer, opcode
  request is issued to file/socket/block layer
  completion CQE is written
  userspace reads CQE and recycles resources
```

若 buffer 在 completion 前释放，异步路径可能使用悬空内存；若 fd 在请求执行前关闭，内核必须通过引用计数保证对象要么仍有效，要么请求明确失败；若应用不消费 CQE，完成队列会背压。

fd 生命周期和引用计数。

并发 close 和 I/O 是系统调用实现的经典难点。用户关闭 fd 后，fd 表项删除，新的 open 可能复用同一整数。但已经拿到 struct file 引用的内核 I/O 可以继续完成。

```
sys_read(fd):
  file = fget(fd)           // increments file refcount if fd valid
  if file == null:
    return -EBADF
  ret = vfs_read(file)
  fput(file)                // may release after last ref
  return ret

sys_close(fd):
  file = remove fd table entry
  fput(file)
```

这解释了为什么 fd 整数不能长期当对象身份。close 后同一个数字可能马上指向另一个文件。内核内部必须使用对象引用，用户态高层库也要小心 fd reuse 带来的取消和多线程 race。

文件系统日志和 page cache 的关系。

page cache 保存文件数据，文件系统日志通常保护元数据或特定模式下的数据顺序。dirty page 写回和日志提交是相关但不同的状态机。fsync 必须把两者推进到一致点。

```
fsync(file):
  write dirty file data pages
  wait data writeback completion
  commit filesystem metadata transaction
  issue flush/FUA as needed
  return only after required durability point
```

不同文件系统和挂载选项会改变数据与元数据顺序。例如 ordered 模式要求相关数据块在提交元数据前写出；writeback 模式可能只保证元数据一致；journal data 模式连数据也进日志。应用不能凭感觉推断崩溃语义，必须理解目标文件系统契约。

把一次请求完整编号。

分析复杂 I/O 时，给每一层对象编号能避免混乱。

编号	对象	状态问题
1	用户 fd 整数	是否仍指向预期 file
2	<code>struct file</code>	引用计数、offset、flags
3	inode/address space	文件大小、page cache、权限
4	folio/page	uptodate、dirty、locked、writeback
5	bio/request	块映射、队列、超时
6	DMA buffer/descriptor	map/unmap、所有权、completion
7	device command	status、错误码、reset 影响

任何一层状态没写清楚，都会产生 bug。例如 fd 已 close 但 file 引用还在，page dirty 但 writeback 未完成，request 已超时但设备稍后仍 completion，reset 后旧 DMA 仍可能写内存。

实现级测试矩阵。

- dup 后两个 fd 共享 offset；重新 open 的 fd 不共享 offset。
- `O_CLOEXEC` fd 在 exec 后关闭，普通 fd 按规则保留。
- 坏用户指针在 read/write 中返回 `EFAULT`，不导致内核崩溃。
- `MAP_PRIVATE` 写入后文件内容不变，`MAP_SHARED` 按规则变 dirty。
- fork 后 COW 写入不影响父子另一方。
- close 与并发 read/write 不造成 use-after-free，fd reuse 不混淆对象。
- page cache 命中路径不发块 I/O，冷缓存路径能观察到块 I/O。
- fsync 后再模拟崩溃，文件数据和目录项语义符合预期。
- 非阻塞 fd 在未 ready 时返回 `EAGAIN`，epoll 事件后仍能处理竞态。
- io_uring 请求在 completion 前不能释放 buffer 或丢失对象引用。

第一阶段源码走读：x86-64 系统调用入口

第一阶段不能只说“系统调用进入内核”。真正要过关，必须能把一条 `syscall` 指令映射到 x86-64 寄存器约定、低级入口汇编、`pt_regs`、C 层分发、用户指针复制、错误码和返回路径。这样后面讲 `open`、`read`、`mmap`、`exec`、信号和 `seccomp` 时，读者知道每个对象为什么能被内核检查，而不是把系统调用当成神秘函数表。

核心思想

x86-64 `syscall` 入口是 OS 最核心的安全边界之一。它不是“跳到一个 C 函数”，而是先由 CPU 和入口汇编建立可信执行环境，再由 C 层根据系统调用号、寄存器参数、当前任务、`seccomp`、tracing、权限和用户指针决定能否推进。

先固定源码入口和读法。

读 Linux 时先看这些路径。不同内核版本函数名会有细微变化，但对象和边界基本稳定。

源码路径	先看什么	读法
------	------	----

arch/x86/entry/e	entry_SYSCALL_64、返回用	看寄存器怎样保存、栈怎
ntry_64.S	户态路径	样切换、何时走慢路径
arch/x86/entry/c	do_syscall_64、exit work	看系统调用号、seccomp、
ommon.c		tracing、信号、调度检查
arch/x86/entry/s	系统调用号表	看 ABI 如何把数字绑定
yscalls/syscall_		到实现名
64.tbl		
include/linux/sy	SYSCALL_DEFINE* 声明	看 C 实现怎样接入统一
scalls.h		表项
arch/x86/include	struct pt_regs	看 trap frame 在 C 层
/asm/ptrace.h		怎样表示
lib/usercopy.c、ar	usercopy	看用户指针复制怎样和
ch/x86/lib/userc		异常修复配合
opy_64.c		

读入口代码时不要从第一行汇编硬啃。先带着四个问题读：用户态把什么放进寄存器，CPU 自动改了什么，入口汇编补保存了什么，C 层分发前还有哪些安全工作。

用户态 ABI：参数不是压到栈上。

x86-64 Linux 系统调用 ABI 把系统调用号放在 `rax`，最多六个参数放在 `rdi/rsi/rdx/r10/r8/r9`。注意第四个参数用 `r10`，不是 C 函数调用 ABI 的 `rcx`。原因是 `syscall` 指令会用 `rcx` 保存用户返回 RIP，用 `r11` 保存用户 RFLAGS。

寄存器	syscall ABI 含义	为什么重要
<code>rax</code>	系统调用号，返回时放返回值	调度到哪个 <code>sys_*</code> 实现
<code>rdi</code>	第 1 参数	例如 <code>write(fd,...)</code> 的 <code>fd</code>
<code>rsi</code>	第 2 参数	例如用户 <code>buffer</code> 指针
<code>rdx</code>	第 3 参数	例如长度
<code>r10</code>	第 4 参数	<code>rcx</code> 被硬件占用，不能放参数
<code>r8/r9</code>	第 5/6 参数	六参数系统调用使用
<code>rcx</code>	用户返回 RIP	<code>sysretq</code> 快速返回需要
<code>r11</code>	用户 RFLAGS	返回时恢复用户可见标志

这张表解释了为什么系统调用 `wrapper` 不是普通 C 调用。`libc` 或内联汇编必须把参数移动到这些寄存器，再执行 `syscall`。如果你在调试器里看系统调用入口，先看寄存器，而不是看用户栈。

CPU 做了什么，入口汇编又补了什么。

`syscall` 指令本身只做一部分特权转移：保存返回 RIP 到 `rcx`，保存 RFLAGS 到 `r11`，把 RIP 改成内核配置的入口地址，按 MSR 配置更新特权相关状态。它不会自动保存所有通用寄存器，也不会替你验证用户参数。

```
before syscall:
    rax = syscall number
    rdi/rsi/rdx/r10/r8/r9 = arguments
    rip = address of syscall instruction
    rsp = user stack

after CPU syscall transition:
    rcx = user return rip
    r11 = user rflags
    rip = IA32_LSTAR entry address
    execution enters ring 0 entry path
```

随后 `entry_64.S` 的入口代码补齐内核需要的上下文：处理 `swaps`，找到当前

CPU 的内核数据，切换到当前线程的内核栈，按约定把寄存器保存成 `pt_regs` 形状，再调用 C 层分发函数。

```
entry_SYSCALL_64 outline:
  swappgs or equivalent per-cpu base transition
  save user rsp
  load current task kernel stack
  push user ss, user rsp, rflags, user cs, user rip style frame
  save general-purpose registers into pt_regs
  call do_syscall_64(regs, nr)
```

这里的关键不是背汇编每一行，而是理解入口汇编在构造一个 C 层可读的事实包：这个任务从哪里来，用户 RIP/RSP/RFLAGS 是什么，参数寄存器是什么，返回时要恢复到哪里。

pt_regs：低级入口交给 C 代码的证据包。

`pt_regs` 可以理解为“这次陷入的证据包”。系统调用、page fault、中断和调试异常都会需要类似对象，只是字段含义和错误码不同。系统调用路径尤其关心参数寄存器、系统调用号、用户返回 RIP、用户栈和返回值位置。

pt_regs 类字段	来源	用途
通用寄存器	用户态执行 <code>syscall</code> 前的寄存器	提取参数、保存用户可见状态
<code>orig_ax</code>	原始系统调用号	tracing/seccomp/restart <code>syscall</code> 需要原号
<code>ip</code>	用户返回 RIP	信号、ptrace、返回路径验证
<code>sp</code>	用户 RSP	信号帧构造和调试输出
<code>flags</code>	用户 RFLAGS	单步、方向标志、中断相关状态
<code>ax</code>	返回值位置	C 层结果写回给用户态

入口代码漏保存字段，后面不是“少打印一个值”，而是可能破坏调试、信号、系统调用重启、seccomp、ptrace 和返回用户态。第一阶段读源码时要形成这个判断：低级结构体字段是安全边界的一部分。

C 层分发：不只是查表调用。

C 层分发通常可以抽象成下面的流程，但真实路径还要处理 tracing、audit、seccomp、ptrace、系统调用重启、信号和返回用户态前的 work。

```
do_syscall_64(regs, nr):
  if nr is outside syscall table:
    regs.ax = -ENOSYS
    return
  if syscall_enter_work is pending:
    run tracing/audit/seccomp hooks
    nr may be changed or rejected
  result = syscall_table[nr](regs arguments)
  regs.ax = result
  if syscall_exit_work is pending:
    run tracing/audit/signal/resched checks
```

这说明系统调用号表只是中间一步。比如 seccomp 可能在真正执行前拒绝某个系统调用；ptrace 可能观察或改写参数；audit 可能记录事件；信号可能导致阻塞系统调用返回 `-EINTR` 或被重启。把系统调用理解成“数组下标调函数”会漏掉安全和可观测性。

以 `write(fd, buf, len)` 追一条完整路径。

用户态调用 `write(1, buf, len)` 时，硬件入口只知道三个整数：fd、用户地

址、长度。它不知道这是终端、文件、pipe 还是 socket。内核必须逐层把整数还原成受保护对象。

```
write(fd, user_buf, len):
    fd table lookup -> struct file
    check file was opened for writing
    copy or pin user buffer under user access rules
    dispatch file->f_op->write_iter
    update file offset or stream queue
    return bytes written or negative errno
```

如果 fd 无效，返回 `-EBADF`。如果用户 buffer 不可读，返回 `-EFAULT`。如果对象暂时不能写且是非阻塞 fd，返回 `-EAGAIN`。如果信号打断等待，可能返回 `-EINTR`。这些错误不是装饰，它们分别来自 fd 表、用户地址空间、对象等待语义和信号状态。

用户指针复制：fault 可以是正常错误路径。

用户指针的特殊性在于：它来自不可信地址空间，但内核必须按请求访问它。直接解引用用户指针会把用户错误变成内核错误。正确路径是 `usercopy`：访问前后有范围、权限、异常修复和对象生命周期约束。

```
copy_from_user kernel idea:
while bytes remain:
    try load from user address
    if page fault happens at a usercopy instruction:
        jump to exception-table fixup
        return bytes not copied or -EFAULT
    store into kernel-owned buffer
```

异常表的价值在这里体现出来：同样是 page fault，如果 faulting RIP 在 `usercopy` 的异常表范围内，它是用户输入错误；如果发生在普通内核指针解引用上，它更可能是内核 bug。第一阶段讲 fault 时必须把这两类区分开。

返回路径：快返回和慢返回。

系统调用返回不是简单 `return result`。内核要决定能否走快速 `sysretq` 风格路径，还是必须走更通用的 `iretq` 风格路径。若用户返回 RIP/RFLAGS、tracing、信号、单步调试、抢占或兼容状态需要特殊处理，就可能走慢路径。

返回条件	快路径倾向	慢路径原因
普通系统调用，无 pending work	可快速恢复寄存器并返回	无
有信号待投递	不适合直接快返	需要构造用户信号帧
需要调度	不适合直接快返	返回前要切换任务
ptrace、seccomp、audit	不适合直接快返	需要通知观察者或改写结果
返回状态不满足快速指令限制	走通用返回	iretq 能表达更完整状态恢复

这也是为什么返回路径经常比入口更容易出安全问题。入口时内核取得控制权；返回时内核把控制权交回给不可信程序，必须保证所有用户可见状态都被正确恢复，内核私密状态没有泄漏。

观察证据：用真实系统校准理解。

原理卷不要求读者修改内核，但要求能把系统调用入口的解释和系统证据对上。

命令或文件	能观察什么	如何判读
-------	-------	------

<code>strace -e trace=</code>	用户态发起的系统调用和	只能看到边界结果，看不到内核内部等待
<code>write ./a.out</code>	<code>errno</code>	
<code>perf trace -e sys</code>	<code>syscall enter/exit</code>	能看到系统调用号、参数和返回时序
<code>scalls:sys_enter_</code>	<code>tracepoint</code>	
*		
<code>grep syscall /pr</code>	内核符号是否可见	受权限和 <code>kallsyms</code> 设置影响
<code>oc/kallsyms</code>		
<code>cat /proc/<pid>/s</code>	线程当前 <code>syscall</code> 和参数	只在任务正在系统调用中时有意义
<code>ysyscall</code>		
<code>bpftrace tracepoi</code>	<code>syscall tracepoint</code>	可按 <code>pid</code> 、 <code>comm</code> 、 <code>errno</code> 过滤
<code>nt:syscalls:*</code>		

用这些证据时要保持边界感。`strace` 看到 `write(1,...)=5`，只能证明用户/内核边界返回了 5；它不能证明数据已经落盘，也不能说明底层是否发生 `page fault`、锁等待、设备 I/O 或信号处理。要解释更深路径，需要结合 `perf`、`ftrace`、`eBPF`、`/proc/<pid>/stack` 和具体子系统日志。

第一阶段验收题。

读完本节，应能不看书回答这些问题。

1. 为什么 x86-64 Linux 第四个系统调用参数放在 `r10` 而不是 `rcx`？
2. `rcx` 和 `r11` 在 `syscall` 入口中分别保存什么？
3. 为什么入口汇编要构造 `pt_regs`，而不是直接调用 C 函数？
4. `do_syscall_64` 查表前后为什么还有 `seccomp`、`tracing`、`audit` 和 `signal work`？
5. `copy_from_user` 遇到 `page fault` 为什么通常返回 `EFAULT` 而不是让内核崩溃？
6. `sysretq` 和 `iretq` 都能回用户态，为什么还要区分快路径和慢路径？
7. `strace` 能证明什么，不能证明什么？

这些问题能答清，系统调用就不再是“用户调用内核函数”，而是一条从 x86-64 硬件入口、低级汇编、C 层分发、对象权限、用户内存和返回路径共同组成的受控边界。

第 6 章 exec、ELF 装载与用户栈

6.1 原理

`fork` 复制了一个进程的执行环境，而 `exec` 把当前进程的用户态程序替换成另一个程序。这个替换不是“创建新进程”，因为 `pid`、父子关系、部分资源限制和很多内核对象仍然属于同一个进程；它也不是普通文件读取，因为内核必须验证可执行文件格式、建立新的地址空间、设置入口点、构造用户栈、处理解释器、重置部分信号语义并关闭带 `close-on-exec` 标志的 `fd`。

ELF 装载器是系统调用边界、文件系统、虚拟内存和权限模型的交汇点。用户给内核一个路径和参数数组，内核必须在不信任用户内存的前提下读取这些字符串；路径解析得到 `inode` 后，还要检查执行权限、文件类型、挂载选项和安全策略；读取 ELF 头后，装载器根据 `program header` 而不是 `section header` 建立映射。`section` 是链接和调试视角，`program segment` 才是运行时装载视角。

核心思想

`exec` 的核心契约是原子替换用户态映像：成功后旧程序不能继续执行；失败时旧程序必须仍然可运行，除非失败发生在明确不可恢复的提交点之后。

真实系统还要处理动态链接。若 ELF 带 `PT_INTERP`，内核并不直接跳到主程序入口，而是同时映射解释器，把控制权交给动态链接器。动态链接器再加载共享库、重定位符号，最后跳到用户程序的入口。内核只负责把 `aux vector`、入口、`program header` 信息和随机数等必要元数据放到用户栈。

6.2 实践

一个教学内核中的 `execve(path, argv, envp)` 可以拆成九个阶段：

1. 从用户内存复制 `path`、`argv`、`envp`，设置长度上限。
2. 路径解析并打开可执行 `inode`。
3. 检查文件类型、执行权限、`noexec` 挂载选项和安全策略。
4. 读取并验证 ELF header。
5. 扫描 `program header`，计算新地址空间布局。
6. 创建临时地址空间，映射 `PT_LOAD` 段和用户栈。
7. 若存在 `PT_INTERP`，装载解释器并设置实际入口。
8. 构造用户栈：`argc`、`argv` 指针、`envp` 指针、`auxv`、字符串和对齐填充。
9. 提交：切换进程 `mm`、寄存器入口和栈指针，关闭 `CLOEXEC` `fd`。

这里最容易讲错的是失败回滚。前七步应尽量在临时对象中完成：临时 `mm`、临时 `VMA`、临时页、临时 `fd` 引用。只有全部验证成功后，才进入提交阶段并销毁旧地址空间。否则一个坏 ELF 就可能把调用者进程破坏到无法返回错误码。

对象	成功后状态	失败回滚
旧 <code>mm</code>	被新 <code>mm</code> 替换并释放引用	保持可运行
新 <code>mm</code>	成为 <code>current</code> 的地址空间	释放所有 <code>VMA</code> 和页
文件引用	可执行文件可关闭或保留映射引用	put <code>inode/file</code> 引用

fd table	关闭 <code>FD_CLOEXEC</code> 条目	不应改变
信号处理	捕获处理器重置，忽略语义按规则保留	不应改变
寄存器	RIP 指向入口，RSP 指向新用户栈	不应改变

把这条路径放进一个具体命令会更清楚。假设 shell 进程 pid 为 3182，执行：

```
execve("/usr/bin/grep",
      ["grep", "-n", "main", "a.txt"],
      envp);
```

进入内核时，`path`、`argv` 和 `envp` 都还在旧地址空间里。这里的旧地址空间不是“快要没用的垃圾”，而是内核目前唯一能读到这些字符串的地方。内核不能先释放旧 mm 再慢慢读参数；一旦旧页表被拆掉，`argv[2]` 指向的 “main” 可能就再也翻译不出来。因此第一轮工作是复制输入：先复制路径字符串，再按指针数组逐个复制 `argv/envp` 字符串，并统计总字节数。若 `argv[3]` 恰好跨到一个不可读页，正确结果是 `-EFAULT`，旧 shell 继续运行，fd table、signal handler 和旧地址空间都不能被改坏。

阶段	内核手里的稳定证据	此时失败应怎样收场
复制用户参数	内核缓冲区里的 <code>path</code> 、 <code>argv</code> 、 <code>envp</code> 副本	返回 <code>EFAULT/E2BIG</code> ，旧程序继续
路径解析	指向 <code>/usr/bin/grep</code> 的 <code>inode/file</code> 引用	返回 <code>ENOENT/EACCES</code> ，释放引用
ELF 验证	ELF header 和 <code>program header</code> 副本	返回 <code>ENOEXEC</code> ，旧 mm 不变
临时装载	新 mm、VMA、栈页、解释器信息	释放临时 mm，旧程序继续
提交	新入口、新栈、新凭据、 <code>CLOEXEC</code> 关闭集合	成功后不再回到旧用户代码

接着内核解析路径，得到稳定的 `inode` 和 `file` 引用。权限检查必须绑定这个对象，而不是只检查字符串 “/usr/bin/grep”。检查通过后，内核读取 ELF header。若这是动态链接的程序，`program header` 里通常有 `PT_INTERP`，例如指向 `/lib64/ld-linux-x86-64.so.2`。这意味着“用户写的是执行 `grep`”，但第一条用户态指令很可能属于动态链接器；动态链接器根据 `auxv` 找到主程序的 `program header`，完成共享库装载和重定位，最后再跳到 `grep` 的入口。这个过程解释了为什么 `exec` 栈上要有 `AT_PHDR`、`AT_PHNUM`、`AT_ENTRY`、`AT_BASE` 和 `AT_RANDOM`，它们不是装饰字段，而是用户态运行时启动所需的证据。

再看文件描述符。shell 可能提前打开了管道：

```
grep -n main a.txt | less
```

这时 `grep` 进程继承的 `STDOUT_FILENO` 可能不是终端，而是管道写端。`exec` 成功后，普通 fd 继续指向同一个 open file description；带 `FD_CLOEXEC` 的 fd 则在提交时关闭。这个规则必须在提交点附近执行：太早关闭会导致 `exec` 失败后旧程序少了 fd；太晚关闭会让新程序短暂看到本不该继承的能力。于是 `FD_CLOEXEC` 不是“关闭一些数字”这么简单，它是在程序映像替换边界上裁剪继承能力。

最后把新用户栈具体展开。假设栈顶从高地址向低地址增长，内核通常先把字符串放到栈下方，再压入指针表和 `auxv`。进入动态链接器或程序入口时，用户态看到的是：

```
low address
```

```

argc = 4
argv[0] -> "grep"
argv[1] -> "-n"
argv[2] -> "main"
argv[3] -> "a.txt"
argv[4] = NULL
envp[0] -> ...
...
NULL
auxv entries: AT_PHDR, AT_PHNUM, AT_ENTRY, AT_RANDOM, ...
padding for alignment
strings: "grep\0-n\0main\0a.txt\0..."
high address

```

如果这里指针顺序错了，`main(int argc, char** argv)` 看到的参数就会错；如果 16 字节对齐错了，某些 ABI 约定和运行时启动代码会在很早的位置崩溃；如果 `AT_RANDOM` 指向了旧地址空间或未映射地址，libc 初始化 canary 时会 fault。用户栈构造不是格式化输出，而是内核和用户态启动代码之间的二进制契约。

6.3 算法与实现分析

ELF header 验证。ELF 验证要防止把任意文件当代码执行。最小检查包括 magic、class、endianness、machine、type、program header 范围和整数溢出。

```

validate_elf(file):
    eh = read_at(file, 0, sizeof(Elf64_Ehdr))
    require eh.magic == 0x7f 'E' 'L' 'F'
    require eh.class == ELFCLASS64
    require eh.machine == EM_X86_64
    require eh.phentsize == sizeof(Elf64_Phdr)
    require eh.phnum <= MAX_PHDRS
    require checked_mul(eh.phnum, eh.phentsize, &ph_size)
    require checked_add(eh.phoff, ph_size, &ph_end)
    require ph_end <= file.size or file supports sparse read policy

```

复杂度是 $O(\text{phnum})$ 。真正关键的是边界检查：`p_offset + p_filesz`、`p_vaddr + p_memsz`、页对齐和权限组合都可能溢出。装载器不能相信编译器或链接器一定生成合法文件，因为攻击者可以手工构造 ELF。

段映射。`PT_LOAD` 段描述文件区间到虚拟地址区间的映射。文件大小 `p_filesz` 是需要从文件读出的内容，内存大小 `p_memsz` 包括 BSS 零填充。

```

map_load_segment(mm, file, ph):
    require ph.memsz >= ph.filesz
    require same_page_offset(ph.vaddr, ph.offset)
    start = page_down(ph.vaddr)
    end = page_up(ph.vaddr + ph.memsz)
    file_start = page_down(ph.offset)
    prot = prot_from_flags(ph.flags)
    map_file(mm, start, end, file, file_start, prot)
    zero_range(mm, ph.vaddr + ph.filesz, ph.vaddr + ph.memsz)

```

若采用 demand paging，`map_file` 不必立即读入所有页面，只需建立 VMA，等 page fault 时按文件偏移填充页面。这样 `exec` 延迟低，内存占用也低。若采用 eager loading，实现简单但启动大程序会有明显 I/O 峰值。

用 `/usr/bin/grep` 的第一条指令看 demand paging。`exec` 提交完成后，寄存器 RIP 已经指向解释器或主程序入口，RSP 指向新用户栈；但入口所在的代码页可能还没有真正读入物理内存。CPU 返回用户态后开始取指，MMU 查入口虚拟地址的

PTE，发现 `present=0`，于是触发 `instruction page fault`。内核重新进入缺页处理，先用 `fault` 地址找到对应 VMA，再发现它来自可执行文件的 `PT_LOAD` 段，权限是 `read+exec`，文件偏移由 `vaddr - segment.vaddr + segment.offset` 算出。若 `page cache` 里没有这一页，内核提交文件读；读完后安装只读可执行 PTE，返回用户态重试同一条取指。

时刻	状态	关键边界
exec 提交	新 mm 已安装，代码段只有 VMA 或部分 PTE	旧程序不能继续运行
第一次取指	入口页 PTE 不 <code>present</code> ，CPU 产生缺页异常	<code>faulting</code> 指令还没执行
缺页处理	根据 VMA 找到 ELF 文件和页偏移	必须检查 VMA 可执行权限
读入页面	<code>page cache miss</code> 时等待存储 I/O	当前 task 可睡眠，调度器运行别人
安装 PTE	PTE 标 成 <code>user</code> 、 <code>present</code> 、 <code>read</code> 、 <code>exec</code> 、 <code>not writable</code>	权限不能超过 ELF 段权限
重试取指	同一 RIP 再次执行，这次翻译成功	<code>demand paging</code> 对用户程序透明

这条路径解释了两个现象。第一，大程序 `exec` 可以很快返回到用户态，因为只物化了马上需要的页；后面真正访问到的代码和数据才陆续缺页。第二，`exec` 已经成功不代表后续每次取指和读数据都不会失败：底层文件系统 I/O 错误、映射被异常截断、权限标志错误，都可能在第一次访问某页时暴露。教材若只说“`exec` 装载 ELF”，读者会误以为装载一定是一次性读完整个文件；真实边界是“建立地址空间契约”和“按需把页物化”两步。

用户栈构造。用户栈是 ABI 契约的一部分。教学系统可以简化 `auxv`，但不能把 `argv` 字符串和 `argv` 指针混为一谈。

```
build_user_stack(argv, envp, auxv):
    sp = USER_STACK_TOP
    for s in reverse(strings(argv, envp)):
        sp -= len(s) + 1
        copy_to_new_user(sp, s)
        remember_pointer(s, sp)
    sp = align_down(sp, 16)
    push_null()
    push_auxv(auxv)
    push_null()
    push_pointers(envp)
    push_null()
    push_pointers(argv)
    push(argc)
    return sp
```

栈构造要设置总大小上限，防止用户传入巨量参数耗尽内核内存。所有用户字符串必须先复制到内核或临时页中，因为在 `exec` 期间旧地址空间即将被替换；若边复制边销毁旧 mm，就可能读到无效地址。

提交点。提交阶段要尽量短，并且顺序清晰：

```
commit_exec(new_image):
    old_mm = current.mm
    current.mm = new_image.mm
    current.regs.ip = new_image.entry
```

```

current.regs.sp = new_image.stack_pointer
reset_caught_signal_handlers(current)
close_cloexec_fds(current.files)
activate_mm(new_image.mm)
drop_mm(old_mm)
return_to_user()

```

提交点之后不应再执行可能失败且需要返回旧程序的操作。比如关闭 `CLOEXEC` fd 一般可以在提交逻辑中完成，因为关闭失败不应阻止 `exec` 成功；但读取解释器、分配栈页和权限检查必须放在提交点之前。

6.4 测试

1. 正常 ELF：静态程序能打印 `argc/argv/envp`，入口地址和栈对齐正确。
2. 坏 magic：返回 `ENOEXEC`，旧程序继续运行。
3. 无执行权限：返回 `EACCES`，fd table 不变化。
4. 段溢出：构造 `p_vaddr + p_memsz` 溢出的 ELF，装载器必须拒绝。
5. BSS 清零：全局未初始化数组初值必须为零。
6. `FD_CLOEXEC`：带标志 fd 在新程序中不可见，普通 fd 仍可用。
7. 大 argv：超过限制返回 `E2BIG`，不破坏旧地址空间。
8. 动态链接入口：带 `PT_INTERP` 时入口应进入解释器而不是主程序 `e_entry`。

本章合格标准：能区分 `fork` 和 `exec`；能解释 ELF program header 的运行时意义；能画出用户栈布局；能指出 `exec` 的失败回滚边界和提交点。

本章压缩进来的装载与身份账本。`exec` 很容易被误讲成“把一个 ELF 读进内存”。真正的主线更宽：它同时替换地址空间、重建用户栈、改变入口寄存器、裁剪 fd 继承、可能改变凭据、处理多线程、交给解释器或动态链接器，并把旧程序的错误恢复边界切断。把这些细节压成一张账本，读源码时才不会只盯着 `load_elf_binary` 里某几次 `mmap`。

账本	exec 要修改什么	不能出错的边界
地址空间	新 <code>mm</code> 、VMA、栈、 <code>brk</code> 、 <code>mmap base</code> 、 <code>VDSO</code>	提交前失败必须保留旧 <code>mm</code> ，提交后不能再回旧代码
文件与路径	可执行文件、解释器、脚本原文件、 <code>mount flags</code>	权限检查必须绑定稳定 <code>inode/file</code> ，而不是路径字符串
fd 继承	普通 fd 继承， <code>FD_CLOEXEC</code> fd 关闭	不能在 <code>exec</code> 失败时提前关闭旧程序 fd
凭据与安全	<code>setuid</code> 、 <code>file capability</code> 、 <code>LSM</code> 、 <code>no_new_privs</code> 、 <code>dumpable</code>	提权 <code>exec</code> 要清理危险环境和调试关系
线程组	成功后只保留调用 <code>exec</code> 的线程	旧用户态锁和其他线程栈不能进入新映像
用户态启动	<code>argc/argv/envp/auxv</code> 、入口 <code>RIP</code> 、栈对齐	动态链接器和 <code>libc</code> 在 <code>main</code> 前就依赖这些字段

这张账本能解释很多看似零散的规则。脚本 `#!/bin/sh` 需要重新构造 `argv`，是因为真正执行的是解释器；动态 ELF 入口先到 `ld-linux`，是因为重定位和共享库

加载属于用户态运行时；`no_new_privs` 会压制提权，是因为 `seccomp` 和沙箱不能允许“先收紧入口、再 `exec` 一个 `setuid` 程序拿回更大权限”；多线程 `exec` 要清掉其他线程，是因为旧程序里的用户态 `mutex`、`TLS`、栈和信号现场对新程序没有可解释语义。

再用一个带提权和动态链接的例子收束。进程执行一个 `setuid root` 的动态链接程序时，内核不能只检查 ELF magic。它要先解析可执行 `inode`，计算新凭据，判断 `no_new_privs`、文件 `capability` 和 `LSM` 策略；若发生权限边界变化，运行时还要进入 `secure mode`，忽略或清理会影响动态链接器的环境变量，例如 `LD_PRELOAD`。否则攻击者可以让高权限程序在启动前加载自己控制的共享库。

```
privileged_exec_boundary(file):
    old_cred = current_cred
    new_cred = compute_exec_cred(file, old_cred)
    if privilege_will_change(old_cred, new_cred):
        enable_secureexec()
        suppress_unsafe_loader_environment()
        restrict_dump_and_ptrace()
    install_new_image_and_cred_at_commit()
```

这里的提交点必须把“新程序身份”和“新程序映像”作为一组状态安装。若凭据已经提升但地址空间仍是旧程序，旧代码就拿到了不该有的身份；若地址空间已经换成新程序但 `fd` 和环境还没裁剪，新程序又可能短暂看到不该继承的能力。好的教学实现即使简化 `setuid` 和动态链接，也要保留这条不变量：`exec` 成功后看到的是一组自洽的新运行身份，`exec` 失败后旧程序仍保持原身份和原资源。

读 `exec` 源码时，至少标出四个点：仍可回滚到旧程序的点、开始销毁旧用户映像的点、新凭据和新地址空间一起生效的点、返回用户态且永不返回旧代码的点。缺少这四个点，就很难判断错误路径是否安全。

6.5 源码级补充：load_elf_binary、auxv 与 binfmt

ELF 结构字段。读 ELF 装载源码时要区分三类头：`Elf64_Ehdr` 描述整个文件，`Elf64_Phdr` 描述运行时段，`Elf64_Shdr` 描述链接/调试节。内核运行装载主要看 `program header`；`section header` 即使被 `stripped`，也不影响程序运行。

字段或类型	含义	装载用途
<code>PT_LOAD</code>	可映射段	建立 VMA，设置 R/W/X 权限
<code>PT_INTERP</code>	动态解释器路径	装载 <code>ld-linux.so</code> 并把入口交给它
<code>PT_PHDR</code>	<code>program header</code> 自身位置	传给用户态动态链接器
<code>PT_GNU_STACK</code>	栈权限提示	决定用户栈是否可执行
<code>e_entry</code>	ELF 入口虚拟地址	静态程序入口或解释器最终跳转参考

Linux 源码入口是 `fs/binfmt_elf.c` 的 `load_elf_binary`。读时抓住阶段：读取 `header`，读取 `program header`，检查 `interpreter`，建立新 `mm`，映射 `load segments`，设置 `brk`，构造栈和 `auxv`，设置寄存器。

aux vector 布局。`auxv` 是内核放到用户栈上的键值数组，给动态链接器和 `libc` 传递信息。常见项包括 `AT_PHDR`、`AT_PHENT`、`AT_PHNUM`、`AT_ENTRY`、`AT_BASE`、`AT_PAGESZ`、`AT_RANDOM`、`AT_EXECFN`、`AT_NULL`。


```

user stack top:
  strings for argv/envp
  random bytes
  auxv: AT_PHDR, value
        AT_PHENT, value
        AT_PHNUM, value
        AT_ENTRY, value
        AT_RANDOM, pointer
        AT_NULL, 0
  envp pointers, NULL
  argv pointers, NULL
  argc

```

`AT_RANDOM` 为 stack canary 和 ASLR 提供随机种子；`AT_BASE` 指向动态链接器基址；`AT_PHDR` 让动态链接器找到主程序段表。若 `auxv` 构造错误，程序可能在 `main` 前就崩溃。

解释器脚本和 `binfmt`。`#!/bin/sh` 脚本不是 ELF。内核识别 shebang 后，把解释器路径和脚本路径重写成新的 `argv`，再递归执行解释器。Linux 用 `binfmt` 框架支持多种二进制格式：ELF、脚本、misc 注册格式等。

```

execve("script.sh", ["script.sh", "a"]):
  read first line "#!/bin/sh"
  new argv = ["/bin/sh", "script.sh", "a"]
  execve("/bin/sh", new argv)

```

这里要限制递归深度，否则脚本解释器互相指向会无限递归。`binfmt_misc` 允许用户注册自定义格式，例如通过魔数把某类文件交给解释器。安全检查必须覆盖最终解释器和原脚本，不能只检查第一层路径。

`vfork` + `exec` 为什么高效但危险。`vfork` 让子进程在 `exec` 或 `exit` 前与父进程共享地址空间，父进程通常被挂起。它避免了复制页表和 COW 设置，适合立刻 `exec` 的路径；但子进程若在 `exec` 前修改内存或调用复杂函数，会破坏父进程状态。

```

child = vfork()
if child == 0:
    // only async-signal-safe, minimal setup
    dup2(fd, STDOUT_FILENO)
    execve(path, argv, envp)
    _exit(127)

```

现代系统常用 `posix_spawn` 在库和内核之间选择更安全高效的实现。理解 `vfork` 的意义，不是鼓励随便使用，而是理解 `exec` 前后地址空间提交点的重要性。

`setuid`、凭据变化与 `no_new_privs`。`exec` 不只是换代码。执行 `setuid/setgid` 文件时，进程凭据可能变化；`capability` 也会根据文件能力、`bounding set`、`ambient set` 和 `no_new_privs` 重新计算。动态链接器和环境变量处理必须考虑提权执行，避免用户用 `LD_PRELOAD` 一类环境影响高权限程序。

```

exec_security_transition(file):
  read inode mode, owner, file capabilities
  if no_new_privs:
    suppress privilege gain
    compute new effective/permitted/inheritable caps
    clear dangerous personality and env effects
    install new credentials at commit

```

这解释了为什么 `exec` 路径会调用 LSM hook 和凭据提交逻辑。安全检查不能只看 ELF 格式合法，还要看执行后主体身份是否改变、哪些 fd 继承、`dumpable` 标

志如何更新、ptrace 是否允许继续。

ASLR、PIE 与栈保护输入。exec 建立新地址空间时，会选择随机化的 mmap base、stack base、brk 和 VDSO 位置。PIE 主程序也可随机基址加载。内核还向 auxv 提供 AT_RANDOM，供 libc 初始化 stack canary。

机制	exec 阶段输入
ASLR	随机选择 mmap、stack、brk、VDSO、PIE 基址
stack canary	AT_RANDOM 提供随机字节
NX stack	PT_GNU_STACK 和默认策略决定栈可执行性
RELRO/动态链接	program header 和 dynamic section 给链接器信息

这些保护不是 exec 独立完成的，但 exec 负责把初始地址布局 and auxv 种子交给用户态运行时。若地址布局固定或随机数可预测，后续保护会明显削弱。

exec 与多线程。多线程进程调用 exec 时，成功后只留下调用 exec 的线程，其他线程被清理。因为新程序映像不可能继承旧程序的线程栈、锁状态和用户态同步对象。内核必须停止同一线程组中的其他线程，并让共享资源进入一致状态。

```
de_thread_for_exec(current):
    signal other threads to exit
    wait until thread group is single-threaded
    take over thread group leader identity if needed
    proceed to install new mm
```

这也是为什么 exec 提交点必须非常谨慎：旧线程可能持有用户态锁，但 exec 后这些锁没有意义；内核对象如 fd table、credentials、signal struct 还要按规则继承或重置。

binfmt_misc 与解释器安全边界。binfmt 框架允许注册自定义格式，把某些 magic 或扩展名交给解释器。它方便运行脚本、模拟器格式或语言运行时，但也扩大了 exec 策略面。解释器路径、参数重写、递归深度、权限继承和 mount noexec 都必须纳入检查。

```
registered format:
    magic = "\xCA\xFE"
    interpreter = "/usr/bin/custom-runner"
exec file:
    kernel rewrites argv:
        custom-runner original-file original-args...
```

安全模型必须说明权限检查作用于原文件、解释器，还是二者都检查。只检查脚本可执行而不检查解释器，或绕过 noexec 挂载，都会形成边界漏洞。

第 7 章 同步、IPC 与锁

多个执行流共享 CPU 和内存，需要同步原语防止互相破坏。本章从 `futex` 到 RCU，从 `pipe` 到信号，把等待关系和锁顺序讲完整。

7.1 原理

多线程程序的核心风险不是“同时执行”这个事实，而是共享状态缺少清晰的不变量。两个线程同时修改一个队列，硬件只能执行 `load`、`store`、`CAS`、`fence` 等局部操作；它不知道 `head`、`tail`、`size`、`closed` 之间应该满足什么关系。操作系统和运行库提供 `mutex`、`condition variable`、`semaphore`、`futex`、`pipe`、`signal` 等机制，帮助程序把共享状态、等待和唤醒写成可检查的协议。

操作系统进入并发世界以后，错误不再只来自单个函数写错变量，而来自两个执行流同时触碰同一份状态。中断处理程序可能和普通内核线程同时访问设备队列；两个 CPU 可能同时把任务放入 `runqueue`；一个进程关闭 `fd` 时，另一个线程可能正在通过同一个 `open file description` 发起 I/O。锁的目的不是让代码“看起来串行”，而是给共享对象规定一个可证明的临界区：谁能读，谁能写，写入完成前谁必须等待，等待时是否允许睡眠。

从裸机单片机演进过来时，最早的“锁”通常是关中断。它简单有效，因为只有一个 CPU，且并发主要来自中断打断主循环。进入多核后，关本地中断不能阻止另一个 CPU 访问同一对象，于是需要自旋锁；进入可能长时间等待的路径后，自旋浪费 CPU，于是需要 `mutex`、`semaphore` 和等待队列。每一种同步原语都绑定一个上下文限制：中断上下文不能睡眠；持有自旋锁时不能调用可能阻塞的函数；持有文件系统锁时不能随意回调用用户内存复制路径；调度器自己的锁必须在切换前后保持严格顺序。

核心思想

锁不是性能技巧，而是状态所有权的边界。设计锁时先写对象不变量，再决定哪些操作必须在同一把锁下完成。

内核锁设计要同时回答四个问题：保护哪个对象，持锁期间允许做什么，等待者在哪里排队，失败路径如何释放。很多死锁来自第四个问题被忽略：正常路径释放锁，错误路径提前返回却忘了释放；或者关闭路径拿到 A 锁后等待 B，而另一个路径拿到 B 后等待 A。

互斥锁保护的不是某个变量，而是一组字段之间的不变量。条件变量也不是消息队列，它表达的是“某个谓词暂时不满足，线程可以睡眠，等状态可能改变后再重新检查”。正确等待必须同时包含谓词、锁和退出路径。只写 `wait()` 而不写完整谓词，会遇到虚假唤醒、丢失唤醒和关闭卡死。

一个有界队列足以说明同步原理。队列状态包括 `buffer`、`capacity`、`head`、`tail`、`size`、`closed`。生产者等待 `size < capacity || closed`，消费者等待 `size > 0 || closed`。`push` 改变 `buffer`、`tail`、`size`；`pop` 改变 `buffer`、`head`、`size`；`close` 改变 `closed` 并唤醒所有等待者。任何一个字段在锁外修改，都会破坏状态机。

IPC 是跨进程同步和数据交换。`pipe` 把字节流连接到 `fd`，`socket` 把通信扩展到本机或网络，`shared memory` 共享页但不自动提供同步，`message queue` 提供消息边界，`eventfd` 把计数型事件暴露成 `fd readiness`。IPC 的关键不是哪个接口高级，而是数据在哪里、所有权如何转移、背压如何发生、关闭如何传播。

信号是异步控制流。它可以通知进程发生了外部事件，例如终端中断、子进程退出、非法内存访问、定时器到期。信号难学，是因为它打断正常执行路径，而且信号处理函数只能安全调用很少一部分函数。工程上常把信号转成更普通的事件，例如写 `pipe`、设置原子标志或使用 `signalfd`，再交给主循环处理。

7.2 实践

同步实践先写不变量表。

字段	含义	受谁保护
capacity	最大槽数，固定大于 0	构造后只读
head	下一个可读槽	mutex
tail	下一个可写槽	mutex
size	当前元素数量	mutex
closed	是否拒绝新写入	mutex

然后写等待谓词：

- 生产者等待：`size < capacity || closed`
- 消费者等待：`size > 0 || closed`
- 关闭等待：`running_workers == 0` 或 `queue_empty && all_workers_stopped`

下面是常见内核锁原语的实践边界：

原语	适用场景	禁止事项	典型对象
关中断	单核短临界区，中断共享变量	多核共享保护，长时间计算	tick 计数、单核设备寄存器影子状态
自旋锁	多核短临界区，不能睡眠的上下文	持锁 I/O、持锁 page fault、持锁等待用户事件	runqueue、描述符环、引用计数
mutex	进程上下文中可能等待的互斥	中断上下文获取，持有后进入不可重入路径	inode 元数据、文件系统目录更新
读写锁	读多写少且临界区短	长读者导致写者饥饿，升级锁顺序混乱	mount 表、路由表快照
RCU	读路径极热，读者不阻塞写者	立即释放旧对象，读区内睡眠	fdtable、路径缓存、配置表
semaphore	计数资源，允许多个持有者	用它表达复杂对象不变量	buffer 数量、并发 I/O 限额

一个合理的锁设计文档至少包含下面字段：

```
Object:
    runqueue[cpu]

Invariant:
    task.state == RUNNABLE implies task is on exactly one runqueue
    rq.nr_running == number of tasks linked from rq.head

Lock:
    rq.lock protects rq.head, rq.nr_running, task.rq_link

Allowed while holding:
    enqueue/dequeue, choose_next, update vruntime

Forbidden while holding:
    page allocation with reclaim, disk I/O, copy_from_user, waiting on mutex
```

等待队列必须把“检查条件”和“入队睡眠”放在同一把锁保护下，否则会出现 lost wakeup。正确模型不是“先看条件，不满足就睡”，而是“在保护条件的锁内，把

自己放到等待集合中，然后原子地释放锁并切换状态”。唤醒方也在同一把锁下改变条件并唤醒等待者。

IPC 实践也按同样方式写。pipe 的状态包括读端引用数、写端引用数、内核缓冲区、读写阻塞队列。写端关闭后，读端读空返回 EOF；读端关闭后，写端写入会失败或收到信号；缓冲区满时写阻塞或返回 EAGAIN；缓冲区空时读阻塞或返回 EAGAIN。

信号实践要避免在 handler 里做复杂逻辑。推荐的纸上模型是：handler 只记录最小事实，例如设置原子标志；主循环在安全位置检查标志并执行真正清理。若系统使用 signalfd，则信号也进入 fd/event loop 模型，和其他 I/O 事件统一处理。

7.3 算法与实现分析

环形缓冲区。环形缓冲区是设备驱动、pipe、socket buffer 和日志队列的基础结构。它用固定数组保存数据，用 head 指向下一个可读位置，用 tail 指向下一个可写位置。若额外维护 size，判断空满很直接；若不维护 size，则常留一个空槽，用 `head == tail` 表示空，用 `next(tail) == head` 表示满。

```
bool push(Ring* r, byte b) {
    if (r->size == r->capacity) return false;
    r->buf[r->tail] = b;
    r->tail = (r->tail + 1) % r->capacity;
    r->size += 1;
    return true;
}

bool pop(Ring* r, byte* out) {
    if (r->size == 0) return false;
    *out = r->buf[r->head];
    r->head = (r->head + 1) % r->capacity;
    r->size -= 1;
    return true;
}
```

push/pop 都是 $O(1)$ 。难点不在算法复杂度，而在并发所有权。单生产者单消费者可以用两个原子索引实现无锁队列；多生产者多消费者通常先用 mutex 保护不变量，等有证据证明锁是瓶颈，再考虑更复杂结构。

mutex 和等待队列。mutex 的关键不是把自旋改成睡眠，而是在失败时把当前线程从 runnable 集合移到等待队列：

```
mutex_lock(m):
    disable_preempt()
    spin_lock(m.wait_lock)
    if m.owner == null:
        m.owner = current
        spin_unlock(m.wait_lock)
        enable_preempt()
        return
    enqueue(m.waiters, current)
    current.state = BLOCKED
    schedule_unlocking(m.wait_lock)
    enable_preempt()
    retry mutex_lock(m)

mutex_unlock(m):
    spin_lock(m.wait_lock)
    if empty(m.waiters):
        m.owner = null
```



```

else:
    next = dequeue(m.waiters)
    m.owner = next
    next.state = RUNNABLE
    enqueue_runqueue(next)
    spin_unlock(m.wait_lock)

```

这里三个不变量：`owner` 要么为空要么指向持有线程；等待队列里的线程必须是 `BLOCKED`；被唤醒线程必须重新进入 `runnable` 集合。若 `unlock` 先释放 `owner` 再唤醒，另一个新线程可能插队，造成等待者饥饿。若唤醒时没有持有等待队列锁，等待节点可能和 `timeout/cancel` 路径并发释放。

条件变量。条件变量的算法核心是“检查谓词、睡眠、醒来后重新检查”。虚假唤醒不是异常，而是接口允许的行为；因此 `wait` 必须写在 `while` 循环中。

```

lock(m)
while !predicate():
    wait(cv, m)
// predicate is true while holding m
do_state_transition()
unlock(m)

```

`notify` 的语义也要讲清：通知不是把数据交给等待者，只是说明谓词可能变真。若生产者先通知再修改状态，消费者醒来仍看不到新状态；若修改状态后忘记通知，等待者可能永远睡眠；若关闭时只通知一个等待队列，其他等待者可能卡住。

信号量。信号量表达可用资源数量。`P/acquire` 操作在计数大于 0 时减一，否则等待；`V/release` 操作加一并唤醒等待者。它适合连接池许可、队列空槽、I/O in-flight 限额，不适合保护多字段复杂不变量。

```

acquire(sem):
    lock(sem.lock)
    while sem.count == 0:
        wait(sem.cv, sem.lock)
    sem.count -= 1
    unlock(sem.lock)

release(sem):
    lock(sem.lock)
    sem.count += 1
    notify_one(sem.cv)
    unlock(sem.lock)

```

信号量的常见 bug 是许可泄漏：`acquire` 成功后，错误路径忘记 `release`。工程上要用作用域对象或 `finally` 路径保证释放。另一个 bug 是把信号量当成关闭广播；关闭是额外状态，必须单独表达。

自旋锁。最小自旋锁可以用原子交换实现：

```

lock(spinlock *l):
    while atomic_exchange(l->held, true) == true:
        cpu_relax()

unlock(spinlock *l):
    atomic_store_release(l->held, false)

```

这个算法的互斥性来自原子交换：同一时刻只有一个 CPU 能看到旧值为 `false`。复杂度不是固定常数，因为竞争时等待时间取决于持锁者运行时间和 CPU 调度。实现上必须使用 `acquire/release` 语义，否则 CPU 或编译器可能把临界区内存访问

重排到锁外。真实内核还会加入持有者记录、递归检查、抢占关闭、IRQ 保存和等待统计。

普通 test-and-set 自旋锁在高竞争下会让所有 CPU 反复写同一个 cache line。ticket lock 用取号保证公平：

```
lock(ticket_lock *l):
    my = fetch_add(l->next, 1)
    while load_acquire(l->owner) != my:
        cpu_relax()

unlock(ticket_lock *l):
    store_release(l->owner, l->owner + 1)
```

ticket lock 的优点是 FIFO 公平，缺点是所有等待者仍然轮询 owner cache line。队列锁进一步让每个等待者轮询自己的节点，降低缓存抖动。教学内核可以先实现 test-and-set，再用压力测试观察竞争成本，最后引入 ticket 或 MCS 解释公平性和 cache line 迁移。

RCU 的读写分离。RCU 解决的是读路径极热、写路径较少的场景。读者进入 RCU 临界区时不获取传统互斥锁，只保证自己不会在临界区内被回收对象悬空；写者发布新对象后，必须等所有旧读者经过 quiescent state 才释放旧对象。

```
reader():
    rcu_read_lock()
    p = rcu_dereference(global_ptr)
    use(p)
    rcu_read_unlock()

writer():
    old = global_ptr
    new = clone_and_update(old)
    rcu_assign_pointer(global_ptr, new)
    call_rcu(old, free_after_grace_period)
```

RCU 的不变量是：任何读者看到的对象在读侧临界区结束前仍然有效。它不保证读者看到最新状态，也不适合读区内睡眠的普通实现。把 RCU 当成“无锁万能替代品”会导致非常隐蔽的 use-after-free。

IPC 的背压算法。pipe 和 socket 都需要背压。缓冲区满时，写入者要么阻塞，要么在非阻塞模式返回 `EAGAIN`。背压算法的本质是有界队列：

```
write(pipe, bytes):
    while bytes not empty:
        if pipe.buffer_full():
            if nonblocking: return bytes_written_or_EAGAIN
            sleep_until_not_full_or_closed()
        copy_some_bytes_into_buffer()
        wake_readers()
```

这里的 `copy_some_bytes` 说明短写是正常路径。只要缓冲区剩余空间小于请求长度，写入就可能部分完成。调用者必须把“已完成多少字节”作为状态保存，而不是假设一次 write 完成整个消息。

死锁条件与锁顺序。经典死锁需要四个条件同时成立：互斥、持有并等待、不可抢占、循环等待。内核无法完全消除前三个条件，因为对象确实需要互斥，很多资源不能强制抢占。因此工程上主要打破循环等待：给锁分层，规定全局顺序。

```
Allowed lock order:
    tasklist_lock
```

```

-> mm.lock
    -> vma.lock
        -> page.lock
superblock.lock
    -> inode.lock
        -> pagecache.lock
            -> buffer.lock
netns.lock
    -> socket.lock
        -> skb_queue.lock

```

锁顺序不是文档装饰，而要能被测试和运行时检查。简化 lockdep 可以在每个线程记录当前持有锁栈，每次获取新锁时加入一条“已持有锁 -> 新锁”的有向边；若图出现环，就报告潜在死锁。

```

on_lock_acquire(new_lock):
    for held in current.held_locks:
        graph_add_edge(held.class, new_lock.class)
        if graph_has_cycle():
            report_deadlock_risk(current, held, new_lock)
    push(current.held_locks, new_lock)

```

这个检查的成本取决于锁类数量和边数量，不能在极热路径无节制开启，但作为调试构建非常有价值。更重要的是，它把“某次测试没有卡死”升级成“锁顺序图没有出现已知环”。

7.4 测试

并发测试要覆盖正常路径、等待路径和关闭路径。只测单线程 push/pop 没有意义，因为它没有触发同步问题。

- 多生产者多消费者下，元素不丢、不重、不乱破坏队列不变量。
- 队列空时消费者等待，生产者 push 后消费者能醒来。
- 队列满时生产者等待，消费者 pop 后生产者能醒来。
- close 能唤醒所有生产者和消费者，不留下永久睡眠线程。
- timeout 或 cancel 路径不会破坏 size/head/tail。
- pipe 读写能处理 EOF、EPIPE、EAGAIN 和短读短写。
- 信号只改变允许改变的最小状态，不在 handler 中执行不安全操作。

测试报告必须写出等待谓词和唤醒来源。若测试只说“不会死锁”，但没有说明哪个线程等什么、谁负责唤醒、关闭时怎样退出，这个测试不可维护。

锁测试不能只跑正常路径。最低测试包括：

1. 互斥测试：多个 CPU 同时递增共享计数器，最终值必须等于操作次数。
2. lost wakeup 测试：让唤醒和入睡交错，等待者不能永久睡眠。
3. 中断上下文测试：持有自旋锁时触发同类中断，检查是否需要 irqsave。
4. 死锁检测测试：故意构造 A 后 B 与 B 后 A，lockdep 必须报告环。
5. 饥饿测试：大量短任务竞争 mutex，等待时间不能无限增长。
6. 退出路径测试：加锁后在每个错误分支注入失败，所有锁计数必须归零。

核心思想

本章合格标准：能说明每把锁保护哪个对象；能判断自旋锁、mutex、RCU 的上下文限制；能写出 lost wakeup 的反例和修复；能用锁顺序图解释死锁风险。

7.5 源码级补充：锁实现、IPC 与 lockdep

raw spinlock、ticket lock 与 queued spinlock。简单 test-and-set 锁在竞争下会让所有 CPU 反复写同一 cache line。ticket lock 提供 FIFO 公平，但所有等待者仍然轮询同一个 owner。queued spinlock 把等待者组织成队列，思想接近 MCS：每个 CPU 主要等待自己的节点，减少 cache line 抖动。

```
mcs_lock(lock, node):
    node.locked = true
    pred = xchg(lock.tail, node)
    if pred != null:
        pred.next = node
        while node.locked:
            cpu_relax()

mcs_unlock(lock, node):
    if node.next == null:
        if cmpxchg(lock.tail, node, null):
            return
    wait until node.next != null
    node.next.locked = false
```

Linux 的 qspinlock 比这段伪代码更紧凑，读 [kernel/locking/qspinlock.c](#) 时重点看三件事：pending bit 如何处理短竞争，MCS queue 如何承载长竞争，锁路径如何配合 preempt 和 paravirt。不要把 queued lock 理解成“更复杂所以一定更快”，它是为高竞争多核场景付出的复杂度。

rwlock、rwsem 和乐观自旋。读写锁允许多个读者并行，一个写者独占。rwlock 通常用于短临界区，不能睡眠；rwsem 是可睡眠读写信号量，适合 VMA、文件系统等较长路径。rwsem 还可能做 optimistic spinning：如果持有者正在 CPU 上运行，等待者短暂自旋，期望持有者很快释放，避免睡眠/唤醒成本。

原语	优点	风险
rwlock	读者并行，路径短	长读者或写频繁时收益差，不能睡眠
rwsem optimistic spin	可睡眠，适合较长临界区 避免短等待睡眠成本	饥饿、公平性、唤醒策略复杂 持有者不运行时浪费 CPU

读源码可看 [kernel/locking/rwsem.c](#)。关注 wait list、owner、reader count、writer handoff。若一个路径持有 rwsem 后等待另一个也需要此 rwsem 的工作完成，就容易形成隐藏死锁。

seqlock：写者优势的读一致性。seqlock 用一个递增序列号表达写入是否发生。读者开始时读取 seq，复制数据，结束时再读 seq；如果 seq 变化或为奇数，重试。写者持锁并把 seq 变奇数，写完后变偶数。

```
read:
    do {
        seq = read_seqbegin(&s)
        local = shared_data
    } while (read_seqretry(&s, seq))
```

```
write:
    write_seqlock(&s)
    shared_data = new_value
    write_sequnlock(&s)
```

seqlock 适合读多写少、读者可重试、数据可复制的场景，例如时间数据。它不适合读者不能重试、数据包含指针且对象生命周期复杂、写者频繁导致读者长期重试的场景。

per-CPU 与 preempt/migrate 控制。per-CPU 变量让每个 CPU 有自己的副本，减少锁和 cache line bouncing。但访问 per-CPU 数据时，要确保当前执行不会迁移到另一个 CPU，否则先取到 CPU0 的指针，随后迁移到 CPU1 继续使用，会破坏语义。

```
this_cpu_counter_add(delta):
    preempt_disable()
    per_cpu(counter, smp_processor_id()) += delta
    preempt_enable()
```

有些路径用 `migrate_disable` 而不是完全禁止抢占，表示任务可以被抢占但不能迁移 CPU。读调度和网络栈源码时，这类边界经常出现。per-CPU 不是无锁魔法，它把共享变成分片，同时引入汇总、CPU hotplug 和迁移限制。

内存屏障原语。

原语	语义
<code>smp_rmb</code>	约束读-读顺序
<code>smp_wmb</code>	约束写-写顺序
<code>smp_mb</code>	完整内存屏障，约束读写重排
<code>smp_store_release</code>	发布数据前的写入对获取方可见
<code>smp_load_acquire</code>	看到发布指针后能看到发布前初始化

屏障必须服务于具体协议。若无法说出“谁发布、谁获取、保护哪组字段”，就不应随便加 `smp_mb`。错误屏障可能既没修 bug，又增加维护负担。

pipefs 与 pipe_buffer。Linux 管道不是用户态数组，而是内核 pipe inode 和一组 `pipe_buffer`。每个 buffer 可以引用匿名页、page cache 页或 splice 传递的页片段。管道容量有默认值和上限，常见默认容量是 65536 字节量级，但会受内核版本和配置影响。

```
pipe_write(file, iov):
    pipe = file.private_data
    lock(pipe.mutex)
    while pipe_full(pipe):
        if nonblocking: return -EAGAIN
        wait(pipe.wr_wait, pipe.mutex)
    copy_or_splice_pages_into_pipe_buffers()
    wake_readers(pipe.rd_wait)
    unlock(pipe.mutex)
```

读源码时看 `fs/pipe.c`: `pipe_write`、`pipe_read`、`pipe_wait`、`pipe_buffer`。关注点不是函数名，而是关闭语义：写端全关且缓冲为空时读返回 EOF；读端全关时写失败并可能触发 `SIGPIPE`；非阻塞模式返回 `EAGAIN`。

System V 与 POSIX IPC。System V IPC 提供 semaphore、message queue、shared memory，典型接口是 `semget/semop`、`msgget/msgsnd/msgrcv`、`shmget/shmat`。POSIX IPC 提供命名 semaphore、POSIX message queue 和共享内存对象，接口如 `sem_open`、`mq_open`、`shm_open`。

机制	特点	适用边界
System V semaphore	内核维护计数和 undo 语义	传统系统和跨进程资源计数
POSIX semaphore	named/unnamed 两类	进程间或线程间许可控制
System V msgqueue	类型化消息，内核队列	需要消息边界但吞吐不极致
POSIX mq	描述符化，可通知	与 fd/event loop 集成更方便
shared memory	共享页，不自带同步	大数据交换，需 futex/锁配合

共享内存常和 futex 一起使用：数据放在共享页，锁字也放在共享页；无竞争时用户态原子操作完成，有竞争时 futex 让内核按这个用户地址建立等待队列。这样数据路径不必每次进入内核。

eventfd、timerfd、signalfd 和 memfd。Linux 把很多事件转成 fd，是为了让 epoll 能统一等待。

接口	内核语义	常见用途
eventfd	64 位计数器，read 消费，write 增加	线程/进程通知、io_uring、KVM kick
timerfd	定时器到期次数可读	event loop 中处理超时
signalfd	把被 mask 的信号作为 fd 读取	避免复杂 async signal handler
memfd	匿名内存文件，可 seal	共享内存、沙箱传递只读 blob

eventfd 的 EFD_SEMAPHORE 模式每次 read 只消耗 1，否则 read 返回当前计数并清零。这个小差异决定它表达的是“累计事件”还是“许可”。memfd sealing 可以禁止继续写、增长或收缩，让发送方把一段内存变成稳定对象再交给接收方。

信号投递路径。信号不是立即调用用户函数。内核先把信号加入 pending 集合；线程返回用户态前检查 pending signal；若未被屏蔽且有 handler，内核在用户栈上构造 signal frame，把返回地址改成 handler；handler 执行后通过 sigreturn 恢复原上下文。

```

send_signal(task, sig, info):
    add_to_pending(task.signal_pending, sig, info)
    if task.sleeping_interruptibly:
        wake_up(task)

return_to_user(task):
    if has_unblocked_pending_signal(task):
        build_signal_frame_on_user_stack()
        task.regs.ip = handler_address
        task.regs.sp = signal_frame_sp

```

这解释了为什么 handler 可用函数受限：它运行在异步插入的用户控制流里，可能打断 malloc、printf、锁持有路径。signalfd 的工程价值在于把异步信号改造成普通 fd 事件，降低重入风险。

IPC 性能决策树。

需求	推荐思路
少量控制事件	eventfd、pipe、socketpair
需要消息边界	Unix datagram socket、POSIX mq、自定义帧协议
大块数据同机传输	shared memory + futex/eventfd 通知
要跨机器扩展	TCP/QUIC/RPC，明确超时、重试和幂等
要把文件数据转发给 socket	sendfile、splice、zero-copy 路径
要统一事件循环	fd 化 接 □：epoll + eventfd/timerfd/signalfd

性能不是只看平均延迟。管道有内核拷贝和容量上限；共享内存吞吐高但同步复杂；socket 语义完整但协议开销高；message queue 保留边界但容量和优先级策略会影响尾延迟。选择 IPC 时要写出数据大小、频率、是否需要消息边界、是否跨主机、背压和关闭语义。

lockdep 报告怎么读。lockdep 报告通常会给出两个锁类、已观察到的获取顺序、当前试图形成的新顺序和调用栈。读报告时先找锁类名，再画边：A 后 B，B 后 A 是否形成环；然后检查是否真实同一实例、同一上下文，还是锁类标注过粗。

```
Observed order:
inode_lock -> page_lock
New order:
page_lock -> inode_lock
Result:
possible circular locking dependency
```

不要把 lockdep 当成“测试失败噪声”。它发现的是潜在等待环，可能在当前负载下未死锁，但在错误时序下会卡死。高质量内核代码要能解释每条锁边的必要性，或调整锁类/顺序消除环。

第三部分

内存管理

本部分导读

上一部分解决”谁运行、谁等待”；这一部分解决”谁能访问什么内存”。虚拟内存、page cache 和内核分配器放在一起，是因为它们都在维护授权边界和提交边界。

虚拟内存把地址访问交给 VMA/PTE/page fault 决策；`mmap` 和 page cache 把文件内容、内存页面和 I/O 提交连在一起；内核分配器从 buddy 到 slab 管理物理页和内核对象。它们看似分属不同子系统，实质上都在回答同一个问题：验证在哪里完成，临时状态放在哪里，提交点是哪一行，失败后哪些对象必须回滚。

读这一部分时要始终区分三层完成：用户 API 返回、内核对象状态更新、底层资源真正提交。

第 8 章 虚拟内存与地址空间

8.1 原理

虚拟内存把每个进程看到的地址空间和物理内存分开。进程使用虚拟页，页表把虚拟页映射到物理页，并附带 readable、writable、executable、present 等权限。一次访问是否合法，不只取决于地址是否存在，还取决于访问类型和权限位。

先看一个最容易误判的程序：

```
char* p = malloc(1 << 30); // request 1 GiB virtual range
p[0] = 1;                  // touch first page
p[4096] = 2;               // touch second page
```

很多人会把 `malloc` 成功理解成“系统已经给了 1 GiB 物理内存”。这通常不对。用户程序拿到的是一段虚拟地址范围；真正的物理页可能在第一次写入某个页时才分配。也就是说，虚拟内存首先是一套承诺和检查机制：地址范围是否被允许，访问类型是否被允许，物理页是否已经准备好，没准备好时能不能补上。

把这条路径拆开，会看到三个层次：

1. 用户代码只拿到虚拟地址，例如 `p + 4096`。
2. 内核用 VMA 记录“这段地址理论上属于堆，允许读写，可以按需分配”。
3. CPU/MMU 用页表项记录“这一页当前是否有物理页，权限是什么”。

因此“没有 PTE”和“非法地址”不是同一件事。一个地址如果不在任何 VMA 中，访问就是非法；一个地址在 VMA 中但还没有 PTE，可能只是第一次访问，需要缺页处理分配或调入页面。理解这层区别，后面的 `mmap`、COW、swap、page cache 才不会混在一起。

隔离来自默认不可见。一个进程不能随便读写另一个进程的页；没有映射的地址会触发缺页；只读页不能写；不可执行页不能取指。操作系统通过这些规则同时提供保护、共享和按需分配。

虚拟地址空间不仅是页表。内核还维护 VMA 或等价区域结构，用来描述一段虚拟地址的来源和权限：匿名内存、栈、堆、共享库、文件映射、设备映射。页表项是某一页当前是否能翻译，VMA 是这一段地址理论上是否合法、缺页时应该怎样处理。没有页表项不一定是错误，可能只是尚未按需分配。

一次访问的大致路径是：CPU 发出虚拟地址；TLB 命中则直接得到物理地址和权限；TLB 未命中则硬件或内核进行页表遍历；页表项不存在或权限不符时触发异常；内核根据 VMA 判断是合法缺页、权限错误还是非法访问；合法缺页可能分配物理页、从文件读页、建立 COW 私有页或映射零页。

这条路径可以用一个具体地址走一遍。假设页大小是 4096，进程访问 `0x401234`：

```
virtual address = 0x401234
virtual page    = 0x401
offset in page  = 0x234

find VMA containing 0x401234
yes: [0x400000, 0x500000) readable executable file mapping

look up PTE for virtual page 0x401
present: physical page 0x83000, readable executable

physical address = 0x83000 * 4096 + 0x234
```

若 PTE 不存在，内核不会立刻把程序杀掉，而是先问 VMA：这是不是文件映射的合法页？是不是栈增长？是不是匿名页按需分配？若 VMA 也找不到，才是典型的

非法访问。若 VMA 找到了但访问类型不允许，例如往只读代码页写，就是权限错误。缺页异常只是入口，真正结论由 VMA、PTE、访问类型和错误码共同决定。

copy-on-write 是虚拟内存最有代表性的协议。fork 时，父子进程可以先共享物理页，并把页表标记为只读和 COW。任何一方写入时触发保护异常，内核复制物理页，更新写入方页表，再继续执行。这样 fork 不需要立刻复制整个地址空间，但必须正确维护引用计数和权限。

核心思想

虚拟内存不是“把地址翻译成另一个地址”这么简单。它同时维护三件事：哪些虚拟区间被承诺，当前哪些页已经物化成物理页，每次读写执行是否被授权。

8.2 实践

实验的 PageTable 支持按页映射。读访问要求 present 和 readable；写访问要求 writable；执行访问要求 executable。地址翻译保留页内 offset，帮助读者理解“页号转换，页内偏移不变”。

输入	计算
page size	4096
virtual address	0x401234
virtual page number	0x401
offset	0x234
physical page number	查页表得到
physical address	$\text{physical_page} * 4096 + \text{offset}$

权限矩阵要单独写：

访问	present	readable	writable	executable	结果
load	yes	yes	no	no	成功
store	yes	yes	no	no	权限错误
fetch	yes	yes	no	no	NX 错误
fetch	yes	yes	no	yes	成功
load	no	-	-	-	缺页或非法

观察真实系统时，要区分虚拟大小、RSS、page cache、minor fault 和 major fault。虚拟地址空间很大，不代表物理内存已经占用；RSS 增加，可能是匿名页，也可能是文件页；minor fault 可能只是建立映射，major fault 更可能等待存储设备。

8.3 算法与实现分析

物理页分配：free list、bitmap 和 buddy。最简单的物理页分配器是 free list。每个空闲页框放进链表，分配时弹出一个，释放时放回去。它的分配和释放都是 $O(1)$ ，适合固定大小页框；缺点是难以快速找到连续页，也难以检测重复释放和碎片分布。

```

page_alloc():
    if free_list.empty(): return null
    page = free_list.pop()
    page.refcount = 1
    return page

page_free(page):

```

```
assert(page.refcount == 0)
free_list.push(page)
```

bitmap 分配器用一个 bit 表示一个页框是否空闲。它节省元数据，容易扫描连续区域，但朴素扫描是 $O(n)$ 。可以通过分层 bitmap 或缓存上次搜索位置降低平均成本。

buddy allocator 用 2 的幂大小管理连续页块。分配 8 页时，寻找 order=3 的块；如果只有更大的块，就不断拆分；释放时，如果相邻 buddy 也空闲，就合并成更大块。buddy 的分配和释放大约是 $O(\log N)$ ，能较好处理连续页需求，但仍会产生内部碎片。

虚拟地址查找：VMA 和页表。页表回答“这一页当前映射到哪里”，VMA 回答“这一段地址理论上是否合法”。缺页时，内核先查 VMA：若地址不属于任何 VMA，就是非法访问；若属于可按需分配的匿名 VMA，就分配零页；若属于文件映射 VMA，就从 page cache 找页或发起 I/O。

VMA 可以用有序区间结构保存。查找地址属于哪个区间，朴素线性扫描是 $O(n)$ ；平衡树查找是 $O(\log n)$ ；现代内核还会使用更适合区间查询和缓存局部性的结构。教学模型可以先用有序数组，因为它能把语义讲清楚。

```
handle_page_fault(addr, access):
    vma = find_vma(addr)
    if vma == null: signal(SIGSEGV)
    if !vma.allows(access): signal(SIGSEGV)
    page = materialize_page(vma, addr)
    install_pte(addr, page, vma.permissions)
```

COW 的引用计数。COW 的不变量是：共享页必须只读，物理页有引用计数；写入共享页时，写入方得到新副本。若 refcount 为 1，可以直接把页改成可写；若 refcount 大于 1，必须复制。

```
handle_write_fault(vaddr):
    pte = lookup_pte(vaddr)
    page = pte.page
    if !pte.cow: signal(SIGSEGV)
    if page.refcount == 1:
        pte.writable = true
        pte.cow = false
    else:
        new_page = alloc_page()
        copy_page(new_page, page)
        page.refcount -= 1
        install_pte(vaddr, new_page, writable=true, cow=false)
```

这段算法最容易错在引用计数和权限更新顺序。若忘记把共享页设为只读，写入不会触发 fault，父子进程会互相污染；若复制后忘记减少旧页引用计数，会泄漏；若 TLB 中仍缓存旧权限，必须刷新或失效对应条目。

用一个父子进程的实际页来走一遍。父进程有一页虚拟地址 `0x7000`，物理页是 `P42`，内容是字符串 `"abc"`。`fork` 前，父进程 PTE 指向 `P42`，权限可读可写，`P42.refcount = 1`。`fork` 时，内核不复制 `P42` 的 4096 字节，而是让父子页表都指向同一个 `P42`，同时把两个 PTE 都改成只读，并在软件元数据里标记 COW，`P42.refcount = 2`。

时刻	父进程 PTE	子进程 PTE
fork 前	<code>0x7000 -> P42, writable</code>	不存在
fork 后	<code>0x7000 -> P42, read-only, COW</code>	<code>0x7000 -> P42, read-only, COW</code>

子进程写入 `P42.refcount = 2` store 触发写保护 fault
 前
 子进程仍指向 `P42`, `read-only` 指向新页 `P99`, `writable`
 fault 后 `y, COW`
 后续父进程若 `P42.refcount = 1` 可直 已经和父进程分离
 写入 接恢复 `writable`

关键点是“写入必须在 store 提交前被拦住”。CPU 执行子进程的 `*p = 'x'` 时, 地址翻译发现 PTE present 但不可写, 于是保存 faulting address、错误码和 RIP 进入内核。因为 store 还没有修改 `P42`, 内核才能安全复制旧页到 `P99`, 把子进程 PTE 改为 `P99, writable`, 再返回原指令重试。重试后写入落在 `P99`, 父进程继续看到旧的 “abc”。

失败路径也要讲清楚。若 fault 地址不在允许写的 VMA 内, 不能执行 COW, 而应给进程发段错误。若分配 `P99` 失败, 可能触发回收或 OOM。若更新 PTE 后没有失效旧 TLB, CPU 可能继续使用旧只读翻译, 导致重复 fault; 更危险的是撤销可写权限时没有失效旧可写翻译, 会让某个 CPU 绕过新的只读保护继续写共享页。COW 的正确性不是“复制一页”这么简单, 而是 VMA 权限、PTE 权限、物理页引用计数、TLB 失效和 fault 可重试性共同成立。

页面置换。当物理内存不足时, 内核要回收页面。理想算法 OPT 总是淘汰未来最晚再访问的页面, 但它需要知道未来, 不能实现。LRU 淘汰最近最久未使用的页, 更接近实际局部性, 但精确 LRU 维护成本高。Clock 算法用访问位近似 LRU: 扫描环形列表, 访问位为 1 就清零并跳过, 访问位为 0 就淘汰。

```
clock_evict():
    while true:
        page = clock_hand.page
        if page.referenced:
            page.referenced = false
            advance(clock_hand)
        else if page.dirty:
            schedule_writeback(page)
            advance(clock_hand)
        else:
            remove_and_return(page)
```

置换策略必须和文件系统、匿名页、swap、dirty writeback 结合。干净文件页可以直接丢弃, 未来再从文件读; 脏文件页要先写回; 匿名页若要回收, 需要 swap 或压缩; 被锁定或正在 I/O 的页不能随便回收。

多级页表。64 位地址空间太大, 不能为每个虚拟页都预留一个线性页表。多级页表把虚拟页号拆成多段索引, 只为实际使用的区域分配中间页表页。以四级页表为例, 虚拟地址被拆成 `L4`、`L3`、`L2`、`L1` 和页内 offset。

```
walk_page_table(root, vaddr, create):
    table = root
    for level in [L4, L3, L2]:
        index = index_for(level, vaddr)
        if table[index] not present:
            if not create:
                return null
            table[index] = allocate_page_table()
            table = table[index].next_table
    return &table[index_for(L1, vaddr)]
```

查找复杂度按层数是常数, 但每层都可能带来内存访问。TLB 的价值就在于缓存

最近翻译，避免每次访问都走多级页表。页表设计还影响内核内存占用：稀疏地址空间只分配用到的页表页，密集映射则消耗更多元数据。

TLB shutdown。改变页表后，CPU TLB 中可能仍缓存旧翻译。单核系统可以本地失效；多核系统中，同一地址空间可能正在其他 CPU 运行，需要发送 shutdown IPI 让它们失效对应条目。

```
unmap_page(mm, vaddr):
    pte = walk_page_table(mm.root, vaddr, false)
    old = clear_pte(pte)
    cpus = mm.active_cpus
    for cpu in cpus except current_cpu:
        send_tlb_shutdown(cpu, mm, vaddr)
    local_invlpg(vaddr)
    wait_for_shutdown_ack(cpus)
    put_page(old.page)
```

顺序很重要。若在其他 CPU 失效 TLB 前就释放物理页，该页可能被重新分配给别的对象，而旧 CPU 仍通过过期 TLB 写入它，形成严重内存破坏。TLB shutdown 是虚拟内存和多核同步的交叉点。

再把这个危险展开成具体事故。进程 X 有虚拟页 `0x9000 -> P10`，它的两个线程分别在 CPU0 和 CPU1 上运行。CPU1 的 TLB 里已经缓存了这条翻译。CPU0 执行 `munmap(0x9000)`，把页表项清掉，然后若立刻把 `P10` 释放给物理页分配器，分配器可能马上把 `P10` 交给内核网络栈当作 packet buffer。此时 CPU1 如果还没收到 shutdown，仍可用旧 TLB 把用户态 `0x9000` 翻译到 `P10`，一次普通 store 就会写坏网络 buffer。这个 bug 表面可能表现成网络包校验失败、随机内核崩溃或安全越权，根因却是页表撤销和 TLB 旧副本之间的生命周期错误。

因此 shutdown 的完成点必须在“物理页可重用”之前。教学模型可以记成三步：先让新页表状态不可再产生新翻译；再让所有可能持有旧翻译的 CPU 丢掉旧 TLB；最后释放旧物理页或改变其用途。

阶段	必须完成的事	如果提前进入下一步
清 PTE	后续 page walk 不能再得到旧映射	新 TLB 项仍可能被硬件装入
本地失效	当前 CPU 不再使用旧翻译	当前线程可能继续访问已撤销页
远端 IPI	其他 active CPU 执行 <code>invlpg</code> 或等价操作	远端 CPU 可能写入已释放物理页
等待 ack	确认旧翻译不可再被使用	释放页会造成 use-after-free 到物理内存
释放旧页	页框回到分配器或引用计数下降	到这一步才安全改变页用途

这也是为什么 `mprotect`、`munmap`、COW 和进程退出在多核上不是“改一张表”就结束。内核还要知道哪些 CPU 可能正在运行这个地址空间，哪些映射范围需要 flush，能否批量合并 shutdown，是否允许延迟释放。优化可以减少 IPI 次数，但不能破坏上面的安全边界。

overcommit 与 OOM。虚拟内存允许系统承诺比物理内存更多的虚拟地址。例如 `mmap` 或 `malloc` 成功不代表物理页已经分配，首次写入才可能触发分配。overcommit 提高灵活性，但内存真正不足时必须选择失败策略。

```
handle_anon_fault(vma, addr):
    page = alloc_page()
```

```

if page == null:
    victim = oom_select_process()
    if victim == null:
        return SIGBUS_OR_ENOMEM
    kill(victim)
    retry allocation
zero_page(page)
install_pte(addr, page, WRITE | USER)

```

OOM killer 不是随意杀进程。它通常综合内存占用、权限、oom score、是否关键服务、cgroup 限制和可回收性。容器场景下，cgroup 内 OOM 应优先限制在该 cgroup 内，而不是扩大到整个宿主。

把 overcommit 放进一个常见服务事故。机器有 8GiB 内存，服务 A 启动时 `malloc(6GiB)` 做缓存索引，服务 B 也 `malloc(6GiB)` 准备批处理。两个 `malloc` 都可能成功，因为内核只是给它们各自建立虚拟区间和 VMA，并没有立即分配 12GiB 物理页。监控里 VSZ 很大，但 RSS 还不高。事故发生在两者开始真正写入页面时：

```

for each 4096-byte page in allocated range:
    p[i] = 0;

```

每写一个尚未物化的匿名页，CPU 都会触发缺页；内核确认 VMA 允许写，分配物理页，清零，安装 PTE，然后重试 store。刚开始这很顺利；随着空闲页减少，内核先回收干净文件页，再尝试写回脏页，再考虑 swap 或压缩；若仍不能满足分配，就进入 OOM 决策。此时失败点不是 `malloc`，而是某一次普通 store 的缺页处理。

阶段	内核状态	应用看到什么
<code>malloc</code> 成功	VMA/堆区扩展，物理页可能未分配	返回非空指针
首次写前几页	匿名页缺页，分配物理页并清零	store 正常完成，只是可能变慢
内存压力上升	回收 page cache、写回脏页、触发 swap	延迟抖动，RSS 上升
无法回收	OOM 在进程或 cgroup 内选择受害者	某进程被杀，或 fault 返回失败语义
cgroup 限制命中	只在该资源组内回收和 OOM	宿主其他服务不应被拖下水

这解释了为什么服务不能只用“`malloc` 成功”作为容量保证。真正可靠的设计要设置 cgroup memory.max、应用层缓存上限、分批预热、失败回退和监控 RSS/PSI。overcommit 的价值是让稀疏使用的大地址空间可行；代价是失败可能延迟到第一次触碰具体页面时才出现。

8.4 测试

测试要覆盖成功翻译、写只读页失败、访问未映射页失败、unmap 后访问失败、页内 offset 保留。错误结果必须携带可读诊断，不能只返回一个空值。

- `fork` 后父子共享只读 COW 页，任一方写入后得到私有副本。
- `mmap` 文件页首次访问触发填页，第二次访问走热路径。
- 栈增长必须受上限约束，不能无限扩张。
- `mprotect` 改变权限后，旧访问策略失效。
- `munmap` 后再次访问必须失败。
- 用户态指针传入系统调用时，内核必须检查可读/可写范围。

权限测试尤其重要。没有权限测试的虚拟内存模型，会把地址转换误写成普通哈希表查找，丢掉操作系统最重要的保护语义。

本章压缩进来的内存工程账本。虚拟内存的细节很多：page fault、VMA 切分、PTE 标志、TLB shutdown、COW、swap、THP、mlock、PSI、OOM 和 cgroup。主线里先把它们压成一张“缺页账本”。每次 CPU 访问地址失败，内核都要回答：地址属于哪个 VMA，访问类型是否被允许，页面内容从哪里来，PTE 怎样安装，旧 TLB 是否要失效，失败时返回信号还是 errno，线程是否需要睡眠等待 I/O 或回收。

场景	内核主要动作	关键风险
匿名首次写	分配物理页、清零、安装 writable PTE	overcommit 后在首次触碰时 OOM
COW 写 fault	检查引用、复制页面、替换 PTE、flush TLB	父子进程互相污染或复制后权限错误
文件映射 fault	查 page cache、必要时发起读 I/O、安装映射	把 I/O 等待误判成 CPU 慢
mprotect	VMA 切分、改权限、更新页表、失效 TLB	旧 TLB 保留过宽权限
内存回收	扫 LRU、写回脏页、swap out、唤醒等待者	回收不可回收页或引发严重抖动

读源码时可以按对象追踪：`mm_struct` 是进程地址空间账本，VMA 是区间权限和来源账本，PTE 是硬件当前可执行的翻译账本，`struct page/folio` 是物理页生命周期账本，`rmap` 把物理页反查到映射它的地址空间。任何优化都不能跳过这些账本的一致性。THP 减少 TLB 压力，但拆分时仍要恢复普通页语义；swap 降低内存压力，但 fault 延迟会进入请求尾部；mlock 保护实时路径或密钥页，但会减少可回收集合；PSI 不解决内存压力，只告诉你线程正在因为内存等待而失去前进能力。

内核自己的内存分配也属于这张账本。页分配器负责按页框分配和回收，buddy 系统处理连续页块；slab/SLUB 把页切成固定大小对象，服务 `task_struct`、inode、dentry、socket 等高频对象；`vmalloc` 提供虚拟连续但物理不连续的内核映射。教学实现可以先只有 page allocator 和几个对象池，但必须保留三个边界：中断上下文不能随意睡眠等待内存，DMA buffer 可能要求物理连续或满足设备地址限制，对象缓存释放后不能仍被 RCU 读者或异步 completion 引用。

分配器	适合的对象	验收问题
页分配器	页表页、大块缓冲、匿名页和 page cache 页	压力下是否能回收、写回或明确失败
SLUB/slab	大量同类型小对象	析构、引用计数、越界和 use-after-free 是否可检测
vmalloc	大型内核虚拟连续缓冲	不能交给要求物理连续 DMA 的设备
per-CPU allocator	每 CPU 统计、队列和热路径缓存	CPU 热插拔和迁移时账本是否一致

本章验收问题：给一个虚拟地址和一次读/写/取指访问，能否说明它命中哪个 VMA、PTE 应有哪组权限、缺页后页面从哪里来、是否可能睡眠、失败给用户什么信号或错误、哪些观测指标能证明判断。能回答这些，才算把虚拟内存读成 OS 机制，而不是地址转换表。

8.5 源码级补充：PTE 标志、THP、swap、mlock 与 PSI

页表项标志。x86-64 PTE 典型标志包括 `present`、`writable`、`user`、`accessed`、`dirty`、`global`、`NX` 等。操作系统把 VMA 权限、文件打开模式、COW 状态和硬件标志合成 PTE。注意 COW 不是单纯硬件位，通常由软件在只读 PTE 和附加元数据中表达。

标志	语义
<code>present</code>	PTE 可用于地址翻译，否则触发缺页
<code>writable</code>	store 是否允许
<code>user</code>	ring 3 是否可访问
<code>accessed</code>	CPU 访问后置位，用于回收近似 LRU
<code>dirty</code>	CPU 写入后置位，用于写回和 COW 判断
<code>NX</code>	禁止取指执行

权限收缩必须刷新 TLB。例如把页从可写改成只读，如果旧 TLB 仍保留可写权限，写入可能绕过新的 COW 或 `mprotect` 规则。

`mprotect`、`munmap` 和 VMA 切分。`mprotect` 改变区间权限，`munmap` 删除映射。二者都可能把一个 VMA 切成三段：前半保持原权限，中间修改或删除，后半保持原权限。

```
before:
    [0x1000, 0x9000) RW
mprotect [0x3000, 0x5000) R
after:
    [0x1000, 0x3000) RW
    [0x3000, 0x5000) R
    [0x5000, 0x9000) RW
```

VMA 切分后，要更新页表权限、失效 TLB、处理锁定页和文件映射引用。教学实现若只改一个 `flags` 字段，会在部分区间修改时出错。

Transparent Huge Page。普通页常为 4KiB，大页可为 2MiB 或更大。THP 尝试把连续匿名页合并成 PMD 级大页，减少 TLB miss。代价是分配连续内存更难，拆分和 COW 成本更高，延迟可能出现尖峰。

```
khugepaged:
    scan VMAs marked hugepage-friendly
    find 512 contiguous 4KiB pages
    allocate 2MiB huge page
    copy or remap contents
    replace PTE table with PMD huge mapping
```

THP 是性能优化，不应改变用户语义。发生 `mprotect`、`munmap`、COW 或内存压力时，大页可能被拆回普通页。性能报告要说明 THP 是否开启，否则 TLB 和内存延迟数据难以比较。

swap cache 和 swappiness。匿名页回收需要把内容写到 swap。swap slot 是匿名页在交换区的位置；swap cache 避免同一 swap 页被重复读写。swappiness 影响内核在文件页和匿名页之间的回收倾向。

```
swap_out(page):
    slot = allocate_swap_slot()
    write_page_to_swap(slot, page)
    replace_pte_with_swap_entry(vaddr, slot)
    free_physical_page(page)

swap_in(fault):
```

```
slot = fault.swap_entry
page = swap_cache_lookup_or_read(slot)
install_pte(fault.addr, page)
```

swap 让系统在内存压力下继续运行，但延迟可能变得非常高。服务端系统常用 PSI、cgroup memory.high 和限流来避免进入严重 swap 抖动。

mlock 和不可回收页。mlock 把页锁在内存中，常用于实时路径、密码材料或避免 page fault 的场景。锁定页不能被普通回收换出，因此必须受权限和 rlimit 限制。

```
mlock(addr, len):
    check RLIMIT_MEMLOCK
    for each VMA page in range:
        fault page in if needed
        mark unevictable
```

滥用 mlock 会减少可回收内存，影响整机。教学 OS 要把“不可回收”作为页面状态加入回收测试，证明回收器不会错误丢弃 locked page。

系统调用与内存深讲：一次请求如何安全穿过边界

系统调用和内存管理是 OS 隔离能力的核心。系统调用控制“用户能请求什么”，页表控制“用户能访问什么”。二者常常一起出现：用户把指针传给系统调用，内核必须检查这段用户内存；缺页可能在系统调用复制参数时发生；exec 会替换地址空间并重建用户栈。

为什么用户指针不能直接解引用。

用户传入的地址可能不存在、只读、跨页、被另一个线程并发修改，甚至在检查后被 munmap。内核若直接解引用，轻则崩溃，重则形成安全漏洞。因此内核要用专门的 copy 接口，并准备处理 copy 期间的 page fault。

```
sys_write(fd, user_buf, len):
    file = lookup_fd(fd)
    if file == null:
        return -EBADF
    if not file.can_write:
        return -EBADF
    kbuf = allocate_temp(min(len, MAX_RW_COUNT))
    ret = copy_from_user(kbuf, user_buf, len)
    if ret < 0:
        return -EFAULT
    return vfs_write(file, kbuf, len)
```

注意顺序：先查 fd 和写权限，再复制用户数据；复制失败不能留下半提交文件状态。对很大的写入，真实系统不会一次复制全部，而是分段处理；每段成功后返回可能是短写，调用者必须循环。

page fault 的三种结论。

同样是 page fault，内核结论可能完全不同。

结论	条件	处理
合法缺页	地址属于 VMA，权限允许，只是 PTE 不存在	分配页、读文件页或建立 COW 页
权限错误	地址属于 VMA，但访问类型不允许	发送 SIGSEGV 或返回 EFAULT
非法地址	不属于任何 VMA	发送 SIGSEGV 或返回 EFAULT

区别取决于上下文。用户态普通 load/store 错误通常给进程发信号；内核在

`copy_from_user` 中遇到用户地址错误，通常让系统调用返回 `EFAULT`。内核访问自己错误地址则是内核 bug，不能当普通用户错误处理。

fork、COW 与 exec 的组合。

shell 启动程序常见流程是 fork 后在子进程中设置 fd，再 exec。若 fork 立刻复制整个地址空间，启动大进程会很慢。COW 让父子先共享只读页，只有写入时才复制。exec 成功后，子进程旧地址空间被替换，很多 COW 页根本不用复制。

```
parent shell:
    pid = fork()

child after fork:
    dup2(file, STDOUT)
    execve(program)

kernel optimization:
    fork shares pages as read-only COW
    exec discards old user mappings
    only pages written before exec are copied
```

这说明机制之间是组合出来的。fork 的语义简单，COW 让它便宜，exec 让它变成启动新程序，fd 继承让 shell 能做重定向。很多 OS 设计的美感来自小机制组合，而不是一个巨大的 spawn 黑盒。

内存回收为什么会影响系统调用延迟。

系统调用可能看起来只是写 socket 或打开文件，却在中途分配内存。若内存不足，分配器可能触发回收；回收可能写回脏页；写回可能等待块设备；块设备完成再唤醒当前线程。于是一个普通请求的尾延迟可能来自内存和 I/O，而不是业务代码。

```
sys_send()
-> allocate socket buffer
-> memory pressure
-> reclaim pages
-> dirty page writeback
-> block I/O wait
-> scheduler wakeup
-> return to send path
```

这也是为什么真实性能分析要看 off-CPU 时间、page fault、writeback 和 block I/O。只看 CPU profiler 会漏掉大部分等待。

系统调用与内存练习。

1. 为什么 `copy_from_user` 失败应该返回 `EFAULT`，而不是让内核 panic？
2. fork 后父子进程共享 COW 页，父进程写入一页后，页表和物理页引用计数怎样变化？
3. 为什么 exec 失败前应尽量保留旧地址空间可运行？
4. mmap(MAP_PRIVATE) 写入文件映射页后，文件内容为什么不变？
5. 为什么内存压力会让看似无关的系统调用变慢？

系统调用错误矩阵。

高质量系统调用实现要能区分不同失败原因。把所有失败都返回一个通用错误，会让调用者无法恢复。

错误	典型原因	调用者策略
----	------	-------

EBADF	fd 不存在或模式不允许	程序 bug 或句柄生命周期错误
EFAULT	用户指针不可访问	程序 bug，不能重试同一指针
EINTR	阻塞调用被信号打断	根据已完成字节数决定是否重试
EAGAIN	非阻塞对象暂时不能推进	等 readiness 后重试
ENOMEM	内存不足或限制触发	降级、释放资源或失败返回
EIO	设备或底层 I/O 错误	上报、切换副本、进入恢复流程
EACCES	权限不足	请求授权或拒绝操作

错误码本身就是 API 的一部分。比如 `EAGAIN` 表示状态可能稍后改变，`EFAULT` 表示调用者传了坏地址，二者的恢复策略完全不同。

页表权限和文件权限的区别。

初学者容易混淆“我能打开文件”和“我能访问内存”。文件权限是文件系统对象上的访问控制；页表权限是 CPU 执行每次内存访问时检查的硬件权限。二者可以关联，但不是同一个东西。

```
open file read-only:
    file.can_write = false

mmap file private writable:
    VMA may allow write
    first write creates anonymous COW page
    file content is not modified

mmap file shared writable:
    fd must allow write
    dirty page belongs to file mapping
    msync/fsync affects persistence
```

这就是为什么 `MAP_PRIVATE` 可以让只读文件映射在进程内“可写”：写入的是私有匿名副本，不是文件本体。反过来，`MAP_SHARED` 可写映射必须受文件打开模式和文件系统权限限制。

本章小结。

- 系统调用跨越信任边界，必须检查 fd、权限、用户指针和对象状态。
- page fault 不是单一错误；它可能是合法懒加载、权限错误或非法地址。
- fork、COW、exec、fd 继承共同组成高效进程启动模型。
- 内存压力可能把普通系统调用拖入回收和 I/O 等待。
- 错误码要表达恢复策略，而不是只表达“失败了”。

系统调用入口的生命周期。

系统调用入口不是简单跳到一个 C 函数。CPU 从用户态进入内核后，内核要切换到受信任内核栈，保存用户寄存器，建立当前线程的系统调用上下文，验证系统调用号和参数，再调用具体实现。返回前还要处理信号、抢占、审计、ptrace/seccomp 等边界。

```
syscall_entry:
    switch_to_kernel_stack()
    save_user_registers()
    nr = user_registers.syscall_number
```

```

if seccomp_denies(nr):
    return_error_or_kill()
ret = syscall_table[nr](arg0, arg1, ...)
if signal_pending(current):
    prepare_signal_frame_or_restart()
if need_reschedule:
    schedule()
restore_user_registers(ret)
return_to_user()

```

这条路径解释了几个现象。系统调用可能被信号打断，也可能在返回用户态前被抢占；同一个系统调用在 tracing 或 seccomp 下成本不同；错误返回不是异常抛出，而是 ABI 规定的返回值和错误码。系统调用接口必须稳定，因为它是用户程序和内核之间的长期二进制契约。

用户指针的 TOCTOU 问题。

即使内核先检查用户地址范围，再复制数据，也不能假设这段内存存在检查后不变。另一个线程可能修改 buffer、调用 `munmap`，或改变内存内容。正确模型是：检查只能证明“此刻看起来可访问”，真正复制必须能处理 `fault`；若需要稳定数据，内核要复制到内核缓冲区后再验证和使用。

```

bad_path(user_header):
    if validate_user_header(user_header):
        // user thread may change fields here
        use_header_fields_directly(user_header)

better_path(user_header):
    hdr = copy_from_user(user_header, sizeof(Header))
    if !validate_header(hdr):
        return -EINVAL
    use_private_copy(hdr)

```

不是所有用户数据都必须一次复制完。大 I/O 通常分段复制或使用 pinned pages、iov iterator、zero-copy 机制。但无论哪种方式，所有权和生命周期都要写清楚：谁能修改这段内存，设备或内核何时完成访问，失败时如何解除 pin 或引用。

VMA、PTE 和 page cache 的三层关系。

很多虚拟内存错误来自混淆三层对象。VMA 是虚拟地址区间的策略；PTE 是某个虚拟页当前的硬件映射；page cache 是文件内容在内存中的缓存页。文件 `mmap` 首次访问时，VMA 说明这段地址来自哪个文件和偏移；page cache 提供或创建对应页；PTE 把该页映射进进程地址空间。

对象	回答的问题	典型变化
VMA	这段地址是否合法，权限和来源是什么	<code>mmap/munmap/mprotect</code> 改变
PTE	这一页现在是否可由硬件访问	缺页、换出、COW、 <code>unmap</code> 改变
page cache page	文件某偏移的内容是否在内存	<code>read</code> 、 <code>mmap fault</code> 、 <code>writeback</code> 、 <code>reclaim</code> 改变

因此，没有 PTE 不一定是错误；可能是合法 VMA 的懒加载。没有 VMA 通常才是非法地址。page cache 中有页，也不代表某进程已经有 PTE；多个进程可以通过不同 PTE 映射同一个 page cache 页。

缺页处理的完整分支。

缺页处理要先分类，再处理。分类至少包括：地址是否在用户范围，是否属于

VMA，访问权限是否允许，PTE 是否存在但权限不符，是否 COW，是否文件页，是否栈增长，是否内核 copy 用户数据时发生。

```
handle_fault(mm, addr, access, context):
    if !is_user_range(addr):
        return fault_result(context, BAD_ADDRESS)
    vma = find_vma(mm, addr)
    if vma == null:
        return fault_result(context, NO_VMA)
    if is_stack_growth(vma, addr):
        expand_stack_or_fail(vma, addr)
    if !vma.allows(access):
        return fault_result(context, PERMISSION)
    pte = walk_pte(mm, addr, create=true)
    if pte.present and access.write and pte.cow:
        return handle_cow_fault(pte)
    if !pte.present:
        return materialize_page(vma, addr, access)
    return fault_result(context, PERMISSION)
```

`fault_result` 取决于上下文。用户态访问失败通常发信号；系统调用复制用户内存失败通常返回 `EFAULT`；内核访问内核坏地址通常 `panic`。把这些路径混成一个错误码，会掩盖边界。

页面回收的实际顺序。

内存回收不是随机找页释放。回收器通常先尝试容易回收的页：干净文件页可以直接丢；脏文件页要写回；匿名页需要 swap 或压缩；被 `mlock`、DMA pin、正在 I/O 或引用计数异常的页不能回收。

```
reclaim_scan():
    for page in inactive_list:
        if page.pinned or page.under_writeback:
            skip(page)
        else if page.clean and page.file_backed:
            remove_from_page_cache(page)
            free(page)
        else if page.dirty and page.file_backed:
            start_writeback(page)
        else if page.anonymous and swap_available:
            write_to_swap(page)
        else:
            keep_or_escalate_pressure()
```

这个顺序解释了为什么内存压力会传导到存储 I/O。系统缺内存时，回收器可能把脏页写回设备；设备慢时，回收也慢；回收慢又会让分配路径阻塞，最终表现为系统调用尾延迟升高。

TLB、PCID 和 shutdown 成本。

TLB 缓存虚拟地址翻译。切换地址空间时，若没有 x86-64 PCID 这样的标签，CPU 需要清空大量 TLB 项；有 PCID 时，可以保留不同地址空间的翻译，但页表修改仍要失效相关条目。多核下，其他 CPU 可能正在用同一地址空间运行，必须通过 IPI 通知它们失效。

操作	必要性	成本来源
local invalidate	当前 CPU 旧翻译不能继续用	pipeline/TLB 扰动
remote shutdown	其他 CPU 旧翻译不能继续用	IPI、等待 ack、跨核同步

batch unmap	合并多个失效请求	延迟释放与复杂生命周期
PCID	减少切换地址空间的全局 flush	标识管理、复用和一致性规则

这也是为什么高频 `mmap/munmap/mprotect` 可能拖慢多线程程序。它们不是只改一张表，还可能引发跨 CPU 协议。优化思路通常是批量映射、复用区域、减少权限翻转和避免在热路径频繁改变页表。

分配器层次：页、slab 和用户堆。

内核内存分配有层次。页分配器管理物理页框；slab/slub 等对象分配器管理固定大小内核对象；用户态 `malloc` 在进程地址空间内管理堆，并通过 `brk/mmap` 向内核申请更大区域。

分配层	管理对象	典型问题
page allocator	物理页和连续页块	外部碎片、NUMA、本地性
slab allocator	inode、task、dentry 等固定对象	对象复用、构造析构、缓存热度
vm allocator	内核虚拟连续区域	页表开销、TLB、地址空间碎片
user malloc	用户堆小对象	arena、碎片、RSS 与虚拟大小差异

理解层次能避免错误诊断。用户进程 RSS 增长不一定是内核泄漏；slab 增长可能来自 dentry/inode cache；虚拟地址空间变大不一定占用物理页；页分配失败可能是连续块碎片而不是总内存为零。

exec 的失败原子性。

`execve` 成功后旧程序消失；失败时通常应让调用进程还能继续运行并看到错误码。这要求内核在替换地址空间时谨慎排序。若过早销毁旧地址空间，然后发现 ELF 无效或无法分配栈，就可能让进程处于不可恢复半状态。

```
execve(path, argv, envp):
    file = open_executable(path)
    image = validate_elf(file)
    new_mm = build_new_address_space(image, argv, envp)
    if any_step_failed:
        destroy_partial_new_mm()
        return -errno
    old_mm = current.mm
    current.mm = new_mm
    switch_page_table(new_mm)
    destroy_old_mm(old_mm)
    start_at_entry(image.entry)
```

真实实现还要处理解释器脚本、动态链接器、setuid、安全标签、fd 的 `CLOEXEC`、信号处理器重置和多线程 `exec` 语义。核心原则仍然是：新镜像完全准备好之前，不要破坏失败后必须保留的旧执行环境。

内存与系统调用测试矩阵。

测试	触发方式	期望
坏用户读指针	<code>write(fd, bad, len)</code>	返回 <code>EFAULT</code> ，内核不崩溃
坏用户写指针	<code>read(fd, bad, len)</code>	返回 <code>EFAULT</code> 或短读语义清楚

COW 写入	fork 后父子分别写同页	内容分离，引用计数正确
权限缺页	写只读映射或执行 NX 页	信号或错误，不能绕过权限
munmap 竞态	复制期间另线程 unmap	返回错误或短完成，不 use-after-free
TLB 失效	mprotect 后旧权限访问	旧权限不能继续生效
内存压力	限制内存后触发分配	有 OOM/ENOMEM 证据和恢复路径

这些测试共同证明：边界错误不会变成内核错误，权限变化能真正影响硬件访问，资源不足有可解释失败，部分完成语义被调用者看见。

第一阶段源码走读：x86-64 #PF 到 VMA、PTE 与 COW

前面已经讲过虚拟地址、VMA、PTE、TLB 和 COW，但只讲概念还不够。真正读懂操作系统，必须能把一次用户态 load/store 失败拆成硬件异常、入口现场、VMA 判定、PTE 状态、缺页修复、TLB 失效和返回重试。x86-64 上这条路径的核心异常是 #PF，Linux 里它从 `arch/x86/mm/fault.c` 进入，再落到 `mm/memory.c` 的通用内存管理逻辑。

核心思想

#PF 不是“程序访问错了内存”的同义词。它只是 CPU 告诉内核：“这次地址翻译或权限检查没有按当前页表完成。”内核必须结合 CR2、error code、当前上下文、VMA 和 PTE，才能决定是分配零页、读文件页、执行 COW、返回 `-EFAULT`，还是给进程发送信号。

硬件交给内核的最小证据。

x86-64 上发生 page fault 时，CPU 至少提供三类证据：faulting virtual address、错误码和被打断现场。faulting address 由 CR2 保存；错误码说明这是不是 present 页上的保护错误、是不是写访问、是不是用户态访问、是不是取指访问等；入口保存的寄存器现场说明 fault 发生在什么 RIP、什么 RSP、什么 CPL。

证据	典型来源	OS 需要回答的问题
fault address	CR2	这个地址属于哪个 VMA，是否 canonical
error code.P	page fault error code	是 PTE 不存在，还是 present 页权限拒绝
error code.W/R	page fault error code	是读、写还是取指触发
error code.U/S	page fault error code	是 Ring 3 访问，还是内核路径访问
trap frame	异常入口保存	返回后重试哪条指令，还是转成信号/错误

这张表解释了为什么“缺页处理”不能只写成 `if page missing then allocate`。同样是 #PF，缺少 PTE 可能是匿名懒分配，present 页写保护可能是 COW，用户态访问无 VMA 是 `SIGSEGV`，内核 `copy_from_user` 访问坏地址应该返回 `-EFAULT`，内核普通指针 fault 则可能是 oops。

源码地图：从 x86-64 异常到通用 mm。

阅读 Linux 时，先不要在整个源码树里搜索“page fault”到处跳。应该按层次看：

路径	角色	阅读重点
<code>arch/x86/mm/fault.c</code>	x86-64 page fault 入口	读取 CR2、解释 error code、区分用户/内核上下文
<code>arch/x86/entry/entry_64.S</code>	异常入口汇编	trap frame、寄存器保存、进入 C 函数前的状态
<code>include/linux/mm_types.h</code>	mm 核心结构	<code>mm_struct</code> 、 <code>vm_area_struct</code> 、页表相关对象
<code>mm/mmap.c</code>	VMA 创建与查找	<code>mmap</code> 、 <code>mprotect</code> 、VMA 合并拆分
<code>mm/memory.c</code>	通用 fault 主干	<code>handle_mm_fault</code> 、匿名 fault、文件 fault、COW 页被哪些映射引用，回收和 COW 需要的关系
<code>mm/rmap.c</code>	反向映射	page cache、文件映射、读入文件页
<code>mm/filemap.c</code>	文件页 fault	本地 flush、远端 shutdown、PCID 相关路径
<code>arch/x86/mm/tlb.c</code>	TLB 失效	

这条地图的分层很重要：`arch/x86/mm/fault.c` 负责架构相关入口，`mm/memory.c` 负责大部分架构无关语义。不要把二者混成一个函数理解。x86-64 入口知道 CR2 和 error code；通用 mm 知道 VMA、folio、匿名页、文件页、COW、回收和锁。

第一步：地址形式和 VMA 查找不是同一件事。

用户态传来的值先要满足 x86-64 canonical address 规则。非 canonical 地址不是“找不到 VMA”那么简单，它在页表遍历前就可以被硬件拒绝。通过这一关后，内核才用 fault address 查找当前进程的 VMA。

```
page fault classification:
address is not canonical:
    reject before treating it as a normal VMA miss
no VMA covers address:
    user instruction -> SIGSEGV
    kernel usercopy -> -EFAULT
VMA covers address but permissions reject access:
    protection failure, not demand allocation
VMA covers address and permissions allow access:
    inspect PTE and repair if possible
```

VMA 是虚拟地址区间的语义说明。它回答“这段地址是否应该存在，允许读写执行吗，来自匿名内存还是文件，私有还是共享”。PTE 是当前硬件翻译状态。一个合法 VMA 允许暂时没有 PTE；一个没有 VMA 的地址不能靠分配页修复。

第二步：error code 决定走缺失路径还是保护路径。

page fault error code 的 present/protection 位把路径分成两类。若不是 present 页，常见情况是懒分配、文件页未读入或 swap 页未换入。若是 present 页上的保护 fault，常见情况是 COW、写只读映射、用户态访问 supervisor 页或取指违反 NX。

硬件现象	内核可能动作	不能误判成什么
not-present 读 匿名页	分配零页，安装 PTE，重试指令	程序错误
not-present 读 文件页	查 page cache，必要时发起 I/O	简单 malloc

present	写只读	若 VMA 允许写且 PTE 是普通权限失败	
PTE		COW, 则复制或提升	
present	写只读	发送信号或返回错误	COW
VMA			
present	取指	发送信号或安全违规处理	数据页缺失
NX 页			

这里的关键是同时看 VMA 和 PTE。VMA 允许写、PTE 不可写，可能是 COW；VMA 本身不允许写，PTE 不可写就是权限拒绝。只看 PTE 会把只读文件映射误当成 COW；只看 VMA 会看不见硬件当前状态。

匿名懒分配：合法地址为什么第一次访问才有物理页。

`mmap` 或堆增长可以先建立 VMA，不立刻分配物理页。第一次读写触发 `not-present #PF`，内核分配一个清零页，安装 PTE，返回用户态重试 `faulting instruction`。用户程序感觉这条 `load/store` 成功了，但中间发生过一次异常和内核修复。

```
anonymous demand fault:
  find VMA covering fault address
  verify VMA permits the access
  allocate zero-filled physical page
  install user PTE with VMA-derived permissions
  return to the same user RIP
  CPU retries the faulting load/store
```

这就是为什么 VIRT 可以很大而 RSS 很小。VMA 代表承诺，RSS 代表当前真的驻留的物理页。OS 用这个策略避免提前为大量可能永远不会访问的地址分配内存。

文件映射缺页：`read` 和 `mmap` 在 `page cache` 汇合。

文件 `mmap` 后，用户第一次读某个页面也会触发 `not-present #PF`。内核根据 VMA 找到文件和偏移，再到 `page cache` 查对应页。若 `page cache` 命中，这是 `minor fault`；若需要从存储设备读入，这是 `major fault`。读入后安装 PTE，用户指令重试。

```
file-backed fault:
  fault address -> VMA
  VMA file + page offset -> page-cache index
  if page cache hit:
    map cached folio and return
  else:
    submit storage read, wait for completion
    mark folio uptodate
    map folio and return
```

因此 `read` 和 `mmap` 共享的不只是“文件内容”，而是内核里的 `page cache` 对象。前者通常把 `page cache` 内容复制到用户 `buffer`；后者把 `page cache` 页映射进用户地址空间。理解这一点，后面讲 `page cache` 回收、`dirty`、`writeback` 和文件系统日志才不会断开。

COW 的完整条件：VMA 允许写，PTE 暂时禁止写。

`fork` 后父子共享匿名物理页。为了发现第一次写，父子 PTE 都要清掉 `writable`，并标记 COW。用户写入时，CPU 看到 `present` PTE 但不可写，于是产生保护型 `#PF`。内核确认 VMA 允许写、PTE 是 COW，再决定复制还是提升。

```
COW write fault:
  write to present but read-only PTE
  VMA says private mapping is writable
  PTE carries COW state
```

```

if physical page refcount > 1:
    allocate new page and copy old content
    decrement old page refcount
    install writable PTE to new page
else:
    only restore writable bit
    invalidate stale TLB translation
    return and retry the store

```

复制和提升的差别体现了 COW 的性能设计。若物理页仍被父子或其他映射共享，必须复制；若只剩当前映射，直接把 PTE 改回 writable 即可。无论哪种，都要处理 TLB，因为 CPU 可能还缓存着旧的只读或旧的可写翻译。

为什么 fork 时父进程也必须写保护。

常见误解是“fork 只要把子进程页表设成只读”。这是错的。父进程如果保留 writable PTE，它可以继续写共享物理页，子进程会直接看到父进程的新内容，COW 就失效了。正确做法是父子双方都写保护，并清理父进程可能已有的 writable TLB 项。

```

fork private pages:
  for each writable private PTE:
    increment physical page refcount
    clear parent writable bit
    mark parent PTE as COW
    install child read-only COW PTE
    invalidate parent stale writable TLB entry

```

这一步把“fork 很快”变成可理解的状态机：fork 不复制所有页，只复制页表项和引用计数；第一次写才复制物理页。代价是 fork 时要批量改 PTE 权限，并承担 TLB 失效成本。

TLB shutdown: COW 的隐藏正确性条件。

如果父进程在 fork 前刚写过某页，CPU TLB 可能缓存了 writable 翻译。fork 清掉父 PTE writable 后，若没有失效旧 TLB，父进程下一次写可能不触发 #PF，而是直接修改共享物理页。这不是性能问题，而是隔离正确性问题。

```

stale TLB failure:
  parent writes page, TLB caches writable PTE
  fork clears parent PTE writable but forgets TLB invalidate
  parent writes again through stale TLB
  child observes modified shared frame
  COW handler never runs

```

多核时问题更明显：同一个地址空间的线程可能在多个 CPU 上运行，远端 CPU 也可能缓存旧翻译。内核需要本地 TLB invalidate，也可能需要 IPI 触发远端 CPU 执行 shutdown。后面看到 `mprotect`、`munmap`、COW、页迁移性能问题时，要把这笔成本算进去。

用户指针 fault: 为什么有时是 -EFAULT 而不是信号。

同一个坏地址，在用户指令里访问和在系统调用参数复制时访问，结果不同。用户程序自己执行 `mov rax, qword ptr [rdi]` 访问坏地址，通常应得到信号。若用户把坏指针传给 `read` 或 `write`，内核在 `copy_to_user` 或 `copy_from_user` 中 fault，则系统调用应返回 `-EFAULT`。

```

; Intel syntax, user instruction fault example
mov rax, qword ptr [rdi]

; syscall usercopy fault example, conceptual
copy_from_user(kernel_buffer, user_pointer, length)

```

```
if page fault is not repairable:
    return -EFAULT
```

Linux 用异常表一类机制标记某些内核访问用户内存的指令：这里 `fault` 不代表内核坏了，而是用户参数不可访问，应跳到修复位置返回错误。没有这个区分，坏用户指针就会把内核打崩。

可观察证据：不要只看 `maps`。

调试缺页路径时，`/proc/<pid>/maps` 只能告诉你 VMA，不告诉你每页是否已经有 PTE、是否驻留、是否 `dirty`。需要把多个证据合起来看：

命令或文件	能看到什么	不能单独推出什么
<code>/proc/<pid>/maps</code>	VMA 区间、权限、文件映射	物理页是否已分配
<code>/proc/<pid>/smaps</code>	RSS、dirty、shared/private 统计	每一次 <code>fault</code> 的源码分支
<code>perf stat -e page-faults</code> <code>perf stat -e minor-faults,</code> <code>major-faults</code>	page fault 总量	major/minor 和 <code>fault</code> 原因细节
<code>strace</code>	缺页是否涉及 I/O 系统调用返回 <code>EFAULT</code> 等错误	COW 与匿名零页的区别 用户指令自己的 <code>page fault</code>

真正的分析要能把现象落回路径。例如 VMA 存在但 RSS 没涨，说明还没实际触页；minor fault 增加但 major fault 不增，通常没有存储 I/O；fork 后父子写同一页 RSS 增加，可能是 COW 分裂；系统调用返回 `EFAULT`，说明坏用户指针被 `usercopy` 路径捕获。

配套 C++20 模型覆盖哪些失败。

下面的模型不是完整 Linux，也不模拟真实指令译码。它只把第一阶段最容易讲空的断言变成测试：

- 合法匿名 VMA 第一次访问会 `minor fault`，分配零页后重试成功。
- 只读 VMA 的写入不能被当成 COW，必须失败且不能偷偷分配页。
- fork 后父子共享页，第一次写触发 COW，父子内容分离。
- 如果 fork 时忘记失效父进程 `writable TLB`，父进程会绕过 COW 污染子进程。
- 文件映射第一次 `fault` 通过 `page cache` 建立映射，并记录 `major fault`。
- `syscall usercopy` 遇到坏用户指针返回失败，而不是模拟成内核崩溃。
- 非 `canonical` 地址不能被误讲成普通 VMA miss。

第一阶段关于虚拟内存的最低验收：能解释一条用户态写指令为什么可能正常完成、minor fault 后完成、major fault 后完成、COW 后完成、返回 `-EFAULT`，或变成信号；并能指出每种结果依赖哪个 VMA/PTE/TLB 条件。

完整 C++20 模型源码。

```
#include <array>
#include <cstdint>
#include <cstdint>
#include <iostream>
#include <map>
```

```

#include <optional>
#include <stdexcept>
#include <string>
#include <string_view>
#include <unordered_map>
#include <vector>

using Byte = std::uint8_t;
using Word = std::uint64_t;
using VirtualAddress = std::uint64_t;
using PageNumber = std::uint64_t;
using Pcid = std::uint16_t;
using FrameId = std::uint32_t;

constexpr std::uint64_t X86_VIRTUAL_ADDRESS_BITS = 48U;
constexpr std::uint64_t X86_CANONICAL_SIGN_BIT =
    1ULL << (X86_VIRTUAL_ADDRESS_BITS - 1U);
constexpr VirtualAddress X86_CANONICAL_LOW_MASK =
    (1ULL << X86_VIRTUAL_ADDRESS_BITS) - 1ULL;
constexpr VirtualAddress X86_CANONICAL_HIGH_MASK = ~X86_CANONICAL_LOW_MASK;
constexpr std::uint64_t X86_PAGE_SHIFT = 12U;
constexpr std::size_t X86_PAGE_SIZE = 1ULL << X86_PAGE_SHIFT;
constexpr VirtualAddress X86_PAGE_OFFSET_MASK =
    static_cast<VirtualAddress>(X86_PAGE_SIZE - 1U);
constexpr Word X86_PF_PROTECTION_BIT = 1ULL << 0U;
constexpr Word X86_PF_WRITE_BIT = 1ULL << 1U;
constexpr Word X86_PF_USER_BIT = 1ULL << 2U;
constexpr std::size_t X86_FAULT_RETRY_LIMIT = 3U;
constexpr std::size_t HASH_COMBINE_SHIFT = 1U;

constexpr Pcid TEST_PARENT_PCID = 41U;
constexpr Pcid TEST_CHILD_PCID = 42U;
constexpr Pcid TEST_FILE_PCID = 43U;
constexpr VirtualAddress TEST_ANON_PAGE = 0x0000'0000'0040'0000ULL;
constexpr VirtualAddress TEST_ANON_VA = TEST_ANON_PAGE + 0x30ULL;
constexpr VirtualAddress TEST_READ_ONLY_PAGE = 0x0000'0000'0050'0000ULL;
constexpr VirtualAddress TEST_READ_ONLY_VA = TEST_READ_ONLY_PAGE + 0x20ULL;
constexpr VirtualAddress TEST_COW_PAGE = 0x0000'0000'0060'0000ULL;
constexpr VirtualAddress TEST_COW_VA = TEST_COW_PAGE + 0x10ULL;
constexpr VirtualAddress TEST_FILE_PAGE = 0x0000'0000'0070'0000ULL;
constexpr VirtualAddress TEST_FILE_VA = TEST_FILE_PAGE + 0x80ULL;
constexpr VirtualAddress TEST_BAD_USER_VA = 0x0000'0000'0080'0000ULL;
constexpr VirtualAddress TEST_NON_CANONICAL_VA = 0x0000'8000'0000'0000ULL;
constexpr std::uint64_t TEST_FILE_PAGE_INDEX = 9U;
constexpr std::size_t TEST_FILE_BYTE_OFFSET =
    static_cast<std::size_t>(TEST_FILE_VA & X86_PAGE_OFFSET_MASK);
constexpr Byte TEST_ZERO_BYTE = 0x00U;
constexpr Byte TEST_ANON_BYTE = 0x5AU;
constexpr Byte TEST_ORIGINAL_BYTE = 0x11U;
constexpr Byte TEST_PARENT_BYTE = 0x22U;
constexpr Byte TEST_CHILD_BYTE = 0x33U;
constexpr Byte TEST_FILE_BYTE = 0x7EU;

enum class AccessKind {
    Read,
    Write,
    Execute,
};

enum class VmaKind {
    Anonymous,
    FileBacked,
};

enum class FaultContext {
    UserInstruction,
    UserCopy,
};

enum class FaultDisposition {
    Handled,
    Signal,
    BadUserPointer,
};

enum class AccessFailureKind {
    SegmentationViolation,
    BadUserPointer,
    RetryLimitExceeded,
};

struct PageTableEntry {
    FrameId frame = 0;
    bool present = false;
    bool writable = false;
    bool user = true;
    bool executable = false;
    bool cow = false;
};

struct Translation {
    FrameId frame = 0;
    std::size_t offset = 0;
};

struct Vma {
    VirtualAddress start = 0;

```

```

VirtualAddress end = 0;
bool readable = false;
bool writable = false;
bool executable = false;
bool private_mapping = true;
VmaKind kind = VmaKind::Anonymous;
std::uint64_t file_page_index = 0;
};

struct FaultStats {
    std::size_t minor_faults = 0;
    std::size_t major_faults = 0;
    std::size_t cow_copies = 0;
    std::size_t cow_promotions = 0;
    std::size_t signals = 0;
    std::size_t bad_user_pointers = 0;
};

struct PageCacheResult {
    FrameId frame = 0;
    bool hit = false;
};

struct TlbKey {
    Pcid pcid = 0;
    PageNumber vpn = 0;

    bool operator==(const TlbKey& other) const {
        return this->pcid == other.pcid && this->vpn == other.vpn;
    }
};

struct TlbKeyHash {
    std::size_t operator()(const TlbKey& key) const {
        const std::size_t pcid_hash = std::hash<Pcid>{}(key.pcid);
        const std::size_t vpn_hash = std::hash<PageNumber>{}(key.vpn);
        return pcid_hash ^ (vpn_hash << HASH_COMBINE_SHIFT);
    }
};

bool is_canonical(VirtualAddress address) {
    const bool sign_bit_set = (address & X86_CANONICAL_SIGN_BIT) != 0;
    const VirtualAddress high_bits = address & X86_CANONICAL_HIGH_MASK;
    if (sign_bit_set) {
        return high_bits == X86_CANONICAL_HIGH_MASK;
    }
    return high_bits == 0;
}

VirtualAddress page_base(VirtualAddress address) {
    return address & ~X86_PAGE_OFFSET_MASK;
}

PageNumber virtual_page_number(VirtualAddress address) {
    return address >> X86_PAGE_SHIFT;
}

std::size_t page_offset(VirtualAddress address) {
    return static_cast<std::size_t>(address & X86_PAGE_OFFSET_MASK);
}

void require(bool condition, std::string_view message) {
    if (!condition) {
        throw std::runtime_error(std::string(message));
    }
}

class PageFault : public std::runtime_error {
public:
    PageFault(VirtualAddress fault_address, AccessKind fault_access,
              Word fault_error_code, std::string_view message)
        : std::runtime_error(std::string(message)),
          address_(fault_address),
          access_(fault_access),
          error_code_(fault_error_code) {}

    VirtualAddress address() const {
        return this->address_;
    }

    AccessKind access() const {
        return this->access_;
    }

    Word error_code() const {
        return this->error_code_;
    }

private:
    VirtualAddress address_ = 0;
    AccessKind access_ = AccessKind::Read;
    Word error_code_ = 0;
};

class AccessFailure : public std::runtime_error {
public:
    AccessFailure(AccessFailureKind failure_kind, std::string_view message)

```

```

        : std::runtime_error(std::string(message)), kind_(failure_kind) {}

    AccessFailureKind kind() const {
        return this->kind_;
    }

private:
    AccessFailureKind kind_ = AccessFailureKind::SegmentationViolation;
};

template <typename Callable>
void require_access_failure(AccessFailureKind expected, Callable action) {
    try {
        action();
    } catch (const AccessFailure& failure) {
        require(failure.kind() == expected, "unexpected access failure kind");
        return;
    }
    throw std::runtime_error("expected access failure was not raised");
}

class PhysicalMemory {
public:
    FrameId allocate_zero_frame() {
        const FrameId frame = static_cast<FrameId>(this->frames_.size());
        this->frames_.push_back(Frame{});
        this->refcounts_.push_back(1U);
        return frame;
    }

    FrameId allocate_copy_frame(FrameId source) {
        const FrameId frame = this->allocate_zero_frame();
        this->frames_.at(frame) = this->frames_.at(source);
        return frame;
    }

    void increment_ref(FrameId frame) {
        ++this->refcounts_.at(frame);
    }

    void decrement_ref(FrameId frame) {
        std::size_t& refcount = this->refcounts_.at(frame);
        require(refcount > 0, "physical frame refcount underflow");
        --refcount;
    }

    std::size_t refcount(FrameId frame) const {
        return this->refcounts_.at(frame);
    }

    Byte load8(FrameId frame, std::size_t offset) const {
        require(offset < X86_PAGE_SIZE, "load offset must stay inside page");
        return this->frames_.at(frame).bytes.at(offset);
    }

    void store8(FrameId frame, std::size_t offset, Byte value) {
        require(offset < X86_PAGE_SIZE, "store offset must stay inside page");
        this->frames_.at(frame).bytes.at(offset) = value;
    }

    std::size_t frame_count() const {
        return this->frames_.size();
    }

private:
    struct Frame {
        std::array<Byte, X86_PAGE_SIZE> bytes{};
    };

    std::vector<Frame> frames_;
    std::vector<std::size_t> refcounts_;
};

class PageCache {
public:
    PageCacheResult get_or_read_page(PhysicalMemory& memory,
                                     std::uint64_t file_page_index) {
        const auto iterator = this->pages_.find(file_page_index);
        if (iterator != this->pages_.end()) {
            return PageCacheResult{.frame = iterator->second, .hit = true};
        }

        const FrameId frame = memory.allocate_zero_frame();
        memory.store8(frame, TEST_FILE_BYTE_OFFSET, TEST_FILE_BYTE);
        this->pages_[file_page_index] = frame;
        return PageCacheResult{.frame = frame, .hit = false};
    }

    std::size_t size() const {
        return this->pages_.size();
    }

private:
    std::map<std::uint64_t, FrameId> pages_;
};

class Tlb {

```



```

public:
    std::optional<PageTableEntry> lookup(Pcid pcid, PageNumber vpn) const {
        const auto iterator = this->entries_.find(TlbKey{.pcid = pcid, .vpn = vpn});
        if (iterator == this->entries_.end()) {
            return std::nullopt;
        }
        return iterator->second;
    }

    void insert(Pcid pcid, PageNumber vpn, const PageTableEntry& entry) {
        this->entries_[TlbKey{.pcid = pcid, .vpn = vpn}] = entry;
    }

    void invalidate(Pcid pcid, PageNumber vpn) {
        this->entries_.erase(TlbKey{.pcid = pcid, .vpn = vpn});
    }

    std::size_t size() const {
        return this->entries_.size();
    }

private:
    std::unordered_map<TlbKey, PageTableEntry, TlbKeyHash> entries_;
};

class AddressSpace {
public:
    explicit AddressSpace(Pcid pcid) : pcid_(pcid) {}

    Pcid pcid() const {
        return this->pcid_;
    }

    void add_vma(const Vma& vma) {
        require(is_canonical(vma.start), "VMA start must be canonical");
        require(is_canonical(vma.end - 1U), "VMA end must be canonical");
        require(vma.start < vma.end, "VMA must not be empty");
        this->vmas_.push_back(vma);
    }

    const Vma* find_vma(VirtualAddress address) const {
        for (const Vma& vma : this->vmas_) {
            if (address >= vma.start && address < vma.end) {
                return &vma;
            }
        }
        return nullptr;
    }

    void map_page(VirtualAddress virtual_page, const PageTableEntry& entry) {
        require((virtual_page & X86_PAGE_OFFSET_MASK) == 0,
            "mapped virtual address must be page aligned");
        this->entries_[virtual_page_number(virtual_page)] = entry;
    }

    const PageTableEntry* find_pte(PageNumber vpn) const {
        const auto iterator = this->entries_.find(vpn);
        if (iterator == this->entries_.end()) {
            return nullptr;
        }
        return &iterator->second;
    }

    PageTableEntry* find_pte_for_mutation(PageNumber vpn) {
        const auto iterator = this->entries_.find(vpn);
        if (iterator == this->entries_.end()) {
            return nullptr;
        }
        return &iterator->second;
    }

    const std::vector<Vma>& vmas() const {
        return this->vmas_;
    }

    const std::map<PageNumber, PageTableEntry>& entries() const {
        return this->entries_;
    }

    std::map<PageNumber, PageTableEntry>& entries_for_mutation() {
        return this->entries_;
    }

    std::size_t mapped_page_count() const {
        return this->entries_.size();
    }

private:
    Pcid pcid_ = 0;
    std::vector<Vma> vmas_;
    std::map<PageNumber, PageTableEntry> entries_;
};

class Mmu {
public:
    Translation translate(const AddressSpace& address_space, Tlb& tlb,
        VirtualAddress virtual_address, AccessKind access) const {

```

```

if (!is_canonical(virtual_address)) {
    throw PageFault(
        virtual_address, access, this->error_code(access, false),
        "x86-64 rejected non-canonical virtual address before VMA lookup");
}

const PageNumber vpn = virtual_page_number(virtual_address);
const std::optional<PageTableEntry> cached = tlb.lookup(address_space.pcid(), vpn);
if (cached.has_value()) {
    this->check_entry(virtual_address, access, cached.value());
    return Translation{.frame = cached->frame, .offset = page_offset(virtual_address)};
}

const PageTableEntry* walked = address_space.find_pte(vpn);
if (walked == nullptr || !walked->present) {
    throw PageFault(virtual_address, access, this->error_code(access, false),
        "page walk found no present PTE");
}
this->check_entry(virtual_address, access, *walked);
tlb.insert(address_space.pcid(), vpn, *walked);
return Translation{.frame = walked->frame, .offset = page_offset(virtual_address)};
}

private:
void check_entry(VirtualAddress address, AccessKind access,
    const PageTableEntry& entry) const {
    if (!entry.user) {
        throw PageFault(address, access, this->error_code(access, true),
            "ring 3 access reached supervisor PTE");
    }
    if (access == AccessKind::Write && !entry.writable) {
        throw PageFault(address, access, this->error_code(access, true),
            "write access rejected by PTE");
    }
    if (access == AccessKind::Execute && !entry.executable) {
        throw PageFault(address, access, this->error_code(access, true),
            "execute access rejected by PTE");
    }
}

Word error_code(AccessKind access, bool protection) const {
    Word code = X86_PF_USER_BIT;
    if (protection) {
        code |= X86_PF_PROTECTION_BIT;
    }
    if (access == AccessKind::Write) {
        code |= X86_PF_WRITE_BIT;
    }
    return code;
}
};

class KernelMemoryManager {
public:
    KernelMemoryManager(PhysicalMemory& memory, PageCache& page_cache, Tlb& tlb)
        : memory_(memory), page_cache_(page_cache), tlb_(tlb) {}

    const FaultStats& stats() const {
        return this->stats_;
    }

    FaultDisposition handle_page_fault(AddressSpace& address_space,
        const PageFault& fault,
        FaultContext context) {
        const Vma* vma = address_space.find_vma(fault.address());
        if (vma == nullptr || !this->vma_allows(*vma, fault.access())) {
            return this->reject_fault(context);
        }

        const PageNumber vpn = virtual_page_number(fault.address());
        PageTableEntry* pte = address_space.find_pte_for_mutation(vpn);
        const bool protection_fault =
            (fault.error_code() & X86_PF_PROTECTION_BIT) != 0;
        if (!protection_fault) {
            return this->handle_missing_pte(address_space, *vma, vpn);
        }
        if (fault.access() == AccessKind::Write && pte != nullptr && pte->cow) {
            return this->handle_cow_fault(address_space, vpn, *pte);
        }
        return this->reject_fault(context);
    }

    void fork_private(AddressSpace& parent, AddressSpace& child,
        bool invalidate_parent_tlb) {
        for (const Vma& vma : parent.vmas()) {
            child.add_vma(vma);
        }

        for (auto& [vpn, entry] : parent.entries_for_mutation()) {
            PageTableEntry child_entry = entry;
            const VirtualAddress virtual_page = vpn << X86_PAGE_SHIFT;
            const Vma* vma = parent.find_vma(virtual_page);
            if (entry.present && vma != nullptr) {
                this->memory_.increment_ref(entry.frame);
                if (vma->private_mapping && vma->writable) {
                    entry.writable = false;
                    entry.cow = true;
                }
            }
        }
    }
};

```

```

        child_entry.writable = false;
        child_entry.cow = true;
        if (invalidate_parent_tlb) {
            this->tlb_.invalidate(parent.pcid(), vpn);
        }
    }
    child.map_page(virtual_page, child_entry);
}

private:
bool vma_allows(const Vma& vma, AccessKind access) const {
    if (access == AccessKind::Read) {
        return vma.readable;
    }
    if (access == AccessKind::Write) {
        return vma.writable;
    }
    return vma.executable;
}

FaultDisposition reject_fault(FaultContext context) {
    if (context == FaultContext::UserCopy) {
        ++this->stats_.bad_user_pointers;
        return FaultDisposition::BadUserPointer;
    }
    ++this->stats_.signals;
    return FaultDisposition::Signal;
}

FaultDisposition handle_missing_pte(AddressSpace& address_space, const Vma& vma,
                                   PageNumber vpn) {
    if (vma.kind == VmaKind::Anonymous) {
        const FrameId frame = this->memory_.allocate_zero_frame();
        address_space.map_page(
            vpn << X86_PAGE_SHIFT,
            PageTableEntry{
                .frame = frame,
                .present = true,
                .writable = vma.writable,
                .user = true,
                .executable = vma.executable,
                .cow = false,
            });
        ++this->stats_.minor_faults;
        return FaultDisposition::Handled;
    }

    PageCacheResult result =
        this->page_cache_.get_or_read_page(this->memory_, vma.file_page_index);
    this->memory_.increment_ref(result.frame);
    address_space.map_page(
        vpn << X86_PAGE_SHIFT,
        PageTableEntry{
            .frame = result.frame,
            .present = true,
            .writable = vma.writable && !vma.private_mapping,
            .user = true,
            .executable = vma.executable,
            .cow = false,
        });
    if (result.hit) {
        ++this->stats_.minor_faults;
    } else {
        ++this->stats_.major_faults;
    }
    return FaultDisposition::Handled;
}

FaultDisposition handle_cow_fault(AddressSpace& address_space, PageNumber vpn,
                                  PageTableEntry& pte) {
    if (this->memory_.refcount(pte.frame) == 10) {
        pte.writable = true;
        pte.cow = false;
        this->tlb_.invalidate(address_space.pcid(), vpn);
        ++this->stats_.cow_promotions;
        return FaultDisposition::Handled;
    }

    const FrameId old_frame = pte.frame;
    const FrameId new_frame = this->memory_.allocate_copy_frame(old_frame);
    this->memory_.decrement_ref(old_frame);
    pte.frame = new_frame;
    pte.writable = true;
    pte.cow = false;
    this->tlb_.invalidate(address_space.pcid(), vpn);
    ++this->stats_.cow_copies;
    return FaultDisposition::Handled;
}

PhysicalMemory& memory_;
PageCache& page_cache_;
Tlb& tlb_;
FaultStats stats_;
};

```

```

class UserCpu {
public:
    UserCpu(const Mmu& mmu, PhysicalMemory& memory, Tlb& tlb,
            KernelMemoryManager& kernel)
        : mmu_(mmu), memory_(memory), tlb_(tlb), kernel_(kernel) {}

    Byte load8(AddressSpace& address_space, VirtualAddress virtual_address) {
        for (std::size_t attempt = 0; attempt < X86_FAULT_RETRY_LIMIT; ++attempt) {
            try {
                const Translation translation =
                    this->mmu_.translate(address_space, this->tlb_, virtual_address,
                                         AccessKind::Read);
                return this->memory_.load8(translation.frame, translation.offset);
            } catch (const PageFault& fault) {
                this->handle_or_throw(address_space, fault, FaultContext::UserInstruction);
            }
        }
        throw AccessFailure(AccessFailureKind::RetryLimitExceeded,
                            "load exceeded page fault retry limit");
    }

    void store8(AddressSpace& address_space, VirtualAddress virtual_address,
               Byte value) {
        for (std::size_t attempt = 0; attempt < X86_FAULT_RETRY_LIMIT; ++attempt) {
            try {
                const Translation translation =
                    this->mmu_.translate(address_space, this->tlb_, virtual_address,
                                         AccessKind::Write);
                this->memory_.store8(translation.frame, translation.offset, value);
                return;
            } catch (const PageFault& fault) {
                this->handle_or_throw(address_space, fault, FaultContext::UserInstruction);
            }
        }
        throw AccessFailure(AccessFailureKind::RetryLimitExceeded,
                            "store exceeded page fault retry limit");
    }

    bool copy_from_user(AddressSpace& address_space, VirtualAddress virtual_address,
                       Byte& output) {
        for (std::size_t attempt = 0; attempt < X86_FAULT_RETRY_LIMIT; ++attempt) {
            try {
                const Translation translation =
                    this->mmu_.translate(address_space, this->tlb_, virtual_address,
                                         AccessKind::Read);
                output = this->memory_.load8(translation.frame, translation.offset);
                return true;
            } catch (const PageFault& fault) {
                const FaultDisposition disposition =
                    this->kernel_.handle_page_fault(address_space, fault,
                                                    FaultContext::UserCopy);
                if (disposition == FaultDisposition::Handled) {
                    continue;
                }
                return false;
            }
        }
        return false;
    }

private:
    void handle_or_throw(AddressSpace& address_space, const PageFault& fault,
                        FaultContext context) {
        const FaultDisposition disposition =
            this->kernel_.handle_page_fault(address_space, fault, context);
        if (disposition == FaultDisposition::Signal) {
            throw AccessFailure(AccessFailureKind::SegmentationViolation,
                                "page fault became a user signal");
        }
        if (disposition == FaultDisposition::BadUserPointer) {
            throw AccessFailure(AccessFailureKind::BadUserPointer,
                                "page fault became a bad user pointer");
        }
    }

    const Mmu& mmu_;
    PhysicalMemory& memory_;
    Tlb& tlb_;
    KernelMemoryManager& kernel_;
};

Vma anonymous_vma(VirtualAddress page, bool writable) {
    return Vma{
        .start = page,
        .end = page + X86_PAGE_SIZE,
        .readable = true,
        .writable = writable,
        .executable = false,
        .private_mapping = true,
        .kind = VmaKind::Anonymous,
        .file_page_index = 0,
    };
}

Vma file_vma(VirtualAddress page) {
    return Vma{
        .start = page,
    };
}

```

```

        .end = page + X86_PAGE_SIZE,
        .readable = true,
        .writable = false,
        .executable = false,
        .private_mapping = true,
        .kind = VmaKind::FileBacked,
        .file_page_index = TEST_FILE_PAGE_INDEX,
    };
}

struct TestMachine {
    PhysicalMemory memory;
    PageCache page_cache;
    Tlb tlb;
    Mmu mmu;
    KernelMemoryManager kernel;
    UserCpu cpu;

    TestMachine()
        : memory(),
          page_cache(),
          tlb(),
          mmu(),
          kernel(memory, page_cache, tlb),
          cpu(mmu, memory, tlb, kernel) {}
};

void test_anonymous_fault_allocates_and_retries() {
    TestMachine machine;
    AddressSpace process(TEST_PARENT_PCID);
    process.add_vma(anonymous_vma(TEST_ANON_PAGE, true));

    require(process.mapped_page_count() == 0U,
        "anonymous VMA should not allocate a PTE before access");
    require(machine.cpu.load8(process, TEST_ANON_VA) == TEST_ZERO_BYTE,
        "anonymous demand fault should return a zero-filled byte");
    machine.cpu.store8(process, TEST_ANON_VA, TEST_ANON_BYTE);
    require(machine.cpu.load8(process, TEST_ANON_VA) == TEST_ANON_BYTE,
        "anonymous page did not preserve user store");
    require(machine.kernel.stats().minor_faults == 1U,
        "anonymous demand paging should be a minor fault in this model");
}

void test_vma_permission_rejects_write() {
    TestMachine machine;
    AddressSpace process(TEST_PARENT_PCID);
    process.add_vma(anonymous_vma(TEST_READ_ONLY_PAGE, false));

    require_access_failure(AccessFailureKind::SegmentationViolation, [&machine,
                                                                    &process] {
        machine.cpu.store8(process, TEST_READ_ONLY_VA, TEST_ANON_BYTE);
    });
    require(machine.kernel.stats().signals == 1U,
        "write to read-only VMA should become a user-visible failure");
    require(process.mapped_page_count() == 0U,
        "permission failure must not allocate a page");
}

void test_cow_split_preserves_parent_child_isolation() {
    TestMachine machine;
    AddressSpace parent(TEST_PARENT_PCID);
    parent.add_vma(anonymous_vma(TEST_COW_PAGE, true));

    machine.cpu.store8(parent, TEST_COW_VA, TEST_ORIGINAL_BYTE);
    const FrameId shared_frame =
        parent.find_pte(virtual_page_number(TEST_COW_VA))>>frame;
    AddressSpace child(TEST_CHILD_PCID);
    machine.kernel.fork_private(parent, child, true);

    require(machine.memory.refcount(shared_frame) == 2U,
        "fork should share the original frame before either process writes");
    machine.cpu.store8(parent, TEST_COW_VA, TEST_PARENT_BYTE);
    require(machine.cpu.load8(parent, TEST_COW_VA) == TEST_PARENT_BYTE,
        "parent COW write did not install private data");
    require(machine.cpu.load8(child, TEST_COW_VA) == TEST_ORIGINAL_BYTE,
        "child should still see the pre-COW byte");
    require(machine.kernel.stats().cow_copies == 1U,
        "shared COW page should require one copy");

    machine.cpu.store8(child, TEST_COW_VA, TEST_CHILD_BYTE);
    require(machine.cpu.load8(child, TEST_COW_VA) == TEST_CHILD_BYTE,
        "child COW promotion did not make the child page writable");
    require(machine.kernel.stats().cow_promotions == 1U,
        "last private COW reference should be promoted without copying");
}

void test_missing_tlb_shutdown_breaks_cow() {
    TestMachine machine;
    AddressSpace parent(TEST_PARENT_PCID);
    parent.add_vma(anonymous_vma(TEST_COW_PAGE, true));

    machine.cpu.store8(parent, TEST_COW_VA, TEST_ORIGINAL_BYTE);
    AddressSpace child(TEST_CHILD_PCID);
    machine.kernel.fork_private(parent, child, false);

    machine.cpu.store8(parent, TEST_COW_VA, TEST_PARENT_BYTE);
    require(machine.cpu.load8(child, TEST_COW_VA) == TEST_PARENT_BYTE,

```

```

    "without shutdown, stale writable TLB should corrupt the shared page");
    require(machine.kernel.stats().cow_copies == 0U,
        "stale TLB write should bypass the COW handler entirely");
}

void test_file_fault_and_bad_user_pointer() {
    TestMachine machine;
    AddressSpace process(TEST_FILE_PCID);
    process.add_vma(file_vma(TEST_FILE_PAGE));

    require(machine.cpu.load8(process, TEST_FILE_VA) == TEST_FILE_BYTE,
        "file-backed fault did not load page-cache content");
    require(machine.kernel.stats().major_faults == 1U,
        "first file-backed page should model a major fault");
    require(machine.page_cache.size() == 1U,
        "file fault should populate exactly one page-cache entry");

    Byte copied = TEST_ZERO_BYTE;
    require(!machine.cpu.copy_from_user(process, TEST_BAD_USER_VA, copied),
        "bad user pointer should be reported to the syscall layer");
    require(machine.kernel.stats().bad_user_pointers == 1U,
        "bad user pointer statistic was not updated");
}

void test_non_canonical_address_is_not_a_vma_miss() {
    TestMachine machine;
    AddressSpace process(TEST_PARENT_PCID);
    process.add_vma(anonymous_vma(TEST_ANON_PAGE, true));

    require(!is_canonical(TEST_NON_CANONICAL_VA),
        "test address must be non-canonical");
    require_access_failure(AccessFailureKind::SegmentationViolation, [&machine,
                                                                    &process] {
        static_cast<void*>(machine.cpu.load8(process, TEST_NON_CANONICAL_VA));
    });
}

int main() {
    test_anonymous_fault_allocates_and_retries();
    test_vma_permission_rejects_write();
    test_cow_split_preserves_parent_child_isolation();
    test_missing_tlb_shutdown_breaks_cow();
    test_file_fault_and_bad_user_pointer();
    test_non_canonical_address_is_not_a_vma_miss();
    std::cout << "x86-64 page fault and COW model checks passed\n";
}

```


第 9 章 mmap、page cache 与文件映射

9.1 原理

`read` 和 `write` 把文件内容显式复制到用户缓冲区，而 `mmap` 把文件区间映射到进程地址空间，让普通 `load/store` 触发文件页面的调入和回写。它看起来像内存功能，实际是虚拟内存、文件系统和 `page cache` 的共同契约：`VMA` 记录虚拟地址区间；`page cache` 记录文件偏移到物理页的缓存；缺页处理把两者连接起来；写回系统决定脏页何时落盘。

先写一个常见误解：

```
addr = mmap(file, length)
# wrong mental model:
# kernel reads the whole file into memory now
# addr points to that private copy
```

这个模型会立刻解释不了三个现象。第一，映射一个很大的文件可以很快返回，因为内核通常只建立 `VMA`，不会立刻把所有页面读入内存。第二，多个进程映射同一个文件区间，读到的可能是同一份 `page cache` 页，而不是每个进程一份私有拷贝。第三，另一个进程把文件 `truncate` 后，当前进程访问原来映射的越界部分可能得到 `SIGBUS`，因为虚拟地址仍在 `VMA` 里，但背后的文件页已经不存在。

正确过渡是：

```
mmap:
  create VMA [addr, addr + length)
  remember file and file offset
  do not install every PTE yet

first load from addr + x:
  page fault
  find VMA
  compute file page index
  find or load page cache page
  install PTE
  resume user instruction
```

所以 `mmap` 的核心不是“少一次拷贝”这一句话，而是把文件偏移延迟绑定到虚拟地址访问上。这个延迟绑定带来性能，也带来错误边界：缺页可能等待 I/O，写入可能变成 `COW` 或 `dirty page`，持久化要靠 `msync/fsync` 推进，文件大小变化可能让合法 `VMA` 里的访问失败。

用一个小文件把状态走完整。文件 `data.bin` 有 9000 字节，页大小 4096。进程执行：

```
fd = open("data.bin", O_RDWR);
p = mmap(NULL, 8192, PROT_READ | PROT_WRITE,
          MAP_SHARED, fd, 0);
x = p[5000];
```

`mmap` 返回时，常见状态并不是“两个页都已经在内存里”。更准确的账本是：

对象	刚 <code>mmap</code> 后	第一次读 <code>p[5000]</code> 后
<code>VMA</code>	记录虚拟区间、文件、offset	不变
	0、共享可写	

PTE	两个虚拟页通常都还没有 present 映射	第二个虚拟页 present，指向 page cache 页
page cache	可能没有对应页，也可能已有别的进程留下的页	(inode, index=1) 有缓存页
文件系统 I/O	通常没有立即读 8192 字节	若 cache miss，提交读第 1 个文件页
用户指令	还没执行 load	fault 返回后重新执行 load，读到字节

这里的 `p[5000]` 落在映射的第二个虚拟页，因为 $5000 / 4096 = 1$ 。缺页处理先用 fault 地址找到 VMA，再把 `addr - vma.start` 换算成文件页索引 1，然后到 inode 的 page cache 中查 `index=1`。若缓存命中，内核直接安装 PTE；若缓存未命中，内核分配一个 locked page，把它先放入 page cache，再发起磁盘读。读完成后 faulting 线程睡眠；完成后解锁页面，安装 PTE，回到用户态重新执行那条 load。注意是重新执行 faulting 指令，不是从下一行继续，因为第一次 load 还没有真正完成。

再看边界。映射长度是 8192，只覆盖文件前两个页；文件实际有 9000 字节，所以这两个页都在文件范围内。若改成映射 16384 字节，再访问第三页的一部分，VMA 可能仍然覆盖该地址，但文件没有对应内容，内核不能凭空制造稳定文件页，应该按系统语义触发 `SIGBUS` 或等价错误。这也是 `SIGSEGV` 和 `SIGBUS` 的区别：前者常表示地址不属于合法映射或权限不对；后者常表示地址形式上属于映射，但背后的文件对象不能提供该页。

为什么需要 `mmap`？第一，它减少系统调用和复制，适合随机访问大文件。第二，它让多个进程共享同一份文件页，天然支持共享库和共享内存。第三，它把文件 I/O 纳入页表权限和缺页机制，统一了“按需加载”。但它也带来更复杂的一致性：一个进程通过 `write` 修改文件，另一个进程通过 `mmap` 读同一区间，看到的应是同一份 page cache 语义，而不是两个互相独立的缓存。

核心思想

`mmap` 不是把文件一次性读进内存，而是建立“虚拟地址 -> 文件偏移 -> page cache 页面”的延迟绑定。

`MAP_SHARED` 和 `MAP_PRIVATE` 是理解文件映射的分水岭。共享映射的写入会把 page cache 页面标脏，之后由 `msync`、`fsync` 或后台写回落盘；私有映射初始可共享文件页，但第一次写入触发 COW，产生匿名页，文件本身不被修改。共享库代码段通常是只读文件映射；进程堆和栈通常是匿名映射；数据库会谨慎选择 `mmap` 或 direct I/O，因为错误的写回假设会破坏延迟和持久化边界。

9.2 实践

一个教学 `mmap` 需要以下对象：

对象	保存什么	不变量
VMA	虚拟区间、权限、flags、file、offset	VMA 不重叠，权限不超过文件打开模式
page table entry	虚拟页到物理页、权限、dirty/accessed	PTE 权限不能超过 VMA 权限
page cache page	inode、文件页索引、物理页、dirty、refcount	同一 inode+index 对应一个缓存页
anon page	私有 COW 后的页面	不再回写到文件
writeback item	脏页、目标块、提交状态	写回完成前页面不能被当成干净页回收

文件映射缺页路径如下：

```
handle_page_fault(addr, access):
    vma = find_vma(current.mm, addr)
    if vma == null or access not allowed by vma:
        send_sigsegv()
    page_index = vma.file_offset_pages + ((addr - vma.start) / PAGE_SIZE)
    page = page_cache_get_or_load(vma.file, page_index)
    if access == WRITE and vma.private:
        page = copy_page(page)
        mark_anon(page)
    install_pte(addr, page, pte_prot(vma, access))
```

mmap 实现中最重要的错误路径是长度、偏移和权限检查。映射长度为零、地址未对齐、偏移未按页对齐、写共享映射但 `fd` 没有写权限、文件系统不支持映射、区间越过用户地址空间，都必须明确返回错误。不要把所有失败都压成一个 `EINVAL`，否则测试和调试会失去边界信息。

9.3 算法与实现分析

VMA 查找。最简单的 VMA 管理可以用按起始地址排序的数组或链表，查找复杂度为 $O(n)$ 。这对教学模型足够，但进程有大量映射时会拖慢 `page fault`。真实内核会使用树结构，Linux 近年使用 `maple tree` 管理 VMA，以降低查找、插入和遍历成本。

```
find_vma(mm, addr):
    for vma in mm.vmas_by_start:
        if vma.start <= addr and addr < vma.end:
            return vma
        if addr < vma.start:
            return null
    return null
```

插入 VMA 时必须检查相邻区间是否可合并。若权限、`flags`、`file` 和连续 `offset` 一致，就可以合并，减少 VMA 数量。若频繁切分和合并出错，会导致 `munmap` 后仍然能访问、或合法区间被错误 `SIGSEGV`。

page cache 索引。`page cache` 的键通常是 `(inode, page_index)`。这个键保证 `read/write` 与 `mmap` 共享同一页：

```
page_cache_get_or_load(file, index):
    key = (file.inode, index)
    lock(page_cache.lock)
    page = radix_tree_lookup(page_cache, key)
    if page != null:
        page.refcount++
        unlock(page_cache.lock)
        return page
    page = allocate_page_locked()
    radix_tree_insert(page_cache, key, page)
    unlock(page_cache.lock)
    read_file_page(file, index, page)
    unlock_page(page)
    return page
```

这里先插入 `locked page`，再执行 I/O，是为了阻止多个线程同时读取同一文件页。后来的缺页者发现页面存在但 `locked`，就等待页面解锁。否则并发 `page fault` 会造成重复 I/O，甚至最后一个完成者覆盖前一个完成者的状态。

shared 与 private 写故障。写故障要区分三种情况：VMA 不允许写，私有映射需要 COW，共享映射需要让 PTE 可写并跟踪脏页。

```
handle_write_fault(vma, addr, old_page):
    if not vma.writable:
        send_sigsegv()
    if vma.private:
        new_page = allocate_page()
        copy(new_page, old_page)
        install_pte(addr, new_page, WRITE | USER)
        put_page(old_page)
        return
    mark_page_dirty(old_page)
    install_pte(addr, old_page, WRITE | USER | SHARED)
```

COW 的复杂度是一次页面复制 `O(PAGE_SIZE)`。它用写时成本换取 fork/exec 和私有映射的低启动成本。共享映射的复杂性则在持久化：PTE dirty、page dirty、文件系统 journal dirty 和块设备队列不是同一件事。用户调用 `msync(MS_SYNC)` 期望脏页写回并等待完成，但这仍不必然等价于包含目录项的业务提交，后者需要文件系统和上层协议共同保证。

继续用 `data.bin`。两个进程 A 和 B 都用 `MAP_SHARED` 映射文件第一页。A 执行：

```
p[10] = 'X';
```

如果 A 的 PTE 当前是只读但 VMA 允许写，CPU 会产生写权限 fault。内核确认这是共享可写映射后，可以把同一份 page cache 页装成可写 PTE，并在合适的时机把页面标脏。此后 B 通过 `read(fd, buf, ...)` 或自己的共享映射读同一文件页，看到的可能就是 page cache 中已修改的内容。此时状态应分层看：

层次	<code>p[10]='X'</code> 之后发生了什么
CPU/PTE	A 的页表项允许写；硬件 dirty 位可能被置位
page cache	<code>(inode,0)</code> 对应页内容已变，页面需要写回
文件系统	还未分配或提交所有元数据事务
块层/设备	还未收到写请求，更未必完成持久化
应用语义	写了内存，不等于业务事务已经提交

这就是很多 mmap 程序出错的地方：它们看到另一个进程能读到新字节，就以为数据已经“保存到磁盘”。实际上那只是 page cache 可见性，不是掉电后的持久性。若随后机器掉电，而 dirty page 还没写回，磁盘上的文件可能仍是旧内容。若应用需要“写完记录后断电也能恢复”，至少要推进到 `msync(MS_SYNC)` 或 `fsync`，并且还要考虑文件长度、目录项、日志或 manifest 的提交顺序。

`MAP_PRIVATE` 的同一行代码完全不同。A 第一次写时，内核复制一页，把 A 的 PTE 指向匿名页；page cache 仍保存文件原始内容，B 也仍看到原始内容。此时“写入成功”只对 A 的进程地址空间成立，不对文件成立。私有映射不是“共享映射但晚点落盘”，而是写入后脱离文件页的匿名内存。

回收与写回。内存压力下，干净文件页可以直接回收，因为它们能从文件重新读入；脏文件页必须先写回；匿名页若没有 swap，就不能随意丢弃。

```
reclaim_page(page):
    if page.refcount != 0 or page.locked:
        return BUSY
    if page.file_backed and not page.dirty:
        remove_from_page_cache(page)
        free(page)
```

```

return OK
if page.file_backed and page.dirty:
    enqueue_writeback(page)
    return RETRY_LATER
if page.anon and swap_available:
    write_to_swap(page)
    free(page)
    return OK
return BUSY

```

这个算法说明 page cache 不只是性能缓存，也是内存回收策略的一部分。错误地回收 dirty page 会丢数据；错误地保留所有文件页会让匿名内存被挤爆。

9.4 测试

1. 基本映射：映射文件第一页，读取字节应等于文件内容。
2. 懒加载：mmap 后不访问不应触发磁盘读，第一次访问才缺页。
3. 权限：只读 fd 不能建立可写 MAP_SHARED。
4. 私有写：MAP_PRIVATE 写入后文件内容不变。
5. 共享写：MAP_SHARED 写入后 read 同区间能看到 page cache 更新。
6. munmap：解除映射后访问同地址触发 SIGSEGV 或等价错误。
7. 截断：文件被 truncate 后访问越界映射要触发 SIGBUS 或教学系统定义的文件映射错误。
8. 回写：脏页经过 msync 后，模拟块设备能观察到写请求。

本章合格标准：能画出 VMA、PTE、page cache、inode 的关系；能解释 private/shared 映射的写故障差异；能说明 msync、fsync 和业务提交不是同一个边界。

本章压缩进来的持久化与恢复。文件映射、page cache 和写回最终都要服务恢复语义。普通应用常把“别的进程现在能读到新字节”误认为“断电后仍能读到新字节”；存储系统则必须把每一步写成可恢复协议。主线里先抓住四层提交：内存可见、page cache 变脏、块设备完成、文件系统和目录项持久。业务提交通常还要在这些层之上，再写 manifest、日志或版本号。

边界	已经保证什么	仍未保证什么
memcpy 到映射区	当前进程地址空间已改变	文件页未必写回，私有映射不改文件
其他进程读到新字节	共享 page cache 可见	掉电后未必存在
msync(MS_SYNC)	指定映射脏页写回并等待结果	目录项、rename、其他文件未必提交
fsync(file)	文件数据和必要元数据推进到持久层	多文件事务顺序未必正确
rename + fsync(dir)	目录项切换可恢复	应用协议仍要能识别旧版/新版

可靠更新通常需要单调证据。先写新数据并同步，再写临时 manifest 并同步，随后 rename 替换正式 manifest，最后同步目录。崩溃恢复时，程序只相信通过校验和、长度、版本号或事务记录证明完整的版本。若先更新 manifest 再同步数据，

恢复程序可能看到“版本 42 已提交”，但数据页还停在旧内容。持久化错误最危险的地方是正常测试很难暴露，只有把崩溃点插在每个提交边界之后，才能证明恢复算法不会相信半成品。

9.5 源码级补充：do_mmap、XArray、writeback、O_DIRECT 与 DAX

do_mmap 到 mmap_region。Linux 的 `mmap` 路径会从系统调用参数进入 `do_mmap`，经过权限、地址选择、长度和 `flags` 校验后，调用 `mmap_region` 建立或合并 VMA。文件映射还会调用 `file->f_op->mmap`，让具体文件系统或设备设置 `vm_ops`。

```
sys_mmap
-> ksys_mmap_pgoff
-> vm_mmap_pgoff
-> do_mmap
-> get_unmapped_area
-> mmap_region
-> file->f_op->mmap
```

读源码时看 `mm/mmap.c`、`mm/filemap.c`。重点关注 `vm_flags`: `VM_READ/WRITE/EXEC/SHARED/MAYWRITE/LOCKED/HUGEPAGE` 等标志共同决定 `fault`、`mprotect`、`mlock`、回收和写回行为。

address_space 与 XArray。`page cache` 不挂在 `fd` 上，而挂在文件的 `address_space` 上。一个 `inode` 的 `i_mapping` 用 `XArray` 以页索引为 `key` 保存缓存页或 `folio`。这样不同进程、不同 `fd`、`read/write/mmap` 都能找到同一份文件页。

```
filemap_fault(vmf):
mapping = vmf.vma.file.f_mapping
index = offset_to_page_index(vmf.address)
folio = xa_load(mapping.i_pages, index)
if folio missing:
    folio = page_cache_sync_readahead(...)
    read_folio_from_filesystem(...)
install_pte(vmf.address, folio)
```

`XArray` 替代老的 `radix tree`，支持稀疏索引、并发查找和特殊 `entry`。读源码时不必先精通 `XArray` 实现，但要知道 `page cache` 的 `key` 是文件偏移，不是虚拟地址。

readahead 与访问模式。顺序读时，内核会预读后续页；随机读时，过度预读会浪费 I/O 和内存。应用可用 `madvise` 给出提示：`MADV_SEQUENTIAL`、`MADV_RANDOM`、`MADV_DONTNEED`、`MADV_HUGEPAGE`。

```
on_page_cache_miss(file, index):
if sequential_pattern_detected(file):
    submit_readahead(index + 1, window)
else:
    read_single_page(index)
```

预读失败案例：数据库随机读大文件，错误地被当成顺序流，`page cache` 被无用页污染；或者媒体顺序读取未触发预读，设备队列深度上不去。性能报告要注明访问模式和缓存状态。

writeback 与错误范围。脏页写回由 `flusher` 线程、内存压力、`fsync/msync` 等触发。Linux 用 `writeback_control` 描述写回目标，用 `address_space` 记录写回错误。一个 `write` 成功后发生的异步写回错误，可能在之后的 `fsync` 才报告。


```
fsync(file):
    filemap_fdatawrite(mapping)
    filemap_fdatawait(mapping)
    err = file_check_and_advance_wb_err(file)
    if err:
        return err
    return filesystem_fsync_metadata(file)
```

错误范围跟踪很重要。否则一个进程造成的写回错误可能被另一个不相关进程消费，或者永远没人知道。可靠程序必须检查 `fsync` 返回值。

O_DIRECT 和 DAX。`O_DIRECT` 尝试绕过 page cache，把用户页直接用于块 I/O。它通常要求地址、长度和文件偏移对齐设备或文件系统要求。绕过 page cache 不等于绕过一致性：若同一文件同时被 buffered I/O 和 direct I/O 访问，内核必须同步或失效相关缓存页。

DAX 面向持久内存，允许文件直接映射到持久介质地址，跳过 page cache。这样 load/store 可直接访问持久介质，但持久化顺序、cache flush 和失败原子性更需要应用和文件系统谨慎处理。

模式	优点	风险
buffered I/O	page cache、预读、合并、易用	双重缓存、fsync 边界易误解
<code>O_DIRECT</code>	减少缓存污染，数据库可控	对齐、短 I/O、混用一致性复杂
DAX	跳过 page cache，低延迟持久内存	cache flush、持久顺序、硬件依赖强

截断、SIGBUS 与映射失效。文件映射后，文件可能被另一个进程 truncate。若映射区间超过新的文件大小，访问超出部分通常触发 `SIGBUS`，因为虚拟地址属于 VMA，但背后的文件页不再存在。这和访问完全未映射地址的 `SIGSEGV` 不同。

```
file_truncate(inode, new_size):
    invalidate page cache pages beyond new_size
    unmap mappings beyond new_size or mark invalid
    wake waiters

fault_mapped_file(addr):
    if page_index beyond inode.size:
        signal(SIGBUS)
```

这个边界常被忽略。mmap 程序不能假设映射后文件大小永远不变，尤其是共享文件、日志文件和被管理进程替换的文件。

msync、fsync 和 fdatasync。`msync` 面向映射区间，把 dirty mapped pages 写回；`fsync` 面向文件，等待文件数据和必要元数据持久；`fdatasync` 通常只保证读取文件数据所需的元数据。它们重叠但不等价。

接口	语义重点
<code>msync</code>	映射页写回，可按地址范围控制
<code>fsync</code>	文件数据和必要元数据持久化
<code>fdatasync</code>	文件数据和读取必需元数据，不一定同步所有时间戳
<code>syncfs</code>	某挂载文件系统的脏数据同步

数据库和存储服务必须明确使用哪个边界。只调用 `msync` 某段映射，不一定同

步目录项或业务 manifest；只调用 `fsync` 文件，不一定说明应用的多文件事务已提交。

用一个索引文件更新看清楚持久化边界。程序维护两个文件：`data.dat` 保存记录，`manifest` 保存“当前有效数据文件和版本”。它用 `mmap` 修改 `data.dat` 的某一页，然后更新 `manifest` 指向新版本：

```
memcpy(mapped_data + off, record, len);
msync(mapped_data + page_start, PAGE_SIZE, MS_SYNC);
write(manifest_fd, "version=42\n", 11);
fsync(manifest_fd);
```

这段看起来已经很谨慎，但还要继续问：`manifest` 文件是覆盖写还是 `rename` 新文件？目录项是否 `fsync`？`data` 文件长度是否已经扩展并持久？如果 `record` 跨两页，只 `msync` 了一页会怎样？如果 `msync` 成功但 `fsync(manifest)` 失败，恢复时应该相信哪个版本？持久化不是“调用一个同步函数”就完了，而是要把恢复算法需要的证据按顺序写到介质上。

动作	它推进了哪层状态	仍未保证什么
写 <code>mmap</code> 内存	改了 <code>page cache</code> 或私有匿名页	未必到块设备
<code>msync(MS_SYNC)</code>	等待指定映射脏页写回	未必同步目录项或其他文件
<code>fsync(data_fd)</code>	推进 <code>data</code> 文件数据和必要元数据	不代表 <code>manifest</code> 已提交
<code>rename(tmp, manifest)</code>	原子切换目录项视图	目录项持久化仍要看目录 <code>fsync</code>
<code>fsync(dirfd)</code>	推进目录项更新	不修复上层协议顺序错误

可靠更新通常要让恢复路径有单调证据。例如先把新数据页写入并同步，再写临时 `manifest`，`fsync` 临时文件，`rename` 替换正式 `manifest`，最后 `fsync` 目录。崩溃后要么看到旧 `manifest` 指向旧数据，要么看到新 `manifest` 指向已经同步过的新数据。若顺序反过来，先提交 `manifest` 再写 `data`，恢复时可能读到“版本 42 已提交”但数据页仍是旧内容。这里的核心不是某个固定配方，而是每一步都要说明：崩溃发生在这一行之后，恢复程序会看到哪些对象状态。

userfaultfd 和用户态缺页处理。`userfaultfd` 允许用户态接管某些缺页事件，用于虚拟机迁移、按需加载和用户态分页。内核把 `fault` 事件交给用户态管理进程，后者填充页面并唤醒 `faulting` 线程。

```
fault on registered range:
kernel queues userfault event
faulting thread sleeps
pager reads event
pager copies page data into address
pager wakes faulting thread
```

这个机制说明 `page fault` 不只属于内核内部，也可以成为用户态系统软件的接口。但它要求严格权限和资源控制，否则恶意 `pager` 可以让线程永久睡眠或制造内存压力。

第 10 章 内核内存分配器

10.1 原理

虚拟内存章节讲页表和地址空间，本章看内核自己怎样管理内存。内核需要分配页表页、进程结构、文件对象、socket buffer、目录项缓存、驱动 DMA buffer、日志记录和临时缓冲区。这些对象大小不同、生命周期不同、对齐要求不同，有些可以睡眠等待内存，有些在中断上下文不能睡眠。

内核分配器通常分层。最底层管理物理页框，例如 buddy allocator；上层把页切成固定大小对象，例如 slab/slub；再上层可能有专用缓存，例如 task struct cache、inode cache、dentry cache。分层的原因是单一分配器无法同时高效处理连续页、大量小对象、构造析构、缓存局部性和调试需求。

10.2 实践

实践先写对象分类表：

对象	分配来源	约束
页表页	buddy order=0 页	页对齐，通常不可移动
DMA buffer	连续物理页或 IOMMU 映射	设备可访问，对齐和边界限制
task 结构	slab cache	高频分配释放，需要初始化
socket buffer	slab + page fragment	网络吞吐和生命周期复杂
日志缓冲	kmalloc 或专用池	失败路径也要可用

然后为每类对象写：是否可睡眠、是否可回收、是否可移动、是否需要清零、是否可能从中断上下文分配。

10.3 算法与实现分析

buddy allocator。buddy allocator 把空闲内存按 order 管理，order=k 表示 2^k 个连续页。分配时找最小足够 order；没有就拆更大的块。释放时尝试和 buddy 合并。

```
alloc(order):
    for k in order..MAX_ORDER:
        if free[k] not empty:
            block = free[k].pop()
            while k > order:
                k -= 1
                buddy = split(block, k)
                free[k].push(buddy)
            return block
    return null

free(block, order):
    while order < MAX_ORDER:
        buddy = find_buddy(block, order)
        if buddy not free: break
        remove buddy from free[order]
        block = merge(block, buddy)
        order += 1
    free[order].push(block)
```

buddy 的优势是能管理连续页，释放时能合并。问题是内部碎片：请求 9 页必须分配 16 页。长期运行后，高 order 连续块也可能稀缺。

slab cache。slab 分配器为固定大小对象建立缓存。一个 slab 通常由一页或多页组成，切成多个对象槽。分配就是从 freelist 取一个槽；释放就是放回 freelist。对象构造可以在 slab 创建时完成，减少重复初始化成本。

```
slab_alloc(cache):
    if cache.partial not empty:
        obj = pop_free_object(cache.partial.first)
        return obj
    slab = allocate_new_slab(cache)
    cache.partial.push(slab)
    return pop_free_object(slab)
```

slab 的分配释放接近 $O(1)$ 。它适合大量同类型对象，例如 inode、dentry、task。实现要处理 per-CPU cache，以减少多核锁竞争；还要处理调试模式下的红区、poison、double free 检测。

kmalloc 和大小类别。通用 **kmalloc** 通常把请求大小映射到一组 size class，例如 32、64、128、256 字节。每个 size class 背后是一个 slab cache。这样可以用固定对象缓存服务变长请求。

```
kmalloc(size):
    class = size_to_class(size)
    return slab_alloc(kmalloc_cache[class])
```

size class 的代价是内部碎片。请求 129 字节可能分到 256 字节槽。分配器设计要在碎片、速度和缓存局部性之间折中。

GFP 标志和上下文。内核分配 API 通常带标志，说明是否允许睡眠、是否允许 I/O 回收、是否要求 DMA 区域等。中断上下文不能睡眠，所以只能使用不会阻塞的分配；普通进程上下文可以等待回收。

```
if in_interrupt():
    page = alloc_page(GFP_ATOMIC)
else:
    page = alloc_page(GFP_KERNEL)
```

这解释了为什么内核代码不能随便分配内存。分配失败不是理论问题，尤其在中断、持锁、内存压力和回收路径中。高质量代码必须定义失败策略：返回错误、丢包、使用预留池、降级还是 panic。

per-CPU 缓存。多核系统中，所有小对象分配都争同一把 slab 锁会很慢。per-CPU cache 让每个 CPU 维护少量本地空闲对象，常见分配释放不需要全局锁。

```
kmem_cache_alloc(cache):
    cpu_cache = cache.percpu[current_cpu()]
    if cpu_cache.freelist not empty:
        return cpu_cache.freelist.pop()
    refill_from_global(cache, cpu_cache)
    return cpu_cache.freelist.pop()

kmem_cache_free(cache, obj):
    cpu_cache = cache.percpu[current_cpu()]
    cpu_cache.freelist.push(obj)
    if cpu_cache.count > HIGH_WATERMARK:
        drain_to_global(cache, cpu_cache)
```

per-CPU 缓存降低锁竞争，但会增加内存滞留：每个 CPU 都可能囤积一批对象。高质量实现要有高低水位、CPU 下线 drain、内存压力回收和统计对账。

内存调试与防御。分配器是许多内核漏洞的放大器。越界写、use-after-free、double free 和未初始化读取都可能破坏任意内核对象。调试构建应加入红区、poison、隔离队列和调用栈记录。

```
debug_free(obj):
    if obj.magic != LIVE_MAGIC:
        panic("double free or corrupted object")
    fill(obj.memory, POISON_FREE)
    obj.magic = FREE_MAGIC
    quarantine_push(obj)

debug_alloc(cache):
    obj = real_alloc(cache)
    fill(obj.memory, POISON_ALLOC)
    obj.magic = LIVE_MAGIC
    return obj
```

隔离队列会延迟复用刚释放对象，使 use-after-free 更容易在测试中暴露。代价是内存占用和性能下降，所以通常只在调试或安全强化配置中开启。

10.4 测试

- buddy 分配释放后，空闲页总数守恒。
- 分裂和合并后，不存在重叠空闲块。
- slab 分配对象满足对齐，释放后能复用。
- double free、越界写和 use-after-free 在调试模式能被检测。
- GFP_ATOMIC 路径不能睡眠，失败时有明确降级策略。
- 内存压力下，分配器统计能解释碎片和失败原因。

10.5 源码级补充：buddy、SLUB、GFP、vmalloc 与 kmemleak

buddy 的 free_area 和迁移类型。Linux buddy 不只是按 order 分链表，还按迁移类型分组，例如 movable、unmovable、reclaimable、CMA 等。这样做是为了降低长期碎片：可移动页尽量放在一起，未来需要高 order 连续页时更容易通过 compaction 得到。

```
struct free_area {
    list_head free_list[MIGRATE_TYPES];
    unsigned long nr_free;
};
```

读 mm/page_alloc.c 时重点看 __rmqueue、__free_pages_ok、watermark、zone 和 migratetype。一次 order-9 分配失败，不一定表示系统没有 512 页空闲，可能是没有连续的 512 页。

SLAB、SLUB、SLOB 的取舍。SLAB 强调对象缓存和复杂队列；SLUB 简化元数据，成为常见默认；SLOB 面向极小系统，结构简单但伸缩性弱。现代主线常见是 SLUB。读 mm/slub.c 时看 kmem_cache、per-CPU freelist、partial slab、debug red zone 和 poison。

```
kmem_cache_alloc(cache):
    if current_cpu.freelist:
        return pop(current_cpu.freelist)
    if cache.node.partial:
        slab = take_partial(cache.node)
        load_cpu_freelist(slab)
```



```
return pop(current_cpu.freelist)
slab = new_slab_from_buddy()
return first_object(slab)
```

SLUB 的快路径很短，但 debug 选项会显著增加检查成本。教材中讲“分配是 $O(1)$ ”时，要同时说明这是平均快路径；慢路径可能进入 buddy、回收、compaction 或 OOM。

kmalloc size class 与内部碎片。kmalloc(130) 可能进入 192 或 256 字节 size class，取决于架构和配置。向上取整带来内部碎片，但换来固定大小缓存的速度和对齐。对象若频繁分配，最好建立专用 kmem_cache，可以提供构造函数、调试名和更好统计。

GFP_NOIO、GFP_NOFS 和回收递归。GFP_KERNEL 允许睡眠和回收；GFP_ATOMIC 用于中断或持自旋锁路径；GFP_NOIO 禁止通过 I/O 回收；GFP_NOFS 禁止进入文件系统回收路径。后两者用于避免递归死锁。

```
filesystem_write_path():
    lock(inode)
    buf = kmalloc(size, GFP_NOFS)
    // avoid reclaim entering filesystem and taking inode locks again
```

若文件系统持有 inode 锁时用 GFP_KERNEL 分配，内存压力下回收可能进入同一文件系统并等待 inode 锁，形成递归死锁。GFP flag 是锁和回收的边界，不是性能装饰。

vmalloc 与物理不连续。kmalloc 通常返回物理连续内存，适合 DMA 或小对象；vmalloc 返回虚拟连续但物理页可不连续的内存，适合大缓冲或模块映射。代价是需要建立页表，TLB 局部性更差，不能直接给不支持 scatter-gather 的设备 DMA。

```
vmalloc(size):
    area = allocate_virtual_area(size)
    pages = alloc_individual_pages(npages)
    map_pages(area, pages)
    return area.addr
```

kmemleak、KASAN 和 KFENCE。内存泄漏和 use-after-free 很难靠肉眼找。kmemleak 通过扫描可达指针寻找未释放对象；KASAN 使用 shadow memory 检测越界和释放后访问；KFENCE 用低频保护页检测堆错误，开销比 KASAN 低但覆盖率也低。

工具	能发现	代价
kmemleak	长期未释放对象	扫描和误报，需要调试配置
KASAN	越界、use-after-free	内存和性能开销高
KFENCE	采样式堆越界和 UAF	开销低，非确定覆盖

内存碎片、compaction 和 CMA。内存碎片分为内部碎片和外部碎片。内部碎片来自 size class 向上取整；外部碎片来自空闲页分散，无法满足高 order 连续页。compaction 通过迁移可移动页，把空闲页聚合成连续块。CMA 预留可迁移区域，给需要连续物理内存的设备使用。

```
compact_zone(zone):
    free_scanner = end_of_zone
    migrate_scanner = start_of_zone
    while scanners_not_met:
        page = find_movable_page(migrate_scanner)
        free = find_free_page(free_scanner)
        migrate(page, free)
```


并非所有页都能迁移。页表页、驱动 pin 住的 DMA 页、mlock 页和某些内核对象不可移动。高 order 分配失败时，诊断要看总空闲页、最大连续块、不可移动页比例和 compaction 失败原因。

内存控制组和分配归属。cgroup v2 可以限制一组进程的内存。分配器需要把许多用户可归因的内存记到 memcg，例如 page cache、匿名页、部分 slab 对象。这样容器内泄漏不会无限吃掉宿主内存。

```
charge_memcg(page, current.memcg):
    if memcg.current + page.size > memcg.max:
        reclaim_memcg(memcg)
        if still over limit:
            trigger_memcg_oom(memcg)
    memcg.current += page.size
```

memcg OOM 和全局 OOM 要区分。前者应限制在 cgroup 内，后者说明整机资源不足。测试容器内存限制时，要观察 `memory.current`、`memory.events` 和 OOM victim。

对象生命周期对账。内核内存质量不只看分配成功，还要能证明对象生命周期闭合。每类对象应有 alloc/free 计数、当前活跃数、高水位和失败次数。对复杂对象，还应记录 owner 和引用来源。

```
object_accounting:
    alloc_count++
    active_count++
    high_watermark = max(high_watermark, active_count)

free:
    active_count--
    free_count++
```

测试结束时 active count 不为零，就是泄漏线索；free count 大于 alloc count，说明 double free 或计数损坏；高水位异常说明负载下对象滞留。生产系统不一定记录所有栈，但调试构建应支持按对象类型打开详细追踪。

第四部分

设备、I/O 与文件系统

本部分导读

前三部分已经有了任务、边界和内存；这一部分把它们接到外部世界。设备不会直接理解进程和文件，它只理解寄存器、队列、DMA 地址、中断和 completion。内核要把这些异步硬件事件包装成 fd、文件、socket、块请求和可恢复的持久化语义。

设备驱动先把寄存器、队列、DMA 映射和 completion 翻译成 request；块层和 VFS 再把 bio、fd、inode、page cache 和目录项组织成统一接口；文件系统把磁盘块翻译成有名字、有结构、有崩溃恢复的存储对象；socket 把网卡队列和协议状态包装成字节流。

只要有 DMA、块 I/O 或 page cache，就必须追踪 buffer 归谁所有、completion 对应哪个请求、错误在哪层返回、成功是否等于持久。

第 11 章 设备驱动、中断下半部与 DMA

11.1 原理

设备驱动把硬件寄存器、中断、DMA 和内核对象连接起来。应用看到的是 fd、socket、块设备或帧缓冲；驱动看到的是 MMIO 寄存器、描述符环、设备状态位、中断线、DMA 地址和错误码。驱动的难点在于硬件和 CPU 并发运行：CPU 写描述符，设备异步读取；设备写内存，CPU 稍后读取；中断可能在任意时刻到来。

先从一个错误直觉开始：

```
disk_write(buf, len):
    return disk.write(buf, len)
```

这像普通函数调用，但真实设备不会沿着你的调用栈返回。CPU 能做的是写设备寄存器、填一段内存中的描述符、敲 doorbell 通知设备；设备随后按自己的时钟和状态机工作。它可能成功，可能失败，可能很久以后才完成，也可能在 reset 中丢掉请求。驱动的工作就是把这种异步硬件行为包装成内核可管理的请求。

把最小路径写清楚：

```
submit_write(buf):
    req = allocate_request()
    req.buffer = buf
    req.state = PREPARED
    dma = map_buffer_for_device(buf)
    fill_descriptor(req.id, dma, len)
    publish_descriptor_with_barrier()
    ring_doorbell()
    req.state = IN_FLIGHT
    sleep_until_completion(req)
```

这段伪代码第一次引出几个概念。descriptor 是给设备看的请求摘要，通常包含 DMA 地址、长度、标志和请求身份。doorbell 是通知设备“我放了新活”的寄存器写入。completion 是设备稍后交回的完成结果。request 是内核自己的对象，用来把用户请求、buffer、描述符、等待任务和最终错误码连在一起。

最简单设备可以用 programmed I/O：CPU 直接读写设备寄存器或数据端口。PIO 易懂，但吞吐低，CPU 被数据搬运占住。DMA 让设备直接读写内存，CPU 只准备描述符和缓冲区。DMA 提高吞吐，也引入缓存一致性、地址映射、内存屏障和 buffer 生命周期问题。

PIO 和 DMA 的区别不是“老技术”和“新技术”的区别，而是 CPU 参与程度不同：

方式	CPU 做什么	风险边界
PIO	每次读写设备寄存器搬运数据	CPU 忙等、吞吐低、长时间占用核心
DMA	准备 buffer 和描述符，让设备直接搬运内存	buffer 生命周期、cache 一致性、IOMMU 映射
中断	设备完成后通知 CPU	中断风暴、上半部不能睡眠、completion 要能找回请求
轮询	CPU 定期检查完成队列	延迟/吞吐折中、budget 背压

驱动通常分成上半部和下半部。中断上半部快速确认设备、读取状态、屏蔽或清除中断，把复杂工作推迟到底半部、工作队列、软中断或普通内核线程。这样可以

减少中断关闭时间，避免在硬中断上下文执行可能睡眠的操作。

驱动和前面章节的连接也要明确：提交请求时可能让当前 task 睡眠，completion 到来后要唤醒 wait queue；DMA buffer 来自页分配器和虚拟内存系统；设备错误最终要通过 fd、socket、块层或文件系统返回给用户；权限和 namespace 决定用户能否打开这个设备。驱动不是孤立章节，而是硬件异步事件进入 OS 对象系统的入口。

11.2 实践

先画一个网卡或磁盘的描述符环：

```
descriptor ring:
head: device consumes descriptors
tail: driver produces descriptors
desc[i]:
    dma_address
    length
    flags
    status
```

驱动发送路径通常是：从内核分配 buffer，映射成设备可见 DMA 地址，填描述符，执行内存屏障，更新 tail，通知设备。完成路径通常是：设备 DMA 完成并写 status，触发中断，驱动回收描述符，解除 DMA 映射，释放 buffer，唤醒等待者或上报 completion。

11.3 算法与实现分析

MMIO 寄存器访问。MMIO 把设备寄存器映射到物理地址。访问 MMIO 不能当成普通内存优化：编译器不能随便重排，CPU 也要遵守设备要求的顺序。驱动通常使用专用 readl/writel 一类接口。

```
device_start(dev):
    writel(dev->mmio + CTRL, CTRL_RESET)
    wait_until(readl(dev->mmio + STATUS) & READY)
    writel(dev->mmio + IRQ_MASK, RX_IRQ | TX_IRQ)
```

轮询 wait 要有超时。设备可能坏掉，寄存器可能永远不变。驱动若在内核里无限等待，会拖死系统。

描述符环。描述符环是设备驱动的核心队列。生产者和消费者可以是 CPU 和设备，也可以反过来。环满时不能继续提交，环空时不能继续回收。

```
submit_tx(desc_ring, packet):
    next = (tail + 1) % ring_size
    if next == head:
        return -EAGAIN
    desc[tail].addr = dma_map(packet.data)
    desc[tail].len = packet.len
    desc[tail].flags = OWNED_BY_DEVICE
    memory_barrier()
    tail = next
    writel(DOORBELL, tail)
```

这里的屏障非常关键。CPU 必须先把描述符内容写完，再更新 tail 或 doorbell 通知设备；否则设备可能看到半初始化描述符。

把一次网卡发送请求走细一点。用户态 send 最终让内核拿到一段要发送的数据，网络栈构造 packet buffer，驱动要把这个 buffer 交给网卡。网卡不认识 struct sk_buff，也不能读用户虚拟地址；它通常只读一段描述符内存，描述符里写

着 DMA 地址、长度、校验和 offload 标志、结束位和 request tag。CPU 是生产者，设备是消费者。

步骤	驱动和硬件动作	不能错的边界
保留槽位	驱动读取 <code>head/tail</code> ，确认环未滿，选择 <code>tail</code> 槽位	满环必须返回或停止上层队列，不能覆盖设备未消费的描述符
映射 buffer	DMA API 把 packet buffer 映射成设备可见 DMA 地址	设备不能拿用户指针或内核虚拟地址
填写描述符	写入地址、长度、flags、tag，最后设置 <code>owned/valid</code> 位	<code>valid</code> 位不能早于其他字段对设备可见
发布顺序	执行 DMA 写屏障，保证描述符内容先于 doorbell	少屏障会让设备读到旧字段或半初始化字段
敲门铃	写 MMIO doorbell 或更新 <code>tail</code> ，通知设备取新描述符	doorbell 只是通知，不代表发送完成
设备 DMA	网卡读取描述符和数据，经 IOMMU 访问内存，发送到链路	buffer 在 completion 前不能释放或复用
完成回收	设备写回 completion 或更新 <code>head</code> ，驱动 <code>unmap</code> 并释放 buffer	回收必须以设备完成证据为准

这里有两个常见误解。第一，`tail = next` 只是软件变量变化，设备未必看见；真正通知设备的通常是 MMIO doorbell 或共享队列尾指针的可见更新。第二，`completion` 不等于用户业务语义完成。网卡发送 `completion` 通常说明设备已经不再需要这个 DMA buffer，未必说明对端应用已经收到数据；块设备 `completion` 说明设备处理了请求，是否持久还要看 `flush/FUA` 和文件系统语义。驱动层必须先把“buffer 能否释放”这件事讲清楚，再谈上层协议。

迟到 `completion` 是取消路径最容易出错的地方。假设上层超时，驱动 `reset` 队列并把 request 对象释放；如果设备或旧中断稍后又报告同一个 tag 完成，而 tag 已经被新请求复用，驱动可能唤醒错任务或释放错 buffer。可靠实现通常给 request tag、队列 generation、in-flight bitmap 和 `reset` 状态建立明确规则：`reset` 开始后停止新提交，标记旧请求失败，等待或屏蔽旧 `completion`，确认设备不再 DMA 后再复用 tag 和 buffer。

DMA 和缓存一致性。若 CPU cache 和设备 DMA 不自动一致，驱动必须在设备读内存前清理 cache，在设备写内存后失效 cache。即使硬件支持 IOMMU，也要建立设备可访问地址映射。

```
prepare_dma_to_device(buf):
    cache_clean(buf)
    dma_addr = iommu_map(buf.phys)
    return dma_addr

finish_dma_from_device(buf):
    cache_invalidate(buf)
    parse_received_data(buf)
```

DMA buffer 的生命周期必须覆盖设备访问全过程。提交后不能释放或复用 buffer，直到 `completion` 表明设备不再访问它。这个规则和异步 I/O 的 buffer 所有权完全一致。

缓存一致性要分方向看。设备从内存读数据，叫 TO_DEVICE：CPU 先把数据写进 buffer，但这些写入可能还停在 cache 中，设备从内存读会看到旧内容；所以提交前要 `clean` 或通过 `coherent` 映射保证设备可见。设备向内存写数据，叫 FROM_DEVICE：设备已经把新数据写进内存，但 CPU cache 里可能还有旧 cache line；所以 `completion` 后 CPU 读取前要 `invalidate` 或同步。

方向	提交前/完成后要做什么	忘记后的现象
TO_DEVICE	CPU 写 buffer 后, clean cache, 再让设备 DMA 读	设备发送旧包、旧磁盘数据或半初始化描述符
FROM_DEVICE	设备 completion 后, invalidate cache, 再让 CPU 解析	CPU 读到旧包内容, 协议栈校验失败或数据错乱
BIDIRECTIONAL	两个方向都要同步, 并严格定义所有权切换	CPU 和设备同时修改, 结果半新半旧
coherent DMA	平台保证 CPU/设备可见性, 但仍需要顺序屏障	数据可见不等于 doorbell 顺序自动正确

再看一个接收包场景。驱动预先分配 RX buffer, 把它 DMA map 成 FROM_DEVICE, 挂到 RX ring。挂上去之后, buffer 暂时归设备所有, CPU 不应把它当普通内存写。网卡收到包后 DMA 写入 buffer, 并在描述符里写入长度、校验状态和完成位。驱动 poll 到完成位后, 先执行 DMA 同步或 invalidate, 再读取包头和长度, 构造 skb 交给协议栈。若顺序反了, CPU 可能先读旧 cache line, 看到旧长度或旧以太网头; 这类 bug 往往不是每次复现, 因为它取决于 cache 状态、CPU 迁移和包到达时机。

IOMMU 还提供了另一个边界: 设备只能访问被映射的 IOVA 范围。驱动 unmap 后, 设备若继续 DMA, 正确结果应是 IOMMU fault 或设备错误, 而不是静默覆盖任意物理内存。可这要求驱动先确认设备不再持有该 descriptor, 再 unmap 并释放页。IOMMU 是安全网, 不是生命周期管理的替代品。

中断合并和轮询。高吞吐设备如果每个包都中断一次, CPU 会被中断淹没。中断合并让设备积累多个完成再中断; NAPI 一类机制在高负载下从中断转为轮询, 批量处理包, 降低中断成本。代价是单个包延迟可能增加。

```
rx_interrupt():
    disable_rx_irq()
    schedule_poll()

poll(budget):
    processed = 0
    while processed < budget and rx_desc_ready():
        handle_packet()
        processed += 1
    if no_more_packets():
        enable_rx_irq()
```

这里的 budget 是背压工具。一次 poll 不能无限处理, 否则其他任务得不到 CPU。

11.4 测试

- MMIO 等待必须有超时, 设备无响应时返回错误。
- 描述符环满时返回背压, 不覆盖未完成描述符。
- 提交描述符前执行必要内存屏障。
- DMA buffer 在 completion 前不能释放或复用。
- 中断处理上半部不执行可能睡眠操作。
- 批量 poll 遵守 budget, 不能垄断 CPU。
- 设备 reset 后, 驱动能清理 in-flight 请求并通知上层失败。

本章压缩进来的块层与提交顺序。块层原本可以单独讲很久，但放在设备章里看更清楚：它是把文件系统的逻辑读写转换成设备队列 request 的中间层。文件系统提交的是 bio，描述“文件系统需要这些扇区读写”；blk-mq 把 bio 合并、排序、分派给硬件队列；驱动把 request 变成设备描述符；completion 再把错误和字节数一路传回等待者。

层次	保存什么账本	不能混淆的边界
bio	一组页、扇区范围、读写方向和完成回调	bio 完成不一定等于文件系统事务提交
request	合并后的设备队列项、tag、超时和状态	request tag 复用必须等旧 completion 收敛
hardware queue	CPU/NUMA 亲和的提交队列	多队列提高吞吐，也引入负载和顺序问题
flush/FUA	写缓存顺序和持久化约束	普通写完成不等于掉电安全
I/O scheduler	合并、排序、公平和延迟控制	吞吐优化不能破坏屏障语义

存储顺序比普通异步完成更严格。日志文件系统、数据库和键值存储经常要求“先写数据，再提交元数据或 manifest”。如果设备或调度器为了吞吐随意重排，就可能在崩溃后看到新 manifest 指向旧数据。块层用 flush、FUA、barrier 或等价机制表达这些顺序约束；文件系统还要把自己的 journal commit、checkpoint 和目录项更新放在正确位置。设备章至少要记住一句话：DMA completion 说明设备不再需要某个 buffer，持久化 completion 才说明恢复路径能相信某个状态。

11.5 源码级补充：Linux 设备模型、PCI、NVMe、virtio 与 IOMMU

设备模型：bus、driver、device、class。Linux 设备模型把发现、匹配、生命周期和 sysfs 统一起来。总线负责枚举和匹配，driver 提供 probe/remove，device 表示具体硬件或虚拟设备，class 给用户空间暴露类别视图，kobject/kset 负责引用计数和 sysfs 层次。

```
struct bus_type      -> match(device, driver)
struct device_driver -> probe(device), remove(device)
struct device        -> parent, bus, driver, kobject
struct class         -> /sys/class view
```

probe 成功后驱动必须拥有明确资源集合：MMIO 映射、IRQ、DMA mask、队列、字符/块/网络注册对象。remove 路径按相反顺序撤销，并拒绝新请求、等待 in-flight 请求完成。

```
PCI 驱动骨架。
static const struct pci_device_id ids[] = {
    { PCI_DEVICE(VENDOR, DEVICE) },
    { 0 }
};

probe(pdev, id):
    pci_enable_device(pdev)
    pci_request_regions(pdev)
    mmio = pci_iomap(pdev, bar)
    dma_set_mask_and_coherent(pdev, DMA_BIT_MASK(64))
    request_irq(pdev->irq, handler, IRQF_SHARED, name, dev)
    init_queues(dev)

remove(pdev):
```

```

stop_queues(dev)
free_irq(pdev->irq, dev)
pci_iounmap(pdev, mmio)
pci_release_regions(pdev)
pci_disable_device(pdev)

```

错误路径要能从任一步失败回滚。例如 `request_irq` 失败时，要释放 MMIO 和 PCI region；队列初始化失败时，要释放 IRQ。真实源码中这类 `goto err_*` 标签比正常路径更能体现驱动质量。

字符、块、网络三类设备。

类别	核心对象	典型回调
字符设备	<code>cdev</code> 、 <code>file_operations</code>	<code>open/read/write/ioctl/poll</code>
块设备	<code>gendisk</code> 、 <code>request_queue</code> 、 <code>blk-mq</code>	<code>submit_bio</code> 、 <code>queue_rq</code> 、 <code>completion</code>
网络设备	<code>net_device</code> 、 <code>net_device_ops</code> 、NAPI	<code>ndo_start_xmit</code> 、 <code>poll</code> 、 <code>set_rx_mode</code>

同一个硬件可能暴露多个接口。比如 NVMe 是块设备，网卡是网络设备，串口是字符设备。VFS 统一 `fd` 入口，但驱动对象和 `completion` 路径完全不同。

streaming DMA 与 coherent DMA。`dma_alloc_coherent` 返回 CPU 和设备一致可见的内存，适合描述符环；`dma_map_single/sg` 把已有 buffer 临时映射给设备，适合数据包或块 I/O。streaming DMA 要在方向、同步和 `unmap` 上严格配对。

```

tx_submit(buf):
    dma = dma_map_single(dev, buf.data, buf.len, DMA_TO_DEVICE)
    if dma_mapping_error(dev, dma): return -EIO
    fill_desc(dma, buf.len)
    dma_wmb()
    ring_doorbell()

tx_complete(desc):
    dma_unmap_single(dev, desc.dma, desc.len, DMA_TO_DEVICE)
    free_buf(desc.buf)

```

IOMMU 为设备建立地址空间，限制设备只能 DMA 到授权页。没有 IOMMU 或映射错误时，设备 bug 可能覆盖任意内存；有 IOMMU 时，错误 DMA 会变成 IOMMU fault，至少能阻止破坏扩大。

NAPI、NVMe 与 virtio 阅读入口。网络驱动可读 `drivers/net/ethernet/intel/e1000e/` 或 `virtio-net`；NVMe 可读 `drivers/nvme/host/`；virtio 核心可读 `drivers/virtio/`。阅读时按同一模板：

1. probe 如何发现设备、分配队列、注册上层对象。
2. submit 路径如何填描述符、DMA map、通知设备。
3. completion 路径如何从中断/NAPI/poll 回收资源。
4. reset/remove 路径如何停止新请求并清理 in-flight。
5. 错误统计在哪里暴露给 `ethtool`、`sysfs`、`dmesg` 或 `block stats`。

ACPI platform 设备与 PCIe 自描述设备。并非所有设备都能像 PCIe 一样完整自描述。x86-64 PC/服务器上，可枚举高速设备通常来自 PCIe 配置空间；板载控制器、固件虚拟设备、电源和热管理资源常由 ACPI 表描述。platform driver 的匹配不依赖 `vendor/device id`，而依赖 ACPI id 或平台总线注册信息。

ACPI resource sketch:

```
HID/UID identify the platform device
MMIO resource describes register window
IRQ resource describes interrupt routing
_CRS reports current resource settings
_PS0/_PS3 may describe power-state transitions
```

probe 阶段要把描述转换成内核资源：`platform_get_resource` 取得 MMIO 区间，`devm_ioremap_resource` 建立映射，`platform_get_irq` 取得中断，电源管理回调处理设备 suspend/resume。教学系统可以把 ACPI 资源简化成静态设备表，但仍要保留“描述硬件”和“驱动绑定”两个阶段。

托管资源和错误回滚。Linux 驱动大量使用 devm 托管资源。资源绑定到 device 生命周期，remove 或 probe 失败时自动释放。托管资源不是为了偷懒，而是为了减少 probe 多阶段失败路径中的泄漏。

```
probe(dev):
    state = devm_kzalloc(dev, sizeof(*state))
    mmio = devm_ioremap_resource(dev, res)
    irq = platform_get_irq(dev, 0)
    ret = devm_request_threaded_irq(dev, irq, top, thread, flags, name, state)
    if ret:
        return ret
    ret = register_upper_object(state)
    if ret:
        return ret
    return 0
```

但 devm 不能替代所有生命周期管理。已经暴露给上层的字符设备、网络设备或块设备必须先停止新请求、撤销注册、等待 in-flight 完成，再让托管资源释放。否则用户态仍可能通过旧 fd 进入已经释放的 MMIO 或队列。

ioctl、sysfs 与用户态控制面。驱动给用户态暴露控制面时，常见三种方式：ioctl、sysfs 属性和 netlink。ioctl 适合和 fd 绑定的设备私有命令；sysfs 适合稳定、简单、可文本化的设备属性；netlink 适合网络和复杂事件通知。控制面设计必须有版本、长度、权限和兼容性边界。

```
ioctl(file, cmd, user_arg):
    switch cmd:
        case GET_STATUS:
            status = snapshot_device_status()
            return copy_to_user(user_arg, status)
        case RESET_DEVICE:
            if !capable(CAP_SYS_ADMIN):
                return -EPERM
            return reset_device_safely()
        default:
            return -ENOTTY
```

ioctl 的常见漏洞来自信任用户结构体长度、未检查 padding、把内核指针泄露给用户、或在 reset 中没有和 I/O 路径同步。控制命令不是旁路，它和 read/write/interrupt 一样会并发访问设备状态。

runtime PM、suspend 与 resume。现代设备需要省电。runtime power management 允许设备空闲时关闭时钟或电源域；系统 suspend/resume 要在整机睡眠和唤醒时保存恢复设备状态。驱动必须区分“设备逻辑对象仍存在”和“硬件暂时不可访问”。

```
runtime_suspend(dev):
```

```
stop_new_io(dev)
wait_inflight(dev)
save_registers(dev)
disable_irq(dev)
disable_clock(dev)

runtime_resume(dev):
    enable_clock(dev)
    restore_registers(dev)
    enable_irq(dev)
    restart_queues(dev)
```

如果 resume 后忘记恢复 DMA mask、队列基址或中断 mask，设备可能表现为偶发超时。电源管理 bug 常常只在压力、热插拔、suspend 循环或低功耗平台上出现，因此测试必须覆盖这些状态转换。

驱动并发不变量。驱动至少要维护下面不变量：

- 提交给设备的描述符在 completion 前不能释放。
- 设备可 DMA 的 buffer 必须有有效映射，方向必须匹配。
- remove/reset 开始后，不再接受新请求。
- 中断处理看到 dying 状态时只确认和丢弃，不再提交新工作。
- 用户 fd 持有期间，驱动私有对象引用不能降到零。
- 上半部只做不可睡眠操作，复杂恢复进入线程化 IRQ 或 workqueue。

把这些不变量写进测试，才能发现“正常收发能跑”之外的问题。驱动不是能读写设备就合格，而是要在错误、中断风暴、热拔、reset 和并发 close 下仍然收敛。

第 12 章 块设备、I/O 调度与文件路径

从 bio 到用户 buffer，I/O 要穿过块层、调度器和 VFS。本章把 request 队列、merge/sort、fd→file→inode 链和 buffered/direct 路径串成一条线。把这条线走通，才能说清 write 为什么快返回、fsync 为什么慢，以及掉电后哪些数据会丢。

12.1 块层原理与对象

文件系统最终要把逻辑块读写交给块设备。若每个上层请求都直接变成一次磁盘操作，系统会遭遇两个问题：机械磁盘上寻道开销巨大，随机请求顺序会让吞吐崩溃；即使在 SSD/NVMe 上，请求大小、队列深度、合并策略和 flush 顺序也直接影响延迟和持久化语义。因此操作系统在文件系统和驱动之间放置块层，负责把 bio 合并、拆分、排序、限流、派发和完成。

块层的目标不是“越重排越好”。对普通读写，重排可以提高吞吐；对日志提交、数据库事务和文件系统 barrier，错误重排会破坏崩溃一致性。I/O 调度器必须同时理解性能和顺序约束：哪些请求可合并，哪些可越过，哪些必须在 flush 后才能报告完成。

核心思想

块层把“文件系统想读写哪些块”转化为“设备能高效且按约束完成的请求序列”。它是性能优化层，也是持久化契约层。

从设备演进看，机械磁盘偏好按扇区顺序扫描；SATA SSD 降低了寻道成本但仍受擦写块和队列深度影响；NVMe 提供多队列并行，单一全局电梯队列反而会成为瓶颈。教材不能只讲一个 SCAN 算法就结束，而要说明为什么算法随硬件变化。

块层常用对象如下：

对象	含义	关键字段
bio	上层的一段块 I/O 意图	sector、len、page list、op、flags、endio
request	设备队列中的可派发请求	合并后的 bio 链、方向、优先级、deadline
request queue	块设备的软件队列	pending、inflight、depth、scheduler
hardware queue	多队列设备的硬件提交队列	doorbell、completion、CPU 亲和
flush request	缓存刷新或持久化屏障	preflush、FUA、barrier dependency

一次写入的路径可以写成：

```
filesystem_writeback(page):
    bio = build_bio(page.block, page.data, WRITE)
    submit_bio(bio)

submit_bio(bio):
    q = bio.device.queue
    if can_merge(q.last_request, bio):
        merge(q.last_request, bio)
    else:
```



```
req = make_request(bio)
scheduler_insert(q, req)
dispatch_if_possible(q)
```

这里的关键是 bio 的生命周期。bio 完成并不等于调用者数据结构一定可释放，除非 endio 回调已经把 page 状态、错误码和等待者唤醒都处理完。对写请求而言，“设备报告完成”也不必然等于掉电后数据存在，除非请求带有 FUA 或之前执行了正确 flush。

12.2 I/O 调度算法

合并与拆分。合并减少请求数量，提高吞吐。前向合并是新 bio 紧接已有请求后方；后向合并是新 bio 紧接已有请求前方。

```
try_merge(req, bio):
    if req.op != bio.op or req.device != bio.device:
        return false
    if req.end_sector == bio.start_sector:
        append(req, bio)
        return true
    if bio.end_sector == req.start_sector:
        prepend(req, bio)
        return true
    return false
```

合并复杂度取决于查找结构。只看队尾是 $O(1)$ ，但合并机会少；用按 sector 排序的树查找相邻请求是 $O(\log n)$ ，合并率更高。拆分发生在请求跨越设备最大段数、最大字节数、DMA 边界或 zone 限制时。教学模型可以只实现连续扇区合并，但要保留“设备约束可能要求拆分”的接口。

SCAN 和 deadline。SCAN 电梯算法按当前磁头方向服务请求，到尽头后反向。它适合解释机械磁盘为什么怕随机寻道。

```
pick_scan(queue, head, direction):
    candidates = requests_in_direction(queue, head, direction)
    if empty(candidates):
        direction = reverse(direction)
        candidates = requests_in_direction(queue, head, direction)
    return nearest_by_sector(candidates, head, direction)
```

SCAN 的吞吐较好，但远处请求可能等待很久。deadline 调度器给读写请求设置截止时间，选择时优先处理过期请求，否则继续按 sector 顺序提升吞吐。

```
pick_deadline(queue):
    if has_expired(queue.read_deadline):
        return pop_oldest(queue.read_fifo)
    if has_expired(queue.write_deadline):
        return pop_oldest(queue.write_fifo)
    return pop_by_sector_order(queue)
```

deadline 的本质是用最大等待时间约束吞吐优化。它不保证实时性，但能避免单纯电梯算法在混合负载下让读请求被大量写请求拖垮。

CFQ/BFQ 与公平性。桌面和多租户环境还需要进程间公平。按进程或控制组建立 I/O 队列，可以避免一个大顺序写任务压制交互读。BFQ 这类预算公平队列给每个实体分配服务预算，用“服务了多少字节”和“等待了多久”共同决定下一个队列。

```
pick_fair(queue_set):
    entity = min_virtual_finish_time(queue_set.active)
```

```

req = entity.pop_next_request()
entity.service += req.bytes
entity.vfinish = entity.vstart + entity.service / entity.weight
if entity.has_more():
    reinsert(entity)
return req

```

公平调度会牺牲部分全局顺序性，尤其在机械磁盘上可能增加寻道。工程选择取决于目标：批处理吞吐、桌面响应、数据库尾延迟还是云主机隔离。

12.3 持久化语义与多队列

flush、FUA 与 barrier。现代存储有缓存。普通写完成可能只表示数据到达设备缓存，不表示掉电持久。flush 请求要求设备把缓存写入介质；FUA 要求当前写绕过或强制落到持久介质；barrier 规定顺序，防止前后的写被重排穿越。

```

submit_ordered_write(data_req):
    if data_req.requires_preflush:
        submit(FLUSH)
        wait_flush_done()
    submit(data_req with FUA if requested)
    wait_data_done()
    if data_req.requires_postflush:
        submit(FLUSH)
        wait_flush_done()
    complete_to_filesystem()

```

这段逻辑解释了为什么 **fsync** 慢：它不只是把 page cache 交给设备，还可能要求等待队列中相关写完成，并发出 flush 来建立掉电后的顺序证据。

多队列 NVMe。NVMe 设备支持多个 submission/completion queue。块层会把请求映射到 CPU 本地硬件队列，减少全局锁竞争。

```

submit_nvme(req):
    hctx = map_cpu_to_hw_queue(current.cpu)
    tag = allocate_tag(hctx)
    cmd = build_nvme_command(req, tag)
    sq = hctx.submission_queue
    sq.entries[sq.tail] = cmd
    sq.tail = next(sq.tail)
    ring_doorbell(hctx)

```

多队列提高并行度，但也削弱了全局排序。需要顺序语义的请求必须带明确依赖，不能假设“提交顺序就是完成顺序”。

blk-mq 对象关系。blk-mq 把软件提交队列映射到硬件队列，减少单锁瓶颈。核心对象包括 **request_queue**、**blk_mq_tag_set**、hardware context、software context 和 tag。tag 是请求在硬件队列中的身份，用 completion 找回 request。

```

submit_bio
-> blk_mq_submit_bio
-> get request tag
-> map current CPU to hctx
-> driver queue_rq
-> device completion interrupt
-> blk_mq_complete_request
-> bio end_io

```

读源码看 **block/blk-mq.c**、**block/bio.c** 和具体驱动的 **queue_rq**。关注

request 从 free tag 到 in-flight, 再到 completion 释放 tag 的生命周期。

12.4 块层的统计、限流与错误处理

IO 统计从哪里来。`/proc/diskstats`、`/proc/[pid]/io`、`iostat` 看到的数字来自块层和任务 I/O 统计。它们能说明请求数、扇区数、队列时间、服务时间, 但不能直接告诉你应用层哪个业务请求慢。需要把应用 trace、系统调用和块层事件串起来。

指标	解释注意点
read/write IOPS	请求次数, 受合并影响, 不等于应用调用次数
sectors	数据量, 可和吞吐换算
await/latency	队列等待加服务时间, 受队列深度影响
util	设备忙碌时间比例, 多队列设备上不等同饱和度

错误诊断不要只看 util。NVMe util 高不一定坏, 机械盘 util 高更可能意味着排队; await 上升要结合队列深度、flush、写回和 cgroup 限流。

IO 优先级和 cgroup。`ionice`、IO priority class 和 blk-cgroup 可影响不同任务的 I/O 服务顺序。云主机和容器环境常通过 cgroup v2 的 io controller 控制权重或限速。公平性会改变尾延迟: 低权重任务可能明显变慢, 高权重任务不应被后台写回完全淹没。

writeback、dirty throttling 与块层的关系。应用 `write` 把页标脏后, 脏页并不会无限积累。内核用 dirty ratio、dirty bytes、writeback 线程和 per-bdi 状态控制写回。当脏页过多时, 写入者可能被迫参与平衡, 表现为一次普通 `write` 变慢。

```
balance_dirty_pages(task, mapping):
    dirty = global_dirty_pages()
    if dirty < dirty_background:
        return
    wake_writeback_threads()
    if dirty > dirty_limit:
        sleep_or_writeback_until_below_limit(task)
```

这个机制把存储压力反馈给应用。没有 throttling, 快 CPU 可以瞬间制造大量脏页, 把内存挤爆; 有 throttling, 局部写入延迟会上升, 但系统保持可控。诊断写慢时要区分: 是用户态复制慢, page cache 分配慢, dirty throttling, 块层排队, 还是设备 flush。

zone 设备和顺序写约束。某些设备把空间划成 zone, 并要求一个 zone 内按顺序写入。文件系统和块层必须知道 zone write pointer, 不能随意把随机写重排进去。这个例子说明块层不只是性能优化, 还承载设备语义。

```
submit_zone_write(req):
    zone = lookup_zone(req.sector)
    if req.sector != zone.write_pointer:
        return -EIO
    dispatch(req)
    zone.write_pointer += req.length
```

对普通块设备, 逻辑块可随机覆盖; 对 zone 设备, 文件系统可能要采用 log-structured 布局或显式 reset zone。硬件约束改变, 上层算法就必须改变。

请求取消、超时与错误完成。块请求提交后可能超时、被设备 reset、被上层取消或完成失败。取消不是“把请求从队列里删除”这么简单, 因为请求可能已经在设备

硬件队列中，甚至设备正在 DMA。

```
timeout_request(req):
    mark req timed_out
    ask driver eh_timed_out(req)
    if driver says reset:
        freeze_queue()
        reset_device()
        complete_lost_requests(EIO)
        unfreeze_queue()
    else if request completed meanwhile:
        ignore timeout
```

这里存在竞态：timeout 线程准备处理请求时，中断也可能完成请求。request 状态机必须保证 completion 只发生一次，tag 只释放一次，bio endio 只调用一次。块层测试要专门构造 timeout 和 completion 交错。

从 fio 数字到 OS 解释。I/O benchmark 不能只报 MB/s 和 IOPS。高质量报告要写明 direct/buffered、sync/async、iodepth、block size、read/write mix、文件系统、是否预热、是否清缓存、是否包含 fsync。

参数	含义	影响
bs	每次 I/O 大小	合并、吞吐和延迟
iodepth	in-flight 请求数量	设备并行度和排队
direct	是否绕过 page cache	缓存干扰和对齐要求
fsync	是否等待持久化	flush 延迟和事务成本
rw	顺序/随机、读/写混合	调度器和介质特性

如果不说明这些参数，“磁盘很快”或“磁盘很慢”没有工程意义。OS 学习里的性能测试必须可复现，也必须能解释观测值背后的路径。

本章块层部分的合格标准：能说明 bio、request、queue 的区别；能分析 SCAN、deadline、公平调度的取舍；能解释 flush/FUA/barrier 为什么属于正确性而不仅是性能。

12.5 VFS 文件层

文件描述符是用户态访问内核对象的句柄。它可以指向普通文件、管道、socket、设备或匿名内核对象。进程拿到的是一个小整数，但内核维护的是对象、offset、权限、引用计数和关闭状态。

先看一段最普通的代码：

```
int fd = open("log.txt", O_CREAT | O_APPEND | O_WRONLY, 0644);
write(fd, "hello\n", 6);
close(fd);
```

初学时容易把它理解成三步：打开文件、把 6 个字节写到磁盘、关闭文件。真实路径更长：路径字符串要解析成目录项和 inode；open 要创建 open file description 并放入进程 fd 表；write 要验证 fd 权限、复制用户 buffer、更新

文件 offset、修改 page cache 或提交设备请求；close 只是释放一个引用，不保证已经持久落盘。

把错误版本写出来更清楚：

```
write(fd, buf, len):
    inode = fd_to_inode(fd)
    disk.write(inode.block, buf, len)
    return len
```

这个版本至少错在五处。第一，fd 不直接指向 inode，中间还有 open file description，它保存 offset 和 status flags。第二，O_APPEND 必须在内核里原子决定写入位置，不能让用户态先 lseek。第三，普通文件写通常先进 page cache，write 返回不等于磁盘完成。第四，写入可能短写或被信号打断。第五，错误可能异步出现，例如后台 writeback 失败，之后 fsync 才报告。

文件 I/O 的第一层对象关系是：路径字符串经过路径解析找到目录项和 inode；open 产生 open file description；进程 fd 表保存一个小整数到 open file description 的引用。dup 产生新的 fd，但通常仍指向同一个 open file description，因此共享 offset 和状态。再次 open 同一路径通常产生新的 open file description，因此 offset 分离。

普通文件读写经常经过 page cache。读命中 page cache 时不碰设备；读未命中时可能提交块设备请求并等待完成。写通常先把用户字节复制进 page cache，把页标记为 dirty，再由后台 writeback 或 fsync 推进到设备。write 返回不等于数据安全落盘，这一点必须反复强调。

设备可以看作有特殊行为的文件对象。终端、磁盘、网卡、timerfd、eventfd、epoll、pipe、socket 都可以放进 fd/event 模型，但它们的 read/write 语义不同：有的返回字节流，有的返回结构化事件，有的可能阻塞，有的可能短读短写，有的只表示 readiness。

阻塞、非阻塞和异步是三种控制流模型。阻塞 I/O 让当前线程睡眠直到有结果；非阻塞 I/O 在不能推进时返回 EAGAIN，调用者等待 readiness 后重试；异步 I/O 提交请求，completion 稀后返回结果。三者没有绝对高低，关键是请求身份、buffer 生命周期、短读短写、错误传播和背压。

背压是 I/O 系统的安全阀。无界队列能让短 benchmark 看起来很快，因为生产者从不等待；一旦设备变慢，内存会持续增长，尾延迟扩大，最终进程可能被杀。可靠系统要同时限制 item 数、字节数和 in-flight 请求数，并在关闭时 drain 或 cancel。

核心思想

fd 是入口，file 是打开实例，inode 是文件身份，page cache 是缓存和脏页账本，block request 是设备请求。文件 I/O 的连贯理解来自把这几层一条串起来，而不是把 read/write 当成直接磁盘函数。

fd 分配表。进程的 fd table 可以先看作数组：下标是 fd，元素是指向 open file description 的引用。open 要找最小空闲 fd；close 清空槽位并减少引用计数；dup 找新槽位并增加引用计数。

```
alloc_fd(file):
    for i in 0..max_fd:
        if table[i] == null:
            table[i] = file
            file.refcount += 1
            return i
    return -EMFILE

close(fd):
```



```

file = table[fd]
if file == null: return -EBADF
table[fd] = null
file.refcount -= 1
if file.refcount == 0:
    file.ops.release(file)

```

线性扫描是 $O(n)$ ，但简单。真实系统会用 bitmap 或 hint 记录下一个可能空闲位置，把常见分配降到接近 $O(1)$ 。无论怎么优化，不变量都是：fd 只是进程局部索引，open file description 才保存 offset、flags 和对象引用。

路径解析。路径解析把字符串变成目录项和 inode。绝对路径从根目录开始，相对路径从当前工作目录开始；每个路径分量都要查目录；遇到符号链接可能要跳转；权限和 mount namespace 会改变可见结果。

```

resolve(path):
    node = path.starts_with("/") ? root : current.cwd
    for component in split(path, "/"):
        if component == "." continue
        if component == "..": node = node.parent
    else:
        node = lookup_child(node, component)
        if node is symlink:
            node = resolve_symlink(node)
    return node

```

路径解析的复杂度大致和路径分量数相关，每个目录查找又取决于文件系统目录结构。缓存 dentry 可以避免每次都从磁盘读目录。安全实现还要限制符号链接递归深度，避免循环。

fd、file 和 inode 的并发引用。fd 是进程 fd 表下标，不是对象身份。内核执行 `read(fd)` 时，先把 fd 转成 `struct file` 引用；只要引用拿到，即使另一个线程随后 `close(fd)`，当前 I/O 也能在 file 对象上完成。close 删除的是 fd 表槽位，并减少 file 引用；它不能把正在执行的 I/O 用到的 file 直接释放。

```

sys_read(fd, buf, len):
    file = fget(fd)
    if file == null:
        return -EBADF
    ret = vfs_read(file, buf, len)
    fput(file)
    return ret

sys_close(fd):
    file = files_table_remove(fd)
    if file == null:
        return -EBADF
    fput(file)
    return 0

```

这个规则解释 fd reuse 风险：线程 A 保存整数 fd=5，线程 B `close(5)`，线程 C `open` 得到新的 fd=5。A 再用整数 5 时，访问的可能已经是另一个对象。内核靠 file 引用保护内部生命周期，用户态也要避免把裸 fd 长期跨线程传递而没有所有权约定。

open flags 的作用域。flag 不是都绑定在同一层。fd flag 绑定 fd 表项；file status flag 绑定 open file description。dup 后两个 fd 指向同一个 open file description，所以共享 file status flag 和 offset；但 close-on-exec

是 fd flag，每个 fd 可以不同。

标志	作用范围	边界
<code>O_CLOEXEC</code>	fd 表项	exec 提交时关闭该 fd，不影响其他 fd 的标志
<code>O_NONBLOCK</code>	open file description	dup 后共享，影响 read、write、accept、connect 的等待行为
<code>O_APPEND</code>	open file description	每次写前在内核内原子移动到文件尾
<code>O_DIRECT</code>	file I/O 路径	对齐、页 pin、缓存一致性和短 I/O 更复杂
<code>O_SYNC/O_DSYNC</code>	写入持久化语义	返回前要推进数据或元数据到更强边界

`O_APPEND` 不是用户态先 `lseek` 到末尾再 `write`。后者在并发下会竞争：两个线程都看到同一个旧文件尾，然后互相覆盖。append 语义要求内核在持有文件系统相关锁的状态下决定写入位置。

源码阅读入口。fd 表和引用看 `fs/file.c`、`include/linux/fdtable.h`；open 和路径解析看 `fs/open.c`、`fs/namei.c`；read/write 主干看 `fs/read_write.c`；page cache 看 `mm/filemap.c`；epoll 看 `fs/eventpoll.c`；io_uring 看 `io_uring/`。阅读时跟住一个 fd：它从整数变成 file，如何增加引用，如何进入具体 file operations，错误码在哪里返回，最后一个引用在哪里释放。

12.6 I/O 路径

VFS 的 read/write 并不直接落到设备。普通文件读写经过 page cache，未命中时才生成 bio 进入块层；direct I/O 绕过 page cache 但仍走块层提交。本节把缓存命中与未命中、buffered 与 direct、同步与异步的路径分别拆开。

page cache 读写。普通文件读路径先查 page cache。若缓存命中，从内核页复制到用户 buffer；若未命中，分配 page cache 页，提交块 I/O，等待完成，再复制。写路径通常复制用户数据到 page cache，标记 dirty，并在之后由 writeback 或 fsync 推进设备写。

```
buffered_read(file, offset, len):
    while len > 0:
        page_index = offset / PAGE_SIZE
        page = page_cache_lookup(file, page_index)
        if page == null:
            page = read_page_from_storage(file, page_index)
        n = copy_page_to_user(page, offset % PAGE_SIZE, len)
        offset += n
        len -= n
```

这个算法解释了冷读和热读差异。热读可能完全不碰设备；冷读可能等待存储。benchmark 若不说明缓存状态，就无法比较。

把一次冷读走完整。进程调用 `read(fd, buf, 6000)`，当前文件 offset 是 3000，页大小是 4096。第一次要读的是文件第 0 页的后 1096 字节，第二次要读第 1 页的前 4096 字节，最后还要读第 2 页的 808 字节。若这三页都不在 page cache，内核要为每页建立 page cache 条目，发起块 I/O，把线程挂到等待队列。设备 completion 到来后，页状态从 locked/loading 变成 uptodate，等待线程被唤醒，再把页内容复制到用户 buffer。

阶段	page cache 状态	线程看到什么
查第 0 页	miss, 分配 cache 页并标记 locked	不能直接返回, 需要等待 I/O
提交 I/O	块层生成 request, 设备 DMA 填页	线程 sleeping, 不占 CPU
completion	页标记 uptodate, 清除 locked, 唤醒等待者	线程变 runnable, 不一定立即运行
复制用户 buffer	从 page cache 页复制对应片段	可能再次遇到用户页 fault
推进 offset	成功复制多少, offset 推进多少	短读时不能假设请求全完成

热读路径不同：若页已经 uptodate，内核只需要拿到页引用并复制到用户 buffer，不发设备 I/O。这里的“快”来自 page cache 保存了文件内容的内存副本，而不是文件系统跳过权限和边界检查。读性能测试若第一轮冷读、第二轮热读，却把两者当同一种读，就会得出错误结论。

写路径可以按这条线理解：

```
sys_write(fd, user_buf, len):
    file = fget(fd)
    check file allows write
    decide offset, handling O_APPEND under lock
    copy bytes from user into page cache pages
    mark pages dirty
    update file size if needed
    return copied bytes

later:
    writeback finds dirty pages
    filesystem maps file offsets to blocks
    block layer submits requests
    device completes DMA
    fsync reports delayed writeback errors
```

这条线解释了三个常见困惑。第一，`write` 很快返回，是因为它可能只完成了“复制到内核缓存并标脏”。第二，系统掉电时，dirty page 还没写到持久介质就会丢。第三，`fsync` 的意义不是“再写一遍”，而是把先前的脏数据和必要元数据推进到更强的持久化边界，并把异步错误交还给调用者。

设备队列和完成。块设备、网卡和异步 I/O 都可以抽象成提交队列和完成队列。提交时生成 request，放入设备或驱动队列；完成时中断或轮询拿到结果，唤醒等待任务或投递 completion。

```
submit(req):
    req.id = next_id()
    req.state = IN_FLIGHT
    device_queue.push(req)
    kick_device()

complete(req_id, status, bytes):
    req = lookup(req_id)
    req.status = status
    req.bytes = bytes
    req.state = COMPLETED
    wake(req.waiter)
```

请求身份非常重要。异步完成可能乱序返回；取消后的请求也可能迟到完成；buffer 所有权必须跟着请求身份走。没有 request id，就无法把 completion 正确对应到提交者。

短读短写不是异常。普通文件顺序读常尽量返回请求长度，但很多 fd 都允许短读短写。pipe 可能只剩一部分数据；socket 是字节流，接收缓冲里有什么就先给什么；非阻塞 fd 无法继续推进时返回 `EAGAIN`；信号可能让系统调用返回 `EINTR`；存储错误可能让已经完成的部分和错误同时出现。

```
write_all(fd, buf, len):
    done = 0
    while done < len:
        n = write(fd, buf + done, len - done)
        if n > 0:
            done += n
            continue
        if errno == EINTR:
            continue
        if errno == EAGAIN:
            wait_writable(fd)
            continue
        return error
    return done
```

读路径也需要协议层状态机。对 TCP，`recv` 返回 20 字节不表示一个业务消息完整；对 UDP，一次 `recvmsg` 对应一个 datagram，buffer 太小会截断；对终端，行规程可能改变返回时机。OS 提供 fd 语义，应用协议必须自己定义消息边界。

短写也要用具体事故理解。假设应用要把 1MiB 日志写到非阻塞 socket，第一次 `write` 返回 64KiB，第二次返回 `EAGAIN`。如果应用把第一次返回当作“整条日志已经发送”，就会丢掉后 960KiB；如果它在 `EAGAIN` 时忙循环重试，会把 CPU 打满；如果它没有保存剩余 buffer 和当前偏移，等 socket 可写时也不知道从哪里继续。正确实现需要一个输出队列，记录每条待发送数据、已发送 offset 和连接关闭时的清理策略。

```
connection_on_writable(conn):
    while !conn.outq.empty():
        item = conn.outq.front()
        n = write(conn.fd, item.buf + item.off, item.len - item.off)
        if n > 0:
            item.off += n
            if item.off == item.len:
                conn.outq.pop_front()
                continue
        if errno == EAGAIN:
            arm_epollout(conn)
            return
        if errno == EINTR:
            continue
        fail_connection(conn, errno)
    return
```

这段代码把“短写不是异常”落实成状态机：数据没写完时必须保留；`EAGAIN` 表示当前 fd 暂不可推进；真正错误才关闭连接或上报。文件 fd、pipe、socket、终端的短 I/O 原因不同，但调用者都不能假设一次系统调用等于业务消息完整提交。

buffered I/O 和 direct I/O 的不同风险。buffered I/O 经过 page cache，适合普通应用和共享缓存；direct I/O 尽量绕过 page cache，适合数据库一类自己管理缓存和写入顺序的程序。direct I/O 不是“更高级的 read/write”，它把对齐、页

pin、设备队列、缓存一致性和错误处理暴露得更多。

路径	优点	代价
buffered read	命中 page cache 很快，可共享缓存	多一次复制，缓存污染可能影响其他任务
buffered write	快速标脏，后台批量写回	write 返回不代表持久化
direct	减少 page cache 干扰，适合自管缓存	对齐、pin 页、短 I/O 和并发一致性复杂
read/write		
mmap	load/store 直接访问映射页	缺页、SIGBUS、dirty/writeback 语义更隐蔽

direct I/O 和 buffered I/O 混用尤其危险。一个路径绕过 page cache，另一个路径读写 page cache，若文件系统没有正确失效或同步缓存，应用可能看到旧数据。真实系统会在文件系统层处理这类一致性，但应用设计仍应尽量避免无计划混用。

epoll item 绑定的是 file。 `epoll_ctl` 把一个 fd 对应的 file 和 interest mask 加进 epoll 实例。之后整数 fd 关闭并复用，不代表旧 epoll item 自动变成新文件的监听项。事件返回里带着用户当初登记的 fd 数字，但内核里的等待关系绑定的是旧 file。

```
epoll_ctl(epfd, ADD, fd):
    file = fget(fd)
    epitem.file = file
    epitem.fd_number_for_user = fd
    attach_poll_callback(file, epitem)

event delivery:
    file state changes
    callback enqueues epitem
    epoll_wait returns stored fd_number and event mask
```

事件循环通常要给每个连接对象加 generation 或指针身份，而不是只根据 fd 整数找状态。close、dup、fork、exec、线程并发都可能让 fd 数字的直觉失效。

io_uring 的提交、完成与取消。 io_uring 用共享提交队列 SQ 和完成队列 CQ 减少系统调用往返，支持固定 buffer、固定文件、poll、timeout、链式请求和异步取消。它不是“绕过内核”，而是把提交和完成批量化，让内核在更少入口中处理更多请求。

io_uring 把“提交请求”和“收完成结果”拆成两个环。SQE 中的 `user_data` 是应用识别 completion 的关键；CQE 中的 `res` 是最终字节数或负 `errno`。提交成功只表示内核接收了请求，不表示 I/O 完成。

```
userspace:
    sqe = get_sqe()
    sqe.op = READV
    sqe.fd = fixed_file
    sqe.buf = fixed_buffer
    sqe.user_data = request_id
    submit_sq_tail()
    io_uring_enter()          # or rely on SQPOLL

kernel:
    consume SQEs
    issue async work or direct submit
    post CQEs with res = bytes or -errno
```

fixed buffer/file 提高性能，但也把生命周期责任推给应用：buffer 在 completion 前不能释放，取消可能和完成竞态，部分完成必须按 CQE 结果处理。

取消请求要按竞态理解：取消到达时，请求可能还在队列里，可能已经派发给设备，可能刚刚完成。应用必须能接受“取消成功”“取消失败因为已经完成”“先收到完成后收到取消结果”等情况。只要 completion 可能到来，buffer 和请求对象就不能提前复用。

更具体地说，io_uring 取消不是“把那次 I/O 从世界上抹掉”。假设应用提交读请求 R，user_data=42，buffer 指向一块复用池内存。100ms 后应用超时，提交 cancel(42)。此时可能有三种真实状态：R 还在内核队列，取消可以把它摘掉；R 已经交给块设备，设备可能无法撤销，只能等 completion；R 已经完成，CQE 还没被应用收走。应用若在提交 cancel 后立刻把 buffer 给另一个请求使用，就可能和迟到的 R completion 冲突。

取消到达时	内核可能返回什么	应用必须怎样处理
请求未派发	cancel CQE 表示取消成功，原请求不会再完成	可以释放 buffer，但仍要消费 cancel 结果
请求已派发	cancel 可能失败或标记取消中，原请求仍会完成	buffer 继续保持有效，直到看到原请求 CQE
请求已完成	cancel 返回未找到或已完成	根据原 CQE 的结果处理，不重复释放
完成和取消乱序到达	先看到哪一个都可能	用 request 状态机合并结果，避免双重释放

所以异步 I/O 的基本规则是：提交记录、buffer 所有权和 completion 消费必须绑定。真正能释放或复用 buffer 的点，不是“我不想等了”，而是“所有可能引用这个 buffer 的请求状态都已经收敛”。这和驱动层 DMA request 的规则是同一件事，只是对象从设备 descriptor 换成了用户可见的 SQE/CQE。

12.7 实践

纸上实践一：画 fd 引用图。

```
process fd table:
fd 3 -> open file description A -> inode X
fd 4 -> open file description A -> inode X    # dup
fd 5 -> open file description B -> inode X    # open same path again
```

这张图能解释 offset 行为：fd 3 和 fd 4 共享 offset，fd 5 有独立 offset。并行文件读取如果使用 fd 3 和 fd 4，就可能因为共享 offset 得到不可预期的分片；使用 pread 或独立 open file description 更清楚。

纸上实践二：写出短写循环。任何 write 都可能只写入一部分字节，尤其是 pipe、socket、非阻塞 fd 或被信号打断时。正确代码必须记录已完成字节数，继续写剩余部分，遇到 EINTR 重试，遇到 EAGAIN 进入等待，而不是把短写当成完整成功。

纸上实践三：区分 readiness 和 message。epoll 告诉你 fd 可能可读或可写，不告诉你协议帧已经完整。网络程序必须有用户态 receive buffer、解析状态机和背压策略。把 fd readiness 当成完整请求，是事件驱动程序常见错误。

```
PreparedBuffer
-> Submitted
-> InFlight
-> Completed(bytes, error)
-> ConsumedByProtocol
```


-> BufferReleased

异步 I/O 中, buffer 在 completion 前不能被上游复用; completion 失败不能被提交路径吞掉; cancel 后也可能收到迟到 completion, 所以请求身份必须稳定。

12.8 测试

块层侧的测试覆盖合并、拆分、调度、flush 顺序和多队列身份:

1. 合并测试: 连续写 bio 应合并成较少 request, 非连续不得错误合并。
2. 拆分测试: 超过最大请求大小时必须拆分, 完成回调仍只向上层报告一次整体结果或清晰的部分错误。
3. SCAN 测试: 给定 head 和方向, 选择顺序可手算。
4. deadline 测试: 过期读请求应越过大量未过期写请求。
5. flush 顺序测试: 日志数据、flush、commit block 的完成顺序不能被重排破坏。
6. 多队列测试: 同一队列 tag 唯一, completion 能找到原请求。
7. 错误注入: 设备返回 I/O error 时, page、bio、request、等待者状态都能收敛。

文件 I/O 侧的测试覆盖 open/write/read、dup 后共享内容、close 一个 fd 后另一个 fd 仍有效、关闭所有引用后访问失败、未知 fd 返回诊断。还要测试 fd 分配单调或复用策略是否有明确规则。

- dup 后 offset 共享, 独立 open 后 offset 分离。
- pread 不改变 current offset。
- pipe 空读阻塞或非阻塞返回 EAGAIN, 写端关闭后读端得到 EOF。
- 读端关闭后写 pipe 失败, 不应静默丢数据。
- socket 或 pipe 短写后, 调用者继续写剩余字节。
- readiness 事件只触发解析尝试, 不被当成完整消息。
- 异步请求完成前, buffer 所有权不能释放。
- 队列达到字节上限时, 生产者受到背压。

第一版必须先保证句柄生命周期可解释。持久化和崩溃恢复不要另起一条线, 而要回到 fd、file、inode、page cache、block request 和目录项这些对象上验收。

12.9 文件系统恢复与崩溃一致性

VFS 给用户一个统一 fd 接口, 具体文件系统负责目录项、inode、块映射、权限、日志和恢复。路径解析得到 dentry/inode, open 生成 file, read/write 进入 file operations, 普通文件再落到 address_space、page cache 和文件系统块映射。主线里先抓住对象边界, 而不是先背某个文件系统格式。

对象	保存什么	关键不变量
dentry	名字到 inode 的缓存关系	rename/unlink 后缓存不能让旧名字错误复活
inode	文件身份、权限、大小、块映射、时间戳	多个路径和 fd 可引用同一 inode

file	一次打开实例、offset、status flags、ops	dup 共享，独立 open 分离
address_space	文件页缓存和写回状态	read/write/mmap 必须看到同一份 page cache 账本
journal/log	元数据或数据提交记录	崩溃恢复只能重放完整事务

文件系统恢复的核心不是“写得更勤”，而是“崩溃后能判断哪些写已经构成完整状态”。以创建文件并重命名为例，数据块、inode 大小、目录项、父目录元数据、日志 commit 和设备 flush 之间必须有顺序。若应用写临时文件、`fsync(tmp)`、`rename(tmp, final)`，但忘记 `fsync` 父目录，崩溃后目录项是否存在就依赖文件系统和挂载语义，不能当作通用保证。

```
durable_replace(tmp, final):
    write tmp contents
    fsync(tmp_fd)
    rename(tmp, final)
    fsync(parent_dir_fd)
```

这段不是所有场景的万能配方，但它展示了恢复协议的写法：每一步都要说明崩溃发生后恢复程序能看到什么。若看到旧文件，旧版本必须仍完整；若看到新名字，新内容必须已经同步；若看到临时文件，恢复程序应能删除或继续提交。文件系统、块层和设备只提供若干提交边界，业务正确性还要靠应用自己的版本号、校验和、manifest 或事务日志。

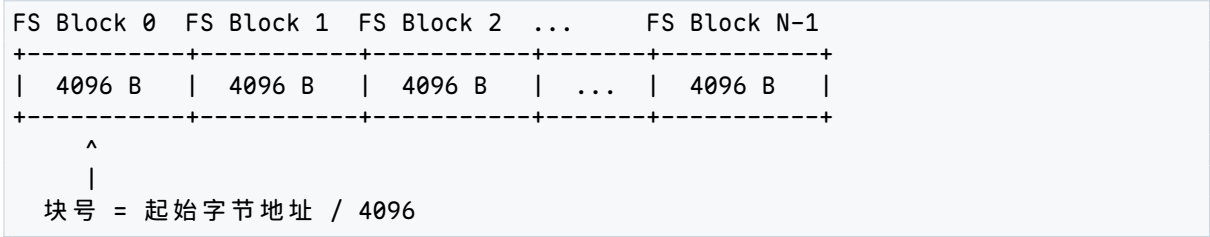
文件与 I/O 章的最终验收：跟住一个字节从用户 buffer 到 page cache、bio、request、设备 completion、writeback error、`fsync` 返回和恢复程序判断。若不能指出每层的状态对象和错误返回点，就不能声称理解了文件 I/O。

第 13 章 文件系统：从磁盘字节到现代设计

13.1 为什么需要文件系统

裸磁盘模型。在我们引入文件系统之前，先回到最底层：一块磁盘（无论是机械硬盘 HDD 还是固态硬盘 SSD）对 CPU 而言是什么？

从操作系统块层看，它可以先抽象成一个有固定容量、可按逻辑块寻址的线性空间。这里的“块”不是宇宙通用的 4096 字节：设备逻辑块、设备物理块、文件系统块和内存页是四个相关但不同的粒度。为便于后面的算例，本节暂时假设文件系统块为 4096 字节。

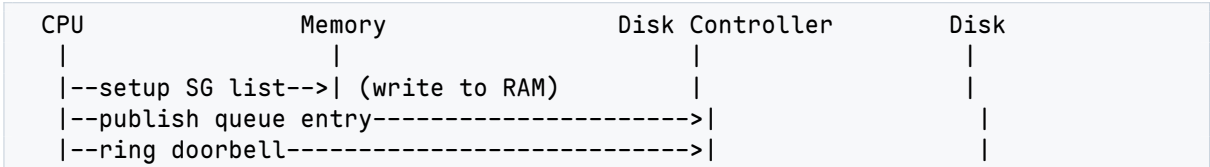


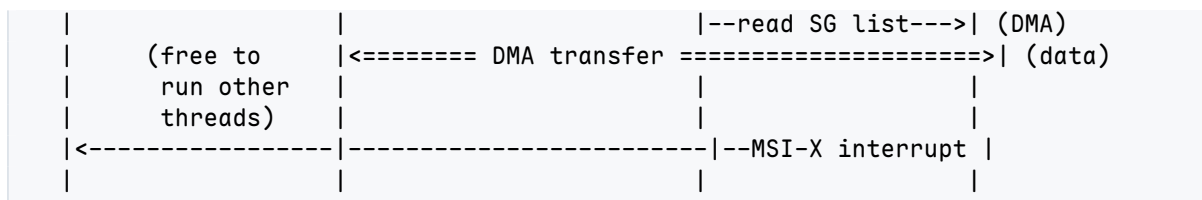
图中的 4096 字节只是本章示例。实际设备会报告 logical block size、physical block size、minimum I/O size 和 optimal I/O size；文件系统再选择自己的块大小，内存管理也有独立的页大小。CPU 不能直接用 `mov` 指令读写这些块—磁盘不在内存地址空间中。CPU 必须通过 磁盘控制器 (disk controller) 间接访问。

DMA 路径：CPU 如何让磁盘干活。现代高性能块设备通常用 直接内存访问 (Direct Memory Access, DMA) 传输数据。下面是队列式设备的简化路径；具体设备可能使用 submission queue、descriptor ring 或厂商自定义命令格式，不能把某一种寄存器布局当成所有设备的标准。

1. 内核分配并 pin 住传输缓冲区，通过 DMA API 把它映射成设备可见的 DMA 地址；启用 IOMMU 时，这通常是 IOVA，不等同于 CPU 物理地址。
2. 驱动在内存队列或描述符环中准备命令和散布-收集列表，并在发布前完成必要的内存排序。
3. 驱动更新队列尾指针或写 doorbell MMIO 寄存器，通知控制器有新命令。
4. 控制器开始工作：它自行读取 scatter-gather 描述符，然后通过总线在设备与已映射缓冲区之间传输数据。传输本身不依赖 CPU 执行逐字节复制指令，但 CPU 仍可能在提交前填充写缓冲区、在完成后解析或复制数据。
5. 数据传输完成后，控制器向 CPU 发出一个中断（现代设备用 MSI-X 消息信号中断）。
6. CPU 的中断处理程序被调用，确认 I/O 完成。

关键洞察：在 DMA 传输期间，发起 I/O 的线程通常不必占用 CPU 等待；CPU 可以执行其他线程，也可以按设备与负载选择中断或轮询完成队列。DMA 解决的是数据搬运方式，轮询与中断解决的是完成通知方式，两者不是互斥的替代关系。





一个能工作的裸磁盘程序。假设我们有一个简单的单任务环境。一个程序想把 8192 字节的数据存到磁盘上，过一会儿再读回来。它可以这样设计：

```
// 写数据到磁盘：块 100--101，共 8192 字节
int fd = open("/dev/sda", O_RDWR);
lseek(fd, 100 * 4096, SEEK_SET); // 定位到块 100
write(fd, my_data, 8192);        // 写 2 个块

// ... 时间流逝 ...

// 读回来
lseek(fd, 100 * 4096, SEEK_SET);
read(fd, my_data, 8192);
```

这在单任务、单程序、文件数量很少的场景下可以工作。程序自己记住“数据在块 100-101”，就像你在仓库里记住“东西放在第 100 号货架上”。

但只要场景稍微复杂一点，问题就来了。

裸磁盘的问题清单。下表列出了裸磁盘模型在面对真实多任务、持久化需求时的核心问题：

问题	具体困难	举例
命名	块只有编号，没有名字。程序必须自己记住“块 100 是我的数据”。用户不可能记住每个文件在哪个块。	<code>open("/home/alice/report.pdf")</code>
分配	谁来跟踪哪些块是空闲的？程序 A 用了块 100，程序 B 不能再用。没有全局的空闲块记录。	两个程序同时选了块 100
增长	文件写完后可能还要追加数据。如果块 100 后面的块 101 已被别人占用，文件无法连续增长。	<code>append()</code> 导致需要新块
共享	两个程序想读同一份数据，但它们各自记录不同的块号。如何让多个程序引用同一份数据？	Web 服务器和缓存代理读同一页
隔离	程序 A 不应该能覆盖程序 B 的数据。裸磁盘上任何程序都能写任何块。	恶意程序覆盖块 0（引导扇区）
崩溃	断电时写操作可能只完成了一半。块 100 写好了，块 101 没写。重启后数据不一致。	写到一半断电
缓存	同一个块被反复读取时没有缓存层。每次都走完整 DMA 路径。	反复读同一文件的头部
并发	两个程序同时写同一个块，后写的覆盖先写的，数据丢失。	两个进程同时 <code>write()</code> 同一偏移
碎片化	空闲块散布在各处，新文件的数据块不连续，导致随机 I/O 增多。	删删写写后空间碎片化
访问控制	谁能读哪些块？裸磁盘没有权限概念。	用户 A 读取用户 B 的邮件
持久化顺序	断电后哪些写会“活下来”？DMA 顺序 $\neq$ 发出 <code>write()</code> 的顺序。	先写的数据反而丢了

这 11 个问题不是独立的——它们相互交织。例如，崩溃恢复需要知道哪些写已经

落盘（持久化顺序），而并发写需要某种锁机制，锁机制又需要命名来标识被锁的对象。

文件系统（file system）就是解决这些问题的软件层。它在裸磁盘之上构建了一组抽象：文件（命名的字节序列）、目录（命名的层级结构）、权限（访问控制）、原子写（崩溃一致性）、缓存（性能）。

核心思想

文件的本质是**变换**：它把一个无名的、块号的线性数组变换为有名的、层级组织的、崩溃可恢复的、并发可访问的存储对象集合。这个变换需要四类磁盘上的数据结构（超级块、inode、目录项、空闲空间位图）和一组管理它们的算法（分配、查找、回收、缓存、日志）。

13.2 块设备接口：从 CPU 到磁盘字节

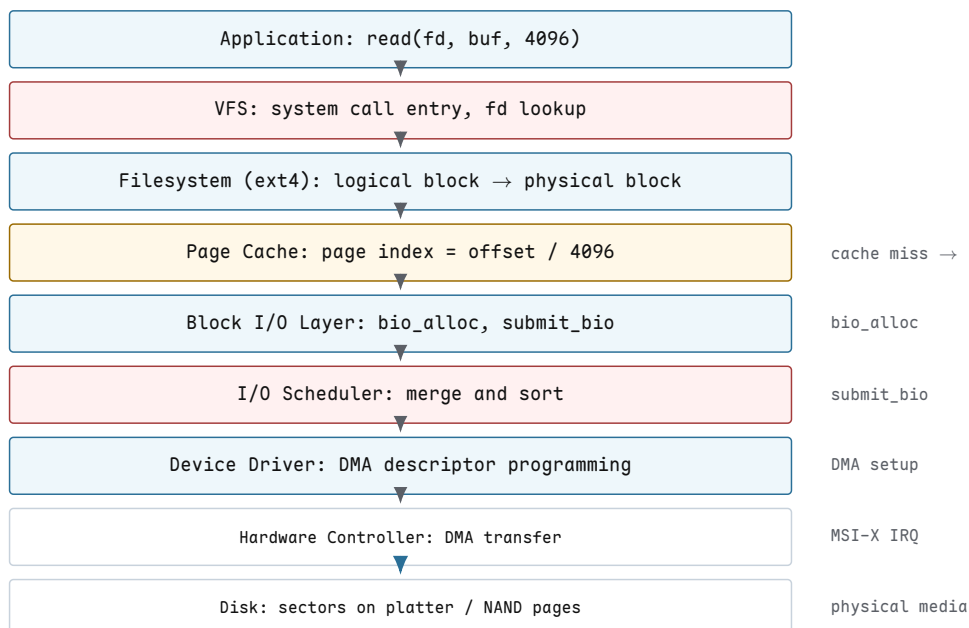
块设备作为文件。Linux 统一了设备接口：一切皆文件。磁盘 `/dev/sda` 也是一个文件。当你执行：

```
int fd = open("/dev/sda", O_RDWR);
```

你得到一个文件描述符，它支持 `read`、`write`、`lseek` 和部分同步操作。但它有固定容量，而且 I/O 必须服从设备拓扑和访问模式的约束：

- buffered I/O 可以接受字节范围，内核再通过缓存、合并和拆分生成满足设备限制的请求；`O_DIRECT` 等路径通常要求地址、偏移和长度满足对齐约束，否则可能返回 `EINVAL`。
- 小于物理写入粒度或跨越不利边界的写，可能在块层、设备固件或闪存 FTL 中触发读-修改-写，但这不是每次短写都由 Linux 内核固定执行的步骤。
- 同步操作要区分页缓存写回、块层 `flush/FUA` 和设备持久化承诺；具体语义依文件类型、驱动和设备能力而定。
- 块设备的容量就是可寻址范围。到达容量边界后读取会得到 EOF 或短读，越界写会失败；它与普通文件的区别是容量通常不能通过普通 `write` 自动增长。

Linux 块 I/O 栈。从用户空间的 `read()` 系统调用到磁盘上的物理扇区，数据要穿过多层软件栈。理解这个栈对理解文件系统的性能行为至关重要。



图示：数据从上到下穿过 8 层。上半部分（VFS、filesystem、page cache）在内核内存中操作页和 inode；下半部分（bio、scheduler、driver、hardware）操作扇区和 DMA 描述符。cache miss 是分水岭：命中时数据从内存返回，miss 时才需要走完整 I/O 路径。

下面逐层分析。

VFS 层。VFS (Virtual File System) 是所有文件系统的统一入口。当你调用 `read(fd, buf, 4096)`：

1. 内核通过 `fd` 在当前进程的文件描述符表中找到 `struct file`。
2. `struct file` 包含一个指向 `struct inode` 的指针和当前文件偏移量 `file->f_pos`。
3. VFS 检查偏移量是否超出文件大小。如果是，返回 0 (EOF)。
4. VFS 调用 `file->f_op->read` 或 `file->f_op->read_iter`，这是由具体文件系统（如 ext4）注册的回调。

VFS 本身不碰磁盘——它只是路由。

文件系统层：逻辑块到物理块的映射。ext4 文件系统收到 VFS 传来的请求后，需要把文件内的逻辑块号映射为磁盘上的物理块号。例如，用户想读文件偏移 8192 处的 4096 字节：

1. 逻辑块号 = $8192 / 4096 = 2$ 。
2. ext4 查询 inode 的 `i_block[15]` 数组，找到逻辑块 2 对应的物理块号（假设为 5000）。
3. 物理扇区号 = $5000 \times 8 = 40000$ （每个 4K 块 = 8 个 512B 扇区）。

这个映射可能需要多次磁盘读（如果需要遍历间接指针，见 13.3 节），也可能只需要一次内存查找（如果 inode 已在内存中）。

页缓存层。Linux 的页缓存（page cache）是文件数据的内存缓存。每个文件的数据以 4096 字节的页为单位缓存。页缓存的查找键是（inode, page_index）。

当文件系统需要读取文件的某页时：

1. 先在页缓存中查找 `find_get_page(mapping, index)`。
2. 如果找到且页标记为 `PG_uptodate`（数据有效），直接从内存复制到用户缓冲区——完全不碰磁盘。
3. 如果没找到（cache miss），分配一个新页，标记为 `PG_locked`，然后向块 I/O 层提交读取请求。
4. 当 I/O 完成后，页被标记为 `PG_uptodate` 并解锁。等待的 `read()` 系统调用被唤醒，从页缓存复制数据到用户缓冲区。

核心思想

页缓存是 Linux 文件 I/O 的核心设计：所有文件读写都经过页缓存。`read()` 先查缓存，miss 时才走 I/O 栈。`write()` 先写缓存页（标记为 dirty），由 `pdflush / writeback` 线程异步刷回磁盘。这意味着 `write()` 返回后数据不一定在磁盘上——只有 `fsync()` 保证落盘。

块 I/O 层：bio 结构。当页缓存 miss 时，文件系统需要从磁盘读取数据。它通过构造一个 `bio` (block I/O) 对象来描述这次 I/O 请求。


```

/* Simplified from Linux's include/linux/blk_types.h */
struct bio {
    sector_t        bi_sector;    /* starting sector on device */
    unsigned int     bi_size;      /* total bytes in this I/O */
    struct block_device *bi_bdev;  /* target block device */
    struct bio_vec    *bi_io_vec; /* array of page fragments */
    unsigned short   bi_vcnt;     /* count of bio_vecs */
    unsigned short   bi_idx;      /* current index into bi_io_vec */
    unsigned int     bi_opf;      /* operation flags (READ/WRITE) */
    bio_end_io_t     *bi_end_io;  /* completion callback */
    void             *bi_private; /* caller-private data */
};

struct bio_vec {
    struct page      *bv_page;    /* page in page cache */
    unsigned int     bv_len;      /* length of this segment */
    unsigned int     bv_offset;   /* offset within the page */
};

```

一个 **bio** 可以包含多个 **bio_vec**，每个 **bio_vec** 指向页缓存中的一个页面片段。这种设计允许一次 I/O 操作将磁盘上连续的扇区 散布 (scatter) 到内存中不连续的多个页面，或反向收集 (gather)。

一个 1 MiB 读取如何变成多个 bio。 用户调用 `read(fd, buf, 1048576)` (读 1 MiB)。假设文件偏移从 0 开始，块大小 4096，页大小 4096：

1. 1 MiB = 1048576 字节 = 256 个页。
2. VFS 为每个页检查页缓存。假设全部 miss。
3. 文件系统为每个页生成一个 **bio_vec**：bv_page 指向页缓存中新分配的页，bv_len = 4096，bv_offset = 0。
4. ext4 可能将逻辑上连续的多个页合并到一个 **bio** 中（前提是它们映射到磁盘上连续的物理块）。假设文件在磁盘上连续存放，256 个页可以合并成 1 个 **bio**：bi_sector = 起始扇区，bi_size = 1048576，bi_vcnt = 256。
5. 如果文件在磁盘上不连续（例如跨了两个物理区域），则拆分为 2 个 **bio**。

I/O 调度器：合并与排序。 bio 被 `submit_bio()` 提交到块设备的 I/O 队列后，由 I/O 调度器 (I/O scheduler) 处理。调度器的核心任务：

- **合并 (merging)**：如果新请求的扇区与队列中已有请求的扇区相邻，合并为一个更大的请求。减少请求开销。
- **排序 (sorting)**：对队列中的请求按扇区号排序，使磁头按顺序扫过盘面，减少寻道时间。

不同设备需要不同策略：

调度器	策略	适用设备	关键特性
none	不调度，直接提交	NVMe SSD	随机访问延迟一致， 无需排序
deadline	读写分离队列，按扇区 排序 + 截止时间	HDD，通用	防止请求饥饿
cfq (历史)	单队列时代按进程分配 时间片	旧式 HDD 桌面系统	已被 blk-mq 时代的 策略取代，不应作为 当前默认方案
bfq	预算公平队列，权重分配	HDD，桌面/移动	交互响应好

调度器	策略	适用设备	关键特性
mq-deadlin	多队列版 deadline	NVMe + HDD	兼顾排序与多队列

NVMe 没有机械寻道，因此按扇区排序的收益通常远小于 HDD，但不能据此规定“永远使用 `none`”。`none`、`mq-deadline` 或 BFQ 的选择还取决于延迟隔离、公平性、虚拟化层和工作负载。应在目标设备上比较吞吐与尾延迟，而不是把设备类型直接映射成唯一调度器。

DMA 深入：从描述符到中断。当 I/O 调度器将请求派发给设备驱动后，驱动程序需要设置 DMA 传输。现代设备常用描述符环、散布-收集列表或 `submission/completion queue`。以队列式设备为例：

1. CPU 在内存中构造一个或多个 DMA 描述符。每个描述符包含：
 - 设备可见的 DMA 地址与长度；启用 IOMMU 时地址通常是 IOVA
 - 传输长度（字节数）
 - 标志位（方向、中断使能）
 - 下一个描述符索引、地址或队列位置，具体由设备格式定义
2. 驱动把描述符发布到设备约定的内存队列；有的设备在初始化时配置队列基址，有的设备使用链式描述符。
3. 驱动执行必要的内存屏障，再更新队列尾指针或写 `doorbell`，触发控制器处理。
4. 控制器自行通过总线读取内存中的描述符（这是 DMA 本身的第一次使用——用 DMA 来读 DMA 描述符！）。
5. 控制器按照描述符链执行数据传输：直接在磁盘和 RAM 之间搬移数据，**不经过 CPU 寄存器**。
6. 每个描述符完成后，控制器可以发出一个中断；或者等所有描述符完成后发一个中断。现代设备通常用 MSI-X(Message Signaled Interrupts with eXtended)，设备发起约定地址与数据格式的内存写事务，由平台中断控制器路由到目标 CPU，无需传统中断引线。
7. CPU 收到中断，调用驱动注册的中断处理函数。处理函数标记 `bio` 为完成状态，调用 `bi_end_io` 回调。

核心思想

DMA 的本质是**让磁盘控制器成为总线主控 (bus master)**。控制器不再依赖 CPU 逐字节搬运数据，而是自行访问内存。CPU 的角色从“搬运工”变为“调度员”：设置描述符、发门铃、等中断。在 NVMe 协议中，每个 I/O 队列有多达 65536 个条目，控制器可以一口气处理大量请求而不需要 CPU 逐个干预。

完整追踪：`read(fd, buf, 4096)` 在偏移 8192 处。让我们追踪一次完整的 `read()` 调用，从用户空间到物理磁盘字节。假设文件 `fd` 对应 `inode` 中记录逻辑块 2 → 物理块 5000，块大小 4096，磁盘扇区 512 字节。

```
Step 1: VFS
- read(fd, buf, 4096) enters kernel via syscall
- fd -> file struct, file->f_pos = 8192
- file size check: 8192 + 4096 <= file_size? yes
- call file->f_op->read_iter()
```

```
Step 2: Page Cache lookup
```

- `page_index = 8192 / 4096 = 2`
- `find_get_page(mapping, 2) -> NULL` (cache miss)
- alloc new page, set `PG_locked`, add to page cache

Step 3: Filesystem block mapping

- `logical_block = 8192 / 4096 = 2`
- lookup inode `i_block[]`: logical 2 -> physical block 5000
- `sector = 5000 * 8 = 40000` (8 sectors per 4K block)

Step 4: bio construction

- `bio_alloc(bdev, sector=40000, size=4096, op=READ)`
- `bio_vec: bv_page = newly allocated page`
`bv_len = 4096`
`bv_offset = 0`
- `bi_vcnt = 1`

Step 5: submit_bio()

- bio enters device request queue
- I/O scheduler checks for merge opportunity:
 is there an adjacent pending request? (maybe merge)
 for NVMe: no sort, just submit to SQ (submission queue)
 for HDD: insert into sorted position by sector

Step 6: Device driver

- driver pops request from queue
- builds NVMe command or DMA descriptor:
`opcode = READ`
`slba = 40000` (starting LBA)
`nlb = 7` (number of logical blocks = 8 sectors - 1, 0-based)
`prp1 = DMA address returned by the DMA mapping API`
- writes command to submission queue tail
- rings doorbell register (`SQyTDBL`)

Step 7: DMA transfer

- NVMe controller fetches command from submission queue (via DMA)
- controller reads 4096 bytes from NAND at LBA 40000
- controller writes 4096 bytes through the mapped DMA address
- CPU is free to run other threads during this transfer
- transfer latency depends on device, queue depth, cache state and workload

Step 8: Interrupt

- controller writes completion entry to completion queue
- controller fires MSI-X interrupt vector N
- CPU enters driver interrupt handler (top half)
- driver reads completion queue entry, checks status
- driver calls `bio_end_io(bio)` -> marks page `PG_uptodate`, clears `PG_locked`
- wakes up the `read()` that was sleeping on this page

Step 9: Copy to user

- VFS resumes: page is now valid (`PG_uptodate` set)
- `copy_to_user(buf, page_address(page) + 0, 4096)`
- `read()` returns 4096

注意 Step 7: 设备通过 DMA 映射把数据写入内核页, CPU 不执行设备到内存的逐字节复制。Step 9 的 `copy_to_user` 仍由 CPU 完成; 若使用 direct I/O、零拷贝或用户页 pin, 数据路径又会不同。因此“DMA 不占用 CPU 搬运设备数据”不能简写成“整条 I/O 路径 CPU 不碰数据”。

Step 8 的完成上下文取决于驱动和配置: NVMe 可以在中断处理路径中处理完成队列, 也可以使用 polling 等模式。NAPI 是网络收包的中断缓和与轮询框架, 不

能作为通用 NVMe 完成机制的名称。

`read()` 成功返回表示请求的字节已经交给调用者；它不建立新的持久化承诺，也不需要随后调用 `fsync()` 来让这次读取“更可靠”。持久化问题属于写路径：`write()` 返回、脏页写回、块层完成和设备稳定介质承诺是不同边界。页缓存只说明读取可能从内存命中，不能据此推断数据此前是否由本次 I/O 从设备取回。

13.3 最小文件系统：四类对象

磁盘参数。为了具体化讨论，我们定义一个教学文件系统（Minimal FS, MDFS），运行在一块 1 GiB 磁盘上：

- 磁盘容量：1 GiB = 2^{30} 字节 = 1,073,741,824 字节
- 块大小：4096 字节（4 KiB）
- 总块数： $2^{30}/2^{12} = 2^{18} = 262144$ 块
- inode 大小：256 字节
- inode 总数：512
- inode 表大小： $512 \times 256/4096 = 32$ 块

磁盘布局如下：

块号	内容	说明
0	引导扇区	保留，包含引导加载程序。文件系统不使用。
1	超级块	文件系统全局元数据：总块数、空闲块数、inode 表位置等。
2	inode 位图	每位对应一个 inode，1 = 已分配。512 位 = 64 字节，远小于 1 块。
3-10	块位图	每位对应一个数据块，1 = 已分配。262144 位 = 32768 字节 = 8 块。
11-42	inode 表	$32 \text{ 块} \times 4096 / 256 = 512$ 个 inode。inode N 在块 $11 + N/16$ ，偏移 $(N\%16) \times 256$ 。
43-262143	数据块	实际存放文件数据。

这个文件系统需要管理四类磁盘上的数据结构：

1. 超级块（superblock）：全局元数据，1 块。
2. inode 表（inode table）：每个文件的元数据，32 块。
3. 目录项（directory entries）：目录的内容，存在数据块中。
4. 空闲空间位图（block bitmap + inode bitmap）：分配状态， $8 + 1 = 9$ 块。

超级块：全局元数据。

```
/* Block 1: superblock (64-byte header, rest is padding) */
struct superblock {
    uint32_t magic;                /* 0x4D534653 = "MDFS" magic number */
    uint32_t total_blocks;         /* 262144 */
    uint32_t free_blocks;         /* 262100 (initially, after mkfs) */
    uint32_t total_inodes;        /* 512 */
    uint32_t free_inodes;         /* 511 (inode 1 used by root dir) */
    uint32_t block_size;          /* 4096 */
    uint32_t inode_size;          /* 256 */
};
```

```

uint32_t root_inode;           /* 1 (root directory's inode number) */
uint32_t bitmap_start;        /* 2 (block where inode bitmap starts)*/
uint32_t inode_table_start;    /* 11 */
uint32_t data_start;           /* 43 (first data block) */
uint32_t reserved[5];          /* future use */
}; /* total: 16 * 4 = 64 bytes, rest of 4096-byte block is zero */

```

当 `mkfs` 创建文件系统后，块 1 的前 64 字节（小端序）如下：

偏移	字节 (hex)
0000:	53 46 53 4D 00 00 04 00 FF FF 03 00 00 02 00 00
	magic---- total_blk- free_blk-- total_ino
0010:	FF 01 00 00 00 10 00 00 00 01 00 00 01 00 00 00
	free_ino- block_sz- inode_sz- root_ino-
0020:	02 00 00 00 0B 00 00 00 2B 00 00 00 00 00 00 00
	bitmap--- inode_tbl- data_strt reserved0
0030:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
	reserved1 reserved2 reserved3 reserved4

逐字段解读：

- `magic = 0x4D534653`：小端序读为字节 `53 46 53 4D`，即 ASCII “MDFS”（反序）。文件系统挂载时检查此值，不匹配则拒绝挂载。
- `total_blocks = 0x00040000 = 262144`。
- `free_blocks = 0x0003FFD5 = 262101`。初始时 262144 块减去 43 块元数据 = 262101。再减去根目录占用的 1 块（块 43）= 262100。
- `total_inodes = 0x00000200 = 512`。
- `free_inodes = 0x000001FF = 511`（inode 1 已分配给根目录）。
- `block_size = 0x00001000 = 4096`。
- `inode_size = 0x00000100 = 256`。
- `root_inode = 1`，`bitmap_start = 2`，`inode_table_start = 11`（`0x0000000B`），`data_start = 43`（`0x0000002B`）。

inode：文件的元数据。inode（index node）是文件系统中每个文件的元数据锚点。它存储除文件名以外的所有文件属性。我们的教学文件系统使用 256 字节的 inode（与 ext4 默认值相同），但核心字段布局沿用 ext2 的经典 128 字节格式：

```

/* 256-byte inode (first 128 bytes shown, matching ext2 layout) */
struct mdfs_inode {
    uint16_t i_mode;           /* file type + permission bits */
    uint16_t i_uid;            /* owner user ID (low 16 bits) */
    uint32_t i_size;           /* file size in bytes */
    uint32_t i_atime;          /* last access time (Unix epoch) */
    uint32_t i_ctime;          /* inode change time */
    uint32_t i_mtime;          /* data modification time */
    uint32_t i_dtime;          /* deletion time (0 if alive) */
    uint16_t i_gid;            /* group ID (low 16 bits) */
    uint16_t i_links_count;    /* hard link count */
    uint32_t i_blocks;         /* sectors used (NOT block count!) */
    uint32_t i_flags;          /* ext4 flags (immutable, append, etc.) */
    uint32_t i_osd1;           /* OS-specific data */
    uint32_t i_block[15];      /* 12 direct + 1 indirect +
                               /* 1 double + 1 triple indirect */
    uint32_t i_generation;     /* inode version (for NFS) */
    uint32_t i_file_acl;       /* extended attribute block */

```

```

uint32_t i_dir_acl;      /* high 32 bits of size (dirs) */
uint32_t i_faddr;       /* fragment address (obsolete) */
uint8_t i_osd2[12];     /* OS-specific (uid/gid high,
                        /* reserved blocks, etc.) */
/* --- ext4 extension: bytes 128--255 --- */
uint32_t i_ctime_extra;  /* sub-second ctime precision */
uint32_t i_mtime_extra; /* sub-second mtime precision */
uint32_t i_atime_extra; /* sub-second atime precision */
uint32_t i_crtime;      /* creation time */
uint32_t i_crtime_extra; /* sub-second crtime precision */
uint32_t i_version_hi;   /* high 32 bits of i_generation */
uint32_t i_projid;       /* project ID */
uint8_t i_checksum[8];  /* inode checksum (crc32c) */
};

```

inode 的字节级视图。假设有一个 10000 字节的普通文件，拥有者 uid=0, gid=0, 权限 0644, 创建时间戳为 1700000000 (2023-11-14 UTC)。文件数据占用 3 个块 (块 68, 69, 70)，因为 $\lceil 10000/4096 \rceil = 3$ 。

该 inode (前 64 字节, 小端序) 如下:

偏移	字节 (hex)	字段
0000:	A4 81 00 00 10 27 00 00 00 F1 53 65 00 F1 53 65	i_mode--- i_size--- i_atime- i_ctime-
0010:	00 F1 53 65 00 00 00 00 00 00 01 00 18 00 00 00	i_mtime- i_dtime-- uid gid i_links- i_blocks
0020:	00 00 00 00 00 00 00 00 44 00 00 00 45 00 00 00	i_flags- i_osd1--- block[0]- block[1]-
0030:	46 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	block[2]- block[3]- block[4]- block[5]-

逐字段解读:

- **i_mode** = 0x81A4。高 4 位 0x8 表示 S_IFREG (普通文件)。低 12 位 0x1A4 = 0644 (rw-r--r--)。完整值: S_IFREG | 0644 = 0100644 (八进制) = 0x81A4。
- **i_size** = 0x00002710 = 10000。文件精确大小 10000 字节。
- **i_atime** / **i_ctime** / **i_mtime** = 0x6553F100 = 1700000000, 即 2023-11-14T22:13:20Z。
- **i_dtime** = 0 (文件未被删除)。
- **i_uid** = 0 (root), **i_gid** = 0 (root)。
- **i_links_count** = 0x0001 = 1 (1 个硬链接)。
- **i_blocks** = 0x00000018 = 24。注意: 不是 3 (块数), 而是 24 (512 字节扇区数)。3 块 × 8 扇区/块 = 24 扇区。
- **i_block[0]** = 0x00000044 = 68 (物理块号)。i_block[1] = 69, i_block[2] = 70。i_block[3..14] = 0 (未使用)。

i_blocks 的语义极其反直觉: 它不是文件使用的块数, 而是文件使用的 512 字节扇区数, 包括元数据块 (如间接指针块、扩展属性块)。一个 1 字节文件在 4K 块系统上 **i_blocks** = 8 (1 块 × 8 扇区), 而不是 1。这个设计源于 ext2 时代, 为了兼容只有 512 字节扇区的设备。

目录项：文件名的存放。文件名不在 *inode* 中——文件名存放在目录文件的数据块里，以 目录项(directory entry, *dirent*) 的形式组织。*ext2/ext4* 的目录项格式如下：

```
/* ext4 directory entry (variable length) */
struct ext4_dirent {
    uint32_t inode;           /* inode number of this entry */
    uint16_t rec_len;         /* total size of this entry (bytes) */
    uint8_t name_len;         /* length of filename (bytes) */
    uint8_t file_type;        /* file type (see enum below) */
    char name[];              /* filename, NOT null-terminated */
    /* padded to 4-byte boundary */
};

/* file_type values */
#define EXT4_FT_UNKNOWN 0
#define EXT4_FT_REG_FILE 1 /* regular file */
#define EXT4_FT_DIR 2 /* directory */
#define EXT4_FT_CHRDEV 3 /* character device */
#define EXT4_FT_BLKDEV 4 /* block device */
#define EXT4_FT_FIFO 5 /* named pipe */
#define EXT4_FT_SOCK 6 /* socket */
#define EXT4_FT_SYMLINK 7 /* symbolic link */
```

rec_len 的设计是 *ext2/ext4* 目录项的核心：它允许目录项在块内 紧凑排列，同时支持*rec_len* 覆盖被删除的条目（删除时不移动后续条目，只把被删条目的 *inode* 置 0 并把它 *rec_len* 加到前一条目）。

目录项的字节级视图。假设根目录(*inode* 1)的数据块中有一个条目指向文件 “foo” (*inode* 5)，以及标准的 “.” 和 “..” 条目：

目录数据块（块 43）内容：

偏移	字节 (hex)	解读
0000:	01 00 00 00 0C 00 01 02 2E 00 00 00	<i>inode</i> =1, <i>rec_len</i> =12, <i>namelen</i> =1, <i>type</i> =DIR, “.”
000C:	01 00 00 00 0C 00 02 02 2E 2E 00 00	<i>inode</i> =1, <i>rec_len</i> =12, <i>namelen</i> =2, <i>type</i> =DIR, “..”
0018:	05 00 00 00 0C 00 03 01 66 6F 6F 00	<i>inode</i> =5, <i>rec_len</i> =12, <i>namelen</i> =3, <i>type</i> =REG, “foo”
0024:	00 00 00 00 F4 0F 00 00 ...	<i>inode</i> =0 (deleted), <i>rec_len</i> =4012 (rest of block)

逐条解读：

- 偏移 0x0000: “.” 条目。*inode*=1 (根目录自身), *rec_len*=12 (8 字节头 + 1 字节名 + 3 字节填充), *name_len*=1, *file_type*=2 (DIR), 名字 = “.”。
- 偏移 0x000C: “..” 条目。*inode*=1 (父目录也是根目录自身), *rec_len*=12, *name_len*=2, *file_type*=2 (DIR), 名字 = “..”。
- 偏移 0x0018: “foo” 条目。*inode*=5, *rec_len*=12, *name_len*=3, *file_type*=1 (REG_FILE), 名字 = “foo” (0x66 0x6F 0x6F), 填充 1 字节 0x00。
- 偏移 0x0024: 剩余空间。*inode*=0 标记为空闲, *rec_len*=4012 (从 0x0024 到块末 = 0x1000 的剩余字节数)。

空闲空间管理：块位图与 *inode* 位图。MDFS 使用位图(bitmap) 管理空闲资源。每个块对应位图中一位：1 = 已分配，0 = 空闲。

- **块位图：**262144 个块需要 262144 位 = 32768 字节 = 8 个块 (块 3-10)。
- ***inode* 位图：**512 个 *inode* 需要 512 位 = 64 字节，远少于 1 个块 (块 2)。

块位图在创建 3 个文件后的状态。假设 `mkfs` 后，创建了根目录（占用块 43），然后创建 3 个文件：`/foo`（100 字节，块 68）、`/bar`（200 字节，块 69）、`/baz`（50 字节，块 70）。

已分配的块：0-42（元数据）、43（根目录数据）、68、69、70。

位图前 10 字节如下（bit 0 = 块 0，LSB first）：

```
字节 0 (块 0-7):   FF = 11111111  块 0-7 全部已分配
字节 1 (块 8-15):  FF = 11111111  块 8-15 全部已分配
字节 2 (块 16-23): FF = 11111111  块 16-23 全部已分配
字节 3 (块 24-31): FF = 11111111  块 24-31 全部已分配
字节 4 (块 32-39): FF = 11111111  块 32-39 全部已分配
字节 5 (块 40-47): 0F = 00001111  块 40-43 已分配, 44-47 空闲
字节 6 (块 48-55): 00 = 00000000  全部空闲
字节 7 (块 56-63): 00 = 00000000  全部空闲
字节 8 (块 64-71): 70 = 01110000  块 68-70 已分配, 其余空闲
字节 9 (块 72-79): 00 = 00000000  全部空闲
...
```

`ext2/ext4` 中位图是小端序 **bit order**：字节内的 bit 0 (LSB) 对应最低编号的块。即字节 `0x0F = 0b00001111` 表示块 0-3 已分配，块 4-7 空闲。这与大端序的直觉相反。x86 架构下，`testb` 指令测试的是 LSB，与这种 bit order 一致。

完整追踪：创建 `/foo`（100 字节）。现在我们追踪一次完整的 `create_file` 操作，从路径解析到磁盘写入。

```
=== create_file("/foo", 100 bytes) ===

Step 1: path_resolve("/foo")
- Start at root inode (superblock.root_inode = 1)
- Load inode 1 from inode table
- Root is a directory (i_mode & S_IFDIR), proceed

Step 2: dir_lookup(root_inode=1, "foo")
- Read root's data block (block 43)
- Scan dirents: ".", "..", (deleted entry)
- "foo" not found -> ENOENT (good, we can create it)

Step 3: alloc_inode()
- Load inode bitmap (block 2)
- Scan for first 0 bit: bit 5 is free
- Set bit 5 -> inode 5 is now allocated
- Mark inode bitmap block dirty

Step 4: init_inode(5)
- Write inode 5 at block (11 + 5/16) = block 11, offset (5%16)*256 = 1280
- Fields:
  i_mode      = S_IFREG | 0644 = 0x81A4
  i_uid       = 0
  i_size      = 0 (will be 100 after write)
  i_atime     = i_ctime = i_mtime = current_time
  i_dtime     = 0
  i_gid       = 0
  i_links_count = 1
  i_blocks    = 0 (no data blocks yet)
  i_block[0..14] = 0 (all zero)
```

- Mark inode table block 11 dirty

Step 5: alloc_blocks(1)

- Load block bitmap (blocks 3-10)
- Scan for first free bit after data_start (block 43)
- Block 68 is free (bit 68 in bitmap = 0)
- Set bit 68 -> block 68 allocated
- Mark block bitmap block dirty
- Update superblock: free_blocks--

Step 6: set inode 5 block[0] = 68

- Write i_block[0] = 68 in inode 5's on-disk record
- Update i_blocks = 8 (1 block * 8 sectors/block)
- Update i_size = 100
- Mark inode table block 11 dirty

Step 7: write data

- Copy 100 bytes of user data into block 68
- Zero-fill remaining 4096 - 100 = 3996 bytes of block 68
- Mark block 68 dirty in page cache

Step 8: dir_add_entry(root_inode=1, "foo", inode=5)

- Read root's data block (block 43, already in page cache)
- Find the "deleted" entry at offset 0x0024 (inode=0, rec_len=4012)
- Split: new entry takes 12 bytes (8 header + 4 name) remaining rec_len = 4012 - 12 = 4000
- Write dirent:
 - inode = 5
 - rec_len = 12
 - name_len = 3
 - file_type = 1 (EXT4_FT_REG_FILE)
 - name = "foo\0" (padded to 4 bytes)
- Update the following entry's rec_len to 4000
- Mark root's data block (block 43) dirty
- Update root inode: i_mtime = current_time, i_size may grow
- Mark root inode dirty

Step 9: mark dirty

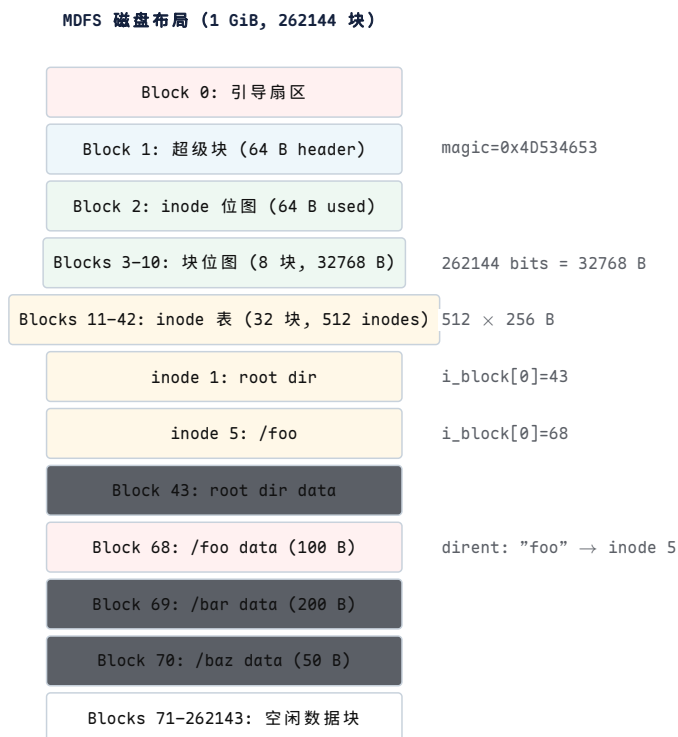
- Inode 5 dirty (metadata changed)
- Inode 1 (root) dirty (directory modified)
- Inode bitmap block dirty (bit 5 set)
- Block bitmap block dirty (bit 68 set)
- Superblock dirty (free_blocks decremented)
- Block 68 dirty (file data written)
- Block 43 dirty (dirent added)

Step 10: writeback

- Eventually, pdflush/writeback thread wakes up
- Flushes all dirty pages to disk via submit_bio()
- Order may vary (depends on journal mode)
- On ext4 with journal=data: metadata changes go through journal first
- On ext4 with journal=ordered: data is written before metadata
- fsync() waits for all these writes to complete

Step 10 是最危险的一步。如果在 Step 8 写入了目录项但 Step 5 的块位图还没落盘，断电后磁盘状态不一致：inode 5 的数据块 68 被标记为已分配（因为目录项指向 inode 5），但块位图说 68 空闲，导致块泄漏（leaked block）。反之，如果块位图先落盘但目录项没落盘，则块 68 被标记为已分配但没有文件引用它——也是泄漏。ext4 用日志（journal）解决这个问题：将元数据变更先写入日志区，原子提交后再应用到主区域。

磁盘布局图。



图示：红色 = 数据块，蓝色 = 超级块，绿色 = 位图，琥珀色 = inode 表，灰色 = 根目录数据。注意块 43（根目录数据）和块 68（/foo 数据）都在数据区（块 43+），它们没有特殊的存储位置——文件系统通过 inode 的 `i_block[]` 指针找到它们。

13.4 inode 的深层结构

inode 作为元数据容器。inode 是文件系统中最关键的数据结构。它存储了一个文件除了名字以外的所有信息：

- 身份：inode 编号唯一标识文件。
- 类型：普通文件、目录、符号链接、设备文件……
- 权限：谁可以读、写、执行。
- 大小：文件有多少字节。
- 时间戳：访问、修改、创建、删除时间。
- 数据位置：通过 `i_block[15]` 数组指向数据块。
- 链接计数：有多少个目录项指向这个 inode。

文件名不在 inode 中。文件名在目录文件的数据块中，作为目录项的 `name` 字段。一个 inode 可以有多个名字（硬链接），也可以没有名字但仍然存在（被进程通过 `open()` 持有但已从目录中删除）。

ext4 on-disk inode 格式。以下是 ext4 的实际 on-disk inode 结构（简化自 `fs/ext4/ext4.h`），标注了每个字段的偏移和用途：

```
/* ext4 on-disk inode (256 bytes, little-endian) */
struct ext4_inode {
    /* --- bytes 0--127: ext2-compatible core --- */
    __le16 i_mode;           /* +0:  file mode (type + perms)      */
    __le16 i_uid;            /* +2:  low 16 bits of owner UID       */
    __le32 i_size_lo;        /* +4:  low 32 bits of size (bytes)    */
    __le32 i_atime;          /* +8:  last access time              */
    __le32 i_ctime;          /* +12: inode change time             */
    __le32 i_mtime;          /* +16: data modification time        */
    __le32 i_dtime;          /* +20: deletion time                */
    __le16 i_gid;            /* +24: low 16 bits of group GID      */
    __le16 i_links_count;    /* +26: hard link count              */
    __le32 i_blocks_lo;      /* +28: low 32 bits of sector count    */
    __le32 i_flags;          /* +32: ext4 flags                   */
    __le32 i_osd1;           /* +36: OS-specific (Linux: version) */

    /* +40: block pointers (60 bytes) */
    __le32 i_block[15];      /* 12 direct, 1 indirect,           */
                             /* 1 double indirect, 1 triple      */

    __le32 i_generation;     /* +100: inode version (NFS)         */
    __le32 i_file_acl_lo;    /* +104: extended attribute block    */
    __le32 i_size_high;      /* +108: high 32 bits of size        */
    __le32 i_obso_faddr;     /* +112: obsolete fragment address   */

    /* +116: OS-specific area (12 bytes) */
    __le16 i_blocks_hi;      /* high 16 bits of i_blocks          */
    __le16 i_file_acl_hi;    /* high 16 bits of file ACL          */
    __le16 i_uid_high;       /* high 16 bits of UID               */
    __le16 i_gid_high;       /* high 16 bits of GID               */
    __le16 i_checksum_lo;    /* low 16 bits of checksum           */
    __le16 i_reserved2;      /* reserved                          */

    /* --- bytes 128--255: ext4 extensions --- */
    __le32 i_ctime_extra;     /* +128: extra ctime (ns precision)  */
    __le32 i_mtime_extra;     /* +132: extra mtime (ns precision)  */
    __le32 i_atime_extra;     /* +136: extra atime (ns precision)  */
    __le32 i_crttime;         /* +140: creation time               */
    __le32 i_crttime_extra;   /* +144: extra crttime (ns precision) */
    __le32 i_version_hi;     /* +148: high 32 bits of version     */
    __le32 i_projid;         /* +152: project ID                  */
    /* bytes 156--255: checksums, inline data, etc. */
};
```

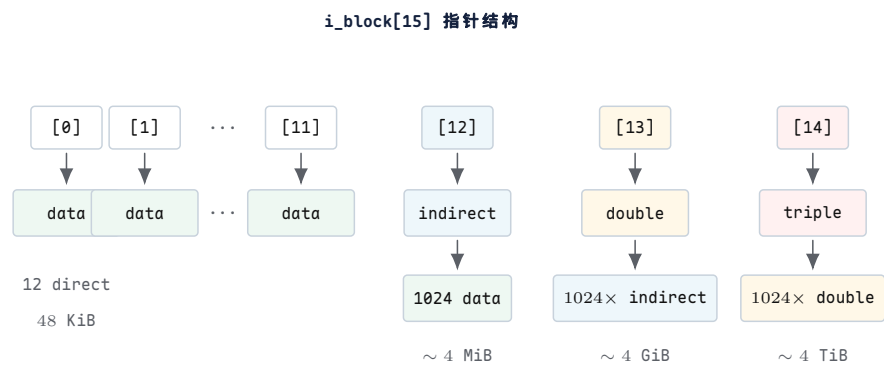
逐字段深入。

字段	偏移	含义与设计理由
<code>i_mode</code>	+0	16 位。高 4 位是文件类型（ <code>S_IFREG=0100</code> ， <code>S_IFDIR=0040</code> ， <code>S_IFLNK=0120</code> 等），低 12 位是权限位（ <code>rw-rw-rw</code> + <code>setuid/setgid/sticky</code> ）。用 16 位而非 32 位是为了节省空间——512 个 inode × 2 字节 = 1 KiB 节省。
<code>i_uid</code>	+2	16 位低部。加上 <code>i_uid_high</code> (+118) 组成 32 位 UID。早期 Unix 只有 16 位 UID（最多 65536 用户），足够小型系统。

字段	偏移	含义与设计理由
<code>i_size_lo</code>	+4	32 位低部。加上 <code>i_size_high</code> (+108) 组成 64 位文件大小。ext2 时代 <code>i_size_high</code> 用于目录 (<code>i_dir_acl</code>)，ext4 重新定义为大小高位，支持 > 4 GiB 文件。
<code>i_atime</code>	+8	最后访问时间。32 位 Unix epoch 秒。ext4 扩展 <code>i_atime_extra</code> 提供纳秒精度。注意： <code>atime</code> 更新会产生写 I/O！现代系统用 <code>noatime</code> 挂载选项避免此开销。
<code>i_ctime</code>	+12	inode 元数据修改时间（改权限、改属主时更新）。
<code>i_mtime</code>	+16	数据修改时间（写文件内容时更新）。
<code>i_dtime</code>	+20	删除时间。文件被删除时写入此字段。用于恢复工具（如 <code>extundelete</code> ）判断文件何时被删。非零意味着此 inode 曾被删除。
<code>i_links_count</code>	+26	硬链接计数。每创建一个硬链接 +1，每删除一个 -1。降为 0 时，inode 被释放（位图中对应位清零，数据块回收）。
<code>i_blocks_low</code>	+28	512 字节扇区数，不是块数！包括间接指针块和扩展属性块。加上 <code>i_blocks_high</code> (+116) 组成 48 位值。
<code>i_flags</code>	+32	ext4 特性标志： <code>EXT4_EXTENTS</code> （使用区段树而非经典间接指针）、 <code>EXT4_IMMUTABLE_FL</code> （不可修改）、 <code>EXT4_APPEND_FL</code> （仅追加）等。现代 ext4 默认设置 <code>EXT4_EXTENTS</code> ，此时 <code>i_block[15]</code> 的含义改变为区段树头。
<code>i_block[15]</code>	+40	60 字节的块指针数组。经典模式下：12 直接 + 1 间接 + 1 双重间接 + 1 三重间接。ext4 区段模式下：存储区段树根。
<code>i_generation</code>	+100	inode 版本号。每次 inode 被回收再分配时递增。NFS 用它检测“这个 inode 号是否还指向原来的文件”（防止 ABA 问题）。

i_block[15]：经典间接指针方案。当 `i_flags` 不包含 `EXT4_EXTENTS` 时，`i_block[15]` 使用经典的多级间接指针方案。这是 ext2/ext3 的设计，也是理解 inode 数据定位的基础。

15 个 `uint32_t` 槽位分为 4 组：



图示：蓝色 = 单重间接，琥珀色 = 双重间接，红色 = 三重间接，绿色 = 数据块。每个间接块包含 1024 个 4 字节指针（4096 / 4 = 1024）。颜色越深表示间接层级越多、寻址时需要的磁盘读次数越多。

精确容量计算。块大小 4096 字节，指针 4 字节，每块可容纳 $4096/4 = 1024$ 个指针。

层级	计算	容量 (字节)	磁盘读次数
12 直接	12×4096	49,152 (48 KiB)	1
1 间接	1024×4096	4,194,304 (4 MiB)	2
1 双重间接	$1024^2 \times 4096$	4,294,967,296 (4 GiB)	3
1 三重间接	$1024^3 \times 4096$	4,398,046,511,104 (4 TiB)	4
总计	以上之和	~ 4 TiB	最多 4

核心思想

多级指针方案的核心权衡是**查找深度 vs 元数据紧凑性**：小文件 (< 48 KiB) 只需 1 次磁盘读 (直接指针直达数据)，大文件需要 2-4 次。由于大多数文件很小 (Unix 系统中 > 90% 的文件 < 48 KiB)，这种设计在实践中非常高效。ext4 的区段 (extent) 方案进一步优化：用一棵 B+ 树替代 15 个指针槽，使连续区域用单个区段描述符表示。

追踪：读取偏移 5,000,000 处的数据。假设文件使用了所有四种指针类型。我们追踪读取文件内偏移 5,000,000 字节处所在的数据块。

目标：读取 byte offset 5,000,000

Step 1: 计算逻辑块号

```
logical_block = 5,000,000 / 4096 = 1220 (整数除法)
byte_offset_in_block = 5,000,000 % 4096 = 3120
```

Step 2: 判断指针层级

```
Direct blocks cover logical 0..11    -> 12 blocks
1220 >= 12, so NOT in direct range.
offset_in_indirect = 1220 - 12 = 1208
```

```
Single indirect covers logical 12..1035 -> 1024 blocks
1208 >= 1024, so NOT in single indirect range.
offset_in_double = 1208 - 1024 = 184
```

```
Double indirect covers logical 1036..1,049,611 -> 1,048,576 blocks
184 < 1,048,576, so it IS in double indirect range. ✓
```

Step 3: 读取双重间接块

```
double_indirect_block = read_block(inode.i_block[13])
-- DISK READ #1: read the double indirect block
-- This block contains 1024 pointers to single indirect blocks
-- We need entry index = 184 / 1024 = 0
```

Step 4: 读取单重间接块

```
indirect_ptr = double_indirect_table[0] -> block 6000 (example)
indirect_block = read_block(6000)
-- DISK READ #2: read the single indirect block
-- This block contains 1024 pointers to data blocks
-- We need entry index = 184 % 1024 = 184
```

Step 5: 读取数据块

```
physical_block = indirect_table[184] -> block 8000 (example)
data = read_block(8000)
-- DISK READ #3: read the actual data block
-- The byte we want is at offset 3120 within this block
```



```
Result: byte at physical block 8000, offset 3120
```

为什么是 3 次磁盘读？。在没有缓存的情况下，读取这一个数据块需要 3 次磁盘 I/O：

1. 读双重间接块 (`i_block[13]` 指向的块)。
2. 读单重间接块 (双重间接块中第 0 项指向的块)。
3. 读数据块 (单重间接块中第 184 项指向的块)。

如果磁盘是 HDD，每次随机读约需 5–10 ms，则总延迟 15–30 ms。如果是 NVMe SSD，每次约 50 μ s，总延迟约 150 μ s。

实际中，Linux 的预读 (readahead) 机制会提前读取间接块周围的数据，以及 inode 所在的整个块组。页缓存也会缓存间接块，后续访问同一区域的间接块时不再需要磁盘 I/O。

别混淆逻辑块和物理块。逻辑块号 1220 是文件内的第 1220 个块 (从文件头算起)。物理块号 8000 是磁盘上的第 8000 个块 (从磁盘头算起)。inode 的 `i_block[]` 数组做的正是从逻辑块号到物理块号的映射。这个映射可能不是连续的 (逻辑块 1220 在物理块 8000，逻辑块 1221 可能在物理块 5000)，这就是碎片化。

稀疏文件 (Sparse Files)。当 `i_block[]` 中的某个指针为 0 时，意味着该逻辑块没有被分配。这不是错误—读取该块时，内核返回一个全零页，而不是报错。这就是稀疏文件 (sparse file) 的工作原理。

```
// 创建一个稀疏文件：只有块 0 和块 1000 有数据
int fd = open("sparse.dat", O_CREAT | O_WRONLY, 0644);
write(fd, "hello", 5);           // 写块 0 (offset 0)
lseek(fd, 1000 * 4096, SEEK_SET); // 跳到块 1000
write(fd, "world", 5);           // 写块 1000
close(fd);

// inode 的 i_block[] 状态:
// i_block[0] = 68      (块 0 有数据)
// i_block[1..11] = 0    (块 1-11 是 "hole", 未分配)
// i_block[12] = 5000    (间接块, 指向块 1000 的间接指针)
// 间接块中: entry[988] = 70 (块 1000 的物理块号)
// 间接块中: entry[0..987] = 0 (块 12-999 是 hole)
//
// 磁盘实际占用: 3 块 (块 0 的数据 + 间接块 + 块 1000 的数据)
// 逻辑大小: 1001 * 4096 + 5 = 4,102,405 字节
// 实际占用: 3 * 4096 = 12,288 字节 (节省 99.7%)

// 读取 hole 区域:
char buf[4096];
lseek(fd, 500 * 4096, SEEK_SET); // 偏移在 hole 中间
read(fd, buf, 4096);              // 返回全零页, 不产生磁盘 I/O!
// 内核发现 i_block[] 指针 = 0, 直接返回 zero-filled page
```

核心思想

稀疏文件的核心机制是：`i_block[]` 中的 0 指针不是“无效”，而是“该逻辑块未分配，内容全为零”。读取 hole 不产生任何磁盘 I/O—内核直接返回一

个预制的零页 (zero page)。这使得创建巨大的稀疏文件（如虚拟机磁盘镜像、数据库预分配空间）几乎不占用磁盘空间。

inode 表的布局。inode 表是一段连续的磁盘块区域，存储所有 inode 的 on-disk 记录。每个 inode 256 字节，每个块 4096 字节，因此每个块容纳 $4096/256 = 16$ 个 inode。

```
inode 表布局 (blocks 11--42, 32 blocks, 512 inodes):

Block 11: inode 0    inode 1    inode 2    ... inode 15
Block 12: inode 16   inode 17   inode 18   ... inode 31
Block 13: inode 32   inode 33   inode 34   ... inode 47
...
Block 42: inode 496  inode 497  inode 498  ... inode 511

inode N 的位置:
    block  = inode_table_start + N / 16
            = 11 + N / 16
    offset  = (N % 16) * 256

例: inode 5
    block  = 11 + 5 / 16 = 11 + 0 = 11
    offset = (5 % 16) * 256 = 5 * 256 = 1280

    inode 5 位于块 11 的字节偏移 1280 (0x500)
```

ext2 中 inode 编号从 1 开始 (inode 0 无效)，但 inode 表在磁盘上从偏移 0 开始存储。因此 inode 1 在表偏移 0，inode N 在表偏移 $(N - 1) \times \text{inode_size}$ 。MDFS 为了简化教学，让 inode N 在表偏移 $N \times \text{inode_size}$ ，即 inode 1 在偏移 256。真实 ext2/ext4 中需要减 1。

硬链接与符号链接。

硬链接。硬链接 (hard link) 是目录中的一个额外条目，指向一个已存在的 inode 编号。创建硬链接不创建新 inode，而是：

1. 在目标目录的数据块中追加一个 dirent，inode 字段填为被链接文件的 inode 编号，name 填为新名字。
2. 将被链接文件的 i_links_count 加 1。

```
// 创建硬链接
link("/foo", "/bar");
// 效果:
//   /foo 的目录项: inode=5, name="foo"
//   /bar 的目录项: inode=5, name="bar" (新增)
//   inode 5 的 i_links_count: 1 -> 2
//
// 删除 /foo:
unlink("/foo");
//   /foo 的目录项被删除 (inode=0, rec_len 合并)
//   inode 5 的 i_links_count: 2 -> 1
//   inode 5 不被释放 (links_count > 0)
//
// 删除 /bar:
```

```

unlink("/bar");
// /bar 的目录项被删除
// inode 5 的 i_links_count: 1 -> 0
// inode 5 被释放！数据块回收，inode 位图清零

```

符号链接。符号链接 (symbolic link, symlink) 是一个独立的文件，其 `i_mode` 为 `S_IFLNK`。文件内容是目标路径的字符串。

```

// 创建符号链接
symlink("/foo", "/baz");
// 效果：
// 分配新 inode 6
// inode 6: i_mode = S_IFLNK | 0777
// inode 6: i_links_count = 1
// inode 6: 数据 = "/foo" (4 bytes, 存在 i_block[] 内联)
//
// 路径解析 open("/baz"):
// 1. dir_lookup(root, "baz") -> inode 6
// 2. read inode 6: i_mode = S_IFLNK
// 3. read symlink data: "/foo"
// 4. 重新解析 open("/foo")
// 5. dir_lookup(root, "foo") -> inode 5
// 6. open inode 5

// 删除 /foo 后：
unlink("/foo");
// inode 5 被释放 (links_count -> 0)
// /baz 仍然存在，但指向一个不存在的路径 -> "dangling symlink"

```

对比。

方面	硬链接	符号链接
实现	目录项指向同一 inode 编号	独立 inode，内容为目标路径字符串
inode	与原文件共享同一 inode	有自己的 inode
<code>i_links_count</code>	创建时 +1	新 inode 的 <code>links_count</code> = 1
原文件删除	数据不丢 (<code>links_count</code> > 0)	链接悬空 (dangling)
跨文件系统	不行 (inode 号是 FS 局部的)	可以 (只是路径字符串)
链接到目录	通常禁止 (防止循环)	允许
路径解析	直接找到 inode (1 次 lookup)	需读链接内容、重新解析 (2+ 次 lookup)
inode 大小	无额外空间 (仅目录项)	需要新 inode + 路径字符串

核心思想

硬链接和符号链接代表了两种不同的引用模型：硬链接是物理引用（直接指向存储对象的编号），符号链接是逻辑引用（通过路径名间接指向）。硬链接的代价是不能跨文件系统、不能链接目录；符号链接的代价是解析时有额外开销，且原文件移动/删除后链接失效。Unix 的设计哲学是同时提供两种机制，让用户根据场景选择。

inode 分配：Orlov 分配器。ext2/ext3/ext4 面临一个经典问题：新创建的 inode 应该放在 inode 表的哪个位置？这个决策影响数据局部性 (data locality) — 如果同一目录下的文件 inode 被放在相邻的块组 (block group) 中，读取这些文件时磁头移动更少。

ext4 默认使用 *Orlov* 分配器（由 Grigory Orlov 提出），其策略是：

1. **目录分散**：新目录应分散到不同块组，避免同一块组中出现过多目录（目录会产生大量子文件，导致该块组 inode 和数据块耗尽）。Orlov 倾向于选择空闲 inode 和空闲块最多的块组。
2. **文件聚集**：新文件应与父目录在同一块组中。这样，遍历目录时的 I/O 集中在一个块组内，磁头移动最小。

Orlov 策略示意：

Block Group 0	Block Group 1	Block Group 2
+-----+	+-----+	+-----+
inode: root/	inode: /docs/	inode: /media/
/src/	/docs/a	/media/x
/src/a	/docs/b	/media/y
/src/b	/docs/c	/media/z
+-----+	+-----+	+-----+

/root 和 /src 在 BG0；/docs 及其文件在 BG1；/media 及其文件在 BG2
目录分散到不同 BG；文件与父目录同 BG

Orlov 的核心启发式：目录是“重”的（会产生大量子文件），应分散；文件是“轻”的（不会产生子节点），应聚集。这种“重者散、轻者聚”的策略在大规模文件系统中显著减少了碎片化和寻道开销。

ext4 默认启用 `EXT4_EXTENTS` 标志（`i_flags` 中），此时 `i_block[15]` 不再是 12+1+1+1 的经典指针方案，而是存储一棵区段树（extent tree）的根节点。区段树用一个（`logical_block`, `length`, `physical_block`）三元组描述一段连续的物理块，大幅减少了大文件的元数据开销。经典间接指针方案仅在不支持区段的旧文件系统中使用。但理解经典方案对于理解 inode 的设计哲学仍然至关重要——它是区段方案的前身和对照。

13.5 块映射：从直接指针到 extent 树

问题：间接指针的大文件代价。经典的 UNIX 文件系统（ext2、UFS）在 inode 中保存若干直接指针、一个单间接指针、一个双间接指针和一个三间接指针。对小文件这足够了，但当文件增大时，随机访问一个偏移需要多次磁盘 I/O 才能定位数据块。

考虑一个 1 GiB 的文件。在 4 KiB 块、4 字节指针的前提下：直接指针只能覆盖前 12 个块（48 KiB），其余都要走间接。在 inode 已经驻留内存而间接块均未缓存时，双间接区要读取 2 个间接块，三间接区要读取 3 个间接块，然后才能读取数据块。对 HDD，这些元数据若分散在不同位置，机械寻道会显著放大延迟；具体数值取决于缓存命中、布局和设备，不能用一个固定毫秒数代表。

间接指针的本质缺陷：元数据散布在许多不连续的磁盘块中。一个 100 MiB 的文件需要 25 个间接块，它们随机散布在磁盘各处，导致寻道时间随文件大小线性增长。

各级指针的精确容量。设块大小 $B = 4096$ 字节，指针宽度 $P = 4$ 字节。一个间接块可容纳的指针数为：

$$N = \frac{B}{P} = \frac{4096}{4} = 1024 \text{ (个 uint32 指针)}$$

各级指针的覆盖范围如下表所示。

指针级别	指针数	覆盖块数	覆盖字节
直接 (12 个)	12	12	48 KiB
单间接 (1 个)	1024	1024	4 MiB
双间接 (1 个)	1024^2	1048576	4 GiB
三间接 (1 个)	1024^3	1073741824	4 TiB

表 5.1: ext2 inode 各级指针容量 (4 KiB 块、4 字节指针)。

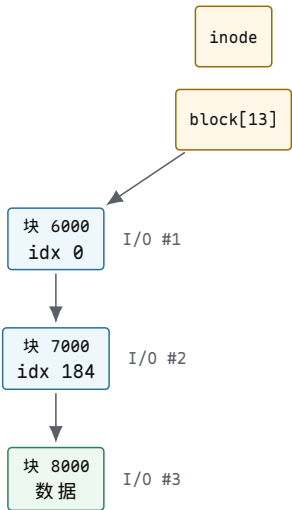
注意双间接理论上可覆盖 4 GiB，但 ext2 inode 的 `i_size` 字段限制了实际最大文件。真正的大文件落在三间接区，随机读需要读 3 个间接块再读数据块 = 4 次 I/O。

追踪：在双间接区读取偏移 5000000。给定文件偏移 `offset = 5\protect \protect \leavevmode@ifvmode \kern +.1667em\relax 000\protect \protect \leavevmode@ifvmode \kern +.1667em\relax 000` 字节，块大小 4096：

```
logical_block = 5000000 / 4096 = 1220    // 落在双间接区
// 直接区：0..11, 单间接区：12..1035, 双间接区：1036..(1036+1024*1024-1)
// 1220 在双间接区：index_into_double = 1220 - 1036 = 184
// first_level_index = 184 / 1024 = 0
// second_level_index = 184 % 1024 = 184
```

逐步追踪：

1. `logical_block = 1220`，落在双间接区 (1036 起)。
2. inode 已在内存中，其 `block[13]` 给出双重间接根块号 6000；读取块 6000 (第 1 次 I/O)。
3. 在块 6000 中取索引 0 的指针得到二级间接块号 7000；读取块 7000 (第 2 次 I/O)。
4. 在块 7000 中取索引 184 的指针得到数据块号 8000。
5. 读块 8000 → 数据。
6. 总计：2 次间接块 I/O + 1 次数据读 = 3 次 I/O (inode 已在内存、间接块均未缓存时)。若连 inode 所在磁盘块也未缓存，还需另计读取 inode 的 I/O。



图示：双间接读偏移 5000000 的三次 I/O 链：两个间接块加一个数据块。实际延迟由缓存命中、介质与队列状态决定。

间接指针的问题总结。

- **元数据膨胀**：100 MiB 文件需要 $\lceil (100 \times 1024 / 4) \rceil = 25600$ 个数据块，其中间接块约 25 个（每 1024 个数据块 1 个间接块），共 $25 \times 4 = 100$ KiB 纯元数据。
- **随机访问延迟**：三间接读在最坏的无缓存路径上需要 3 次间接块 I/O 加 1 次数据 I/O；SSD 消除了机械寻道，但没有消除额外请求、队列和控制器处理成本。
- **元数据碎片化**：间接块本身可能不连续，进一步加剧寻道。
- **无范围信息**：无法高效表达“这 1000 个块连续”，导致 **FIEMAP** 类查询效率低。

Extent：用 (start,length) 表达连续性。

核心思想

Extent 把“一个数据块指针”升级为“一段连续数据块的描述”：一个 extent 记录 (logical_block, start_block, length)，能覆盖任意长的连续区域。100 MiB 的连续文件只需 1 个 extent = 12 字节，而间接指针方案需要 25 个间接块 = 100 KiB 元数据。

ext4 extent 结构（源自 fs/ext4/extents.h）。

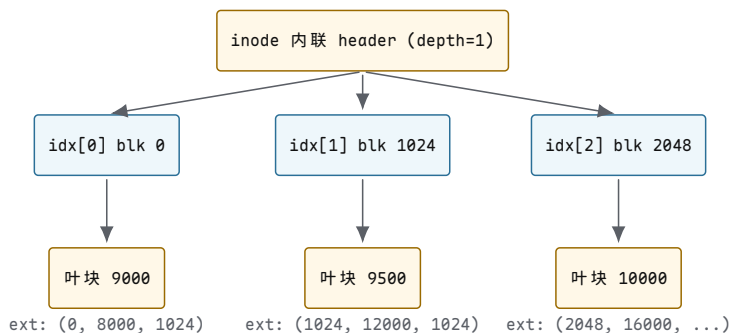
```
// 叶子节点中的 extent：描述一段连续物理块
struct ext4_extent {
    __le32 ee_block;        // 逻辑块号（文件内偏移）
    __le16 ee_len;          // 覆盖的块数（最多 32768）
    __le16 ee_start_hi;     // 物理起始块高 16 位
    __le32 ee_start_lo;    // 物理起始块低 32 位
};

// 索引节点：指向子节点（另一 extent 块）
struct ext4_extent_idx {
    __le32 ei_block;        // 此索引覆盖的逻辑块号下界
    __le32 ei_leaf_lo;      // 子节点块号低 32 位
    __le16 ei_leaf_hi;      // 子节点块号高 16 位
    __u16  ei_unused;       // 保留
};

// 每个 extent 块（含 inode 内联）的头部
struct ext4_extent_header {
    __le16 eh_magic;        // 0xF30A，标识 extent
    __le16 eh_entries;      // 实际条目数
    __le16 eh_max;          // 最大条目数
    __le16 eh_depth;        // 0=叶子，1=一层索引，2=两层
    __le16 eh_generation;   // 用于一致性校验
};
```

inode 尾部的 60 字节 (`i_block[15]`) 内联一个 extent header + 4 个 extent。当 `eh_depth=0` 时这 4 个 extent 直接描述数据；文件更大时 `eh_depth=1`，内联区变为 4 个 `ext4_extent_idx`，每个指向一个叶子块；再大则 `eh_depth=2`，极端情况可覆盖 ~ 4 PiB。

Extent 树结构。



图示：ext4 extent 树。根内联于 inode，depth=1 时指向若干叶块。

追踪：在 depth=1 树中查找逻辑块 1500。前提：inode 内联 header depth=1，有 3 个索引条目。

1. 读内联 header：depth=1，3 个 ext4_extent_idx。
2. 二分查找：ei[0].block=0，ei[1].block=1024，ei[2].block=2048。1500 落在 [1024, 2048)，即 ei[1]。
3. 读 ei[1].leaf → 叶块 9000。
4. 块 9000 header：depth=0，2 个 extent：ext[0]={block=1024, start=8000, len=1024}，ext[1]={block=2048, start=12000, ...}。
5. 1500 落在 ext[0]：物理块 = 8000 + (1500 - 1024) = 8476。
6. 总计：1 次 I/O（叶块）+ 1 次数据读 = 2 次 I/O。

对比：在 inode 驻留内存且元数据均未缓存时，双间接方案需要 2 次间接块 I/O，extent 树在这个示例里只需读取 1 个叶块，因为内联 header 不需要 I/O，而索引层只多一层。

Extent 的分裂与合并。

- **合并**：当新写入恰好紧接在已有 extent 末尾（逻辑块连续、物理块连续），直接把 ee_len 加 1，不新增条目。
- **分裂**：当 extent 块已满 (eh_entries == eh_max)，需要分裂：把一半条目移到新块，在父层插入新索引。若父层也满，递归分裂。
- **延迟分裂**：ext4 默认不立即分裂，先尝试在现有 extent 上扩展 ee_len，减少树高度。

Extent 树中的洞。两个 extent 之间的逻辑块间隙即为洞 (hole)。读洞时返回零页—ext4 不为洞分配物理块。FIEMAP ioctl 返回 FIEMAP_EXTENT_UNKNOWN 标志标记洞。写入洞的某个偏移时，ext4 分配块并在 extent 树中插入新条目。

XFS B+ 树：无历史包袱的设计。XFS 从设计之初就以 B+ 树为映射结构，没有间接指针的历史包袱。其 extent 记录格式更简洁：

```

// XFS B+ tree 叶子节点 (extent 记录)
struct xfs_bmbt_rec {
    __be64 br_startoff; // 逻辑块号
    __be64 br_startblock; // 物理块号
    __be64 br_blockcount; // 块数
};
// 内部节点：键 + 指针
struct xfs_bmbt_key { __be64 br_startoff; };
struct xfs_bmbt_ptr { __be64 br_startblock; };
  
```

XFS 的 B+ 树是真正的平衡树：插入/删除后自动调整高度与分裂/合并，而 ext4 extent 树的深度在多数文件上固定为 0-2。XFS 的优势在于动态伸缩：小文件 1 层即可，超大文件自动增到 3-4 层，查找代价始终 $O(\log n)$ 。

映射方案对比。

方案	元数据大小 (100 MiB 连续文件)	查找复杂度	碎片化倾向
ext2 间接指针	~ 100 KiB (25 间接块)	$O(\text{层数})$, 最多 4 I/O	高
ext4 extent	12 字节 (1 extent)	$O(\log n)$, 1-2 I/O	低
XFS B+ 树	~ 24 字节 (1 叶记录)	$O(\log n)$, 严格平衡	最低

表 5.2：三种块映射方案的定量对比。

13.6 空间分配：bitmap、块组与局部性

分配问题。当 `write()` 写入新数据时，文件系统必须决定把新块放在磁盘哪里。看似简单的选择，却影响三个关键指标：

- 连续性：连续布局使预读和顺序读受益，HDD 上减少寻道。
- 空间利用率：低碎片化意味着空闲空间可被大请求复用。
- 分配延迟：`write()` 路径上的分配必须快——它是热路径。
- 局部性：数据块应靠近其 inode 和同目录的其他文件，使目录遍历（如 `ls -R`）时的寻道最小。

这些目标常常冲突：best-fit 减少碎片但扫描慢；first-fit 快但可能产生外部碎片。现代文件系统用多层策略逼近帕累托前沿。

Bitmap 分配。最朴素的方案：每个块用 1 bit 标记空闲 (0= 空闲, 1= 已用)。1 GiB 磁盘、4 KiB 块：

块数 = $\frac{1\text{GiB}}{4\text{KiB}} = 262\,144 \Rightarrow \text{bitmap 字节} = \frac{262\,144}{8} = 32\,768\text{B} = 32\text{KiB}$

即 8 个块大小的 bitmap。分配时扫描 bitmap 找第一个 0 bit，复杂度 $O(n)$ (n 为块数)。对 1 TiB 磁盘，bitmap 达 32 MiB，全扫描不可接受，因此实际中 bitmap 按块组分区，配合缓存与索引。

first-fit / best-fit / worst-fit。

策略	规则	碎片化	速度
first-fit	选第一个足够大的空闲区	中等，外部碎片在前部	快
best-fit	选最小的足够大空闲区	低内部碎片，高外部碎片	慢（全扫描）
worst-fit	选最大的空闲区	高（浪费大区）	慢

表 6.1：三种经典分配策略。

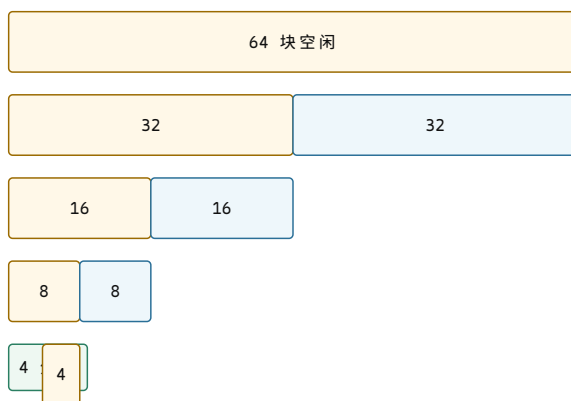
first-fit 追踪（空闲区链表：[0-31], [40-71], [80-95]，请求 8 块）：选 [0-31] 的前 8 块 → 剩 [8-31]。下次请求 28 块：[8-31] 只有 24 不够 → 跳到 [40-71]，分走 28，剩 [68-71]。外部碎片出现在前部。

best-fit 追踪（同上，请求 8 块）：扫描全部：[0-31](32)、[40-71](32)、[80-95](16)。8 最接近 16，选 [80-95]，分 8 块 → 剩 [88-95](8)。碎片被挤到小区域，但每次都要全扫描。

Buddy 分配器。Buddy 系统把空闲空间按 2^k 大小组织。分配请求 m 块时向上取整到 $2^{\lceil \log_2 m \rceil}$ ，递归分裂更大的块。

追踪：初始一个 64 块的空闲区，请求 4 块：

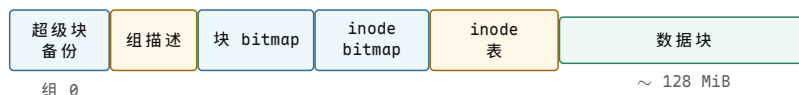
```
free[6] = {64}           // 只有 1 个大小为 64 的区
请求 4 块：
  分裂 64 -> 32 + 32,    选第一个 32
  分裂 32 -> 16 + 16,    选第一个 16
  分裂 16 -> 8 + 8,      选第一个 8
  分裂 8 -> 4 + 4,       选第一个 4 -> 分配
free[3] = {4}            // 剩下的 4
free[4] = {8}            // 剩下的 8
free[5] = {16}           // 剩下的 16
free[6] = {32}           // 剩下的 32
```



图示：Buddy 分配器：64 块区为服务 4 块请求，分裂 3 次，产生 3 个“伙伴”空闲块。

Buddy 优势：合并 $O(1)$ （释放时检查伙伴是否也空闲），但内部碎片高：请求 5 块也要分 8 块。ext4 的 `mballoc` 在 buddy 之上做了多块优化。

ext2 块组。ext2 把磁盘分成块组 (block group)，每组约 32768 个块 (4 KiB 块时 ~ 128 MiB/组)。每组内部布局：



图示：ext2 块组布局。元数据与数据同居一组，利于 HDD 局部性。

核心思想

块组的三大设计理由：

1. **局部性：**inode 与其数据块在同一组内，目录遍历寻道小。
2. **并行性：**不同组可被不同 CPU 并行分配（各组有独立 bitmap 锁）。
3. **故障隔离：**一个组 bitmap 损坏不影响其他组。

ext4 mballoc：多块分配器。ext4 的 `mballoc` (multi-block allocator) 在块组之上叠加多层策略：

- **Buddy bitmap：**每组维护 buddy 结构，查找连续 2^k 块的代价 $O(\log(\text{组内块数}))$ 而非 $O(n)$ 。
- **Locality group (局部性组)：**尝试把同一 inode 的数据和同目录的新文件分配到同一组。

- **Preallocation (预分配)**：对正在增长的文件，预留 8-32 KiB 连续块，使后续 `write()` 直接命中预留区。
- **Goal (目标)**：记录上次分配的最后块号，新分配尽量靠近 goal，使 extent 可合并。
- **Multi-block 一次性分配**：一次 `writeback` 可能脏了上百个页，`mballoc` 一次分配所有块，减少锁竞争。

延迟分配 (delalloc)。

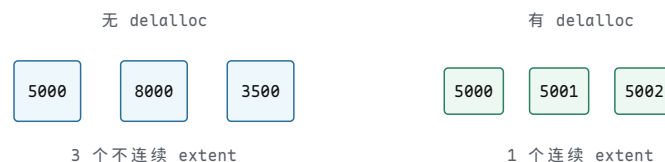
核心思想

延迟分配的核心思想：`write()` 只把数据写入页缓存，不分配物理块。等到 `writeback` 时，分配器看到完整的脏页范围，一次性分配连续块。

对比两种场景（写 3 个 4 KiB 页）：

```
// 无 delalloc: 每次 write() 立即分配
write(0..4095):    alloc block 5000
write(4096..8191): alloc block 8000    // 5000 已被占用，跳到 8000
write(8192..12287): alloc block 3500    // 又跳到 3500
结果：3 个 extent, 高碎片化

// 有 delalloc: writeback 看到全部脏页
write(0..4095):    page cache only
write(4096..8191): page cache only
write(8192..12287):page cache only
flush / writeback: alloc 5000-5002 (连续)
结果：1 个 extent, 无碎片
```



图示：延迟分配让 `writeback` 一次性分配连续块，消除逐次分配导致的碎片化。

delalloc 的 ENOSPC 困境。

延迟分配带来一个微妙问题：`write()` 在页缓存中成功，但 `writeback` 时可能发现磁盘已满 (ENOSPC)。此时 `write()` 早已返回 0 (成功)，用户以为数据已落盘，却在后台刷盘时失败。

ext4 的缓解措施：

- **预留块**：默认为 root 保留 5% 的块，使 root 进程 (含 `pdflush`) 有空间完成 `writeback`。
- **写时检查**：在 `write()` 路径上做粗略空间检查 (`ext4_nospace_check`)，但这是尽力而为。
- **回退到立即分配**：当剩余空间低于阈值，关闭 `delalloc` 改为同步分配，保证 ENOSPC 在 `write()` 时即报错。

Orlov 分配器。Orlov 策略针对目录分配：新目录应分散到不同块组，避免所有热目录挤在一个组导致该组 inode 表与 bitmap 成为瓶颈。规则：

- 新目录选空闲率最高的组（`free_blocks` 和 `free_inodes` 都较高）。
- 同一目录下的文件（非子目录）仍跟随父目录，保证目录内局部性。
- 目录分散 → 子树分散 → 负载均衡。

XFS 分配组 (Allocation Group, AG)。XFS 把磁盘分成独立的 AG，每组有自己的超级块、inode 表、两棵 B+ 树：

- **by-size 树**：键 = 空闲区长度，便于 best-fit 查找。
- **by-location 树**：键 = 起始块号，便于局部性分配。

两棵树互补：by-size 满足空间效率，by-location 满足局部性。不同 AG 可被不同 CPU 并行分配，每组各持一把自旋锁，`mp_io_resource` 级别的锁竞争极低。

13.7 目录结构：线性表、hash、B-tree

目录问题。目录本身也是一个文件，其“数据”是名字 → inode 号的映射表。关键问题：如何组织这张表使查找快、增删快、且向后兼容？

理想目录应满足：查找 $O(\log n)$ 或 $O(1)$ ，插入 $O(1)$ amortized，删除不移动数据（避免大目录整块搬移）。不同文件系统选择了不同平衡点。

线性目录 (ext2/ext4 基础格式)。ext2/ext4 的目录数据块是一串变长 `dirent`：

```
struct ext4_dirent {
    __le32 inode;      // 0 表示已删除
    __le16 rec_len;    // 本条目总长度 (含 padding)
    __u8  name_len;    // 文件名实际字节数
    __u8  file_type;   // DT_REG, DT_DIR, DT_LNK, DT_CHR, DT_BLK, DT_FIFO, DT_SOCK
    char  name[];      // name_len 字节, NUL 填充到 4 字节边界
};
```

`rec_len` 是变长布局的关键：它允许条目大小不等，删除时只需把被删条目的 `rec_len` 加到前一条目上——零数据搬移。

具体目录块（块 4096 字节，含 `..`、`...`、`foo.txt`、`bar.c`）：

偏移	inode	rec_len	name_len	file_type	name
0	12	12	1	DT_DIR	.
12	2	12	2	DT_DIR	..
24	15	20	7	DT_REG	foo.txt
44	16	16	5	DT_REG	bar.c
60	0	4036	0	—	(剩余空间, 空闲)

表 7.1：一个 ext4 目录块的具体布局。`rec_len` 使最后一条目吞掉所有剩余空间。

删除 `foo.txt`：把偏移 24 的条目 `rec_len` 累加到偏移 12 的 `..`；`..` 的 `rec_len` 变为 $12 + 20 = 32$ 。`foo.txt` 的空间被“折叠”进前一条，无需移动 `bar.c`。查找时遇到 `inode==0` 直接跳过。

线性查找复杂度。线性扫描是 $O(n)$ 。对 10 个文件的目录， $< 1 \mu s$ ，完全可接受。对 100 000 个文件的目录（如 `/usr/lib`、邮件 spool），每次 `stat` 都要扫半个目录——灾难性。这驱动了 HTree 和 B-tree 目录的诞生。

HTree: ext3/ext4 的 hash B-tree。HTree（也叫 `dx_tree`）是在线性目录之上叠加的 hash 索引。目录数据块仍是线性 `dirent`，但额外维护一个 B-tree 索引：

- **Hash 函数**：`half_md4` 或 `dx_hash`（tear、rupasov 等）。对文件名算 hash，hash 决定它落在哪个数据块。

- **索引节点**：一个 `(hash, child_block)` 对的数组。
- **查找**：`hash(name)` → 在索引中二分 → 读 `child_block` → 在该块的线性 `dirent` 中扫描。

```
struct dx_node {
    struct dx_entry entries[]; // 变长
};
struct dx_entry {
    __le32 hash;           // 子树的最小 hash
    __le32 child_block;    // 子目录块号
};
// 根索引内联于目录第一个数据块的开头
// (与 dirent 共存, 通过 dirent 的 rec_len 隐藏索引部分)
```

查找追踪（目录有 4 个数据块，HTree depth=1）：

1. `hash("foo.txt") = 0xA3B7`。
2. 根索引 4 个 entry: `[0x0000$ \rightarrow $blk100, 0x4000$ \rightarrow $blk200, 0x8000$ \rightarrow $blk300, 0xC000$ \rightarrow $blk400]`。
3. 二分：`0xA3B7` 落在 `[0x8000, 0xC000)` → `blk300`。
4. 读 `blk300`，线性扫描找 `name=="foo.txt"` → `inode 15`。
5. 总计：1 次索引 I/O + 1 次数据 I/O = 2 I/O（对比纯线性 4 I/O）。

核心思想

HTree 的精妙在于向后兼容：不理解 HTree 的工具（旧版 `e2fsck`、只读挂载）看到的仍是线性 `dirent`，因为索引被巧妙隐藏在第一个条目的 `rec_len` 里。只有 `EXT4_INDEX_FL` 标志位告知新内核“这里有索引”。

B-tree 目录 (XFS、Btrfs)。XFS `dir2` 和 Btrfs 把目录本身做成 B-tree，不再有线性 `dirent` 层：

- **XFS dir2**：叶节点存 `dirent` 数据，内部节点存 `(hash, ptr)` 索引。查找 $O(\log n)$ ，插入/删除自动再平衡。
- **Btrfs**：所有元数据统一为 B-tree，键为 `(objectid, type, offset)`，目录条目的 `type=BTRFS_DIR_ITEM_KEY`，值为完整 `dirent`。整个文件系统就是一棵大 B-tree 的不同子树。

B-tree 目录的优势：所有操作 $O(\log n)$ ，轻松扩展到百万级条目（如容器镜像层的 `/usr/lib`）。代价是结构复杂、小目录有额外开销（一个节点至少一个块）。

dentry 缓存 (dcache)。dcache 是 VFS 层的路径解析结果缓存，与具体文件系统无关：

- **正向 dentry**：名字存在，缓存了对应 `inode` 指针。再次访问该名字直接命中，无需调用底层 `lookup`。
- **负向 dentry**：名字不存在。缓存“不存在”这一事实，避免对缺失文件的反复磁盘查找（典型场景：`PATH` 搜索中 90% 的路径不存在）。

dcache 是基于路径的：`/home/user/foo` 被缓存为一串 dentry: `root $ \rightarrow $ home $ \rightarrow $ user $ \rightarrow $ foo`。每段都是独立缓存条目，可被不同路径复用。

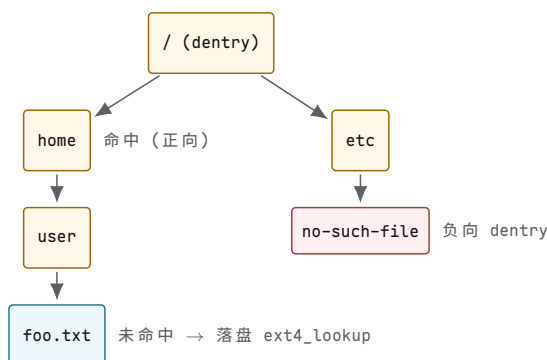
RCU-walk: Linux 3.2+ 引入的无锁路径遍历。遍历 dentry 树时用 RCU (Read-Copy-Update) 保护，不取任何锁，只在需要修改（创建/删除 dentry）时降级为 ref-walk。热路径 `stat("/usr/bin/gcc")` 几乎全在 RCU 下完成。

失效: `rename`、`unlink`、跨挂载点变化必须使受影响 dentry 失效 (`d_invalidate`)，否则返回过时结果。

带 dcache 的完整路径解析。

```
vfs_lookup("/home/user/foo.txt"):
    root_dentry = dcache_lookup(NULL, "/")           // 命中: 根 dentry
    home_dentry = dcache_lookup(root_dentry, "home") // 命中?
    if !home_dentry:
        home_inode = ext4_lookup(root_inode, "home") // 落盘
        home_dentry = d_add(root_dentry, "home", home_inode)
    user_dentry = dcache_lookup(home_dentry, "user") // 命中?
    if !user_dentry:
        user_inode = ext4_lookup(home_inode, "user")
        user_dentry = d_add(home_dentry, "user", user_inode)
    foo_dentry = dcache_lookup(user_dentry, "foo.txt") // 命中?
    if !foo_dentry:
        foo_inode = ext4_lookup(user_inode, "foo.txt")
        foo_dentry = d_add(user_dentry, "foo.txt", foo_inode)
    return foo_dentry->d_inode
```

热路径下，每一级 `dcache_lookup` 都命中，`ext4_lookup` 永不被调用——整个解析在内存中 $O(\text{路径深度})$ 。



图示: dcache 树: 正向 dentry 缓存存在路径, 负向 dentry 缓存“不存在”。

13.8 VFS: 统一接口与分发

VFS 问题。Linux 支持 `ext4`、`xfs`、`btrfs`、`nfs`、`proc`、`sysfs`、`tmpfs`、`fuse` 等数十种文件系统。应用程序只想用 `open/read/write/close`，不关心底层是 `ext4` 还是网络协议。

核心理念

VFS (Virtual File System Switch) 是内核中的一层抽象接口与分发器: 它定义了一组抽象对象和操作方法表, 每个具体文件系统实现这些方法。应用层看到统一 API, 内核根据挂载信息把调用分发到具体实现。

四个核心 VFS 对象。

super_block: 每个已挂载文件系统一个。

```
struct super_block {
    unsigned long s_blocksize;    // 块大小 (字节)
    unsigned long s_maxbytes;     // 最大文件大小
    unsigned long s_flags;        // MS_RDONLY, MS_NOSUID, ...
```

```

    unsigned long s_magic;           // 0xEF53 (ext2/3/4), 0x58465342 (xfs)
    struct inode *s_root;            // 根目录 inode
    struct xarray s_inodes;          // 本 fs 的所有 inode
    const struct super_operations *s_op;
    // ...
};
struct super_operations {
    struct inode *(*alloc_inode)(struct super_block *sb);
    void (*destroy_inode)(struct inode *);
    void (*evict_inode)(struct inode *);
    int (*write_inode)(struct inode *, struct writeback_control *);
    int (*sync_fs)(struct super_block *, int wait);
    // ...
};

```

super_block 在 `mount()` 时由 `fill_super` 回调创建，卸载时销毁。它持有全文件系统级别的状态（空闲块/inode 计数、脏标志）。

inode：每个文件一个（在内存中）。

```

struct inode {
    umode_t i_mode;                // 文件类型与权限
    loff_t i_size;                 // 文件大小（字节）
    blkcnt_t i_blocks;            // 占用的 512 字节块数
    const struct inode_operations *i_op;
    const struct file_operations *i_fop;
    struct address_space *i_mapping; // 页缓存
    struct super_block *i_sb;
    // ...
};
struct inode_operations {
    struct dentry *(*lookup)(struct inode *, struct dentry *, unsigned int);
    int (*create)(struct inode *, struct dentry *, umode_t, bool);
    int (*link)(struct dentry *, struct inode *, struct dentry *);
    int (*unlink)(struct inode *, struct dentry *);
    int (*mkdir)(struct inode *, struct dentry *, umode_t);
    int (*rename)(..., unsigned int);
    const char *(*get_link)(struct dentry *, struct inode *, void **);
    int (*permission)(struct inode *, int);
    // ...
};

```

内存 inode 由磁盘 inode 实例化，但额外维护运行时状态（页缓存引用、脏标志、锁、引用计数）。

dentry：路径组件缓存。

```

struct dentry {
    struct qstr d_name;            // 名字 + hash
    struct inode *d_inode;         // NULL = 负向 dentry
    struct dentry *d_parent;
    const struct dentry_operations *d_op;
    struct list_head d_subdirs;
    unsigned char d_flags;         // DCACHE_MOUNTED, ...
    int d_lockref;                // 引用计数
};

```

dentry 形成镜像目录结构的树，是 dcache 的载体。

file：每次 open() 一个。

```

struct file {
    struct path f_path;           // dentry + vfsmount
    struct inode *f_inode;
};

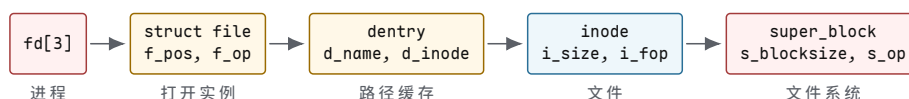
```

```

const struct file_operations *f_op;
loff_t f_pos;           // 当前偏移
fmode_t f_mode;         // FMODE_READ, FMODE_WRITE
unsigned int f_flags;    // O_RDONLY, O_NONBLOCK, ...
atomic_long_t f_count;  // 引用计数
// ...
};
struct file_operations {
    ssize_t (*read_iter)(struct kiocb *, struct iov_iter *);
    ssize_t (*write_iter)(struct kiocb *, struct iov_iter *);
    int (*open)(struct inode *, struct file *);
    int (*release)(struct inode *, struct file *);
    int (*mmap)(struct file *, struct vm_area_struct *);
    int (*fsync)(struct file *, loff_t, loff_t, int);
    long (*ioctl)(struct file *, unsigned int, unsigned long);
    // ...
};

```

对象关系：fd → file → dentry → inode → super_block。



图示：VFS 对象链：fd 引用 file，file 指向 dentry 和 inode，inode 属于 super_block。

分发机制。VFS 的核心是函数指针表分发。`vfs_read` 不直接读数据，而是调用 `file->f_op->read_iter`，后者对 ext4 是 `ext4_file_read_iter`，对 xfs 是 `xfs_file_read_iter`：

```

ssize_t vfs_read(struct file *file, char __user *buf, size_t len, loff_t *pos) {
    if (file->f_op->read_iter)
        return new_sync_read(file, buf, len, pos); // 调 read_iter
    else if (file->f_op->read)
        return file->f_op->read(file, buf, len, pos); // 旧接口
    return -EINVAL;
}
// new_sync_read 最终调用：
// file->f_op->read_iter(kiocb, iov_iter)
// ext4: ext4_file_read_iter -> generic_file_read_iter -> pagecache
// xfs:  xfs_file_read_iter  -> generic_file_read_iter
// nfs:  nfs_file_read_iter  -> 网络 RPC

```

核心理想

VFS 分发的本质是延迟绑定：`open()` 时 `file->f_op = inode->i_fop`（由具体文件系统在 `alloc_inode` 时设置），此后每次 `read/write/fsync` 都通过这张函数指针表分发到正确的实现。应用代码、VFS 通用代码、文件系统特定代码三者解耦。

`open()` 完整路径追踪。

1. `sys_open(path, flags, mode) → do_sys_open`。
2. `path_lookupat(path, LOOKUP_FOLLOW)`：解析路径得到 `dentry + inode`。遍历过程中若遇挂载点，`follow_mount()` 跳转到挂载的子树。
3. 权限检查：`inode_permission(inode, MAY_OPEN) → 调用 inode->i_op->permission (ext4: ext4_permission)`。

4. 分配 `struct file`, 设置 `f_pos=0`, `f_op = inode->i_fop`, `f_inode = inode`。
5. 调用 `file->f_op->open` (若存在) 做文件系统特定初始化, 再调 `security_file_open` (SELinux/AppArmor 钩子)。
6. `fd_install`: 把 `file` 装入当前进程的 `fd` 表, 返回 `fd` 号。

挂载命名空间。每个容器/命名空间有自己的挂载树。`/proc/mounts` 只显示当前命名空间的挂载。`mount --bind /src /dst` 为同一 `inode` 子树创建第二个 `dentry` 入口—两路径访问同一数据。`mount --move` 可在树间搬移子树。

特殊文件系统。

文件系统	后端存储	语义	典型用途
<code>tmpfs</code>	内存 (+swap)	易失, 读写	<code>/tmp</code> , <code>/dev/shm</code>
<code>procfs</code>	无	读即生成	<code>/proc/<pid>/status</code>
<code>sysfs</code>	无	属性 ↔ 驱动函数	<code>/sys/class/...</code>
<code>overlayfs</code>	lower+upper	写时复制	容器镜像层叠
<code>fuse</code>	用户态守护进程	请求经 <code>/dev/fuse</code> 往返	自定义文件系统

表 8.1: 常见特殊文件系统。

tmpfs。数据在页缓存中, 无磁盘 `inode`。可用 `swap` 作后端: 内存压力时换出。`shmem_alloc` 在 `shmem_fs_type` 下注册。`ls /dev/shm` 看到的“文件”实为 `shmem_inode_info`。

procfs。`/proc` 无后端存储。`read()` 触发 `proc_generate()` 动态生成内容。如 `/proc/meminfo` 的 `seq_file` 回调 `meminfo_show` 实时遍历 `global_node_page_state` 等计数器。

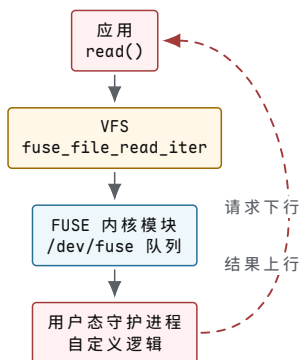
sysfs。`/sys` 是 `kobject` 层次的镜像。每个 `kobject` 对应一个目录, 每个 `attribute` 对应一个文件。`read(attribute)` 调用驱动注册的 `show` 回调, `write(attribute)` 调用 `store` 回调—直接映射到驱动函数, 无“文件”实体。

overlayfs。叠加 `lower` (只读) 和 `upper` (可写) 层。读时自顶向下找第一层命中; 首次写时写时复制 (`copy-up`): 把 `lower` 文件复制到 `upper` 再修改。Docker 镜像层叠正是基于 `overlayfs`。

FUSE。`fuse` 把文件操作路由到用户态守护进程:

1. 内核 VFS 调用 `fuse_file_read_iter`。
2. 内核把请求写入 `/dev/fuse` 字符设备, 守护进程阻塞在 `read(/dev/fuse)`。
3. 守护进程处理请求 (可能是网络、内存、任意逻辑), 把结果 `write` 回 `/dev/fuse`。
4. 内核唤醒等待的 `vfs_read`, 返回数据给应用。

代价是每次 I/O 两次内核 ↔ 用户态切换, 延迟 $\sim \mu\text{s}$ 级, 不适合高吞吐场景。优势是文件系统逻辑可在用户态实现, 无需内核权限。



图示：FUSE 请求往返：应用 → VFS → FUSE 内核 → 用户态守护进程，结果原路返回。

小结。本节覆盖了从块映射到 VFS 的完整栈：

- 块映射从间接指针演化到 extent 树与 B+ 树，把大文件的元数据从 100 KiB 降到 12 字节，查找从 4 I/O 降到 2 I/O。
- 空间分配用块组 + buddy + 延迟分配的组合，在连续性、利用率、延迟间逼近帕累托前沿。
- 目录结构从线性表升级到 HTree 和 B-tree，把 $O(n)$ 查找变为 $O(\log n)$ ，并辅以 dcache 实现热路径零 I/O。
- VFS 用函数指针表分发统一 API，使 ext4/xfs/nfs/tmpfs/fuse 共享同一套 open/read/write 接口。

这些机制层层叠叠：VFS 分发 → 文件系统特定操作 → extent 树查找 → 块分配 → 目录索引 → dcache 命中。理解每一层的权衡与数据结构，才能诊断从“大文件随机读慢”到“容器挂载层叠”的各类性能问题。

13.9 page cache 与写回

问题：磁盘比内存慢一千倍。上一节讨论了 VFS 的统一接口与分发机制，但还没有回答一个更根本的问题：每次 read 和 write 都访问磁盘，系统会慢到什么程度？

先看数字。DDR4 内存访问延迟 ~80 ns，带宽 ~25 GB/s。NVMe SSD 读取延迟 ~100 μs（慢 1250 倍），带宽 ~3 GB/s（慢 8 倍）。HDD 随机读取延迟 ~10 ms（慢 125000 倍），带宽 ~200 MB/s（慢 125 倍）。如果用户程序每次 read 都要等磁盘返回，一个 4 KiB 的读请求在 HDD 上就要花 10 ms；如果一个程序依次读 10000 个 4 KiB 页面，总耗时 100 秒。而如果把这 40 MiB 数据缓存在内存中，10000 次内存拷贝只需 ~0.8 ms。差距是五个数量级。

因此，现代操作系统几乎不会让用户 read 直接接触磁盘。内核在内存中维护一层文件数据缓存—page cache—作为用户进程与块设备之间的中介。读操作先查缓存，命中则零磁盘 I/O；写操作只写到缓存就返回，磁盘写入延迟到后台异步完成。

核心理想

page cache 的本质是把磁盘内容映射到内存页帧。每个缓存页由 (inode, offset) 唯一标识。读路径优先在缓存中查找；写路径只修改缓存中的页，标记为脏页后立即返回。磁盘 I/O 被推迟到 writeback 线程异步执行。

address_space：从 (inode, offset) 到 page。page cache 不是一张全局哈希表。内核为每个 inode 关联一个 address_space 对象，它是该 inode 所有缓存页的管理容器。

```

struct address_space {
    struct inode      *host;           // 拥有此 address_space 的 inode

```

```

struct xarray      i_pages;           // page cache 页的索引
pgoff_t           writeback_index;    // 上次 writeback 停止位置
const struct address_space_operations *a_ops; // 操作表
unsigned long      nrpages;           // 已缓存页数
unsigned long      wb_pages;          // 正在 writeback 的页数
struct rb_root_cached i_mmap;         // 私有映射的 VMA 树
struct rw_semaphore i_mmap_rwsem;     // mmap 信号量
spinlock_t         xa_lock;           // XArray 自旋锁
unsigned long      flags;             // AS_EIO, AS_ENOSPC 等
};

```

`host` 指回拥有此 `address_space` 的 `inode`。`i_pages` 是一棵 XArray，键是页索引（`pgoff_t`，即文件内偏移除以页大小），值是指向 `struct page` 的指针。`nrpages` 记录当前缓存了多少页；一个 1 GiB 文件的 `nrpages` 在全部缓存时为 $2^{30}/2^{12} = 262144$ 。

`a_ops` 是操作表，决定了具体的读、写、写回行为如何落到底层文件系统：

```

struct address_space_operations {
    int (*readpage)(struct file *, struct page *);
    int (*writepage)(struct page *, struct writeback_control *);
    int (*write_begin)(struct file *, struct address_space *,
                        loff_t, unsigned, unsigned,
                        struct page **, void **);
    int (*write_end)(struct file *, struct address_space *,
                     loff_t, unsigned, unsigned,
                     struct page *, void *);
    ssize_t (*direct_IO)(struct kiocb *, struct iov_iter *);
    int (*set_page_dirty)(struct page *);
    void (*invalidatepage)(struct page *, unsigned int, unsigned int);
    int (*releasepage)(struct page *, gfp_t);
    int (*migratepage)(struct address_space *,
                       struct page *, struct page *, enum migrate_mode);
    bool (*is_dirty_writeback)(struct page *, bool *, bool *);
    int (*error_remove_page)(struct address_space *, struct page *);
};

```

`ext4` 填充的是 `ext4_aops`，XFS 填充的是 `xfs_address_space_operations`。当 VFS 调用 `a_ops->readpage` 时，实际执行的函数取决于文件系统类型。这就是 VFS 分发给 `page cache` 层的体现。

XArray：页索引的 0(1) 查找。`address_space.i_pages` 是一棵 XArray（前身是 radix tree）。键是 `pgoff_t`（无符号长整型，表示页在文件中的序号），值是 `struct page` 指针。XArray 是一棵多叉树，每个内部节点有 64 个槽位（`XA_CHUNK_SHIFT = 6`），每个槽位指向下一级节点或叶子。

```

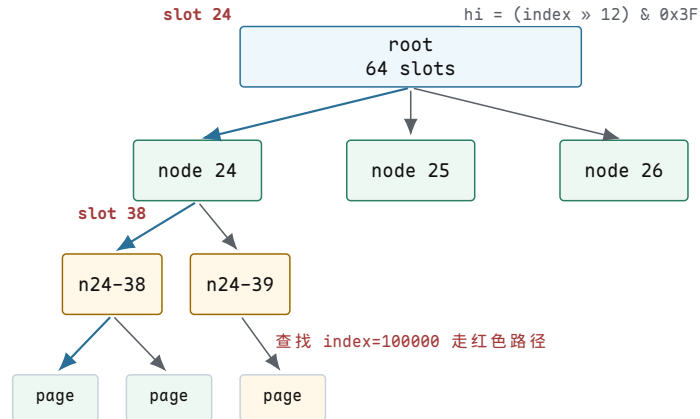
// 1 GiB 文件，页大小 4 KiB
// 总页数 = 1 GiB / 4 KiB = 262144 = 2^18
// XArray 层级计算：
//   每级 64 个槽 (2^6)
//   2^18 个键需要 ceil(18/6) = 3 级
//   level 0 (root):    64 个槽 → 覆盖 2^6 = 64 页
//   level 1:           64^2 个槽 → 覆盖 2^12 = 4096 页
//   level 2:           64^3 个槽 → 覆盖 2^18 = 262144 页
//
// 查找 page index = 100000:
//   index = 100000 = 0x186A0
//   slot[0] = (100000 >> 0) & 0x3F = 0x20 = 32 // 实际用高 12 位
//   实际分三段：高6位、中6位、低6位
//   hi = (100000 >> 12) & 0x3F = 24 // root → level 1 节点 24

```



```
// mi = (100000 >> 6) & 0x3F = 38 // level 1 → level 2 节点 38
// lo = (100000 >> 0) & 0x3F = 32 // level 2 → slot 32 = page*
```

三层 XArray 的查找只需要 3 次内存访问（每次一次指针解引用），这与文件大小无关——1 GiB 文件和 1 TiB 文件的查找都是 $O(1)$ （常数级内存访问次数，层数随 $\log_{64}(n)$ 增长极慢）。



图示：XArray 查找路径。1 GiB 文件需要 3 级 XArray（每级 64 槽）。查找 page index=100000 时，从 root 的 slot 24 进入 level 1 节点 24，再从其 slot 38 进入 level 2 节点，取出 page 指针。3 次内存访问完成 $O(1)$ 查找。

读路径深追踪。下面追踪一次 `vfs_read(file, buf, 4096, offset=8192)` 的完整执行路径。文件使用 ext4，页大小 4 KiB，offset 8192 对应 page index = 2。

```
// 步骤 1: VFS 入口
ssize_t vfs_read(struct file *file, char __user *buf,
                 size_t count, loff_t *pos)
{
    // offset = 8192, count = 4096
    // 检查权限、锁、文件模式
    if (ret = rw_verify_area(READ, file, pos, count))
        return ret;
    // 分发到 file->f_op->read_iter
    // ext4: ext4_file_read_iter -> generic_file_read_iter
    return generic_file_read_iter(iocb, to);
}

// 步骤 2: 查找 page cache
pgoff_t index = 8192 / 4096; // index = 2
struct page *page = pagecache_lookup(
    file->f_inode->i_mapping, // address_space
    index                     // page index 2
);
// XArray 查找: i_pages.xa_head → level 0 → slot 2

// 步骤 3a: 如果命中 (page 存在且 PageUptodate)
if (page && PageUptodate(page)) {
    // copy_page_to_user: 把 page 内偏移拷到用户 buffer
    // 8192 在 page 2 内的偏移 = 8192 % 4096 = 0
    copy_page_to_iter(page, 0, 4096, buf);
    // 没有设备 I/O; 耗时取决于复制长度、CPU cache、缺页和调度状态
    return 4096;
}

// 步骤 3b: 如果未命中
```

```
// 分配一个新 page，加入 address_space
page = page_cache_alloc(file->f_inode->i_mapping);
page->index = index;
// 将 page 插入 XArray（其他线程现在可以看到它）
__add_to_page_cache(page, mapping, index, gfp);

// 步骤 4：调用文件系统的 readpage
// ext4: ext4_readpage → ext4_readpage_inline 或 mpage_readpage
mapping->a_ops->readpage(file, page);
// ext4_readpage 内部：
// 1. 查 extent 树，找到 page index=2 对应的物理块
// 2. 构造 bio: bio_alloc(bdev, sector, 1 page)
// 3. submit_bio(READ, bio) → 交给块层 → 驱动 → 磁盘
// 4. page 被标记 PG_locked，等待 I/O 完成

// 步骤 5：等待 I/O 完成
wait_on_page_locked(page);
// I/O 完成后，中断处理程序调用 SetPageUptodate(page)
// 并解锁 page

// 步骤 6：拷贝数据到用户空间
copy_page_to_iter(page, 0, 4096, buf);
// page 留在 cache 中，下次读同一页直接命中
return 4096;
```

整个读路径的两种结局：

路径	执行步骤	磁盘 I/O	延迟
缓存命中	XArray 查找 → <code>copy_page_to_user</code>	无	~100 ns
缓存未命中	XArray 查找 → 分配 page → <code>readpage</code> → <code>submit_bio</code> → 等 待 → 拷贝	1 次 4 KiB 读	~100 μ s (NVMe)

第二次读同一页面时走缓存命中路径，延迟降到 ~100 ns。这就是 page cache 对重复读的加速比：~1000 倍。

预读：检测顺序访问并预取。当内核检测到访问模式是顺序的，会主动把后续页面提前读入 cache，避免每次都 cache miss。预读窗口按指数增长：

```
// read-ahead 窗口增长（简化）
// 第一次读 offset=0: cache miss, 读 1 页
// 预读窗口 = 4 KiB (1 page)
// 第二次读 offset=4096: cache miss, 读 1 页 + 预读 3 页
// 预读窗口 = 8 KiB (2 pages), 实际预读 4-16 KiB
// 第三次读 offset=8192: cache hit (被预读)
// 预读窗口增长到 16 KiB (4 pages)
// 内核继续预读下一批
// 第四次读 offset=12288: cache hit
// 预读窗口 = 32 KiB (8 pages)
// ...
// 最终预读窗口稳定在 128 KiB (32 pages)

// 内核使用 onstack_readahead_control 结构
struct readahead_control {
    struct file *rac_file;           // 关联的 file
    struct address_space *mapping;
    pgoff_t _index;                  // 预读起始页索引
    unsigned int _nr_pages;          // 本次预读页数
```

```

    unsigned int _batch_count;    // 已提交页数
};

// 预读决策在 page_cache_sync_readahead 中
void page_cache_sync_readahead(mapping, file, page, index)
{
    // ondemand_readahead 判断是否应该预读
    if (ondemand_readahead(mapping, ra, file, true, index)) {
        // 预读窗口增长逻辑
        ra->size = next_ra_size(ra); // 指数增长
        // ra_pages = min(ra->size, 128) // 上限 128 KiB
        __do_page_cache_readahead(mapping, file,
                                   ra->start, ra->size);
    }
}

```

次序	offset	用户读	预读动作	预读窗口
1	0	4 KiB	读 1 页，启动预读	4 KiB
2	4K	4 KiB	命中（预读命中），继续 预读 4 页	8 KiB
3	8K	4 KiB	命中，预读 8 页	16 KiB
4	12K	4 KiB	命中，预读 16 页	32 KiB
5	16K	4 KiB	命中，预读 32 页	64 KiB
6	20K	4 KiB	命中，预读 64 页	128 KiB
7+	...	4 KiB	命中，稳定预读 128 KiB	128 KiB

预读的关键收益：当窗口稳定在 128 KiB 时，后续每次 4 KiB 读都命中 cache，没有磁盘 I/O。磁盘在后台以一次大 I/O 读取 128 KiB（32 页），而非 32 次小 I/O。大 I/O 的吞吐远高于小 I/O——HDD 上一次 128 KiB 顺序读 ~0.6 ms，而 32 次 4 KiB 随机读 ~320 ms，快 500 倍。

预读是投机 (speculation)。如果用户程序只读了文件头部就退出，预读的 128 KiB 全部浪费。内核的预读算法因此需要检测随机访问模式并关闭预读：如果连续两次读的 offset 不连续，预读窗口立即缩减到 0。

写路径深追踪。写路径与读路径有一个根本区别：写操作不等待磁盘。数据拷入 page cache、标记为脏页后立即返回。

```

// vfs_write(file, buf, 4096, offset=8192)
ssize_t vfs_write(struct file *file, const char __user *buf,
                  size_t count, loff_t *pos)
{
    // offset = 8192, count = 4096
    // 分发到 generic_file_write_iter
    return generic_file_write_iter(iocb, from);
}

// generic_file_write_iter 内部：
// 1. 获取 page cache 中的 page
pgoff_t index = 8192 / 4096; // index = 2
struct page *page;

// write_begin 回调 (ext4: ext4_write_begin)
// 如果 page 不在 cache 中，分配并加入
page = grab_cache_page_write_begin(mapping, index);

```

```
// ext4_write_begin 可能还需要处理延迟分配预留

// 2. 从用户空间拷贝数据到 page
// page 内偏移 = 8192 % 4096 = 0
copied = copy_from_user(page_address(page) + 0, buf, 4096);
// 此时数据在内核 page cache 中

// 3. write_end 回调 (ext4: ext4_write_end)
// 更新 inode 的 i_size
// 标记 page 为脏
set_page_dirty(page);
// SetPageDirty(page) → 把 page 加入 address_space 的脏页树

// 4. 解锁 page，返回用户空间
unlock_page(page);
put_page(page);
// write() 系统调用返回 4096
// 数据在内存中，磁盘上还没有
```

`write()` 返回时，数据的安全级别取决于缓存层次：

时刻	数据位置	断电后
<code>write()</code> 返回	page cache (内存)	丢失
<code>writeback</code> 完成	磁盘写缓存	可能丢失 (易失缓存)
<code>fsync()</code> 返回	磁盘非易失介质	保留

这就是为什么 `write()` 返回成功不代表数据已持久化——它只代表数据到达了内存。

脏页追踪。page cache 需要区分”干净页”(内容与磁盘一致)和”脏页”(内存中修改过，磁盘还没更新)。`address_space` 用一棵独立的脏页树 (也是 XArray) 来追踪脏页。

```
// 每个 page 有一个 page->flags 字段
// 其中包含 PG_dirty 位：
//   PG_dirty = 1: 页被修改过，需要 writeback
//   PG_dirty = 0: 页与磁盘一致 (或刚从磁盘读入)

// SetPageDirty(page) 的简化逻辑：
int set_page_dirty(struct page *page)
{
    struct address_space *mapping = page->mapping;
    // 把 page 加入 mapping 的脏页 XArray
    // 脏页树与正常页树可以共享结构，
    // 但内核用 page->flags 区分
    if (!TestSetPageDirty(page)) {
        // 之前不是脏页，加入脏页链表
        __xa_set_mark(&mapping->i_pages, page->index,
                     PAGECACHE_TAG_DIRTY);
        // 增加全局脏页计数
        __inc_node_page_state(page, NR_FILE_DIRTY);
        // 增加 inode 的脏页计数
        __mark_inode_dirty(mapping->host, I_DIRTY_PAGES);
        return 1;
    }
    return 0;
}
```

脏页的追踪使用三个标记：`PAGECACHE_TAG_DIRTY`（需要写回）、`PAGECACHE_TAG_WRITEBACK`（正在写回中）、`PAGECACHE_TAG_TOWRITE`（等待写回）。`writeback` 代码通过扫描这些标记找到需要处理的页。

写回机制：阈值与触发。Linux 的脏页写回由 per-CPU 计数器和全局阈值驱动。核心计数器是 `nr_dirty`（全局脏页数）和 `nr_writeback`（正在写回的页数），通过 per-CPU 变量维护以减少锁竞争。

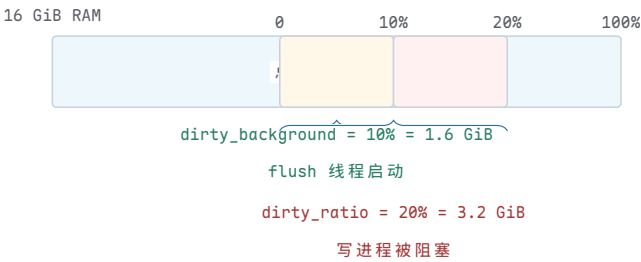
参数	默认值	可调范围	含义
<code>vm.dirty_ratio</code>	20%	0-100	脏页达到总内存的比例时，写进程被阻塞
<code>vm.dirty_backg round_ratio</code>	10%	0-100	脏页达到此比例时，后台 flush 线程启动
<code>vm.dirty_expire_ centisecs</code>	3000 (30s)	—	脏页存活超过此时长后优先写回
<code>vm.dirty_write back_centisecs</code>	500 (5s)	—	flush 线程周期性唤醒间隔

以 16 GiB RAM 为例：

```
// 16 GiB RAM 的脏页阈值计算
total_memory = 16 * 1024 * 1024 KiB = 16777216 KiB
// 假设每页 4 KiB
total_pages = 16777216 / 4 = 4194304

dirty_background_ratio = 10% // 默认
dirty_background_threshold = 4194304 * 0.10 = 419430 pages
                                = 419430 * 4 KiB = 1638 MiB
// 脏页超过 1638 MiB → kworker flush 线程启动

dirty_ratio = 20% // 默认
dirty_threshold = 4194304 * 0.20 = 838860 pages
                                = 838860 * 4 KiB = 3276 MiB
// 脏页超过 3276 MiB → 写进程被阻塞 (throttle)
// 写进程被迫同步执行 writeback，不能继续产生脏页
```



图示：脏页阈值示意。16 GiB RAM 时，脏页超过 1.6 GiB (10%) 触发后台写回；超过 3.2 GiB (20%) 阻塞写进程。黄色区域为后台写回触发范围，红色区域为写进程阻塞范围。

写回追踪。当脏页达到阈值或时间过期，`kworker` 线程被唤醒执行 `wb_workfn`：

```
// 写回核心逻辑（简化自 mm/page-writeback.c）
void wb_workfn(struct writeback_control *wbc)
{
    while (wbc->nr_to_write > 0) {
        // 1. 收集脏页：从 address_space 的脏页树中
        //    扫描最多 1024 页
        struct page *pages[1024];
```

```

int nr = find_dirty_pages(mapping, wbc,
                          pages, 1024);

if (nr == 0)
    break; // 没有脏页了

for (int i = 0; i < nr; i++) {
    struct page *page = pages[i];

    // 2. 如果是延迟分配，此时才分配物理块
    if (test_bit(PG_delalloc, &page->flags)) {
        // ext4: ext4_mballocc 在块组中找连续块
        ext4_lblk_t lblk = page->index;
        ext4_fsblk_t pblk;
        int allocated = ext4_mb_new_blocks(
            handle, &ar, &pblk);
        // 设置 extent: (lblk, pblk, 1)
        ext4_ext_map_blocks(handle, inode, &map,
                            EXT4_GET_BLOCKS_CREATE);
        clear_bit(PG_delalloc, &page->flags);
    }

    // 3. 构造 bio
    struct bio *bio = bio_alloc(GFP_NOIO, 1);
    bio->bi_bdev = inode->i_sb->s_bdev;
    bio->bi_iter.bi_sector =
        pblk * (blocksize >> 9);
    // bio 记录 page、页内偏移和长度；DMA 映射在更低层建立
    bio->bi_io_vec[0].bv_page = page;
    bio->bi_io_vec[0].bv_len = 4096;
    bio->bi_io_vec[0].bv_offset = 0;

    // 4. 标记为正在写回
    SetPageWriteback(page);
    ClearPageDirty(page);
    // 从脏页树移到 writeback 树

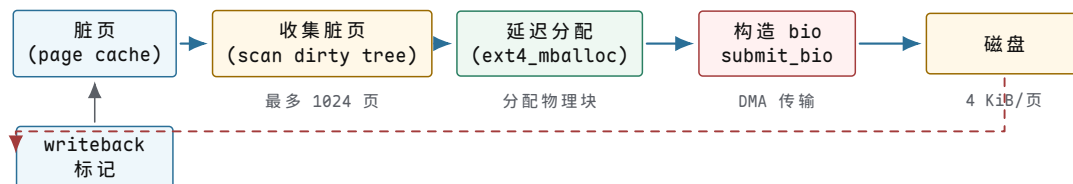
    // 5. 提交给块层
    submit_bio(WRITE, bio);
}

// 6. 等待这批 bio 完成
// (部分 writeback 模式异步等待)
wbc->nr_to_write -= nr;
}

// bio 完成回调（中断上下文）
void bio_endio(struct bio *bio)
{
    for (page in bio->bi_io_vec) {
        // 清除 writeback 标记
        end_page_writeback(page);
        // 从 writeback 树移除
        // page 现在是干净页，留在 cache 中
    }
    bio_put(bio); // 释放 bio
}

```

脏页到磁盘的完整数据流：



图示：写回数据流。脏页从 page cache 出发，经过脏页扫描、延迟分配、bio 构造、DMA 传输到达磁盘。I/O 完成后中断回调清除 writeback 标记，page 变为干净页留在 cache 中。

fsync：从内存到非易失介质。 `fsync(fd)` 是用户进程强制持久化的唯一可靠手段。它执行三个步骤，确保数据真正到达磁盘的非易失介质：

```
// ext4_fsync (简化)
int ext4_fsync(struct file *file, loff_t start, loff_t end, int datasync)
{
    struct inode *inode = file->f_inode;

    // 步骤 1: 把此 inode 的所有脏页刷到磁盘
    // filemap_write_and_wait 遍历 address_space 的脏页树
    // 对每个脏页: 构造 bio -> submit_bio -> 等待完成
    ret = filemap_write_and_wait_range(
        inode->i_mapping, start, end);
    // 此时数据已到磁盘 (或磁盘写缓存)

    // 步骤 2: 如果文件系统有日志, 强制提交日志事务
    // 确保元数据修改也持久化
    if (!datasync || ext4_should_journal_data(inode)) {
        ret = ext4_fc_commit(journal, start, end);
        // 或旧版: jbd2_complete_transaction(journal, tid)
        // 这会触发 jbd2 的 commit 流程:
        //   写 descriptor + metadata + commit block
        //   flush 日志区域
    }

    // 步骤 3: 发送 CACHE FLUSH 命令到磁盘
    // 强制磁盘把内部写缓存刷到非易失介质
    ret = blkdev_issue_flush(inode->i_sb->s_bdev);
    // blkdev_issue_flush 构造一个 barrier bio
    // 驱动把它翻译成 SCSI SYNCHRONIZE CACHE
    //   或 NVMe FLUSH CACHE 命令
    // 磁盘控制器把 DRAM 写缓存写入 NAND/HDD 介质
    // 完成后返回, 此时数据在非易失介质上

    return ret;
}
```

许多磁盘有易失性写缓存 (volatile write cache)。磁盘控制器收到写请求后，数据先进入磁盘内部的 DRAM 缓存，磁盘立即向主机返回“完成”。如果此时断电，缓存中的数据丢失。`blkdev_issue_flush` 发送的 FLUSH 命令强制磁盘把缓存内容写到非易失介质，并在完成后才返回。没有这一步，`fsync` 返回后断电仍可能丢数据。

某些磁盘声称写缓存是非易失的 (电池备份)，此时 FLUSH 可以是空操作。但文件系统无法信任磁盘的声明——许多消费级 SSD 声称非易失但在断电时实际丢失数据。

屏障与磁盘命令顺序。在有易失写缓存的磁盘上，写操作的完成顺序与提交顺序可能不同。内核用屏障 (barrier) 来保证顺序。考虑以下场景：先写数据块 5000，再写元数据块 6000，必须保证 5000 先到非易失介质。

```
// 磁盘命令序列 (有屏障)
WRITE block 5000    // 数据块
WRITE block 5001    // 数据块
FLUSH_CACHE        // ← 屏障：强制 5000/5001 到非易失介质
--barrier--        // 后续写必须在此之后
WRITE block 6000    // 元数据块 (依赖 5000 已落盘)
FLUSH_CACHE        // 确保 6000 也落盘

// 如果在 FLUSH_CACHE 之前断电：
//   block 5000/5001 可能没落盘
//   block 6000 一定没落盘 (还在屏障之后)
//   元数据不会指向未落盘的数据块 → 一致

// 如果没有屏障，磁盘可能重排：
WRITE block 6000    // 先落盘
WRITE block 5000    // 后落盘
// 断电：block 6000 在盘上，block 5000 不在
//   元数据指向不存在的数据 → 不一致
```

现代 SATA 和 NVMe 协议提供了原语的 FUA (Force Unit Access) 和 FLUSH 命令来支持屏障语义。内核在 `submit_bio` 时设置 `BIO_RW_BARRIER` 或使用 `blkdev_issue_flush` 来插入屏障。

Direct I/O：绕过 page cache。有些应用程序不需要内核 page cache——它们自己管理缓存。数据库系统 (Oracle、PostgreSQL) 在用户态维护 buffer pool，数据缓存在自己的内存中。如果再经过 page cache，就会产生双重缓存：同一份数据既在数据库 buffer pool 中，又在内核 page cache 中，浪费内存且增加一次拷贝。

`O_DIRECT` 标志让 `read/write` 绕过 page cache，直接在用户 buffer 和块设备之间通过 DMA 传输数据。

```
// O_DIRECT 读路径
vfs_read(file, buf, 4096, offset=8192, O_DIRECT):
    // 1. 不查 page cache, 不分配 page
    // 2. 检查对齐约束
    //   buf 必须页对齐 (buf % 4096 == 0)
    //   offset 必须块对齐 (offset % 512 == 0)
    //   len 必须块对齐
    if ((unsigned long)buf % blocksize != 0)
        return -EINVAL;
    if (offset % blocksize != 0)
        return -EINVAL;
    if (len % blocksize != 0)
        return -EINVAL;

    // 3. 直接构造 bio, 用户 buffer 作为 DMA 目标
    //   需要 pin 住用户 page 防止换出
    bio = bio_alloc(GFP_KERNEL, 1);
    bio->bi_bdev = inode->i_sb->s_bdev;
    bio->bi_iter.bi_sector = offset / 512;
    bio->bi_io_vec[0].bv_page = virt_to_page(buf);
    bio->bi_io_vec[0].bv_offset = offset_in_page(buf);
    bio->bi_io_vec[0].bv_len = 4096;

    // 4. 提交给块层, 等待完成
    submit_bio(READ, bio);
```

```
wait_for_completion(bio);

// 5. 不经过 page cache, 不产生缓存页
// 数据直接 DMA 到用户 buffer
return 4096;
```

条件	要求	原因
buffer 对齐	页对齐	DMA 需要物理连续页帧
offset 对齐	块对齐 (512B 或 4K)	块设备最小寻址单元
length 对齐	块对齐	不能 DMA 半个块

何时使用 `O_DIRECT`:

- 数据库 (Oracle、PostgreSQL、MySQL InnoDB): 自己管理 buffer pool, 避免双重缓存, 减少一次内存拷贝。
- 大文件顺序写 (视频录制、日志归档): 不需要 page cache 的读缓存, 直接写入即可。

何时不使用 `O_DIRECT`:

- 小文件随机读: 没有预读收益, 每次都直接访问磁盘。
- 写密集型小写: 没有写回批量化, 每次写直接落盘。
- 通用文件访问: page cache 的读缓存和写批量对小 I/O 至关重要。

DAX: 持久内存直接访问。 DAX (Direct Access) 面向可由 CPU load/store 访问的持久内存 (persistent memory, PMEM) 或支持 DAX 的设备映射。具体延迟依平台而变; 稳定特征是它能绕过传统 page cache 数据路径, 并保留非易失性介质的持久化要求。

DAX 模式下, 文件 `mmap` 返回用户虚拟地址, 页表把该地址映射到持久内存页。应用可用 CPU load/store 访问映射内容, 数据不经过传统 page cache, 也不靠设备 DMA 完成每次 load/store; 但地址仍然是受页表权限约束的虚拟地址。

```
// DAX 写路径
struct data_t { int data; };
int fd = open("/pmem/file.dat", O_RDWR);
struct data_t *p = mmap(NULL, 4096,
    PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
// p 是映射到持久内存页的用户虚拟地址

p->data = 42;    // CPU store 指令直接写 PMEM
                // 这是一条 mov 指令, 不经过 page cache

// 但 CPU 有自己的 L1/L2 cache
// store 先进入 CPU cache, 可能还没到 PMEM
// 要保证持久化, 需要显式刷 cache line

clwb(p);        // cache line writeback
                // 把包含 p 的 cache line 刷到 PMEM
sfence();       // store fence
                // 确保之前的所有 store (包括 clwb) 完成
                // 之后才能继续执行
// 此刻 p->data = 42 已在持久内存上, 断电不丢

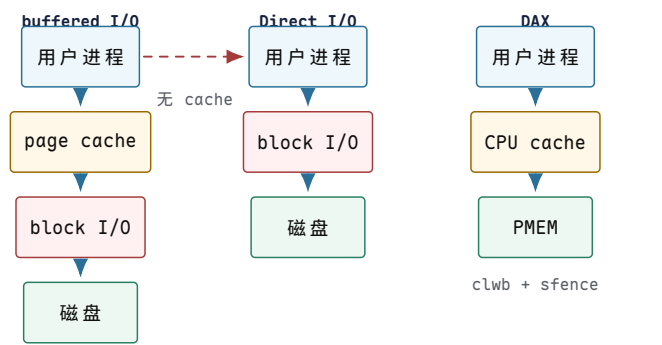
munmap(p, 4096);
close(fd);
```

DAX 下的 `fsync` 语义完全不同：

```
// DAX fsync
int dax_fsync(struct inode *inode)
{
    // 遍历此 inode 的所有 dirty PMEM 映射
    // 对每个 dirty cache line:
    //   clwb(addr) // flush to PMEM
    //   sfence()   // ensure ordering
    // 不需要 submit_bio, 不需要 blkdev_issue_flush
    // 因为 PMEM 本身就是非易失介质

    struct address_space *mapping = inode->i_mapping;
    // 扫描 DAX dirty 标记的页面
    pgoff_t index = 0;
    while ((page = find_get_page_tag(mapping, &index,
        PAGECACHE_TAG_DIRTY))) {
        // 对 PMEM 中的地址执行 clwb
        void *addr = dax_get_addr(page);
        arch_wb_cache_pmem(addr, PAGE_SIZE);
    }
    arch_wmb_cache(); // sfence
    return 0;
}
```

	buffered I/O	Direct I/O	DAX
数据路径	page cache → block I/O	block I/O	CPU store → PMEM
<code>fsync</code>	刷脏页 + 屏障	屏障	<code>clwb</code> + <code>sfence</code>
缓存层	内核 page cache	应用自管	CPU cache line
延迟	~100 ns (命中)	~100 μs	~300 ns
对齐要求	无	页/块对齐	无
适用	通用	数据库	持久内存设备



图示：三种 I/O 路径对比。buffered I/O 经过 page cache 和 block I/O 两层；Direct I/O 绕过 page cache 但仍走 block I/O；DAX 完全绕过 page cache 和 block I/O，CPU store 直接写持久内存。

13.10 崩溃一致性与日志

崩溃问题：多步写不是原子的。文件系统的一次逻辑操作（如创建文件）在磁盘上需要修改多个独立的块。磁盘每次只写一个块（或者一个扇区），而电源可能在任意时刻中断。这导致磁盘上的状态可能停留在“半完成”的状态——部分块已更新，部分块还是旧的。

以创建文件 `/foo` 为例，需要四步磁盘写入：

```
// 创建 /foo 需要的磁盘修改
create_file(root_dir, "foo"):
    // 步骤 1: 分配 inode 5
    //   修改 inode bitmap: 标记 inode 5 为已用
    //   写磁盘块 2 (inode bitmap)
    mark_inode_used(inode_bitmap, 5)
    write_block(2, inode_bitmap)

    // 步骤 2: 初始化 inode 5
    //   修改 inode table: 写 inode 5 的内容
    //   写磁盘块 4 (inode table 中 inode 5 所在块)
    init_inode(5, mode=REG_FILE, size=0, ...)
    write_block(4, inode_table_block)

    // 步骤 3: 添加目录项
    //   修改根目录数据块: 添加 "foo" -> 5
    //   写磁盘块 68 (根目录数据块)
    dir_add_entry(root_dir, "foo", inode=5)
    write_block(68, dir_data_block)

    // 步骤 4: 分配数据块 (如果文件有内容)
    //   修改 block bitmap: 标记块 68 为已用
    //   写磁盘块 3 (block bitmap)
    mark_block_used(block_bitmap, 68)
    write_block(3, block_bitmap)
```

崩溃在不同时刻发生，造成不同的不一致状态：

崩溃时刻	已完成的写	不一致后果
步骤 1 之后	块 2 (inode bitmap)	inode 5 标记为已用，但 inode table 中内容未初始化。inode 5 有垃圾数据。
步骤 2 之后	块 2 + 块 4	inode 5 已分配且初始化，但无目录项指向它。inode 泄漏。bitmap 说已用，但无路径可达。
步骤 3 之后	块 2 + 4 + 68	目录指向 inode 5，但 block bitmap 未标记块 68 为已用。另一个文件可能分配到同一块。
步骤 4 之后	全部完成	一致。

步骤 3 之后崩溃尤其危险：目录项说 `"foo" → inode 5`，inode 5 的 `block[]` 指向块 68，但 block bitmap 仍标记块 68 为空闲。下次分配时，另一个文件可能拿到块 68，两个文件共享同一数据块—数据损坏。

核心思想

崩溃一致性的核心困难是：一次逻辑修改需要多个磁盘写，而磁盘写之间没有原子性。任何时刻断电都可能留下部分更新的状态。文件系统必须能恢复到一致状态，无论崩溃发生在哪一步。

fsck：旧方案的代价。ext2 没有日志，崩溃恢复依赖 `e2fsck` 在挂载前扫描整个文件系统。`fsck` 分五步检查：

```
e2fsck(dev):
    // Pass 1: 检查每个 inode
    //   遍历 inode table 中所有 inode
    //   验证 mode 合法、size 非负
```

```

// 验证 block 指针在合法范围 [0, total_blocks]
// 统计每个 inode 使用的块
for i in 0..inode_count:
    inode = read_inode(i)
    if inode.mode == 0: continue // 未使用
    for block in inode.block_pointers:
        if block < 0 or block >= total_blocks:
            mark_bad_inode(i)
            block_usage[block].add(i) // 谁用了这个块

// Pass 2: 检查目录结构
// 从根目录开始 BFS
// 每个目录项指向的 inode 必须存在且类型正确
// "." 必须指向自身, ".." 必须指向父目录
bfs_from_root():
    queue = [root_inode]
    while queue:
        dir = dequeue(queue)
        for entry in dir.entries:
            if not inode_exists(entry.inode):
                error("dangling dir entry")
            if entry.name == "." and entry.inode != dir:
                error("bad . entry")
            enqueue(entry.inode)

// Pass 3: 检查连通性
// 从根目录可达的 inode 集合 vs 全部已用 inode
// 不可达的 inode 是“孤儿” (orphan)
// 将孤儿 inode 链到 lost+found
reachable = bfs_from_root()
for i in 0..inode_count:
    if inode_used(i) and i not in reachable:
        link_to_lost_found(i)

// Pass 4: link count 验证
// inode.links 字段 vs 实际目录引用数
// 不匹配则修正 links 字段
for i in 0..inode_count:
    actual_links = count_dir_references(i)
    if inode[i].links != actual_links:
        fix_links(i, actual_links)

// Pass 5: bitmap 一致性
// inode bitmap: 标记已用的 inode 是否确实在使用
// block bitmap: 标记已用的块是否被某 inode 引用
// 不匹配则修正 bitmap
for block in 0..total_blocks:
    if bitmap[block] == USED and block not in block_usage:
        bitmap[block] = FREE
    if bitmap[block] == FREE and block in block_usage:
        bitmap[block] = USED // 可能是泄漏

```

`fsck` 的复杂度是 $O(\text{total_blocks})$ —必须扫描每个块和每个 inode。一个 1 TiB 的文件系统，4 KiB 块，共 $2^{38}/2^{12} = 2^{26} \approx 6700$ 万个块。即使每秒检查 100000 块，也需要 ~ 11 分钟。更大的文件系统 (10 TiB+) 可能需要数小时。

`fsck` 在大型文件系统中恢复时间长达数十分钟到数小时。服务器重启窗口要求几秒到几分钟，`fsck` 的恢复时间是不可接受的。这正是日志文件系统被发明的直接动机：把恢复时间从 $O(\text{文件系统大小})$ 降到 $O(\text{日志大小})$ 。

日志：write-ahead logging。 日志的核心思想是写前记录 (write-ahead logging, WAL)：在修改文件系统的实际数据结构之前，先把修改的内容记录到一块专门的日志区域。日志中的记录描述“我要把块 X 的内容改成 Y”。修改提交后，再把实际数据写到文件系统的对应位置。

```
// 带日志的文件创建
journal_create_file(root_dir, "foo"):
    // 1. 开启一个日志事务 (handle)
    handle = journal_start()

    // 2. 在日志中记录所有要做的修改
    //     这些记录描述了修改后的“完整块内容”
    journal_log(handle, "inode 5: set mode=FILE, size=0")
    journal_log(handle, "dir block 68: add entry foo->5")
    journal_log(handle, "inode bitmap: set bit 5")
    journal_log(handle, "block bitmap: set bit 68")

    // 3. 提交日志事务
    //     commit record 写到日志区域的磁盘上
    //     这是“事务完成”的不可撤销标记
    journal_commit(handle)
    // ← 此刻之前断电：事务未提交，恢复时丢弃
    // ← 此刻之后断电：事务已提交，恢复时重放

    // 4. 把修改写到主文件系统位置
    //     (checkpoint, 可延迟执行)
    write_inode(5, ...)
    write_dir_block(68, ...)
    write_inode_bitmap(...)
    write_block_bitmap(...)
```

崩溃恢复规则因此变得简单：

崩溃时刻	日志状态	恢复动作
<code>journal_commit</code> 之前	无 commit 块	丢弃此事务，所有修改未应用
<code>journal_commit</code> 之后	有 commit 块	重放日志中的修改到主文件系统
<code>checkpoint</code> 之后	日志已回收	无需恢复（修改已在主位置）

核心思想

日志的关键规则是日志先行 (write-ahead)：描述修改的日志记录必须在主文件系统上的实际修改之前持久化到磁盘。否则崩溃后无法判断磁盘上的半成品是否应该保留。commit 块是事务的原子边界—有 commit 块就重放，没有就丢弃。

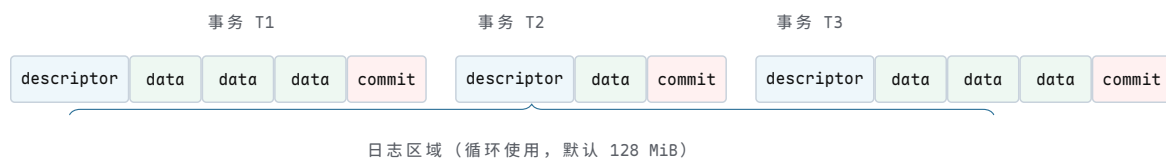
jbd2 日志格式。 ext4 的日志由 jbd2 (Journaling Block Device v2) 实现。日志是一块循环使用的磁盘区域，内部由一系列块组成。每个事务在日志中的布局如下：

```
// jbd2 日志区域布局（磁盘上）
```

```
// 每个块 4 KiB
//
// 事务 T1:
// [descriptor][data][data][data][commit]
// 块 100      101    102    103    104
//
// 事务 T2:
// [descriptor][data][commit]
// 块 105      106      107
//
// 事务 T3:
// [descriptor][data][data][data][commit]
// 块 108      109     110     111     112

// descriptor block 格式:
struct journal_header2_t {
    uint32_t h_magic;      // 0x4a533334 = "J3S4"
    uint32_t h_blocktype;  // 2 = descriptor
    uint32_t h_sequence;   // 事务序号
    uint32_t h_blocknr;    // 本块在日志中的物理块号
};
// 后跟一组 block_tag:
struct journal_block_tag_s {
    uint32_t t_blocknr;    // 被修改块在主 FS 中的块号
    uint16_t t_checksum;   // 数据块校验和 (CRC32C)
    uint16_t t_flags;      // JBD2_FLAG_ESCAPE 等
    uint32_t t_blocknr_high; // 高 32 位块号 (大 FS)
};

// commit block 格式:
struct journal_commit_header {
    journal_header2_t h;    // h_blocktype = 6 (commit)
    uint8_t chksum_type;    // 校验类型
    uint8_t flags[3];
    uint32_t chksum[16];    // CRC32C 校验数组
    // 每个 checksum 对应事务中的一个数据块
};
```



图示: jbd2 日志布局。每个事务以 descriptor 块开始 (列出后续数据块对应的主 FS 块号), 中间是数据块 (修改后的完整块内容), 最后以 commit 块结束 (含 CRC32C 校验和)。恢复时, 有 commit 块且校验通过的事务被重放。

崩溃恢复追踪。系统启动挂载文件系统时, jbd2 执行日志恢复:

```
journal_recover(journal):
    // 从日志区域的头部开始扫描
    log_start = journal->j_head;    // 下一个要写的位置
    log_end   = journal->j_tail;    // 最老的未 checkpoint 位置

    // 扫描每个事务
    seq = journal->j_tail_sequence;
    block = log_end;

    while block < log_start:
        // 读 descriptor 块
```

```

desc = read_block(block);
if desc.h_magic != MAGIC || desc.h_blocktype != DESCRIPTOR:
    break; // 不是有效的日志内容

// 读后续的数据块，直到遇到 commit 块
data_blocks = [];
tag = desc.tags;
while tag != NULL:
    data = read_block(block + offset);
    // 校验：data 的 CRC32C == tag.t_checksum?
    if crc32c(data) != tag.t_checksum:
        // 数据损坏，事务不可信
        break;
    data_blocks.append({target: tag.t_blocknr,
                        content: data});
    tag = tag->next;

// 读 commit 块
commit = read_block(block + data_count + 1);
if commit.h_blocktype == COMMIT:
    // 有 commit 块 → 事务已提交
    // 校验 commit 块的 checksum
    if verify_commit_checksum(commit, data_blocks):
        // 重放：把每个数据块写到主 FS 位置
        for entry in data_blocks:
            write_block(entry.target, entry.content);
        // 事务重放完成
    else:
        // commit 校验失败，跳过
        break;
else:
    // 没有 commit 块 → 事务未完成，丢弃
    break;

// 恢复完成
// 恢复时间 = 0(日志大小)，通常几秒
// 128 MiB 日志 / 4 KiB 块 = 32768 块
// 每块读 + 可能重写 = 最坏 32768 * 2 次 I/O
// NVMe 上约 32768 * 0.1ms = 3.3 秒

```

恢复时间与日志大小成正比，与文件系统总大小无关。128 MiB 日志的恢复在 NVMe 上约 3 秒，远优于 `fsck` 的 10–30 分钟。

ext4 + jbd2 事务模型。jbd2 有两级抽象：

对象	粒度	生命周期
<code>handle</code>	一次逻辑操作（创建一个文件、一次 <code>rename</code> ）	线程局部， <code>journal_start</code> 到 <code>journal_stop</code>
<code>transaction</code>	一个 <code>commit</code> 周期内的所有 <code>handle</code>	一个 jbd2 线程唤醒周期，批量提交

一个 `transaction` 可以收集来自不同线程的多个 `handle`。jbd2 线程定期唤醒（默认 5 秒或脏数据达到阈值），把当前 `transaction` 的所有修改一次性提交到日志。

```

// jbd2 commit 流程（简化自 fs/jbd2/commit.c）
void jbd2_commit_transaction(journal_t *journal)
{
    transaction_t *commit_transaction =

```

```

        journal->j_running_transaction;

// 1. 等待所有 handle 关闭
// 新的 handle 会进入下一个 transaction
wait_for_all_handles_closed(commit_transaction);
journal->j_running_transaction = new_transaction();

// 2. 切换到 commit 状态
commit_transaction->t_state = T_LOCKED;

// 3. 写 descriptor 块
// descriptor 列出本事务中所有被修改的 buffer
// 及其在校 FS 中的目标块号
write_descriptor_block(journal, commit_transaction);

// 4. 写数据/元数据块到日志
// 把每个被修改的 buffer 的完整内容写入日志
for (jh in commit_transaction->t_buffers):
    // 如果是 data=journal 模式, 数据块也写入
    // 如果是 data=ordered, 此时不写数据块
    journal_write_buffer(journal, jh);

// 5. 写 commit 块
// 包含本事务所有块的 CRC32C 校验和
write_commit_block(journal, commit_transaction);

// 6. flush 日志区域
// 确保日志内容在非易失介质上
blkdev_issue_flush(journal->j_dev);

// 7. 标记旧 transaction 可以被 checkpoint
commit_transaction->t_state = T_COMMIT;

// 8. 后续 checkpoint (可以延迟):
// 把已提交的修改从日志复制到主 FS 位置
// 完成后释放日志空间
}

```

三种日志模式。ext4 支持三种日志模式，在一致性和性能之间做不同取舍：

模式	日志内容	数据顺序	崩溃后保证
<code>data=journal</code>	元数据 + 数据	数据先入日志	最强：数据不丢。写放大 2 倍。
<code>data=ordered</code>	只元数据	数据先写盘，再提交元数据	中等：要么旧数据要么新数据。
<code>data=writeback</code>	只元数据	无顺序保证	最弱：可能看到旧垃圾数据。

下面追踪三种模式下 `write(fd, buf, 4096); fsync(fd);` 的磁盘 I/O 序列：

```

// ===== data=journal 模式 =====
// write() 触发：数据进入 page cache (无 I/O)
// fsync() 触发：
// 1. jbd2 提交事务，包含数据和元数据
// 日志：[data: buf 内容][metadata: inode 更新][commit]
// flush 日志区域
// 2. checkpoint：把数据写入主 FS 位置
// 主 FS：[data: buf 内容][metadata: inode 更新]

```

```
// 3. flush 主 FS
//
// 磁盘写：数据写 2 次（日志 + 主 FS），元数据写 2 次
// 写放大：2x
// 崩溃后：日志重放恢复数据，数据完整

// ===== data=ordered 模式（ext4 默认） =====
// write() 触发：数据进入 page cache（无 I/O）
// fsync() 触发：
// 1. 先把数据脏页刷到主 FS
// 主 FS: [data: buf 内容] ← 先落盘
// flush 数据 (barrier)
// 2. jbd2 提交元数据事务
// 日志: [metadata: inode 更新][commit]
// flush 日志
// 3. checkpoint: 元数据写入主 FS
// 主 FS: [metadata: inode 更新]
//
// 磁盘写：数据写 1 次（主 FS），元数据写 2 次（日志 + 主 FS）
// 写放大：~1.5x（元数据部分）
// 崩溃后：
// 如果 commit 之前崩溃：数据在盘上，元数据不在
// → inode 不指向新数据 → 看到旧数据（或 hole）
// 如果 commit 之后崩溃：数据和元数据都在
// → 看到新数据，完整一致

// ===== data=writeback 模式 =====
// write() 触发：数据进入 page cache（无 I/O）
// fsync() 触发：
// 1. jbd2 提交元数据事务
// 日志: [metadata: inode 更新][commit]
// flush 日志
// 2. checkpoint: 元数据写入主 FS
// 主 FS: [metadata: inode 更新]
// 3. 数据可能在 fsync 时也刷了，也可能没有
// （取决于实现，writeback 模式不强制数据顺序）
//
// 磁盘写：元数据写 2 次（日志 + 主 FS），数据写 0 或 1 次
// 写放大：~1x（最快）
// 崩溃后：
// 元数据可能指向一个块，但数据没写
// → 读到的是磁盘上的旧垃圾数据
// → 可能暴露上一个文件的残留内容！
```

data=writeback 模式在崩溃后可能暴露旧数据：元数据（inode 的块指针）已更新指向新分配的块，但数据内容还没写。这个块上可能存着另一个已删除文件的内容。这是安全漏洞——一个用户可能看到另一个用户的旧数据。**data=ordered**（默认模式）通过强制数据先于元数据落盘来避免此问题。

checkpoint 与日志空间回收。日志区域大小有限（ext4 默认 128 MiB）。提交后的事务修改记录留在日志中，直到被 checkpoint 到主文件系统位置后才能回收。日志因此是一块循环使用的环形缓冲区。

```
// 日志环形缓冲区
// j_head: 下一个写入位置（指向日志的最新事务末尾之后）
// j_tail: 最老的未 checkpoint 事务的起始位置
```

```
// j_free: j_head 到 j_tail 之间的可用空间
//
// ... [T_old][T_newer][T_current] ... [free space] ... [T_oldest]
//                               ↑head           ↑tail
// (日志是循环的, head 可能绕回 tail 之前)

// checkpoint 流程
jbd2_checkpoint(journal):
    // 找到最早未 checkpoint 的 transaction
    t = journal->j_checkpoint_transactions;
    while t:
        // 把 t 中记录的修改写到主 FS 位置
        for jh in t->t_buffers:
            // jh->b_bh 是被修改的 buffer
            // buffer 内容已在日志中, 现在写到主位置
            write_block(jh->b_blocknr, jh->b_data);
        // 此事务的日志空间可以回收
        free_log_space(journal, t);
        t = t->t_next;

    // 更新 j_tail
    journal->j_tail = next_oldest_start;
    // 写日志 superblock (更新 tail 指针)
    update_journal_superblock(journal);
```



图示：日志环形缓冲区。head 指向下一个写入位置，tail 指向最老的未 checkpoint 事务。已 checkpoint 的事务（T0、T1）空间可回收。如果 checkpoint 跟不上新事务产生速度，free 空间耗尽，新事务必须等待—导致写入停顿。

13.11 Copy-on-Write 文件系统

COW 原理：不覆盖，只追加。传统文件系统（ext4、XFS）在原地覆盖写：文件块更新时，直接写回原来的物理块。如果写入过程中断（断电），块中一半是旧数据一半是新数据—数据损坏。ext4 用日志来修复这种情况：日志保证要么全部回滚要么全部重放。

COW（Copy-on-Write）文件系统采用另一种策略：永远不在原地覆盖。修改一个块时，分配一个新块写入数据，然后更新所有指向旧块的指针指向新块，最后原子切换根指针。

```
// 传统原地覆盖写
update_block(inode, offset, new_data):
    block = lookup(inode, offset) // 找到物理块 5000
    write_block(5000, new_data)   // 直接覆盖块 5000
    // 如果写到一半断电：
    // 块 5000 一半旧一半新 → 数据损坏
    // 需要 journal 回滚

// COW 方式
cow_write(inode, offset, new_data):
    // 1. 分配新物理块
    new_data_block = alloc_block() // 分配块 8000
    write_block(8000, new_data)    // 写新块，旧块不动
```



```
// 2. 更新 B-tree 叶子指向新块
// 叶子也要 COW (不能原地改)
new_leaf = copy_and_modify(old_leaf, offset -> 8000)
new_leaf_block = alloc_block() // 分配块 8100
write_block(8100, new_leaf)

// 3. 父节点也要 COW (因为叶子指针变了)
new_parent = copy_and_modify(old_parent,
                             leaf_ptr -> 8100)
new_parent_block = alloc_block() // 分配块 8200
write_block(8200, new_parent)

// ... 递归 COW 到根 ...

// 4. 原子更新超级块的根指针
atomic_write(superblock.root, new_root_block) // 块 8400
// 这是唯一的“切换”操作
// 切换前：根指针指向旧树 → 旧数据可见
// 切换后：根指针指向新树 → 新数据可见
// 不存在“中间状态”

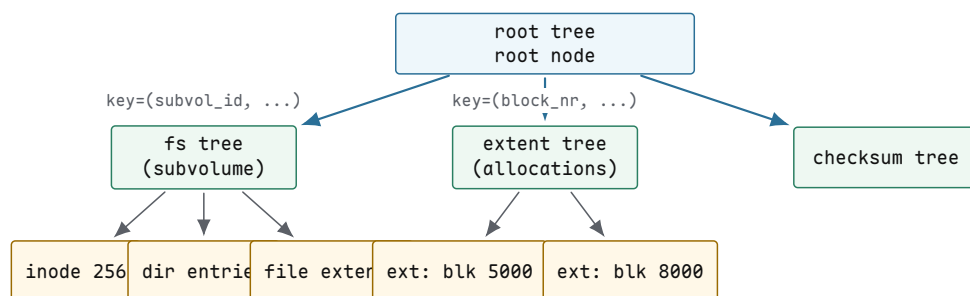
// 旧块 (5000, 旧叶子, 旧父节点...) 仍被旧根引用
// 如果有 snapshot, 旧根仍然有效
// 如果没有 snapshot, 旧块可以回收
```

	原地覆盖 (ext4/XFS)	COW (Btrfs/ZFS)
崩溃一致性	需要日志修复	天然一致 (根指针原子切换)
snapshot	需要额外 LVM 层	原生支持 (保留旧根指针)
写放大	低 (1 次写服务 1 次用户写)	高 (3-6 倍)
碎片	低 (原地覆盖)	高 (每次写新位置)
空间回收	不需要	需要 refcount 追踪

Btrfs：一切都是 B-tree。 Btrfs (B-tree file system) 的核心设计是把所有数据结构统一为 B-tree。超级块、extent 分配、目录、inode、校验和都是不同类型的 B-tree。

每个 B-tree 节点是一个磁盘块 (4 KiB-64 KiB)。键是三元组 (`objectid`, `type`, `offset`)，这种统一键空间让不同类型的树可以共享同一套查找、插入、删除代码。

B-tree	key	value
root tree	tree id	指向子树根节点
extent tree	逻辑块号	extent ref (物理块号 + 引用计数 + owner)
chunk tree	逻辑 chunk 号	物理 chunk 映射 (设备号 + 偏移)
dev tree	物理 extent	设备号 + 偏移
fs tree	(inode#, type, (per-subvolume) offset)	inode / dir entry / file extent
checksum tree	(inode#, offset)	数据块校验和



图示：Btrfs 的多棵 B-tree 结构。root tree 是顶层，指向 fs tree（每个 subvolume 一棵）、extent tree（块分配跟踪）、checksum tree 等。所有树共享统一的（objectid, type, offset）键空间。

Btrfs snapshot: 0(1) 创建。Btrfs 的 snapshot 创建只需复制 root tree 的根节点指针，不复制任何数据块：

```
// Btrfs snapshot 创建
btrfs_snapshot(src_subvol, dst_subvol):
    // src_subvol 的 fs tree root 在块 5000
    // 创建 dst_subvol:
    //   1. 在 root tree 中添加一个新条目
    //   指向 src 的 fs tree root (块 5000)
    //   2. 增加块 5000 的引用计数 (从 1 到 2)
    //   完成！没有复制任何数据块
    dst_subvol->fs_tree_root = src_subvol->fs_tree_root;
    // 即块 5000
    increase_refcount(5000); // extent tree 中 refcount 1->2
    // 耗时：写一个 root tree 节点 + 更新 extent ref
    //         = 0(1) 磁盘 I/O

// 之后在 dst 中修改文件：
btrfs_write(dst_subvol, inode, offset, data):
    // COW 路径：
    //   1. alloc new block 8000, write data
    //   2. COW leaf: 新叶子指向 8000 (新块 8100)
    //   3. COW parent: ... (新块 8200)
    //   4. ...
    //   5. COW 到 fs tree root: 新 root 在块 8400
    //   6. 更新 dst_subvol->fs_tree_root = 8400
    //   dst 现在有独立的树，指向自己的新块

    // src 的 fs tree root 仍然指向块 5000
    // 块 5000 的 refcount 降回 1 (被 src 引用)
    // 旧数据块 (5000 的子树) 仍被 src 引用，不回收
```

snapshot 创建后，两个 subvolume 共享所有未修改的数据块。只有被修改的块及其元数据路径才会被复制。这使得 Btrfs 的 snapshot 在空间和时间上都是极其高效的。

核心理想

Btrfs snapshot 是 COW 的直接收益：旧树根仍然有效，新树根通过 COW 路径逐渐分叉。共享的数据块通过 extent tree 中的引用计数管理——refcount > 1 的块被多棵树引用，refcount 降到 0 时才回收。snapshot 创建本身只是增加一次引用，0(1) 完成。

ZFS：事务对象存储。ZFS (Zettabyte File System) 的设计哲学与 Btrfs 类似 (COW + snapshot)，但组织方式和数据完整性保证不同。

概念	含义	作用
pool	存储池，由多个 vdev 组成	聚合空间、提供冗余
vdev	虚拟设备 (disk、mirror、RAID-Z)	冗余层
dataset	文件系统或卷	独立的 snapshot/quota/属性
objset	dataset 内的对象集合	包含 dnode 和数据
dnode	ZFS 的 inode 等价物	存储元数据和 block pointer
block pointer	带校验和的块指针	端到端数据完整性
TXG	transaction group	批量原子提交
ZIL	ZFS Intent Log	同步写加速
ARC	Adaptive Replacement Cache	内存读缓存
L2ARC	SSD 二级缓存	读缓存扩展

ZFS 的每个 block pointer 都携带校验和 (256 位 SHA-256 或 fletcher4)。读数据时，ZFS 先读块、再校验 checksum。如果不匹配，说明数据静默损坏 (silent data corruption) —— 磁盘返回了数据但内容不对。此时 ZFS 从冗余副本 (mirror 或 RAID-Z) 读取并修复。

```
// ZFS block pointer 结构 (简化)
struct blkptr_t {
    uint64_t blk_dva[3];    // 3 个 Data Virtual Address
                          // (可存 3 个副本的位置)
    uint64_t blk_prop;      // 压缩级别、加密、类型
    uint64_t blk_birth;     // 创建此块的 TXG 号
    uint64_t blk_fill;      // 填充信息
    uint64_t blk_cksum[8];  // 256-bit checksum
                          // (ZIO_CHECKSUM_SHA256)
};

// 读取一个 block pointer 指向的数据:
zio_read(bp):
    data = read_block(bp->blk_dva[0]); // 从第一副本读
    if checksum(data) == bp->blk_cksum:
        return data; // 校验通过, 数据完好

    // 校验失败 → 数据损坏!
    // 从第二副本读 (如果有 mirror/RAID-Z)
    data = read_block(bp->blk_dva[1]);
    if checksum(data) == bp->blk_cksum:
        // 第二副本完好
        // 修复第一副本: 重写 blk_dva[0]
        zio_rewrite(bp->blk_dva[0], data);
        return data;

    // 都坏了 → 不可恢复, 返回 EIO
```

ZFS 的 TXG (Transaction Group) 机制批量提交写操作。每隔几秒 (或 ZIL 满时), ZFS 把所有未提交的修改组装成一个 TXG, COW 写入新块, 更新所有 block pointer, 最后原子切换 uberblock (ZFS 的超级块等价物)。

```
// ZFS TXG 提交流程
txg_commit(pool):
    // 1. 冻结当前 TXG, 收集所有 dirty 数据
    txg = pool->txg_current;
    txg->state = TXG_STATE_QUIESCING;
```

```

// 新写入进入下一个 TXG

// 2. COW 写所有新数据块
for dnode in txg->dirty_dnodes:
    for block in dnode->dirty_blocks:
        new_block = alloc_block(); // 从空间分配
        write_block(new_block, block.new_data);
        // 更新 block pointer: 指向新块, 带新 checksum
        block.bp = make_blkptr(new_block,
                                checksum(block.new_data),
                                txg->txg_num);
        // 旧块不覆盖, 旧 block pointer 还在旧 uberblock

// 3. 更新所有 dnode (COW)
for dnode in txg->dirty_dnodes:
    new_dnode_block = alloc_block();
    write_block(new_dnode_block, dnode);
    // dnode 指向新的 block pointers

// 4. 更新 objset (COW)
new_objset_block = alloc_block();
write_block(new_objset_block, objset);
// objset 指向新的 dnode blocks

// 5. 原子写 uberblock
// uberblock 包含指向 root objset 的指针
new_uberblock = make_uberblock(
    new_objset_block, txg->txg_num, timestamp);
atomic_write_uberblock(pool, new_uberblock);
// 旧 uberblock 仍然存在 (ZFS 存多个 uberblock)
// 崩溃恢复: 读最新的有效 uberblock

```

ZIL (ZFS Intent Log) 是 ZFS 对 `fsync` 的加速机制。没有 ZIL 时, 每次 `fsync` 需要等待整个 TXG 提交 (COW 全路径), 延迟可能数秒。ZIL 是一块独立的日志区域, `fsync` 时先把同步写请求记录到 ZIL, 立即返回。后续 TXG 提交时把数据正式 COW 到主存储, 然后释放 ZIL 空间。崩溃恢复时, 如果 uberblock 指向的 TXG 已提交, ZIL 内容被忽略; 如果未提交, 重放 ZIL 中的同步写。

写放大: COW 的核心代价。COW 文件系统的写放大是核心代价。一次 4 KiB 的随机写, 最坏情况下需要:

```

// 4 KiB 随机写在 Btrfs 中的写放大
// 假设 B-tree 深度 3 (root → internal → leaf → data)

// 步骤 1: 写新数据块
new_data_block = alloc_block(); // 块 8000
write_block(8000, user_data); // 1 次 4 KiB 写

// 步骤 2: COW 叶子节点 (data → 8000)
new_leaf = copy_modify(old_leaf);
new_leaf_block = alloc_block(); // 块 8100
write_block(8100, new_leaf); // 1 次 4 KiB 写

// 步骤 3: COW 父节点 (leaf → 8100)
new_parent = copy_modify(old_parent);
new_parent_block = alloc_block(); // 块 8200
write_block(8200, new_parent); // 1 次 4 KiB 写

// 步骤 4: COW 祖父节点

```

```

new_grandparent = copy_modify(old_gp);
new_gp_block = alloc_block();    // 块 8300
write_block(8300, new_gp);      // 1 次 4 KiB 写

// 步骤 5: COW 根节点
new_root = copy_modify(old_root);
new_root_block = alloc_block();  // 块 8400
write_block(8400, new_root);    // 1 次 4 KiB 写

// 步骤 6: 原子更新超级块根指针
atomic_write(superblock.root, 8400); // 1 次 4 KiB 写

// 总计: 6 次 4 KiB 写, 服务 1 次用户写
// 写放大 = 6x

```

步	写什么	块号	说明
1	数据块	8000	新数据内容
2	叶子节点	8100	COW: extent 指向 8000
3	父节点	8200	COW: 指向新叶子 8100
4	祖父节点	8300	COW: 指向新父节点 8200
5	根节点	8400	COW: 指向新祖父 8300
6	超级块	—	原子更新根指针 → 8400

Btrfs 和 ZFS 通过以下策略缓解写放大：

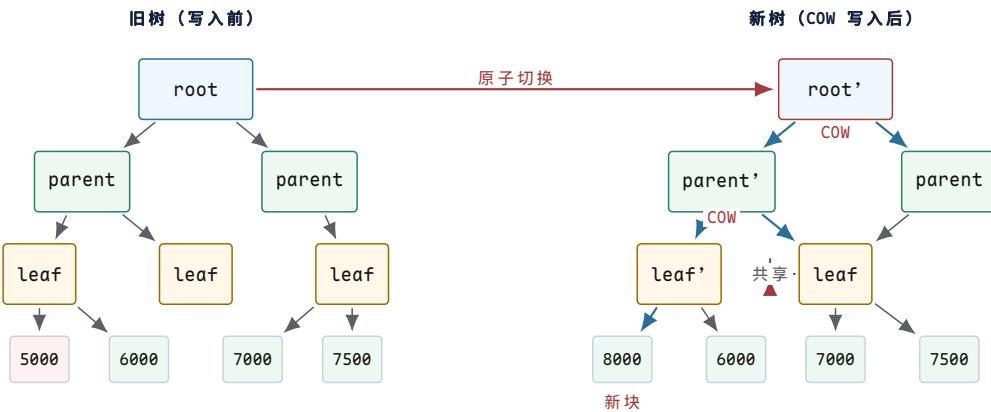
- 批量提交：把许多写操作收集到一个 TXG/transaction 中，一次提交。多用户写共享同一批 COW 元数据节点，分摊开销。
- ZIL / journal：同步写先记到小日志，不等完整 COW 路径。
- Clone-friendly allocator：分配器尽量让新块在物理上靠近，减少碎片和元数据分散。
- B-tree 节点变大：Btrfs 可用 64 KiB 节点，每个节点容纳更多条目，减少树深度和 COW 层数。

对比：ext4 vs Btrfs vs ZFS。

	ext4 (journal)	Btrfs (COW)	ZFS (COW)
崩溃恢复	日志重放	根指针原子切换	uberblock 原子切换
snapshot	需 LVM	原生 0(1)	原生 0(1)
写放大	1-1.5x	3-6x	3-6x
碎片	低（原地覆盖）	高（COW 散布）	高（COW 散布）
空间效率	需日志空间	需 refcount 元数据	需校验和 + refcount
数据完整性	无校验	有校验（checksum tree）	端到端校验 + 自修复
RAID 支持	无（需 mdadm）	无（已移除）	原生 RAID-Z
压缩	否	是（zlib/zstd）	是（lz4/gzip）

核心理想

COW 文件系统用写放大换取了崩溃一致性和 snapshot 能力。原地覆盖写文件系统用日志换取同样的崩溃一致性，但写放大更低。两者不是“谁更好”的问题，而是不同的优先级：COW 优先一致性和功能，原地覆盖优先吞吐和低延迟。



图示：COW 更新路径。写入块 5000 的数据时，不覆盖块 5000，而是分配新块 8000 写入数据。从叶子到根的路径上的每个节点都 COW 出新版本（红色标记）。旧树（左侧）保持完整，根指针原子切换到新树根。旧块 5000 仍被旧树引用，可用于 snapshot。共享的子树（parent 右子树、leaf 和 6000/7000/7500 块）不被复制。

13.12 闪存文件系统

NAND Flash 的物理约束。NAND Flash 的物理特性与 HDD 有根本区别，直接决定了闪存文件系统的设计方向。

特性	NAND Flash	HDD
读写单位	page (4-16 KiB)	sector (512 B)
擦除单位	block (128-512 KiB = 32-128 pages)	无需擦除，直接覆盖
写约束	只能写已擦除的页（全 1）	无约束
寻址	电气 (μ s 级)	机械寻道 (ms 级)
寿命	P/E 循环有限	无限（理论上）

NAND 的核心约束是写之前必须先擦除，而擦除单位远大于写入单位。一个 block 包含 64-128 个 page。要修改一个 page 的内容，不能直接覆盖—必须先擦除包含该 page 的整个 block，然后重新写入。但 block 中其他 page 可能还有有效数据，擦除会丢失它们。

```
// NAND Flash 的操作约束
// page: 4 KiB (读写单位)
// block: 256 KiB = 64 pages (擦除单位)
//
// 写一个 page:
// 如果该 page 已被擦除 (所有 bit = 1):
//   page_program(page_addr, data) // 直接写入
// 如果该 page 之前写过 (有非 1 bit):
//   不能直接覆盖! 会损坏数据
//   必须:
//   1. 找一个已擦除的新 page
//   2. 写入新数据到新 page
//   3. 旧 page 标记为 "invalid" (stale)
//   4. 当整个 block 的有效页很少时:
//      a. 把有效页复制到新 block
//      b. erase_block(old_block) → 全部变 1
//      c. 旧 block 可以重新写
//
// P/E 循环寿命 (Program/Erase cycles)
//   SLC: 100,000 次/block
//   MLC: 3,000-10,000 次/block
//   TLC: 1,000-3,000 次/block
```



```
// QLC: 100-1,000 次/block
// 超过 P/E 寿命的 block 变成坏块
```

除了 P/E 寿命限制，NAND 还有其他可靠性问题：

- *Read Disturb* (读干扰)：读取同一个 page 太多次，可能导致相邻 page 的 bit 翻转。典型阈值：每个 block 读取 10^5 – 10^6 次后需要刷新。
- *Write Disturb* (写干扰)：编程一个 page 时，高压脉冲可能影响同一 block 中相邻 page 的电荷。
- *Retention* (保持性)：存储的电荷随时间泄漏。TLC 闪存在室温下数据保持约 1-3 年，高温环境保持时间更短。
- *Bad blocks* (坏块)：出厂时有初始坏块（通常 $\leq 2\%$ ），运行中会持续产生新的坏块。

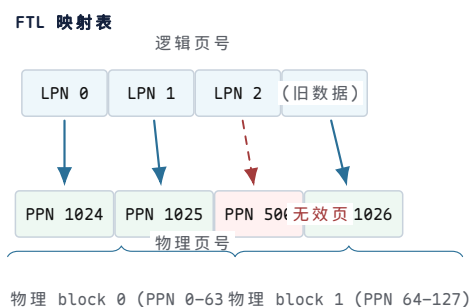
NAND Flash 的写前擦除约束是所有闪存文件系统设计的出发点。传统文件系统 (ext4) 假设可以随时覆盖任何块，这在 NAND 上不成立——要么通过 FTL 翻译层隐藏这个约束，要么设计专门的日志结构文件系统 (F2FS) 直接适配它。

FTL：闪存翻译层。大多数 SSD 内部有 FTL (Flash Translation Layer)，把 NAND 的页/块语义翻译成块设备的扇区语义，对上层文件系统透明。FTL 的核心职责是地址映射、垃圾回收和磨损均衡。

```
// FTL 地址映射表
// 逻辑扇区号 → 物理页号
//
// 1 TiB SSD, 4 KiB pages
// 逻辑扇区数 = 1 TiB / 512 B = 2 * 10^9
// 逻辑页数 = 1 TiB / 4 KiB = 256 * 10^6 = 256M
// 映射表项 = 256M entries * 4 bytes = 1 GiB
//
// 这张表存在 SSD 的 DRAM 中 (SSD 有自己的 DRAM)
// 断电时持久化到 NAND 的保留区域

struct ftl_mapping_entry {
    uint32_t logical_page; // 逻辑页号
    uint32_t physical_page; // 物理页号
};
// 映射表: logical_page -> physical_page

// FTL 写操作
ftl_write(logical_page, data):
    // 1. 分配一个已擦除的物理页
    physical_page = alloc_erased_page();
    // 2. 编程写入数据
    nand_page_program(physical_page, data);
    // 3. 更新映射表
    old_physical = mapping_table[logical_page];
    mapping_table[logical_page] = physical_page;
    // 4. 旧物理页标记为无效
    mark_page_invalid(old_physical);
    // 注意: 旧页不能直接擦除
    //      要等整个 block 的有效页都迁移后才能擦除
```



图示：FTL 地址映射表。逻辑页号 (LPN) 映射到物理页号 (PPN)。LPN 2 的旧映射指向 PPN 500，已被新写入标记为无效。当物理 block 0 的有效页比例足够低时，FTL 垃圾回收会迁移有效页并擦除整个 block。

FTL 垃圾回收。当空闲的已擦除 block 不足时，FTL 执行垃圾回收，回收包含大量无效页的 block：

```
ftl_gc():
// 1. 选择 victim block: 无效页最多的 block
victim = find_block_with_most_invalid_pages();
// 假设 victim 有 60 个无效页, 4 个有效页

// 2. 分配一个已擦除的新 block
new_block = alloc_erased_block();

// 3. 遍历 victim 中的每个 page
new_page_offset = 0;
for page in victim.pages:
    if is_page_valid(page):
        // 有效页: 复制到新 block
        data = nand_page_read(page.physical);
        nand_page_program(new_block[new_page_offset], data);
        // 更新映射表
        mapping_table[page.logical] = new_block[new_page_offset];
        new_page_offset++;
    // 无效页: 跳过, 不复制

// 4. 擦除旧 block
nand_block_erase(victim);
// 现在 victim 是全 1, 可以重新写入

// 5. 新 block 还有剩余已擦除页, 加入空闲池
// victim 有 64 pages, 4 个被复制, 60 个空闲
// → 新 block 提供 60 个空闲页

// 写放大计算:
// 用户写入 4 个页 (4 * 4 KiB = 16 KiB)
// GC 迁移了 4 个有效页 (4 * 4 KiB = 16 KiB)
// 总闪存写 = 用户写 + GC 写 = 16 + 16 = 32 KiB
// 写放大 = 32 / 16 = 2x
//
// 如果 victim 只有 1 个有效页:
// 用户写 63 页, GC 迁移 1 页
// 写放大 = (63 + 1) / 63 ≈ 1.02x (很好)
//
// 如果 victim 有 32 个有效页:
// 用户写 32 页, GC 迁移 32 页
// 写放大 = (32 + 32) / 32 = 2x
//
// 最坏情况: 几乎所有页都有效, GC 频繁迁移
```

```
// 写放大可达 3-4x 甚至更高
```

磨损均衡。P/E 循环有限，必须把写操作均匀分布到所有 block，避免热点 block 过早耗尽。磨损均衡分两种：

```
// 动态磨损均衡 (Dynamic Wear Leveling)
// 分配器选择 P/E 循环最少的已擦除 block
ftl_alloc_page_dynamic():
    // 从空闲 block 池中选择 erase_count 最小的
    block = min(free_blocks, key=lambda b: b.erase_count);
    return block.alloc_page();

// 静态磨损均衡 (Static Wear Leveling)
// 后台任务：把冷数据从高磨损 block 迁移到低磨损 block
// 释放高磨损 block 给新写入使用
ftl_static_wl():
    // 找到 erase_count 最高的 block (写多，存冷数据)
    hot_block = max(all_blocks, key=lambda b: b.erase_count);
    // 找到 erase_count 最低的 block (写少，存热数据)
    cold_block = min(all_blocks, key=lambda b: b.erase_count);

    if hot_block.erase_count - cold_block.erase_count > threshold:
        // 交换：把冷数据移到低磨损 block
        for page in hot_block.valid_pages:
            data = read_page(page);
            new_page = cold_block.alloc_page();
            write_page(new_page, data);
            update_mapping(page.logical, new_page);
        // 高磨损 block 现在空了，擦除后可用于新写入
        erase_block(hot_block);
```

TRIM/DISCARD。文件系统删除文件时只更新元数据 (bitmap、inode)，不擦除数据块内容。在 HDD 上这不影响性能—磁盘不需要擦除就能覆盖写。但 SSD 的物理块需要先擦除才能写入，FTL 不知道哪些逻辑块对应的物理块可以回收。

TRIM (也叫 **DISCARD**) 命令让文件系统告诉 SSD “这些逻辑块不再被使用”：

```
// 文件系统删除文件后发 TRIM
unlink(path):
    inode = resolve(path)
    // 释放 inode 和数据块 (更新文件系统元数据)
    free_inode(inode)
    free_blocks(inode.block[])
    // 告诉 SSD 这些逻辑块不再使用
    // blkdev_discard 发送 DISCARD/TRIM 命令
    blkdev_discard(bdev,
        inode.block[] * blocksize,
        inode.block_count * blocksize)

// FTL 收到 TRIM 后
ftl_trim(logical_blocks):
    for lb in logical_blocks:
        physical = mapping_table[lb]
        // 标记物理页为无效
        mark_page_invalid(physical)
        // 不需要立即擦除
        // 垃圾回收时这些无效页不需要迁移
        // → GC 更快，写放大更低
```

```
// ext4 挂载选项：
// mount -o discard /dev/nvme0n1 /mnt
//   → 同步 TRIM：删除时立即发 TRIM
// mount -o discard=async /dev/nvme0n1 /mnt
//   → 异步 TRIM：收集后批量发
// mount -o noDiscard /dev/nvme0n1 /mnt
//   → 禁用 TRIM
```

	无 TRIM	有 TRIM
GC 迁移	迁移已删除数据的无效页	不迁移（已知无效）
写放大	高	低
空闲块	少（GC 不知哪些可回收）	多

F2FS：日志结构闪存文件系统。F2FS (Flash-Friendly File System) 是 Samsung 开发的 Linux 文件系统，专门为 NAND Flash 优化。核心设计是日志结构 (log-structured)：所有写操作都是顺序追加，不覆盖旧数据。

```
// F2FS 磁盘布局
// 整个设备分为以下区域（按顺序）：
//
// [Superblock] [Checkpoint] [SIT] [NAT] [SSA] [Main Area]
//
// Superblock: 全局参数（块大小，段大小，总段数）
// Checkpoint: 两个交替的 checkpoint（崩溃恢复）
//   每个 checkpoint 记录：
//     - NAT 的根位置
//     - SIT 的根位置
//     - Main Area 的空闲段位置
//     - 活跃 segment 列表
//
// SIT (Segment Info Table):
//   每个 segment 的状态：有效块数，无效块数
//   GC 用 SIT 找到无效块最多的 segment
//
// NAT (Node Address Table):
//   inode 号 → node block 的物理地址
//   类似于 ext4 的 indirect block，但用一张表而非指针
//   作用：数据块地址不直接存在 inode 中
//     inode -> NAT -> node block -> data block
//
// SSA (Segment Summary Area):
//   每个 segment 内每个块的：(inode 号，偏移)
//   GC 用 SSA 判断一个块是否有效
//   有效 = 该逻辑块仍被某 inode 引用
//
// Main Area: 6 种 segment 类型
//   hot node: 直接 inode block（频繁更新）
//   warm node: 间接 node block
//   cold node: 目录/很少更新的 node
//   hot data: 目录数据（频繁更新）
//   warm data: 普通文件数据
//   cold data: 很少访问的数据（多媒体等）
```



图示：F2FS 磁盘布局。Superblock 和 Checkpoint 是元数据入口。SIT/NAT/SSA 是三张辅助表，分别管理段状态、节点地址映射和块有效性。Main Area 分 6 种 segment 类型 (hot/warm/cold × node/data)，按热度分离写入。所有写入都是顺序追加到 log tail。

F2FS 写路径与 GC。 F2FS 的写路径完全顺序化：新数据始终追加到 log tail 的下一个空闲 segment。旧数据所在的 segment 逐渐变为只有少量有效页。

```
// F2FS 写路径
f2fs_write(inode, offset, data):
    // 1. 找到当前 log tail 的位置
    //    (hot/warm/cold 决定写入哪个 segment 类型)
    segment = get_active_segment(SEG_WARM_DATA);
    page_offset = segment.next_free_page;

    // 2. 写数据到 segment
    write_page(segment, page_offset, data);
    segment.next_free_page++;

    // 3. 更新 NAT: inode 的 node block 指向新数据位置
    //    node block 本身也要 COW (日志结构)
    node_block = get_node_block(inode);
    new_node_block = copy_modify(node_block,
        offset -> segment:page_offset);
    node_segment = get_active_segment(SEG_WARM_NODE);
    new_node_addr = node_segment.next_free_page;
    write_page(node_segment, new_node_addr, new_node_block);

    // 4. 更新 NAT 表: inode -> new_node_block 地址
    nat_update(inode, new_node_addr);
    // 旧 node block 所在 page 变为无效

    // 5. 更新 SSA: 记录新数据页和 node 页的归属
    ssa_update(segment, page_offset, (inode, offset));
    ssa_update(node_segment, new_node_addr, (inode, NODE));

    // 6. 如果当前 segment 写满，分配新 segment
    if segment.next_free_page >= segment_size:
        segment = alloc_new_segment(SEG_WARM_DATA);

// F2FS GC
f2fs_gc():
    // 1. 用 SIT 找无效块最多的 segment
    victim = find_segment_with_most_invalid_blocks();
    //    例如: segment 有 512 blocks, 480 无效, 32 有效

    // 2. 用 SSA 确定每个有效块的归属
    for block in victim.valid_blocks:
        (inode, offset) = ssa_lookup(block);
        // 3. 复制有效块到 log tail
        data = read_block(block);
        f2fs_write(inode, offset, data); // 追加到新位置
        //    NAT 和 SSA 自动更新
```

```
// 4. 整个 segment 现在全是无效块，回收
free_segment(victim);
// segment 加入空闲池，可重新使用
```

F2FS 的 hot/warm/cold 分离是关键优化。元数据 (hot node, 频繁更新) 与用户数据 (warm/cold data) 写入不同的 segment。这样 GC 时不会因为迁移热元数据而连带迁移冷数据—热 segment 会更频繁地被 GC 回收 (因为无效页增长快), 但冷 segment 不受影响。

segment 类型	内容	GC 特点
hot node	直接 inode block	更新频繁，无效快，GC 频繁但小
warm node	间接 node block	中等频率
cold node	目录 node	很少更新，GC 几乎不碰
hot data	目录数据	频繁更新
warm data	普通文件数据	中等
cold data	多媒体等	写一次读多次，GC 不碰

日志结构文件系统原理 (LFS)。 F2FS 的设计思想源自 LFS (Log-structured File System)，由 Rosenblum 和 Ousterhout 在 1992 年提出。LFS 的核心原则：磁盘是一个只追加的日志。

```
// LFS 核心原理
// 1. 所有写操作都是追加：
//    数据块、inode、目录项、间接块
//    全部追加写入日志尾部
//    旧版本数据变成 "garbage" (垃圾)

// 2. inode map (类似 F2FS 的 NAT):
//    inode 号 -> inode 最新版本的磁盘位置
//    inode map 本身也写入日志

// 3. 读操作：
//    查 inode map 找到 inode 位置
//    读 inode 获取数据块指针
//    读数据块 (位置在日志中，可能很分散)
//    → 随机读可能慢 (数据散布在日志各处)

// 4. 清理 (cleaner / GC):
//    后台进程扫描旧 segment
//    复制有效块到 log tail
//    释放旧 segment

// LFS 的基本权衡：
// 写：快 (顺序追加，无需寻道/擦除)
// 读：可能慢 (数据散布在日志中，随机访问)
// GC：后台开销，可能影响前台延迟
```

	LFS/F2FS (日志结构)	ext4 (原地覆盖 + 日志)
写模式	顺序追加	原地覆盖 + 独立日志
写放大	低 (无覆盖写)	中 (日志 + 主 FS)
随机读	慢 (数据散布)	快 (extent 连续)
GC 开销	有 (cleaner)	无
空间开销	NAT/SIT/SSA 元数据	bitmap + 日志
碎片	随时间增长	延迟分配缓解

核心理念

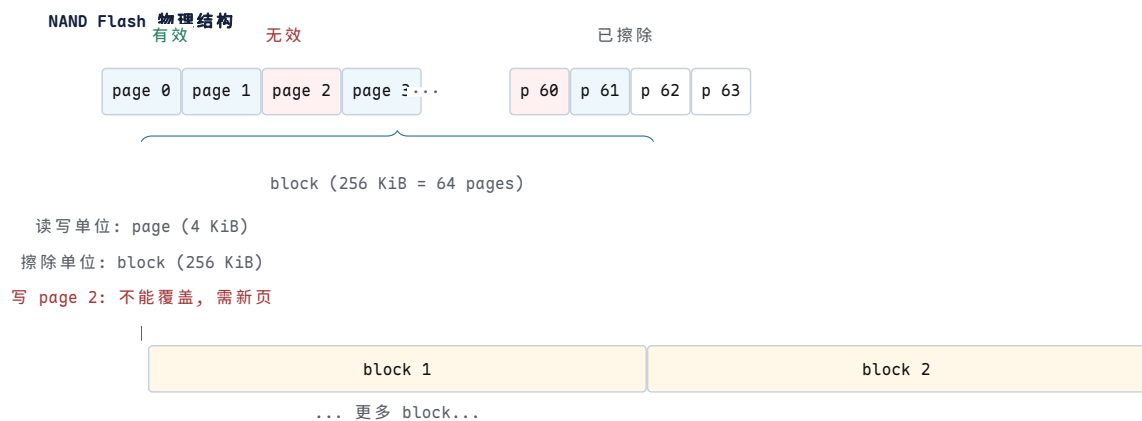
日志结构文件系统的核心收益是写性能：NAND Flash 的顺序写比随机写快得多（无需先擦除，利用 page program 的顺序特性），且写放大更低。代价是读性能可能下降（数据散布在日志各处）和 GC 的后台开销。F2FS 通过 hot/warm/cold 分离、NAT 间接寻址和前台 GC 控制来缓解这些代价。

F2FS vs ext4 on SSD。在 SSD 上，F2FS 和 ext4 都能工作，但性能特征不同：

	F2FS	ext4
写模式	顺序追加（闪存友好）	原地覆盖（FTL 翻译）
写放大	低（无覆盖，GC 高效）	中（FTL GC + ext4 日志）
元数据开销	高（NAT + SIT + SSA）	低（bitmap + inode table）
随机读	中（NAT 间接一层）	快（extent 直接映射）
崩溃恢复	checkpoint 交替	jbd2 日志重放
TRIM	集成到 GC	需挂载选项

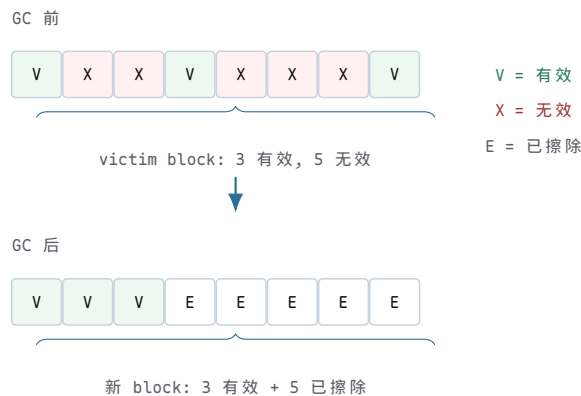
F2FS 的优势在写密集场景：闪存的顺序追加写不需要 FTL 做额外的地址翻译和 GC，写放大接近 1x。ext4 的原地覆盖写触发 FTL 的写前擦除约束，FTL 内部 GC 产生额外写放大。

ext4 的优势在读密集和元数据密集场景：extent 树直接映射逻辑块到物理块，而 F2FS 需要通过 NAT 间接一层。对于小文件和目录操作，ext4 的 bitmap 分配比 F2FS 的 segment 管理更轻量。



图示：NAND Flash 物理结构。一个 block 包含 64 个 page (256 KiB)。绿色为有效页，红色为无效页（旧数据），白色为已擦除页（全 1，可写）。写单位是 page，擦除单位是 block。修改 page 2 不能直接覆盖——必须分配新页写入，旧页标记无效。当 block 的无效页足够多时，GC 迁移有效页并擦除整个 block。

FTL 垃圾回收过程



图示：FTL 垃圾回收过程。GC 前 victim block 有 3 个有效页和 5 个无效页。GC 把 3 个有效页复制到新 block，然后擦除 victim block。结果是一个新 block 包含 3 个有效页和 5 个已擦除页，可以接受新写入。写放大 = (3 迁移 + 3 用户写) / 3 = 2x。

核心思想

从 page cache 到 COW 到闪存文件系统，本章展示了文件系统设计的核心张力：性能、一致性、介质适应性三者之间的取舍。page cache 用内存换延迟；日志用额外写换恢复速度；COW 用写放大换崩溃一致性和 snapshot；日志结构用顺序写换闪存友好性。没有免费的午餐——每个优化都在另一个维度产生代价。理解这些取舍，才能根据工作负载和硬件选择正确的文件系统。

13.13 现代优化技术

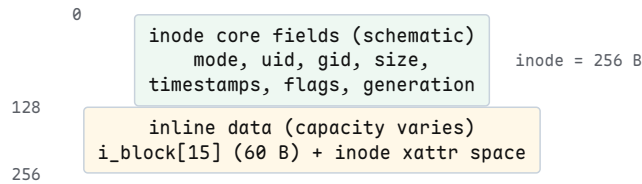
前几节建立了文件系统的基础机制：索引节点、目录项、日志、B+ 树、空间分配。然而，真实世界中的文件系统还必须应对一系列工程挑战——小文件膨胀导致的空间浪费、静默数据腐烂 (bit rot)、闪存设备的写放大、延迟敏感型工作负载、容器镜像的层叠结构。本节深入分析现代文件系统采用的核心优化技术，每一项都是资源 (CPU、内存、复杂性) 与收益 (I/O、空间、可靠性) 之间的权衡。

inline data：小文件内嵌。一个典型的配置文件——`/etc/hostname`——通常只有 10–20 字节。在传统的 ext4 布局中，即使文件只有 1 字节，文件系统也会为其分配至少一个 4 KiB 数据块。inode 本身占据 256 字节，其中 `i_block[15]` 数组 (60 字节) 用于存放区段树 (extent tree) 的根节点。对于小文件而言，这些空间大部分被浪费。

ext4 的 *inline data* 特性 (标志 `EXT4_FEATURE_INCOMPAT_INLINE_DATA`) 解决了这个问题：当文件足够小时，直接将文件数据内嵌到 inode 结构中，不需要任何独立的数据块。`i_block[]` 数组原本用于存放区段指针，现在被重新利用为内嵌数据区域。对于一个 256 字节的 inode，标准元数据占据前 128 字节，剩余 128 字节的额外 inode 空间 (extra inode space) 也可以用于内嵌数据，合计 ~ 128 字节。ext4 进一步支持通过 `i_data` 字段 (位于额外 inode 空间内) 将内嵌容量扩展到 ~ 384 字节。

```
/* fs/ext4/ext4.h - simplified on-disk inode */
struct ext4_inode {
    __le16 i_mode;           /* 0 file mode */
    __le16 i_uid;            /* 2 low 16 bits of UID */
    __le32 i_size_lo;        /* 4 size (low 32 bits) */
    __le32 i_atime;          /* 8 access time */
    __le32 i_ctime;          /* 12 inode change time */
    __le32 i_mtime;          /* 16 modification time */
    __le32 i_dtime;          /* 20 deletion time */
    __le16 i_gid;            /* 24 low 16 bits of GID */
    __le16 i_links_count;    /* 26 hard link count */
    __le32 i_blocks_lo;      /* 28 blocks count */
    __le32 i_flags;          /* 32 inode flags */
    __le32 i_osd1;           /* 36 OS-specific */
    __le32 i_block[15];      /* 40 block pointers / extents
                               repurposed for inline_data */
    __le32 i_generation;     /* 100 version (NFS) */
    __le32 i_file_acl_lo;    /* 104 extended attr block */
    /* --- standard inode ends at offset 128 --- */
    __le32 i_size_high;      /* 108 size (high 32 bits) */
    __le32 i_obso_faddr;     /* 112 obsolete fragment addr */
    __le16 i_checksum_lo;    /* CRC32C checksum (low) */
    /* --- extra inode space: up to 256 bytes total --- */
    /* inline data continues here via xattr mechanism */
}
```

```
};
```



图示：ext4 inline data inode 布局示意。前一部分数据可放在 60 字节的 `i_block[]` 区域，更多内容可使用 inode 的扩展属性空间；实际容量取决于 inode 大小和已有 xattr，不能固定写成 128 字节。足够小的文件可以不分配独立数据块。

Btrfs 提供了类似但更慷慨的内嵌支持。Btrfs 的 *inline extent* 可以将整个小文件存储在 B 树叶子节点中，容量上限取决于叶子大小（通常 4 KiB）减去头部和其他条目，实际上可达 ~ 3.8 KiB。由于 Btrfs 的 B 树叶子已经在内存中，内嵌文件的读取不需要额外的 I/O。

影响分析：考虑一个典型的 `/etc` 目录，包含 ~ 3000 个配置文件，其中大多数 < 128 字节。在不启用 inline data 时，每个小文件至少需要 1 个 inode 块（与其他 inode 共享）加 1 个数据块 = 8 KiB I/O（最坏情况）。启用 inline data 后，文件数据随 inode 一起读取，零额外 I/O。对于遍历 `/etc` 的全量扫描，I/O 从 ~ 3000 次数据块读取降至 0 次——所有数据已在 inode 块中。

核心理念

inline data 的本质是将 元数据存储与 数据存储合并到同一个物理块。当文件大小 $\leq$ inode 内嵌容量时，数据访问退化为元数据访问——只需一次 I/O 即可同时获取元数据和数据。

元数据校验和。存储介质会发生 静默数据腐烂（silent data corruption, bit rot）。原因包括：宇宙射线导致的位翻转、控制器固件 bug、NAND 读取扰动（read disturb）、磁介质磁畴退化、固件写入不完整。一个被损坏的 inode 块如果不被检测到，会导致文件系统将错误的物理块号映射给文件——这是灾难性的。

ext4 的 *metadata checksums* 特性（标志 `EXT4_FEATURE_RO_COMPAT_METADATA_CSUM`）为以下结构添加 CRC32C 校验和：

结构	校验和字段	覆盖范围
superblock	<code>s_checksum</code>	superblock 全部字段
group descriptor	<code>bg_checksum</code>	group descriptor 全部字段
inode	<code>i_checksum_lo/hi</code>	inode + inode number + generation
extent header	<code>eh_checksum</code>	extent header + 所有 extent 条目
directory htree	<code>dx_tail</code> (dx_node 尾部)	目录块全部内容
block bitmap	(group descriptor 中的校验和)	bitmap 块全部内容

CRC32C（Castagnoli 多项式 0x1EDC6F41）被选中是因为 x86 SSE4.2 提供了硬件加速指令 `crc32`，使得校验计算几乎零开销。ARMv8 也提供了 `CRC32` 指令。在没有硬件支持的平台，内核回退到软件 CLMUL 实现。

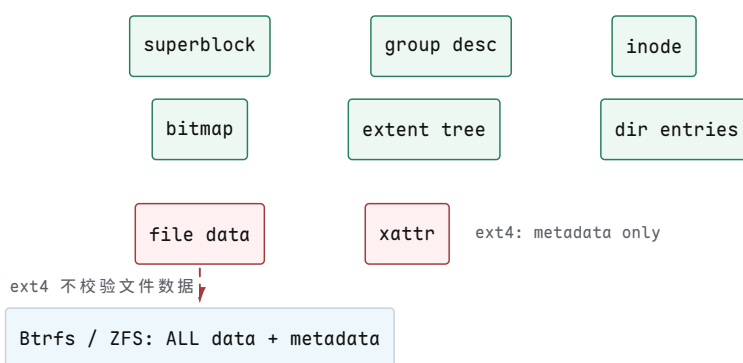
```
/* fs/ext4/extents.c - extent checksum computation (simplified) */
static __le32 ext4_extent_block_csum(struct inode *inode,
```

```

                                struct ext4_extent_header *eh)
{
    struct ext4_inode_info *ei = EXT4_I(inode);
    struct ext4_extent_tail *tail;

    /* checksum covers: extent header + all extent entries
       (but NOT the tail itself, which holds the checksum) */
    tail = find_ext4_extent_tail(eh);
    return cpu_to_le32(
        ext4_crc32c(inode->i_sb,
                    (void *)eh,
                    (__u8 *)tail - (__u8 *)eh, /* data to checksum */
                    ei->i_crc_seed));           /* seed = inode
                                                number + generation */
}

```



图示：ext4 仅对元数据计算 CRC32C 校验和。文件数据块本身没有校验。Btrfs 和 ZFS 对所有数据和元数据均计算校验和，并在每次读取时验证。

ext4 的元数据校验和不保护文件数据内容。如果一个数据块在磁盘上发生了位翻转，ext4 在读取该块时不会检测到错误，应用程序会收到损坏的数据。只有 Btrfs 和 ZFS 的全数据校验才能在读取时检测并（通过镜像/校验）修复数据损坏。

ZFS 的校验策略更为彻底：每个块（数据块和元数据块）都有 256 位校验和（默认 `fletcher4`，可选 `sha256` 或 `edon-R`）。ZFS 在每次读取时都会重新计算并验证校验和。如果校验失败，ZFS 会尝试从镜像副本或 RAID-Z 校验数据中恢复正确的块。Btrfs 默认使用 CRC32C，也支持 `xxhash64`、`sha256` 和 `blake2b`。

透明压缩。存储 I/O 是许多工作负载的瓶颈。如果数据可以被压缩，那么在写入时压缩、在读取时解压，可以减少实际 I/O 量——以 CPU 时间换取 I/O 带宽。Btrfs、ZFS 和 F2FS 支持透明压缩（transparent compression），应用程序无需感知。

写入路径：应用程序调用 `write()`，数据进入页缓存（page cache）。在写回时，文件系统对每个数据块执行压缩算法。如果压缩后的块比原始块小，文件系统分配更少的物理空间并记录压缩信息（压缩算法、压缩后大小）。**读取路径：**文件系统读取压缩后的块，执行解压，返回原始数据。

```

/* ZFS compression: lz4 early-abort logic (simplified) */
int lz4_compress(void *src, void *dst, size_t s_len, size_t d_len)
{
    /* Early abort: compress first 4 KiB.
       If ratio < 1/8 (12.5% improvement), skip entirely. */
    if (s_len >= 4096) {

```

```

    int prefix = lz4_compress_4k(src, dst);
    if (prefix > 4096 * 7 / 8) {
        /* Not compressible enough - store uncompressed */
        return -1; /* signals: store raw block */
    }
}
/* Full compression attempt */
return lz4_compress_full(src, dst, s_len, d_len);
}

```

文件系统	默认算法	可选算法	早期中止	特点
ZFS	lz4	gzip, zstd	lz4: 前 4 KiB	lz4 速度 ~ 500 MB/s 压缩, ~ 2.5 GB/s 解压
Btrfs	zlib	zstd, lzo, none	zlib/zstd: 是	zstd 压缩比更好, 速度接近
F2FS	lz4	zstd, none	是	针对 NAND 优化的压缩
ext4	—	—	—	不支持透明压缩

具体数字：1 GiB 文本文件，4 KiB 块大小 = 262144 个块。假设压缩比为 3:1（典型文本文件），压缩后需要 87381 个块 = $87381 \times 4 \text{ KiB} = 341 \text{ MiB}$ 。I/O 减少了 67%。

```

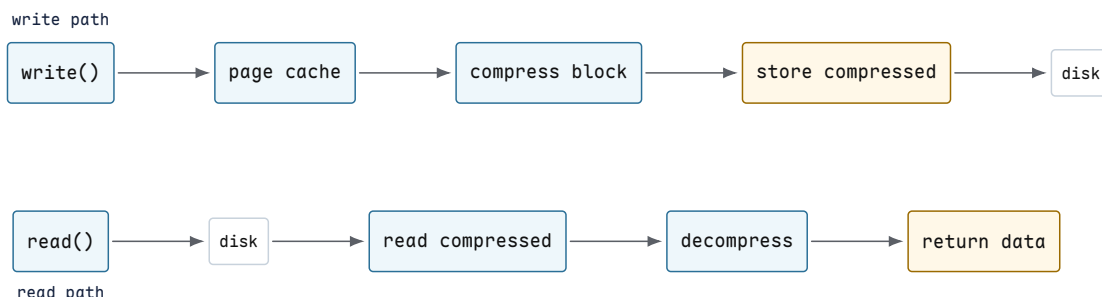
=== 压缩 I/O 节省计算 ===
原始: 1 GiB / 4 KiB = 262144 blocks
压缩比: 3:1
压缩后: 262144 / 3 = 87381 blocks (向上取整)
空间: 87381 * 4 KiB = 341 MiB (节省 683 MiB, 67%)
I/O: 写入减少 67%, 读取减少 67%
CPU: 压缩和解压成本必须在目标 CPU、数据集和算法级别上实测

```

```

=== 性能权衡 ===
I/O-bound: CPU 空闲等 I/O → 压缩用 CPU 换 I/O → 吞吐 ↑
CPU-bound: CPU 已满 → 压缩增加 CPU 负载 → 吞吐 ↓
存储受限且 CPU 有余量 → 压缩可能提升吞吐
CPU 已饱和或数据不可压缩 → 压缩可能降低吞吐

```



图示：透明压缩的写入与读取路径。写入时压缩每个数据块，仅存储压缩后的数据。读取时先读取压缩块，再解压返回。对 I/O-bound 的工作负载，CPU 用在压缩上的时间通常远小于节省的 I/O 等待时间。

块级去重。许多存储系统包含大量重复数据：100 台虚拟机运行相同的操作系统镜像，容器镜像共享相同的基础层，备份系统存储逐日累积的相似快照。块级去重(block-level deduplication)在块内容粒度上消除冗余：如果两个不同的逻辑块包含完全相同的字节序列，文件系统只存储一份物理副本。

机制：ZFS 维护一张去重表（Dedup Table, DDT）。对于每个写入的块，ZFS 计算 SHA-256 哈希，然后在 DDT 中查找：

```
/* ZFS dedup write path (simplified) */
int ddt_lookup(zfs_sb_t *zsb, ddt_entry_t *dde, void *data)
{
    /* 1. Compute SHA-256 hash of block content */
    dde->dde_hash = sha256_hash(data, blocksize);

    /* 2. Check dedup table */
    ddt_entry_t *existing = ddt_table_lookup(zsb, &dde->dde_hash);
    if (existing != NULL) {
        /* 3a. Block already exists: increment refcount, return */
        existing->dde_phys.ddp_refcnt++;
        dde->dde_phys = existing->dde_phys; /* physical block addr */
        return DDT_EXISTING; /* no actual write performed */
    }

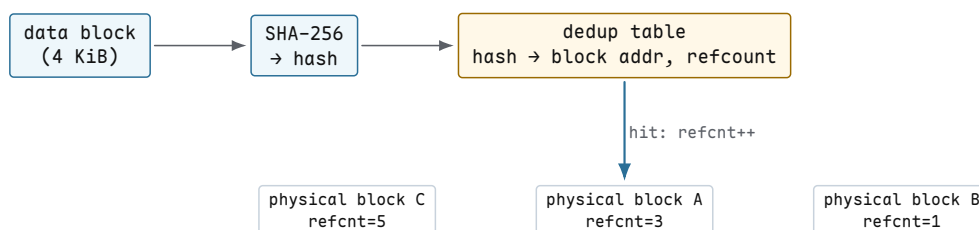
    /* 3b. New block: write to disk, add to DDT */
    ddt_table_add(zsb, dde);
    return DDT_NEW; /* caller performs actual write */
}
```

内存代价：去重表必须常驻内存（或在 ARC 缓存中），否则每次写入都需要额外的 I/O 来查表。对于 1 TiB 存储空间，4 KiB 块大小：

```
=== 去重表内存计算 ===
存储容量：    1 TiB = 2^40 B
块大小：      4 KiB = 2^12 B
总块数：      2^40 / 2^12 = 2^28 = 268,435,456 (≈ 256M)

每个 DDT 条目：
SHA-256 哈希：    32 B
物理块指针：      8 B
引用计数：        4 B
哈希链表指针：    16 B
填充/对齐：       ~40 B
-----
合计：            ≈ 100 B

总内存：268M × 100 B ≈ 25.6 GiB
→ 1 TiB 存储需要 25 GiB RAM 仅仅用于去重表
→ 10 TiB 需要 256 GiB RAM → 不现实
```



图示：块级去重流程。写入时计算块内容的 SHA-256 哈希，在去重表中查找。如果哈希已存在，增加引用计数并返回已有物理块地址——不执行实际写入。新块才真正写入磁盘并添加到去重表。

去重表的内存在规模化后变得不可接受。对于 10 TiB 存储，DDT 需要 ~ 256 GiB RAM。如果 DDT 不在内存中，每次写入的哈希查找都需要磁盘 I/O，性能

急剧下降。这就是为什么 ZFS 文档反复强调：“dedup is always a bad idea unless you know exactly what you are doing”。

最佳用例：VM 镜像（多台虚拟机安装相同操作系统，90%+ 块重复）、容器镜像（共享基础层，层间大量重复块）、备份目标（每日增量备份的快照间重复率极高）。对于这些工作负载，去重可以节省 50-90% 的存储空间，且 DDT 大小可控（因为重复块数量有限）。

预读和预取。当应用程序顺序读取文件时，每次 `read()` 触发一次页缓存缺失(cache miss)，导致一次磁盘 I/O。如果文件系统能预测应用程序接下来会读哪些页，它可以提前将这些页加载到页缓存中。Linux 内核的 *read-ahead* 机制实现了这一预测。

Linux 的预读算法是一个 自适应状态机（位于 `mm/readahead.c`）。它跟踪每个文件的访问模式，动态调整预读窗口大小：

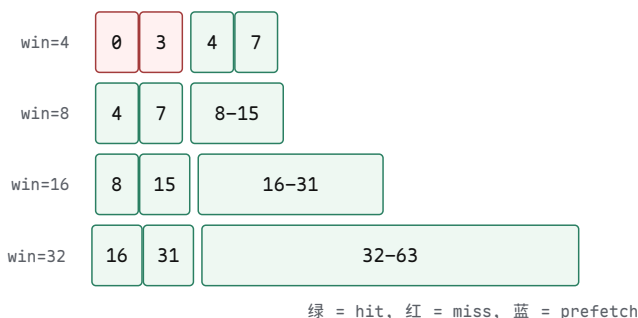
=== 顺序读取：预读窗口增长 ===

```
read(0):  cache miss → read pages [0-3],    prefetch [4-7]      win=4
read(4):  cache hit  → win × 2 = 8,         prefetch [8-15]     win=8
read(8):  cache hit  → win × 2 = 16,        prefetch [16-31]    win=16
read(16): cache hit  → win × 2 = 32,        prefetch [32-63]    win=32
read(32): cache hit  → win × 2 = 64,        prefetch [64-127]   win=64
read(64): cache hit  → win × 2 = 128,       prefetch [128-255]  win=128 (cap)
```

→ 128 pages = 512 KiB 预读窗口上限（4 KiB page）

=== 随机回退读取：窗口重置 ===

```
read(0):  cache miss → reset win=4, read [0-3]      win=4
          (检测到回退 → 标记为随机访问 → 停止预读)
```



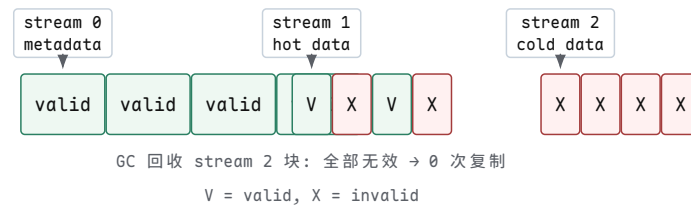
图示：预读窗口增长过程。每次缓存命中（绿色）后，窗口大小翻倍。蓝色区域为预取的页。窗口从 4 页增长到 128 页（512 KiB），在连续命中 5 次后达到上限。

预读算法的关键在于区分顺序访问与随机访问。如果检测到 `lseek()` 回退（当前读取位置在上次预读窗口之前），算法判定为随机访问，将窗口重置为 0（不预读）。这避免了为随机访问工作负载预取无用的页——随机 4 KiB 读取时预读 512 KiB 会浪费 99% 的 I/O 带宽。

多流写入。SSD 的垃圾回收 (garbage collection, GC) 是写放大的主要来源。当 GC 需要回收一个擦除块时，必须先将其中的有效页复制到其他块。如果一个擦除块中混合了长寿命数据（如文件系统元数据）和短寿命数据（如临时文件），GC 在回收短寿命数据的同时被迫复制长寿命数据——这完全是不必要的写放大。

NVMe streams (NVMe 1.3+ 规范) 通过 *Directive* 机制解决了这个问题。SSD 暴露多个流 (stream)，每个流被映射到不同的擦除块组。文件系统将不同生命周期的写入分配到不同的流：

- 流 0：文件系统元数据 (journal, inode table, directory blocks) ——长寿命
- 流 1：热数据 (数据库活跃表、日志文件) ——中寿命
- 流 2：冷数据/临时数据 (日志轮转、临时文件) ——短寿命



图示：NVMe 多流写入。不同生命周期的数据分配到不同流，SSD 内部将流映射到不同的擦除块组。当 GC 回收 stream 2 的块时，所有页已无效—零次有效页复制，写放大 = 1.0。

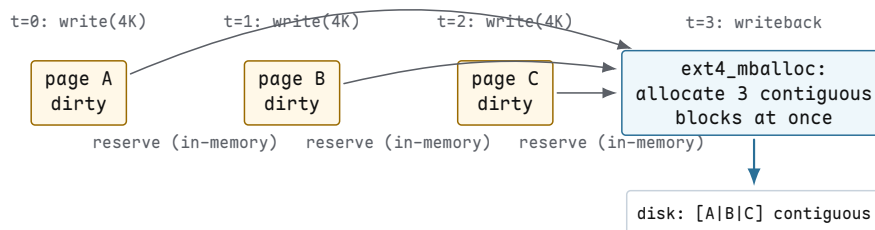
收益：当 GC 回收 stream 2 的擦除块时，由于该流中的数据都是短寿命的，很可能所有页都已经失效—无需复制任何有效页，直接擦除即可。而对于 stream 0 的块，由于元数据是长寿命的，GC 很少需要回收它们。多流将生命周期相似的数据隔离到不同的擦除块，从根本上减少了 GC 的有效页复制量。

ext4 和 XFS 的多流支持通过 `write_hint` 接口 (`f_hint` 字段在 `struct inode` 中) 实现，传递到块层的 `REQ_OP_WRITE` 请求中，最终通过 NVMe Directive 命令传递到 SSD。

延迟分配深入。传统文件系统在 `write()` 系统调用时立即分配物理块。但这有几个问题：(1) 如果应用程序先写 4 KiB 块 A，再写 4 KiB 块 B (不相邻的偏移量)，文件系统可能为 A 和 B 分配不相邻的物理块—即使 B 的写入随后紧随 A。(2) 对于追加写入 (append)，立即分配无法利用未来的写入模式来优化布局。

ext4 的延迟分配 (delayed allocation, `delalloc`) 将物理块分配推迟到写回 (writeback) 时间：

1. `write()` 系统调用：数据进入页缓存，标记为脏页。ext4 在内存中为该 inode 预留所需块数 (per-inode reservation)，但不分配物理块。`write()` 返回成功。
2. 写回触发 (定时器、脏页比例超限、`fsync()`)：ext4 的多块分配器 `ext4_mballocc` (`fs/ext4/mballoc.c`) 一次性为该 inode 的所有脏页分配物理块。分配器在选择最佳连续区段 (contiguous extent) 时拥有全局视角—它看到所有待写入的页，可以选择最大连续的物理区段。
3. 数据写入磁盘：所有脏页按分配的连续物理块顺序写入，最大化 I/O 效率。



图示：延迟分配时间线。三次 `write()` 调用仅在内存中预留空间。写回时，`ext4_mballocc` 一次性为所有脏页分配连续物理块，实现最优布局。

预留与超额预留：每个 inode 按需预留块数，但多个 inode 的预留可能重叠 (即总预留量可能超过实际可用空间)。ext4 使用全局空闲块计数器 (`s_freeclusters_counter`) 作为权威判断：在 `write()` 时检查全局计数器是否有足够空间，但实际分配在写回时才发生。如果写回时发现空间不足 (其他 inode 的写回消耗了空间)，`write()` 已经返回了成功—应用程序会收到一个延迟的 `ENOSPC` 错误。

延迟分配的 `ENOSPC` 延迟问题：`write()` 返回成功但写回时空间不足。ext4 通过 `li_get_block_create_hint` 和预留机制尽量避免此问题，但在接近满盘时仍可能发生。应用程序应当检查 `write()` 和 `fsync()` 的返回值——`fsync()` 会冲刷所有延迟分配的块并返回最终的错误状态。

持久内存与 DAX。非易失性内存 (non-volatile memory, NVM) 可以提供字节寻址或可直接映射的持久存储。Intel Optane PMEM (基于 3D XPoint) 是历史上的典型代表，虽然产品已经停产，DAX、缓存行写回和持久化域等概念仍然重要。延迟会随介质、CPU、NUMA 拓扑和访问模式变化，本章不使用固定纳秒数代表整个类别。

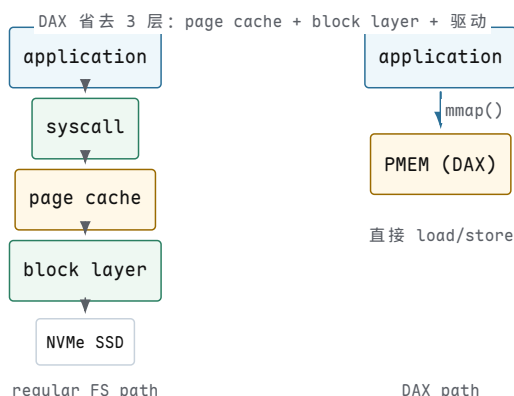
DAX 模式 (Direct Access, `CONFIG_FS_DAX`)：当文件系统以 DAX 模式挂载时 (`mount -o dax`)，`mmap()` 系统调用返回的虚拟地址直接映射到 PMEM 的物理地址。没有页缓存 (page cache) 介入，没有块设备层 (block layer) 介入。CPU 的 `load/store` 指令直接访问持久内存。

```
/* PMEM 持久性写入流程 (PMDK libpmem, simplified) */
void pmem_persist(const void *addr, size_t len)
{
    /* 1. 将脏缓存行写回 PMEM */
    pmem_clwb(addr, len);          /* clwb (Cache Line Write Back) */

    /* 2. 确保所有 store 对其他 CPU 和 PMEM 可见 */
    asm volatile("sfence" ::: "memory"); /* sfence (Store Fence) */

    /* 此后 addr 处的数据已持久化到 PMEM。
       断电也不会丢失。 */
}

/* 对比：传统文件系统写持久化 */
void fs_persist(int fd)
{
    write(fd, buf, len); /* 数据进入页缓存 */
    fsync(fd);           /* 刷页缓存到块设备 → I/O syscall */
    /* 需要 syscall 上下文切换 + 块层 + 驱动 */
}
```



图示：DAX 与传统文件系统路径对比。传统路径需要 5 层（应用 → 系统调用 → 页缓存 → 块层 → 驱动 → SSD），每次 I/O 需要系统调用。DAX 路径仅需 `mmap` 后直接 `load/store` 到 PMEM，零系统调用。

持久性保证：DAX 模式下，CPU `store` 指令将数据写入 L1/L2 缓存，然后通过写缓冲 (write buffer) 异步刷新到 PMEM。如果在刷新完成前断电，数据会丢失。必须显式执行 `clwb` (Cache Line Write Back) 将缓存行写回 PMEM，然后执行 `sfence` (Store Fence) 确保所有之前的 `store` 指令对 PMEM 可见。ext4 和 XFS 在 DAX 模

式下的 `fsync()` 会遍历 inode 的脏页，对每页执行 `clwb + sfence`。

大页支持。页缓存 (page cache) 传统上使用 4 KiB 页。对于大文件（例如 10 GiB 数据库），4 KiB 页意味着 2.5×10^6 个页表项，产生大量 TLB miss。x86-64 的 TLB 容量有限（L1 dTLB 通常 64-72 项 for 4 KiB 页），覆盖范围仅 256-288 KiB。

大页 (large folios / huge pages) 使用 2 MiB 页（甚至 1 GiB 巨页）替代 4 KiB 页。一个 2 MiB 页覆盖相同数据量只需要 1 个 TLB 项—TLB 覆盖范围扩大 512 倍。Linux 6.5+ 内核中，ext4 和 XFS 的页缓存支持透明大页 (Transparent Huge Pages for page cache)：当文件系统检测到顺序访问大文件时，内核可以使用 2 MiB folio 替代 4 KiB 页。

对于顺序读取 10 GiB 文件：

- 4 KiB 页： 2.5×10^6 个页，TLB miss 频繁
- 2 MiB 大页：4883 个页，TLB miss 减少 512 倍
- I/O 层面：2 MiB bio 请求比 4 KiB bio 更高效—块层、驱动和 SSD 都更擅长处理大 I/O 请求

overlayfs 与容器。容器镜像通常由多个层 (layer) 组成：基础 OS 层 + 应用层 + 依赖层。每个层是一个只读的目录树。容器运行时需要一个可写层来记录修改。OverlayFS (`fs/overlayfs/`) 将这些层叠合并为一个统一的目录视图。

- *lowerdir* (下层)：一个或多个只读目录，叠加顺序从下到上
- *upperdir* (上层)：一个可写目录，所有修改写入此层
- *workdir* (工作目录)：overlayfs 内部使用，用于 copy-up 原子操作

挂载命令：

```
mount -t overlay overlay -o lowerdir=/lower1:/lower2,upperdir=/upper,workdir=/work /merged
```

copy-up 机制：当应用程序尝试写入一个位于 lower 层的文件时，overlayfs 不能直接修改只读的 lower 层。它先将文件从 lower 层完整复制到 upper 层，然后在 upper 层的副本上执行写入。lower 层的原始文件保持不变（其他容器可能正在使用它）。

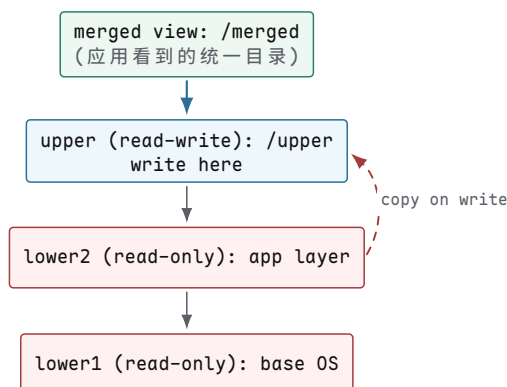
```
=== overlayfs copy-up trace ===
open("/etc/passwd", O_WRONLY)
├─ lookup: /etc/passwd found in lower layer (read-only)
├─ cannot write to lower layer
├─ copy /etc/passwd: lower → upper (full file copy, atomic)
├─ open upper copy for writing
└─ return fd (points to upper copy)

write(fd, "newuser:x:1000:1000:./home/newuser:/bin/bash\n", 47)
├─ writes 47 bytes to upper copy
└─ lower layer copy is unchanged

close(fd)
└─ upper layer now has modified /etc/passwd

=== 后续读取 ===
open("/etc/passwd", O_RDONLY)
```

└ overlayfs checks upper layer first → returns upper copy
(lower layer shadowed by upper copy)



图示：OverlayFS 层叠结构。merged 是应用程序看到的统一视图。upper 是可写层。lower1/lower2 是只读层。copy-up：首次写入 lower 层文件时，文件被复制到 upper 层，后续操作在 upper 副本上进行。

性能注意：copy-up 是同步操作——大文件的首次写入需要先完整复制文件到 upper 层。对于一个 1 GiB 的日志文件，首次 `open(O_WRONLY)` 会触发 1 GiB 的复制 I/O。此外，overlayfs 的目录查找需要在多个层中搜索，增加了 `lookup()` 的开销。

13.14 设计选择与对比

不同的文件系统做出了不同的设计选择，反映了对工作负载、兼容性和故障模型的取舍。功能开关、容量上限和算法会随版本、格式参数和操作系统集成变化，因此这里比较稳定的设计轴，不把易过期的“yes/no”和最大容量数字伪装成永久事实。

表 13.34：主流文件系统的稳定设计取舍

系统	更新与索引思路	主要优势	边界与代价
ext4	extent 映射, jbd2 日志保护元数据更新	成熟、兼容性广，适合通用 Linux 工作负载	不原生提供 Btrfs/ZFS 式子卷快照和透明压缩
XFS	allocation group、B+ 树和元数据日志	大文件、并行分配和高吞吐扩展性强	管理工具与恢复模型不同于 ext 系列；reflink 不等同于快照产品
Btrfs	数据和元数据采用 COW B 树	校验和、子卷、快照、压缩和 send/receive 集成	重写型负载可能碎片化；defrag 会破坏 extent sharing 并增加空间占用
ZFS	COW 树、事务组、ZIL 与存储池	端到端校验、快照、压缩和冗余修复能力强	内存、运维模型和平台集成成本较高
F2FS	日志结构写入，NAT/SIT、checkpoint 与 GC	面向闪存，支持冷热数据分离和自适应分配	空间压力下前台 GC 会形成尾延迟；不提供原生通用快照
NTFS	MFT、run list 与 \$LogFile 元数据日志	与 Windows 权限、加密和管理生态紧密集成	VSS 是上层卷影复制机制，不能简单等同于文件系统原生快照
APFS	COW 元数据、空间共享、clone 与 snapshot	面向 Apple 平台，集成加密、快照和快速克隆	实现与可用特性受 Apple 系统版本约束，跨平台可观测性有限

设计哲学。ext4：兼容演进是重要约束。ext4 继承 ext2/ext3 的磁盘格式家族，并通过 compatible/incompatible feature flags 区分旧实现能否安全挂载；新建文件系统也可以选择更大的 inode 和新特性。ext4 通过区段树 (extent tree) 扩展块映射，通过 jbd2 日志提供崩溃一致性，通过 inline_data 和元数据校验和增量地添

加现代特性。这种保守的演进路径使 ext4 成为 Linux 最广泛使用的文件系统，但也限制了它支持快照和透明压缩等需要根本性结构改变的功能。

XFS: 源自 SGI IRIX 系统，设计之初就面向大文件吞吐量。XFS 使用并行分配组 (allocation groups, AG)，每个 AG 有自己的 inode 表和空闲空间 B+ 树，允许多 CPU 并行分配而无需全局锁。XFS 的 B+ 树框架 (xfs_btree) 是一个通用实现——空闲空间、inode 分配、区段映射和大目录都使用同一框架。XFS 不支持快照和压缩，但其对大文件顺序 I/O 的优化和并行性使其在高性能计算和数据库工作负载中表现优异。

Btrfs: Oracle 赞助开发，目标是成为 Linux 的“下一代文件系统”。Btrfs 的核心是 COW B 树——元数据组织在多棵 B 树中，文件数据由 extent item 引用，更新通过 COW 发布新树路径。这使得创建快照通常很快，因为可以共享既有 extent。Btrfs 支持透明压缩、子卷 (subvolume)、快照、发送/接收 (send/receive) 等现代特性。但 COW 的代价是碎片化——频繁修改的文件会被分散到不同位置。Btrfs 支持在已挂载文件系统上整理文件；代价是整理会重写 extent，并可能打破 reflink 或快照之间的 extent sharing，显著增加空间占用。

ZFS: Sun Microsystems (后被 Oracle 收购) 工程团队设计，将数据完整性置于一切之上。ZFS 的设计原则是“永不静默损坏”：每个块 (数据和元数据) 都有校验和，每次读取都验证。ZFS 的 COW + intent log (ZIL) 提供了强一致性保证。ZFS 的存储池 (zpool) 模型将物理设备与文件系统解耦——多个文件系统可以共享一个池的存储空间。ZFS 支持透明压缩、去重、快照、克隆和纠错 (RAID-Z)。代价是复杂性和内存消耗 (ARC 缓存通常占用大量 RAM)。

F2FS: Samsung 开发，专门为 NAND 闪存优化，主要目标是 Android 设备。F2FS 的设计围绕闪存的物理特性：日志结构写入 (log-structured write) 将随机写转换为顺序写，减少闪存擦除次数；冷热数据分离 (warm/cold separation) 将不同生命周期的数据写入不同区域，减少 GC 开销。后台 GC 可以在 I/O 空闲时清理；空间压力触发的前台 GC 则可能直接增加应用延迟。F2FS 不提供原生通用快照和去重，其目标是在闪存上平衡写放大、吞吐和尾延迟。

NTFS: Microsoft 为 Windows NT 设计，使用 \$LogFile 日志保证元数据一致性。小文件数据可以作为 resident attribute 保存在 MFT 记录中，大文件则用 run list 描述簇范围。NTFS 支持每文件压缩和 EFS 加密；Windows 的 VSS 与服务器数据去重属于更大的存储栈，讨论时不能直接当成 NTFS 磁盘格式本身的能力。

APFS: Apple 为 macOS/iOS 设计，2017 年推出。APFS 针对闪存优化——所有写入都是 COW，原生支持 TRIM，并为 SSD 优化了空间分配。APFS 的快照可被 Time Machine 等上层功能使用。APFS 原生支持加密、snapshot 和 clone；clone 利用共享 extent 避免创建时复制全部数据，但后续写入仍会分配新空间，不能把它理解成永远零成本。

核心思想

没有“最好的”文件系统，只有不同约束下的权衡。ext4 追求稳定兼容，XFS 追求大文件吞吐，Btrfs 追求功能丰富，ZFS 追求数据完整性，F2FS 追求闪存友好，NTFS 追求 Windows 生态，APFS 追求 Apple 生态。选择取决于工作负载、硬件平台和可靠性要求。

13.15 测试

文件系统的正确性和性能不能仅靠代码审查保证。一个文件系统必须在各种异常条件下经过系统性测试——包括崩溃恢复、并发竞争、空间碎片和闪存磨损。以下测试分类和具体场景构成了一个文件系统验证的最小集合。

测试分类。

Correctness（正确性）

- 基本 API 正确性：`create`、`unlink`、`rename`、`link`、`fsync`、`fdatasync` 在所有文件类型上的行为
- 边界条件：空文件、最大文件、路径长度上限、文件名特殊字符
- 崩溃恢复：在每个操作序列的每个中间状态下断电，验证恢复后状态

Performance（性能）

- 顺序读/写吞吐量：1 MiB 块大小，测量 MB/s 和 IOPS
- 随机读/写吞吐量：4 KiB 块大小，队列深度 1/16/64
- 不同块大小（512 B – 1 MiB）下的吞吐量曲线
- 元数据操作延迟：`stat`、`open`、`readdir` 的 p99 延迟

Scalability（可扩展性）

- 单目录 1M 文件：`readdir`、`lookup`、`unlink` 性能
- 10 TiB 单文件：顺序读写性能不随偏移量退化
- 100 万并发文件描述符：内核内存消耗和调度延迟

Crash Recovery（崩溃恢复）

- 故障注入：在每个操作（`create`、`write`、`fsync`、`rename`、`unlink`）的每个步骤后断电
- 日志恢复：验证 `jbd2/ZIL/checkpoint` 在各种中断场景下能正确恢复
- 校验和验证：注入位翻转后，文件系统是否能检测并报告损坏

Concurrency（并发）

- 100 个进程在同一目录中并发创建文件—无数据竞争，无死锁
- 并发 `rename` 到同一目标路径—最终状态一致
- 并发 `fsync` 与 `write`—崩溃后数据状态可预测

Space Efficiency（空间效率）

- 1M 次 `create` + `delete` 循环后测量碎片化程度
- 填充至 99% 后随机 `create/delete`—验证分配器在满盘时的退化
- 空闲空间报告准确性：`statfs()` 报告的空闲块数与实际一致

Flash Optimization (闪存优化)

- 写放大测量：写入 100 GiB 数据，测量 SSD 实际闪存写入量
- TRIM 有效性：删除大量文件后验证 SSD 回收了擦除块
- 对齐：验证文件系统块大小与 SSD 页大小对齐

具体测试场景。以下场景覆盖了最常见的文件系统故障模式：

1. **崩溃在同步之前：**创建并写文件 → 在任意步骤崩溃 → 恢复。未建立持久化承诺时，文件可能不存在，也可能以某个已提交状态存在；测试应验证文件系统结构合法、旧状态仍可解释，并且不会暴露越界块、悬空目录项或未初始化数据，而不是强行要求“文件不应存在”。
2. **新文件的持久发布：**创建并写临时文件 → `fsync(file)` → `rename` 到目标名 → `fsync(parent_dir)` → 崩溃 → 恢复 → 文件名和正确数据都必须存在。只同步文件不能单独保证包含它的目录项已经持久化。
3. **每 4K fsync 的吞吐量：**顺序写入 1 MiB，每 4 KiB 后调用 `fsync` → 测量吞吐量和 p50/p95/p99 延迟。这是数据库 WAL 的典型模式。报告必须记录设备型号、缓存与断电保护、文件系统版本、挂载参数、队列深度和测试工具；没有这些条件，不给固定“预期 IOPS”。
4. **并发创建：**100 个进程各在同一目录中创建 1000 个文件 → 无文件丢失、无数据竞争、无死锁。验证目录锁的正确性。
5. **满盘碎片化：**填充至 99% → 随机 create/delete 10000 次 → 测量碎片化（最大连续空闲区段 vs 总空闲空间）。验证分配器不会退化到无法分配连续区段。
6. **SSD 写放大：**在 SSD 上写入 100 GiB → 读取 SSD SMART `Media_Wearout_Indicator` 或厂商提供的 NAND 写入计数 → 计算实际闪存写入量与逻辑写入量之比。不同 SSD 对 SMART 字段的定义不同，必须先验证计数单位，并把文件系统写放大与设备 FTL 写放大分开报告。
7. **快照一致性：**创建快照 → 修改 10% 的数据 → 验证快照仍然保存原始数据 → 删除快照 → 验证空间被正确回收。
8. **累积崩溃：**在持续写入工作负载下，执行 1000 次随机断电 → 每次恢复后验证文件系统一致性 → 无累积损坏。验证日志恢复是幂等的。

13.16 源码级补充

以下补充材料从 Linux 内核源码的角度深入文件系统的核心实现路径。涉及的源文件路径基于 Linux 6.x 内核。

Linux VFS 核心路径

路径查找 (path lookup) 是文件系统最频繁的操作——每次 `open()`、`stat()`、`read()` 都需要将路径名解析为 inode。Linux VFS 在 `fs/namei.c` 中实现了这一路径，核心函数是 `path_lookupat`。

路径查找有两条路径：`RCU walk` (无锁快速路径) 和 `ref walk` (引用计数慢速路径)。RCU walk 使用 Read-Copy-Update 机制在无锁状态下遍历 dentry 和 inode——这是 `stat` 和 `open` 热路径上的常见情况 (缓存命中)。当遇到复杂情况 (跨挂载点、符号链接、dentry 缺失、竞争检测到修改) 时，RCU walk 回退到 `ref walk`，后者使用 `d_lock` 和 `i_rwsem` 保证安全。

```

/* fs/namei.c - path lookup (simplified) */
static int path_lookupat(struct nameidata *nd,
                        unsigned flags, struct path *path)
{
    /* Phase 1: try RCU walk (lockless, fast path) */
    rcu_read_lock();
    nd->flags |= LOOKUP_RCU;
    err = link_path_walk(nd);          /* walk path components */
    if (!err && can_lookup_fast(nd))
        err = walk_component(nd);      /* try dcache hit */
    rcu_read_unlock();

    /* Phase 2: fallback to ref walk on contention */
    if (err == -ECHILD || err == -ESTALE) {
        err = link_path_walk(nd);      /* ref walk: d_lock + i_rwsem */
    }
    return err;
}

/* walk_component: fast path vs slow path */
static int walk_component(struct nameidata *nd)
{
    /* lookup_fast: dcache hit (RCU-protected) */
    dentry = lookup_fast(nd);          /* d_lookup in dcache */
    if (dentry)
        return step_into(nd, dentry); /* consume one component */

    /* lookup_slow: dcache miss → call filesystem */
    dentry = lookup_slow(nd);          /* inode->i_op->lookup() */
    return step_into(nd, dentry);
}

```

RCU walk 的性能优势在于完全避免了原子操作和缓存行争用。在高度并行的场景下（例如 Web 服务器处理数千并发请求），RCU walk 能显著减少共享锁和引用计数争用。不过路径遍历仍与路径分量数量相关，并没有从算法上的 $O(n)$ 变成 $O(1)$ ；改善的是并发扩展性和常数开销。

ext4 extent 查找。

ext4 的区段树（extent tree）查找由 `ext4_ext_map_blocks` 实现（`fs/ext4/extents.c`）。给定一个逻辑块号，该函数在 B+ 树中查找对应的物理块号。

```

/* fs/ext4/extents.c - extent lookup (simplified) */
int ext4_ext_map_blocks(handle_t *handle, struct inode *inode,
                        struct ext4_map_blocks *map, int flags)
{
    struct ext4_extent_header *eh;
    struct ext4_extent *ex;
    ext4_lblk_t block = map->m_lblk; /* logical block number */

    /* Start from inline header in inode */
    eh = ext_inode_hdr(inode); /* i_block[0] as extent header */

    /* Traverse B+ tree: descend from root to leaf */
    while (eh->eh_depth > 0) {
        /* Index node: find correct child pointer */
        struct ext4_extent_idx *idx;
        idx = ext4_ext_find_index(eh, block);
        bh = sb_bread(sb, ext4_idx_pblock(idx)); /* read child */
        eh = ext_block_hdr(bh);
    }
}

```

```

}

/* Leaf level: search for extent covering 'block' */
ex = ext4_ext_search_right(eh, block);
if (ex == NULL) {
    /* No extent covers this range → it's a hole */
    map->m_flags |= EXT4_MAP_HOLE;
    map->m_pblk = 0; /* zero-filled page on read */
    return 0;
}

/* Found: compute physical block */
map->m_pblk = ext4_ext_pblock(ex) + (block - ex->ee_block);
map->m_len = ext4_ext_get_actual_len(ex)
            - (block - ex->ee_block);
return map->m_len;
}

```

空洞 (*hole*) 处理：如果逻辑块号不在任何 extent 的覆盖范围内，`ext4_ext_map_blocks` 返回 `EXT4_MAP_HOLE` 标志。VFS 层据此返回一个全零页—读取空洞区域得到零字节，不需要分配物理块。

jbd2 提交。

ext4 的日志由 jbd2 (Journaling Block Device 2) 驱动，核心提交函数是 `jbd2_journal_commit_transaction (fs/jbd2/commit.c)`。提交过程将内存中的事务（一组已记录的元数据修改）写入磁盘上的日志区域。

```

/* fs/jbd2/commit.c - transaction commit (simplified) */
int jbd2_journal_commit_transaction(journal_t *journal)
{
    transaction_t *commit_txn = journal->j_running_transaction;

    /* Phase 1: write all descriptors and data buffers */
    /* For each journal block: write to log area */
    err = journal_write_data_buffers(commit_txn);

    /* Phase 2: write commit block */
    commit_blk = jbd2_journal_get_descriptor(journal);
    commit_blk->h_magic = JBD2_MAGIC_NUMBER;
    commit_blk->h_blocktype = JBD2_COMMIT_BLOCK;

    /* CRC32C checksum of all blocks in this transaction */
    commit_blk->h_chksum_type = JBD2_CRC32C;
    commit_blk->h_chksum[0] = crc32c_journal(commit_txn);

    /* Write commit block → this is the atomic commit point */
    submit_bio(WRITE, commit_bio);
    wait_for_completion(); /* Durable! */

    /* Phase 3: checkpoint (fs/jbd2/checkpoint.c) */
    /* Move committed data from log to final locations,
       then free journal space for reuse */
    jbd2_journal_checkpoint_transaction(journal);
}

```

提交块 (commit block) 是事务的原子提交点—只有在提交块成功写入后，事务才算“已提交”。如果在写入提交块之前崩溃，恢复时该事务被视为未提交，所有相关修改回滚。提交块中的 CRC32C 校验和用于在恢复时验证提交块本身没有损坏。

XFS B+ 树框架。

XFS 的所有核心数据结构都使用一个通用的 B+ 树框架 (`fs/xfs/libxfs/xfs_btree.c`)。这个框架提供了查找、插入和删除的通用实现，具体的数据结构通过函数指针定制比较和键值函数。

```
/* fs/xfs/libxfs/xfs_btree.c - generic B+ tree (simplified) */
int xfs_btree_lookup(struct xfs_btree_cur *cur,
                    xfs_lookup_t dir, int *stat)
{
    /* Traverse from root to leaf, comparing keys */
    for (level = cur->bc_nlevels - 1; level >= 0; level--) {
        /* Binary search within node */
        xfs_btree_binary_search(cur, level, &key, &ptr);
        if (ptr > xfs_btree_get_numrecs(block))
            ptr = xfs_btree_get_numrecs(block);
        if (level > 0)
            block = xfs_btree_read_block(cur, ptr); /* descend */
    }
    *stat = (found_exact_match) ? 1 : 0;
    return 0;
}

/* Users of the generic framework:
 * - xfs_btree_cur for free space by block number (AGFL)
 * - xfs_btree_cur for free space by block count (AGFL)
 * - xfs_btree_cur for inode allocation (AGI)
 * - xfs_btree_cur for extent map (BMBT)
 * - xfs_btree_cur for directory (large dirs) (BMDA)
 */
```

所有 XFS 的 B+ 树（空闲空间索引、inode 分配、区段映射、大目录）都通过 `struct xfs_btree_cur` 配置不同的比较函数和键值类型，共享同一套遍历、插入和删除代码。这种设计避免了代码重复，也保证了所有 B+ 树操作的一致性。

Btrfs 事务提交。

Btrfs 的事务提交 (`btrfs_commit_transaction`, `fs/btrfs/transaction.c`) 通过 COW B 树和原子根指针更新实现崩溃一致性。与 ext4 的日志不同，Btrfs 不需要单独的日志区域—B 树的 COW 特性本身就是日志。

```
/* fs/btrfs/transaction.c - commit (simplified) */
int btrfs_commit_transaction(struct btrfs_trans_handle *trans)
{
    struct btrfs_transaction *cur = trans->transaction;

    /* Phase 1: COW all modified B-tree nodes */
    /* Each modified node is written to a NEW disk location.
     * Old locations remain intact until superblock is updated. */
    btrfs_write_dirty_block_groups(trans);
    btrfs_write_dirty_roots(trans); /* COW B-tree nodes */

    /* Phase 2: write superblock with new root pointers */
    /* Btrfs has multiple superblock slots (usually 4).
     * The new root pointer is written atomically. */
    btrfs_write_super(trans);

    /* Phase 3: mark old data as reclaimable */
    /* Old B-tree nodes are no longer referenced.
     * Future writes can reuse those disk blocks. */
    btrfs_run_delayed_refs(trans);
}
```

```

/* Two ways to join a transaction:
 * btrfs_join_transaction: read-only join, does NOT force commit
 * btrfs_start_transaction: write join, MAY force commit if
 *                           transaction is large or old
 */

```

Btrfs 的超级块有多个槽位（通常 4 个），分布在不同位置。提交时，新根指针写入下一个槽位——这是一个原子操作（单个扇区写入是原子的）。如果在写入新超级块之前崩溃，旧超级块仍然指向旧根，文件系统状态回滚到上次提交点。Btrfs 的 COW 模型意味着旧数据永远不会被覆盖——提交本质上是切换根指针的“地址”，而非修改已有数据。

page cache 与写回。

Linux 的脏页管理由 `mm/page-writeback.c` 和 `fs/fs-writeback.c` 共同实现。前者管理全局脏页比例和写回节流，后者实现写回线程的主循环和每-inode 写回逻辑。

```

/* mm/page-writeback.c - global dirty throttling (simplified) */
/* vm_dirty_ratio (default 20%): max dirty pages as % of memory */
/* vm_dirty_background_ratio (default 10%): when writeback starts */

void global_dirty_limits(unsigned long *pbackground,
                        unsigned long *pdirty)
{
    unsigned long total = global_page_state(NR_FREE_PAGES)
                        + global_reclaimable_pages();
    *pbackground = total * vm_dirty_background_ratio / 100;
    *pdirty       = total * vm_dirty_ratio / 100;
    /* When dirty pages > pdirty: new write() blocks
       When dirty pages > pbackground: writeback thread wakes */
}

/* fs/fs-writeback.c - writeback thread (simplified) */
void wb_workfn(struct work_struct *work)
{
    while (!list_empty(&bdi->work_list)) {
        /* Pop next inode to write back */
        inode = wb_writeback_single(bdi, &wbc);
        __writeback_single_inode(inode, &wbc);
        /* Calls inode->i_op->writepages() → ext4_writepages()
           which calls ext4_mballocc to allocate blocks
           for all dirty pages at once (delayed alloc!) */
    }
}

```

`__writeback_single_inode` 调用文件系统的 `writepages` 方法（例如 `ext4_writepages`），后者负责将脏页写入磁盘。对于延迟分配的文件系统，`writepages` 是实际物理块分配发生的时刻——多块分配器 `ext4_mballocc` 在此处一次性为所有脏页分配连续物理块。

dcache RCU 遍历。

目录项缓存（`dcache`，`fs/dcache.c`）缓存了路径名到 `inode` 的映射。`dcache` 的查找使用 RCU 实现无锁快速路径。

```

/* fs/dcache.c - dcache lookup (simplified) */
struct dentry *d_lookup(const struct dentry *parent,
                      const struct qstr *name)
{
    /* RCU fast path: lockless dcache lookup */
}

```



```

rcu_read_lock();
hlist = rcu_dereference(
    parent->d_flags & DCACHE_OP_COMPARE
    ? d_compare_rcu : d_hash_rcu);
hlist_for_each_entry_rcu(dentry, hlist, d_hash) {
    if (dentry->d_parent != parent) continue;
    if (qstr_casecmp(&dentry->d_name, name)) continue;
    if (d_unhashed(dentry)) continue; /* being removed */
    /* Hit! But must verify under RCU */
    if (lockref_get_not_dead(&dentry->d_lockref)) {
        rcu_read_unlock();
        return dentry; /* cache hit */
    }
}
rcu_read_unlock();
return NULL; /* cache miss → call filesystem lookup */
}

```

RCU walk 路径在 `fs/namei.c` 中通过 `link_path_walk` → `walk_component` → `lookup_fast` → `d_lookup` 链接起来。整个路径在 `rcu_read_lock()` 保护下执行，不持有任何自旋锁。当 `d_lookup` 返回 `NULL`（缓存缺失）或检测到 `dentry` 正在被删除时，VFS 回退到 `lookup_slow`，后者获取 `i_rwsem` 信号量并调用文件系统的 `i_op->lookup` 方法。

应用层一致性不能外包给文件系统。

文件系统的日志保证的是 文件系统自身元数据的一致性——inode 表、目录项、空闲块位图不会因为崩溃而损坏。但文件系统 不保证应用层面的多文件事务一致性。如果一个应用需要原子地更新两个文件（例如：写入数据文件 A 和更新指向 A 的索引文件 B），文件系统的日志无法帮助——每个文件系统操作（`write`、`fsync`）是独立的，崩溃可能发生在两者之间。

应用必须自行实现原子性。清单发布协议（manifest publish protocol）是一种常用模式，利用 `rename` 的原子性作为提交点：

```

/* Manifest publish protocol - atomic multi-file update */

/* Step 1: Write data object to a unique, versioned path */
int fd = open("objects/v42/data", O_WRONLY|O_CREAT|O_EXCL, 0644);
write_all(fd, data, data_len);
fsync(fd); /* persist data to disk */
close(fd);

/* Step 2: Write manifest referencing the object */
fd = open("manifests/v42", O_WRONLY|O_CREAT|O_EXCL, 0644);
write_all(fd, manifest_content); /* includes checksums, metadata */
fsync(fd); /* persist manifest */
close(fd);

/* Step 3: Atomic publish via rename */
rename("manifests/v42", "CURRENT");
/* rename is atomic: either readers see old CURRENT or new CURRENT,
   never an intermediate state */

/* Step 4: Persist the directory entry change */
int dirfd = open(".", O_RDONLY|O_DIRECTORY);
fsync(dirfd); /* ensure rename is durable */
close(dirfd);

/* Readers: always read CURRENT, which points to a consistent
   manifest+data pair. No partial state is ever visible. */

```

核心理念

不同层次的一致性由不同的日志机制保护，各司其职，不可混淆：

1. 文件系统日志 (jbd2/ZIL/Btrfs COW) 保护文件系统结构的一致性——inode、目录项、位图不会损坏。
2. 数据库 WAL (Write-Ahead Log) 保护数据库事务的一致性——事务要么全部提交，要么全部回滚。
3. 应用清单 (manifest + atomic rename) 保护应用版本的一致性——读者要么看到旧版本，要么看到新版本，不会看到中间状态。

文件系统的崩溃一致性保证止步于元数据。应用程序的跨文件原子性是应用自身的责任——不能外包给文件系统。

第 14 章 socket、网络入口与内核边界

14.1 原理

网络系统会在第三册展开，本章只讲操作系统必须提供的网络入口：socket 为什么是 fd，连接和监听如何变成内核对象，阻塞和非阻塞 I/O 如何与调度器协作，poll/epoll 为什么不是“循环 read”的语法糖。把 socket 放在文件描述符模型里，是 Unix 设计的强大统一：进程用 fd 表引用网络端点，权限、关闭、继承、阻塞、异步通知和事件等待都能复用已有 OS 机制。

socket 与普通文件的差异在于它不是稳定字节数组，而是协议状态机和队列集合。TCP socket 至少包含发送缓冲、接收缓冲、连接状态、重传计时器、拥塞控制状态和等待队列。UDP socket 没有连接字节流，但有端口绑定、报文边界和接收队列。应用看到的是 send/recv，内核维护的是协议栈状态、网卡队列和计时器。

核心思想

socket 是 fd 表中的一个内核对象，但它的可读可写由协议状态、缓冲区、水位线和错误队列共同决定。

网络入口必须先把 OS 契约讲清楚，再进入协议细节。阻塞 recv 在没有数据时会让线程睡在 socket 等待队列；非阻塞 recv 返回 EAGAIN；poll 把“是否可读可写”注册为等待条件；epoll 用就绪队列降低大量 fd 的重复扫描成本。这些都是调度、等待队列和 fd 生命周期的延伸。

14.2 实践

最小 socket 子系统可以包含：

对象	作用	不变量
socket fd	进程引用网络端点的句柄	fd 引用有效时 socket refcount 大于零
socket object	保存协议族、类型、状态、ops	状态决定允许的系统调用
bind table	本地地址端口到 socket 的映射	同一四元组不能被非法重复绑定
listen queue	半连接和已完成连接队列	backlog 限制必须生效
send buffer	用户写入但未发送或未确认的数据	字节数受 SO_SNDBUF 限制
receive buffer	已到达但未被用户读取的数据	可读事件与缓冲水位一致

典型 TCP 服务端路径：

```
s = socket(AF_INET, SOCK_STREAM)
bind(s, 0.0.0.0:8080)
listen(s, backlog=128)
while true:
    c = accept(s)
    handle_client(c)
```

内核对应状态是：socket 创建未绑定对象；bind 加入端口表；listen 把对象转成监听状态并初始化队列；三次握手完成后，新连接进入 accept queue；accept 取出一个已完成连接，创建新的 fd 指向新的 connected socket。监听 socket 和

连接 socket 不是同一个对象。

14.3 算法与实现分析

bind 冲突检查。端口绑定要考虑地址、端口、协议、reuse 标志和 namespace。教学模型可以先实现严格唯一：

```
bind(sock, local_addr):
    lock(bind_table.lock)
    if bind_table.contains(sock.netns, sock.proto, local_addr):
        unlock(bind_table.lock)
        return EADDRINUSE
    sock.local = local_addr
    sock.state = BOUND
    bind_table.insert(sock)
    unlock(bind_table.lock)
    return OK
```

真实系统的 `SO_REUSEADDR` 和 `SO_REUSEPORT` 会放宽规则，但不能简化成“允许重复”。复用规则还要区分监听 socket、已连接四元组、`TIME_WAIT` 和负载均衡策略。先掌握严格唯一模型，再扩展复用规则，能避免把安全边界讲错。

listen 与 accept 队列。监听 socket 有两个不同队列：握手未完成的半连接队列，以及握手完成等待用户 accept 的队列。简化模型可以只保留完成队列：

```
tcp_connection_established(listener, child):
    lock(listener.lock)
    if listener.acceptq.len == listener.backlog:
        drop_or_reset(child)
    else:
        enqueue(listener.acceptq, child)
        wake_waiters(listener.read_waitq)
    unlock(listener.lock)

accept(listener):
    lock(listener.lock)
    while empty(listener.acceptq):
        if listener.nonblock:
            unlock(listener.lock)
            return EAGAIN
        sleep_on(listener.read_waitq, listener.lock)
    child = dequeue(listener.acceptq)
    fd = install_fd(child)
    unlock(listener.lock)
    return fd
```

不变量是：accept queue 中的 child socket 已经是 connected 状态；队列非空时监听 socket 对 `poll` 表现为可读；关闭监听 socket 必须唤醒等待 `accept` 的线程并让它们返回错误。

send/recv 与背压。发送路径不是把用户数据直接交给网卡。数据先进入 socket send buffer，再由协议栈分段、排队、重传和确认。

```
send(sock, user_buf, len):
    copied = 0
    lock(sock.lock)
    while copied < len:
        space = sock.sndbuf_limit - sock.sndbuf_used
        if space == 0:
            if sock.nonblock:
```

```

        break_or_EAGAIN(copied)
        sleep_on(sock.write_waitq, sock.lock)
        continue
    n = min(space, len - copied)
    copy_from_user_into_skb(sock, user_buf + copied, n)
    copied += n
    tcp_push_pending(sock)
    unlock(sock.lock)
    return copied

```

这段伪代码解释短写：阻塞 socket 通常尽量等待空间，非阻塞 socket 在缓冲区满时可能只写入部分数据或返回 `EAGAIN`。接收路径类似：接收缓冲为空时阻塞或返回 `EAGAIN`；对 TCP 返回字节流，对 UDP 每次读取保留报文边界语义。

背压要从两端同时看。发送方应用调用 `send(sock, buf, 1MiB)`，内核不可能把 1MiB 立即塞进网卡。它先受本机 socket send buffer 限制；再受 TCP 拥塞窗口和接收方通告窗口限制；再受 qdisc、驱动队列和网卡 DMA ring 限制。任一层满了，`send` 都可能停住、短写或返回 `EAGAIN`。

队列或窗口	它限制什么	满了以后会怎样
socket sndbuf	应用已交给内核但尚未确认释放的字节	阻塞 <code>send</code> 、短写或 <code>EAGAIN</code>
TCP send queue	已分段、待发送、待重传的数据	受拥塞控制和对端窗口约束
qdisc 队列	本机网络调度层待发送 packet	排队延迟增加，可能丢包或限速
驱动 TX ring	网卡可消费的 DMA 描述符槽位	停止网络队列，等 completion 回收槽位
对端 receive window	接收方还能缓存多少未读数据	发送方必须减速，甚至进入零窗口等待

一个常见现象是：程序以为 `send` 返回成功就表示对方收到了消息。实际并不是。返回成功通常只说明数据已经被本机内核接收进入发送路径，后续还可能因为连接 reset、重传失败、超时或本机崩溃而没有到达对端应用。若业务需要“对方已经处理”，必须在应用协议里设计确认，而不是把 socket 写入成功当业务提交点。

接收方向也一样。网卡收到包后，驱动把 skb 交给协议栈，TCP 按序重组后放进 socket receive queue。应用不读，receive queue 会涨满；涨满后接收窗口变小，发送方会被迫降速。若应用只看“网络带宽不够”，就会误诊。真正问题可能是本进程处理慢、单线程事件循环被阻塞、receive buffer 太小，或者应用协议把消息边界处理错。

poll 与 epoll。朴素 poll 每次扫描 fd 数组，复杂度 $O(n)$ ：

```

poll(fds, timeout):
    ready = []
    for fd in fds:
        mask = fd.file.ops.poll(fd.file)
        if mask != 0:
            ready.append((fd, mask))
    if ready not empty:
        return ready
    register_waiters(fds, current)
    schedule_timeout(timeout)
    retry scan

```

`epoll` 把关注集合保存在内核中。当 socket 状态变化时，回调把对应 epitem 放入 ready list，用户调用 `epoll_wait` 主要消费 ready list。这样大量空闲连

接不会每次都被全量扫描。

```
socket_state_change(sock):
    for epitem in sock.poll_subscribers:
        if sock.poll_mask & epitem.interest:
            enqueue_once(epitem.epoll.ready_list, epitem)
            wake_waiters(epitem.epoll.waitq)
```

边沿触发和水平触发的区别也来自 ready 条件。水平触发下，只要缓冲区仍有数据，下一次等待还会报告；边沿触发只在状态从不可读变可读时报告，用户必须一直读到 `EAGAIN`，否则可能再也收不到事件。

用一个具体 bug 看边沿触发为什么容易错。socket `S` 原来不可读，随后内核收到 100 字节，状态从不可读变为可读，`epoll` edge-triggered 把 `S` 放入 ready list。应用从 `epoll_wait` 取到事件，只调用一次 `recv(S, buf, 20)`，读走 20 字节后就回到等待。此时 socket 里还剩 80 字节，但状态没有从“不可读”再次变成“可读”，所以边沿触发可能不会再报告。应用就把 80 字节卡在内核 receive queue 里。

正确写法是：收到边沿事件后，对非阻塞 fd 循环读取，直到 `recv` 返回 `EAGAIN`。这个 `EAGAIN` 不是错误，而是证明“当前 ready 条件已经被消费干净”。写方向也类似，收到可写事件后应尽量写到 `EAGAIN` 或应用输出队列为空。

```
on_epollin_edge(fd):
    while true:
        n = recv(fd, buf, sizeof(buf), NONBLOCK)
        if n > 0:
            append_to_protocol_parser(buf, n)
            continue
        if n == 0:
            close_connection(fd)
            break
        if errno == EAGAIN:
            break
        if errno == EINTR:
            continue
        handle_error(fd, errno)
        break
```

`epoll` 还要求应用区分“事件身份”和“fd 数字”。线程 A 把 `fd=7` 加入 `epoll`，线程 B 关闭 `fd=7`，线程 C 立即打开新连接又得到 `fd=7`。旧 `epoll` item 在内核里绑定的是旧 file，不是新连接；但事件返回给用户时可能仍带着当初保存的数字 7。健壮事件循环通常把 `data.ptr` 指向连接对象，并给对象加 generation 或关闭标记，避免用裸 fd 数字误操作新连接。

TCP 状态边界。操作系统教材不必在本册完整推导 TCP，但要能解释 socket 系统调用面对的状态：

状态	用户可见行为	内核重点
LISTEN	可 accept，不能普通 recv 数据	accept queue、backlog、SYN 处理
SYN-SENT	connect 进行中	超时、错误返回、非阻塞完成事件
ESTABLISHED 半关闭	可 send/recv FIN-WAIT 或 CLOSE-WAIT	缓冲、重传、拥塞控制、窗口 EOF、shutdown、剩余数据读取
TIME-WAIT	fd 通常已关但四元组保留	防旧包污染新连接

14.4 测试

1. fd 生命周期：dup socket 后关闭一个 fd，连接对象不能提前释放。
2. bind 冲突：同一地址端口重复绑定应失败，释放后可再次绑定。
3. listen backlog：超过 backlog 的完成连接必须被拒绝或延迟，不能无限增长。
4. accept 阻塞：无连接时阻塞，有连接到达后唤醒；非阻塞返回 EAGAIN。
5. send 背压：发送缓冲满时阻塞或短写，空间释放后唤醒写等待者。
6. recv EOF：对端关闭后，已缓冲数据读完再返回 EOF。
7. poll 水平触发：缓冲仍有数据时重复报告可读。
8. epoll 边沿触发：未读尽数据时不会凭空产生新边沿事件。

本章合格标准：能说明 socket 为什么是 fd；能画出 bind/listen/accept/connect/send/recv 的对象变化；能解释阻塞、非阻塞、poll、epoll 的等待语义。

本章压缩进来的网络可靠性边界。网络补充材料可以继续展开路由、Netfilter、NAPI、skb、拥塞控制和持久化服务协议。主线里先守住一个边界：TCP 的可靠字节流不等于业务请求可靠提交。操作系统能告诉你连接状态、字节是否进入本机内核、对端是否关闭、socket 错误是什么；它不能替应用判断“远端服务是否已经执行这笔订单、写入这条记录或只执行了一半”。

现象	OS/协议层含义	应用层仍要处理什么
send 返回正数	字节被本机发送路径接收	对端应用未必收到或处理
recv 返回 0	对端有序关闭且缓冲已读完	未完成请求可能成功也可能失败
ECONNRESET 超时	连接被复位，流状态不再可信 等待期限内未得到事件	当前业务操作结果未知 远端可能稍后处理或已处理但响应丢失
重连成功	新 TCP 流建立	旧流上的请求语义不能自动继承

可靠服务要在 TCP 之上建立自己的提交证据。常见方法包括请求 ID、幂等键、服务端去重表、事务日志、单调版本、应用级 ACK 和恢复查询。客户端超时后重试同一个请求 ID，服务端若发现已经提交，就返回旧结果；若未提交，才执行并记录。这样网络断开、进程崩溃、重复发送和响应丢失都能收敛到同一条业务语义，而不是把 socket 错误码当作业务真相。

```
server_handle(req):
    if committed_log.contains(req.id):
        return committed_log.result(req.id)
    result = execute_transaction(req.payload)
    committed_log.append(req.id, result)
    fsync(committed_log)
    return result
```

这个模型也解释了为什么网络服务常和文件系统持久化绑在一起。若服务端先回 ACK 再写日志，崩溃后客户端以为成功而服务端忘记结果；若先写日志再回 ACK，客户端即使没收到响应，也能用同一个请求 ID 查询或重试。OS 网络栈负责传字节和报告连接状态，业务一致性由应用协议和存储提交共同证明。

14.5 源码级补充：sk_buff、协议栈入口、NAPI 与 Netfilter

sk_buff 是网络栈的“页”和“bio”。Linux 网络栈中，**sk_buff** 是数据包在内核中的主要载体。它不仅保存字节，还保存协议头指针、校验和状态、路由信息、引用计数、分片信息和共享数据区。理解 **skb**，才能把驱动、协议栈和 **socket** 接起来。

```
sk_buff:
    head, data, tail, end
    len, data_len
    mac_header, network_header, transport_header
    dev, sk
    checksum state
    shared_info: fragments, gso, refcount
```

skb_clone 共享数据但复制元数据，**skb_copy** 复制数据，**kfree_skb** 释放引用。性能问题常来自不必要 **copy**；正确性问题常来自共享 **skb** 被误写、引用计数错误或线性区和 **fragment** 区理解错误。

接收路径：从 NAPI 到 socket 队列。高吞吐网络接收通常走 **NAPI**。中断只负责调度 **poll**，**poll** 在 **budget** 内批量取包，把 **skb** 送入协议栈。

```
nic_interrupt:
    disable_rx_interrupt()
    napi_schedule()

napi_poll(budget):
    while work < budget and rx_desc_ready:
        skb = build_skb_from_rx_desc()
        napi_gro_receive(skb)
    if no_more_work:
        napi_complete()
        enable_rx_interrupt()

ip_rcv -> tcp_v4_rcv -> tcp_v4_do_rcv
-> append to socket receive queue
-> wake waiters
```

GRO 可以把多个小包合并成较大 **skb**，降低协议栈成本；但它也改变了观测粒度。抓包、**trace** 和应用 **recv** 的边界不一定一一对应。测试网络栈时要区分 **wire packet**、**skb**、**TCP segment** 和应用读出的字节。

发送路径：tcp_sendmsg 到驱动队列。发送路径从用户 **buffer** 复制或引用到 **skb**，进入 **TCP** 发送队列，按拥塞窗口、接收窗口和 **MSS** 分段，最后到 **qdisc** 和驱动。

```
sendmsg
-> inet_sendmsg
-> tcp_sendmsg
-> copy into skb or attach pages
-> tcp_push_pending_frames
-> ip_queue_xmit
-> qdisc enqueue/dequeue
-> dev_queue_xmit
-> driver ndo_start_xmit
```

这个路径解释了 **send** 成功的边界：数据进入本机 **TCP** 发送状态，不等于对端应用收到，更不等于业务提交。若连接稍后 **reset**，应用必须根据协议状态判断请求是否需要重试。

Netfilter hook 点。Netfilter 在 IP 路径上提供 hook: PRE_ROUTING、LOCAL_IN、FORWARD、LOCAL_OUT、POST_ROUTING。防火墙、NAT、容器端口映射和连接跟踪都依赖这些点。

hook	典型用途
NF_INET_PRE_ROUTING	入站后、路由前，DNAT、早期过滤
NF_INET_LOCAL_IN	目的地是本机，进入 socket 前过滤
NF_INET_FORWARD	转发路径过滤
NF_INET_LOCAL_OUT	本机发出包，路由后处理
NF_INET_POST_ROUTING	出站前，SNAT、最后修改

Netfilter 让网络语义不仅由 socket 代码决定，还受规则表、namespace、连接跟踪和 NAT 影响。诊断“包为什么没到应用”时，要检查驱动、IP、路由、Netfilter、socket 绑定和应用读取，而不是只看一个层次。

socket option 与内核策略。socket option 是应用调整内核协议策略的入口。例如 SO_RCVBUF 和 SO_SNDBUF 控制缓冲区，TCP_NODELAY 影响 Nagle，SO_REUSEPORT 允许多个监听 socket 分担连接，SO_KEEPALIVE 打开保活探测。

选项	影响	风险
TCP_NODELAY	小包立即发送	更多包，可能降低吞吐
SO_REUSEPORT	多 socket 绑定同端口	负载分配和安全检查更复杂
SO_RCVBUF	接收缓冲上限	太小丢吞吐，太大耗内存
SO_LINGER	close 时处理未发送数据	可能阻塞或丢弃

选项不是调优魔法。每个选项都改变状态机和资源边界，必须通过负载测试和故障测试验证。

connect 的阻塞、非阻塞和错误取回。connect 不是简单地“连上或失败”。阻塞 TCP connect 会发起握手并睡眠等待结果；非阻塞 connect 通常返回 EINPROGRESS，之后用户用 poll/epoll 等待可写，再用 getsockopt(SO_ERROR) 取真正结果。可写事件只表示连接状态有变化，不表示一定成功。

```

nonblocking_connect(sock, addr):
    ret = connect(sock, addr)
    if ret == 0:
        connected
    else if errno == EINPROGRESS:
        wait EPOLLOUT
        err = getsockopt(sock, SOL_SOCKET, SO_ERROR)
        if err == 0:
            connected
        else:
            connect failed with err
    else:
        immediate failure

```

程序只看到 EPOLLOUT 就开始发送，是常见错误。连接拒绝、超时、路由不可达都可能通过 socket 错误状态返回。内核把异步错误挂在 socket 上，应用必须取

走并处理。

recv 返回值的含义。TCP 是字节流，`recv` 的返回值表达当前 socket 状态，不表达业务协议状态。

结果	内核含义	应用动作
<code>n > 0</code>	从接收队列取出 <code>n</code> 字节	放入协议解析缓冲，继续解析消息边界
<code>0</code>	对端有序关闭，且本端已读完缓冲数据	处理 EOF，不能再期待新字节
<code>-EAGAIN</code> <code>-EINTR</code>	非阻塞 socket 暂无数据 被信号打断	等待下一次可读事件 根据业务决定重试或退出
<code>-ECONNRESET</code>	连接被复位	丢弃未完成请求，按协议决定是否重试

EOF 不等于错误，reset 也不等于普通 EOF。若对端发送 FIN，已经在接收缓冲里的数据仍可读；读完后才返回 0。若收到 RST，未完成的字节流不再可信，很多协议必须把当前请求标成结果未知。

TCP 不是消息队列。应用写入两次，接收端可能一次读到合并后的字节；应用写入一次，接收端可能分多次读到。TCP 保证有序字节流，不保留 `write` 调用边界。业务协议必须显式 framing。

```
length_prefixed_protocol:
    read bytes into buffer
    while buffer has at least 4 bytes:
        len = parse_u32(buffer[0:4])
        if buffer has less than 4 + len:
            break
        message = buffer[4:4+len]
        dispatch(message)
        remove consumed bytes
```

这个状态机必须处理半包、粘包、超长长度、恶意长度、连接关闭和超时。把一次 `recv` 当成一条完整消息，是 TCP 服务端最常见的教学级错误。

Nagle、delayed ACK 和小包延迟。`TCP_NODELAY` 关闭 Nagle 后，小写入更快发出，但包数增加；Nagle 打开时，小包可能等待未确认数据；接收端 delayed ACK 又可能延迟确认。两者组合会让“写了几个字节为什么几十毫秒后才到”变成真实问题。

机制	目的	可能副作用
Nagle	合并小包，减少网络开销	请求/响应小消息延迟上升
Delayed ACK <code>TCP_NODELAY</code> <code>TCP_CORK</code>	减少纯 ACK 包 降低小消息等待 手动聚合发送	与 Nagle 组合造成等待包数和 CPU 开销上升 忘记 uncork 会拖延发送

调优不能只背“低延迟就开 `TCP_NODELAY`”。如果协议能批量发送，保留聚合可能更好；如果是交互 RPC，小消息阻塞会直接进入尾延迟。

超时、重试和幂等不由 socket 自动解决。socket 超时只能告诉应用某个等待没有在期限内完成，不能告诉远端是否处理了请求。发送超时、接收超时、连接 reset、进程崩溃都可能让请求处在“可能提交、可能未提交”的灰区。应用协议要用请求 ID、

幂等键、事务日志或去重表解决语义。

```
rpc_with_idempotency:
    id = allocate_request_id()
    send(id, operation, payload)
    if timeout:
        reconnect_or_retry(id)
server:
    if id already committed:
        return old result
    execute operation
    record result by id
    return result
```

OS 负责提供字节流、错误码、关闭语义和等待机制；“这笔业务是否执行过”是应用协议层的责任。网络章节若不讲这个边界，读者会把 TCP 可靠传输误解成业务可靠提交。

第五部分

安全、恢复与观测

本部分导读

最后一部分不再引入新的资源类型，而是把前面所有资源放进工程验收框架。系统会被攻击，会 panic，会返回错误，会变慢，也会留下误导性证据。

安全隔离要说明谁被允许触碰哪些对象；持久化要说明崩溃后哪些状态可以信任；错误处理要说明失败怎样收敛而不是扩散；可观测性要说明证据从哪里来、能证明什么、不能证明什么。工程验收不是另一个附加主题，而是前面所有契约在真实故障里的检查方法。

读完本册的最低标准不是“知道很多术语”，而是能对任一机制写出：状态对象、入口、权限、等待、提交、失败路径和证据。

第 15 章 安全、隔离与最小权限

15.1 原理

操作系统的隔离不是一个单独功能，而是贯穿 CPU、内存、文件、进程、设备和网络的约束集合。没有隔离，一个进程可以读写任意内存、伪造设备请求、修改别人的文件、消耗全部 CPU 和内存。安全机制的目标不是让程序“不会出错”，而是让错误和恶意行为被限制在授权边界内。

先从一个具体事故模型进入：一个图片处理服务接收用户上传文件，解析库里有漏洞，攻击者能让它执行任意代码。此时问题已经不是“程序会不会出 bug”，而是“这个被接管的进程还能碰到什么”。如果它能读所有文件、调用所有系统调用、无限 fork、访问宿主设备、修改网络配置，那么一个进程漏洞就会变成整机失陷。OS 安全机制要做的是把损害限制在最小授权范围里。

错误设计通常长这样：

```
run_service():
    start as root
    mount host filesystem
    allow all syscalls
    no memory or process limit
    parse untrusted image
```

这个设计把“程序正常运行需要的能力”和“程序被攻破后拥有的能力”混成了一件。正确思路要反过来：先列主体、对象和操作，再逐层收紧。

层次	限制什么	失败时挡住什么
CPU 特权级和页表	用户代码不能执行特权指令或访问内核页	任意代码直接改内核内存
权限模型层	哪个主体能访问哪个文件、socket、设备	越权读写对象
namespace	进程能看见哪些名字和对象集合	通过全局路径或 PID 找到宿主资源
cgroup/rlimit	能消耗多少 CPU、内存、进程和 I/O	fork bomb、内存耗尽、I/O 拖垮整机
沙箱收紧层	未来还能调用哪些入口、访问哪些路径	漏洞后扩大攻击面

最基础隔离来自 CPU 特权级和页表权限。用户态不能执行特权指令，不能访问内核页，不能修改页表。系统调用是受控入口，内核在入口处检查参数和权限。文件权限、用户 ID、组 ID、capability、namespace、cgroup、seccomp 和 LSM 则在更高层限制对象访问和资源消耗。

这些机制不是互相替代。页表保护内核地址，但不决定某个用户能不能写 `/etc/passwd`；文件权限能拒绝写文件，但不能限制进程创建十万个子进程；namespace 改变进程看到的文件树和 PID 视图，但共享宿主内核；seccomp 能减少系统调用入口，但不能把原本没有的文件权限授予进程。安全设计要叠加多层边界，每层解决不同失败模式。

本章第一次使用“主体”和“对象”两个词。主体是发起动作的一方，例如进程、线程、用户身份、容器或服务账号。对象是被访问的一方，例如虚拟页、inode、socket、设备、namespace、cgroup 文件。权限检查不是抽象口号，它必须落到“主体 S 对对象 O 做操作 A 是否允许”这个三元关系上。

继续展开图片处理服务的事故。攻击者上传一张畸形图片，触发解析库越界写，拿到了服务进程内的任意代码执行。此时 CPU 和页表已经无法阻止它在本进程地址空间里做事，因为它就是这个进程本身；OS 隔离要从“它接下来想越界访问别的对

象”开始拦截。

攻击动作	若没有边界	哪层机制应当挡住
读取 <code>/etc/shadow</code>	泄露宿主口令哈希	DAC、capability、mount namespace、只读根文件系统
打开容器引擎 socket	创建特权容器控制宿主	mount namespace 不暴露 socket，文件权限和 LSM 拒绝
调用 <code>ptrace</code> 注入同机服务	横向控制其他进程	uid 边界、Yama/LSM、缺少 <code>CAP_SYS_PTRACE</code> 、seccomp
无限 <code>fork</code>	占满 pid 和调度资源	pids cgroup、rlimit
分配大量内存	触发宿主回收和 OOM	memory cgroup、oom 策略
创建设备节点或挂载文件系统	访问宿主设备或改写视图	缺少 <code>CAP_SYS_ADMIN</code> ，seccomp/LSM 拒绝

按这张表看，安全配置不是“开一个沙箱”就结束。假如服务以普通 uid 运行，但把宿主 Docker socket 挂进容器，攻击者不需要 root 也可能通过 socket 请求宿主创建新容器；假如没有 pids 限制，攻击者即使读不了敏感文件，也能 fork bomb 拖垮机器；假如 seccomp 允许 mount、bpf、ptrace、危险 ioctl，一次普通解析漏洞就有机会变成内核攻击面。每一层都只负责一类失败，叠加起来才叫最小权限。

把收紧后的设计写成状态变化，而不是口号：

```
before accepting untrusted images:
create service uid without shell login
mount only /app and /tmp/work, /app read-only
map container uid 0 to unprivileged host uid if using user namespace
drop all capabilities except the exact required set
set pids.max, memory.max, cpu.max, io.max
set no_new_privs
install seccomp profile for the real hot path
apply LSM/Landlock rules for file access
exec worker
```

这个顺序也有意义。先准备 namespace 和 cgroup，是为了让后续进程在受限视图和资源账本里出生；drop capability 和 no_new_privs 要发生在执行不可信工作前；seccomp 一旦安装，未来系统调用入口被缩小；最后通过 exec 进入 worker，可以顺便清理地址空间、环境变量和不该继承的 fd。若顺序反了，例如先开始解析图片再安装 seccomp，漏洞就可能发生在边界生效之前。

15.2 实践

安全分析先写主体、对象、操作、权限：

主体	对象	操作	检查
进程	虚拟页	read/write/exec	页表权限、VMA
进程	文件	open/read/write	mode、owner、capability、mount
进程	系统调用	exec/fork/ioctl	seccomp、LSM、资源限制
容器	CPU/内存	消耗资源	cgroup quota/limit

然后写威胁模型：攻击者控制什么输入，目标对象是什么，期望阻止什么。没有威胁模型的“安全”讨论会变成口号。

15.3 算法与实现分析

访问控制检查。访问控制算法接收主体、对象和操作，返回允许或拒绝。Unix mode 位是最小模型：如果主体是 owner，看 owner bits；否则如果主体属于 group，看 group bits；否则看 other bits。

```
may_access(subject, inode, op):
    if subject.uid == inode.uid:
        bits = inode.mode.owner
    else if subject.groups contains inode.gid:
        bits = inode.mode.group
    else:
        bits = inode.mode.other
    return bits.allow(op)
```

capability 是对 root 权限的拆分。与其让 uid 0 拥有全部能力，不如把网络配置、挂载、修改时间、加载模块等能力拆开。这样服务可以只拿最小需要的能力。

namespace 和 cgroup。namespace 改变“看见什么”，cgroup 限制“能用多少”。PID namespace 让进程看到自己的进程树；mount namespace 改变文件系统视图；network namespace 隔离网卡、路由和端口。cgroup 限制 CPU、内存、I/O 和进程数量。

```
container_start():
    create_namespaces(pid, mount, net, ipc, uts)
    configure_cgroup(cpu_quota, memory_limit, pids_max)
    drop_capabilities()
    apply_seccomp_filter()
    exec_init_process()
```

容器不是虚拟机。它共享宿主内核，所以内核漏洞仍可能突破容器边界。容器安全依赖 namespace、cgroup、capability、seccomp、LSM 和镜像文件系统共同工作。

seccomp 过滤。seccomp 允许进程限制自己未来能调用哪些系统调用。典型服务启动后，只保留 read、write、epoll、futex、clock 等必要调用，拒绝 mount、ptrace、模块加载等危险入口。

```
seccomp_check(syscall_nr, args):
    action = bpf_program.evaluate(syscall_nr, args)
    if action == ALLOW: continue_syscall()
    if action == ERRNO: return -EPERM
    if action == KILL: terminate_process()
```

seccomp 的价值是缩小攻击面。即使攻击者控制了进程内存，也很难调用被过滤的内核入口。但过滤规则写错也会让程序在正常路径崩溃，所以必须用测试覆盖。

15.4 测试

- 用户态访问内核地址必须失败。
- 只读映射写入必须触发权限错误。
- 文件权限拒绝时，不创建、不截断、不写入目标文件。
- drop capability 后，相关特权操作必须失败。
- namespace 中不可见的对象不能通过旧路径访问。
- cgroup 内存限制触发时，系统有明确 OOM 或失败策略。
- seccomp 规则允许正常路径，拒绝非必要危险系统调用。

本章压缩进来的故障、观测与验收。安全隔离不是最后一章唯一主题。一个真实系统还必须回答：错误怎样返回，panic 后哪些状态不可信，恢复依据是什么，观测证据从哪里来。把 panic、错误路径和可观测性放在安全章收束，是因为它们都在检查同一件事：边界被突破或状态不一致时，系统能不能停止扩散、留下证据、支持恢复。

场景	系统应做什么	验收证据
普通错误	返回明确 errno，回滚未提交对象	负面测试证明没有副作用和泄漏
可恢复资源压力	限流、回收、失败或局部 OOM	cgroup 事件、PSI、日志和请求错误率
内核不变量破坏	oops/panic，停止继续写不可信状态	panic 日志、kdump、最后 trace 事件
设备或 I/O 错误	停新请求、完成 in-flight、上报 delayed error	dmesg、block stats、fsync 错误、驱动计数器
安全拒绝	拒绝入口并记录主体、对象、操作	audit、LSM 日志、seccomp 统计和负面测试

观测工具也要按问题选择。tracepoint 和 ftrace 适合看内核路径是否经过；perf 适合看 CPU 时间、cache miss 和热点；eBPF 适合在线聚合延迟、队列长度和错误码；日志适合记录低频状态转换；crash dump 适合 panic 后离线检查。观测不是越多越好，证据必须能回答具体假设：线程是在等 CPU、等锁、等 I/O、等内存回收，还是被安全策略拒绝。

```
incident_report:
  symptom: request p99 rises from 20ms to 2s
  hypothesis: blocked on page cache writeback
  evidence:
    off-cpu stack shows balance_dirty_pages
    PSI io full rises during incident
    block queue latency p99 rises
    application CPU remains low
  conclusion:
    not an algorithm CPU hotspot; storage backpressure
```

整册的最终验收可以压成七项。任一机制都要写出状态对象、入口、权限、等待、提交、失败路径和证据。调度有 task/runqueue/wait queue；内存有 mm/VMA/PTE/page；文件有 fd/file/inode/page cache/request；网络有 socket/skb/队列/错误状态；安全有主体/对象/策略/审计。若某个解释只剩术语，没有这七项，就还不能指导实现和排障。

本册收束标准：拿到一次系统调用、缺页、I/O、网络请求或安全拒绝，能说清它改变了哪些对象，在哪个点可能睡眠，哪个点对用户可见，哪个点对持久化或远端可见，失败如何回滚，哪条观测证据能证明判断。做到这一点，章节压缩后内容仍然是厚的。

15.5 源码级补充：DAC、MAC、capability、namespace、Landlock 与硬化

访问控制模型。DAC 由对象拥有者决定访问，典型是 UNIX mode、uid、gid；MAC 由系统策略决定，典型是 SELinux 和 AppArmor；RBAC 按角色授权；ABAC 按属性决策。Linux 实际路径通常叠加 DAC、capability、LSM、mount flags 和 namespace。

读 `fs/namei.c`、`security/security.c`、`security/selinux/` 时，要追踪同一个对象从路径解析到权限 hook 的全过程。

```
open(path, O_WRONLY):
    resolve path to inode and mount
    check mount is writable
    check DAC mode or CAP_DAC_OVERRIDE
    call security_inode_permission()
    check file type and open flags
    create struct file
```

安全检查必须绑定实际对象。只检查字符串路径，再使用路径，是 TOCTOU 风险；解析后持有 inode、dentry 或 file 引用，再检查权限，才有稳定对象。权限失败时还要保证没有副作用：不能先 truncate 再发现权限不足。

capability 集合。进程有 permitted、effective、inheritable、bounding、ambient 等 capability 集合。bounding set 给进程树设置上界；ambient set 影响 exec 后保留的能力。容器安全经常要求清空不必要 capability，尤其避免 `CAP_SYS_ADMIN`、`CAP_SYS_PTRACE`、`CAP_NET_RAW`。

```
drop_privileges():
    clear_ambient_capabilities()
    drop_bounding_set_except(required_caps)
    set_no_new_privs()
    install_seccomp_filter()
```

能力检查要放在对象所属 user namespace 下解释。`ns_capable(user_ns, CAP_NET_ADMIN)` 和全局 `capable` 不是同一个边界。容器内 root 若只在容器 user namespace 有能力，不应自动获得宿主网络或宿主挂载的管理权。

namespace 与 cgroup v2。

namespace	隔离内容
PID	进程编号和父子视图
Mount	挂载树和路径视图
Network	网络设备、路由、端口空间
IPC	System V/POSIX IPC 对象
UTS	hostname/domainname
User	uid/gid 映射和 namespace 内 capability
CGroup/Time	cgroup 视图、部分时间视图

cgroup v2 用 `cgroup.controllers` 声明可用 controller，用 `cgroup.subtree_control` 向子树下发 controller，用 `memory`、`cpu`、`io`、`pids` 等文件表达限制。资源限制也是安全边界：没有 pids 限制，fork bomb 会扩大到宿主；没有 memory 限制，单服务可触发全局回收和 OOM；没有 io 限制，后台写入会拖慢其他租户。

Landlock、seccomp user notification 和 BPF LSM。Landlock 允许非特权进程给自己加文件访问规则，是基于 LSM 的沙箱。seccomp user notification 可把某些系统调用交给监督进程决策。BPF LSM 允许加载受验证的 BPF 程序到安全 hook。这些机制的共同边界是：它们可以进一步限制自己或被管理进程，但不能凭空授予原本没有的权限。

```
sandbox_process:
    create Landlock ruleset
    add allowed read-only paths
    restrict_self()
    install seccomp allowlist
```



```
exec workload
```

这类沙箱必须有负面测试：禁止路径打开失败、禁止 `syscall` 被拒绝、规则安装后不能再获得新权限。只验证正常路径能运行，不足以证明沙箱有效。

TPM、measured boot 和 sealed storage。TPM 可记录启动链测量值，remote attestation 让远端验证机器启动了预期固件、bootloader、内核和策略；sealed storage 让密钥只在特定测量状态下解封。它解决的是“这台机器是不是以预期状态启动”，不是运行时所有访问控制。内核被测量启动后，仍然要依赖权限、LSM、seccomp 和更新机制维护运行时边界。

ASLR、stack canary、CFI 与 KASLR。ASLR 随机化栈、堆、mmap 和 PIE 基址；stack canary 检测栈溢出覆盖返回地址；CFI 限制间接控制流；KASLR 随机化内核基址。它们是缓解措施，不是权限模型替代品。

机制	防什么	不能防什么
ASLR/KASLR	降低地址可预测性	信息泄漏后效果下降
stack canary	简单栈返回地址覆盖	任意写、逻辑漏洞
CFI	非法间接跳转	合法路径上的权限绕过
WX/NX	数据页执行	JIT 配置错误、ROP

安全测试要同时证明正常路径可用、禁止路径失败、失败有日志、敏感数据不泄露。只打开硬化选项但没有负面测试，不能说明边界有效。

open 权限检查不能只看路径字符串。路径是输入，inode 和 file 才是稳定对象。安全代码若先检查字符串路径，再在另一步打开文件，中间目录项可能被替换，形成 TOCTOU。更可靠的做法是让内核路径解析在同一次操作里完成权限、mount、LSM 和 open flag 检查，或者使用目录 fd、`openat2` 约束和已经打开的 fd 作为稳定能力。

```
unsafe pattern:
  if path starts with "/safe":
    open(path)

safer pattern:
  dirfd = open("/safe", O_PATH | O_DIRECTORY)
  openat2(dirfd, relative_path,
          RESOLVE_BENEATH | RESOLVE_NO_SYMLINKS)
```

这里的关键不是某个 API 名字，而是对象绑定：检查和使用必须落在同一个解析结果上。符号链接、bind mount、rename、并发 unlink、mount namespace 都会改变“同一个字符串”指向的对象。

exec 时 capability 和 no_new_privs 改变边界。权限不是进程启动后固定不变。exec 会重新计算凭据、环境、dumpable 状态、secureexec、文件 capability 和 setuid 影响。若进程设置 `no_new_privs`，后续 exec 不能获得比当前更多的权限；seccomp 过滤器通常要求先设置这个标志，防止低权限进程安装过滤器后再通过 exec 提权。

```
exec credential sketch:
  read current cred
  inspect executable mode and file capabilities
  if no_new_privs:
    suppress privilege gain
  compute new permitted/effective/ambient sets
  apply secureexec rules if privilege boundary changed
  commit new cred at exec commit point
```

ambient capability 容易被误用。它能跨 exec 传递给非特权可执行文件，所以启动服务前应清理不需要的 ambient set，并收紧 bounding set。否则子进程可能在没预期的路径上保留网络、ptrace 或管理能力。

seccomp 规则要按参数和返回动作设计。只按 syscall 号 allow/deny 往往太粗。例如 `ioctl`、`prctl`、`clone`、`socket` 的危险性高度依赖参数。高质量 seccomp profile 至少要区分常用参数组合，并为拒绝路径选择明确动作：返回 `errno`、kill 进程、记录日志或交给 user notification。

```
seccomp policy sketch:
allow read/write/close/futex/clock_gettime
allow mmap only without PROT_EXEC|PROT_WRITE together
allow socket only AF_UNIX if service does not need network
reject mount/ptrace/bpf/module loading
return EPERM for expected probes
kill process for impossible dangerous calls
```

规则还要随库和运行时变化测试。动态链接器、线程库、DNS、日志库都可能调用你没想到的系统调用。部署前应在真实启动路径、热路径、错误路径和 reload 路径下跑 profile，而不是只跑主函数。

Landlock 的边界：只能收紧，不能授权。Landlock 适合让普通进程给自己加文件系统沙箱。它的 ruleset 声明要处理哪些访问类型，再把路径 fd 加入 allow list，最后 `restrict_self`。一旦限制生效，进程不能用 Landlock 获得原来没有的权限，只能在原权限基础上进一步收紧。

```
landlock flow:
ruleset = create_ruleset(handled_access_fs)
dirfd = open(allowed_dir, O_PATH | O_DIRECTORY)
add_path_beneath_rule(ruleset, dirfd, allowed_access)
prctl(PR_SET_NO_NEW_PRIVS, 1)
landlock_restrict_self(ruleset)
exec or run workload
```

测试要覆盖允许目录、禁止目录、符号链接、rename、临时文件和继承。若只测试“程序还能启动”，无法说明沙箱真的挡住了写入、删除、执行或路径逃逸。

BPF LSM 和传统 LSM 的关系。LSM hook 把安全检查插入到 inode、file、task、socket、bpf 等对象操作中。传统 LSM 如 SELinux、AppArmor 使用预先加载的策略；BPF LSM 允许受验证的 BPF 程序挂到安全 hook 上，适合做可观测、渐进式或环境相关的策略。

```
security_inode_permission(inode, mask):
for each registered LSM:
    ret = lsm->inode_permission(inode, mask)
    if ret != 0:
        return ret
return 0
```

BPF LSM 仍受 verifier、权限和 hook 上下文限制。程序不能任意睡眠，不能随便访问内核内存，返回值语义也要遵守 hook 约定。它是内核安全框架的一部分，不是绕过 DAC/capability 的后门。

容器逃逸常见入口。容器共享宿主内核，所以边界经常坏在“多给了一个能力或设备”。常见风险包括挂载宿主敏感路径、保留 `CAP_SYS_ADMIN`、开放容器引擎 socket、允许特权设备、未限制 `/proc` 和 `/sys`、seccomp profile 过宽、cgroup pids/memory 未限。

风险	为什么危险	收紧方向
<code>CAP_SYS_ADMIN</code>	覆盖大量管理操作	默认删除，只按具体需要添加
宿主容器引擎 socket	等价于控制宿主容器引擎	不挂载，改用受限代理
特权设备节点	可能直接访问内核或硬件接口	<code>devices cgroup</code> 和最小设备集
宽松 <code>/proc</code>	泄露进程、内核和 <code>namespace</code> 信息	<code>hidepid</code> 、只读挂载、裁剪视图
无 <code>pids/memory</code> 限制	<code>fork bomb</code> 或内存压力拖垮宿主	<code>cgroup v2</code> 限额

安全隔离要按“最小必要对象”配置，而不是按“程序跑起来需要什么就全开”。每放开一个 `capability`、设备、挂载或 `syscall`，都要写清楚它对应的业务需求和负面测试。

第 16 章 持久化、崩溃一致性与恢复

16.1 原理

持久化问题的核心是：系统可能在任意两步之间崩溃。程序调用 `write`，内核可能只把数据放进 `page cache`；设备可能只完成了一部分请求；目录项和文件内容可能以不同顺序持久化；应用日志可能写了意图但没有应用数据；`manifest` 可能发布了一个并不完整的输出。崩溃一致性讨论的不是“正常运行时看起来对不对”，而是“重启后能否解释磁盘状态”。

`write`、`fsync`、`rename` 和 `manifest` 分别处在不同层次。`write` 把字节交给内核；`fsync` 推进文件内容和部分元数据的持久化；`rename` 可以提供目录项替换的原子可见性，但目录本身是否持久也需要考虑；`manifest` 是业务层承认某批输出有效的提交点。只有把这些层次分开，才能写可靠输出。

文件系统常用日志或 `copy-on-write` 技术维护自己的元数据一致性，但这不自动等于应用数据一致。文件系统保证的是它自己的结构能恢复，不保证你的两个文件之间满足业务事务。应用如果需要“数据文件和索引文件同时生效”，就必须设计应用层提交协议。

最小可靠写出协议通常是：写临时文件，写完后 `fsync` 临时文件，校验大小和 `checksum`，`rename` 到最终路径，`fsync` 父目录，最后写或更新 `manifest`。这个协议不是唯一答案，但它展示了 `prepared` 和 `committed` 的区别。

日志恢复要区分 `redo` 和 `undo`。`redo` 日志记录“应该完成什么”，崩溃后可以重放；`undo` 日志记录“如何撤销未完成修改”。`write-ahead logging` 的关键规则是：描述修改的日志必须先于实际修改持久化，否则崩溃后无法判断磁盘上的半成品是否应该保留。

16.2 实践

纸上实践先写提交状态机：

```
NoOutput
-> TempCreated
-> DataWritten
-> DataSynced
-> RenamedVisible
-> DirectorySynced
-> ManifestCommitted
```

每个状态都要写出崩溃后恢复规则：

崩溃点	恢复策略
TempCreated	删除临时文件或忽略
DataWritten	校验失败则删除，不能发布
DataSynced	仍未提交，等待 <code>rename</code> 或清理
RenamedVisible	若 <code>manifest</code> 未提交，按业务规则决定是否采用
DirectorySynced	文件名持久，但业务仍以 <code>manifest</code> 为准
ManifestCommitted	读取者可以信任并校验文件

实践中要避免把“文件存在”当成业务成功。目录中有最终文件名，只说明某次 `rename` 可能发生过；`manifest` 提交并通过 `checksum`，才说明业务层承认它属于某个版本或批次。

再写一份日志协议：

```

append log record: transaction T intends to write block B with checksum C
fsync log
write data block
fsync data
append commit record for T
fsync log

```

恢复时扫描日志：有 commit 的事务必须 redo；没有 commit 的事务按规则忽略或 undo；checksum 不匹配的记录停止或进入人工恢复。这里的重点不是实现数据库，而是理解日志先行、提交记录和幂等恢复。

16.3 算法与实现分析

原子 rename 发布。很多应用用临时文件加 rename 发布结果。算法分为写入、同步、替换、同步目录四步：

```

write_atomic(path, bytes):
    tmp = path + ".tmp." + unique_id()
    fd = open(tmp, O_CREAT | O_EXCL | O_WRONLY)
    write_all(fd, bytes)
    fsync(fd)
    close(fd)
    rename(tmp, path)
    dirfd = open(parent_dir(path))
    fsync(dirfd)
    close(dirfd)

```

这个算法保证读者要么看到旧文件，要么看到新文件，不应看到半个最终文件。但它仍然只是单文件发布。若业务需要多个文件同时生效，还需要 manifest 或事务日志。

manifest 提交。manifest 把多个输出文件绑定成一个版本。数据文件可以先各自写好并 fsync；manifest 最后写入，列出版本号、文件名、大小和 checksum。读取者只读取 manifest 指向的文件。

```

commit_version(version, files):
    for file in files:
        write_atomic(file.tmp_path, file.bytes)
        verify_checksum(file.tmp_path)
        rename(file.tmp_path, file.final_path)
    manifest = build_manifest(version, files)
    write_atomic("MANIFEST.new", manifest)
    rename("MANIFEST.new", "MANIFEST")

```

manifest 的优点是恢复简单：重启后读取 MANIFEST，把它当作权威事实；不在 manifest 里的临时文件或孤儿文件可以清理。缺点是提交粒度较粗，manifest 本身也需要可靠写出。

redo 日志。redo 日志适合“提交后必须能重放”的场景。日志记录足够的信息，使恢复程序可以重新执行已经提交但未完全应用的修改。日志记录必须先持久化，数据页才能写出。

```

transaction_write(T, block, data):
    log_append(INTENT, T, block, checksum(data))
    fsync(log)
    write(block, data)
    log_append(COMMIT, T)
    fsync(log)

```

恢复算法扫描日志：看到 INTENT 但没有 COMMIT 的事务不应用；看到 COMMIT 的事务检查数据是否已经应用，未应用则 redo。redo 操作必须幂等，即重复执行不会改变最终正确结果。

检查点。日志无限增长不可接受，所以系统要做 checkpoint。checkpoint 把当前稳定状态写成快照，并记录“恢复只需从这个日志位置之后开始”。算法难点是 checkpoint 期间仍可能有新事务进入，因此要记录 epoch 或 LSN。

```
checkpoint():
    begin_lsn = log.current_lsn()
    flush_dirty_pages_up_to(begin_lsn)
    write_checkpoint_record(begin_lsn)
    fsync(log)
    truncate_log_before(begin_lsn)
```

checkpoint 的正确性依赖顺序：只有当某个 LSN 之前的脏数据都已经安全写出，才能截断之前的日志。否则崩溃后会丢失 redo 所需信息。

16.4 测试

持久化测试必须做故障注入。只在正常路径写文件再读取，不足以证明崩溃一致性。

- 在每个提交状态后模拟崩溃，恢复程序能给出确定结果。
- 临时文件残留不会被读取者当成有效输出。
- manifest 指向缺失文件时，读取者拒绝该版本。
- checksum 错误时，恢复程序能定位坏文件并拒绝提交。
- 重复执行恢复程序是幂等的，不会一次恢复成功、二次恢复破坏状态。
- write 成功但 fsync 失败时，业务提交必须失败。
- rename 后但 manifest 前崩溃，恢复策略必须明确。

测试报告不要只写“断电恢复成功”。必须列出崩溃点、磁盘上允许出现的文件集合、恢复后权威事实、被清理对象和未覆盖边界。可靠性来自状态枚举，不来自运气。

16.5 源码级补充：WAL、COW 文件系统、RAID、LVM 与 media failure

崩溃模型分层。崩溃一致性至少有三类故障：transaction crash，系统在两步之间断电；system crash，内核或整机停止但介质仍可靠；media failure，介质本身丢失或返回坏数据。日志能处理前两类的一部分，但不能让坏盘自动恢复数据，后者需要冗余、校验和副本。

故障	例子	需要的机制
transaction crash	commit 前断电	WAL/journal/manifest 状态机
system crash	内核 panic、掉电	fsync、flush、replay、fsck
media failure	坏块、整盘损坏	RAID、复制、checksum、备份

LSN 与 checkpoint。WAL 系统通常给每条日志记录 LSN。数据页记录 pageLSN，表示该页包含到哪个日志位置的修改。恢复时若 pageLSN 小于 committed LSN，就 redo；checkpoint 记录某个 LSN 之前的脏页已经安全传播。


```
redo_recovery():
    start = last_checkpoint_lsn
    for record in log from start:
        if record.committed and page.page_lsn < record.lsn:
            apply(record)
            page.page_lsn = record.lsn
```

LSN 的价值是让恢复可判定，而不是靠文件时间戳或猜测。数据库、文件系统 journal 和 manifest 版本号都在解决同一问题：给恢复程序一个可比较的事实。

COW 文件系统。btrfs、ZFS 等 COW 文件系统不在原地覆盖元数据，而是写新块，再用新的根指针发布版本。snapshot、clone、send/receive 都建立在“旧版本块仍被引用，新版本写到新位置”的基础上。

```
cow_update(tree, key, value):
    new_leaf = copy_and_modify(old_leaf)
    new_parent = copy_and_point_to(new_leaf)
    ...
    write_all_new_blocks()
    atomically_update_super_root(new_root)
```

COW 避免部分原地覆盖造成的结构破坏，但带来写放大、碎片和空间回收复杂度。snapshot 多时，一个旧块可能被许多版本引用，不能简单释放。

RAID 与 write hole。RAID 0 条带化提高吞吐但无冗余；RAID 1 镜像；RAID 5/6 用奇偶校验节省空间；RAID 10 结合镜像和条带。RAID 5 写入数据和 parity 时若断电，可能出现 write hole：数据和校验不一致。

级别	优点	风险
RAID0	高吞吐、高容量利用	任一盘坏全丢
RAID1	简单冗余，读可并行	容量减半
RAID5/6	容量效率高，有校验	小写放大、write hole、重建压力
RAID10	性能和可靠性较好	成本高

write hole 可用日志、非易失缓存、bitmap 或 copy-on-write parity 缓解。这里再次说明：持久化正确性来自顺序证据和冗余，不来自“写了两次应该差不多”。

LVM 和 device mapper。Linux device mapper 把上层块设备映射到底层设备区域，LVM 的 linear、snapshot、thin provisioning 都建立在此之上。dm-snapshot 用 COW 保存旧块，dm-thin 动态分配块，带来空间耗尽时的错误传播问题。

```
bio to /dev/mapper/vg-lv
-> dm target maps logical sector
-> lower device sector(s)
-> submit cloned bio
-> complete original bio after all lower bios complete
```

读 `drivers/md/dm.c` 和具体 target。关注 bio clone、错误合并、flush 传播和 snapshot 元数据一致性。

第 17 章 panic、错误处理与可观测性

系统会出错、会 panic、会变慢。本章把错误路径收敛、崩溃恢复和可观测性工具放在一起：从 `errno` 到 `kdump`，从 `printk` 到 `eBPF`，从现象到证据到结论。

17.1 原理

操作系统必须区分普通错误、进程错误、内核 bug 和硬件不可恢复错误。普通错误应返回 `errno`，例如文件不存在、权限不足、非阻塞 I/O 暂时不可用；进程错误应向进程发送信号，例如非法地址访问；内核 bug 可能触发 `WARN`、`oops` 或 `panic`；硬件错误可能要求隔离设备、下线页面或重启系统。

若把所有错误都 `panic`，系统不可用；若把所有错误都吞掉，数据会被悄悄破坏。错误分类的责任在于明确每个失败点：哪些返回 `errno`，哪些终止进程，哪些停止系统。

`panic` 的含义是内核认为继续运行比停止更危险。典型原因包括核心不变量破坏、调度器状态自相矛盾、文件系统元数据检测到不可恢复损坏、内核栈溢出、无法处理的机器检查。`panic` 不是调试打印的高级版本，而是系统级 `fail-stop` 策略：停止服务，尽量保留证据，避免进一步破坏状态。

核心思想

高质量 OS 实现不只写正常路径，还要把每个失败点的责任写清楚：返回错误、终止进程、重试、降级、隔离，还是 `panic`。

错误路径的工程难点在资源回收。系统调用可能已经分配页、获取锁、增加引用、插入表项、发起 I/O，然后第七步失败。正确实现必须让失败路径按照相反顺序撤销已完成步骤，并且不能撤销还没完成的步骤。C 语言内核常用 `goto out_...` 标签表达这种线性回滚；C++ 内核或教学模拟器可以用 `RAII` 表达资源所有权，但必须注意中断上下文、`panic` 路径和 `no-throw` 约束。

转向可观测性：操作系统问题很少只靠读代码解决。线程为什么不运行，可能是调度等待、锁等待、I/O 阻塞、`page fault`、信号、`cgroup` 限流或下游服务；内存为什么上涨，可能是真实泄漏、`allocator` 缓存、`page cache`、`mmap`、线程栈或内核 `buffer`；写文件为什么慢，可能是 `page cache` 脏页、设备队列、`fsync`、目录同步或日志提交。没有观测证据，结论只是猜测。

可观测性要把 OS 状态映射到证据。进程和线程状态可以从 `/proc`、`ps`、`top`、调度 `trace` 观察；系统调用可以用 `strace` 观察；CPU 热点可以用采样 `profiler`；`page fault`、`context switch`、I/O 等待和网络状态可以用系统计数器和 `tracing` 工具；应用层必须补充业务 `trace`，记录请求身份、队列等待、状态机阶段和错误码。

核心思想

可观测性的目标不是收集越多越好，而是把 OS 状态映射到证据：`on-CPU` 解释运行时消耗，`off-CPU` 解释等待时间，两者闭合才能定位问题。

调试时要区分 `on-CPU` 和 `off-CPU`。`on-CPU` 说明线程正在执行指令；`off-CPU` 说明线程没有运行，可能在等待。CPU `profiler` 只能解释运行时消耗在哪里，解释不了大部分时间都在睡眠的请求。尾延迟问题常常要从 `off-CPU` 等待开始看。

内存调试要区分 virtual size、resident set、anonymous memory、file-backed page、page cache、allocator arena 和 kernel memory。`malloc` 后不触页，可能只增加虚拟地址；首次写入才分配物理页；释放内存后 RSS 不马上下降，也可能是 allocator 保留 arena。

把所有 RSS 增长都叫泄漏，是不合格诊断。虚拟地址、物理页、page cache、allocator arena 和内核内存各有不同来源，必须先分清再下结论。

I/O 调试要区分 buffered I/O、direct I/O、mmap、page cache hit/miss、dirty writeback、fsync 和设备吞吐。一次写很快，可能只是进入 page cache；一次读很快，可能只是热缓存；一次 fsync 很慢，可能是在偿还之前累计的脏页。实验必须控制冷缓存和热缓存，否则数字不可比较。

17.2 实践

错误分类可以写成下表：

类别	处理方式	示例
可预期用户错误	返回 <code>errno</code>	<code>ENOENT</code> 、 <code>EACCES</code> 、 <code>EAGAIN</code>
用户进程违规	信号或终止该进程	<code>page fault</code> 权限错误、非法指令
资源暂时不足	返回错误、等待或回收	<code>ENOMEM</code> 、队列满、端口耗尽
设备局部故障	重置设备、隔离队列、上报 I/O error	磁盘超时、网卡 reset
内核不变量破坏	<code>WARN/oops/panic</code>	双重释放、 <code>runqueue</code> 计数错误
持久化一致性风险	<code>remount read-only</code> 、 <code>replay journal</code> 、 <code>panic</code>	元数据校验失败

一个系统调用实现应在设计阶段列出失败点：

```
open(path, flags):
  copied_path = copy_path_from_user(path)           may fail EFAULT/ENAMETOOLONG
  parent = resolve_parent(copied_path)              may fail ENOENT/EACCES
  inode = lookup_or_create(parent, name, flags)      may fail ENOENT/EXIST/EROFS
  file = alloc_file(inode, flags)                   may fail ENOMEM
  fd = alloc_fd(current.files)                       may fail EMFILE
  install_fd(fd, file)                               commit point
  return fd
```

提交点之前失败，要释放已拿到的引用；提交点之后失败，通常不能再返回“没有打开”，而要让 fd 表保持一致或执行明确关闭。把提交点写出来，可以避免半初始化 fd 留在表里。

本章的实践模板是“现象、假设、证据、反证、结论”。

字段	请求延迟升高示例
现象	p95 从 20ms 升到 500ms
假设	worker 等锁或等 I/O
证据	trace 中 <code>queue_wait</code> 低， <code>blocked_io</code> 高，线程睡在 writeback
反证	on-CPU profiler 没有热函数占用 500ms
结论	延迟主要来自 I/O 完成等待，不是 CPU 算法变慢

程序卡死时，先列线程状态：哪个线程持有锁，哪个线程等待条件变量，哪个线程在 join，关闭标志是否设置，是否有线程仍可能唤醒等待者。若所有消费者都睡在 not_empty，而关闭路径只通知 not_full，就能定位到等待谓词和通知对象不匹配。

实践中要保留原始证据：命令、时间、输入规模、机器信息、内核版本、容器限制、采样频率、日志片段和无法观察的字段。没有这些，报告不可复现。

17.3 算法与实现分析

errno 返回与用户可见契约。errno 不是随便选的数字。调用者会根据错误码决定重试、降级、提示用户还是终止。

```
copy_from_user_checked(dst, user_ptr, len):
    if not user_range_valid(user_ptr, len, READ):
        return EFAULT
    if len > MAX_COPY:
        return EINVAL
    fault = copy_may_fault(dst, user_ptr, len)
    if fault:
        return EFAULT
    return OK
```

把非法用户地址返回 EFAULT，而不是 panic，是因为这是用户进程可触发的预期错误。内核必须把“不可信输入”当正常情况处理。只有当内核自己的页表、锁状态或对象引用被破坏时，才进入更严重路径。

goto 回滚模板。C 风格内核常见模式：

```
sys_create(path):
    name = null
    inode = null
    file = null

    name = copy_name(path)
    if name == null:
        ret = ENOMEM
        goto out

    inode = create_inode(name)
    if is_error(inode):
        ret = error(inode)
        goto out_name

    file = alloc_file(inode)
    if file == null:
        ret = ENOMEM
        goto out_inode

    ret = install_fd(file)
    if ret < 0:
        goto out_file
    return ret

out_file:
    put_file(file)
out_inode:
    put_inode(inode)
out_name:
    free(name)
```

```
out:
    return ret
```

这个结构的优点是回滚顺序与获取顺序相反，且每个标签只释放已经成功获取的资源。缺点是标签命名和跳转边界容易腐烂，需要测试每个失败注入点。C++ RAII 可以减少样板，但内核中不是所有路径都适合异常；析构函数也不能在持有自旋锁或 panic 路径中做复杂阻塞操作。

WARN、BUG、oops 与 panic。可以把严重程度分成四级：

机制	含义	处理
WARN	发现异常但可能继续	打印栈、计数、继续或降级
BUG/oops	当前内核路径不可继续	杀死当前进程或停止当前 CPU，视上下文而定
panic	整个系统不可安全继续	停机、重启或进入 crash dump
watchdog	系统长时间无响应	触发诊断、中断栈采样或 panic

教学系统可以实现 `assert_kernel_invariant`：调试构建 panic，发布构建记录错误并尽量隔离。但对文件系统元数据损坏这类风险，继续写入可能扩大破坏，`remount read-only` 或 panic 是合理选择。

看门狗与故障转储。watchdog 检测“没有前进”。硬锁死可能表现为中断不再发生；软锁死可能表现为某 CPU 长时间不调度；RCU stall 表示读侧临界区过久导致回收无法完成。

```
watchdog_tick(cpu):
    now = time()
    if now - cpu.last_schedule_time > SOFTLOCKUP_LIMIT:
        dump_cpu_stack(cpu)
        warn("soft lockup")
    if now - cpu.last_interrupt_time > HARDLOCKUP_LIMIT:
        panic("hard lockup")
```

panic 后的目标不是恢复服务，而是保留证据：panic 原因、CPU 寄存器、栈、当前任务、锁持有、最近日志、脏页状态和设备错误。crash dump 的价值在于让下一次启动后能分析，而不是在 panic 路径中做复杂修复。

重试与幂等。不是所有失败都应该立即返回。临时内存不足可以触发回收后重试；设备超时可以 reset 后重试；网络发送可以等待窗口。但重试必须有边界，并且操作要么幂等，要么有明确的提交记录。

```
retry_io(req):
    while req.retries < MAX_RETRIES:
        submit(req)
        status = wait_completion(req, timeout)
        if status == OK:
            return OK
        if not is_transient(status):
            return status
        reset_path_if_needed(req.device)
        req.retries++
    return EIO
```

对写请求，若设备状态不明，简单重试可能造成重复写。对普通块覆盖写通常可接受；对“追加一条记录”这类上层操作，必须有序列号、校验或日志来识别重复提

交。

事件日志。最简单的可观测性算法是事件日志。每个关键状态转换追加一条结构化记录：时间、线程、请求 ID、事件类型、对象 ID、旧状态、新状态和错误码。日志不能只写自然语言，否则无法聚合和验证。

```
record_event(req_id, object, old_state, new_state, reason):
    entry.ts = monotonic_time()
    entry.cpu = current_cpu()
    entry.thread = current_thread()
    entry.req_id = req_id
    entry.object = object
    entry.old_state = old_state
    entry.new_state = new_state
    entry.reason = reason
    ring_append(trace_buffer, entry)
```

事件日志本身也要有背压。无限日志会把故障变成磁盘或内存问题。常见做法是固定大小环形缓冲区、按级别采样、按请求 ID 打开详细追踪，或者把关键错误同步写入可靠日志。

延迟分解。请求延迟不能只记录总耗时。一个可行的分解至少包括 queue wait、runnable wait、on CPU、blocked on lock、blocked on I/O、downstream wait 和 cleanup。实现上可以在状态转换处打点，然后按请求 ID 聚合。

```
transition(req, new_state):
    now = clock()
    req.time_in_state[req.state] += now - req.last_ts
    req.state = new_state
    req.last_ts = now
```

这个算法的复杂度是每次状态转换 $O(1)$ ，但要求状态枚举稳定。若所有等待都记成 blocked，后续无法判断是锁、I/O 还是 page fault。

off-CPU 分析。很多性能问题不在 CPU 火焰图里，因为线程大部分时间不在 CPU 上。off-CPU 分析记录线程从运行态切到睡眠态的原因，以及被谁唤醒。

```
on_schedule_out(task, reason, wait_object):
    task.offcpu_start = clock()
    task.wait_reason = reason
    task.wait_object = wait_object

on_wakeup(task, waker):
    delta = clock() - task.offcpu_start
    offcpu_histogram[task.wait_reason].add(delta)
    record_wakeup_edge(waker, task, task.wait_object)
```

这个数据能区分等锁、等 I/O、等页、等网络、等定时器和等子进程。若 p99 延迟主要来自 off-CPU 的 writeback 等待，优化业务 CPU 代码不会解决问题。

资源对账。资源泄漏调试需要对账算法。每次 open 增加 fd 计数，每次 close 减少；每次分配对象记录 owner，每次释放删除记录；系统退出或测试结束时检查剩余对象。

```
track_alloc(id, type, owner):
    live[id] = {type, owner, stack}

track_free(id):
    live.erase(id)
```



```
assert_no_leak():
    for each object in live:
        report(object)
```

真实系统不能给所有对象都保存完整 `stack`，因为成本太高；可以采样、按类型开启、只在测试构建开启。但思想不变：资源必须能被计数，生命周期必须能闭合。

最小可复现报告。高质量调试报告要让别人能复现判断。最小格式如下：

```
Problem:
    现象、影响范围、首次出现时间

Environment:
    内核版本、硬件/容器限制、文件系统、负载规模

Timeline:
    关键事件按时间排序

Evidence:
    系统调用、trace、线程状态、资源计数、日志、指标

Rejected hypotheses:
    已经排除的解释和证据

Conclusion:
    当前最能解释现象的机制

Boundary:
    仍未验证的假设和下一步实验
```

报告的价值不在于一次猜中，而在于缩小不确定性。写出反证能防止团队反复讨论已经排除的方向。

17.4 测试

可观测性与错误路径都需要测试。一个系统若只能在成功时输出日志，失败时没有状态，就不可维护。

1. `errno` 矩阵：非法指针、权限不足、资源耗尽返回不同错误码。
2. 失败注入：在系统调用每个分配点失败，引用计数和锁计数最终归零。
3. 提交点测试：提交前失败无用户可见副作用，提交后状态自愈。
4. `WARN` 测试：非致命不变量异常记录证据但不破坏后续测试。
5. `panic` 测试：核心不变量破坏后停止调度，输出原因和栈。
6. `watchdog` 测试：构造关中断死循环或不调度循环，能触发诊断。
7. 设备重试：临时错误可重试，永久错误返回 `EIO` 且唤醒等待者。
8. `crash dump`: `panic` 后保存当前任务、寄存器、锁和最近日志。
 - 每个长时间等待点都能输出等待原因或 `trace` 事件。
 - 每个请求都有稳定身份，能串起进入队列、开始执行、阻塞、完成和失败。
 - 错误码保留原始原因，不被统一改写成 `unknown error`。
 - 资源计数可对账：打开 `fd` 数、活跃线程数、队列长度、`in-flight I/O`、分配对象数。

- 性能测试区分冷启动、热缓存、预热、输入规模和测量次数。
- 故障注入能触发日志和恢复路径，而不是只测试成功路径。

本章合格标准：能区分 `errno`、信号、`oops`、`panic`；能为系统调用写出失败回滚路径；能解释提交点、幂等重试和故障证据的关系。调试报告的验收标准是别人能复现你的判断：若报告只写“怀疑是系统问题”，没有线程状态、系统调用、等待点、资源计数或反证，它不是 OS 分析。

17.5 源码级补充：oops、kdump、ftrace、perf 与 eBPF

BUG、WARN、oops 和 panic 的代码路径。Linux 中 `WARN_ON` 打印栈但通常继续；`BUG_ON` 表示当前路径不可继续；`oops` 是内核异常报告，可能杀死当前进程或导致 `panic`；`panic` 停止系统或按配置重启。读 `kernel/panic.c` 和架构异常处理代码时，看报告如何收集寄存器、栈、`taint`、loaded modules 和当前任务。

```
bad_kernel_access:
    fault in kernel mode
    -> do_error_trap or page fault path
    -> oops_begin()
    -> show registers, stack, code bytes
    -> decide die, panic, or kill current task
```

错误严重程度取决于上下文。用户进程传坏指针应返回 `EFAULT`；内核在持有自旋锁时 `page fault`，可能说明严重 bug；文件系统检测到元数据不一致继续写入可能扩大损坏，因此 `remount read-only` 或 `panic` 都是合理策略。

KASAN、KCSAN 和 KFENCE。

工具	发现问题	取舍
KASAN	越界、use-after-free、部分栈/全局错误	高开销，高覆盖
KCSAN	数据竞争，非原子并发访问	采样式，可能需多跑
KFENCE	堆越界和 UAF	低开销，覆盖有限

这些工具不会替代锁设计。它们帮助你发现违反不变量的具体证据，例如哪个对象释放后被哪个路径继续访问，哪个字段被两个 CPU 无同步写入。测试报告应保留工具输出中的对象地址、分配栈、释放栈和访问栈。

fault injection。Linux 提供多种故障注入，例如 `fail_page_alloc`、`failslab`、`fail_make_request`。目标是在可控位置让分配、slab、块 I/O 失败，验证错误路径。

```
test plan:
    fail next kmalloc in sys_open
    expect no fd installed, inode ref dropped

    fail block request number N during fsync
    expect fsync returns EIO and writeback error recorded
```

没有故障注入，错误回滚代码很可能从未执行过。高质量 OS 代码要让每个 `goto out_*` 标签都能被测试触发。故障注入测试必须检查“没有发生什么”：没有泄漏引用、没有遗留锁、没有半初始化对象进入全局表。

kdump 与 /proc/vmcore。`kdump` 预留一段 `crashkernel` 内存。`panic` 后通过 `kexec` 启动捕获内核，导出 `/proc/vmcore`，再用 `crash` 工具结合 `vmlinux` 符号分析。`panic` 路径越少依赖损坏系统，dump 成功率越高。

```
panic
-> stop other CPUs
-> save registers and notes
-> kexec crash kernel
-> expose /proc/vmcore
-> analyze task, locks, stacks, memory
```

生产环境中，panic 策略要结合服务目标：有的系统宁可快速重启，有的系统必须保留 dump，有的存储系统检测到元数据损坏要停止写入避免扩大破坏。

错误恢复的状态机写法。恢复不是“再试一次”。每个可恢复对象都应有明确状态：正常、降级、恢复中、不可用、已隔离。状态转换要有进入条件、退出条件和超时策略。

```
device state:
  ONLINE
    on timeout -> RECOVERING
  RECOVERING
    stop queues
    reset hardware
    replay configuration
    complete lost requests
    if success -> ONLINE
    if fail count exceeded -> FAILED
  FAILED
    reject new requests with EIO
    keep diagnostic evidence
```

没有状态机，恢复路径容易和正常 I/O 并发踩踏：reset 时仍提交新请求，失败请求没有唤醒等待者，或恢复成功后旧 completion 又回来释放同一资源。

错误预算和降级。并非所有错误都要立即 panic 或无限重试。系统可以设置错误预算：短时间少量校验失败触发重试，连续失败触发隔离，核心元数据错误触发只读或 panic。错误预算让系统避免因为瞬时故障过度反应，也避免因为反复失败无限拖延。

现象	初始处理	超过预算后
块 I/O 超时	重试、reset 队列	下线设备或返回 EIO
内存分配失败	回收、压缩、等待	OOM kill 或返回 ENOMEM
网络重传增多	降低发送速率	熔断连接或限流
文件系统校验失败	停止相关写入	remount read-only 或 panic

错误预算必须可观测：重试次数、最后错误、进入降级时间、当前状态和影响对象都要暴露。否则系统处在降级状态时，用户只会看到“偶尔很慢”。

cleanup attribute 与 RAII 的边界。用户态 C 可以用 `__attribute__((cleanup))` 模拟作用域清理，C++ 可以用 RAII。内核 C 常用 `goto out`，是因为错误路径需要精确控制上下文、锁和释放顺序。三种方法没有绝对高低，关键是所有权必须清楚。

```
// conceptual RAII style
FileRef file = open_checked(path)
PageRef page = allocate_page()
LockGuard guard(inode.lock)
commit_update(file, page)
```

RAII 适合普通进程上下文中的资源组合；但析构函数不应隐藏可能睡眠的复杂 I/O，也不应在 panic 路径中试图获取新锁。教学 OS 可以使用 C++20 RAII 管理引用和锁，但要为中断上下文、noexcept 和显式提交点写清规则。

panic 报告模板。panic 输出要稳定、短而有证据。最低字段如下：

```
panic report:
  reason
  cpu id and current task
  instruction pointer and stack pointer
  trap vector or syscall number
  held locks
  taint or loaded modules
  last N kernel log records
  affected device or inode if any
  reboot/dump policy
```

报告不是越长越好。panic 时系统可能已经损坏，输出过多可能再次阻塞。核心原则是保留能定位第一现场的事实，而不是在崩溃路径中做复杂分析。

printk 和 dmesg。printk 有 loglevel、rate limiting、console 输出和 ring buffer。panic 前后的日志可能是唯一证据，但过量 printk 会扰动时序，甚至拖慢系统。驱动错误路径应记录设备、请求 id、状态寄存器、errno，而不是只写“failed”。

ftrace 和 tracepoint。ftrace 可以追踪函数入口、函数图和静态 tracepoint。tracepoint 的价值是稳定字段：调度切换、irq、block、net、syscalls 都有可解析事件。

```
trace-cmd record -e sched:sched_switch -e block:block_rq_issue
trace-cmd report
```

阅读 [kernel/trace/](#) 时，看 trace event 如何定义字段、如何写入 per-CPU ring buffer、如何被用户态读取。tracepoint 是把证据落成可采集事件。

perf 与 PMU。perf stat 看计数，perf record/report 看采样调用栈，perf top 看实时热点。PMU 事件能观察 cycles、instructions、cache misses、branch misses、TLB misses。IPC 低可能来自 cache miss、分支错误、前端供给不足或后端等待，不能只说“CPU 慢”。

```
perf stat -e cycles,instructions,cache-misses,branches,branch-misses ./workload
perf record -g ./workload
perf report
```

eBPF 和 bpftrace。eBPF 程序可挂到 kprobe、uprobe、tracepoint、profile、LSM、cgroup、网络路径。BPF map 保存状态，helper 提供受控内核访问。bpftrace 适合快速一行观测。

```
bpftrace -e 'tracepoint:syscalls:sys_enter_write { @[comm] = count(); }'
bpftrace -e 'kprobe:vfs_read { @[kstack] = count(); }'
```

eBPF 的边界是 verifier、安全权限和观测开销。生产系统应限制采样频率和输出量，避免观测本身制造故障。

strace、ltrace 和 gdb。strace 基于 ptrace 观察系统调用 enter/exit，能回答“程序在请求内核做什么”；ltrace 观察动态库调用，能回答“用户态库在做什么”；gdb/kgdb 用符号和断点观察状态。strace 看不到内核内部等待原因，perf/ftrace/eBPF 可以补上。

`/proc`、`/sys`、`sysctl` 和 `systemd journal`。`/proc/meminfo`、`/proc/interrupts`、`/proc/slabinfo`、`/proc/[pid]/status`、`/proc/[pid]/stack` 是运行状态窗口；`/sys` 暴露 `kobject`、设备、`cgroup`、`block queue` 参数；`/proc/sys` 是 `sysctl` 控制面；`journald` 给系统日志结构化字段。

PSI: Pressure Stall Information。PSI 记录 CPU、memory、IO stall 时间，回答“任务因为资源压力停顿多久”。它比单纯利用率更接近用户感受到的延迟。内存 PSI 高说明任务因回收或内存不足停顿；I/O PSI 高说明任务等待 I/O；CPU PSI 高说明 `runnable` 但等不到 CPU。

```
cat /proc/pressure/cpu
cat /proc/pressure/memory
cat /proc/pressure/io
```

PSI 适合做过载保护信号。服务可以在 `memory/io pressure` 高时限流，避免把队列堆到超时。

工具选择决策表。可观测性工具很多，选择时先问问题类型。

问题	优先工具	证据
程序卡在哪个系统调用	<code>strace -tt -T</code>	<code>syscall</code> 、耗时、 <code>errno</code>
CPU 时间花在哪里 线程为什么不运行	<code>perf record -g</code> <code>off-CPU trace</code> 、 <code>sched trace</code>	on-CPU 调用栈 睡眠原因、唤醒者
磁盘请求慢在哪里	<code>block trace</code> 、 <code>iostat</code> 、 <code>perf sched</code>	队列和 <code>completion</code>
内存压力是否存在	PSI、 <code>/proc/meminfo</code> 、 <code>vmstat</code>	<code>stall</code> 、 <code>reclaim</code> 、 <code>swap</code>
内核路径偶发错误 容器被限制	<code>tracepoint</code> 、 <code>kprobe</code> 、 <code>bpftrace</code> <code>cgroup v2</code> 文件、 <code>journal</code>	事件字段和栈 <code>throttled</code> 、OOM、 <code>pids limit</code>

不要一上来抓所有数据。先用低成本指标定位层次，再用高精度 `trace` 验证假设。观测也会扰动系统，尤其是高频 `kprobe`、过量 `printk` 和全量系统调用 `trace`—把被观察系统变成了另一个系统。

一次延迟诊断的完整演示。假设 HTTP 服务 `p99` 从 `80ms` 升到 `2s`。诊断不应直接改代码，而应建立证据链。

```
step 1: application trace
  parse: 1ms, business: 5ms, persist: 1900ms

step 2: strace
  fsync(fd) = 0 <1.87s>

step 3: /proc/pressure/io
  some avg10 increased

step 4: block trace
  many flush requests wait behind writeback

step 5: conclusion
  tail latency dominated by storage flush and dirty writeback
```

反证也要写：CPU profiler 没有 2s on-CPU 热点；网络 recv/send 没有长时间阻塞；锁等待直方图没有对应峰值。这样结论才不是“看起来像磁盘”。

指标设计：counter、gauge、histogram、event。系统指标至少分四类。

类型	含义	OS 示例
counter	单调增加事件数	syscall 次数、错误次数、重试次数
gauge	当前状态值	队列长度、活跃线程、脏页数量
histogram	分布	请求延迟、锁等待、I/O 完成时间
event	结构化状态转换	任务睡眠、唤醒、请求提交、panic

错误率只用 counter 不够，还要有错误码和对象维度；延迟只报平均值不够，要有 p50/p95/p99 和最大值；队列只报当前值不够，最好有高水位和丢弃次数。

低开销 instrumentation 原则。观测点应贴近状态转换，而不是在循环里无节制打印。推荐做法：

- 热路径用 per-CPU counter，避免全局锁。
- 高频事件采样或按条件开启。
- 错误路径记录结构化字段和稳定 id。
- 日志消息带 request id、task id、object id。
- trace buffer 有固定容量和丢弃计数。
- 观测代码本身不能持有业务关键锁太久。

可观测性也是代码，必须有性能预算。一个 tracing patch 若把锁竞争扩大，就可能把被观察系统变成另一个系统。

验收：可复现诊断包。本卷要求最终报告能够打包复现。最小诊断包包括：

```
diagnostic bundle:
  exact command and workload
  kernel version and hardware/container limits
  application logs with request ids
  relevant /proc and /sys snapshots
  trace or perf data
  benchmark raw results
  analysis notes with rejected hypotheses
```

报告不是把命令输出堆在一起，而是把证据组织成因果链：现象是什么，在哪层出现，哪些解释被排除，当前结论的边界在哪里。能做到这一点，才算真正掌握 OS 调试。

能力清单

- 能从晶体管开关、逻辑门、寄存器和时钟解释状态机为什么能成为 CPU 的基础。
- 能手跑 X64S 教学子集的 `mov/add/jmp/syscall` 程序，说明 `RIP`、`RSP`、寄存器、`RFLAGS`、`CPL` 如何变化。
- 能从轮询设备推导出中断，从死循环推导出定时器抢占，从同权代码推导出特权级和系统调用。
- 能画出一条 x86-64 `load/store` 指令经过 `RIP`、寄存器、`ALU`、`MMU`、`TLB`、`cache` 和异常入口的路径。
- 能区分 `ISA`、微架构和 `ABI`，并说明 OS 哪些代码依赖 `ISA`，哪些性能现象来自微架构。
- 能解释总线、`MMIO`、`DMA`、`IOMMU`、中断控制器和设备描述符环怎样共同完成一次 `I/O`。
- 能说明特权级限制了哪些指令和状态，为什么系统调用必须是受控陷入而不是普通函数调用。
- 能把一次系统调用拆成用户态入口、内核对象、权限检查、状态更新和返回值。
- 能解释线程为何没有推进：等 `CPU`、等锁、等页面、等 `I/O`、等信号还是等子进程。
- 能解释从复位向量到 `kernel_main` 的启动阶段和每个阶段禁止事项。
- 能说明中断上半部、下半部、定时器 `tick`、抢占点和上下文保存。
- 能判断自旋锁、`mutex`、`semaphore`、`RCU` 的上下文限制，并用锁顺序图解释死锁。
- 能解释 `exec` 的 `ELF` 验证、段映射、用户栈构造、动态解释器和提交点。
- 能手算一次虚拟地址翻译，并区分合法缺页、权限错误和非法地址。
- 能把一次 x86-64 `#PF` 拆成 `CR2`、`error code`、`VMA` 查找、`PTE` 判定、`COW/page cache` 修复、`TLB` 失效和返回重试。
- 能画出 `VMA`、`PTE`、`page cache`、`inode` 的关系，并区分 `MAP_SHARED` 与 `MAP_PRIVATE`。
- 能比较 `buddy`、`slab`、`kmalloc size class` 的用途、复杂度和碎片成本。
- 能画出驱动描述符环、`DMA` 映射、`completion` 和 `buffer` 生命周期。
- 能说明 `bio`、`request`、`request queue`、`flush`、`FUA` 和 `barrier` 如何共同影响 `I/O` 顺序。
- 能画出 `fd`、`open file description`、`inode`、`page cache` 和设备请求之间的对象关系。
- 能解释 `inode`、目录项、空闲块 `bitmap`、`VFS` 和 `page cache` 如何协作。
- 能解释 `socket` 作为 `fd` 的生命周期，说明 `bind`、`listen`、`accept`、`connect`、`send`、`recv` 的等待语义。

- 能分析 capability、namespace、cgroup、seccomp 如何共同形成最小权限边界。
- 能区分 errno、信号、WARN、oops、panic，并为系统调用写出失败回滚路径。
- 能为可靠写出设计 prepared、synced、renamed、manifest committed 的提交状态机。
- 能写出包含现象、假设、证据、反证和未验证边界的 OS 调试报告。

补充材料阅读地图

本册主线由 5 部、17 个编号章组成。补充材料不再夹在第二章或第三章后面，也不另占编号章，否则读者刚建立的因果线会被大块细节打断。下面这些材料保留在源码树中，用作选读、改写素材和实践卷素材。

材料类型	放置位置	使用方式
硬件深讲	source/latex/supplements/hardware/	作为计算机组成和硬件补充卷素材。需要追 ALU、最小 CPU、指令编码、ABI、cache/TLB、总线、DMA、IOMMU、APIC、PCIe 和平台设备细节时再读。主线第二章只引用结论，不连续展开这些文件。
裸机前置	source/latex/supplements/pre-os/	作为“从裸机到内核入口”的选读素材。适合在读完第二章之后补 MCU 主循环、中断、定时器、启动、链接脚本和早期初始化，但不再作为 OS 主线的隐形章节。
可运行模型与验收	source/latex/supplements/models/	作为实践与代码卷素材。这里保存 C++20 硬件模型、x86-64 契约模型、第一阶段验收和硬件到 OS 对象映射。主线正文只保留要验证的不变量，完整模型放到实践卷展开。
源码走读	source/latex/chapters/ 中未直接接入主线的 deep/source/lab 文件	调度、同步、系统调用、page fault、内存、块层、文件系统、网络、恢复、观测和参考实现的长篇材料保留为选读与改写素材。主线各章已经压缩吸收其核心账本，不再把这些文件连续插入目录。

补充材料的使用原则：先读主线，能说清“失败场景、硬件事实、内核对象、状态转换、失败回滚和证据”之后，再按问题查补充材料。不要把补充材料当成第二章后面的连续正文。

硬件深讲建议顺序如下：

1. [ch00-hardware-contract-map.tex](#)：先建立读硬件的统一方法。
2. [ch00-hardware-gates-to-cpu.tex](#)、[ch00-digital-logic-to-cpu-deep.tex](#)、[ch00-minimal-computer-from-scratch.tex](#)：从电路、寄存器和时钟走到最小 CPU。
3. [ch00-number-code-assembly-deep.tex](#)、[ch00-call-stack-abi-deep.tex](#)、[ch00-privileged-machine-walkthrough.tex](#)：把编号、汇编、调用栈、ABI 和特权入口接起来。
4. [ch00-cache-tlb-memory-deep.tex](#)、[ch00-memory-bus-io-deep.tex](#)、[ch00-microarchitecture-platform-deep.tex](#)、[ch00-platform-devices-timing-deep.tex](#)：再进入现代平台、缓存、地址翻译、总线、设备和时钟。
5. [ch00-hardware-os-casebook.tex](#)、[ch00-hardware-os-walkthroughs-stage-one.tex](#)、[ch00-hardware-case-lab.tex](#)：最后用端到端案例检查理解。

裸机前置建议顺序如下：

1. `ch02-bare-metal-mcu.tex`：从温控板主循环看为什么轮询和阻塞函数会失败。
2. `ch03-interrupts-timers.tex`：从串口丢字节和坏任务死循环推导中断、定时器和抢占入口。
3. `ch04-boot-linker-kernel-init.tex`：从上电后不能直接调用 C 函数，推导 reset handler、链接脚本、BSS 清零、早期栈和内核入口。

可运行模型建议顺序如下：

1. `ch00-stage-one-gap-closure.tex`：检查第一阶段还缺哪些能力。
2. `ch00-stage-one-hardware-os-mapping.tex`：把硬件事实映射成内核对象。
3. `ch00-stage-one-cpp20-hardware-model.tex` 和 `ch00-stage-one-x86-64-contract-model.tex`：用可编译模型验证权限、MMIO、中断、DMA 和 x86-64 fault 语义。
4. `ch00-stage-one-evidence-and-acceptance.tex`：用测试和观测证据验收理解。

主线章节的压缩归并关系如下：

主章	已压缩吸收的补充方向
进程、线程与调度	调度器实现、进程生命周期、同步状态机、futex、锁、死锁、调度观测和源码实验。
系统调用、权限与内核边界	x86-64 入口、用户指针复制、异常修复表、capability、LSM、seccomp、vDSO、审计和资源限制。
exec、ELF 装载与用户栈	ELF/binfmt、解释器脚本、动态链接器、auxv、CLOEXEC、setuid/capability、ASLR、 <code>vfork/posix_spawn</code> 和多线程 exec 边界。
虚拟内存与地址空间	page fault 走读、COW、VMA/PTE、TLB、THP、swap、mlock、PSI、OOM、内核分配器和 C++20 内存模型素材。
mmap、page cache 与文件映射	<code>do_mmap</code> 、XArray、writeback、 <code>O_DIRECT</code> 、DAX、映射截断、 <code>msync/fsync</code> 和持久化恢复边界。
设备驱动、中断下半部与 DMA	Linux 设备模型、PCI、virtio、NVMe、IOMMU、DMA 生命周期、块层 request、flush/FUA 和提交顺序。
文件、设备与 I/O	VFS、路径解析、fd/file/inode、page cache、epoll、io_uring、文件系统日志、目录项持久化和恢复协议。
socket、网络入口与内核边界	skb、NAPI、Netfilter、socket option、connect/recv 错误语义、小包延迟、超时重试和业务幂等。
安全、隔离与最小权限	DAC/MAC、capability、namespace、cgroup、Landlock、seccomp、BPF LSM、panic、错误路径、观测、调试和全册验收标准。

Linux 源码阅读索引

本册不是 Linux 内核源码注释，但每个重要机制都应能落到真实源码。本版以 Linux 6.16 文档作为接口与术语核对基线；下面的路径采用 Linux 6.x 主线的常见目录。源码阅读时仍应先 checkout 明确的 tag，并以该源码树为准，因为函数名、文件位置和实现策略都会变化。目标不是背路径，而是知道“这个概念在真实系统里由哪些结构和函数承载”。

主题	关键路径	重点对象或函数
x86 启动	arch/x86/boot/, arch/x86/kernel/head_64.S	实模式入口、长模式入口、早期页表
内核初始化	init/main.c, arch/x86/kernel/setup.c	start_kernel、setup_arch、initcall
系统调用入口	arch/x86/entry/entry_64.S, arch/x86/entry/syscalls/syscall_64.tbl	entry_SYSCALL_64、do_syscall_64
x86 page fault	arch/x86/mm/fault.c, arch/x86/mm/tlb.c	CR2、error code、用户/内核 fault、TLB flush 和 shutdown
调度器	kernel/sched/core.c, kernel/sched/fair.c, kernel/sched/rt.c	__schedule、enqueue_task_fair、pick_next_task
进程创建退出	kernel/fork.c, kernel/exit.c	copy_process、do_exit、wait
futex	kernel/futex/	futex_wait、futex_wake、PI futex
锁与 lockdep	kernel/locking/ kernel/locking/lockdep.c	qspinlock、mutex slow path、lock class graph
虚拟内存	mm/memory.c, mm/mmap.c, mm/rmap.c	handle_mm_fault、do_mmap、COW、VMA/PTE 生命周期
页面回收	mm/vmscan.c, mm/swap.c, mm/oom_kill.c	shrink_lruvec、kswapd、OOM killer
内核分配器	mm/page_alloc.c, mm/slub.c, mm/vmalloc.c	buddy、SLUB、kmem_cache_alloc
ELF 装载	fs/binfmt_elf.c, fs/exec.c, fs/binfmt_script.c	load_elf_binary、create_elf_tables、begin_new_exec、install_exec_creds、CLOEXEC 和 binfmt 递归
VFS 与 open	fs/open.c, fs/namei.c, fs/read_write.c	path_openat、vfs_read、vfs_write
page cache	mm/filemap.c, mm/readahead.c, fs/fs-writeback.c	filemap_fault、readahead、writeback
ext4 与 jbd2	fs/ext4/, fs/jbd2/	extent、mballoc、journal commit、recovery
块层	block/blk-mq.c, block/bio.c, block/mq-deadline.c	blk_mq_submit_bio、bio、request、tag
设备模型	drivers/base/ include/linux/device.h	device、driver、bus_type、kobject

PCI、NVMe、virtio	<code>drivers/pci/</code> , <code>drivers/nvme/host/</code> , <code>drivers/virtio/</code>	<code>probe/remove</code> 、 <code>queue</code> 、 <code>interrupt</code> 、DMA
网络栈	<code>net/core/</code> 、 <code>net/ipv4/</code> , <code>include/linux/skbuff.h</code>	<code>sk_buff</code> 、 <code>tcp_v4_rcv</code> 、 <code>tcp_sendmsg</code>
epoll	<code>fs/eventpoll.c</code>	<code>ep_poll_callback</code> 、ready list、wait queue
namespace、cgroup	<code>kernel/nsproxy.c</code> , <code>kernel/cgroup/</code>	namespace clone、cgroup v2 controller
LSM/seccomp	<code>security/</code> , <code>kernel/seccomp.c</code>	<code>security_*</code> hooks、BPF filter、seccomp actions
tracing、perf、 eBPF	<code>kernel/trace/</code> , <code>kernel/events/</code> , <code>kernel/bpf/</code>	tracepoint、perf event、 BPF program/map
panic/kdump	<code>kernel/panic.c</code> , <code>kernel/kexec_core.c</code>	<code>panic</code> 、oops、kdump、 crashkernel

源码阅读最低标准：先找核心结构体，再找对象创建路径、状态转换路径、失败回滚路径和观测点。只搜索一个函数名，通常看不到系统设计。

参考资料与版本基线

本册的机制解释以稳定概念为主；涉及 Linux 接口、调度策略、文件系统特性和持久化语义时，应回到带版本的主资料核对。下面列出本版技术校审使用的主要入口，核对日期为 2026 年 7 月 22 日。网页的“latest”内容会继续变化，做源码走读时仍须记录实际 checkout 的 tag 或 commit。

Linux 内核文档

Linux 6.16 文档总入口：<https://www.kernel.org/doc/html/v6.16/>。

公平调度

EEVDF 调度器说明：<https://www.kernel.org/doc/html/latest/scheduler/sched-eevdf.html>；CFS 历史设计：<https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html>。

用户指针访问

`copy_to_user/copy_from_user` 返回值和上下文约束：<https://docs.kernel.org/6.11/kernel-hacking/hacking.html>。

块设备粒度

Linux block queue 的 logical block size、physical block size 与 minimum I/O size：<https://www.kernel.org/doc/html/v6.16/core-api/kernel-api.html>。

持久化

Linux `fsync(2)`，包括文件与父目录持久化边界：<https://man7.org/linux/man-pages/man2/fsync.2.html>。

Btrfs

在线碎片整理及其对 extent sharing 的影响：<https://btrfs.readthedocs.io/en/latest/Defragmentation.html>。

F2FS

F2FS 设计、checkpoint、后台 GC 与前台 GC：<https://www.kernel.org/doc/html/v6.16/filesystems/f2fs.html>。

开放文件系统文档

OpenZFS 文档：<https://openzfs.github.io/openzfs-docs/>；Microsoft 文件系统协议资料：https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-fscc/efbfe127-73ad-4140-9967-ec6500e66d5e；Apple File System Reference：<https://developer.apple.com/support/downloads/Apple-File-System-Reference.pdf>。

引用的最低标准不是“网页存在”，而是每个易变化断言都能回答三个问题：针对哪个版本，适用哪些配置，原始资料具体支持哪一部分结论。性能数字还必须同时给出硬件、工作负载、队列深度、缓存与持久化设置。